The Super Friendly CCNP Enterprise Study Guide

ENCOR 350-401 (v1.2) + ENARSI 300-410 (v1.1) — Same Friendly Voice, Professional Depth

What is this? The sequel to the CCNA guide. It teaches everything on the two exams that earn **CCNP Enterprise**: the core exam **ENCOR 350-401 v1.2** (architecture, virtualization, infrastructure, assurance, security, automation and AI) and the concentration exam **ENARSI 300-410 v1.1** (advanced routing, VPNs, infrastructure security and services). Same approach as before — plain language, the why behind every mechanism, real configs — but pitched at someone who already holds a CCNA.

How it's organised: Part I (Chapters 1–21) follows the ENCOR blueprint. **Part II** (Chapters 22–32) follows ENARSI, and deliberately builds on Part I: where ENCOR asks you to describe or configure something, ENARSI asks you to troubleshoot it, so the Part II chapters assume the Part I version and go deeper instead of repeating it. **Part III** is the exam strategy and study plans, followed by two **Blueprint Coverage Maps** (one per exam) that point every official topic at the section that teaches it.

What it assumes: you passed CCNA. Subnetting, VLANs, single-area OSPF, ACLs and the packet walk are not re-taught. Where a chapter builds on something from the CCNA guide it says so — e.g. (CCNA guide 16.11) — and **Appendix C** lists the CCNA sections worth re-reading before each part.

How each section teaches: first the **problem** the feature solves (the why), then a ** Story Time** analogy where one helps, then the **mechanism step by step** (the how), and finally a **Worked example** with real addresses and numbers you can match against your own lab output. If a section ever makes sense in the config but not in your head, re-read the Story Time and the walk-through and then try it in EVE-NG.

About the depth sections: a few sections go past what the blueprint asks for — the mechanism underneath rather than the behaviour. They're marked, so you can skip them on a tight timeline.

Every chapter opens with a Blueprint line naming the exact exam topics it covers, so you always know which exam (and which line of it) you're studying for.

Table of Contents

Part I — ENCOR 350-401: Enterprise Core

1. The CCNP Mindset & Enterprise Network Design
2. High Availability: Redundancy, SSO/NSF & Stacking
3. Layer 2 Revisited: Trunking & EtherChannel Troubleshooting

4. Spanning Tree at CCNP Depth: RSTP, MST & Enhancements
5. Routing Concepts: EIGRP vs OSPF, Path Selection & PBR
6. OSPFv2/v3: Multi-Area, Summarization & Filtering
7. BGP Fundamentals & eBGP
8. Time & Translation: NTP, PTP, NAT/PAT
9. First Hop Redundancy: HSRP & VRRP
10. Multicast
11. QoS
12. Device & Data-Path Virtualization: Hypervisors, VRF, GRE & IPsec
13. Network Virtualization: LISP & VXLAN
14. Catalyst SD-WAN & SD-Access
15. Network Assurance Tools: Debugs, SNMP, Syslog, Flexible NetFlow, SPAN, IP SLA
16. Cisco Catalyst Center: Traditional & AI-Powered Workflows
17. Device Access Control, AAA, ACLs & CoPP
18. Security Design & REST API Security
19. Python & JSON for Network Engineers
20. YANG, NETCONF, RESTCONF & Controller APIs
21. EEM, Orchestration Tools & AI in Operations

Part II — ENARSI 300-410: Advanced Routing & Services

22. The Troubleshooting Method & Administrative Distance
23. Route Maps, Prefix Lists & Loop Prevention
24. Redistribution & Summarization
25. PBR, VRF-Lite & BFD
26. EIGRP In Depth
27. OSPF In Depth
28. BGP In Depth
29. MPLS & Layer 3 VPNs
30. DMVPN
31. Infrastructure Security: AAA, ACLs, uRPF, CoPP & IPv6 First-Hop Security
32. Infrastructure Services: Management, SNMP, Logging, DHCP, IP SLA, NetFlow & Assurance

Part III — Passing Both Exams

33. Exam Strategy & Study Plans

Appendices

- Appendix A: ENCOR 350-401 v1.2 Blueprint Coverage Map
- Appendix B: ENARSI 300-410 v1.1 Blueprint Coverage Map
- Appendix C: The CCNA Foundations to Re-Read First
- Glossary

Part I — ENCOR 350-401: Enterprise Core

ENCOR is the **core** exam: required for CCNP Enterprise, and also the qualifying exam for **CCIE Enterprise Infrastructure**. (Since 19 March 2026 the CCIE Enterprise Wireless track has its own core exam, **WLCOR 350-101**, which is why v1.2 could drop wireless.) It is wide rather than deep — six domains running from campus design to Python — and v1.2 (live since **19 March 2026**) renamed the last domain **Automation and Artificial Intelligence**, adopted the current product names (Catalyst Center, SD-WAN Manager) and dropped the wireless topics that v1.1 carried.

Domain	Weight	Chapters
1.0 Architecture	15%	1, 2, 9, 11, 14
2.0 Virtualization	10%	12, 13
3.0 Infrastructure	30%	3–10
4.0 Network Assurance	10%	15, 16, 20
5.0 Security	20%	17, 18
6.0 Automation and Artificial Intelligence	15%	19, 20, 21

Infrastructure is almost a third of the exam on its own — Chapters 3 to 10 are where the marks are.

Chapter 1: The CCNP Mindset & Enterprise Network Design

> **Blueprint:** ENCOR 1.1 — Explain the different design principles used in an enterprise network; ENCOR 1.1.a — High-level enterprise network design such as 2-tier, 3-tier, fabric, and cloud.

1.1 You Passed CCNA. What Actually Changes Now?

Congratulations — you've proven you understand how a network works. CCNP asks a different question. CCNA was mostly **"what does this do, and how do I turn it on?"** CCNP is **"which way should I build this, what breaks when it gets big, and how do I prove it's working?"**

That sounds like a small shift. It isn't. Here's what it looks like in practice:

CCNA asked you…	CCNP asks you…
Configure OSPF on two routers	Design OSPF for 400 routers so one flapping link doesn't make all 400 recalculate

CCNA asked you…	CCNP asks you…
What does HSRP do?	HSRP or VRRP? What's the failover time? What happens to the ARP caches of 2,000 hosts?
Which port does STP block?	Why did the root bridge move at 3 a.m., and which guard would have stopped it?
What is a VLAN?	Should this VLAN span the whole building, or should we route at the access layer?
Is the link up?	Is the path meeting its SLA right now, and how would you know before the users do?

(An SLA, or service-level agreement, is a promised level of performance, for example "under 150 ms of delay and under 1% loss".)

Story Time : CCNA taught you to **drive**: what the pedals do, how to signal, the rules of the road. CCNP asks you to **design the road system** for a city. Now the questions are different. Do we need a ring road, or will traffic lights do? What happens to the whole city when one bridge closes? Should this junction be a roundabout or a flyover? You still need to know how to drive. But the exam now cares about whether the roads you'd build keep moving when something goes wrong.

Why does the exam care so much about "why" and "which"? Because at the professional level, almost every technology has **two or three valid ways** to do the same job, and each comes with a cost. A spanning-tree design is simpler to understand but wastes links; a routed-access design uses every link but needs more IP planning. A big flat OSPF area is easy to configure but fragile at scale; multi-area is robust but needs summarization discipline. CCNP questions are built around those trade-offs, so memorized commands alone won't carry you. You're no longer being tested on whether you know the tool — you're being tested on whether you'd pick the right one.

The Three Habits That Carry You Through Both Exams

1. **Think in failure domains.** For every design, ask: "if this breaks, how much of the network notices?" The answer should always be "as little as possible." You'll see this idea in every chapter — OSPF areas, STP scope, VRFs, EtherChannel, even QoS.
2. **Think in planes.** Every network device does three separate jobs: the **data plane** forwards packets, the **control plane** decides where they should go (OSPF, BGP, STP), and the **management plane** is how you talk to the box (SSH, SNMP, NETCONF). Problems — and security — live in one plane at a time. SD-WAN, SD-Access and Catalyst Center all start by pulling one of these planes apart (Chapters 14 and 16).
3. **Prove it.** At CCNA, "it pings" was success. At CCNP, success is measurable: an IP SLA probe is green, NetFlow shows the traffic took the path you designed, the adjacency has been FULL for 40 days. Chapters 15 and 32 are all about this.

How This Guide Is Laid Out

- **Part I (Chapters 1–21)** follows the **ENCOR 350-401 v1.2** blueprint — the core exam everyone must pass. It's broad: design, switching, routing, services, virtualization, assurance, security and automation.

- **Part II (Chapters 22–32)** follows **ENARSI 300-410 v1.1** — the advanced routing concentration. It's narrow and deep, and most of its blueprint verbs are **"Troubleshoot"**. Those chapters assume the matching ENCOR chapter and go further.
- **Part III** holds the exam strategy, study plans, a coverage map for **every** blueprint line of both exams, and the glossary.

> **What's assumed:** CCNA is the floor. Subnetting in your head, the packet walk (CCNA guide 14.8), single-area OSPF and its internals (CCNA guide 16.10–16.11), STP mechanics (CCNA guide 9.10–9.11), and basic ACLs. If any of those feel shaky, re-read them now — this guide builds on them without stopping.

1.2 The Design Principles Behind Every Enterprise Network

The problem this solves: there are dozens of ways to wire up a network, and each vendor slide shows a different one. Without a short list of rules, every design argument turns into opinion. The principles below are that list. When two designs both "work", the one that follows these principles better is the one that will still work after three years of growth and a few bad nights.

Before any specific architecture, there are a handful of principles every good design follows. Every design choice in this chapter is one of these principles applied to a specific situation.

Principle	What it means	What it buys you
Hierarchy	Split the network into layers, each with one job	Predictable traffic paths; each layer can be sized and changed on its own
Modularity	Build in repeatable blocks (a building, a data center pod, a branch)	Grow by adding a block, not redesigning; one block's failure stays in that block
Resiliency	No single point of failure on important paths	Failures cause a blip instead of an outage (Chapter 2)
Flexibility	Leave room to add services without a redesign	New VLANs, new sites, new overlays slot in
Simplicity	The fewest moving parts that meet the requirements	Fewer things to break; faster troubleshooting at 3 a.m.

Why is "simplicity" listed as a design principle, when it sounds like an absence of design? Because every feature you add is something that can fail, something to configure consistently across hundreds of devices, and something the next engineer has to understand while the network is down. Redundancy is good — but two redundant paths you understand beat four you don't. A design you can't troubleshoot under pressure is not a resilient design, no matter how many backup links it has.

Failure Domains — The Idea Underneath Everything

A **failure domain** is the part of the network affected when one thing breaks.

Story Time : A big ship's hull is split into **watertight compartments** by solid walls called bulkheads. If the hull is holed, only the flooded compartment fills with water and the ship stays afloat. Without the walls, one hole sinks the whole ship. Network design does the same thing: routing boundaries, VLAN boundaries and OSPF areas are the bulkheads. They don't stop things breaking. They stop one breakage from spreading. (Where the analogy stops: ship compartments are sealed. Network "compartments" still pass traffic to each other. What they block is the spread of problems: broadcasts, STP changes, routing churn.)

Some examples you already know:

Thing that fails	Failure domain	How designs shrink it
A broadcast storm in a VLAN	Every port in that VLAN, on every switch it spans	Keep VLANs small and local; route between them
An STP topology change	Every switch in that spanning-tree instance	Routed access; fewer switches per L2 domain
A flapping OSPF link	Every router in that **area** (they all rerun SPF)	Multi-area design with summarization (Chapter 6)
A core switch reload	Everything behind it, unless there's a redundant peer	Dual cores, dual uplinks, SSO (Chapter 2)

Why do "small failure domains" and "routing instead of switching" keep showing up together? Because Layer 2 is a **flooding** technology: unknown unicast, broadcast and STP changes all spread across the whole VLAN. Layer 3 is a **summarizing** technology: a router can hide a thousand prefixes behind one summary and simply not tell the rest of the network about local chaos. Every time you move a routing boundary closer to the edge, you shrink how far a problem can spread.

Worked example: the same loop, two designs. Someone in a classroom plugs both ends of a patch cable into the same desk switch, which has no BPDU guard. A broadcast storm starts in VLAN 10 (10.10.10.0/24).

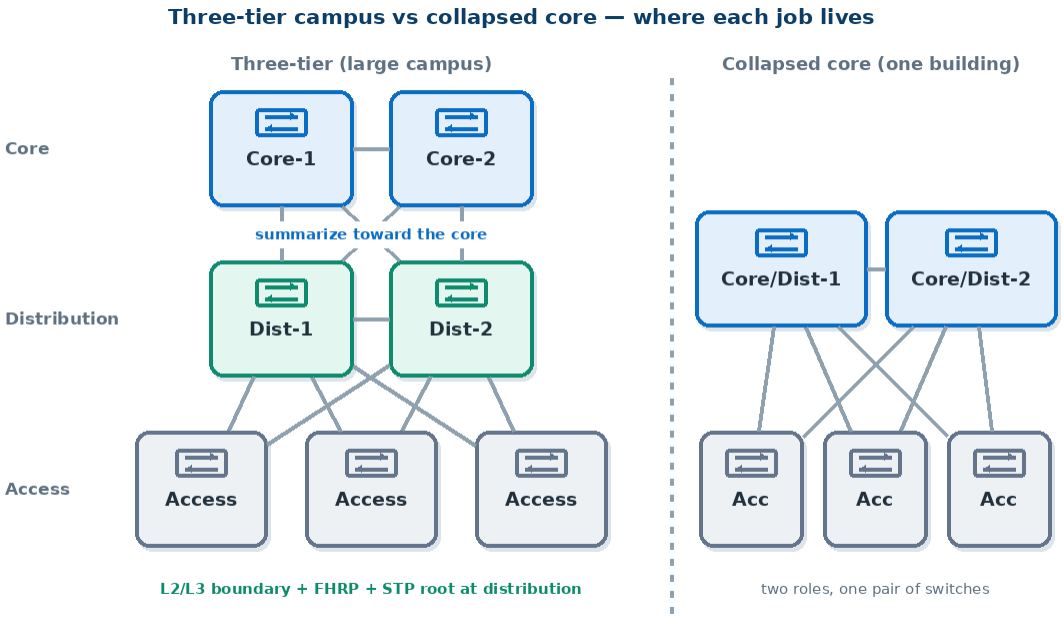
Question	Design A: VLAN 10 spans 12 access switches (L2 to distribution)	Design B: routed access, VLAN 10 exists on one access switch
Where the storm floods	Every port in VLAN 10 on all 12 switches, plus both distribution uplinks	Only the ports on that one access switch
Who notices	Every user in VLAN 10 building-wide; the distribution CPUs spike handling the flood on their SVIs	The ~48 users on that switch
What the rest of the network sees	Possibly a distribution switch too busy to keep its OSPF neighbors up, so wider instability	One subnet unreachable. The routed uplink carries no broadcasts, so nothing else changes

The storm is identical in both. Only the size of the failure domain differs, and that was decided at design time.

1.3 The Three-Tier Hierarchical Campus

The problem this solves: a large campus has thousands of users in dozens of closets. If every switch connected to every other switch "however was convenient", nobody could predict where traffic goes, and one change could ripple everywhere. The three-tier model gives every switch exactly one job, so traffic paths are predictable and each layer can be grown or fixed on its own.

You met this in CCNA (guide 1.7). At CCNP you need to know why each layer does what it does, and what goes wrong when layers blur.



Layer	Job	What lives here	What should NOT live here
Access	Connect endpoints	Access ports, PoE, port security, 802.1X, QoS marking/trust, BPDU guard	Complex policy, inter-site routing
Distribution	Aggregate access, enforce policy, be the L2/L3 boundary	SVIs and FHRP gateways, route summarization, ACLs, STP root	End users
Core	Move packets between distribution blocks as fast as possible	High-speed L3 links, fast-converging routing	Policy, filtering, anything that slows forwarding or fails often

Why should the core be "dumb and fast"? Because the core is the one place where **everything** passes through. Any feature there — an ACL, a QoS policy, NAT — is applied to every packet in the network, and any bug in that feature can take down every building at once. Keep

the core to one job (forward, converge fast) and push intelligence out to the distribution layer, where a mistake only affects one block. The core is the spine of the network; you want it boring.

Why is the distribution layer where summarization happens? Because each distribution pair sits at the edge of a block — a building, a floor group — and knows every subnet inside it. If it advertises one summary (`10.20.0.0/16`) toward the core instead of 200 specific /24s, two good things happen: the core's routing table stays small, and a subnet flapping in that building **doesn't propagate** past the distribution layer, because the summary stays up as long as any component is up. Summarization is how the distribution layer turns "something changed in building 20" into "nothing changed" for everyone else.

Worked example: a flapping subnet behind a summary. Building 20 uses 10.20.0.0/24 through 10.20.255.0/24. Its distribution pair is the OSPF area border router (ABR) for area 20 and is configured with `area 20 range 10.20.0.0 255.255.0.0` (Chapter 6). The lab subnet 10.20.7.0/24 goes up and down every few seconds because of a bad cable.

1. Inside area 20, the change is real. The distribution switches see 10.20.7.0/24 appear and disappear, and run SPF each time.
2. The distribution switches now decide what to tell the core. The summary 10.20.0.0/16 stays advertised as long as **at least one** /24 inside it is up. 255 other subnets are still up, so the summary never changes.
3. The core receives no update at all. Its routing table is calm:

```
CORE1# show ip route 10.20.7.5
Routing entry for 10.20.0.0/16
  Known via "ospf 1", distance 110, metric 3, type inter area
  Routing Descriptor Blocks:
  * 10.0.1.2, from 10.255.0.21, via TenGigabitEthernet1/0/1
```

4. While 10.20.7.0/24 is down, the core still forwards packets for 10.20.7.5 toward building 20. The distribution switch has no /24 for it, matches its own automatically created **discard route** (`10.20.0.0/16 is a summary, Null0`) and drops the packet there.

The trade-off: one building's problem stays in that building, but traffic for a dead subnet travels as far as the distribution layer before it is dropped. That is almost always worth it.

When Do You Need a Separate Core?

The core exists to stop distribution blocks from needing a full mesh to each other. With **n** distribution pairs and no core, each pair needs links to every other pair:

Distribution blocks	Full-mesh links needed — n(n−1)/2	With a core: links per block
2	1	not needed
4	6	2 uplinks each
8	28	2 uplinks each
16	120	2 uplinks each

How to read the formula: each of the **n** blocks needs a link to the other **n − 1**. That gives n(n−1) link ends, and every link has two ends, so divide by 2. With 8 blocks: 8 × 7 = 56, ÷ 2 = **28**

links. Adding a ninth block means cabling it to all 8 existing blocks, which is 8 new links and 8 config changes. With a core, the ninth block needs just its 2 uplinks.

Once you have more than about three distribution blocks, the mesh becomes unmanageable and a core pays for itself.

1.4 Two-Tier (Collapsed Core)

The problem this solves: a separate core only earns its cost when there are many distribution blocks to join. A single building usually has one. Building three tiers there buys extra switches, power and config for no benefit.

In a smaller site — one building, a few floors — the distribution and core roles are **collapsed** into one pair of switches. Access switches uplink to that pair, and that pair also connects to the WAN edge and data center.

Why is collapsed core the most common real-world campus design? Because most sites don't have enough distribution blocks to justify a separate core (see the table above). Collapsing saves two or more expensive chassis, their power and cabling, and an entire tier of configuration — and you can always split the core out later if the site grows. Hierarchy is a logical idea first; you only build the physical tier when the scale demands it.

Design	Fits	Pros	Cons
Two-tier (collapsed core)	Single building, small/medium campus	Cheaper, fewer devices, simpler	Doesn't scale past a few blocks; core and distribution failures are the same failure
Three-tier	Large multi-building campus	Scales by adding blocks; clear roles	More devices, more cost, more config

1.5 Where Layer 2 Ends: Four Access-Distribution Models

The problem this solves: Layer 2 is easy (plug in, same subnet, it works) but floods and needs STP to stop loops. Layer 3 is stable and uses every link but needs IP planning and can't stretch a subnet. Every campus has to decide where to switch from one to the other. That decision fixes how big your failure domains are, whether links sit blocked, and whether you need a first-hop redundancy protocol (FHRP: HSRP, VRRP or GLBP, Chapter 9).

The single most important campus design decision is **where Layer 2 stops and Layer 3 begins**. There are four common answers.

Model	L2/L3 boundary	STP's role	FHRP needed?	VLANs can span access switches?
Layer 2 access (looped)	Distribution	Blocks one uplink per VLAN	Yes, at distribution	Yes

Model	L2/L3 boundary	STP's role	FHRP needed?	VLANs can span access switches?
Layer 2 access (loop-free / "V")	Distribution	Present but nothing to block	Yes	No (each VLAN local to one access switch)
Routed access	**Access switch**	Only on edge ports (no switch-to-switch loops)	**No** — the access switch is the gateway	No
Virtual switching (StackWise Virtual / VSS)	Distribution (one logical switch)	Present, but nothing blocks — uplinks are one EtherChannel	No — one logical gateway	Yes

Why does routed access make spanning tree nearly irrelevant? Because every access-to-distribution uplink becomes a routed point-to-point link. Routed links don't carry broadcasts or BPDUs between switches, so there's no Layer 2 loop to break and nothing for STP to block. Both uplinks forward at once, using **ECMP** (equal-cost multipath) in OSPF or EIGRP, and if one fails, the routing protocol reconverges in well under a second with tuned timers or BFD (Chapter 25). Routed access trades VLAN flexibility for speed, full link utilization, and a tiny failure domain.

(StackWise Virtual and its older Catalyst 6500 ancestor VSS, the Virtual Switching System, join two physical switches into one logical switch with one control plane. Chapter 2 explains how.)

Worked example: routed access failover, step by step. Access switch ACC1 is the default gateway for its users (VLAN 10, 10.1.10.1/24). It has two routed uplinks: Gi1/0/49 (10.0.0.1/31) to DIST1 and Gi1/0/50 (10.0.0.3/31) to DIST2. OSPF runs on both, with equal cost, and BFD at 50 ms × 3.

1. In normal operation, ACC1 installs **two** equal-cost routes to everything beyond the building:

```
ACC1# show ip route 10.99.0.0
Routing entry for 10.99.0.0/16
  Known via "ospf 1", distance 110, metric 3, type inter area
  Routing Descriptor Blocks:
  * 10.0.0.0, from 10.255.0.1, via GigabitEthernet1/0/49
      Route metric is 3, traffic share count is 1
    10.0.0.2, from 10.255.0.2, via GigabitEthernet1/0/50
      Route metric is 3, traffic share count is 1
```

2. Both uplinks carry traffic. ACC1 hashes each flow (source/destination IP, and optionally ports) to one of the two next hops. A single flow always uses the same link, so its packets never arrive out of order.
3. DIST1 fails. If it loses power, Gi1/0/49 goes down at once and OSPF drops the neighbor immediately; no timer is involved. The harder case is a failure that leaves the link light on, such as DIST1's software hanging or a media converter in the path dying. Nothing tells ACC1 the port is "down", so plain OSPF would wait out its 40-second dead interval. BFD on Gi1/0/49 misses three 50 ms hellos instead, so after about **150 ms** it tells OSPF the neighbor is gone.

4. OSPF removes the 10.0.0.0 next hop. Every flow now hashes to Gi1/0/50. No STP reconvergence, no FHRP failover and no ARP change, because the hosts' gateway (ACC1 itself) never moved.
5. When DIST1 returns, the adjacency re-forms and the second next hop is added back.

Compare the looped Layer 2 design. There, one uplink sits blocked by STP until it's needed, and losing the active gateway also means an FHRP failover (Chapter 9).

So why isn't everyone using routed access? Because some applications still need a VLAN to exist in more than one closet — wireless controllers in older designs, certain building-management systems, or an application that expects hosts in the same subnet. Routed access can't stretch a VLAN. That need, at scale, is exactly what overlays like **VXLAN** and **SD-Access** solve: run a routed underlay for stability, and stretch Layer 2 virtually on top of it (Chapters 13 and 14).

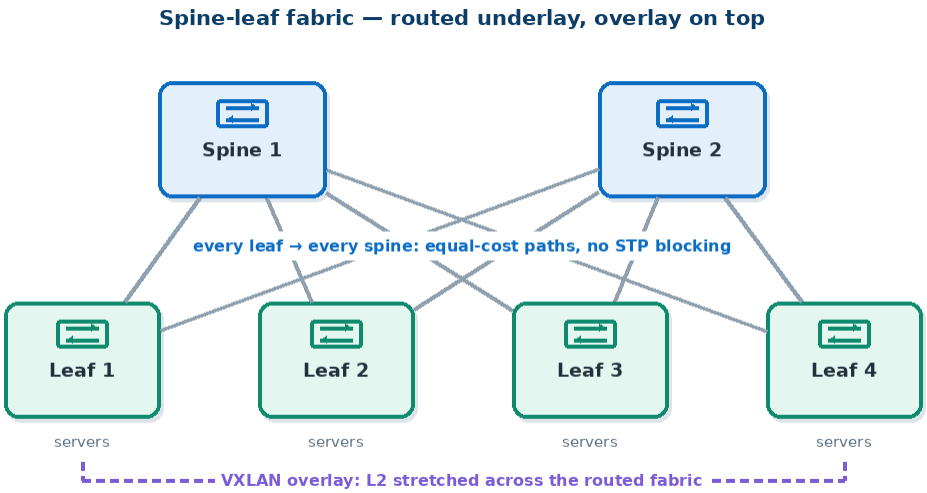
> **Exam trap:** in the looped Layer 2 model, the **STP root** and the **HSRP/VRRP active gateway** for each VLAN should be the **same distribution switch**. If they're split, traffic from a host goes up a blocked-path detour: it crosses the access switch to the STP root, then over the inter-distribution link to the active gateway. Chapter 4 (4.1) and Chapter 9 come back to this.

1.6 Fabric Designs: Spine-Leaf

Data centers carry mostly **east-west** traffic (server to server: app tier to database, VM to storage), not the **north-south** traffic (users to the internet) that a campus mostly carries. A three-tier tree is built for north-south; for east-west it sends traffic up and down the tree through a core that becomes a bottleneck.

Story Time : Picture a building where every office is on one of 20 floors, and every floor has a door onto each of 4 lift shafts. To get from any office to any other you take exactly one lift: walk to a lift, ride, walk out. It doesn't matter which floors you start and end on. The trip is always the same length, and if one lift breaks you use another. The floors are **leaves**, the lifts are **spines**. A three-tier tree is more like a building where some trips need two lifts and a walk across the lobby.

Spine-leaf fixes this with a simple rule: **every leaf connects to every spine, leaves never connect to each other, spines never connect to each other.**



Property	Why it matters
Any leaf to any leaf is **always exactly two hops** (leaf → spine → leaf)	Latency is predictable no matter where workloads sit
Every leaf has **equal-cost paths** through every spine	ECMP spreads load across all spines; no STP blocking
Need more bandwidth? **Add a spine.** Need more ports? **Add a leaf.**	Scales out horizontally instead of buying a bigger box
A spine failure removes **1/n of the capacity**, not connectivity	Graceful degradation

Worked example: sizing and losing a spine. A fabric has 4 spines. Each leaf has one 100 Gbps uplink to each spine, so each leaf has **4 × 100 = 400 Gbps** toward the fabric, spread by ECMP across 4 equal paths.

- Spine 3 fails. Every leaf loses one of its four uplinks: capacity drops to **300 Gbps** per leaf (a 25% loss). Every leaf can still reach every other leaf in two hops through spines 1, 2 and 4.
- The fabric needs more bandwidth. Add a fifth spine and one more uplink per leaf: **500 Gbps** per leaf, with no change to how traffic flows.
- The fabric needs more server ports. Add a leaf and cable it to all spines. Nothing else is touched.

Why is the underlay in a fabric almost always routed (OSPF, IS-IS or eBGP)? Because the whole point is to use **every** spine link at once. Spanning tree would block all but one path to each spine; a routing protocol happily installs every equal-cost path. Workloads still often need Layer 2 adjacency (VM mobility across racks), so a **VXLAN overlay** carries Layer 2 over the routed fabric (Chapter 13). This "routed underlay + overlay" pattern is the single most important architecture idea in modern Cisco design — SD-Access, SD-WAN and data-center fabrics all use it.

The word "fabric" in the blueprint means exactly this pattern: a physical underlay that's simple, routed, and redundant, plus a logical overlay that gives you the segmentation and mobility you want. In the campus, Cisco's fabric is **SD-Access** (Chapter 14).

1.7 The WAN Edge and Branches

The problem this solves: branches, data centers and the cloud all have to reach each other over links you rent rather than own. Those links are slower, cost money every month, and fail in ways you can't fix yourself. WAN design is about choosing which links to rent and how to survive when the carrier has a bad day.

The enterprise doesn't end at the campus. Sites connect through a **WAN edge**, and the design questions change: links are slower, more expensive, and owned by someone else.

WAN option	What it is	Typical use
MPLS L3VPN	Provider-run private network; provider routes your traffic between sites	Predictable SLA, voice/video between sites (Chapter 29)
Internet + IPsec / DMVPN	Encrypted tunnels over cheap broadband	Low cost, flexible, hub-and-spoke or spoke-to-spoke (Chapters 12, 30)
Metro Ethernet	Provider gives you a Layer 2 circuit	Sites in the same city
Cellular (4G/5G)	Wireless WAN link	Backup or temporary sites
SD-WAN	Controller-managed overlay across **all** of the above	Uses every transport at once, picks paths per application (Chapter 14)

Branch designs are usually described by redundancy: **single router, single link** (cheapest, no redundancy), **single router, dual links** (protects against carrier failure), and **dual routers, dual links** (protects against both carrier and hardware failure — the two routers run an FHRP on the LAN side).

Why did SD-WAN take over the WAN edge? Because traditional WAN designs made you choose: expensive MPLS for quality, or cheap internet for bandwidth, with failover only when a link dies completely. SD-WAN measures loss, latency and jitter on every path continuously and steers each application onto whichever path currently meets its needs — voice over MPLS, Microsoft 365 straight out the local internet link. The WAN went from "primary and backup" to "all paths, all the time, chosen per app."

Story Time : Old WAN routing is like driving to work on the same route every day and only changing route when the road is **closed**. SD-WAN is like a live traffic app that checks every road every few seconds. If the motorway is open but jammed, it sends you down the side road. It also gives different answers to different drivers: the ambulance (voice) gets the fastest road, the delivery van (backup traffic) gets the cheapest. (Where it breaks: a traffic app can only advise you. An SD-WAN router actually moves each application's packets onto the chosen path itself.)

1.8 Cloud and Hybrid Designs

The problem this solves: applications that used to sit in your own data center now run at Amazon, Microsoft or a software vendor. You need words for who manages which part (the service models), where the cloud lives (the deployment models), and how your network gets there.

Some of the "network" now lives in someone else's data center.

Story Time : Think about dinner. **On-premises** is cooking at home: you buy the kitchen, the food, and do the washing up. **IaaS** (Infrastructure as a Service) is renting a fully equipped kitchen: the oven and power are provided, but you bring the ingredients and cook. **PaaS** (Platform as a Service) is a meal kit: the ingredients arrive measured and you just assemble. **SaaS** (Software as a Service) is eating in a restaurant: you only choose from the menu. Each step up hands more of the work, and more of the control, to someone else.

Model	You manage	Provider manages	Example
On-premises	Everything	Nothing	Your own data center
IaaS	OS, apps, data, virtual network design	Hardware, hypervisor, physical network	AWS EC2, Azure VMs
PaaS	Apps and data	Everything underneath	Managed databases, app platforms
SaaS	Your data and users	The whole application	Microsoft 365, Webex

Deployment	Meaning
Public cloud	Shared provider infrastructure, rented on demand
Private cloud	Cloud-style self-service on infrastructure dedicated to one organization
Hybrid cloud	On-prem and public cloud connected and used together
Multicloud	More than one public cloud provider in use

How do you actually connect to a cloud? Three common ways: an **IPsec VPN over the internet** (cheap, quick, variable performance), a **dedicated private circuit** such as AWS Direct Connect or Azure ExpressRoute (predictable, costs more, usually runs eBGP to the provider — Chapter 7), or an **SD-WAN cloud on-ramp**, where virtual SD-WAN routers run inside the cloud and join your fabric.

Why does cloud change network design at all, if it's "just another site"? Because traffic patterns flip. When applications lived in your data center, branch traffic hairpinned through headquarters and the HQ firewall. With SaaS, sending a branch's Microsoft 365 traffic across the WAN to HQ, out HQ's internet, and back adds latency for no benefit. Modern designs use **direct internet access (DIA)** at branches for trusted SaaS, with security delivered in the cloud. The center of gravity moved off-site, so the WAN design had to follow it.

1.9 Choosing a Design — Putting It Together

Situation	Sensible starting design	Why
One building, ~500 users	Two-tier collapsed core, L2 access with StackWise Virtual pair	Cheap, simple, no blocked links
Large campus, many buildings	Three-tier, routed or virtual-switching distribution, summarization at distribution	Scales by block, small failure domains
Campus that needs segmentation and mobility at scale	SD-Access fabric	Routed underlay, VXLAN overlay, policy by group
Data center	Spine-leaf with VXLAN EVPN (Ethernet VPN, a BGP-based control plane for VXLAN) overlay	Predictable east-west, scale-out
Many branches, mixed transports	SD-WAN	Per-app path selection across MPLS, internet, LTE

Every row is the same few principles — hierarchy, modularity, small failure domains, and the simplest thing that meets the requirement — applied to a different situation. When an exam question describes a scenario and asks for "the best design," find the requirement that dominates (scale? cost? east-west traffic? segmentation?) and the answer usually picks itself.

Chapter 2: High Availability: Redundancy, SSO/NSF & Stacking

> **Blueprint:** ENCOR 1.1.b — High availability techniques such as redundancy, FHRP, and SSO.

2.1 What "Highly Available" Actually Means

The problem this solves: every device eventually fails. Power supplies die, software crashes, someone unplugs the wrong cable. High availability (HA) is the set of techniques that keep the network working while that happens, so users never notice.

Availability is the fraction of time a service works. It's quoted in "nines," and each extra nine cuts the allowed downtime tenfold:

Availability	Downtime per year	Downtime per month
99% ("two nines")	~3.65 days	~7.3 hours
99.9%	~8.8 hours	~44 minutes

Availability	Downtime per year	Downtime per month
99.99%	~53 minutes	~4.4 minutes
99.999% ("five nines")	**~5.3 minutes**	**~26 seconds**

How the numbers are worked out: a year has 365 × 24 × 60 = **525,600 minutes**. At 99.9% the service may be down 0.1% of that: 525,600 × 0.001 = **525.6 minutes**, about 8.8 hours. At 99.999% it's 525,600 × 0.00001 = **5.3 minutes**.

Why do the nines matter for design, not just for contracts? Because five nines allows only about five minutes of downtime a year. A single router reboot takes longer than that. So a five-nines target is impossible with any single device, however reliable, and that one fact forces every high-availability technique in this chapter. You don't reach five nines by buying a better box. You reach it by making sure that when any one box fails, nobody notices.

High availability has two halves, and good designs need both:

1. **Redundancy:** have a second one of everything that matters (link, device, power supply, supervisor, path).
2. **Fast failover:** when the first one fails, switch to the second one **fast enough that applications don't break**. A redundant path that takes 50 seconds to start forwarding (classic STP) is redundancy without availability.

What does "fast enough" mean? Roughly: a TCP session survives a few seconds of loss; a voice call starts to break up after ~150–200 ms of loss and drops after a few seconds; some trading and industrial applications care about tens of milliseconds. Those are the targets the techniques below are aiming at.

Worked example: why two boxes beat one better box. Suppose each distribution switch is available 99.9% of the time (down about 8.8 hours a year), and failures are independent.

- **In series** (traffic must cross switch A **and** switch B, no alternative): the path works only when both work. 0.999 × 0.999 = **0.998**, i.e. 99.8%. Adding devices in a chain makes things **worse**: about 17.5 hours down a year.
- **In parallel** (traffic can use A **or** B, with instant failover): the path fails only when both are down at the same time. The chance of that is 0.001 × 0.001 = 0.000001, so availability is 1 − 0.000001 = **99.9999%**: about 32 seconds a year.

Two ordinary switches in parallel beat any single switch you can buy. The catch is the words "instant failover". If switching to B takes 50 seconds every time A fails, you lose most of that gain. That's why the rest of this chapter is about making failover fast.

2.2 The Layers of Redundancy

The problem this solves: a network is a chain of parts: power, the switch's brain, the switch itself, cables, the gateway, the path, the carrier. A chain is only as strong as its weakest link, so a spare at one layer is wasted if another layer has none. This table is the checklist.

Redundancy has to exist at every layer, because a failure at any one of them is still an outage.

Layer	Single point of failure	Redundancy technique	Chapter
Power	One PSU, one circuit	Dual PSUs on separate circuits/UPS	—
Supervisor / route processor	One control-plane CPU	Dual supervisors with **SSO**	2.3
Chassis / switch	One switch	**Stacking**, **StackWise Virtual**, dual devices	2.5–2.6
Link	One cable	**EtherChannel**, ECMP routing	3, 5
Default gateway	One router IP for hosts	**FHRP** (HSRP, VRRP, GLBP)	2.4, 9
Layer 2 path	One forwarding path	STP alternate ports (RSTP), virtual switching	4
Layer 3 path	One route	Dynamic routing + **BFD** for fast detection	5, 25
WAN / site	One carrier	Dual carriers, SD-WAN	1.7, 14

Why isn't adding a second device enough on its own? Because the second device only helps if the network **knows to use it** — and knows quickly. Two routers with no FHRP still leave hosts pointing at one dead IP; two uplinks with classic STP still sit 30–50 seconds in listening/learning. Redundancy is the spare tire; the failover mechanism is knowing how to change it in seconds.

2.3 Supervisor Redundancy: RPR, SSO and NSF

A modular switch (Catalyst 9400/9600) or a router with two route processors has two control-plane "brains." The question is how ready the standby brain is when the active one dies.

(Terms: the control plane is the "thinking" part: routing protocols, STP, the CLI. The data plane is the "doing" part: the line-card hardware that forwards packets. The RIB is the routing table the control plane builds; the FIB is the copy of it loaded into the forwarding hardware.)

Story Time : A plane has a pilot and a co-pilot. With **RPR**, the co-pilot is asleep in the cabin. If the pilot collapses, someone has to wake the co-pilot, walk them to the cockpit and brief them, and the plane is flying itself badly in the meantime. With **SSO**, the co-pilot is already in the right-hand seat, has been watching every instrument, and knows the heading, altitude and radio frequencies. They take the controls in a second. **NSF** is the autopilot that keeps the plane level during that second. (Where it breaks: in a switch the "autopilot", the forwarding hardware, keeps working all the way through, whereas a real autopilot has to be switched on.)

Mode	Standby state	Switchover time	What's lost
RPR (Route Processor Redundancy)	Partially booted; config not fully applied	Minutes (line cards reset)	Everything — traffic stops

Mode	Standby state	Switchover time	What's lost
SSO (Stateful Switchover)	**Fully booted**, config and state synchronized continuously	~1–3 seconds or less	Routing adjacencies must re-form — unless NSF is on

With **SSO**, the active supervisor continuously copies its startup and running configuration, plus Layer 2 protocol state, the **FIB** (forwarding table) and adjacency tables, to the standby. When the active fails, the standby takes over with the line cards still forwarding.

Why doesn't SSO alone give you a hitless failover for routing? Because routing protocols run on the supervisor CPU, and their sessions with neighbors don't survive the switchover. The new active supervisor has to re-form OSPF, EIGRP and BGP adjacencies. Worse, when a neighbor sees an adjacency drop, it assumes the whole router died and **reroutes around it** — throwing away a forwarding path that's actually still working, because the line cards never stopped. That's what NSF fixes.

NSF and Graceful Restart

NSF (Non-Stop Forwarding) keeps the data plane forwarding on the **last known FIB** during a switchover, while the control plane rebuilds. It works hand-in-hand with **graceful restart** in the routing protocols:

Role	Meaning
NSF-capable	The restarting router itself — has dual supervisors with SSO, can keep forwarding while its control plane restarts
NSF-aware	A neighbor that understands graceful restart — when the restarting router says "I'm restarting, keep using me," it **holds the routes** instead of tearing them down

The sequence is:

1. Active supervisor fails; the standby takes over via SSO. Line cards keep forwarding using the existing FIB.
2. The new active supervisor signals its neighbors (OSPF uses grace LSAs or Cisco's link-local signaling; BGP uses the graceful-restart capability negotiated when the session came up).
3. NSF-aware neighbors **don't** drop the routes. They help the restarting router resynchronize its databases.
4. The restarted control plane rebuilds the RIB, refreshes the FIB, and stale entries are flushed.

Why do the neighbors need to cooperate? Because the restarting router can't stop its neighbors from reacting to a dropped adjacency. If they don't understand graceful restart, they reroute anyway, and the failover becomes a visible outage. NSF is a team agreement: "I'm only rebooting my brain, not my hands — please keep sending me packets."

Worked example: a supervisor crash with SSO + NSF (OSPF, IETF graceful restart). DIST1 is a Catalyst 9400 with two supervisors in SSO, running `nsf ietf` under OSPF. Its OSPF neighbor CORE1 is NSF-aware (graceful-restart helper support is on by default in IOS XE).

1. **t = 0 s.** Supervisor 1 (active) crashes. Supervisor 2 (STANDBY HOT) detects it and becomes active. The line cards keep forwarding with the FIB they already have.
2. **t ≈ 1 s.** The new active supervisor has the configuration and Layer 2 state (it was synchronized), but no OSPF neighbors yet. Before forming them again, it sends a **grace LSA** (an OSPF Type 9 opaque LSA) to CORE1. In plain words: "I'm restarting. Keep me in your SPF tree for the next 120 seconds." 120 s is the default restart interval.
3. CORE1 enters **helper mode**. It keeps advertising its adjacency with DIST1 as if nothing happened, so no other router reruns SPF and no traffic is moved.
4. DIST1 re-forms the adjacency, gets CORE1's database again, rebuilds its RIB and updates the FIB. Stale entries are removed. It flushes the grace LSA, which ends helper mode.
5. Result: users saw no outage, and no router outside DIST1 and CORE1 even knew there was a restart.

Without NSF, at step 2 CORE1 would have seen DIST1's adjacency drop (after its dead timer, or instantly if BFD was on). It would have flooded new LSAs, every router in the area would have rerun SPF, and traffic would have moved away from a box that was forwarding perfectly well.

```
! Supervisor redundancy (default on most dual-sup platforms)
SW(config)# redundancy
SW(config-red)# mode sso

! Graceful restart per routing protocol
SW(config)# router ospf 1
SW(config-router)# nsf ietf              ! RFC 3623 graceful restart (or: nsf cisco)
SW(config)# router eigrp 100
SW(config-router)# nsf
SW(config)# router bgp 65001
SW(config-router)# bgp graceful-restart
```

```
SW# show redundancy states
       my state = 13 -ACTIVE
     peer state = 8  -STANDBY HOT
           Mode = Duplex
 Redundancy Mode (Operational) = sso
 Redundancy Mode (Configured)  = sso
```

Read it: `STANDBY HOT` is what you want — the peer is fully synchronized and ready for a stateful switchover. A peer stuck in `STANDBY COLD` means it's running RPR-style (often due to a software or configuration mismatch between supervisors), and a failover will be disruptive.

> **Exam trap:** NSF and **fast-converging** timers pull in opposite directions. NSF wants neighbors to wait through a restart; very aggressive hello/dead timers or BFD want neighbors to detect failure instantly. If BFD declares the neighbor down in 150 ms, it can tear down the adjacency before graceful restart finishes. On dual-supervisor boxes, set timers so that a supervisor switchover completes inside them.

2.4 First Hop Redundancy — The Architecture

Hosts are configured with **one** default gateway IP. If that router dies, the host can't reach anything off-subnet, however many other routers exist. A **First Hop Redundancy Protocol (FHRP)** lets two or more routers share a **virtual IP and virtual MAC**, so the gateway the hosts know never disappears.

Story Time : A company prints one support phone number on its website. Behind that number, calls ring whichever engineer is on duty. When one engineer goes home, the next one picks up, and customers never need a new number. The **virtual IP and virtual MAC** are that published number. The routers are the engineers on the rota. (Where it breaks: HSRP and VRRP have exactly one router forwarding for the virtual IP at a time. GLBP is the exception: one router answers ARP, handing out different virtual MACs, so several routers forward together.)

Worked example: an HSRP failover with default timers. VLAN 10 is 10.1.10.0/24. R1 (10.1.10.2, priority 110) and R2 (10.1.10.3, priority 100) share HSRP group 10 with virtual IP **10.1.10.1**. HSRP version 1 builds the virtual MAC from the group number: **0000.0c07.ac0a** (0a is 10 in hex). Every host has 10.1.10.1 as its gateway and 0000.0c07.ac0a in its ARP cache.

1. Normal: R1 is active (higher priority) and owns the virtual MAC. It sends a hello every **3 s**.
2. **t = 0 s.** R1 loses power. R2 stops hearing hellos.
3. **t = 10 s** (at most). The **hold time** of 10 s runs out on R2. R2 becomes active.
4. R2 sends a **gratuitous ARP** for 10.1.10.1 from 0000.0c07.ac0a. The access switches learn that MAC on the ports toward R2.
5. Hosts change **nothing**. Their gateway IP and the MAC in their ARP cache are the same as before. Traffic flows again.

Total loss: up to 10 seconds with default timers. Chapter 9 shows how to cut that to under a second with smaller timers or tracking.

FHRP	Standard	Roles	Load balancing within one VLAN	Default timers (hello/hold)
HSRP	Cisco	Active / Standby	No (split VLANs between routers instead)	3 s / 10 s
VRRP	IETF (RFC 5798 for v3)	Master / Backup	No	1 s advertisement / ~3.6 s master-down interval
GLBP	Cisco	AVG / AVF	**Yes** — different hosts get different virtual MACs	3 s / 10 s

Why is the FHRP decision an architectural one, not just a config detail? Because it has to line up with everything else in the design:

- In a **looped Layer 2 access** design, the FHRP active router for a VLAN should also be the **STP root** for that VLAN (Chapter 4), or traffic takes a detour across the inter-distribution link.

- Spreading load means making **different routers active for different VLANs** (odd VLANs on one distribution switch, even on the other), because HSRP and VRRP don't balance within one VLAN.
- In **routed access**, there's no FHRP at all — the access switch is the only gateway.
- In **StackWise Virtual / VSS**, there's no FHRP either — the two physical switches are one logical gateway with one IP.

Chapter 9 covers HSRP and VRRP configuration, priority, preemption and object tracking in detail.

2.5 Stacking: Many Switches, One Brain

The problem this solves: a wiring closet with 300 users needs several 48-port switches. Managed separately, that's several configs to keep in step, several uplinks fighting STP, and several devices to log in to. Stacking makes them behave as one.

StackWise (on Catalyst 9200 and 9300, e.g. StackWise-480 on the 9300) joins up to eight access switches with dedicated stacking cables into **one logical switch**: one management IP, one configuration, one control plane.

Stack role	Job
Active	Runs the control plane and management for the entire stack
Standby	Synchronized via **SSO**; takes over if the active fails
Members	Forward traffic using tables programmed by the active

The active is elected by: (1) the current active keeps the role (the election is not preemptive), (2) highest **stack priority** (1–15, default 1), then platform tiebreakers ending in (3) the **lowest MAC address**. Because the election only happens when the stack boots or the active fails, pin the active and standby deliberately:

```
Stack# switch 1 priority 15        ! privileged EXEC, not config mode
Stack# switch 2 priority 14        ! intended standby
Stack# show switch
Switch/Stack Mac Address : 7c21.0e5a.1b00 - Local Mac Address
                                             H/W   Current
Switch#   Role    Mac Address     Priority Version  State
-----------------------------------------------------------
*1       Active   7c21.0e5a.1b00     15     V01     Ready
 2       Standby  7c21.0e5a.2c80     14     V01     Ready
 3       Member   7c21.0e5a.3d00     1      V01     Ready
```

Worked example: the election, then a failure. Using the stack above (priorities 15, 14 and 1):

1. All three switches boot together. No active exists yet, so priority decides: switch 1 (15) becomes **active**, switch 2 (14) becomes **standby**, switch 3 is a **member**.
2. Switch 1 fails. Switch 2, already synchronized by SSO, becomes **active** at once. The stack then elects a new standby from the switches left: switch 3.
3. Switch 1 is repaired and rejoins. It has the highest priority, but the election is **not preemptive**, so it does not take the active role back. Switch 2 stays active, switch 3 stays

standby, and switch 1 joins as a **member**.

4. To make switch 1 active again you would have to reload the active (switch 2) during a maintenance window. That is why you set priorities before go-live.

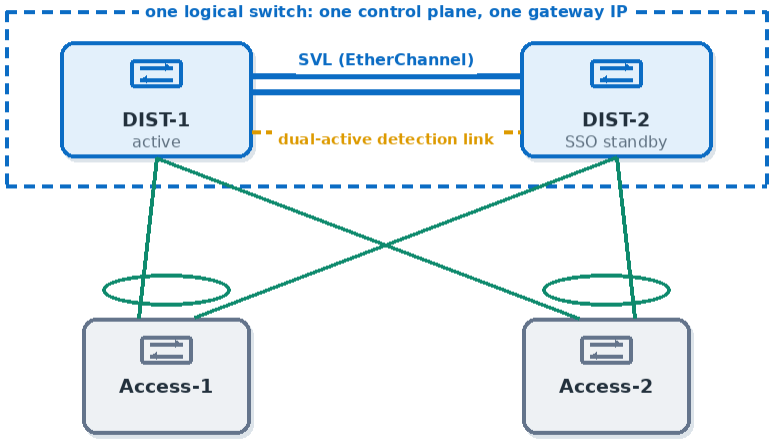
Why is a stack better than separate access switches? Two reasons. First, **cross-stack EtherChannel**: an uplink bundle can use one port on switch 1 and one on switch 3, so losing a whole switch only loses half the uplink bandwidth — with no STP blocking, because the bundle is one logical link. Second, **operational simplicity**: eight switches managed as one. A stack turns "eight devices, eight configs" into "one device with eight line cards."

2.6 StackWise Virtual (and VSS Before It)

The problem this solves: two independent distribution switches give you redundancy, but at a price. STP blocks half the uplinks, HSRP has to fail over the gateway, and you have two control planes to keep consistent. StackWise Virtual keeps the two boxes but removes those costs.

StackWise Virtual (SV) does the same thing for **two distribution or core switches** (Catalyst 9400/9500/9600): two physical chassis become **one logical switch** with one control plane. Its predecessor, **VSS** (Virtual Switching System), did the same on the Catalyst 6500/6800 and 4500-X.

StackWise Virtual — two chassis, one logical switch



MEC: one EtherChannel per access switch, a member to each chassis — nothing blocks

Component	Purpose
StackWise Virtual Link (SVL)	High-speed EtherChannel between the two switches carrying control traffic and data that has to cross between them (VSS called this the **VSL**)
Dual-Active Detection (DAD) link	Separate link used only to detect the case where the SVL fails but both switches are alive
Multichassis EtherChannel (MEC)	An access switch's uplink bundle with one member to each physical switch — looks like one ordinary EtherChannel to the access switch

Component	Purpose
One active control plane	One switch runs the control plane and management; the other is **SSO standby**. Both forward data.

Why does StackWise Virtual remove the need for both STP blocking and FHRP? Look at what the access switch sees: one upstream "switch" and one EtherChannel to it. A single logical link has no loop for STP to block, so **every uplink forwards**. And because there's only one logical distribution switch, there's only one gateway SVI — nothing for HSRP to fail over between. Failover becomes "an EtherChannel member went down," which takes milliseconds. You get the redundancy of two boxes with the simplicity of one.

Why is the Dual-Active Detection link essential? Imagine the SVL fails while both switches are still running. The standby no longer hears the active, assumes it died, and becomes active too. Now there are **two** switches using the same IP addresses, the same router ID and the same MAC addresses — a **dual-active** split brain that corrupts routing, ARP and the forwarding tables of every neighbor. The DAD link (or **ePAgP**, enhanced PAgP, which passes the check through an access switch's EtherChannel) lets the switches discover that they're both alive. The switch that was formerly the active goes into **recovery mode**, shuts down all its interfaces except the SVL and DAD links, and waits for the SVL to come back. Losing half the box is a minor event; having two boxes both believe they're the whole box is a disaster.

How it plays out, step by step:

1. SW1 is active, SW2 is standby. The SVL (Te1/0/47–48 on each) carries their heartbeat.
2. Both SVL cables are cut. SW2 stops hearing SW1 and, by SSO rules, takes over as active.
3. Over the separate DAD link (Te1/0/46), the switches see each other's messages. Both are alive, so this is **dual-active**, not a failure.
4. SW1, the old active, enters **recovery mode** and shuts all its ports except the SVL and DAD ports.
5. SW2 alone forwards. Access switches lose one member of each MEC (multichassis EtherChannel) and carry on over the other.
6. When the SVL is repaired, SW1 reloads and rejoins as standby.

Worked example: losing a whole distribution switch. Access switch ACC1 has Port-channel 1 with Te1/1/1 to SW1 (Te1/0/1) and Te1/1/2 to SW2 (Te2/0/1). To ACC1 it's one 20 Gbps uplink. SW1 loses power:

- ACC1 sees one member go down. The bundle stays up at 10 Gbps, and flows that hashed to Te1/1/1 move to Te1/1/2 within milliseconds.
- SW2 becomes the active control plane through SSO. The gateway SVI (10.1.10.1) was already "on" the logical switch, so hosts don't change anything. No HSRP and no STP change.

```
! On each switch, before conversion (then reload both)
SW1(config)# stackwise-virtual
SW1(config-stackwise-virtual)# domain 10
SW1(config)# interface range TenGigabitEthernet1/0/47 - 48
SW1(config-if-range)# stackwise-virtual link 1          ! these form the SVL
SW1(config)# interface TenGigabitEthernet1/0/46
SW1(config-if)# stackwise-virtual dual-active-detection ! DAD link
SW1# write memory
SW1# reload

SW# show stackwise-virtual          ! domain, switches, SVL members
SW# show stackwise-virtual link     ! SVL port state: should be U (up)
SW# show stackwise-virtual dual-active-detection
SW# show redundancy states          ! one ACTIVE, one STANDBY HOT
```

After conversion, interfaces are numbered with the switch number first, so the same port appears as `Te1/0/1` on switch 1 and `Te2/0/1` on switch 2. A multichassis EtherChannel then uses one interface from each.

Comparing the Options

Technique	Scope	Control planes	FHRP needed	STP blocks uplinks
Two independent switches + HSRP	Distribution	2	Yes	Yes (looped L2)
StackWise	Access (up to 8)	1 (active + SSO standby)	n/a at access	No, with cross-stack EtherChannel
StackWise Virtual / VSS	Distribution / core (2)	1 (active + SSO standby)	**No**	**No** (MEC)
Routed access	Access to distribution	1 per switch	**No**	**No** (no L2 uplinks)

2.7 ISSU — Upgrading Without an Outage

Software has to be upgraded, and a reboot costs minutes — which blows the five-nines budget in one go. **ISSU (In-Service Software Upgrade)** uses the SSO pair to upgrade one half at a time:

1. Load the new image onto the **standby**, which reboots on it.
2. Do a **switchover**: the upgraded standby becomes active (SSO keeps forwarding).
3. Upgrade the old active, which comes back as the new standby.
4. Commit the upgrade (or roll back if something's wrong).

On IOS XE install mode, the whole sequence runs with one command, for example `install add file bootflash:<image>.bin activate issu commit`.

Why does ISSU need SSO underneath it? Because each step in the sequence is a supervisor switchover. If the switchover weren't stateful, every step would be an outage. ISSU is just SSO used on purpose, twice, with an upgrade in between.

2.8 Putting HA Together — Matching Techniques to Failures

The problem this solves: each technique protects against a different failure. On the exam (and at work) you get a failure and have to name the technique that covers it, and how fast recovery will be.

Failure	What saves you	Typical recovery
Supervisor crash	SSO + NSF	Sub-second to ~3 s, routing stays up

Failure	What saves you	Typical recovery
One switch in a stack dies	Stack SSO + cross-stack EtherChannel	Milliseconds for traffic on surviving members
Distribution switch dies (SV pair)	StackWise Virtual + MEC	Milliseconds (EtherChannel member loss)
Distribution router dies (independent pair)	HSRP/VRRP + routing reconvergence	~1–10 s depending on timers
Uplink fiber cut	EtherChannel or ECMP	Milliseconds
Neighbor silently stops forwarding	BFD + routing protocol	~150–900 ms with typical BFD timers
WAN carrier outage	Dual carriers, SD-WAN, IP SLA tracking	Seconds

Every high-availability technique is the same idea: have a second one, keep it warm, and make the switch-over faster than the application can notice.

Chapter 3: Layer 2 Revisited: Trunking & EtherChannel Troubleshooting

> **Blueprint:** ENCOR 3.1.a — Troubleshoot static and dynamic 802.1q trunking protocols; ENCOR 3.1.b — Troubleshoot static and dynamic EtherChannels.

CCNA taught you how to build trunks and EtherChannels (CCNA guide 8.1–8.2, 10.1–10.4). ENCOR's verb is **troubleshoot**. That means knowing every way they fail, what each failure looks like in `show` output, and which single command proves the cause. This chapter is organized around exactly that.

3.1 Static vs Dynamic Trunking — A Quick Recap

The problem this solves: two switches both have users in VLANs 10, 20 and 30. Running one cable per VLAN between them would waste ports and cables. A trunk lets one link carry all of them, as long as each frame says which VLAN it belongs to.

A trunk carries many VLANs over one link by adding an **802.1Q tag** (4 bytes, holding a 12-bit VLAN ID) to every frame except those in the **native VLAN**, which cross untagged.

Story Time : A trunk is a delivery van running between two warehouses. Every crate gets a sticker with its department number (the **tag**), so the receiving warehouse knows which room to put it in. One department's crates travel **without** a sticker: that's the native VLAN. The receiving warehouse puts every unlabelled crate in its own "default" room. That works perfectly until the two warehouses disagree about which room is the default (3.2).

There are two ways to make a link a trunk:

- **Static:** `switchport mode trunk` — the port is a trunk, full stop.
- **Dynamic:** let **DTP (Dynamic Trunking Protocol)**, a Cisco protocol, negotiate. DTP frames are sent about every 30 seconds to the Cisco multicast MAC `0100.0ccc.cccc`.

Mode	What it does	Sends DTP?
`switchport mode access`	Always access	**Yes**: it still sends DTP, to tell the neighbor "I am not a trunk". Stop it with `nonegotiate`
`switchport mode trunk`	Always trunk	**Yes** — still sends DTP unless `nonegotiate`
`switchport mode dynamic desirable`	**Actively asks** to trunk	Yes
`switchport mode dynamic auto`	Trunks only **if asked**	Yes (but only responds)
`switchport nonegotiate`	Turns DTP off (used with trunk or access)	No

On Catalyst 9000 switches, the default for a Layer 2 port is **`dynamic auto`**.

Trunk Encapsulation — the Command That Gets Forgotten

Switches that support both **ISL** (Cisco's legacy trunk encapsulation) and 802.1Q default to **`switchport trunk encapsulation negotiate`** (shown as "Auto"). On those platforms — including many older Catalysts and the **IOL L2 images** you'll use in EVE-NG — a port can't be made a static trunk until you pick the encapsulation:

```
SW1(config-if)# switchport mode trunk
Command rejected: An interface whose trunk encapsulation is "Auto" can not be configured
to "trunk" mode.
SW1(config-if)# switchport trunk encapsulation dot1q     ! choose 802.1Q explicitly
SW1(config-if)# switchport mode trunk                    ! now accepted
```

Catalyst 9000 switches support only 802.1Q, so the `encapsulation` command doesn't exist there and the trunk just works. **Why does the switch refuse rather than guess?** Because a static trunk sends tagged frames without negotiating. If it guessed ISL while the neighbor used 802.1Q, every frame would be unreadable on the far side. Making you choose removes the guess. If `switchport mode trunk` is rejected in a lab, this is almost always why.

The DTP Outcome Matrix (Memorize This)

Side A ↓ / Side B →				
access	Access	Access	Access	**Mismatch**
dynamic auto	Access	**Access**	Trunk	Trunk
dynamic desirable	Access	Trunk	Trunk	Trunk
trunk	**Mismatch**	Trunk	Trunk	Trunk

Why do two `dynamic auto` ports end up as access? Because `auto` is purely passive. It will agree to trunk if the other side asks, but it never asks. Two passive ports wait politely for each other forever, and the link falls back to access. This catches people out because both ports are willing to trunk. Willing on both sides isn't enough — somebody has to start the conversation.

How DTP negotiation runs, step by step (SW1 Gi1/0/1 `dynamic desirable`, SW2 Gi1/0/1 `dynamic auto`, same VTP domain):

1. SW1 sends DTP frames saying "I **desire** a trunk".
2. SW2 is `auto`: it would never start, but it does **answer**. It sees the request and agrees.
3. Both sides agree on the encapsulation (802.1Q on modern switches) and become trunks. `show interfaces trunk` on SW1 shows Mode `desirable`, Status `trunking`. On SW2 it shows Mode `auto`.
4. Change SW1 to `dynamic auto` too, and step 1 never happens. Each side only waits to be asked, so both ports operate as access ports in their access VLAN (VLAN 1 by default).

Why is "trunk on one side, access on the other" a mismatch and not just "access"? Because a static trunk doesn't wait for agreement. It sends tagged frames regardless, while the access side expects untagged frames in one VLAN. Traffic in the native VLAN may still work (it's untagged on both ends), while every other VLAN silently fails. Partial connectivity is the classic symptom of a trunking mismatch.

Why does `switchport nonegotiate` cause problems when the other side is dynamic? With `nonegotiate`, the port sends no DTP at all. A neighbor in `dynamic auto` or `dynamic desirable` never hears a DTP reply, so it never becomes a trunk and stays access, while your side is a hard trunk: a mismatch.

> **Best practice and exam answer:** hard-code both ends — `switchport mode trunk` plus `switchport nonegotiate` on switch-to-switch links, and `switchport mode access` on user ports. DTP is a security risk (an attacker's device can negotiate a trunk and reach every VLAN — **switch spoofing**) and an unpredictability risk.

The VTP Domain Trap

DTP carries the **VTP domain name**. If the two switches have **different VTP domain names**, DTP won't negotiate a trunk (statically configured trunks are unaffected). So if a dynamic trunk refuses to form between switches that look correctly configured, check `show vtp status` on both ends.

Reading `show interfaces switchport`

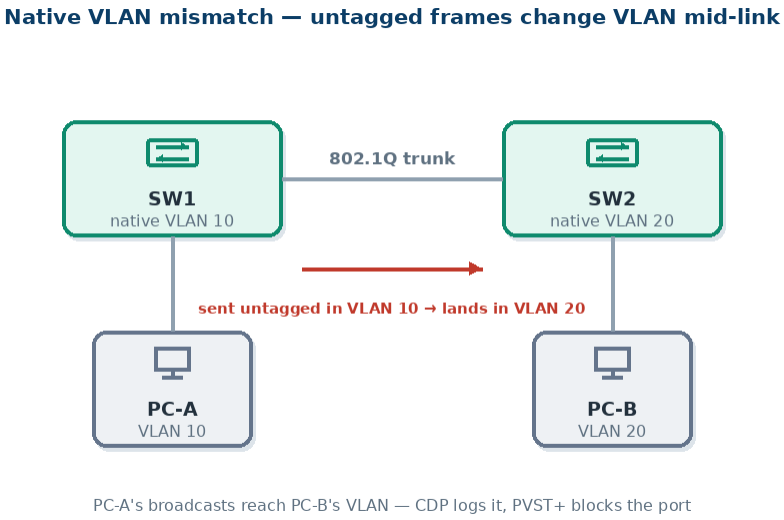
`show interfaces trunk` lists only ports that **are** trunking. When a link should be a trunk and isn't, this is the command that shows what each side was told to do and what actually happened:

```
SW1# show interfaces Gi1/0/1 switchport
Name: Gi1/0/1
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
```

Field	What it tells you
Administrative Mode	What you configured (`dynamic auto`, `trunk`, `static access`...)
Operational Mode	What the port is **actually doing** right now (`trunk` or `static access`)
Administrative Trunking Encapsulation	`dot1q`, `isl` or `negotiate`; "negotiate" on a static trunk explains a rejected command
Negotiation of Trunking	`On` = DTP is running; `Off` = `nonegotiate` is set
Access Mode VLAN	The VLAN used if the port ends up access
Trunking Native Mode VLAN	This side's native VLAN; compare it with the neighbor's
Trunking VLANs Enabled	The allowed list (3.3)

Reading the example: the port is configured `dynamic auto` but is operating as `static access`, and DTP is on. Combined with the matrix above, the neighbor must also be `auto` (or access): nobody asked to trunk. Administrative vs operational mode is the first thing to compare on any "why isn't this a trunk?" question.

3.2 Native VLAN Mismatch



The problem this causes: the native VLAN is the one setting both ends of a trunk must agree on without any tag to check it. Get it wrong and frames silently move from one VLAN into another.

Each end of a trunk decides on its own which VLAN untagged frames belong to. If SW1's native VLAN is 10 and SW2's is 20, an untagged frame SW1 sends from VLAN 10 arrives at SW2 and is placed in **VLAN 20**. Traffic **leaks between VLANs**, which is both a security problem and very confusing.

Worked example: following one frame across the mismatch. SW1's trunk has native VLAN 10; SW2's end has native VLAN 20. PC-A (10.1.10.5, VLAN 10) is on SW1. PC-B (10.1.20.7, VLAN 20) is on SW2.

1. PC-A sends a broadcast ARP. SW1 floods it out the trunk. VLAN 10 is SW1's native VLAN, so the frame leaves **untagged**.
2. SW2 receives an untagged frame. It has no tag to read, so it uses **its own** native VLAN and puts the frame in **VLAN 20**.
3. SW2 floods the broadcast to its VLAN 20 ports. PC-B receives an ARP from a host in a different VLAN and subnet. VLAN 10's broadcasts are now leaking into VLAN 20, and the same happens in reverse.
4. Within seconds, the safety nets react. CDP logs the mismatch, and Rapid PVST+ notices that BPDUs for VLAN 10 arrive on a port whose native VLAN is 20. It marks the port **PVID-inconsistent** and blocks it for VLANs 10 and 20, as below. The leak stops, but so does all VLAN 10 and 20 traffic on that trunk until you fix the config.

Cisco switches detect it in two ways:

- **CDP** reports it: `%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on GigabitEthernet1/0/1 (10), with SW2 GigabitEthernet1/0/1 (20).`
- **PVST+ / Rapid PVST+** carries the sending VLAN inside its BPDUs. When the VLAN in the BPDU doesn't match the receiving port's native VLAN, the port is put into a **PVID-inconsistent** (port VLAN ID) state and **blocked** for the affected VLANs, so the leak doesn't become a loop.

```
SW2# show spanning-tree inconsistentports
Name                 Interface              Inconsistency
-------------------- ---------------------- ------------------
VLAN0010             GigabitEthernet1/0/1   Port VLAN ID Mismatch
VLAN0020             GigabitEthernet1/0/1   Port VLAN ID Mismatch
```

Fix: make the native VLAN match on both ends — and in production, set it to an **unused VLAN** that carries no user traffic, so an untagged frame never lands anywhere useful. That also defeats **double-tagging (VLAN hopping)** attacks, which depend on the attacker's access VLAN being the trunk's native VLAN.

```
SW(config-if)# switchport trunk native vlan 999
SW(config)# vlan dot1q tag native          ! optional: tag native VLAN frames too
```

3.3 Allowed VLAN Lists and Pruning

The problem this solves: not every VLAN needs to cross every trunk. Sending VLAN 30's broadcasts to a switch with no VLAN 30 users wastes bandwidth and widens VLAN 30's failure domain. Allowed lists (manual) and pruning (automatic) trim each trunk to the VLANs it really needs. The flip side: when a VLAN "doesn't work" across a trunk, one of these filters is often the reason.

A trunk can only carry VLANs that are **(1) allowed** on the trunk, **(2) exist and are active** on the switch, and **(3) not pruned** and in STP forwarding state. `show interfaces trunk` shows exactly those three filters in order:

```
SW1# show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gi1/0/1     on           802.1q         trunking      999

Port        Vlans allowed on trunk
Gi1/0/1     10,20,30,999

Port        Vlans allowed and active in management domain
Gi1/0/1     10,20,999

Port        Vlans in spanning tree forwarding state and not pruned
Gi1/0/1     10,999
```

How to read it, line by line:

- VLAN 30 is **allowed** but missing from "allowed and active" → **VLAN 30 doesn't exist** on this switch (`show vlan brief`). Create it.
- VLAN 20 is active but missing from the last list → it's either **blocked by STP** on this port (`show spanning-tree vlan 20`) or **pruned** by VTP because the neighbor has no ports in it.
- `Mode on` means statically configured trunk; `desirable`/`auto` means DTP formed it.

That one command is the whole trunk-troubleshooting flowchart: allowed → exists → forwarding.

> **Exam trap — the most destructive typo in switching:** `switchport trunk allowed vlan 40` **replaces** the entire list with just VLAN 40, instantly cutting every other VLAN on that trunk. To add a VLAN, use `switchport trunk allowed vlan add 40`. (`remove`, `except` and `all` also exist.)

3.4 VTP — Version 3 and Why It Exists

VTP (CCNA guide 8.3) synchronizes the VLAN database from a server to clients. Its notorious failure: a switch with a **higher configuration revision number** joins the domain and **overwrites every switch's VLAN database** with its own — possibly deleting every VLAN in production.

Worked example: the lab switch that deleted production. The production VTP domain `CORP` (version 2, no password) is at **configuration revision 42** and holds VLANs 10, 20 and 30. An engineer takes a switch from the lab, where it was in domain `CORP` too and has been changed many times: it's at **revision 57** but holds only VLAN 1.

1. The lab switch is plugged into a production trunk. It sends a VTP advertisement: domain `CORP`, revision 57.
2. Every production switch compares: 57 is higher than 42, so the advertisement must be "newer". This is true even if the lab switch is a VTP **client**: in v1/v2, any switch with a higher revision is believed.
3. Every switch in the domain replaces its VLAN database with the lab switch's: VLANs 10, 20 and 30 are deleted.
4. Every access port in those VLANs becomes **inactive**, because its VLAN no longer exists. Users across the site lose connectivity.

The prevention under v1/v2 was to reset the revision to 0 before connecting any switch (change the domain name or set transparent mode, then change it back). VTPv3 makes it structural instead:

VTP version 3 addresses this and more:

Feature	VTPv1/v2	VTPv3
Who can change VLANs	Any server	Only the **primary server** (one per domain, per feature)
Accidental overwrite protection	None	A new switch can't become primary without an explicit takeover
Extended VLANs (1006–4094)	Not propagated	**Propagated**
MST instance mapping	No	**Yes** (can propagate MST config)

```
SW1(config)# vtp domain CORP
SW1(config)# vtp version 3
SW1(config)# vtp mode server
SW1# vtp primary vlan               ! exec mode: become primary server (prompts to confirm)
SW1# show vtp status
```

Why is "transparent" or "off" still the most common real-world VTP setting? Because even VTPv3 is a mechanism that lets one switch change many switches' VLAN databases. Most enterprises prefer to configure VLANs explicitly, increasingly through automation (Chapters 19–21), so the blast radius of one mistake stays at one switch.

3.5 EtherChannel Negotiation — The Mode Matrix

The problem this solves: two switches joined by four parallel links would be a loop, so STP would block three of them. EtherChannel tells STP (and everything else) that the four links are **one** logical link. All four carry traffic, and losing one only loses a quarter of the bandwidth.

Story Time : A bundle is a four-lane motorway between two towns. Every car (a **flow**: one conversation between two addresses) is assigned a lane at the on-ramp, by a fixed rule based on where it's coming from and going to, and it stays in that lane. Lots of cars spread across all four lanes. But one very long lorry (a single huge backup flow) can only ever use one lane, which is why one flow never gets more than one link's speed. If a lane closes, its cars are re-assigned to the remaining lanes. (Where it breaks: real lanes let a car change lanes. A flow never changes member unless a link fails or the bundle changes.)

EtherChannel bundles 2–8 active links into one logical **port-channel** (CCNA guide Chapter 10). Members can be bundled statically (`on`) or negotiated by **LACP** (IEEE 802.3ad, now 802.1AX) or **PAgP** (Cisco).

Protocol	Active mode (initiates)	Passive mode (responds)
LACP	`active`	`passive`
PAgP	`desirable`	`auto`
Static	`on` (no protocol)	—

Side A ↓ / Side B →					
on	**Channel**	No	No	No	No
active	No	**Channel**	**Channel**	No	No
passive	No	**Channel**	No	No	No
desirable	No	No	No	**Channel**	**Channel**
auto	No	No	No	**Channel**	No

Read the matrix by three rules: the protocols must match (LACP with LACP, PAgP with PAgP), at least one side must be the initiator (`passive`+`passive` and `auto`+`auto` never form), and **`on` only bundles with `on`**, because `on` sends no protocol frames at all.

Why is `on` the most dangerous mode? Because with no protocol, nothing checks that the other end has really bundled. If SW1 is `on` and SW2 has ports in individual mode (or bundled differently), SW1 sends frames over the members as one logical link while SW2 treats them as separate links. STP on SW2 sees separate ports, while SW1 hashes broadcasts out one member and floods them back from another: the ingredients for a **loop**. LACP and PAgP prevent this by refusing to bundle until both sides agree. Negotiation isn't overhead — it's the safety check.

How an `on`-mode mistake becomes a loop, step by step: SW1 has Gi1/0/1–2 in `channel-group 1 mode on`. SW2's Gi1/0/1–2 are plain trunks (no channel-group).

1. SW1 treats Po1 as one logical port. STP on SW1 has one port toward SW2 and nothing to block there.

2. SW1 sends its BPDUs on Po1, which physically means out **one** member, say Gi1/0/1.
3. SW2 sees two separate trunks. It hears BPDUs only on Gi1/0/1. On Gi1/0/2 it hears none, so STP on SW2 concludes nobody else is on that link and makes Gi1/0/2 **designated, forwarding**. Nothing blocks anywhere.
4. A broadcast from SW2 goes out Gi1/0/1 **and** Gi1/0/2. SW1 receives both copies on Po1 and floods each out its other ports. A broadcast from SW1 that hashes to Gi1/0/1 arrives at SW2, which floods it out Gi1/0/2 straight back to SW1, which floods it again. Frames circle and multiply: a loop.
5. On Catalyst switches the misconfig guard below usually catches this first and err-disables the bundle, which is far better than a loop.

EtherChannel Misconfig Guard

For the `on`-mode danger, STP has a safety net: **EtherChannel misconfiguration guard** (enabled by default on Catalyst switches). If a switch's port-channel receives BPDUs from multiple different source MACs (a sign the far side isn't bundling), it puts the channel's member ports into **err-disabled** state.

```
SW(config)# spanning-tree etherchannel guard misconfig     ! on by default
%PM-4-ERR_DISABLE: channel-misconfig (STP) error detected on Po1, putting Gi1/0/1 in
err-disable state
```

3.6 Why Members Get Suspended — Attribute Mismatches

Even with matching modes, a port joins the bundle **only if its configuration matches the port-channel's**. A mismatched member is **suspended** — up physically, but excluded from the bundle.

Must match across all members (and the port-channel)
Speed and duplex
Switchport mode (access vs trunk) — and Layer 2 vs Layer 3 (`no switchport`)
Access VLAN (if access)
Native VLAN and allowed VLAN list (if trunk)
Trunk encapsulation
LACP/PAgP mode family (can't mix LACP and PAgP members in one channel)

Why must the members match so strictly? Because the switch treats the bundle as **one** interface. If one member allowed VLAN 30 and another didn't, a frame for VLAN 30 hashed onto the wrong member would vanish — and the hash is per-flow (CCNA guide 10.4), so you'd get a bizarre "some conversations in VLAN 30 work, others don't" fault. Suspending the odd member out is the switch refusing to build a link that would behave inconsistently.

Worked example: a suspended member. Po1 is a trunk allowing VLANs 10, 20 and 30. Someone configured Gi1/0/3 directly with `switchport trunk allowed vlan 10,20` before adding it to the channel. The switch logs:

```
%EC-5-CANNOT_BUNDLE2: Gi1/0/3 is not compatible with Po1 and will be suspended
(vlan mask is different)
```

`show etherchannel summary` shows `Gi1/0/3(s)`. The log names the attribute that differs ("vlan mask" means the allowed list). The fix is to make the member's settings match the port-channel's: on Gi1/0/3, set `switchport trunk allowed vlan 10,20,30` (the same list as Po1). Alternatively, `default interface Gi1/0/3`, re-apply the same trunk settings as Po1, then `channel-group 1 mode active`.

> **Best practice:** configure the **port-channel interface**, not the members. Settings applied to `interface Port-channel1` are pushed to every member, so they can't drift apart.

3.7 Reading `show etherchannel summary`

```
SW1# show etherchannel summary
Flags:  D - down        P - bundled in port-channel
        I - stand-alone s - suspended
        H - Hot-standby (LACP only)
        R - Layer3      S - Layer2
        U - in use      f - failed to allocate aggregator
        M - not in use, minimum links not met
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port

Group  Port-channel  Protocol    Ports
------+-------------+-----------+-----------------------------------------
1      Po1(SU)         LACP      Gi1/0/1(P)  Gi1/0/2(P)  Gi1/0/3(s)
2      Po2(SD)         LACP      Gi1/0/5(I)  Gi1/0/6(I)
```

Flag	Meaning	What to check
(SU) on the port-channel	Layer 2, in use — healthy	—
(RU)	Layer 3 port-channel in use — healthy	—
(SD) / **(RD)**	Port-channel is **down**	No member is bundled — look at the member flags
(P)	Member bundled — healthy	—
(s)	**Suspended** — config doesn't match the bundle	`show run int` on the member vs the port-channel; `show interfaces Gi1/0/3 etherchannel`
(I)	**Stand-alone** — no LACP/PAgP partner heard	Far end not configured for the channel, wrong protocol, or `passive`+`passive`
(H)	LACP **hot-standby** (more than 8 members, or max-bundle reached)	Normal by design; lower LACP port priority = preferred active
(D)	Member link down	Layer 1 — cable, optic, `shutdown`
(w)	Waiting to aggregate	Usually transient; persistent means negotiation trouble

Flag	Meaning	What to check
(M)	Min-links not met	`port-channel min-links` set higher than bundled members

Reading the example: Po1 is healthy with two bundled members, but Gi1/0/3 is suspended — it has a configuration mismatch. Po2 is down because both members are stand-alone: they're not hearing LACPDUs, so the far side is probably not configured with LACP (or is `passive` to our `passive`).

Proving What the Partner Is Doing

`show etherchannel summary` shows your side. These two commands show the **partner's** side, as learned from its LACPDUs:

```
SW1# show lacp neighbor
Flags:  S - Device is requesting Slow LACPDUs
        F - Device is requesting Fast LACPDUs
        A - Device is in Active mode       P - Device is in Passive mode

Channel group 1 neighbors

                  LACP port                        Admin  Oper   Port    Port
Port      Flags   Priority  Dev ID          Age    key    Key    Number  State
Gi1/0/1   SA      32768     7c21.0e5a.2c00  12s    0x0    0x1    0x102   0x3D
Gi1/0/2   SA      32768     7c21.0e5a.2c00  15s    0x0    0x1    0x103   0x3D
```

- **Dev ID** is the partner's LACP system ID (its MAC). If two members show **different** Dev IDs, they are cabled to two different switches, and they can't bundle (unless the far side is a StackWise Virtual pair or a stack, which presents one system ID: Chapter 2).
- **Flags** show the partner's mode: `A` active, `P` passive. A member with **no neighbor entry at all** is the `(I)` stand-alone case: no LACPDUs are arriving.
- **Oper Key** must be the same on every member the partner bundles together.

```
SW1# show etherchannel 1 port-channel
                Port-channels in the group:
                ---------------------------
Port-channel: Po1    (Primary Aggregator)
Age of the Port-channel   = 0d:02h:11m:40s
Logical slot/port   = 9/1          Number of ports = 2
Port state          = Port-channel Ag-Inuse
Protocol            =   LACP
Port security       = Disabled

Ports in the Port-channel:
Index   Load   Port       EC state        No of bits
------+------+----------+-----------------+-----------
  0     55     Gi1/0/1    Active             4
  1     AA     Gi1/0/2    Active             4

Time since last port bundled:    0d:02h:11m:32s    Gi1/0/2
```

The **Load** column is the hash-bucket bitmap each member owns (CCNA guide 10.4): two members split the 8 buckets 4 and 4. "Time since last port bundled/unbundled" is a quick way to spot a member that keeps flapping in and out of the bundle.

3.8 LACP Tuning Worth Knowing

```
SW1(config)# lacp system-priority 100                ! lower = this switch decides
SW1(config)# interface range Gi1/0/1 - 4
SW1(config-if-range)# channel-group 1 mode active
SW1(config-if-range)# lacp port-priority 100          ! lower = preferred active member
SW1(config-if-range)# lacp rate fast                  ! LACPDUs every 1 s (default 30 s)
SW1(config)# interface Port-channel1
SW1(config-if)# port-channel min-links 2              ! take Po1 down below 2 members
SW1(config)# port-channel load-balance src-dst-ip     ! global hash input
```

Why do the system priority and port priority exist? LACP supports up to **16** members per channel, but only **8** can be active; the rest are **hot-standby**. The two switches must agree on which 8, so the switch with the better (lower) **system priority** — or lower MAC on a tie — gets to decide, and it uses **port priority** to choose. One side has to be in charge of the decision, or each side could pick a different eight.

Worked example: choosing 8 of 10. SW1 and SW2 are joined by 10 links, all in LACP channel-group 1.

1. Compare **system priority**: SW1 is set to 100, SW2 is at the default 32768. Lower wins, so **SW1 decides** which members are active.
2. SW1 looks at its own **port priorities**: Gi1/0/1–8 are set to 100, Gi1/0/9–10 are at the default 32768. Lower is preferred, so Gi1/0/1–8 become active.
3. Gi1/0/9 and Gi1/0/10 become **hot-standby**. They show `(H)` in `show etherchannel summary` and carry no traffic.
4. Gi1/0/4 fails. The bundle drops to 7 active members, and SW1 promotes one standby port (on a port-priority tie, the lower port number wins) to take its place. The bundle is back to 8.

Worked example: which member will a flow use? You can ask the switch directly, without sending traffic:

```
SW1# test etherchannel load-balance interface port-channel 1 ip 10.1.10.5 10.2.20.9
Would select Gi1/0/2 of Po1
```

Every packet from 10.1.10.5 to 10.2.20.9 will use Gi1/0/2 (with `src-dst-ip` hashing). If one user complains of slowness while the bundle looks lightly loaded, check whether their flow shares a member with a large backup flow.

Why would you want `min-links`? Because a port-channel with only one surviving member is still "up," so routing protocols keep sending full traffic to it — and congest it. `min-links` takes the whole bundle down below a threshold, so routing shifts traffic to a better path instead.

Why `lacp rate fast`? With the default slow rate, a unidirectional failure (one fiber strand broken) can take up to 90 seconds (3 × 30 s) to detect through LACP. Fast rate detects it in about 3 seconds.

Layer 3 EtherChannel

Why use one? Between two Layer 3 switches you could also run two separate routed links with ECMP. A Layer 3 EtherChannel gives you one routing adjacency instead of two, one IP subnet instead of two, and member failure that the routing protocol never sees (the port-channel stays up).

A port-channel can also be a **routed** link, which is common between distribution and core switches. The rule is that the members and the port-channel must **all** be Layer 3 (`no switchport`), and the IP address goes on the **port-channel only**:

```
SW1(config)# interface range Gi1/0/49 - 50
SW1(config-if-range)# no switchport                  ! members become routed ports first
SW1(config-if-range)# channel-group 10 mode active  ! creates Port-channel10 as Layer 3
SW1(config)# interface Port-channel10
SW1(config-if)# no switchport                        ! already L3 if created this way
SW1(config-if)# ip address 10.0.0.1 255.255.255.252 ! the IP lives here, never on members
```

In `show etherchannel summary` a healthy Layer 3 bundle shows **`Po10(RU)`**. **Why must you add `no switchport` to the members before `channel-group`?** Because the port-channel interface is created with the same Layer 2/Layer 3 personality as the first member added. Add Layer 2 members and you get a Layer 2 port-channel. Then a later `no switchport` on the port-channel conflicts with its members and they suspend. Decide the layer first, then bundle.

3.9 Troubleshooting Summary — Symptom → Cause → Proof

Symptom	Likely cause	Command that proves it
Link up, but only some VLANs pass	Trunk/access mismatch, or VLAN not allowed / not created	`show interfaces trunk`, `show interfaces Gi1/0/1 switchport`
`switchport mode trunk` rejected ("encapsulation is Auto")	ISL-capable platform with encapsulation still `negotiate`	`show interfaces Gi1/0/1 switchport` (Administrative Trunking Encapsulation); fix with `switchport trunk encapsulation dot1q`
Port configured dynamic but operating as access	Neither side asks to trunk (`auto`+`auto`), or the far side is access	`show interfaces Gi1/0/1 switchport` (Administrative vs Operational Mode)
Dynamic trunk won't form between correct-looking switches	`auto`+`auto`, `nonegotiate` on one side, or VTP domain mismatch	`show dtp interface Gi1/0/1`, `show vtp status`
Frames leaking between VLANs; port blocked for one VLAN	Native VLAN mismatch	CDP log message, `show spanning-tree inconsistentports`
VLANs vanished across many switches	VTP revision-number overwrite (v1/v2)	`show vtp status` (revision, last modified by)
One trunk VLAN missing from the forwarding list	STP blocking or VTP pruning	`show spanning-tree vlan X`, `show interfaces trunk` (last section)
Port-channel down, members `(I)`	No partner protocol (mode mismatch)	`show etherchannel summary`, `show lacp neighbor`
One member `(s)`	Member config differs from bundle	`show interfaces Gi1/0/3 etherchannel`, compare `show run`
Members err-disabled after enabling `on`	Misconfig guard: far side not bundled	`show interfaces status err-disabled`, log `channel-misconfig`
Bundle up but one flow can't exceed one link's speed	Normal per-flow hashing	`show etherchannel load-balance`; `test etherchannel load-balance`

Two commands answer most Layer 2 link questions: `show interfaces trunk` for VLANs, `show etherchannel summary` for bundles. Learn to read every line of both.

Chapter 4: Spanning Tree at CCNP Depth: RSTP, MST & Enhancements

> **Blueprint:** ENCOR 3.1.c — Configure and verify common Spanning Tree Protocols (RSTP, MST) and Spanning Tree enhancements such as root guard and BPDU guard.

The CCNA guide covered the election (9.3), the flavors (9.8), the guards (9.9) and the mechanics underneath (9.10–9.11). This chapter assumes all of that. It adds **MST**, the RSTP behaviors that matter for troubleshooting, and the full toolkit of enhancements — including where each one belongs in a real campus.

4.1 Root Placement Is a Design Decision

The problem this solves: the root bridge is the center of the spanning tree. Every switch's forwarding path points toward it, and the links furthest from it are the ones that get blocked. If you leave the election to chance, the "winner" is the switch with the lowest MAC address, which is often the oldest, slowest switch in a closet. Then all traffic squeezes through it.

STP will always elect a root. The only question is whether **you** chose it.

Worked example: who wins the election? The bridge ID is **priority + VLAN number** (the extended system ID), then the MAC address as a tiebreak. For VLAN 10:

Switch	Configured priority	Bridge priority for VLAN 10	MAC
DIST-1	24576 (`root primary`)	24576 + 10 = **24586**	7c21.0e5a.1b00
DIST-2	28672 (`root secondary`)	28672 + 10 = 28682	7c21.0e5a.2c00
ACC-old	32768 (default)	32768 + 10 = 32778	0011.2233.4455 (lowest)

Lowest bridge ID wins, and priority is compared before MAC. DIST-1 becomes root. If DIST-1 fails, DIST-2 (28682) takes over. ACC-old has the lowest MAC, and with all priorities left at default **it** would have been root. In a campus, the root for each VLAN belongs at the **distribution layer**, and it must line up with the FHRP:

VLAN	STP root	STP secondary	HSRP/VRRP active
10, 30, 50 (odd)	DIST-1	DIST-2	**DIST-1**
20, 40, 60 (even)	DIST-2	DIST-1	**DIST-2**

```
DIST-1(config)# spanning-tree vlan 10,30,50 root primary     ! priority 24576 (or lower)
DIST-1(config)# spanning-tree vlan 20,40,60 root secondary   ! priority 28672
```

Why must the STP root and the FHRP active router be the same switch? Because the access switch's root port points **toward the root**. If DIST-1 is the STP root but DIST-2 is the active gateway, a host's traffic goes up the forwarding uplink to DIST-1, then crosses the inter-distribution link to reach the gateway on DIST-2 — an extra hop, and that link becomes a bottleneck. Aligning them makes the forwarding path and the gateway the same path. STP decides which way the road goes; the FHRP decides where the exit is. Put the exit at the end of the road.

> **Exam trap:** `root primary` is a **macro**, not a setting. It runs once, looks at the current root's priority, and writes `spanning-tree vlan X priority 24576` into the config (or a value 4096 lower than the current root if that's already 24576 or less). If a switch with priority 0 joins later, `root primary` won't react, because it's already written a fixed number. For a deterministic design, set explicit priorities.
>
> Example: the current root has the default 32768. `root primary` writes **24576**. If instead the current root were already at 24576, it would write 24576 − 4096 = **20480**, to make sure it wins.

Path Cost: Short vs Long

Link speed	Short (802.1D-1998)	Long (802.1t)
10 Mbps	100	2,000,000
100 Mbps	19	200,000
1 Gbps	4	20,000
10 Gbps	2	2,000
100 Gbps	no distinct value — can't be told apart	200

Why does the long method exist? Same problem as OSPF's reference bandwidth (CCNA guide 16.6): the short values bottom out at 10 Gbps, so 10, 40 and 100 Gbps links would look nearly equal. Long costs keep scaling. **MST always uses long costs.**

Worked example: why the method changes the answer. ACC1 can reach the root DIST-1 two ways: a direct 1 Gbps link, or two 10 Gbps hops through ACC2. Root path cost is the sum of the costs of every port the path **enters** on its way down from the root.

Path	Short method	Long method
Direct 1 Gbps	4	20,000
Via ACC2: 10 Gbps + 10 Gbps	2 + 2 = 4	2,000 + 2,000 = 4,000

With short costs it's a **tie** (4 vs 4), so the next tiebreaker decides: lowest **sender bridge ID**. The direct link's BPDUs come from DIST-1, the root itself, which has the lowest bridge ID of all. So ACC1 picks the **slow 1 Gbps link**. With long costs the two 10 Gbps hops clearly win (4,000 vs 20,000). Set the method identically everywhere with `spanning-tree pathcost method long` and confirm it in `show spanning-tree summary`.

STP Topology Tuning — Cost and Port Priority

Root placement decides **where** the tree is anchored. Two more knobs decide **which links** a non-root switch uses to reach it. Both act on the root-port tiebreakers (CCNA guide 9.10): lowest **root path cost**, then lowest **sender bridge ID**, then lowest **sender port priority**, then lowest **sender port number**.

Knob	Command (PVST+ / Rapid PVST+)	MST equivalent	Configured on	Influences
Port cost	`spanning-tree [vlan X] cost N`	`spanning-tree mst N cost N`	The switch **choosing** a root port (the downstream port)	That switch's root path cost via that port, and everything below it
Port priority	`spanning-tree [vlan X] port-priority N`	`spanning-tree mst N port-priority N`	The **upstream** switch, on the port the downstream switch connects to	The neighbor's tiebreak when it has two links to the **same** upstream switch

Port priority is 0–240 in steps of 16 (default **128**). Cost is any value in the switch's cost method.

Why is cost set on the downstream switch but port priority on the upstream one? Because of where each value enters the calculation. Root path cost is added **on the receiving port**: a switch adds its own port's cost to the cost advertised in the BPDU it hears. So lowering a port's cost makes **that** switch prefer that port. Port priority, by contrast, is compared as the **sender's** port ID, the value carried in the BPDU from the upstream switch. So you set it where the BPDU comes from. Cost: "how expensive is my door?" Priority: "which of your doors do I prefer?"

Worked example. ACC1 has two 1 Gbps uplinks, Gi1/0/1 and Gi1/0/2, both to DIST-1 (the root) on DIST-1's Gi1/0/10 and Gi1/0/11. Both paths cost 20,000 (long method), and both BPDUs come from the same bridge ID. The tiebreak falls to the sender port ID: 128.10 beats 128.11, so ACC1's root port is **Gi1/0/1** (the link to Gi1/0/10). You can make Gi1/0/2 the root port either way:

```
! Option 1 — on DIST-1 (upstream): make Gi1/0/11 the preferred sender port
DIST-1(config)# interface Gi1/0/11
DIST-1(config-if)# spanning-tree vlan 10 port-priority 64      ! 64.11 beats 128.10

! Option 2 — on ACC1 (downstream): make Gi1/0/2 the cheaper path
ACC1(config)# interface Gi1/0/2
ACC1(config-if)# spanning-tree vlan 10 cost 10000              ! 10,000 beats 20,000
```

Option 2 is the more common real-world fix because it acts earlier in the decision order (cost before port ID) and works even when the two uplinks go to **different** upstream switches. Port priority only ever matters between parallel links to the **same** neighbor. Verify with `show spanning-tree vlan 10`: the Role column moves `Root` to Gi1/0/2 and `Altn` to Gi1/0/1.

> **Exam trap:** setting `port-priority` on ACC1's own uplinks does **nothing** to ACC1's root-port choice. ACC1 compares the priorities DIST-1 advertises, not its own.

4.2 RSTP Behaviors You Need for Troubleshooting

The problem this solves: classic 802.1D STP waits out timers before it lets a port forward: 15 s listening plus 15 s learning, and up to 20 s more to notice a failure. That's 30–50 seconds of outage every time the topology changes. RSTP (Rapid STP) gets the same loop-free result in well under a second.

Story Time : Classic STP is a traffic light on a fixed timer. Even at 3 a.m. with an empty road, you sit at red for the full 30 seconds, because the light can't see whether anyone is coming. RSTP is two drivers at a quiet junction making eye contact: "you go," "thanks." They agree directly and move off at once. The waiting was never needed for safety. It was there because 802.1D had no way to ask.

RSTP (802.1w) replaced timer-based convergence with **handshakes** (CCNA guide 9.11). On Cisco switches you run it per VLAN as **Rapid PVST+**, enabled with one global command:

```
SW(config)# spanning-tree mode rapid-pvst     ! RSTP per VLAN (alternatives: pvst, mst)
SW# show spanning-tree summary | include mode
Switch is in rapid-pvst mode
```

Recap of the vocabulary:

Port role	Meaning
Root	Best path toward the root (one per non-root switch)
Designated	The one port on each segment that forwards onto it, away from the root (every root-bridge port is designated)
Alternate	A backup path to the root via a **different** switch — discarding, ready to take over instantly
Backup	A backup for a designated port on the **same** shared segment (rare: hubs)

Port state	Learns MACs?	Forwards?
Discarding (replaces disabled, blocking, listening)	No	No
Learning	Yes	No
Forwarding	Yes	Yes

The Proposal/Agreement Handshake, Step by Step

A new uplink is cabled between DIST-1 (root for VLAN 10) Gi1/0/12 and ACC2 Gi1/0/1. ACC2 has host ports Gi1/0/2–48 (PortFast) and a downlink Gi1/0/50 to another switch.

1. The link comes up. Both ends start as **designated, discarding**, and each sends BPDUs with the **proposal** flag set: "I'd like to forward on this link."
2. ACC2 sees DIST-1's BPDU is superior (it comes from the root). ACC2 makes Gi1/0/1 its **root port**.

3. Before agreeing, ACC2 must make sure the new path can't create a loop through it. It runs **sync**: every non-edge designated port (Gi1/0/50) is put into **discarding**. Edge ports (the hosts) are left alone, because they can't create a loop.
4. ACC2 sends an **agreement** back on Gi1/0/1.
5. DIST-1 receives the agreement and moves Gi1/0/12 straight to **forwarding**. No timers.
6. ACC2 now repeats the same proposal/agreement on Gi1/0/50 with its downstream neighbor. The handshake ripples down the tree one link at a time.

The whole sequence typically finishes in milliseconds per hop. It needs two things: a **point-to-point** link (next section) and edge ports correctly marked as edge.

And when a root port fails? ACC1 has root port Gi1/0/1 and alternate Gi1/0/2. When Gi1/0/1 goes down, ACC1 makes Gi1/0/2 its root port and moves it **directly to forwarding**: an alternate port is, by definition, a loop-free backup path to the root. Then it sends a topology change so other switches flush their old MAC entries (below).

Link Types — Why Duplex Matters to STP

RSTP's fast proposal/agreement handshake only works on **point-to-point** links, which RSTP infers from **full duplex**. A half-duplex port is treated as **shared** and falls back to slow timer-based convergence.

```
SW1# show spanning-tree vlan 10
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- ----------------------
Gi1/0/1             Root FWD 4         128.1    P2p
Gi1/0/2             Altn BLK 4         128.2    P2p
Gi1/0/10            Desg FWD 4         128.10   P2p Edge
Gi1/0/24            Desg FWD 19        128.24   Shr
```

Reading it: Gi1/0/2 is an **alternate** — it will become root port immediately if Gi1/0/1 fails. Gi1/0/10 is an **edge** port (PortFast). Gi1/0/24 shows `Shr`: it negotiated half duplex, so if it ever has to move to forwarding it'll wait out the timers. Fix the duplex — or, if a switch link really is full duplex but misdetected, force `spanning-tree link-type point-to-point`.

Topology Changes — The Part That Floods Your Network

In RSTP, a **topology change (TC)** happens only when a **non-edge** port moves **to forwarding**. (Ports going down don't trigger TC in RSTP; edge ports never do.) The switch that sees the change sets the TC flag in its BPDUs and **flushes MAC addresses** learned on all its non-edge ports, and every switch receiving the TC does the same and passes it on.

Why does that matter? Because after a MAC flush, switches don't know where anyone is, so they **flood** unicast traffic until they relearn. One flapping switch port without PortFast on an access switch can cause repeated TCs, repeated flushes, and network-wide flooding that looks like a performance problem. This is the strongest reason to put PortFast on every host port: edge ports don't generate topology changes.

```
SW1# show spanning-tree vlan 10 detail | include occur|from
  Number of topology changes 4127 last change occurred 00:00:09 ago
          from GigabitEthernet1/0/33
```

A rapidly climbing TC count with a recent `from` interface leads you straight to the flapping port. Follow it hop by hop until you find the switch where the `from` port is an access port.

Interop With Legacy STP

RSTP is backward compatible **per port**: if a port hears an 802.1D BPDU, that port falls back to 802.1D behavior and timers. It's correct, just slow, so a single old switch can make part of your network converge in 30–50 seconds again.

4.3 MST — Multiple Spanning Tree (802.1s)

Rapid PVST+ runs **one RSTP instance per VLAN**. That's great for load balancing (CCNA guide 9.8), but it scales badly: 500 VLANs means 500 instances, 500 sets of BPDUs on every trunk every 2 seconds, and 500 elections for the CPU to run. Yet in a typical campus there are only **two or three distinct topologies** — "VLANs that should use the left uplink" and "VLANs that should use the right."

Story Time : Imagine a bus company that prints a **separate timetable for every passenger**, even though everyone is going one of two ways: north or south. That's PVST+ with 500 VLANs. MST prints **two timetables**, one per route, and simply tells each passenger which route they're on. Same journeys, a fraction of the paperwork. An MST **region** is a group of towns that agree on what "route 1" and "route 2" mean. A bus crossing into a town that numbers its routes differently can't keep its number.

MST runs **a few instances** and **maps many VLANs to each one**:

MST instance	VLANs mapped	Root	Effect
0 (IST)	Everything not mapped elsewhere	DIST-1	Default instance; carries the BPDUs
1	10, 30, 50, … (odd)	DIST-1	Odd VLANs use the left uplink
2	20, 40, 60, … (even)	DIST-2	Even VLANs use the right uplink

Same load balancing as PVST+, with **three** instances instead of hundreds.

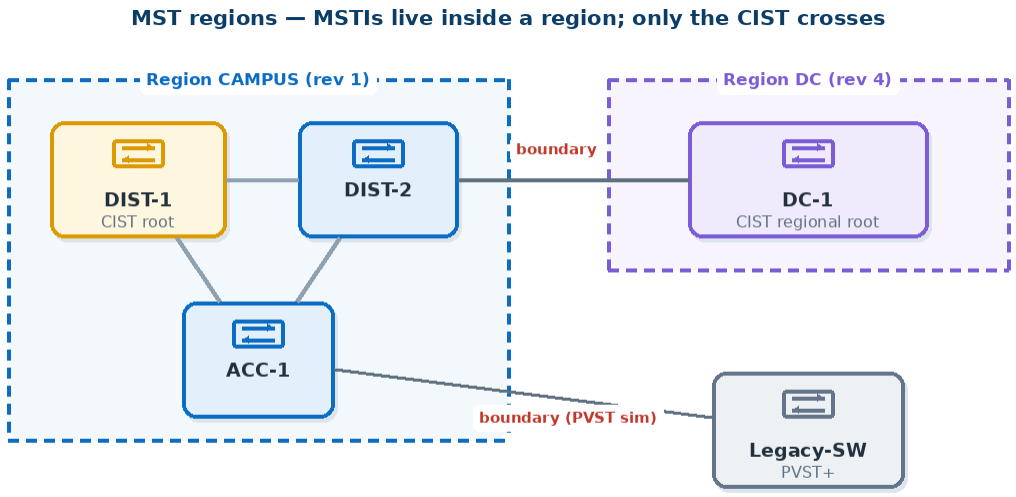
Why does MST beat PVST+ at scale, and what's the catch? The win is CPU and BPDU overhead: MST sends **one BPDU per port** that carries the information for every instance (each instance is an "M-record" inside it). The catch is coordination: every switch must agree on exactly which VLANs belong to which instance, and that agreement is what an MST **region** is.

MST Regions

Switches are in the **same region** only if **all three** of these match exactly:

1. **Region name** (case-sensitive)
2. **Revision number**
3. **VLAN-to-instance mapping table** (all 4094 VLANs)

Any difference makes a different region — even one VLAN mapped differently.



Inside a region: MSTI 1, 2... load-balance. Between regions: one CIST, each region = one big switch

Term	Meaning
IST (instance 0)	The internal spanning tree inside a region; the only instance that sends BPDUs
MSTI (1, 2, ...)	User-created instances; information carried as M-records inside IST BPDUs
CIST	Common and Internal Spanning Tree — the single tree that spans **all** regions and any non-MST switches
CIST root	The root of that whole tree (best bridge ID anywhere)
CIST regional root	The switch in each region closest to the CIST root — the region's "entry point"
Boundary port	A port connecting to a **different region** or to a non-MST switch (802.1D, RSTP or PVST+)

Why can't MSTIs cross region boundaries? Because another region might map VLAN 10 to a completely different instance — or not have that instance at all. Instance 1 means nothing outside the region that defined it. So at the boundary, the whole region looks to the outside like **one big virtual switch** participating in the CIST. Inside the region, MSTIs can load balance; across regions, only the CIST exists. A region is a promise: "we all agree what instance 1 means." Outside the promise, only the common tree applies.

Configuring MST

```
SW(config)# spanning-tree mode mst
SW(config)# spanning-tree mst configuration
SW(config-mst)# name CAMPUS                  ! must match across the region
SW(config-mst)# revision 1                   ! must match
SW(config-mst)# instance 1 vlan 10,30,50     ! mapping must match
SW(config-mst)# instance 2 vlan 20,40,60
SW(config-mst)# show pending                 ! review before it applies
SW(config-mst)# exit                         ! changes apply on exit
SW(config)# spanning-tree mst 0-1 priority 24576   ! on DIST-1: root for IST and MSTI 1
SW(config)# spanning-tree mst 2 priority 28672     ! on DIST-1: secondary for MSTI 2
```

```
SW# show spanning-tree mst configuration
Name      [CAMPUS]
Revision  1     Instances configured 3
Instance  Vlans mapped
--------  ---------------------------------------------------------------
0         1-9,11-19,21-29,31-39,41-49,51-59,61-4094
1         10,30,50
2         20,40,60

SW# show spanning-tree mst 1
##### MST1    vlans mapped:   10,30,50
Bridge        address 7c21.0e5a.1b00  priority      24577 (24576 sysid 1)
Root          this switch for MST1
Interface        Role Sts Cost      Prio.Nbr Type
---------------- ---- --- --------- -------- --------------------------------
Gi1/0/1          Desg FWD 20000     128.1    P2p
Gi1/0/2          Desg FWD 20000     128.2    P2p Bound(PVST)
```

Notice two things. The priority shows `24577` because the extended system ID now carries the **instance number** (1), not a VLAN. And `Bound(PVST)` marks a **boundary port** facing a PVST+ switch.

> **Exam traps:** (1) Changes made in `spanning-tree mst configuration` mode only take effect when you **exit**, and changing the mapping on a live switch briefly puts it in a different region — schedule it. (2) **Map VLANs before you create them**, and keep the mapping identical everywhere — a single mismatched VLAN splits the region and silently kills your MSTI load balancing at that point. (3) Since VTPv3 can propagate the MST configuration (3.4), that's one way to keep it consistent.

What a Region Mismatch Looks Like

Suppose DIST-1 maps VLANs 20, 40 **and 60** to instance 2, but ACC3 was configured with only `instance 2 vlan 20,40`. The name and revision match, but the mapping tables differ, so the two switches compute different configuration digests and treat each other as **different regions**. Nothing errors and nothing is logged. The only evidence is that the link has become a boundary:

```
ACC3# show spanning-tree mst interface Gi1/0/1
GigabitEthernet1/0/1 of MST0 is root forwarding
Edge port: no             (default)        port guard : none        (default)
Link type: point-to-point (auto)           bpdu filter: disable     (default)
Boundary : boundary       (RSTP)           bpdu guard : disable     (default)
Bpdus sent 14, received 2310

Instance Role Sts Cost      Prio.Nbr Vlans mapped
-------- ---- --- --------- -------- ------------------------------
0        Root FWD 20000     128.1    1-19,21-39,41-4094
2        Mstr FWD 20000     128.1    20,40
```

Read it from the top: **`Boundary : boundary (RSTP)`** says the neighbor's BPDUs don't match this region. `show spanning-tree mst` flags the same port as **`Bound(RSTP)`** in its Type column. In the instance table, MSTI 2's role is **`Mstr`** (master): on a boundary port every MSTI simply copies the port's CIST role and state. The consequence is the real damage. ACC3 was supposed to send even VLANs up its other uplink. Now all instances follow the CIST at this link, and **load balancing is silently gone**.

Fix: compare `show spanning-tree mst configuration` on both switches (the `digest` keyword shows the computed MD5 digest, which must match) and make the mapping identical.

MST Hop Count

Inside a region, MST doesn't use the 802.1D message-age timer to age out information. It uses a **hop count**: the regional root sends BPDUs with the hop count set to `max-hops` **(default 20)**, each switch decrements it, and information that reaches 0 is discarded. A region with a diameter larger than 20 switches needs `spanning-tree mst max-hops 30` (range 1–255). Like the other timers, configure it on the regional root.

MST and PVST+ Interoperation

When an MST region meets PVST+ switches, the boundary port runs **PVST simulation**: the MST side sends copies of its IST (CIST) BPDU in every VLAN, and checks the per-VLAN BPDUs it receives against the CIST.

The rule that keeps it working: **the root must be consistent.** Either the CIST root is **inside the MST region** and it's better (lower priority) than every PVST+ VLAN root, or the PVST+ side holds the root for **all** VLANs. If the roots are split — for example, the PVST+ side is root for VLAN 20 but not VLAN 10 — the boundary port goes into a **PVST simulation inconsistent** state and blocks.

Best practice: when migrating to MST, put the CIST root (and all MSTI roots) in the MST core, and migrate from the core outward.

4.4 Edge Ports: PortFast Done Right

The problem this solves: a port facing a PC can't create a loop, but STP doesn't know that. Without PortFast, the switch treats a freshly connected PC like a possible switch and makes it wait. PortFast tells STP "only an end host lives here".

PortFast marks a port as an **edge port**: it goes straight to forwarding and never generates topology changes (4.2).

Worked example: a PC without PortFast. A laptop is plugged into Gi1/0/10 on a Rapid PVST+ switch, and the port is **not** an edge port.

1. The port comes up as designated and sends proposals. A laptop doesn't run STP, so no agreement ever comes back.
2. With no handshake possible, RSTP falls back to timers: **15 s discarding, then 15 s learning** (forward delay × 2).
3. Only at about **30 s** does the port forward. Meanwhile the laptop's DHCP requests go nowhere. Some clients give up and use a 169.254.x.x self-assigned address. The user sees "no network" even though the cable is fine.
4. When the port finally reaches forwarding, it triggers a **topology change**, flushing MAC tables across the VLAN (4.2).

With `spanning-tree portfast`, the same port forwards the moment the link is up, and it never triggers a topology change.

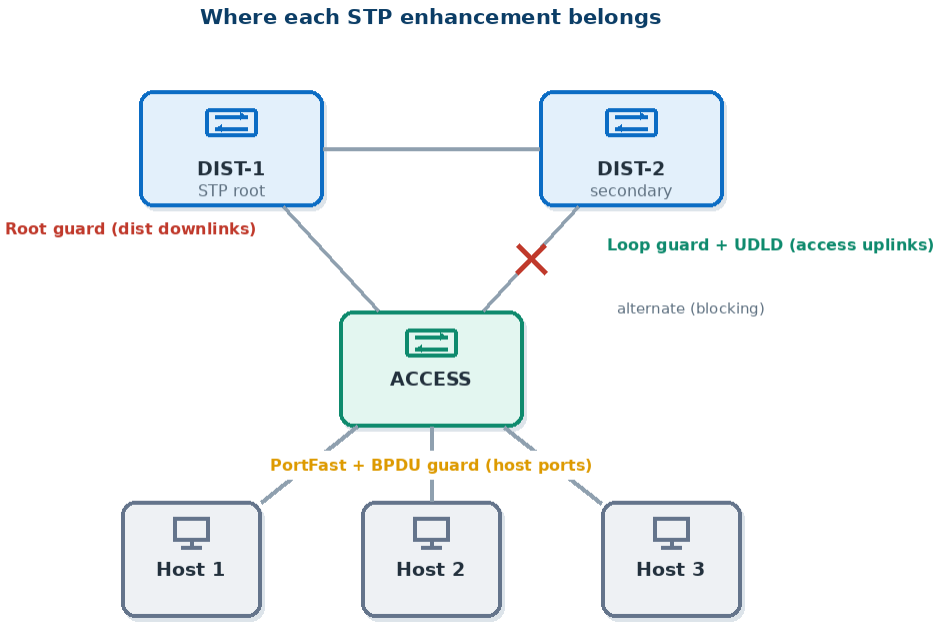
```
SW(config-if)# spanning-tree portfast              ! access port to a host
SW(config-if)# spanning-tree portfast trunk        ! trunk to a hypervisor or AP in FlexConnect
SW(config)# spanning-tree portfast default         ! all ACCESS ports become edge by default
```

(IOS XE 16 and later spell these `spanning-tree portfast edge`, `portfast edge trunk` and `portfast edge default`, and the global guard defaults below become `spanning-tree portfast edge bpduguard default` and `spanning-tree portfast edge bpdufilter default`. Older releases accept the shorter forms. The behavior is identical.)

Why does `portfast trunk` exist when PortFast is supposed to be for hosts? Because some hosts use trunks: a server running a hypervisor with VMs in several VLANs, or a router-on-a-stick. Those endpoints still can't form a switching loop, so the port is safe to treat as an edge port — but plain `spanning-tree portfast` is ignored on a trunk, so you have to say so explicitly.

An edge port that **receives a BPDU** immediately loses its edge status and becomes a normal STP port. That's a safety net, but a weak one: the port has already been forwarding. BPDU guard turns that into a hard stop.

4.5 The Enhancements, One by One



BPDU Guard — "No Switches Here"

```
SW(config-if)# spanning-tree bpduguard enable            ! per port
SW(config)# spanning-tree portfast bpduguard default     ! every PortFast port
SW(config)# errdisable recovery cause bpduguard          ! optional auto-recovery
SW(config)# errdisable recovery interval 300             ! seconds
```

A BPDU arriving on the port **err-disables** it. The port stays down until someone runs `shutdown`/`no shutdown`, or until err-disable recovery brings it back.

Worked example. A user brings a small desk switch from home and plugs it into Gi1/0/10 (PortFast + BPDU guard), then plugs a second cable from that switch into the wall port Gi1/0/11. The desk switch runs STP and sends a BPDU:

```
%SPANTREE-2-BLOCK_BPDUGUARD: Received BPDU on port Gi1/0/10 with BPDU Guard enabled.
Disabling port.
%PM-4-ERR_DISABLE: bpduguard error detected on Gi1/0/10, putting Gi1/0/10 in
err-disable state
```

Gi1/0/10 goes down within a second of the first BPDU, before a loop through the two wall ports can form. `show interfaces status err-disabled` lists Gi1/0/10 with reason `bpduguard`. Remove the desk switch, then `shutdown` / `no shutdown` the port (or, with `errdisable recovery cause bpduguard` configured as above, let it recover after 300 s).

Why prefer the global `portfast bpduguard default` over per-port configuration? Because it ties the two features together automatically: every port you make an edge port is protected, and you can't forget one. Whenever you mark a port as "only hosts here," you also want "...and shut it if that turns out to be false."

BPDU Filter — Two Very Different Behaviors

Configured how	Behavior	Risk
Global: `spanning-tree portfast bpdufilter default`	Applies to **PortFast** ports: sends a few BPDUs at link-up, then stops. If a BPDU is **received**, the port loses PortFast and filtering and runs normal STP	Low — it backs out safely
Per-interface: `spanning-tree bpdufilter enable`	**Never sends, ignores everything received** — STP is effectively off on the port	**High** — a loop through this port will never be blocked

Why use BPDU filter at all? On a port facing a network you deliberately keep out of your STP domain (a service provider handoff, another organization), or to stop sending BPDUs to thousands of host ports. The global form is the safe one. Interface-level BPDU filter turns spanning tree off; never use it toward anything that could be a switch.

> **Exam trap:** if BPDU guard and BPDU filter are both on one port, the filter wins: BPDUs are dropped before guard can see them, so the port never err-disables.

Root Guard — "You May Not Become Root Through This Port"

```
SW(config-if)# spanning-tree guard root
```

Configured on **designated ports** facing switches that should never be on the path to the root — typically distribution downlinks to access switches, or ports to other organizations. If a **superior** BPDU arrives, the port goes **root-inconsistent** (no forwarding). It **recovers automatically** when the superior BPDUs stop.

Why it exists: a **superior BPDU** is one with a better (lower) root bridge ID than the current root. Any switch that sends one will take over as root and re-draw the whole tree around itself. Root guard says: "a new root may never appear on this side."

Worked example. DIST-1 (priority 24576) is root for VLAN 10. Its downlink Gi1/0/5 to ACC5 has `spanning-tree guard root`. A technician connects a lab switch with priority **0** to ACC5.

1. The lab switch sends BPDUs claiming root with bridge priority 0 + 10 = 10. ACC5 passes them on toward DIST-1.

2. DIST-1 receives a superior BPDU on Gi1/0/5. Without root guard, the lab switch would become root for VLAN 10 and the whole campus would re-converge toward a lab bench.
3. With root guard, DIST-1 instead blocks Gi1/0/5 for VLAN 10:

```
%SPANTREE-2-ROOTGUARD_BLOCK: Root guard blocking port GigabitEthernet1/0/5 on VLAN0010.
```

4. `show spanning-tree inconsistentports` lists Gi1/0/5 as **Root Inconsistent**. ACC5's users lose connectivity in VLAN 10, which is a far smaller failure than the whole campus.
5. Once the lab switch is removed and the superior BPDUs stop, the port recovers by itself.

Loop Guard — "Silence Is Not Permission"

```
SW(config-if)# spanning-tree guard loop           ! per port
SW(config)# spanning-tree loopguard default        ! all point-to-point ports
```

Protects **root and alternate ports** (the ports that receive BPDUs). If BPDUs stop arriving on such a port — for example, because of a unidirectional link failure — instead of moving to designated/forwarding, the port goes **loop-inconsistent** (blocking) for that instance. It recovers automatically when BPDUs resume.

Story Time : A lifeguard who stops hearing a swimmer doesn't conclude "the pool must be empty, I can go home." Silence might mean trouble. STP without loop guard reasons the wrong way: "I've stopped hearing BPDUs on this blocked port, so nobody upstream is there, so it must be safe to forward." Loop guard teaches it the lifeguard's rule.

Worked example: the broken fiber strand. ACC1's uplink Gi1/0/2 to DIST-2 is an **alternate** (blocking) port for VLAN 10. The fiber strand carrying DIST-2's transmissions to ACC1 breaks. ACC1's strand toward DIST-2 still works, and the port stays up.

1. ACC1 stops receiving DIST-2's BPDUs on Gi1/0/2. Nothing else looks wrong.
2. **Without loop guard:** after 3 missed hellos (6 s in RSTP), ACC1 ages out the stored BPDU. It now believes it's the only switch on that link, makes Gi1/0/2 **designated**, and starts **forwarding**. DIST-2's end is also designated and forwarding. Frames go ACC1 → DIST-2 → DIST-1 → ACC1: a **loop**.
3. **With loop guard:** at step 2, ACC1 instead puts Gi1/0/2 into **loop-inconsistent** (blocking) for VLAN 10:

```
%SPANTREE-2-LOOPGUARD_BLOCK: Loop guard blocking port GigabitEthernet1/0/2 on VLAN0010.
```

4. No loop forms. When the fiber is repaired and BPDUs arrive again, the port goes back to alternate on its own.

Why can't root guard and loop guard be used on the same port? Because they protect opposite port roles. Root guard belongs on **designated** ports (which send BPDUs, and should never receive superior ones). Loop guard belongs on **root/alternate** ports (which receive BPDUs, and must keep receiving them). IOS refuses to enable both on one interface.

UDLD — Unidirectional Link Detection

A **unidirectional link** is one where traffic flows one way only — classically a fiber pair where one strand is broken or crossed. The port stays up (light is received), so STP can be fooled: the downstream switch stops getting BPDUs and unblocks its port, creating a loop.

UDLD is a Cisco Layer 2 protocol where neighbors **echo each other's identity** back. If a switch stops seeing itself echoed, the link isn't working in both directions.

How it works, step by step: SW1 Gi1/0/49 and SW2 Gi1/0/49 are joined by a fiber pair, with UDLD aggressive on both ends.

1. Every 15 s, SW1 sends a UDLD message: "I'm SW1, port Gi1/0/49, and the neighbors I hear are: SW2 Gi1/0/49." SW2 sends the same about itself.
2. Each switch checks that the other side lists **it** as a heard neighbor. That proves traffic flows both ways.
3. SW1's transmit strand breaks. SW2 stops hearing SW1 entirely. SW1 still hears SW2, but SW2's messages no longer list SW1.
4. SW1 sees that its own identity is missing from the echo: the link is unidirectional, so SW1 **err-disables** Gi1/0/49. SW2 has stopped hearing SW1 altogether. In aggressive mode it tries to re-establish contact with 8 UDLD messages, one per second, and when none is answered it err-disables its end too.
5. With the port down, STP and routing see an ordinary link failure and converge around it. No loop and no traffic black hole.

Mode	When the link is detected unidirectional
Normal	Logs the problem and marks the port; the port keeps running (STP decides what happens)
Aggressive	After the neighbor stops responding, tries to re-establish 8 times (once per second), then **err-disables** the port

```
SW(config)# udld enable                  ! normal mode on all FIBER ports
SW(config)# udld aggressive              ! aggressive mode on all fiber ports
SW(config-if)# udld port aggressive      ! per interface (copper or fiber)
SW# show udld neighbors
SW# show udld Gi1/0/49
```

UDLD must run on **both ends** of the link. The default message interval is 15 seconds.

Loop Guard vs UDLD

Question	Loop guard	UDLD
What it detects	BPDUs stopped arriving (any cause)	Link not bidirectional at Layer 1/2
Scope	Per STP instance/VLAN	Whole port
Works on EtherChannel members	Acts on the bundle	Acts on each physical member
Action	Loop-inconsistent (auto-recovers)	Logs (normal) or err-disable (aggressive)

Why run both? Because they catch different failures. UDLD catches a broken fiber strand even on a link STP isn't blocking, and can disable just one member of an EtherChannel. Loop guard catches BPDU loss from causes UDLD can't see — a software bug, a congested CPU dropping BPDUs. Belt and braces for the one failure STP can't detect on its own.

4.6 The Complete Guard Map

The problem this solves: each guard belongs on a specific kind of port, and the exam loves putting the right feature in the wrong place. This table is the one-page answer: which port, what triggers it, what happens, how it recovers.

Feature	Port type	Triggers on	Port result	Recovery
PortFast	Access/host	—	Instant forwarding, no TCs	—
BPDU guard	Access/host (edge)	**Any** BPDU received	**err-disabled**	Manual or errdisable recovery
BPDU filter (global)	Edge	BPDU received → loses edge status	Normal STP	Automatic
BPDU filter (interface)	Special cases only	—	STP off on the port	—
Root guard	Designated, toward access/outside	**Superior** BPDU	**root-inconsistent**	Automatic
Loop guard	Root / alternate	BPDUs **stop**	**loop-inconsistent**	Automatic
UDLD aggressive	Fiber uplinks (both ends)	Link not bidirectional	**err-disabled**	Manual or errdisable recovery

Verification Cheat Sheet

```
SW# show spanning-tree summary              ! mode, portfast/bpduguard/loopguard defaults
SW# show spanning-tree inconsistentports    ! root-, loop-, PVID- and type-inconsistent ports
SW# show spanning-tree interface Gi1/0/10 detail   ! edge status, guard settings, BPDU counts
SW# show interfaces status err-disabled     ! which ports and why (bpduguard, udld...)
SW# show errdisable recovery                ! which causes auto-recover and the timer
```

The pattern behind every enhancement: STP trusts BPDUs completely. Each guard adds one rule about when a BPDU — or its absence — should be distrusted.

Chapter 5: Routing Concepts — EIGRP vs OSPF, Path Selection & PBR

> **Blueprint:** ENCOR 3.2.a — Compare routing concepts of EIGRP and OSPF (advanced distance vector vs. link state, load balancing, path selection, path operations, metrics, and area types); ENCOR 3.2.d — Describe policy-based routing.

At CCNA you learned that routers learn routes and pick the best one. At CCNP you need to know **how** each protocol thinks, because that's what lets you predict its behaviour and find a fault without guessing. This chapter compares the two interior protocols ENCOR expects you to know — **EIGRP** and **OSPF** — then covers the decision every router makes for every packet, and the one tool that lets you override that decision: **policy-based routing**.

Chapter 26 takes EIGRP to troubleshooting depth, and Chapter 6 (with Chapter 27 behind it) does the same for OSPF. This chapter is the comparison that makes both of them make sense.

5.1 Three Ways to Build a Routing Protocol

The problem every routing protocol solves: a router only knows the networks plugged into it. Everything else it has to learn from other routers, and it has to learn it in a way that never sends packets round in circles (a **routing loop**). The three families below are three different answers to "how do I learn about far-away networks without creating loops?"

The CCNA guide (16.2) split routing protocols into two families. There are really **three**, and EIGRP is the one that doesn't fit the CCNA picture:

Family	Example	What a router knows	What it shares	Who computes the path
Distance vector	RIP	Only what neighbours tell it	Its whole routing table, periodically	Neighbours, effectively — you trust their answer
Advanced distance vector	**EIGRP**	Neighbours' metrics plus enough to prove a path is loop-free	Only changes, only when they happen	Each router, using the DUAL algorithm
Link state	**OSPF**, IS-IS	A full map of its area	Facts about its own links (LSAs)	Each router, running SPF over the map

Story Time : You're driving to the stadium in a city you don't know.

- **Distance vector (RIP)** is asking a stranger at each junction. They point and say "the stadium's 5 km that way", and you believe them. If they're wrong, or their information is old, you could end up driving in a circle.
- **Advanced distance vector (EIGRP)** is still asking strangers, but you add a sanity check. You're 3 km from the stadium. If a stranger says they are 2 km away, they can't be routing you back past where you're standing, so you trust them. If someone says they're 4 km away, you don't take their advice unless you have no other choice and have double-checked.
- **Link state (OSPF)** is everyone in town holding the same printed map. Nobody needs to trust anybody's directions. Each driver plans their own route on the map. The catch is that every driver has to carry the map, and when a road closes, everyone has to get a new copy and replan.

The analogy breaks in one place: real routers don't "drive". The question is always which **next router** to hand the packet to, not the whole trip.

Why is EIGRP called "advanced" distance vector rather than link state? Because it still never sees the map. An EIGRP router knows, for each destination, "my neighbour R2 says it's 1000 away, and the link to R2 costs me 2000" — that's a **vector** (direction) and a **distance**, exactly like RIP. What makes it advanced is what it does with that information. RIP blindly believes the lowest number and needs hop-count limits, hold-down timers and split horizon to stop the loops that blind belief creates. EIGRP instead keeps **every** neighbour's reported distance and uses a mathematical test — the **feasibility condition** (5.3) — to prove which alternatives cannot be loops. It keeps distance vector's light footprint while dealing with the loops that make RIP so fragile.

And what does link state buy that EIGRP doesn't have? A complete view. OSPF routers in an area hold an identical database (CCNA guide 16.11), so every router can compute any path itself. The price is memory and CPU — every router stores the whole area and reruns SPF when it changes — which is exactly why OSPF needs **areas** to scale, and EIGRP doesn't.

Property	Distance vector (RIP)	Advanced DV (EIGRP)	Link state (OSPF)
Updates	Full table every 30 s	Partial, bounded, on change	LSAs on change + refresh every 30 min
Loop prevention	Split horizon, poison, hold-down, max 15 hops	Feasibility condition (DUAL)	Identical LSDB + SPF
Convergence	Slow (minutes)	Very fast with a feasible successor	Fast (SPF run)
Scaling tool	None	Summarization anywhere, stub routers	Areas, summarization at ABR/ASBR
Memory/CPU	Low	Low–moderate	Moderate–high

5.2 EIGRP Fundamentals

The problem: before EIGRP can choose any path, it has to find its neighbours and keep each one accurately informed, while sending as little as possible. This section covers how EIGRP talks; 5.3 covers how it decides.

EIGRP (Enhanced Interior Gateway Routing Protocol) was Cisco-proprietary until Cisco published it as informational **RFC 7868** in 2016. In practice it remains a Cisco-network protocol.

How EIGRP Talks

Item	Value
Transport	Its own IP protocol, **number 88** (not TCP or UDP)
Reliability	**RTP** — Reliable Transport Protocol — acknowledges Updates, Queries and Replies
Multicast address	**224.0.0.10** (IPv6: **FF02::A**)
Hello / hold timers	**5 s / 15 s** on most links; **60 s / 180 s** on slow NBMA (nonbroadcast multiaccess, e.g. legacy Frame Relay) links at T1 speed or below
Administrative distance	**90** internal, **170** external (redistributed), **5** summary
Packet types	Hello, Update, Query, Reply, Ack (plus SIA-Query/SIA-Reply)

Why does EIGRP need its own reliable transport when it isn't using TCP? Because EIGRP sends updates **only once**, only when something changes. RIP could afford to lose an update — another full copy arrives in 30 seconds. EIGRP never re-sends unless asked, so a lost update would leave a neighbour wrong forever. RTP adds acknowledgements and sequence numbers to the packets that matter (Update, Query, Reply) while letting Hellos go unacknowledged. Partial, triggered updates are only safe if delivery is guaranteed — RTP is that guarantee.

How Two EIGRP Routers Become Neighbours

R1 (10.0.12.1) and R2 (10.0.12.2) share the link 10.0.12.0/24 and both run `router eigrp 100`:

1. Both routers send a **Hello** to 224.0.0.10 every 5 seconds. Each Hello carries the AS number, the K-values and the sender's **hold time** (15 s).
2. R1 receives R2's Hello and checks it: same AS (100)? Same K-values? Is 10.0.12.2 in R1's own subnet on that interface? Does authentication pass? All yes, so R1 adds R2 to its **neighbor table**.
3. The routers then swap their full topology in **Update** packets. These are reliable: each one carries a sequence number and must be acknowledged before the next is sent.
4. From now on, only **changes** are sent. Hellos keep flowing every 5 s just to say "I'm still here".
5. If R1 hears nothing from R2 for 15 seconds (R2's advertised hold time), it declares R2 dead and removes every route learned through it.

Worked example: the timer trap. An engineer wants fewer hellos on a link and sets `ip hello-interval eigrp 100 20` on R2 only. EIGRP does **not** adjust the hold time automatically, so R2 still advertises a hold time of 15 s while now sending hellos every 20 s. R1 waits 15 s, hears nothing and drops the neighbour. A Hello then arrives and the adjacency comes back, so it flaps over and over. The log shows `%DUAL-5-NBRCHANGE: ... is down: holding time expired`. The fix is to raise the hold time too: `ip hold-time eigrp 100 60`. The rule: hello and hold don't have to match between neighbours, but on each router the hold time must be comfortably longer than that router's own hello interval (3× is the convention).

The Three Tables

Table	Command	What's in it
Neighbor table	`show ip eigrp neighbors`	Every adjacent EIGRP router, its hold time and RTP counters
Topology table	`show ip eigrp topology`	Every destination, with **every** neighbour's reported distance — the raw material for DUAL
Routing table	`show ip route eigrp`	Only the winners (successors), marked `D` (or `D EX` for external)

What Must Match to Become Neighbors

- Same **autonomous system number** (the `router eigrp 100` number — unlike OSPF's process ID, this one does have to match).
- Same **K-values** (5.4).
- Same **subnet** on the connecting interface (primary addresses).
- Matching **authentication**, if configured.

Hello and hold timers **don't** have to match — each router simply tells its neighbour how long to wait. That's a classic difference from OSPF, where mismatched hello/dead timers block the adjacency.

5.3 DUAL — How EIGRP Picks a Path and a Backup

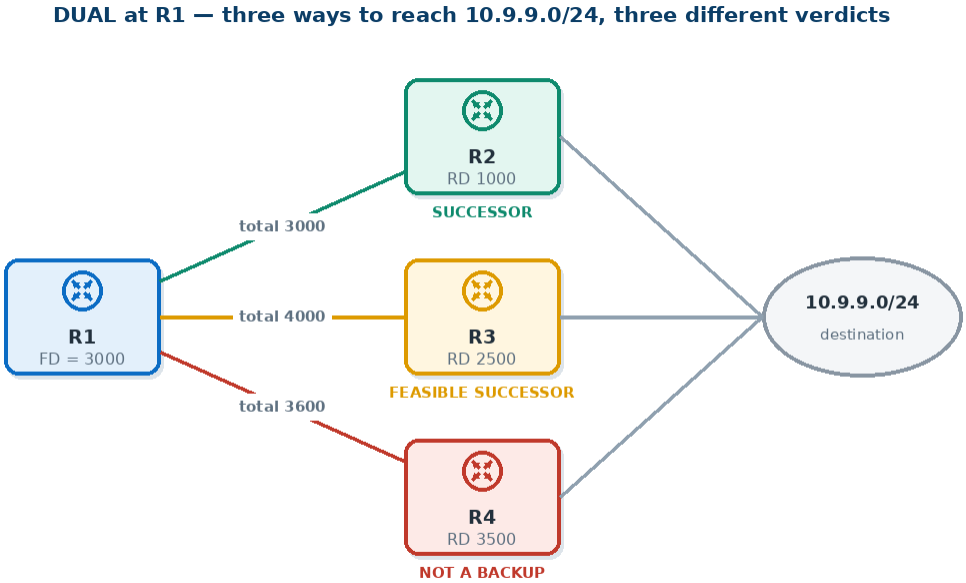
The problem: when the best path fails, a router wants to switch to a backup immediately, but a badly chosen backup might loop the traffic straight back. DUAL solves both halves: it pre-approves backups that are mathematically guaranteed not to loop, and it has a safe procedure for when there is no approved backup.

DUAL (the **Diffusing Update Algorithm**) is the heart of EIGRP. It needs four terms, and the exam uses all of them:

Term	Meaning
Reported distance (RD) — also called advertised distance (AD)	The neighbour's own metric to the destination, as it told you

Term	Meaning
Feasible distance (FD)	Your best total metric to the destination (RD + the cost of the link to that neighbour), for the best path
Successor	The neighbour on the path with the lowest total metric — installed in the routing table
Feasible successor (FS)	A backup neighbour that passes the **feasibility condition**, ready to take over instantly

The feasibility condition: a neighbour is a feasible successor only if its **RD is lower than the current FD**.



Neighbour	RD (their distance)	Total via them	Verdict
R2	1000	3000	**Successor** — lowest total, so FD = 3000
R3	2500	4000	**Feasible successor** — 2500 < 3000
R4	3500	3600	**Not a backup** — 3500 is not < 3000, even though its total (3600) beats R3's

Why does "RD < FD" prove a path is loop-free? Think about what a loop would look like. If R4's path to 10.9.9.0 ran back through R1, then R4's distance would have to include R1's own distance — at least 3000 — plus R4's cost to reach R1, so R4's RD would be **greater** than 3000. Turn that around: if a neighbour reports a distance **smaller** than my own best, its path can't go through me, because nothing that goes through me can be shorter than me. A neighbour who is closer to the destination than you are cannot be routing via you. R4 might actually be fine — but EIGRP can't prove it, so it won't pre-approve it as a backup.

Story Time : You're 3 km from the train station. A friend phones and says, "I'm 2 km from the station." You don't know their route, but you know one thing for certain: it doesn't pass through you. If it did, they'd be at least 3 km away. Another friend says "I'm 3.5 km away." Their route might be fine, or it might run past your front door. You can't tell from one number, so you don't count on them as your backup. That one comparison is the whole feasibility condition.

Why does the FS matter so much? Because it's what makes EIGRP converge in well under a second. When the successor fails and an FS exists, the router simply promotes it — no calculation, no asking anyone. The route stays **passive** (`P` in the topology table).

And with no FS? The route goes **active** (`A`): the router sends **Queries** to its neighbours asking "do you have another way?" and waits for every **Reply**. If replies never come back, the route becomes **stuck in active (SIA)** — a famous EIGRP failure mode that Chapter 26 dissects, along with the stub routers and summarization that prevent it.

Worked Example: Two Failures in a Row

Start from the table above (R2 successor at 3000, R3 feasible successor at 4000, R4 heard at 3600 but not a feasible successor).

Failure 1 — the link to R2 goes down.

1. R1 removes every path learned through R2.
2. R1 looks for a feasible successor for 10.9.9.0/24 and finds R3 (RD 2500 was below the old FD of 3000).
3. R1 installs the route via R3 straight away, with metric 4000. No packets are sent to ask anyone anything. The route stays **passive** the whole time.
4. R1 sends an Update to its remaining neighbours telling them its distance has changed.

Time to recover: about as long as it takes to notice the link is down, typically well under a second.

Failure 2 — now the link to R3 goes down too.

1. R1 has no feasible successor left. R4 is still there, but it never passed the feasibility test: its RD of 3500 is not below the FD. (Switching to a feasible successor does not raise the FD; per the DUAL rules it stays at 3000 until the route next goes active.)
2. R1 marks the route **active** and sends a **Query** to R4 (and any other neighbours): "I've lost 10.9.9.0/24. Do you have a path?"
3. R4 checks its own table. Its path doesn't depend on R1, so it **Replies** with its distance: 3500.
4. Once R1 has a Reply from every neighbour it queried, it picks the best answer, installs the route via R4 (metric 3600) and goes **passive** again.

That's what "diffusing" means: the question spreads out from the router that lost the path, and the answers flow back. It's still fast on a small network, but every queried router has to answer. That's why large EIGRP designs limit how far Queries can travel (Chapter 26).

Reading the Topology Table

```
R1# show ip eigrp topology
EIGRP-IPv4 Topology Table for AS(100)/ID(1.1.1.1)
Codes: P - Passive, A - Active, U - Update, Q - Query, R - Reply,
       r - reply Status, s - sia Status

P 10.9.9.0/24, 1 successors, FD is 3072
        via 10.0.12.2 (3072/2816), GigabitEthernet0/1
        via 10.0.13.3 (5632/2816), GigabitEthernet0/2
```

Each `via` line shows **(total metric / reported distance)**. The first line is the successor (3072 = FD). The second has RD 2816 < FD 3072, so it's a feasible successor — that's why it's listed at all. `show ip eigrp topology` **only shows successors and feasible successors**; to see every path EIGRP heard, including ones that failed the feasibility test, use `show ip eigrp topology all-links`.

5.4 The EIGRP Metric — K-Values, Classic and Wide

The problem: "which path is best?" needs a single number to compare. A hop count (RIP) treats a 10 Mbps link and a 10 Gbps link as equal. EIGRP's metric instead scores a path by how fast it is (bandwidth) and how long it takes (delay), so the router can prefer the path a person would actually choose.

The Classic Composite Metric

EIGRP's metric can combine **bandwidth, delay, load and reliability**, weighted by five **K-values**:

```
metric = 256 × [ K1·BW + (K2·BW)/(256 − load) + K3·delay ] × [ K5/(reliability + K4) ]
```

The last bracket is applied only when K5 is non-zero. With the defaults **K1 = 1, K2 = 0, K3 = 1, K4 = 0, K5 = 0**, the whole thing collapses to:

```
metric = 256 × ( BW + delay )
```

Term	How it's calculated	Whose value?
BW	10,000,000 ÷ slowest bandwidth on the path (in kbps), truncated	The **minimum** along the path
delay	Sum of interface delays on the path, in **tens of microseconds**	The **total** along the path

Worked example. A path crosses a T1 (1544 kbps, delay 20,000 µs) and a GigabitEthernet link (1,000,000 kbps, delay 10 µs):

- BW = 10,000,000 ÷ 1544 = **6476** (the T1 is the bottleneck)
- delay = (20,000 + 10) ÷ 10 = **2001**
- metric = 256 × (6476 + 2001) = **2,170,112**

Why minimum bandwidth but total delay? Because they describe different things. A path's **throughput** is limited by its narrowest pipe — a 10 Gbps link after a T1 doesn't make the T1 faster — so only the bottleneck matters. **Latency** is additive: every link adds its own delay. Bandwidth is a bottleneck; delay is a sum.

Story Time : Think of a road trip. How many cars per hour can make the trip depends on the narrowest stretch of road. A single-lane bridge limits everyone, however wide the motorway after it is. How long the trip takes is every stretch added together. EIGRP scores a path the same way: the worst bandwidth, plus the total delay.

Worked example: steering traffic with delay. R1 reaches 10.9.9.0/24 through two all-GigabitEthernet paths, each two links long:

1. Per path: BW = 10,000,000 ÷ 1,000,000 = **10**; delay = 10 µs + 10 µs = 20 µs = **2** tens of µs.
2. Metric = 256 × (10 + 2) = **3072** for both paths (the same 3072 you'll see in the topology output above). EIGRP load-balances across them equally.
3. You want Gi0/1 to be primary, so you raise the delay on Gi0/2: `interface Gi0/2` → `delay 11`. The `delay` command is in **tens of microseconds**, so this means 110 µs.
4. The Gi0/2 path's delay is now 11 (R1's Gi0/2) + 1 (the next link) = **12**. Its metric becomes 256 × (10 + 12) = **5632**, exactly the second `via` line in the topology output in 5.3.
5. Result: Gi0/1 (3072) is the successor. The Gi0/2 path stays in the topology table as a feasible successor, as long as the neighbour's RD (256 × (10 + 1) = 2816) is below 3072, which it is.

Bandwidth on both interfaces is unchanged, so QoS (quality of service) and other protocols see the true speed.

Why are load and reliability switched off by default? Because they change constantly. A metric built on load would rise when a link got busy, shift traffic away, fall, shift traffic back — a routing protocol oscillating with its own traffic. Cisco's own guidance is to leave K2, K4 and K5 at zero. **K-values must match between neighbours**, or the adjacency won't form (the log says `K-value mismatch`).

> **Exam trap — tune delay, not bandwidth.** To steer EIGRP traffic, change `delay` on an interface. `bandwidth` also feeds QoS and other protocols' calculations, so changing it to influence routing can break things elsewhere.

Wide Metrics — Why Classic Ran Out of Room

Classic EIGRP's bandwidth term is $10^7 \div$ kbps. For 10 Gbps that's $10^7 \div 10^7 =$ **1**; for 40 or 100 Gbps it truncates to **0**. And the smallest delay classic can express is 10 µs. So **every link at 10 Gbps and above looks identical** to classic EIGRP.

It's like measuring with a ruler marked only in whole metres. A 1.2 m table and a 1.9 m table both read "1 m", and you can no longer tell them apart. Wide metrics add finer markings.

Named mode (5.6) uses **wide metrics**: a 64-bit calculation that measures delay in **picoseconds** and scales the bandwidth term by 65,536, so modern links are told apart again. Wide metrics also add a **K6** (for extended attributes such as jitter and energy; 0 by default). Because the routing table still holds a 32-bit metric, the 64-bit value is divided by the **RIB scale** (`metric rib-scale`, default **128**) before installation — which is why named-mode metrics in `show ip route` look different from the topology table.

Same idea as OSPF's reference-bandwidth problem (CCNA guide 16.6): a 1990s scale that ran out of room at modern speeds.

5.5 Load Balancing — Equal and Unequal Cost

The problem: you're paying for two WAN links, but the routing protocol only uses the "best" one, so the other sits idle until something fails. Load balancing puts more than one path in the routing table so both links carry traffic.

Equal-cost (ECMP, equal-cost multipath). Both EIGRP and OSPF install up to **4** equal-cost paths by default (`maximum-paths` raises it; the platform maximum varies, commonly 16 or 32). CEF then shares traffic across them per flow (5.8).

Unequal-cost — EIGRP only. The `variance` multiplier lets EIGRP use paths that are worse than the best:

```
R1(config)# router eigrp 100
R1(config-router)# variance 2          ! use any FS whose metric < 2 x FD
```

A path joins the routing table only if **both** hold:

1. It is a **feasible successor** (RD < FD) — so it's provably loop-free.
2. Its total metric is **less than variance × FD**.

Using the 5.3 numbers with `variance 2`: FD = 3000, so the limit is 6000. R3 (total 4000, FS) is installed alongside R2. R4 (total 3600) is **not** — it's under the limit but fails the feasibility condition. Traffic is then shared **in inverse proportion to metric** (`traffic-share balanced`, the default), so the better path carries more.

Worked example: walking `variance 2` through each path.

Path	Total metric	Feasible successor? (RD < FD 3000)	Under 2 × 3000 = 6000?	Installed?
via R2	3000	successor	yes	**Yes** (best path)
via R3	4000	yes (2500 < 3000)	yes	**Yes**
via R4	3600	**no** (3500 > 3000)	yes	**No**

The share is inverse to the metric: 1/3000 against 1/4000, a ratio of **4 : 3**. So about 57% of flows go via R2 and 43% via R3. `show ip route 10.9.9.0` lists both next hops, each with a `traffic share count` showing that ratio.

Why can EIGRP do unequal-cost balancing but OSPF can't? Because EIGRP has a built-in **proof of loop-freedom** for each alternative — the feasibility condition. OSPF's SPF produces one shortest-path tree; any non-shortest path is, by definition, not on it, and OSPF has no mechanism to prove a longer path won't loop. Unequal-cost balancing is only safe if you can prove the worse path doesn't come back through you — and only EIGRP keeps that proof.

5.6 Configuring EIGRP — Classic and Named Mode

The problem: there are two ways to write the same EIGRP configuration. Classic mode is what you'll find on older routers. Named mode is the modern form and the only one with every feature. You need to read both and know when each applies.

Classic (Autonomous-System) Mode

```
R1(config)# router eigrp 100                          ! AS 100 - must match neighbours
R1(config-router)# eigrp router-id 1.1.1.1            ! stable ID (used for externals, logs)
R1(config-router)# network 10.0.12.0 0.0.0.255        ! enable EIGRP on matching interfaces
R1(config-router)# network 10.0.13.0 0.0.0.255
R1(config-router)# network 192.168.1.0 0.0.0.255
R1(config-router)# passive-interface GigabitEthernet0/3   ! advertise, but no hellos
R1(config-router)# variance 2
```

How the `network` command works. It does not say which networks to advertise. It says which **interfaces** run EIGRP. The router compares each interface's IP address against the address and **wildcard mask** (a 0 bit means "must match", a 1 bit means "don't care"). On a match, that interface sends Hellos and its connected subnet is advertised. With R1's interfaces at Gi0/1 10.0.12.1, Gi0/2 10.0.13.1 and Gi0/3 192.168.1.1:

1. `network 10.0.12.0 0.0.0.255` matches any 10.0.12.x address, so Gi0/1 is in.
2. `network 10.0.13.0 0.0.0.255` matches Gi0/2.
3. `network 192.168.1.0 0.0.0.255` matches Gi0/3, but it's passive: 192.168.1.0/24 is advertised, and no Hellos are sent onto the user LAN, so no one there can form a neighbour relationship.

Leave out the second line and Gi0/2 never forms its adjacency to R3. That's the first thing to check when one neighbour is missing.

Auto-summary is **off** by default since IOS 15.0 — older material that says otherwise predates it. Interface-level settings (timers, authentication, delay) are configured on each interface: `ip hello-interval eigrp 100 5`, `delay 100`.

Named Mode — Everything in One Place

```
R1(config)# router eigrp CORP                         ! name is local only
R1(config-router)# address-family ipv4 unicast autonomous-system 100
R1(config-router-af)# eigrp router-id 1.1.1.1
R1(config-router-af)# network 10.0.12.0 0.0.0.255
R1(config-router-af)# af-interface default
R1(config-router-af-interface)# passive-interface    ! passive everywhere by default...
R1(config-router-af-interface)# exit-af-interface
R1(config-router-af)# af-interface GigabitEthernet0/1
R1(config-router-af-interface)# no passive-interface ! ...except toward real neighbours
R1(config-router-af-interface)# exit-af-interface
R1(config-router-af)# topology base
R1(config-router-af-topology)# variance 2             ! routing-table policy lives here
R1(config-router-af-topology)# maximum-paths 4
R1(config-router-af-topology)# exit-af-topology
R1(config-router-af)# exit-address-family
```

Why did Cisco add named mode? Because classic EIGRP scattered its configuration — process settings under `router eigrp`, timers and authentication under each interface, and IPv6 EIGRP as an

entirely separate process (`ipv6 router eigrp`). Named mode gathers everything into one hierarchy with three levels: **address family** (IPv4 or IPv6, VRF), **af-interface** (per-link settings) and **topology base** (what goes into the routing table). It's also the only way to get **wide metrics** and features like SHA-256 authentication. One AS number can be configured either way, and neighbours don't care which — classic and named mode routers peer happily.

Verifying

```
R1# show ip eigrp neighbors
EIGRP-IPv4 Neighbors for AS(100)
H   Address         Interface   Hold Uptime   SRTT   RTO  Q  Seq
                                (sec)         (ms)       Cnt Num
0   10.0.12.2       Gi0/1         13 02:14:07    1   100  0  42
1   10.0.13.3       Gi0/2         12 02:13:58    2   100  0  37
```

Reading the columns: **H** is the order neighbours were learned; **SRTT** (smooth round-trip time) is the average time for an acknowledgement to come back; **RTO** (retransmission timeout) is how long RTP waits before resending a reliable packet; **Seq Num** is the last sequence number received from that neighbour.

Q Cnt (queue count) should be **0**. A Q Cnt that stays above zero means updates are waiting for acknowledgement — a struggling neighbour or a lossy link, and an early warning of SIA trouble. **Hold** counts down from 15 and resets with each hello; if it keeps reaching low single digits, hellos are being lost.

To confirm the K-values and the other process settings in one place:

```
R1# show ip protocols | include K1|variance|Maximum path
  EIGRP metric weight K1=1, K2=0, K3=1, K4=0, K5=0
  Maximum path: 4
  Maximum metric variance 2
```

In named mode the line also shows **K6=0**, which is a quick way to tell which mode a router is running.

5.7 OSPF, Seen Side by Side

The problem: ENCOR asks you to compare the two protocols, so you need OSPF's answers to the same questions you just answered for EIGRP: how it computes a path, how it measures one, how it balances load, and how it keeps a large network manageable.

You know OSPF from CCNA (16.3–16.11); Chapter 6 extends it to multiple areas. For the comparison ENCOR asks for, what matters is:

- **Path operations — how OSPF gets from links to routes.** Each router describes its own links in **LSAs** (link-state advertisements) and floods them to every router in the area. Every router stores them in the **LSDB** (link-state database), which is the shared map. Each router then runs **SPF** (shortest path first, Dijkstra's algorithm) with itself as the starting point and installs the results. The worked example below walks through it.
- **Metric — cost.** `reference bandwidth ÷ interface bandwidth`, summed along the path. One dimension, no K-values. Set `auto-cost reference-bandwidth` identically everywhere (CCNA guide

16.6), or override per interface with `ip ospf cost`. With the default reference of 100 Mbps, FastEthernet and GigabitEthernet and 10 Gbps all get cost **1**, the same problem classic EIGRP has. With `auto-cost reference-bandwidth 100000` (100 Gbps, the value is in Mbps), GigabitEthernet costs **100** and 10 Gbps costs **10**.

- **Load balancing — equal cost only.** Up to 4 paths by default (`maximum-paths`). No variance.
- **Path selection inside OSPF** — before cost is even compared, OSPF prefers route types in a fixed order: **intra-area (O) → inter-area (O IA) → external type 1 (E1/N1) → external type 2 (E2/N2)**. A cheaper inter-area path never beats an intra-area one. For example, if R1 learns 10.5.5.0/24 both as an intra-area route with cost 50 and as an inter-area route with cost 10, it installs the **intra-area route (cost 50)**. Chapter 27 covers this in depth.
- **Area types** — trade detail for simplicity:

Area type	Keeps out	Gets instead
Backbone (area 0)	nothing	all areas connect through it
Normal	nothing	full detail
Stub	Types 4 and 5 (externals, and the ASBR summaries that locate them)	a default route from the ABR
Totally stubby	Types 3, 4 and 5	a default route only
NSSA	Types 4 and 5 — but can originate its own externals as Type 7	local externals (+ default if configured)
Totally NSSA	Types 3, 4 and 5	local Type 7 externals + a default route

Story Time : A house at the end of a cul-de-sac doesn't need a road atlas. Every journey starts by driving out of the one exit. A single sign saying "everywhere else → this way" is enough. That's a **totally stubby area**: its routers don't need the details of the rest of the network, so the ABR (area border router) replaces them all with one default route. A **stub** area is a cul-de-sac that still keeps the names of the other neighbourhoods in town (inter-area routes) but forgets everything outside town (external routes). An **NSSA** (not-so-stubby area) is a cul-de-sac with its own small side gate to somewhere else, such as a branch internet link. It can bring its own external routes in (as Type 7) while still keeping everyone else's out.

What this looks like on a router inside a totally stubby area: its routing table holds the area's own routes plus one inter-area default, and nothing else learned from outside the area:

```
R5# show ip route ospf
O*IA  0.0.0.0/0 [110/2] via 10.1.15.1, 00:04:12, GigabitEthernet0/0
O     10.1.16.0/24 [110/2] via 10.1.15.6, 00:04:12, GigabitEthernet0/1
```

EIGRP has **no areas** at all — it limits the blast radius of a change with **summarization** (which it can do on any interface, on any router) and **stub routers**, which stop queries from spreading.

Worked Example: SPF on a Four-Router Area

Four routers in area 0, with these interface costs:

Link	Cost
R1–R2	10
R1–R3	5
R3–R2	3
R2–R4	1

R1 runs SPF with itself at the root:

1. Start at R1 (distance 0). Its neighbours are R2 at 10 and R3 at 5. These are tentative, not final.
2. The closest tentative router is **R3 (5)**, so it becomes final. From R3, R2 is reachable at 5 + 3 = **8**, which beats the tentative 10, so R2's entry becomes 8 via R3.
3. The closest tentative router is now **R2 (8)**, so it becomes final. From R2, R4 is reachable at 8 + 1 = **9**.
4. **R4 (9)** becomes final. Nothing is left.

R1's results: R3 at cost 5, R2 at cost 8 **via R3** (not the direct link), R4 at cost 9 via R3. Every prefix attached to R4 is installed with cost 9 and next hop R3. If the R3–R2 link fails, every router floods the change, every router reruns SPF, and R1's path to R2 becomes the direct link at cost 10.

Compare with EIGRP: OSPF had to re-run the whole calculation on every router. EIGRP with a feasible successor would just have switched. That's the "pre-computed backup" versus "recomputed by SPF" row in the table below.

EIGRP vs OSPF — the Comparison Table

Feature	EIGRP	OSPF
Type	Advanced distance vector	Link state
Algorithm	DUAL	Dijkstra SPF
Metric	Composite (bandwidth + delay by default)	Cost (bandwidth-based)
Transport	IP protocol 88, RTP	IP protocol 89
Multicast	224.0.0.10 / FF02::A	224.0.0.5, 224.0.0.6 / FF02::5, FF02::6
AD	90 internal, 170 external	110
Hierarchy	Flat — no areas	Areas, area 0 mandatory backbone
Summarization	Any interface, any router	Only at ABRs and ASBRs (autonomous system boundary routers, which redistribute external routes)
Unequal-cost balancing	Yes (`variance`)	No
Backup path	Pre-computed feasible successor	Recomputed by SPF
Neighbor match	AS number, K-values, subnet, auth	Area, subnet, timers, auth, MTU, stub flag, network type compatibility
Vendor support	Cisco (RFC 7868 informational)	Open standard, every vendor

Why would anyone choose EIGRP over OSPF, or vice versa? EIGRP is simpler to design (no area planning), converges extremely fast with feasible successors, and summarizes anywhere. OSPF is a **multi-vendor standard** with a predictable hierarchical design and far better visibility — every router can show you the full map of its area. Mixed-vendor or very large structured networks lean OSPF; all-Cisco networks that value simplicity often run EIGRP.

5.8 Path Selection — the Order a Router Actually Uses

A router answers "where does this packet go?" by applying three rules in a **fixed order**. Mixing up the order is one of the most common CCNP mistakes.

Step	Rule	Decides between
1	**Longest prefix match**	Different prefixes covering the destination — the most specific wins, always
2	**Administrative distance**	The same prefix learned from different sources — lowest AD gets into the RIB

Step	Rule	Decides between
3	**Metric**	The same prefix from the same protocol — lowest metric wins (ties → ECMP)

Worked example. R1's routing table holds:

Route	Source	AD/metric
10.1.0.0/16	EIGRP	90/3072
10.1.1.0/24	OSPF	110/20
0.0.0.0/0	static	1/0

A packet to **10.1.1.5** follows the **OSPF /24**. EIGRP's lower AD is irrelevant: AD only compares routes for the **same prefix**, and /24 and /16 are different prefixes. Longest match is decided at forwarding time, among everything already in the table. AD decides what gets into the table; longest match decides what gets used.

Three more packets through the same table, step by step:

1. **To 10.1.200.9:** the /24 doesn't cover it (10.1.200.x isn't 10.1.1.x). The /16 does, and so does the /0. The longest is /16, so it goes via **EIGRP**.
2. **To 172.16.4.4:** only 0.0.0.0/0 covers it, so it takes the **static default**.
3. **Same prefix, two sources:** now suppose OSPF also learns 10.1.0.0/16. Both EIGRP (AD 90) and OSPF (AD 110) offer the identical /16, so step 2 applies and only the EIGRP route enters the routing table. The OSPF copy isn't thrown away. It stays in OSPF's own database, and if the EIGRP route disappears, OSPF offers it again and it's installed with AD 110.

Story Time : Longest match is how a post office sorts mail. A letter addressed to "Flat 4, 12 Oak Street, Springfield" goes to the most specific match it has, the Oak Street sorting slot, not the generic "Springfield" slot. The "Everywhere else" bin (the default route) is only for letters nothing else matches. AD is different: it's how the post office chooses between two delivery companies that both offer the same Oak Street route. It picks the one it trusts more.

Administrative Distance Reference

Source	AD
Connected	0
Static	1
EIGRP summary	5
eBGP	**20**
EIGRP (internal)	**90**
OSPF	**110**
IS-IS	115
RIP	120
EIGRP external	**170**
iBGP	**200**
Unknown / unusable	255 (never installed)

Chapter 22 shows how AD causes real-world problems — especially when routes are redistributed — and how to change it with `distance`.

From RIB to FIB — How the Decision Actually Gets Executed

Running a lookup through the routing table for every packet would be far too slow, so IOS uses **Cisco Express Forwarding (CEF)**:

Structure	Built from	Holds
RIB (routing table)	Routing protocols, statics, connected	Best routes, possibly with recursive next hops
FIB (forwarding information base, `show ip cef`)	The RIB, with recursion pre-resolved	Prefix → final next hop + outgoing interface
Adjacency table (`show adjacency detail`)	ARP / ND	Pre-built Layer 2 rewrite headers per next hop

Why pre-compute both tables? Because the slow parts of forwarding — recursive lookups ("the next hop for this BGP route is reached via that OSPF route") and building the new Ethernet header — are done **once, when the topology changes**, not per packet. The per-packet work becomes one longest-match lookup in the FIB and a header copy from the adjacency table, simple enough to do in hardware. That's the **control plane** (building the tables) and **data plane** (using them) split you'll meet again with SD-WAN and SD-Access in Chapter 14.

How one packet to 10.1.1.5 is forwarded with CEF:

1. The packet arrives. The router looks up 10.1.1.5 in the **FIB**: the longest match is 10.1.1.0/24, with next hop 10.0.12.2 out Gi0/1.
2. It takes the pre-built Layer 2 header for 10.0.12.2 from the **adjacency table** (destination MAC from ARP, source MAC of Gi0/1).

3. It rewrites the frame, decrements the TTL (time to live), and sends it out Gi0/1. No routing protocol was consulted.

```
R1# show ip cef 10.1.1.5
10.1.1.0/24
  nexthop 10.0.12.2 GigabitEthernet0/1
```

ECMP and CEF. With multiple equal paths, CEF load-shares **per destination (per flow) by default**, hashing source and destination addresses so one conversation always takes one path. Per-packet sharing (`ip load-sharing per-packet`) exists but reorders packets, which TCP treats as loss — avoid it. Same reasoning as EtherChannel hashing (CCNA guide 10.4).

5.9 Policy-Based Routing — Routing by Something Other Than Destination

Every rule in 5.8 looks at **one** field: the destination address. Sometimes that isn't what you want:

- Guest traffic should leave through the cheap internet link, staff traffic through the MPLS WAN.
- Traffic from the backup server should take the slow link so it doesn't crowd out users.
- Voice should prefer the low-latency path regardless of what OSPF thinks.

Policy-based routing (PBR) lets you match packets on **source address, protocol, ports, packet length** — anything an ACL (access control list) can match — and override the routing table's answer.

Story Time : A company mail room normally sends every parcel by the cheapest service for its destination. That's the routing table. But the mail room has a sticky note on the wall: "Anything from the Legal department goes by courier." The sticky note is PBR. It checks something other than the address (who sent it), and for the parcels it matches, it overrides the normal choice. Parcels that don't match the note are handled exactly as before.

How It's Built

PBR is a **route map** (Chapter 23 covers route maps fully) applied to an **incoming** interface:

```
R1(config)# ip access-list extended GUEST-NET
R1(config-ext-nacl)# permit ip 10.50.0.0 0.0.255.255 any
R1(config)# route-map WAN-POLICY permit 10
R1(config-route-map)# match ip address GUEST-NET      ! which packets
R1(config-route-map)# set ip next-hop 203.0.113.1     ! where they go instead
R1(config)# interface GigabitEthernet0/2              ! the interface guests ARRIVE on
R1(config-if)# ip policy route-map WAN-POLICY
```

Piece	Role
`match ip address`	Selects packets via an ACL
`set ip next-hop`	Forces the next hop, ignoring the routing table

Piece	Role
`set ip default next-hop`	Used **only if** the routing table has no explicit route (default route excluded)
`set interface`	Forces the exit interface (best used on point-to-point links)
`ip policy route-map`	Applies PBR to packets **arriving** on this interface
`ip local policy route-map`	Applies PBR to packets the **router itself** originates (pings, syslog, SNMP)

What happens to traffic the route map doesn't match? It's routed **normally**. A packet that falls through every sequence, or hits a `deny` sequence, simply uses the routing table. PBR is an override, not a replacement. Likewise, if the configured next hop isn't reachable (not on a connected, up subnet), the packet falls back to normal routing.

Worked Example: Three Packets Through WAN-POLICY

The routing table on R1 sends everything (including the default route) to the MPLS WAN at 10.0.99.1.

1. **Guest 10.50.3.7 → 8.8.8.8, arriving on Gi0/2.** The interface has a policy, so R1 checks route-map sequence 10. ACL GUEST-NET permits source 10.50.0.0/16, so it matches. 203.0.113.1 is on a connected, up subnet, so the packet goes to **203.0.113.1** (the internet link). The routing table is never consulted.
2. **Staff 10.20.1.1 → 8.8.8.8, arriving on Gi0/2.** The ACL doesn't match, so the packet falls through the route map and is routed **normally**, via 10.0.99.1.
3. **Guest 10.50.3.7 → 8.8.8.8, with the internet link down.** The sequence matches, but 203.0.113.1 is no longer reachable. The `set` is skipped and the packet is routed normally via 10.0.99.1.

Now swap `set ip next-hop` for **`set ip default next-hop 203.0.113.1`** and send two guest packets:

- **To 10.1.1.5:** the routing table has an explicit route (10.1.1.0/24), so that route is used. PBR does nothing.
- **To 8.8.8.8:** only the default route covers it. A default route doesn't count as explicit, so PBR sends it to **203.0.113.1**.

`set ip next-hop` overrides the routing table; `set ip default next-hop` only fills the gap where the table has nothing specific.

To check that the policy is matching, look at the counters:

```
R1# show route-map WAN-POLICY
route-map WAN-POLICY, permit, sequence 10
  Match clauses:
    ip address (access-lists): GUEST-NET
  Set clauses:
    ip next-hop 203.0.113.1
  Policy routing matches: 1520 packets, 98432 bytes
```

If the match counters stay at zero while guests are sending traffic, the policy is on the wrong interface (remember: it must be the one the traffic arrives on), or the ACL doesn't match the real source addresses.

Why is PBR applied inbound and not outbound? Because the forwarding decision happens when a packet **arrives**. By the time a packet is queued on an outgoing interface, the router has already chosen that interface — there's nothing left to override. That's also why router-generated traffic needs the separate `ip local policy`: those packets never arrive on any interface.

Why not use PBR everywhere? Because it's **invisible to the routing protocols**. OSPF and EIGRP don't know about the policy, so they can't route around a failed policy next hop (unless you add tracking — Chapter 25), and anyone troubleshooting with `show ip route` sees the wrong answer. PBR is a precise tool for exceptions; if most traffic needs it, the routing design is wrong.

Chapter 25 covers PBR configuration in depth — `verify-availability` with IP SLA tracking, `set ip next-hop recursive`, and verifying with `show route-map` and `debug ip policy`.

5.10 Summary — What to Be Able to Explain

You should be able to…	Section
Explain why EIGRP is "advanced distance vector", not link state	5.1
Describe how EIGRP neighbours form and why hold time must exceed the hello interval	5.2
Define RD, FD, successor, FS and apply the feasibility condition	5.3
Walk through what DUAL does with and without a feasible successor	5.3
Calculate a classic EIGRP metric and explain why wide metrics exist	5.4
Explain why only EIGRP does unequal-cost load balancing	5.5
Configure EIGRP in classic and named mode	5.6
Compare EIGRP and OSPF across metric, hierarchy, convergence and area types	5.7
Run SPF by hand on a small topology	5.7
Apply longest-match → AD → metric in the right order	5.8
Describe what PBR does, where it's applied and when it falls back	5.9

Chapter 6: OSPFv2/v3 — Multi-Area, Summarization & Filtering

> **Blueprint:** ENCOR 3.2.b — Configure simple OSPFv2/v3 environments, including multiple normal areas, summarization, and filtering (neighbor adjacency, point-to-point and broadcast network types, and passive-interface).

The CCNA guide built single-area OSPF (16.3–16.10) and then lifted the lid in 16.11: adjacency states, LSA types, and stub areas. This chapter turns that preview into a working multi-area design — the shape almost every real OSPF network has — and adds the two tools that make areas worth having: **summarization** and **filtering**. It finishes with **OSPFv3**, which is OSPF for IPv6 (and, optionally, IPv4 too).

Chapter 27 goes further for ENARSI: NBMA network types, virtual links, the full area-type matrix and authentication.

Terms used throughout (all from CCNA, restated so nothing is assumed): an **LSA** (link-state advertisement) is one fact a router floods, such as "I have these links at these costs". The **LSDB** (link-state database) is the collection of every LSA a router holds, which is its map. **SPF** (shortest path first, Dijkstra's algorithm) is the calculation each router runs over that map. The **RID** (router ID) is the 32-bit dotted number that names each OSPF router. The **DR** and **BDR** (designated and backup designated router) are the routers that collect and redistribute updates on a shared LAN segment.

6.1 Why Multiple Areas — and the Router Roles They Create

Recall the reason areas exist (CCNA guide 16.5): in one big area, every router stores every link and reruns SPF whenever **any** link anywhere flaps. Areas **contain the detail**. Inside an area, routers know the full map; between areas, they exchange only **summaries** of what's reachable.

Story Time : Think of a large company. Inside the Finance department, everyone knows exactly who sits at which desk. Reception on the ground floor doesn't need that detail. It only needs to know "Finance: 3rd floor". When someone in Finance moves desks, only Finance has to update its seating plan. Reception's directory doesn't change. OSPF areas work the same way: routers inside an area hold the full detailed map, and everyone else holds a short "this range of addresses is over there" line. The **ABR** is the receptionist who knows both sides.

Worked example: what a flap costs with and without areas. A user-facing link in area 1 goes up and down every few seconds.

1. **One big area of 200 routers:** the router with the flapping link floods a new Type 1 LSA to all 200 routers. All 200 rerun SPF every time it flaps.
2. **Area 1 with 20 routers, plus area 0 and other areas:** the new Type 1 floods only to the 20 routers in area 1, and only they rerun full SPF. ABR1 updates the one Type 3 LSA for that

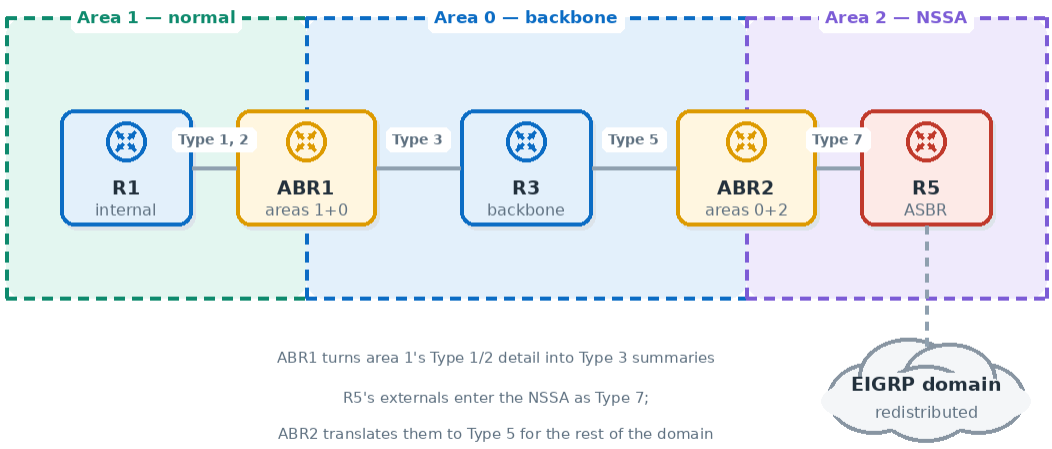
prefix in the other areas. Routers there recalculate just that route, which is cheap.

3. **And with a summary at ABR1 (6.5):** the /22 summary is still valid while any of its component networks is up, so routers outside area 1 see **no change at all**.

That creates four router roles — one router can hold several at once:

Role	Definition	Example below
Internal router	All interfaces in one area	R1
Backbone router	At least one interface in area 0	R3, ABR1, ABR2
ABR (Area Border Router)	Interfaces in area 0 **and** at least one other area	ABR1, ABR2
ASBR (Autonomous System Boundary Router)	Redistributes routes from outside OSPF	R5

Multi-area OSPF — which router speaks which LSA, and where it travels



LSA scope: 1, 2, 3, 7 stay in one area · 4 and 5 flood the whole domain (except stub areas)

The golden rule: every non-backbone area must touch **area 0**, and traffic between two non-backbone areas always crosses area 0. **Why?** Because inter-area routing in OSPF is really **distance vector**. An ABR doesn't pass area 1's map into area 0 — it passes "network X, cost Y", the same kind of rumour RIP trades. Rumours can loop. Forcing every inter-area exchange through a single backbone gives the rumours a strict hub-and-spoke shape with no path back around, so they can't loop. Inside an area OSPF is link state; between areas it's a distance-vector star centred on area 0. (An area that can't physically reach area 0 needs a **virtual link** — Chapter 27.)

6.2 LSA Types — What Each Router Says, and How Far It Travels

The CCNA guide listed the LSA types (16.11). Here's what each one is **for** in a multi-area network, and the command that shows it:

Type	Name	Generated by	Floods within	Becomes route code	[redacted]
1	Router	Every router	Its own area	`O`	`router`
2	Network	The **DR** on a broadcast segment	Its own area	`O`	`network`
3	Summary	**ABR** — one per prefix (or per range)	Into the next area	`O IA`	`summary`
4	ASBR summary	**ABR** — "the ASBR is reachable via me"	Into the next area	— (none)	`asbr-summary`
5	External	**ASBR**	Whole OSPF domain (not stub areas)	`O E1` / `O E2`	`external`
7	NSSA external	**ASBR inside an NSSA**	That NSSA only	`O N1` / `O N2`	`nssa-external`

Following the diagram from left to right:

1. **R1** describes its links in a **Type 1**; a DR on any area 1 LAN adds a **Type 2**. Both stay inside area 1.
2. **ABR1** reads area 1's Type 1/2 database, works out each prefix and its cost, and advertises each prefix into area 0 as a **Type 3**. Area 0 routers see `O IA` routes — "inter-area, reached via ABR1."
3. **R5** redistributes EIGRP routes. Because area 2 is an NSSA (6.3), they enter as **Type 7**.
4. **ABR2** translates each Type 7 into a **Type 5** and floods it to the rest of the domain. In the translated LSA, **ABR2 itself** is the advertising router, not R5. Area 0 routers already know how to reach ABR2 from its Type 1 LSA, so nothing more is needed there. Routers in **area 1** don't have ABR2's Type 1, so **ABR1** advertises a **Type 4** into area 1 saying "ABR2, the router advertising these externals, is reachable through me."

Why is a Type 4 needed at all? A Type 5 says "network X is reachable via ASBR R5" — but routers in area 1 have never seen R5's Type 1 LSA (it's in another area), so they have no idea how to reach R5. The ABR fills the gap with a Type 4: "R5 is reachable through me, at this cost." Type 5 says what's out there; Type 4 says how to get to the router that said it. (When an NSSA ABR translates Type 7 to Type 5, **the ABR itself** becomes the advertising router, so other areas just need a path to the ABR.)

E1 versus E2

External routes come in two metric styles, chosen at redistribution time (`metric-type 1` or `2`):

Type	Metric used	Default?	Use when
E2 / N2	The external **seed metric only** — internal cost to the ASBR is ignored	**Yes** (seed 20; 1 for routes from BGP)	One exit point, or you don't care which ASBR is used
E1 / N1	Seed metric **plus** the internal cost to reach the ASBR	No	Several ASBRs advertise the same prefix and you want the closest one used

With E2, if two E2 routes have the same seed metric, OSPF does break the tie on the cost to the ASBR (the "forward metric") — but it never adds it into the displayed metric.

For the **same prefix**, OSPF also prefers an E1 route over an E2 route whatever their metrics, following the route-type order in 5.7 (intra-area, inter-area, E1, E2).

Worked example: why E1 exists. Two ASBRs both redistribute 172.16.0.0/16. From R1, the internal OSPF cost to ASBR-A is **10** and to ASBR-B is **50**.

Case	Via ASBR-A	Via ASBR-B	R1 picks	Why
E2, both seed 20	20	20	**A**	Tie on the metric, broken by the forward metric (10 < 50). Shown as `[110/20]`
E2, A seed 30, B seed 20	30	20	**B**	E2 compares seeds only. R1 crosses 50 internal cost to save 10
E1, A seed 30, B seed 20	30 + 10 = **40**	20 + 50 = **70**	**A**	E1 counts the trip to the ASBR, so the genuinely closer exit wins

The middle row is the trap: with E2, a router can pick a far-away exit because the seed metric is all it compares. When several exits advertise the same prefixes, use E1 so every router uses the exit nearest to it.

6.3 Area Types at ENCOR Depth

The problem: some areas have only one way out, typically a branch site behind one ABR. Holding hundreds of external routes there costs memory and CPU and adds nothing, because every one of them points at the same ABR anyway. Special area types replace that detail with a default route.

ENCOR 3.2.b asks for **multiple normal areas**; Chapter 5.7 summarised the special types. Configuration is short:

Area type	Config (in)	Where
Stub	`area 1 stub`	**Every** router in area 1
Totally stubby	`area 1 stub no-summary`	ABR only (other routers just need `area 1 stub`)
NSSA	`area 2 nssa`	Every router in area 2
Totally NSSA	`area 2 nssa no-summary`	ABR only (others need `area 2 nssa`)

Why must every router in a stub area agree? Because the stub setting is carried as a **flag in the Hello** (the E-bit). Two routers that disagree about whether external LSAs may exist in their area would build different databases — so OSPF refuses the adjacency. A mismatched stub flag is one of the quieter reasons neighbours never form.

Default routes: stub and totally stubby areas get a Type 3 default from the ABR automatically, and so does a totally NSSA. A plain **NSSA does not** — add `area 2 nssa default-information-originate` on the ABR if area 2 needs one.

Worked example: the same router, three area types. R1 is an internal router in area 1. The network has 50 prefixes in area 1, 400 in other areas and 2,000 externals (redistributed from elsewhere).

Area 1 configured as	What R1's routing table holds	Roughly how many OSPF routes
Normal	`O` for area 1, `O IA` for other areas, `O E2` for externals	50 + 400 + 2,000 = 2,450
Stub (`area 1 stub` everywhere)	`O`, `O IA`, plus `O*IA 0.0.0.0/0` from the ABR	50 + 400 + 1 = 451
Totally stubby (`no-summary` added on the ABR)	`O` plus `O*IA 0.0.0.0/0`	50 + 1 = 51

R1 can still reach every one of those destinations in all three cases. Anything it no longer knows about matches the default and goes to the ABR, which is exactly where it would have gone anyway.

Originating a Default Into Normal Areas

In normal areas, a default route is just another external route, and it's created on the router that owns the exit (typically the internet edge) with **`default-information originate`**:

```
EDGE(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.1          ! the real exit
EDGE(config)# router ospf 1
EDGE(config-router)# default-information originate          ! only while a default is in RIB
EDGE(config-router)# default-information originate always   ! ...or: advertise it regardless
EDGE(config-router)# default-information originate metric 10 metric-type 1  ! optional
```

The command makes EDGE an **ASBR** and floods `0.0.0.0/0` as a **Type 5** LSA, seen elsewhere as `O*E2 0.0.0.0/0` (metric 1, type E2 by default).

Without `always`, nothing is originated unless EDGE has a default route in its routing table from **some other source**, such as a static route, DHCP or BGP. If that default disappears, the Type 5 is withdrawn. **Why is that the safer default behaviour?** Because the default route is a promise: "send me anything you can't route and I'll deliver it." If EDGE has lost its own exit, it can't keep that promise, and withdrawing the advertisement lets the rest of the network fail over to another exit. `always` keeps advertising even with no default in EDGE's RIB. Use it when the default should point at EDGE no matter what, for example when EDGE forwards to a firewall reached by a route that isn't a default. Used carelessly, it **black-holes** traffic at a router that has nowhere to send it.

With two internet exits, advertise the default from both and use `metric-type 1` (E1) so each router picks the **closest** exit, or give one exit a lower `metric` to make it primary.

For example, EDGE-A and EDGE-B both use `default-information originate metric 10 metric-type 1`. R7 is cost 5 from EDGE-A and cost 20 from EDGE-B, so it sees `O*E1 0.0.0.0/0 [110/15]` via EDGE-A (10 + 5, against 10 + 20 = 30 via EDGE-B). A router on the other side of the network, close to EDGE-B, picks EDGE-B. If EDGE-A loses its own default route, it withdraws the Type 5, and R7 moves to EDGE-B at metric 30.

6.4 Configuring Multi-Area OSPFv2

The problem: each interface has to be placed in exactly the right area. An interface in the wrong area won't form an adjacency (the area ID must match), or it will advertise its subnet into the wrong part of the design.

Two ways to put an interface in an area. Both are valid IOS XE, and you'll see both on the exam:

```
! Method 1 - network statements (wildcard match, as in CCNA)
ABR1(config)# router ospf 1
ABR1(config-router)# router-id 2.2.2.2
ABR1(config-router)# network 10.1.12.0 0.0.0.255 area 1
ABR1(config-router)# network 10.0.23.0 0.0.0.255 area 0

! Method 2 - directly on the interface (clearer, and wins over network statements)
ABR1(config)# interface GigabitEthernet0/1
ABR1(config-if)# ip ospf 1 area 1
ABR1(config)# interface GigabitEthernet0/2
ABR1(config-if)# ip ospf 1 area 0
```

Why prefer interface mode? Because it's explicit. A `network` statement is a pattern; if someone later adds an interface whose address happens to match, it silently joins OSPF, in whatever area the pattern says. Interface-level `ip ospf 1 area 1` puts exactly one interface in exactly one area, and reading the interface config tells you everything. Patterns surprise you; statements don't.

An ABR needs nothing special — it **becomes** an ABR the moment it has interfaces in area 0 and another area. Verify with:

```
ABR1# show ip ospf | include border|Number of areas in
 It is an area border router
 Number of areas in this router is 2. 2 normal 0 stub 0 nssa
```

The full `show ip ospf` output then lists each area (`Area BACKBONE(0)`, `Area 1`) with its interface count, authentication and configured ranges.

To check which area each interface landed in, use one line per interface:

```
ABR1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/2        1     0               10.0.23.2/24       1     BDR   1/1
Gi0/1        1     1               10.1.12.2/24       1     DR    1/1
```

Reading it: Gi0/2 is in area 0 and Gi0/1 in area 1, which is what makes ABR1 an ABR. **State** is this router's role on that segment (DR, BDR, DROTHER, or P2P on point-to-point links). **Nbrs F/C** is "neighbours FULL / total neighbours". On a link with only two routers, a value like `0/1` means a neighbour has been seen but the adjacency hasn't completed. On a busy LAN, a DROTHER legitimately shows F lower than C, because it stays in 2WAY with the other DROTHERs.

6.5 Summarization

The problem: without summaries, every prefix in every area is advertised separately everywhere, and every change to any of them ripples through the whole network.

Without summarization, ABR1 sends one Type 3 LSA into area 0 for **every** prefix in area 1. A flap of any one of them ripples into every other area as a changed Type 3. Summarization replaces the list with one advertisement — and a single summary only changes if **every** component route disappears.

OSPF can summarize in exactly **two** places, each with its own command:

Where	Summarizes	Command (under)
ABR	Internal routes (Type 1/2) of an area, into Type 3s	`area <area-id> range <net> <mask>`
ASBR	Redistributed externals (Type 5 or 7)	`summary-address <net> <mask>`

Inter-Area Summary at the ABR

Area 1 contains 10.1.0.0/24, 10.1.1.0/24, 10.1.2.0/24 and 10.1.3.0/24:

```
ABR1(config)# router ospf 1
ABR1(config-router)# area 1 range 10.1.0.0 255.255.252.0    ! one /22 instead of four /24s
```

How to work out the summary. Write the third octet (the one that changes) in binary:

Network	Third octet	Binary
10.1.0.0	0	00000**00**
10.1.1.0	1	00000**01**
10.1.2.0	2	00000**10**
10.1.3.0	3	00000**11**

The first 6 bits are identical in all four, and the last 2 bits take every combination. So the common part is 8 + 8 + 6 = **22** bits: **10.1.0.0/22**, mask 255.255.252.0. The summary covers exactly these four /24s and nothing else. If area 1 had only 10.1.0.0–10.1.2.0, the /22 would also claim 10.1.3.0/24, which doesn't exist. That's legal, and the Null0 route below is what makes it safe.

Story Time : A summary is like a postcode. A sorting office in another city doesn't need every street name in your town. "All of 10.1.x up to 10.1.3.x → ABR1" is enough to get the packet to the right town, and ABR1 knows the streets.

The `area` number is the area **the routes live in** (area 1), not the area the summary is sent to. Now area 0 sees a single `O IA 10.1.0.0/22`. On ABR1 itself a new route appears:

```
ABR1# show ip route ospf | include Null0
O        10.1.0.0/22 is a summary, 00:01:12, Null0
```

Why does the ABR install a summary route pointing at Null0? To prevent a loop. Suppose 10.1.3.0/24 goes down. Other areas still route 10.1.3.x toward ABR1 (the /22 summary is still advertised). ABR1 has no /24 anymore, so without the discard route it would follow its **default route** — possibly straight back into area 0, which sends it back to ABR1, and so on until the TTL dies. The Null0 summary catches anything inside the /22 that has no more specific route and **drops it** on the spot. A summary is a promise; the Null0 route makes sure the router can't keep that promise by bouncing traffic in circles.

External Summary at the ASBR

```
R5(config)# router ospf 1
R5(config-router)# summary-address 172.16.0.0 255.255.0.0   ! summarize redistributed routes
```

> **Exam trap — right command, right router.** `area range` on an ASBR does **nothing** to external routes, and `summary-address` on an ABR does **nothing** to inter-area routes. Inter-area: `area range` on the ABR. External: `summary-address` on the ASBR (or on the NSSA ABR doing the Type 7→5 translation).

The design price of summarizing: you lose detail. Routers outside the area can no longer see that 10.1.3.0/24 is down — they'll keep sending traffic to ABR1, which drops it. And with two ABRs advertising the same summary, a failure behind one of them is invisible to the other areas. Summaries trade precision for stability.

6.6 Filtering

The problem: sometimes part of the network shouldn't be reachable from elsewhere. Examples are a lab network, or a test prefix that mustn't leak into production. You need to stop those routes from spreading without breaking OSPF's database rules.

Filtering in OSPF is unusual, because of the link-state rule that **every router in an area must hold an identical LSDB**. You can't simply stop an LSA from reaching one router in the middle of an area. What you can do depends on where you are:

Tool	Where	What it actually filters	Effect on LSDB
`area X filter-list prefix NAME in|out`	ABR	**Type 3** LSAs entering (`in`) or leaving (`out`) area X	Changes other areas' LSDBs — consistently
`area X range … not-advertise`	ABR	Suppresses the summary (and its components)	Same — no Type 3 is created
`distribute-list … in`	Any router	Routes going from the LSDB **into the routing table**	**None** — LSAs still arrive and flood on
`distribute-list … out`	ASBR	Redistributed routes before they become Type 5/7	Externals never created

Type 3 Filtering at the ABR

Stop area 0 (and everything beyond it) from learning area 1's lab network 10.1.3.0/24:

```
ABR1(config)# ip prefix-list NO-LAB seq 5 deny 10.1.3.0/24
ABR1(config)# ip prefix-list NO-LAB seq 10 permit 0.0.0.0/0 le 32   ! allow everything else
ABR1(config)# router ospf 1
ABR1(config-router)# area 1 filter-list prefix NO-LAB out           ! out OF area 1
```

How ABR1 checks each area 1 prefix against NO-LAB before creating a Type 3 for area 0 (assuming no `area 1 range` is configured):

1. **10.1.3.0/24:** seq 5 matches exactly and says deny, so no Type 3 is created. Area 0 never hears of it.
2. **10.1.2.0/24:** seq 5 doesn't match. Seq 10 (`0.0.0.0/0 le 32`, meaning any prefix of any length) matches and says permit, so the Type 3 is created.
3. Every other area 1 prefix takes the same path as step 2.

On a router in area 0, `show ip ospf database summary 10.1.3.0` now returns nothing, and `show ip route 10.1.3.0` says `% Network not in table`.

`out` means "Type 3s generated **from** area 1's routes"; `in` means "Type 3s being sent **into** area 1." Don't forget the final `permit … le 32`: like an ACL, a prefix list ends in an **implicit deny**, and without that line you'd filter every inter-area route. (Prefix-list syntax in full: Chapter 23.)

The distribute-list "in" Trap

```
R3(config)# router ospf 1
R3(config-router)# distribute-list prefix NO-LAB in
```

This removes 10.1.3.0/24 from **R3's routing table only**. R3 still holds the LSA, still floods it to its neighbours, and they still install the route — and may forward traffic for it **to R3**, which now has no route and drops it (or follows a default). A distribute-list in OSPF is a local blindfold, not a wall. Use it deliberately, for example to stop one router installing a path it shouldn't use; use ABR

filtering when you mean "this area should not know."

6.7 Neighbor Adjacency — Network Types & Passive Interfaces

The problem: routes can only be exchanged once neighbours have formed, and how neighbours form depends on the type of link. On top of that, some interfaces should never form neighbours at all. This section covers getting adjacencies right and keeping them where they belong.

Point-to-Point vs Broadcast

You know both from CCNA (16.10). The ENCOR-level detail is what happens when they **mismatch**:

Network type	DR/BDR	Hello / Dead	Default on
Broadcast	Yes	10 / 40 s	Ethernet
Point-to-point	No	10 / 40 s	Serial (HDLC/PPP), GRE tunnels
Non-broadcast, point-to-multipoint	(Chapter 27)	30 / 120 s	Frame Relay–style NBMA

On an Ethernet link between exactly two routers, it's common to change the type so no DR election is held:

```
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ip ospf network point-to-point     ! configure on BOTH ends
```

DR/BDR on Broadcast Links — a Recap

On a **broadcast** segment, OSPF elects a **DR** and **BDR** (CCNA guide 16.10). Every other router (DROTHER) forms a full adjacency only with those two. The rules that ENCOR still tests:

- **Highest interface priority wins** (`ip ospf priority 0–255`, default 1), then **highest router ID**.
- **Priority 0** = never DR or BDR.
- The election is **not preemptive**. A better router arriving later doesn't take over until the DR fails or OSPF is reset (`clear ip ospf process`).

```
R4(config-if)# ip ospf priority 0            ! R4 may never be DR/BDR on this segment
R4# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
1.1.1.1         100   FULL/DR         00:00:36    10.1.1.1        GigabitEthernet0/0
2.2.2.2          50   FULL/BDR        00:00:33    10.1.1.2        GigabitEthernet0/0
3.3.3.3           1   2WAY/DROTHER    00:00:38    10.1.1.3        GigabitEthernet0/0
```

Read the last line carefully: two DROTHERs stop at **2WAY** with each other by design, so `2WAY/DROTHER` is healthy, not stuck. FULL with the DR and BDR, 2WAY with other DROTHERs, is a correct broadcast segment. A neighbor stuck in 2WAY on a **point-to-point** link is the problem

case.

Why have a DR at all? Without one, every router on a LAN would form a full adjacency with every other router and swap databases with each of them. With n routers that's $n(n-1)/2$ adjacencies. With a DR and BDR, each other router (a **DROTHER**) needs only two.

Routers on the LAN	Full adjacencies without a DR	With DR + BDR
5	10	7
10	45	17

Story Time : A team meeting where everyone has news. If all ten people tell each of the other nine individually, that's a lot of conversations. Instead one person takes the notes (the DR) and a second keeps a copy in case the first leaves (the BDR). Everyone gives their news to the note-taker, and the note-taker sends the summary to everyone. On OSPF terms: DROTHERs send updates to 224.0.0.6 (DR and BDR), and the DR re-floods them to 224.0.0.5 (all OSPF routers).

Worked example: the election, and why it doesn't preempt. Four routers boot on 10.1.1.0/24 at the same time:

Router	Priority	RID
R1	100	1.1.1.1
R2	50	2.2.2.2
R3	1	3.3.3.3
R4	0	4.4.4.4

1. R4 has priority 0, so it's out of the running.
2. Highest priority wins: **R1 (100) becomes DR**, and **R2 (50) becomes BDR**. RID would only matter if priorities tied.
3. R3 is a DROTHER. This is the `show ip ospf neighbor` output above, seen from R4.
4. Later, R5 joins with priority 200. It does **not** take over. R1 stays DR and R5 becomes a DROTHER.
5. R1 fails. **R2 (the BDR) is promoted to DR** straight away, because it already has full adjacencies with everyone. A new BDR election follows, and R5 (200) wins it.

Why no preemption? Every change of DR forces the DROTHERs to build new adjacencies and resynchronise their databases. Keeping a working DR is better than chasing the "best" one. If you want a particular router to be DR, give it the highest priority and make sure it's up first, or reset the process during a maintenance window.

The Sneaky Mismatch

One end set to `ip ospf network point-to-point`, the other left as broadcast. The timers are identical (10/40), so the hellos are accepted and the adjacency may even reach **FULL**. But the broadcast side expects a DR and describes the link through a Type 2 LSA, while the point-to-point side doesn't. The two routers describe the same link differently, SPF can't match them up, and **routes across that link never appear**. FULL neighbours with missing routes should make you check `show ip ospf interface` on both ends.

Neighbour requirements you must be able to recite (and the command that proves each):

Must match	Proven by
Area ID	`show ip ospf interface`
Subnet and mask (on broadcast links)	`show ip interface brief`, `show run interface`
Hello / dead intervals	`show ip ospf interface`
Authentication type and key	`debug ip ospf adj` ("Mismatched authentication")
Stub/NSSA flags	`debug ip ospf hello` ("mismatched Stub/Transit area option bit")
MTU (or the adjacency stalls in ExStart/Exchange)	`show interface | include MTU`
Unique router IDs	`show ip ospf` — duplicates log `%OSPF-4-DUP_RTRID`

Passive Interfaces

```
R1(config)# router ospf 1
R1(config-router)# passive-interface default                ! no hellos anywhere...
R1(config-router)# no passive-interface GigabitEthernet0/1  ! ...except to real neighbours
```

A passive interface **stops sending and processing hellos** — so no neighbour can form on it — but its subnet is **still advertised**. That's exactly right for user LANs: the users' subnet must be reachable, but no device there should be able to become an OSPF neighbour and inject routes.

What goes wrong without it: a user on 192.168.1.0/24 runs a lab router (or malware) that speaks OSPF. Gi0/3 isn't passive, so R1 forms an adjacency with it. The rogue device advertises 0.0.0.0/0 with a low cost, and part of the network starts sending internet traffic to a laptop. A passive interface ignores its Hellos, so the adjacency can never form, and adding OSPF authentication (Chapter 27) closes the gap on interfaces that must stay active.

Why "passive-interface default" rather than listing the user ports? Safety by default. A new interface added next year is passive unless someone consciously makes it a routing link. Fail closed: an interface has to be invited into the routing protocol, not remembered and excluded.

6.8 OSPFv3 — OSPF for IPv6 (and IPv4)

The problem: IPv6 networks need a routing protocol too. OSPFv2 can only carry IPv4, so IPv6 needs OSPFv3. Modern OSPFv3 can carry IPv4 as well, which lets one protocol do both jobs.

OSPFv3 (RFC 5340) is the same protocol — same areas, LSA flooding, SPF, DR/BDR, network types — rebuilt for IPv6. What changed:

Topic	OSPFv2	OSPFv3
Runs over	IPv4 subnets	IPv6 **links** — an interface can have many prefixes
Multicast	224.0.0.5 / 224.0.0.6	**FF02::5 / FF02::6**
Neighbour addresses / next hops	Interface IPv4 address	**Link-local** addresses

Topic	OSPFv2	OSPFv3
Router ID	32-bit	**Still 32-bit, dotted** — set it manually if the router has no IPv4 address
Addressing in Type 1/2	Yes	**No** — topology only; prefixes moved to new LSAs
New LSAs	—	**Type 8** link LSA (link-local + prefixes on a link), **Type 9** intra-area-prefix LSA
Authentication	In OSPF itself	IPsec (or the OSPFv3 authentication trailer)

Why separate topology from addressing? In OSPFv2 a prefix change on a router changes its Type 1 LSA, and a changed Type 1 means a full SPF run for the area. OSPFv3 keeps topology (Type 1/2) and prefixes (Type 8/9) in separate LSAs, so adding or removing a prefix triggers only a cheap partial recalculation. The map of the roads and the list of addresses on each road change at different speeds, so OSPFv3 keeps them on different pages.

Traditional OSPFv3 Configuration

```
R1(config)# ipv6 unicast-routing                    ! IPv6 routing is OFF by default
R1(config)# ipv6 router ospf 1
R1(config-rtr)# router-id 1.1.1.1                   ! required if no IPv4 address exists
R1(config-rtr)# passive-interface GigabitEthernet0/3
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ipv6 address 2001:db8:12::1/64
R1(config-if)# ipv6 ospf 1 area 0                   ! no network statements in OSPFv3
```

Address-Family OSPFv3 — One Process, Both Protocols

Modern IOS XE uses `router ospfv3` with **address families**, which can carry **IPv4 routes too** — over the same IPv6-style protocol machinery:

```
R1(config)# router ospfv3 1
R1(config-router)# router-id 1.1.1.1
R1(config-router)# address-family ipv6 unicast
R1(config-router-af)# passive-interface default
R1(config-router-af)# no passive-interface GigabitEthernet0/1
R1(config-router-af)# area 1 range 2001:db8:1::/48    ! IPv6 inter-area summary
R1(config-router-af)# exit-address-family
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# exit-address-family
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ospfv3 1 ipv6 area 0
R1(config-if)# ospfv3 1 ipv4 area 0                 ! same link, IPv4 AF (needs IPv6 enabled)
```

Each address family runs as a **separate instance** with its own neighbours and database, so `show ospfv3 neighbor` lists them per AF. The IPv4 address family still needs IPv6 enabled on the interface, because OSPFv3 packets themselves always travel over IPv6 link-local addresses.

Verifying OSPFv3

```
R1# show ospfv3 neighbor
          OSPFv3 1 address-family ipv6 (router-id 1.1.1.1)

Neighbor ID     Pri   State           Dead Time   Interface ID    Interface
2.2.2.2           1   FULL/BDR        00:00:36    4               GigabitEthernet0/1
```

How the OSPFv3 adjacency above came up, step by step:

1. Both routers source Hellos from their **link-local** address (FE80::1 and FE80::2 here, set with `ipv6 address fe80::1 link-local` for readability; without that, a link-local address is created automatically when IPv6 is enabled) and send them to **FF02::5**.
2. Each Hello carries the sender's 32-bit RID (1.1.1.1, 2.2.2.2). If a router has no IPv4 address and no `router-id` configured, it has no RID and the process can't start. That's why the config sets one explicitly.
3. The usual OSPF rules apply: matching area, timers and stub flags, then the DR/BDR election (on Ethernet, R2 became BDR), then the database exchange.
4. The routes are installed with the neighbour's **link-local** address as the next hop, because that's the address R1 heard the neighbour speaking from.

Note the **link-local next hop** (`FE80::2`) — that's normal and correct for OSPFv3. The traditional-mode commands (`show ipv6 ospf neighbor`, `show ipv6 ospf interface`) still work for processes configured with `ipv6 router ospf`.

6.9 Verification and Troubleshooting Toolkit

Command	Answers
`show ip ospf interface brief`	Which interfaces run OSPF, in which area, cost, state (DR/BDR/P2P)
`show ip ospf interface Gi0/1`	Network type, timers, DR/BDR, passive status
`show ip ospf neighbor`	Neighbours and adjacency state
`show ip ospf database`	LSDB summary per area and LSA type
`show ip ospf border-routers`	How this router reaches each ABR and ASBR
`show ip route ospf`	What made it into the RIB (`O`, `O IA`, `O E1/E2`, `O N1/N2`)
`show ip protocols`	Router ID, areas, networks, passive interfaces, distribute-lists

Symptom	Likely cause	Prove it with
No neighbour at all	Passive interface, wrong area, ACL blocking 224.0.0.5, subnet mismatch	`show ip ospf interface`, `show ip protocols`
Neighbour stuck in **Init**	One-way hellos — ACL or multicast problem on one side	`debug ip ospf hello`
Stuck in **ExStart/Exchange**	MTU mismatch	`show interface | include MTU`
FULL, but routes missing across the link	Network-type mismatch (p2p vs broadcast)	`show ip ospf interface` on both ends

Symptom	Likely cause	Prove it with
Route present in LSDB but not in RIB	`distribute-list in`, or a better-AD route from another protocol	`show ip protocols`, `show ip route <prefix>`
Inter-area route missing everywhere	`area filter-list` or `range … not-advertise` on the ABR; area not connected to area 0	`show run | section router ospf` on ABRs
External route missing in one area	Area is stub/totally stubby (by design)	`show ip ospf` — area type
Summary not appearing	`area range` used for externals, or `summary-address` used on an ABR	Check which router type you're on

Worked Example: A Stuck Adjacency

R1 and R2 are connected on Gi0/1. Users report that nothing beyond R2 is reachable.

1. `show ip ospf neighbor` on R1 shows R2 in **EXSTART**, and it never moves on. The table points to **MTU**.
2. `show interface Gi0/1 | include MTU` shows **MTU 1500 bytes** on R1 and **MTU 1400 bytes** on R2 (someone set it for a tunnel).
3. **Why it sticks:** in EXSTART/EXCHANGE the routers send Database Description packets that list their LSAs. R1 offers packets up to 1500 bytes. R2 sees that the MTU advertised in R1's packets exceeds its own and rejects them, so the exchange never finishes.
4. **Fix:** make the MTU match on both ends (`mtu 1500` on R2's Gi0/1). The neighbour moves through EXCHANGE and LOADING to **FULL** within seconds, and the routes appear. `ip ospf mtu-ignore` would also bring it up, but it hides the real problem: large packets would still be dropped on that link.

6.10 Summary — What to Be Able to Explain

You should be able to…	Section
Name the router roles and why every area must touch area 0	6.1
Say which router generates each LSA type and how far it floods	6.2
Explain E1 vs E2, and pick the exit each one chooses	6.2
Configure stub/totally stubby/NSSA and say where each command goes	6.3
Configure multi-area OSPF with network statements or interface mode	6.4
Work out a summary in binary; summarize at an ABR vs an ASBR, and explain the Null0 route	6.5
Filter Type 3s at an ABR and explain why distribute-list in is local only	6.6
Run a DR/BDR election and explain why it doesn't preempt	6.7
Diagnose p2p/broadcast mismatches and use passive-interface correctly	6.7
Configure OSPFv3 traditionally and with address families	6.8

Chapter 7: BGP Fundamentals & eBGP

> **Blueprint:** ENCOR 3.2.c — Configure and verify eBGP between directly connected neighbors (best path selection algorithm and neighbor relationships).

BGP is the biggest new protocol of CCNP, and the one the CCNA never touched. It's also the protocol that runs the internet: every ISP, cloud provider and multi-homed enterprise uses it to tell the rest of the world which addresses it owns. ENCOR asks you to bring up **eBGP between directly connected neighbours** and to understand **how BGP picks a best path**. Chapter 28 takes it to ENARSI depth — iBGP, route reflectors, multihop, policy and IPv6.

The single most useful thing to grasp before any command: **BGP is not trying to find the fastest path. It's trying to find the path your policy prefers.**

7.1 Why BGP Exists — Routing Between Organizations

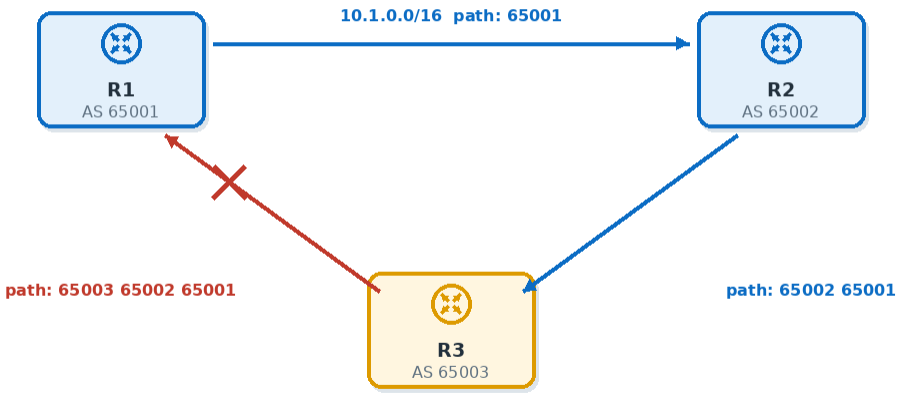
OSPF and EIGRP are **IGPs** (Interior Gateway Protocols): they run inside one organization, where everyone trusts everyone and the goal is the shortest path. Between organizations, both assumptions collapse:

- **Nobody trusts anybody.** Your ISP will not let you join its OSPF and see its internal map, and you shouldn't want to.
- **"Shortest" isn't the goal.** You prefer the ISP you pay less, an ISP prefers its paying customers over its peers, and a bank may require that its traffic never crosses certain countries. Those are **business** decisions.
- **Scale.** The global IPv4 table is around a million prefixes. SPF over that, recomputed on every change anywhere, is impossible.

BGP (Border Gateway Protocol, version 4) answers all three. It's an **EGP** — an Exterior Gateway Protocol — and a **path-vector** protocol.

Why "path vector"? Every BGP route carries the **list of autonomous systems** it has passed through — the `AS_PATH`. That list does two jobs. It's a **rough distance** (fewer ASes is usually better), and it's a **loop detector**: a router that sees its **own AS number** in an incoming path rejects the route, because accepting it would mean routing through itself.

AS_PATH loop prevention — a router rejects any path containing its own AS



R1 sees 65001 in the path → drops the update. No TTL, no hop limit — the path itself is the proof.

Story Time : Think of a passport that gets a stamp at every border it crosses. Each country you pass through adds its stamp. If a border officer opens your passport and finds their own country's stamp from this same trip, they know you've gone round in a circle and turn you back. The AS_PATH is that passport: every AS adds its number as it passes the route on, and an AS that finds its own number refuses the route. Fewer stamps usually means a more direct journey, which is why path length also works as a rough distance.

Worked example: a loop being stopped. R1 in AS 65001 advertises 198.51.100.0/24.

1. R1 sends it to AS 65002 with AS_PATH **65001**.
2. AS 65002 passes it to AS 65003 with AS_PATH **65002 65001** (each AS adds itself to the front).

3. AS 65003 also peers with AS 65001 and sends the route back with AS_PATH **65003 65002 65001**.
4. R1 sees **65001**, its own AS, in the path. It drops the route silently. A route to its own network through two other companies would be a loop.

No hop counts, no feasibility maths, no shared map — the path itself is the proof. That is why BGP needs no areas and no SPF to stay loop-free across tens of thousands of independent networks.

BGP Terms You Need First

Term	Meaning
Autonomous System (AS)	A network under one administrative policy, identified by an **AS number (ASN)**
eBGP	BGP between routers in **different** ASes — AD **20**
iBGP	BGP between routers in the **same** AS — AD **200** (Chapter 28)
Prefix / NLRI	A route BGP advertises (Network Layer Reachability Information)
Path attributes	The properties attached to each prefix (AS_PATH, NEXT_HOP, LOCAL_PREF, MED…) that policy reads and changes

Why is eBGP's AD 20 — better than OSPF — but iBGP's 200? An eBGP route came from **outside** your network, from the only router that actually knows it; nothing inside your AS can know better, so trust it over your IGP. An iBGP route came from **inside**, where your IGP already describes the topology directly; if the IGP and iBGP both know a prefix, the IGP's first-hand knowledge should win. Trust outsiders about the outside, insiders' own map about the inside.

7.2 Autonomous System Numbers

The problem: BGP has to know which organisation each route belongs to and has passed through, so every organisation needs a unique number, just as every network needs unique addresses.

Range	Type	Use
1 – 64,495	Public 2-byte	Assigned by RIRs, the Regional Internet Registries (ARIN, RIPE…)
64,496 – 64,511	Documentation (RFC 5398)	Reserved for examples in books and RFCs, like 192.0.2.0/24 for addresses
64,512 – 65,534	**Private 2-byte**	Internal use, labs, customer-to-ISP links — never seen on the internet

Range	Type	Use
65,536 – 4,199,999,999	Public 4-byte	Assigned since 2-byte numbers ran out
4,200,000,000 – 4,294,967,294	**Private 4-byte**	Internal use
0, 65,535, 4,294,967,295	Reserved	Never used
23,456	AS_TRANS	Stands in for a 4-byte ASN when talking to an old 2-byte-only router

4-byte ASNs can be written as plain numbers (**asplain**, e.g. `65538`) or as two 16-bit halves (**asdot**, e.g. `1.2`). IOS XE uses asplain by default; `bgp asnotation dot` switches the display.

Why do private ASNs exist? For the same reason private IP addresses do (CCNA guide 11.4). An enterprise that connects to a single ISP still needs an ASN to run BGP, but the outside world never needs to see it — the ISP strips the private ASN (`remove-private-as`) and advertises the customer's prefixes as part of its own AS.

For example, the customer uses private AS 65001 and its ISP uses AS 64500 (a documentation number standing in for a real public ASN). The customer advertises 198.51.100.0/24 to the ISP with AS_PATH **65001**. The ISP has `remove-private-as` on that neighbour, so when it advertises the prefix to the rest of the internet, the path is just **64500**. To the world, the customer's block simply belongs to the ISP's AS.

7.3 How a BGP Session Works

The problem: two routers, often owned by different companies, need a reliable, controlled conversation that can carry hundreds of thousands of routes and stay up for months. This section covers how that conversation starts, stays alive and fails.

Neighbours Are Configured, Not Discovered

IGPs find neighbours with multicast hellos. BGP doesn't. Every neighbour is **configured by hand** with its IP address and AS number, and BGP talks to it over **TCP port 179** — ordinary unicast TCP.

Why TCP, when OSPF and EIGRP built their own reliability? Because BGP's updates are huge (a full internet table) and travel between organizations, potentially across many hops. TCP already solves reliable, ordered, flow-controlled delivery of large data — so BGP simply uses it and spends no effort reinventing it. It's also why BGP sends updates **once** and afterwards only changes: TCP guarantees they arrived. BGP is an application that happens to carry routes.

Why configure neighbours manually? Because a BGP session is a **business relationship**. You peer with a specific ISP on a specific link under a contract; automatically forming sessions with whoever answers would be a security disaster on the internet.

Directly Connected eBGP and TTL

By default, eBGP packets are sent with **TTL 1** (time to live: the hop count after which a packet is discarded), so an eBGP neighbour must be **directly connected** — exactly ENCOR's scope. Peering between loopbacks or across another router needs `ebgp-multihop` (or `disable-connected-check`); that's Chapter 28.

Message Types

Message	Purpose
OPEN	Starts the session: BGP version, my AS, hold time, router ID, capabilities
KEEPALIVE	"Still here" — sent every keepalive interval, and to confirm an OPEN
UPDATE	Advertises new prefixes with their attributes, and/or withdraws old ones
NOTIFICATION	An error — sent just before the session is closed
ROUTE-REFRESH	Asks the neighbour to resend its updates (used by soft reset, 7.8)

Timers

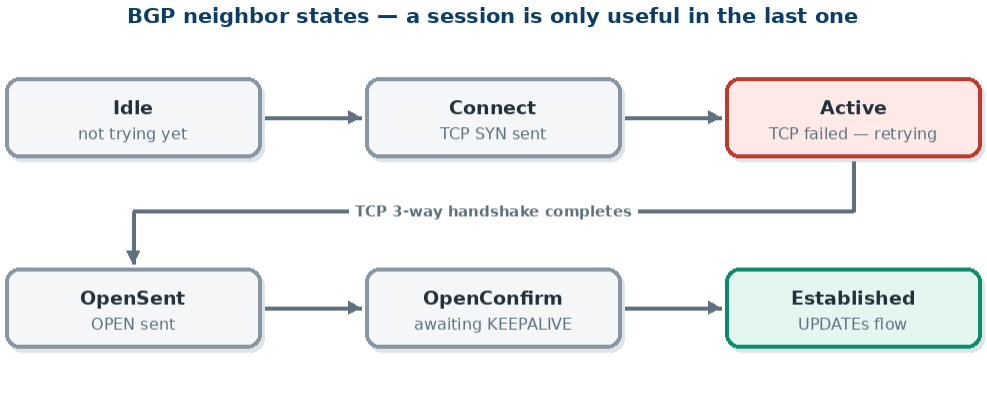
Timer	Cisco default	Notes
Keepalive	**60 s**	
Hold time	**180 s**	Neighbour declared dead if nothing heard for this long

The hold time is **negotiated**: each side proposes one in its OPEN and **both use the lower value**; the keepalive is then a third of that. A hold time of **0** disables keepalives entirely. Timers don't have to match to form a session — unlike OSPF. Change them with `timers bgp <keepalive> <hold>` under `router bgp` or per neighbour.

Why are BGP timers so slow compared with OSPF's 10/40 seconds? Because a BGP session flap is expensive: tearing it down withdraws every prefix learned from that neighbour — possibly a million — and re-establishing re-sends them all. Slow timers avoid declaring a busy neighbour dead. When fast failure detection matters, the modern answer is **BFD** (Chapter 25), not shorter BGP timers.

Neighbour States

Story Time : A BGP session starting up is like a business phone call. First you dial and the phone connects. That's the TCP handshake. Then each side introduces themselves: "This is Acme, AS 65001, my ID is 1.1.1.1, and I'll hang up if I hear nothing from you for 180 seconds." That's the OPEN. Each side checks the introduction is the company it was expecting, then says "OK, go ahead". That's the KEEPALIVE that confirms the OPEN. Then the real business begins, with UPDATEs. During long silences, each side says "still here" now and then so the other doesn't hang up.



State	What's happening	Stuck here usually means
Idle	Not trying yet (or backing off after an error)	No route to the neighbour, or repeated failures
Idle (Admin)	Administratively disabled with `neighbor X shutdown`	Intentional — `no neighbor X shutdown` re-enables it
Connect	TCP SYN sent, waiting	Usually passes quickly
Active	TCP failed; **actively retrying**	No route to the neighbour, wrong neighbour IP, ACL blocking TCP 179, wrong source address
OpenSent	TCP is up; my OPEN sent	Waiting for theirs
OpenConfirm	OPENs exchanged and accepted; waiting for KEEPALIVE	Rarely stuck here
Established	Session up — UPDATEs flow	Healthy

Walk-through: R1 (203.0.113.1, AS 65001) bringing up ISP (203.0.113.2, AS 65002).

1. **Idle → Connect.** R1 has a route to 203.0.113.2 (connected), so it opens a TCP connection to port **179**: SYN, SYN-ACK, ACK.
2. **OpenSent.** TCP is up. R1 sends its OPEN: version 4, My AS **65001**, hold time **180**, BGP identifier **1.1.1.1**, plus capabilities (route refresh, 4-byte ASN, IPv4 unicast).
3. **OpenConfirm.** The ISP's OPEN arrives saying My AS **65002**. That matches R1's `neighbor 203.0.113.2 remote-as 65002`, and the router ID (2.2.2.2) differs from R1's own, so R1 accepts it.

Both proposed 180 s, so the hold time is 180 and the keepalive 60. R1 sends a KEEPALIVE.

4. **Established.** The ISP's KEEPALIVE arrives. The routers now swap their tables in UPDATE messages, and from then on send only changes, plus a KEEPALIVE every 60 seconds.

What if R1 had typed `remote-as 65020` by mistake? At step 3 the OPEN says 65002, which isn't what R1 expects. R1 sends a NOTIFICATION (error 2/2, "bad peer AS"), closes TCP and goes back to **Idle**. A few seconds later it tries again, and fails the same way. That's the "reaches OpenSent, then resets" pattern in the trap below.

Timer negotiation, with numbers: if the ISP had `timers bgp 10 30` and R1 kept the defaults (60/180), both sides would use the lower hold time, **30 s**, with keepalives every **10 s**. The ISP has asked for fast failure detection, and R1 goes along with it.

> **Exam trap — "Active" is bad.** It sounds healthy, but it means the TCP session **failed** and BGP is retrying. A neighbour that cycles between **Idle and Active** almost always has an IP-reachability or addressing problem. One that reaches **OpenSent** and then drops back to Idle has a working TCP connection but a rejected OPEN: a wrong `remote-as` or a router-ID clash. An **MD5 password mismatch** fails earlier than that. The TCP segments themselves fail authentication (`%TCP-6-BADAUTH`), so TCP never comes up and the neighbour never leaves Idle/Active.

7.4 Configuring eBGP Between Directly Connected Neighbours

The problem: R1 must tell its ISP "send traffic for 198.51.100.0/24 to me", and learn the ISP's routes in return, without accidentally advertising anything it shouldn't.

The lab: **R1** (AS 65001, the enterprise) and **ISP** (AS 65002) share 203.0.113.0/30. R1 owns 198.51.100.0/24 and wants to advertise it.

```
R1(config)# router bgp 65001                              ! one BGP process per router - local AS
R1(config-router)# bgp router-id 1.1.1.1                  ! set it, like OSPF
R1(config-router)# bgp log-neighbor-changes               ! log every state change (default on)
R1(config-router)# neighbor 203.0.113.2 remote-as 65002 ! different AS = eBGP
R1(config-router)# neighbor 203.0.113.2 description ISP-A uplink
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# neighbor 203.0.113.2 activate     ! exchange IPv4 routes with it
R1(config-router-af)# network 198.51.100.0 mask 255.255.255.0
R1(config-router-af)# exit-address-family
R1(config)# ip route 198.51.100.0 255.255.255.0 Null0   ! anchor route - see below
```

```
ISP(config)# router bgp 65002
ISP(config-router)# bgp router-id 2.2.2.2
ISP(config-router)# neighbor 203.0.113.1 remote-as 65001
ISP(config-router)# address-family ipv4 unicast
ISP(config-router-af)# neighbor 203.0.113.1 activate
```

The BGP Router ID

Every BGP speaker needs a 32-bit **router ID**, carried in its OPEN message. It's chosen the same way as OSPF's:

1. `bgp router-id` if configured;
2. otherwise the highest IPv4 address on an **up loopback**;
3. otherwise the highest IPv4 address on an **up physical interface**.

Two peers with the **same** router ID can't form a session. On an IPv6-only router there's no IPv4 address to borrow, so `bgp router-id` is mandatory. Set it explicitly, like every other router ID in this book.

What decides eBGP vs iBGP? Nothing but the numbers: if `remote-as` differs from the local AS in `router bgp`, it's eBGP; if it's the same, it's iBGP. There is no separate command.

About `activate`. BGP is multiprotocol — one session can carry IPv4, IPv6, VPN routes and more, each in its own **address family**. On IOS XE, IPv4 unicast is activated automatically for every neighbour (`bgp default ipv4-unicast` is on by default), so the `activate` line above is redundant but harmless. Many engineers turn that default off with `no bgp default ipv4-unicast`, so that no address family is ever exchanged by accident — and then `activate` becomes mandatory. If a neighbour is Established but no routes are exchanged, check `activate` first.

Shutting Down and Securing a Neighbour

```
R1(config-router)# neighbor 192.0.2.6 shutdown            ! keep the config, drop the session
R1(config-router)# neighbor 203.0.113.2 password S3cr3tK3y ! TCP MD5 (RFC 2385) on the session
```

`neighbor shutdown` tears the session down and holds it in **Idle (Admin)** while keeping every line of its configuration: the clean way to take a peer out of service for maintenance. **`neighbor password`** adds an MD5 signature to every TCP segment of the session. Both sides must have the same password. If they don't match (or only one side has one), the receiving router drops the segments and logs:

```
%TCP-6-BADAUTH: Invalid MD5 digest from 203.0.113.2(179) to 203.0.113.1(31744) tabled
%TCP-6-BADAUTH: No MD5 digest from 203.0.113.2(179) to 203.0.113.1(31744) tabled
```

Why authenticate TCP rather than BGP itself? Because the attack BGP fears most is at the TCP layer. A forged TCP reset from anywhere on the internet tears down a session and withdraws every route. Signing every segment means a spoofed SYN or RST without the key is simply discarded. (Chapter 28.3 adds TTL security.)

The `network` Command Is Not What You Think

In OSPF and EIGRP, `network` enables the protocol on interfaces. In BGP it means something completely different:

> **BGP `network` = "advertise this prefix — if, and only if, an exactly matching route (same prefix and same mask) is already in my routing table."**

Situation	Advertised?
`network 198.51.100.0 mask 255.255.255.0`, RIB has 198.51.100.0/24	**Yes**

Situation	Advertised?
Same command, RIB has only 198.51.100.0/25 and .128/25	**No** — no exact /24
Same command, RIB has 198.51.100.0/16	**No** — mask must match exactly
`network 198.51.100.0` with **no mask**	Uses the **classful** mask (/24 for a Class C address)

Why does BGP insist on an exact match? Because advertising a prefix to the world is a promise: "send traffic for this block to me, and I can deliver it." BGP refuses to make that promise unless the router can actually route the block. That's also why the Null0 static above is standard practice: it guarantees the /24 always exists in the RIB, so the advertisement stays **stable** even if internal subnets flap. Traffic for unused parts of the block is simply dropped — the same discard-route idea OSPF uses for summaries (6.5).

Advertising a Summary — `aggregate-address`

The other way to put a prefix into BGP is to summarize routes that are **already in the BGP table**:

```
R1(config-router-af)# aggregate-address 198.51.96.0 255.255.248.0 summary-only
```

The aggregate (here a /21) is created only while **at least one** more-specific component exists in the BGP table, and R1 installs a Null0 discard route for it automatically.

Without `summary-only`, BGP advertises the aggregate **and** every component. With it, the components are **suppressed** (marked `s` in `show ip bgp`) and only the summary is sent. `as-set` keeps the component paths' AS numbers in the aggregate for loop prevention. Chapters 24.6 and 28 cover aggregation in depth.

Worked example. R1's BGP table holds 198.51.100.0/24 (from the `network` command). 198.51.96.0/21 covers 198.51.96.0 to 198.51.103.255, so the /24 is a component of it.

```
R1# show ip bgp | begin Network
     Network          Next Hop            Metric LocPrf Weight Path
 *>  198.51.96.0/21   0.0.0.0                              32768 i
 s>  198.51.100.0/24  0.0.0.0                   0          32768 i
```

The /21 is locally originated (next hop 0.0.0.0, weight 32768). The `s` on the /24 means "suppressed": it's still in R1's table, but only the /21 is sent to the ISP. If 198.51.100.0/24 left the BGP table and it was the only component, the /21 would be withdrawn too.

eBGP for IPv6

The same session machinery carries IPv6 in the **IPv6 unicast address family** (MP-BGP, multiprotocol BGP). The common design peers over IPv6 addresses for IPv6 routes:

```
R1(config)# ipv6 unicast-routing
R1(config)# ipv6 route 2001:db8:100::/48 Null0                 ! anchor route, as for IPv4
R1(config)# router bgp 65001
R1(config-router)# neighbor 2001:db8:0:12::2 remote-as 65002
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# no neighbor 2001:db8:0:12::2 activate   ! no IPv4 routes on this session
R1(config-router)# address-family ipv6 unicast
R1(config-router-af)# neighbor 2001:db8:0:12::2 activate      ! IPv6 AF is never on by default
R1(config-router-af)# network 2001:db8:100::/48               ! exact match rule applies
```

Verify with `show bgp ipv6 unicast summary` and `show bgp ipv6 unicast`. **Why the `no … activate` under IPv4?** Because `bgp default ipv4-unicast` activates IPv4 for **every** neighbour, including one addressed over IPv6. That would try to exchange IPv4 routes whose next hop is an IPv6 address, which is useless. Deactivating it (or using `no bgp default ipv4-unicast` globally) keeps each session carrying only what it should. Chapter 28.4 covers MP-BGP in depth.

7.5 Verifying the Session and the Table

`show ip bgp summary` — Is the Session Up?

```
R1# show ip bgp summary
BGP router identifier 1.1.1.1, local AS number 65001
BGP table version is 4, main routing table version 4
3 network entries using 744 bytes of memory

Neighbor        V    AS MsgRcvd MsgSent   TblVer  InQ OutQ Up/Down  State/PfxRcd
203.0.113.2     4 65002      27      25        4    0    0 00:21:43        2
192.0.2.6       4 65003      31      29        4    0    0 00:19:02        1
192.0.2.10      4 65004       0       0        1    0    0 never    Active
```

The last column is the one that matters. **A number** means the session is **Established**, and the number is how many prefixes were received. **A word** (Idle, Active…) means it is **not** established, and the word is the current state. Here ISP-A is up with 2 prefixes and ISP-B (AS 65003) is up with 1. The neighbour in AS 65004 has **never** come up and is stuck in Active, so check reachability to 192.0.2.10.

`show ip bgp` — the BGP Table

```
R1# show ip bgp
BGP table version is 4, local router ID is 1.1.1.1
Status codes: s suppressed, d damped, h history, * valid, > best, i - internal,
              r RIB-failure, S Stale, m multipath, b backup-path
Origin codes: i - IGP, e - EGP, ? - incomplete

     Network          Next Hop            Metric LocPrf Weight Path
 *>  10.20.0.0/16     203.0.113.2              0             0 65002 i
 *   10.30.0.0/16     192.0.2.6                              0 65003 65002 65030 i
 *>                   203.0.113.2                            0 65002 65030 i
 *>  198.51.100.0/24  0.0.0.0                  0         32768 i
```

Field	How to read it
`*`	**Valid** — the path is usable (next hop reachable)
`>`	**Best** — this path won and is offered to the routing table and to neighbours
`i` (left)	Learned via **iBGP**
`r`	**RIB-failure** — BGP chose it, but a route with lower AD already owns that prefix in the RIB
Next Hop `0.0.0.0`	**Locally originated** by this router (`network` or redistribution)

Field	How to read it
Weight `32768`	Default weight for locally originated routes (others get 0)
Path	The AS_PATH, **nearest AS first**, origin AS last
`i` / `e` / `?` (right)	**Origin**: `i` = `network` command, `e` = EGP (historic), `?` = redistributed

10.30.0.0/16 has two paths: the line without a prefix is a second path to the same network. The one via 203.0.113.2 wins — its AS_PATH is shorter (2 ASes vs 3). Note that only the **best** path is advertised to neighbours, and only the best is offered to the routing table.

`show ip bgp neighbors` — the Session in Detail

When a session misbehaves, this is the command that explains why:

```
R1# show ip bgp neighbors 203.0.113.2
BGP neighbor is 203.0.113.2,  remote AS 65002, external link
 Description: ISP-A uplink
  BGP version 4, remote router ID 2.2.2.2
  BGP state = Established, up for 00:21:43
  Last read 00:00:12, last write 00:00:41, hold time is 180, keepalive interval is 60 seconds
  Neighbor capabilities:
    Route refresh: advertised and received(new)
    Four-octets ASN Capability: advertised and received
    Address family IPv4 Unicast: advertised and received
 ...
  Connections established 1; dropped 0
  Last reset never
 ...
Local host: 203.0.113.1, Local port: 179
Foreign host: 203.0.113.2, Foreign port: 26743
```

Line	What it tells you
`remote AS 65002, external link`	The AS you configured, and whether this is eBGP (`external`) or iBGP (`internal`)
`remote router ID`	The neighbour's BGP router ID: check for duplicates here
`BGP state = …`	The current state (7.3) and how long it's been there
`hold time is 180, keepalive interval is 60`	The **negotiated** timers, the lower of the two proposals
`Neighbor capabilities`	What both sides agreed in their OPENs: route refresh (needed for soft reset), 4-byte ASNs, each address family
`Connections established / dropped` and `Last reset`	How often the session has flapped and the **reason** for the last reset (e.g. hold timer expired, or a NOTIFICATION received)
`Local host … Local port: 179`	Which side **accepted** the TCP connection. Local port 179 means the neighbour initiated; a random local port means you did

More views you'll use constantly:

Command	Shows
`show ip bgp 10.30.0.0/16`	Every path for one prefix, all attributes, and which is best (and why it's valid)
`show ip bgp neighbors 203.0.113.2`	Session detail: state, timers, capabilities, last error/NOTIFICATION
`show ip bgp neighbors 203.0.113.2 advertised-routes`	What you're sending that neighbour
`show ip bgp neighbors 203.0.113.2 routes`	What you accepted from it (after inbound policy)
`show ip route bgp`	What actually made it into the routing table (`B`)

7.6 Path Attributes

The problem: BGP needs more than "how far away is it?" to apply business policy. Each route therefore carries a set of labels, the **path attributes**, that routers read, compare and change.

Everything BGP decides, it decides by reading **path attributes**. They fall into four categories, and the categories decide whether a router must understand and forward them:

Category	Must every BGP router recognise it?	Must it be in every UPDATE?	Passed on to other ASes?	Examples
Well-known mandatory	Yes	**Yes**	Yes	`ORIGIN`, `AS_PATH`, `NEXT_HOP`
Well-known discretionary	Yes	No	Yes (LOCAL_PREF only inside the AS)	`LOCAL_PREF`, `ATOMIC_AGGREGATE`
Optional transitive	No	No	**Yes**, even if not understood	`COMMUNITY`, `AGGREGATOR`
Optional non-transitive	No	No	**No**	`MED` (`MULTI_EXIT_DISC`), `ORIGINATOR_ID`, `CLUSTER_LIST`

The ones that matter for path selection at ENCOR level:

Attribute	Meaning	Scope	Preferred
Weight	Cisco-only, **not a real attribute** — never leaves the router	This router only	**Highest** (default 0; 32768 if local)
LOCAL_PREF	"How much this AS likes this exit"	Shared across the **whole AS** via iBGP, never sent to eBGP peers	**Highest** (default 100)

Attribute	Meaning	Scope	Preferred
AS_PATH	ASes the route crossed	Global	**Shortest**
ORIGIN	How the route entered BGP	Global	**i < e < ?**
MED	"Which of my links you should use to reach me" — a hint to a neighbouring AS	Sent to one neighbouring AS, not beyond	**Lowest** (missing treated as 0)
NEXT_HOP	The IP to forward to	Changed by eBGP; kept by iBGP (Chapter 28)	Must be **reachable**

NEXT_HOP on eBGP

When a router advertises a route to an **eBGP** neighbour, it sets NEXT_HOP to **its own address on that session**, the IP the neighbour peers with. That's why R1's routes from ISP-A show next hop 203.0.113.2. For a directly connected neighbour that address sits on a connected subnet, so the next hop is always reachable and every path is valid at step 0 of the algorithm.

Why does eBGP rewrite the next hop? Because the receiving AS can't reach anything deep inside the advertising AS. The only address it's guaranteed to reach is the one on the shared link. iBGP does **not** rewrite it by default, and that's the source of the most common iBGP problem: routes in the table but not marked best because the next hop is unreachable. Chapter 28.2 covers that problem and its fix, `next-hop-self`.

7.7 The Best-Path Algorithm

The problem: a router that hears the same prefix from two ISPs must pick exactly one path to use and to advertise onwards. It needs a fixed, predictable order of questions, so that every router given the same inputs makes the same choice.

When BGP holds several paths to one prefix, it walks down this list and **stops at the first step that produces a winner**. Paths must first be **valid**: the next hop must be reachable (and, for iBGP with synchronization enabled — off by default since IOS 12.2(8)T — synchronized).

Step	Prefer	Notes
0	Next hop **reachable**	Unreachable next hop = not a candidate at all
1	Highest **Weight**	Cisco-only; local to the router
2	Highest **LOCAL_PREF**	AS-wide outbound policy
3	**Locally originated**	`network`, `aggregate-address` or redistribution on this router
4	Highest **AIGP**	Accumulated IGP metric attribute. Rarely used; ignored unless configured

Step	Prefer	Notes
5	Shortest **AS_PATH**	Each prepended AS counts; `AS_SET` counts as 1
6	Lowest **ORIGIN**	IGP `i` < EGP `e` < incomplete `?`
7	Lowest **MED**	Compared only between paths from the **same neighbouring AS** (by default)
8	**eBGP over iBGP**	External paths are preferred
9	Lowest **IGP metric** to the next hop	"The closest exit"
10	Multipath check	If `maximum-paths` is set, equal paths are installed together
11	**Oldest** eBGP path	Stability — don't flip between equal paths
12	Lowest **router ID** of the advertising neighbour	
13	Shortest **cluster list**	Route-reflector environments
14	Lowest **neighbour address**	Final tie-breaker

Mnemonic: "N WLLA OMNI" — **N**ext hop reachable, **W**eight, **L**ocal pref, **L**ocally originated, **A**S path (with AIGP folded in), **O**rigin, **M**ED, **N**eighbour type (eBGP over iBGP), **I**GP metric. After those, the tie-breakers run oldest → router ID → cluster list → neighbour address.

Why is the order "policy first, distance second"? Look at what comes first: weight and local preference — both values **you** set. AS-path length, the nearest thing BGP has to a distance, is only step 5. That ordering encodes BGP's purpose: business intent overrides topology. If you've expressed a preference, BGP obeys it; only when you haven't does it fall back to "fewer ASes is probably better."

Story Time : Choosing a flight. First you apply your own rules: "I always fly with my loyalty airline" (that's **weight** for you alone, or **local preference** for your whole family). Only if your rules don't decide it do you look at the trip itself: fewest stopovers (**AS_PATH**). After that come smaller details: what the airline itself suggests (**MED**), and finally pure tie-breakers like "book the one I found first" (**oldest path**). The order matters: a loyalty airline with two stopovers beats a direct flight on another airline, because your rule came first.

Worked Example: Walking the List

R1 has two paths to 10.30.0.0/16 (the `show ip bgp` output in 7.5):

Step	Via 192.0.2.6 (ISP-B)	Via 203.0.113.2 (ISP-A)	Result
0 Next hop reachable	yes (connected)	yes (connected)	both valid
1 Weight	0	0	tie

Step	Via 192.0.2.6 (ISP-B)	Via 203.0.113.2 (ISP-A)	Result
2 LOCAL_PREF	100 (default)	100 (default)	tie
3 Locally originated	no	no	tie
4 AIGP	not set	not set	tie
5 AS_PATH length	3 (65003 65002 65030)	**2** (65002 65030)	**ISP-A wins**

The algorithm stops at step 5. Nothing after it is looked at.

Now apply policy: `neighbor 192.0.2.6 weight 200`, followed by `clear ip bgp 192.0.2.6 soft in`. At step 1, ISP-B's path has weight 200 against 0, so **ISP-B wins at step 1** even though its path is longer. That's "policy first, distance second" in one line of config.

A tie that goes all the way down: two paths to 10.40.0.0/16, one from each ISP, each with AS_PATH length 2, origin `i`, no MED.

1. Steps 0–6 tie (weight 0, LOCAL_PREF 100, same length, same origin).
2. Step 7: the paths come from different ASes (65002 and 65003), so MED isn't compared.
3. Step 8: both are eBGP. Tie.
4. Step 9: both next hops are directly connected, so the IGP metric is 0 for both. Tie.
5. Step 10: `maximum-paths` isn't set, so no multipath.
6. Step 11: the path that has been up **longest** wins. If ISP-A's path is older, it stays best, and a new ISP-B path arriving later doesn't cause a change. That's why "oldest" exists: it stops the best path flipping back and forth every time an equal path is re-learned.

Why is MED compared only between paths from the same AS? Because MED is a neighbour's **private suggestion** about its own links ("enter me via the New York link, not Chicago"). AS 65002's MED values are on AS 65002's scale; comparing them with AS 65003's MEDs would be comparing two different companies' internal numbering. (`bgp always-compare-med` overrides this — use it only if every neighbour uses a consistent scheme.)

7.8 Influencing the Path — A Preview of Chapter 28

The problem: with two ISPs, you'll want control over which link carries traffic leaving your network, and you'll want to influence which link the internet uses to send traffic to you. Those are two different problems, solved with different attributes.

The key mental model is **direction**. Which attribute you touch depends on whether you're steering traffic **leaving** your AS or **arriving** at it:

Goal	Attribute	Set on	Applied to updates
Outbound traffic — which exit this router uses	Weight	This router	Inbound (as routes arrive)

Goal	Attribute	Set on	Applied to updates
Outbound traffic — which exit the whole AS uses	LOCAL_PREF	Edge routers	Inbound
Inbound traffic — make a path look worse to others	AS-path prepend	Edge routers	**Outbound** (as you advertise)
Inbound traffic — suggest an entry link to one neighbour AS	MED	Edge routers	**Outbound**

Why is "inbound traffic" controlled by "outbound" updates? Because routes and traffic flow in **opposite directions**. To change how the world sends traffic **to** you, you change what you **tell** the world. And it's only a request — other ASes apply their own policy first, so prepending and MED influence inbound traffic but can't force it. Outbound traffic, by contrast, is entirely yours to decide.

Examples

```
! Prefer ISP-A for everything this router sends out (weight - local only)
R1(config)# router bgp 65001
R1(config-router)# neighbor 203.0.113.2 weight 200

! Make the whole AS prefer ISP-A (local preference - shared via iBGP)
R1(config)# route-map FROM-ISP-A permit 10
R1(config-route-map)# set local-preference 200
R1(config)# router bgp 65001
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# neighbor 203.0.113.2 route-map FROM-ISP-A in

! Make the world prefer the OTHER link to reach us: prepend toward ISP-B
R1(config)# route-map TO-ISP-B permit 10
R1(config-route-map)# set as-path prepend 65001 65001 65001
R1(config)# router bgp 65001
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# neighbor 192.0.2.6 route-map TO-ISP-B out

! Suggest an entry link to a neighbour AS with MED (lower = preferred)
R1(config)# route-map TO-ISP-A permit 10
R1(config-route-map)# set metric 50
R1(config)# router bgp 65001
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# neighbor 203.0.113.2 route-map TO-ISP-A out
```

Worked Example: LOCAL_PREF Across Two Edge Routers

AS 65001 has two edge routers. R1 connects to ISP-A and R2 to ISP-B, and R1 and R2 are iBGP neighbours of each other. Both ISPs advertise 10.20.0.0/16. R1 applies `FROM-ISP-A` (LOCAL_PREF 200) as above.

1. R1 receives 10.20.0.0/16 from ISP-A and sets LOCAL_PREF **200** on it. It's R1's best path, and R1 advertises it to R2 over iBGP with the LOCAL_PREF attached.
2. R2 now has two paths: its own eBGP path from ISP-B (LOCAL_PREF **100**) and the iBGP path via R1 (LOCAL_PREF **200**). This assumes R2 can reach the iBGP path's next hop, normally

because R1 uses `next-hop-self` (Chapter 28.2); otherwise the path fails step 0.

3. Steps 0–1 tie. At **step 2**, 200 beats 100, so R2 picks **the path via R1**. It never gets as far as step 8 (eBGP over iBGP), which on its own would have favoured its own ISP-B link.

4. Result: traffic from the whole AS leaves through ISP-A, and ISP-B is the backup. That's why LOCAL_PREF, not weight, is the tool for AS-wide exit policy. Weight on R1 would have changed nothing on R2.

Worked Example: What Prepending Actually Does

R1 advertises 198.51.100.0/24 to both ISPs and applies `TO-ISP-B` (three extra prepends) toward ISP-B.

1. ISP-A receives AS_PATH **65001** (one entry: R1 adds its AS once when sending over eBGP).

2. ISP-B receives AS_PATH **65001 65001 65001 65001** (the normal entry plus three prepends).

3. A distant AS 65030 that hears both via its own upstreams sees something like `65002 65001` (length 2) and `65003 65001 65001 65001 65001` (length 5). Assuming its policy doesn't decide first, step 5 picks the path via ISP-A.

4. **The catch:** ISP-B itself may still send traffic straight down its link to you. ISPs commonly set a higher LOCAL_PREF on customer routes than on routes from other providers, and step 2 comes before step 5. That's why prepending influences inbound traffic and doesn't control it.

After changing policy, it doesn't apply to routes already exchanged. BGP doesn't re-evaluate history; you must ask for a refresh:

```
R1# clear ip bgp 203.0.113.2 soft in     ! re-apply inbound policy (route refresh)
R1# clear ip bgp 203.0.113.2 soft out    ! resend our updates through outbound policy
```

Never use a plain `clear ip bgp *` on a production router — it **tears down every session**, withdrawing every route. Soft clears re-run policy; hard clears restart the internet connection.

Why prepend your own AS rather than some other number? Because prepending another AS's number would make that AS **reject** the route (loop prevention — 7.1), and it would be lying about the path. Repeating your own ASN makes the path longer without changing who's on it.

7.9 Troubleshooting eBGP

Symptom	Likely cause	Prove it with
Neighbour stuck in **Active** / **Idle**	No route to neighbour IP, wrong neighbour address, ACL blocking TCP 179	`ping 203.0.113.2`, `show ip bgp neighbors … | include state`, `show access-lists`
Session reaches OpenSent, then resets	`remote-as` mismatch (log: `%BGP-3-NOTIFICATION: sent to neighbor … 2/2 (peer in wrong AS)`), or duplicate router ID	`show ip bgp neighbors … | include Last reset`, `debug ip bgp`
Never leaves Idle/Active; neighbour reachable by ping	MD5 password mismatch, or password on one side only	Log `%TCP-6-BADAUTH` (Invalid / No MD5 digest); compare `neighbor … password` on both sides
Shows `Idle (Admin)`	`neighbor … shutdown` configured	`show run | include neighbor`
eBGP to a loopback never comes up	TTL 1 default — not directly connected	Configure `ebgp-multihop` and `update-source` (Chapter 28)
Established, 0 prefixes	Neighbour not `activate`d in the AF, nothing advertised, inbound filter	`show ip bgp summary`, `… advertised-routes` on the peer
Your prefix not advertised	`network` mask doesn't exactly match a RIB route	`show ip route 198.51.100.0 255.255.255.0`
Prefix in BGP table, not `>`	Next hop unreachable, or another path won	`show ip bgp <prefix>`
Best path marked `r`	A route with lower AD (often a static or IGP) already owns the prefix	`show ip route <prefix>`
Policy change has no effect	Routes not refreshed	`clear ip bgp <nbr> soft in/out`

Worked Example: A Neighbour Stuck in Active

`show ip bgp summary` on R1 shows `192.0.2.10 4 65004 0 0 1 0 0 never Active`.

1. **Active means TCP keeps failing**, so start at Layer 3: `ping 192.0.2.10` from R1. It fails.
2. `show ip route 192.0.2.10` returns `% Network not in table`. R1's interface toward AS 65004 is configured as 192.0.2.13/30, so the neighbour is really **192.0.2.14**. The `neighbor` statement has a typo.
3. Fix: `no neighbor 192.0.2.10`, then configure `neighbor 192.0.2.14 remote-as 65004` (and `activate` it if `no bgp default ipv4-unicast` is set).
4. Within seconds the state column moves through Connect/OpenSent to a **number**: the count of prefixes received. Established.

Had the ping worked, the next suspects would be an ACL blocking TCP 179 (`show access-lists` for hits) and, if the session reached OpenSent before failing, the `remote-as` or router ID (step 3 of the walk-through in 7.3).

7.10 Summary — What to Be Able to Explain

You should be able to…	Section
Explain why BGP is path vector and how AS_PATH prevents loops	7.1
Tell eBGP from iBGP, and explain AD 20 vs 200	7.1, 7.4
Recognise private and 4-byte ASNs	7.2
Describe the session: TCP 179, message types, timers, TTL 1	7.3
Walk the state machine and diagnose Active/Idle	7.3, 7.9
Explain how the hold time is negotiated	7.3
Configure eBGP with address families and the exact-match `network` rule	7.4
Read `show ip bgp summary` and `show ip bgp`	7.5
Classify attributes (well-known mandatory etc.)	7.6
Recite the best-path order and explain why policy comes first	7.7
Walk two paths through the algorithm and name the deciding step	7.7
Pick the right attribute for inbound vs outbound traffic	7.8

Chapter 8: Time & Translation — NTP, PTP, NAT/PAT

> **Blueprint:** ENCOR 3.3.a — Interpret network time protocol configurations such as NTP and PTP; ENCOR 3.3.b — Configure NAT/PAT

This chapter pairs two services that look unrelated but share something important. Both are invisible when they work and confusing when they break. A wrong clock does nothing dramatic. It just makes every log you read slightly untrustworthy. A broken NAT rule shows up as "the internet is down for some users", which sounds like a routing problem until you look at the translation table. CCNA introduced both (CCNA guide 17.3 and 17.4). CCNP wants you to read the configurations fluently, understand the precision ladder from NTP to PTP, and know NAT's order of operations well enough to predict what an ACL or a route will see.

8.1 Why Time Is Infrastructure, Not a Nicety

The CCNA guide gave three reasons to synchronize clocks: log correlation, certificates, and time-based features. At CCNP level, add the ones that bite in production:

- **AAA and key rotation.** TACACS+/RADIUS accounting records, Kerberos tickets and key chains with `accept-lifetime` and `send-lifetime` all assume the device knows what time it is. A router whose clock drifted to 1993 after a reload will reject a key chain that "hasn't started yet", and an OSPF or EIGRP adjacency that uses it quietly drops.
- **Assurance data.** Flexible NetFlow records, IP SLA results and Catalyst Center telemetry (Chapters 15–16) are all time series. Unsynchronized timestamps make a latency spike on one device look as if it came after the outage it actually caused.
- **Distributed applications.** Financial trading, industrial control, cellular radio and broadcast video need clocks that agree to the **microsecond or better**. NTP can't deliver that, which is the whole reason PTP exists (8.4).

Story Time : Picture three witnesses to a car crash. One says it happened at 2:04, one at 2:11, one at 1:58, because each wears a watch set differently. The crash is real, but nobody can rebuild the order of events: did the light turn red before or after the car entered the junction? Router logs are those witnesses. If R1 says "interface down 14:02:07" and R2 says "OSPF neighbor lost 14:01:55", you'd conclude R2 noticed the failure before it happened, which is nonsense. Synchronize the watches and the story lines up.

Worked example — the key chain that "isn't valid yet". R1 and R2 run EIGRP with a key chain whose key 1 has `accept-lifetime 00:00:00 Jan 1 2026 infinite`. R2 reloads, has no NTP, and its software clock starts at the factory default (a date years in the past). Here is what happens:

1. R2 boots and checks key 1. From R2's point of view the key's start date is in the future, so the key is not valid.
2. R2 has no valid key to sign or check EIGRP packets with.
3. R1's hellos fail authentication on R2, so the adjacency never forms, and nothing in the config looks wrong.
4. `show clock` on R2 shows the wrong year. Configure NTP, and R2 sets its clock, the key becomes valid, and the neighbor comes up.

A network without synchronized time can still forward packets. It just can't tell you the truth about what happened.

8.2 NTP In Depth

The problem NTP solves: every router has a cheap crystal oscillator that drifts by seconds per day, and each one drifts differently. **NTP (Network Time Protocol)** keeps them all close to one shared, trustworthy time by regularly asking better clocks "what time is it?" and correcting gently.

NTP runs over **UDP 123**. Current IOS XE speaks **NTPv4**, which adds IPv6 support and remains backward-compatible with NTPv3. Clocks are organized by **stratum**, meaning how many hops a clock is from a reference source:

Story Time : Think of the town hall clock as stratum 0, the "truth". The shopkeeper across the street sets her watch by looking straight at it (stratum 1). Her customer sets his watch from hers (stratum 2), and his kids copy his (stratum 3). Each copy adds a little error, so a lower number means you're closer to the truth. Stratum 16 is someone who has never looked at any clock: their watch might be right, but nobody should copy it. Where the analogy breaks: a real NTP client doesn't just copy the number it's told. It also measures how long the question and answer took to travel and corrects for it (next walk-through).

Stratum	Meaning
0	The reference itself: GPS receiver, atomic clock. Never on the network directly
1	A server attached to a stratum-0 source
2–15	Each hop away adds one: syncing to a stratum-3 server makes you stratum 4
16	**Unsynchronized.** Nobody should believe this clock

Worked example — stratum down a campus. A GPS receiver (stratum 0) feeds a public time server at 192.0.2.10 (stratum 1). CORE1 syncs to it and becomes **stratum 2**. The access switches sync to CORE1 and become **stratum 3**. If a switch shows `stratum 4` instead, it is following something one step further away than you designed, for example CORE1 lost its upstream and is now following its peer CORE2, which makes CORE1 stratum 3 and the switch stratum 4.

How One NTP Poll Works

1. The client (CORE1) sends a request and notes its own send time, **T1**.
2. The server (192.0.2.10) stamps when the request arrived, **T2**, and when its reply left, **T3**.
3. The client stamps when the reply arrived, **T4**.
4. Round-trip delay = (T4 − T1) − (T3 − T2): the total time minus the time the server spent thinking.
5. Offset = [(T2 − T1) + (T3 − T4)] ÷ 2: how far the client's clock is from the server's, assuming the trip out took as long as the trip back.
6. The client repeats this every poll interval, filters out bad samples, and nudges its clock toward the server. Small errors are corrected by speeding up or slowing down the clock (slewing) rather than jumping, so log timestamps never run backwards.

Worked example. CORE1's clock is 5 ms ahead of the server. The network takes 10 ms each way and the server takes 1 ms to answer. In milliseconds, using each clock's own reading: T1 = 1000 (CORE1), T2 = 1005 (server: real send time 995 + 10 ms travel), T3 = 1006, T4 = 1021 (CORE1: real arrival 1016 + 5 ms of error).

- Delay = (1021 − 1000) − (1006 − 1005) = 21 − 1 = **20 ms** (10 out + 10 back)
- Offset = [(1005 − 1000) + (1006 − 1021)] ÷ 2 = (5 − 15) ÷ 2 = **−5 ms**: CORE1 must move its clock back 5 ms

That `offset` and `delay` are exactly the columns you read in `show ntp associations` below.

The Three Relationships

Command	Relationship	Who adjusts whom
`ntp server X`	**Client/server**	I adjust to X; X never adjusts to me
`ntp peer X`	**Symmetric active**	We may adjust to each other. Used between equals, like two core routers that each have upstream servers
`ntp master [stratum]`	**Authoritative source**	I serve my own internal clock (default **stratum 8**)
`ntp broadcast` / `ntp broadcast client`	**Broadcast**	The server announces the time on a segment and clients just listen. Rare today, because it can't be authenticated per client

Why would you ever use `ntp master` when real time sources exist? For an isolated lab or an air-gapped network, agreement matters more than accuracy. If every device follows one router's clock, logs still correlate, even if that clock is two minutes off the real world. The deliberately mediocre default of stratum 8 means that any real upstream source (stratum 1–4) wins as soon as one becomes reachable. A confident-but-humble clock: use me only if nothing better shows up.

Why `ntp peer` between core routers instead of making one the server? It removes the single point of failure. Give each core router its own upstream server and peer them. If CORE1 loses its upstream, it can follow CORE2 rather than drifting, and the reverse is also true.

Try It — A Realistic Enterprise NTP Config

```
! --- CORE1: client of two external sources, peer of CORE2, server to the campus
clock timezone EST -5 0
clock summer-time EDT recurring
ntp authentication-key 1 md5 S3cretKey1                ! newer IOS XE also offers SHA
ntp authenticate                                       ! require auth on ALL sources
ntp trusted-key 1                                      ! key 1 is acceptable for sync
ntp source Loopback0                                   ! stable source address
ntp server 192.0.2.10 key 1 prefer                     ! preferred upstream
ntp server 192.0.2.11 key 1
ntp peer 10.255.0.2 key 1                              ! CORE2
ntp update-calendar                                    ! write time to hardware clock
ntp access-group peer 10                               ! who may sync with me
ntp access-group serve-only 20                         ! who may only ask me the time
!
access-list 10 permit 192.0.2.10                       ! upstream servers MUST be here
access-list 10 permit 192.0.2.11
access-list 10 permit 10.255.0.2                       ! CORE2 (peer)
access-list 20 permit 10.0.0.0 0.255.255.255           ! campus clients
!
! --- Access switch: just a client of the cores
ntp server 10.255.0.1
ntp server 10.255.0.2
```

Read the authentication lines as a three-part chain, because all three must be present:

1. `ntp authentication-key` defines a key.

2. `ntp trusted-key` marks it as acceptable for synchronization.
3. `ntp authenticate` turns enforcement on. Without it, the keys exist but unauthenticated sources are still accepted.

Then `key 1` on the `ntp server` line tells the client which key to sign its requests with. **Exam trap:** forgetting `ntp trusted-key` is the classic broken config. The key is defined and authentication is on, but no key is trusted, so the client refuses every server and never syncs.

Why does NTP authentication matter at all? Because whoever controls your clock controls certificate validity, log timelines and key lifetimes. A rogue NTP server that shifts every router forward a year can expire every certificate in the network at once. Authentication proves the time came from a source you chose.

NTP Access Control — the Four Access-Group Types

`ntp access-group` filters NTP itself, separately from any interface ACL. There are four types, from most to least permissive:

Type	Allows the matched hosts to…
`peer`	Request time, send control queries, **and synchronize this router to them**
`serve`	Request time and send control queries, but **not** sync this router
`serve-only`	Request time only
`query-only`	Send control queries only (`ntpq`-style); no time service

When a packet arrives, IOS checks the types in that order and the **first type whose ACL permits the source wins**. A source that matches none of the configured ACLs is denied.

Walk-through with the config above (`peer` ACL 10, `serve-only` ACL 20):

1. A reply arrives from upstream 192.0.2.10. IOS checks `peer` first: ACL 10 permits it, so CORE1 may sync to it.
2. A time request arrives from access switch 10.20.4.2. `peer` ACL 10 doesn't match. `serve` and `query-only` aren't configured. `serve-only` ACL 20 matches 10.0.0.0/8, so CORE1 answers the request, but it will never adjust its own clock to this switch.
3. A packet arrives from 198.51.100.66 on the internet. It matches no configured ACL, so it's dropped.

Exam trap — the upstream server that stops syncing: the moment you configure any `ntp access-group`, every source must be permitted by a type that allows what it needs. Your own upstream servers send replies that sync you, so they must be in the **`peer`** ACL. Add `ntp access-group serve-only 20` for clients but forget to list 192.0.2.10 in ACL 10, and the router silently stops synchronizing. `show ntp associations` shows the server with reach `0` and no `*`, even though `ping 192.0.2.10` works. Access control on a time server must allow both directions: who may ask me, and whom I may believe.

Reading `show ntp associations`

```
CORE1# show ntp associations

  address         ref clock       st   when   poll reach  delay  offset   disp
*~192.0.2.10      .GPS.            1     37     64   377  12.41   -0.212  1.903
+~192.0.2.11      .PPS.            1     12     64   377  14.02    0.388  2.110
 ~10.255.0.2      192.0.2.11       2     50     64   377   0.81    0.501  1.441
 * sys.peer, # selected, + candidate, - outlyer, x falseticker, ~ configured
```

Column / symbol	How to read it
`*`	The **sys.peer**: the source I'm actually synchronized to
`+`	**Candidate**: acceptable, used in the combined estimate
`-`	**Outlier**: discarded by the selection algorithm
`x`	**Falseticker**: disagrees with the majority. NTP thinks this clock is lying
`~`	Configured statically (as opposed to learned dynamically)
`st`	Stratum of that source. Mine will be this + 1
`poll`	Seconds between polls (grows from 64 toward 1024 as the clock stabilizes)
`reach`	**Octal** shift register of the last 8 polls. `377` = all 8 answered; `0` = nothing ever came back
`offset`	How far my clock is from theirs, in **milliseconds**

Worked example — reading `reach` as it fills up. `reach` is an 8-bit register shown in octal. Every poll shifts it left one bit and adds a 1 if a reply came back or a 0 if not. Right after you configure a server, it climbs like this, one poll at a time:

Poll	Bits	
1	00000001	`1`
2	00000011	`3`
3	00000111	`7`
4	00001111	`17`
8	11111111	`377`

Now one reply gets lost: the next poll shifts in a 0, giving 11111110 = `376`. As further good replies arrive, that 0 slides left (`375`, `373`, `367` …) until after eight polls it drops off the end and you are back to `377`. So **`377` = healthy, `0` = never heard back, and anything in between = recent loss**, and you can tell how recent from where the 0 sits.

Why can NTP call a server a "falseticker"? NTP doesn't trust any single source. It compares all of them and assumes the truth lies where most of them agree. A clock that disagrees with the majority gets an `x`. This is why best practice is **at least three** (ideally four) upstream servers: with two that disagree, NTP can't tell which one is lying.

Worked example — outvoting a liar. CORE1 has three servers. Two report times within 1 ms of each other. The third's GPS antenna failed and it is 900 ms off. NTP builds a range of plausible time for each server, finds the range where most of them overlap (here the two that agree), and marks the odd one out `x`. CORE1 keeps synchronizing to one of the good two (`*`) and uses the other as a `+` candidate. With only two servers, one 900 ms off, there is no majority: NTP can't decide which one is wrong.

```
CORE1# show ntp status
Clock is synchronized, stratum 2, reference is 192.0.2.10
nominal freq is 250.0000 Hz, actual freq is 249.9998 Hz, precision is 2**10
...
```

`Clock is unsynchronized, stratum 16` means NTP hasn't settled on a source yet. A fresh sync can take **several minutes**, because NTP needs a few polls before it trusts a source. Don't troubleshoot a config you entered 30 seconds ago.

NTP Troubleshooting

Symptom	Likely cause	Command that proves it
`reach 0` for a server	No reply: routing, ACL blocking UDP 123, wrong source interface	`show ntp associations`, `ping 192.0.2.10 source Loopback0`
Reach is `377` but never `*`	Auth failure (key mismatch, missing `trusted-key`) or server unsynchronized (stratum 16)	`show ntp associations detail` ("authenticated"/"insane"), `debug ntp packets`
Synchronized but logs show wrong hour	Time zone/summer-time not set. NTP always carries **UTC**	`show clock detail`, `show run | include clock`
Server stopped syncing right after adding `ntp access-group`	Upstream server not permitted in the **`peer`** ACL	`show run | include ntp access-group`, then the ACL; `show ntp associations`
Every source shows `x` or `-`	Sources disagree badly with each other	Compare offsets; remove the bad source
Logs have no real timestamps	`service timestamps` not configured	`service timestamps log datetime msec localtime show-timezone`

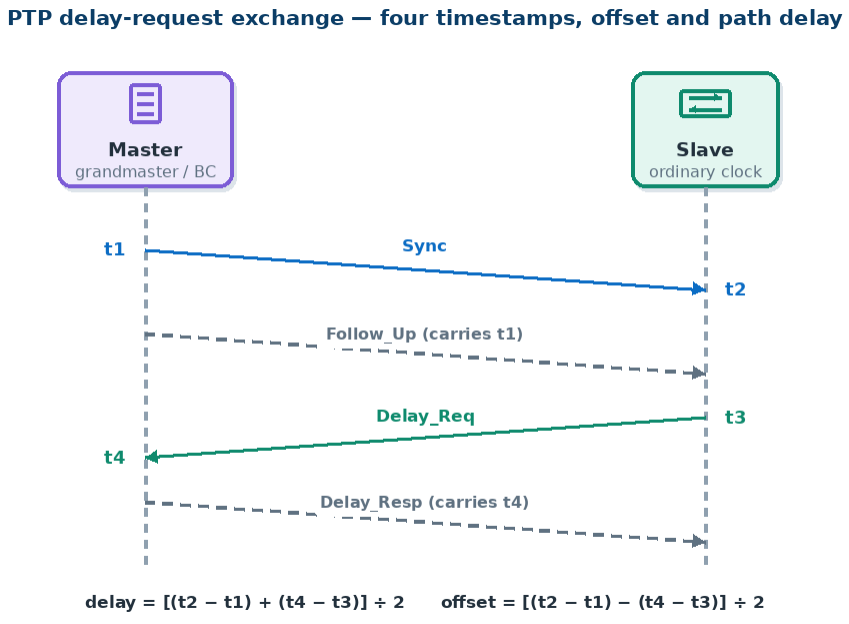
8.3 How NTP and PTP Differ — the Precision Ladder

The problem: NTP is good enough for logs and certificates, where a few milliseconds don't matter. But a robot arm on a factory line, a 5G radio, or a power-grid protection relay needs clocks that agree to a **millionth of a second (a microsecond, µs)** or better. **PTP (Precision Time Protocol, IEEE 1588)** exists for those jobs. It's the same basic idea (exchange timestamps, compute offset) done far more precisely.

Feature	NTP	PTP (IEEE 1588)
Typical accuracy	**Milliseconds** across a WAN, sub-ms on a LAN	**Sub-microsecond**, down to tens of nanoseconds
Where timestamps are taken	In software, by the operating system	In **hardware**, at the physical port, as the packet crosses it
Transport	UDP 123	UDP **319** (event) / **320** (general) to 224.0.1.129, or raw Ethernet (EtherType **0x88F7**)
Hierarchy	Stratum levels, client chooses	**Best Master Clock Algorithm** elects one grandmaster
Network role	Devices are clients and servers	Switches can **actively correct** timing as packets pass
Used for	Logs, certs, AAA: everything	Industrial automation, power utilities, 5G radio, broadcast video (AES67/SMPTE 2110), trading

Why can't NTP just try harder and reach microseconds? Because its error comes mostly from **variable delay it can't see**. NTP timestamps a packet when software handles it, which happens after an unpredictable wait in interrupt handling and output queues. NTP then assumes the path delay is the same in both directions, and every queue along the way breaks that assumption. PTP attacks both problems: it timestamps in **hardware at the wire**, and it lets the **switches in the path** measure and remove their own delay (8.4). NTP estimates across a fog; PTP clears the fog hop by hop.

8.4 PTP — Precision Time Protocol



The Four-Message Exchange

PTP calculates offset and delay from four timestamps:

Step	Message	Timestamp
1	Master sends **Sync**	Master records **t1** at the moment it leaves
2	Master sends **Follow_Up** carrying t1	(two-step clocks; one-step clocks embed t1 in Sync itself)
3	Slave receives Sync	Slave records **t2**
4	Slave sends **Delay_Req**	Slave records **t3**
5	Master replies **Delay_Resp** carrying the arrival time	**t4**

With all four numbers the slave can compute:

- **Mean path delay** = [(t2 − t1) + (t4 − t3)] ÷ 2
- **Offset from master** = [(t2 − t1) − (t4 − t3)] ÷ 2

(**Master** = the clock giving time; **slave** = the clock receiving it. Newer documents say timeTransmitter and timeReceiver, but IOS XE output and most exam material still use master/slave.)

Worked example — the math with real numbers. The real one-way delay is 10 µs in each direction, and the slave's clock is **50 µs ahead** of the master. All values are in µs, each read on the clock that took it:

1. Master sends Sync at **t1 = 0** (master time).
2. It arrives 10 µs later: master time 10, but the slave's clock reads 10 + 50, so **t2 = 60**.
3. The slave sends Delay_Req at **t3 = 100** on its own clock (master time 50).
4. It arrives at the master 10 µs later: **t4 = 60** (master time).

- Mean path delay = [(60 − 0) + (60 − 100)] ÷ 2 = (60 − 40) ÷ 2 = **10 µs**
- Offset = [(60 − 0) − (60 − 100)] ÷ 2 = (60 + 40) ÷ 2 = **50 µs**

The slave now knows it is 50 µs ahead and corrects itself. It does this many times per second, so drift never builds up.

Why does this math depend on the path being symmetric? Because dividing by two assumes the trip out took as long as the trip back. If a switch in the path held the Sync for 40 µs in a queue but forwarded the Delay_Req immediately, the slave would blame half of that 40 µs on its own clock. That's the problem the PTP-aware clock types below exist to fix.

Same example, with that 40 µs queue on the Sync only: t2 becomes 100 instead of 60. Delay = [(100 − 0) + (60 − 100)] ÷ 2 = **30 µs** (wrong: really 10). Offset = [(100 − 0) − (60 − 100)] ÷ 2 = **70 µs** (wrong: really 50). The slave would set its clock **20 µs** off, exactly half of the 40 µs it couldn't see.

Clock Types

Clock type	What it does	Why it exists
Grandmaster	The domain's ultimate source, usually GPS-locked	Someone has to be the truth
Ordinary clock	An endpoint with one PTP port, either master or (usually) slave	Servers, sensors, cameras
Boundary clock (BC)	A switch that is **slave** on its upstream port and **master** on its downstream ports, re-generating time	Breaks a large domain into hops, so no device sees accumulated queuing error and the grandmaster isn't serving thousands of slaves
Transparent clock (TC)	A switch that **doesn't sync itself** but measures how long each PTP packet sat inside it (its **residence time**) and writes that into the packet's **correction field**	Removes the switch's own queuing delay from the math. **E2E** TCs correct residence time only; **P2P** TCs also measure each link's delay

Story Time : You post a letter with the time written on it, and the reader wants to know exactly how long it took in transit. A **transparent clock** is a sorting office that stamps the envelope "held here for 40 µs". The reader subtracts every such stamp and is left with only the true travel time. A **boundary clock** is different: it's a local office that reads the letter, sets its own official clock from it, and then writes fresh letters to everyone downstream. Where it breaks: real sorting offices don't add their times together. In PTP, every transparent clock on the path adds its residence time to the same correction field, so the slave subtracts one total.

Worked example — fixing the 40 µs queue. Put an E2E transparent clock where that switch was. The Sync sits inside it for 40 µs, so it adds **40 µs** to the Sync's correction field. The slave subtracts the correction field from t2 before doing the math: 100 − 40 = 60, which is exactly the clean case above. The answer comes out right: delay 10 µs, offset 50 µs. (Delay_Req messages get the same treatment on the way back.)

The Best Master Clock Algorithm (BMCA)

Every PTP clock advertises itself in **Announce** messages, and every port independently picks the best one it hears. The comparison order is:

1. **Priority1** (0–255, lower wins, default **128**). The administrator's override.
2. **Clock class**: how good the source is (locked to GPS beats free-running).
3. **Clock accuracy**
4. **Offset scaled log variance**: how stable it is.
5. **Priority2**: a second administrative tiebreaker.
6. **Clock identity**: a unique ID derived from the MAC address. The final tiebreaker.

If this reminds you of the STP root bridge election (configurable priority first, then MAC as the tiebreaker), that's because it solves the same problem: elect one root everyone agrees on,

without a central controller.

Worked example — who becomes grandmaster?

Clock	Priority1	Clock class	Identity
GM-A (GPS-locked)	128	6	…:AA
GM-B (GPS-locked)	128	6	…:BB
SW1 (free-running)	128	248	…:11

1. Priority1 is a tie (128 for all), so move to the next item.
2. Clock class: 6 (locked to GPS) beats 248 (a default, not-locked clock). SW1 is out.
3. GM-A and GM-B tie on accuracy, variance and priority2, because they are identical models.
4. Clock identity: …:AA is lower than …:BB, so **GM-A wins**.

Now the administrator sets `ptp priority1 100` on GM-B. Step 1 is no longer a tie: 100 beats 128, so **GM-B wins before clock class is even looked at**. This is exactly how you pin the grandmaster, the way you'd lower STP priority to pin the root bridge. The catch is that priority1 beats quality: give a free-running clock priority1 0 and it wins over GPS, so lower priority1 only on a clock you trust.

Interpreting a PTP Configuration

ENCOR asks you to interpret PTP configs, not memorize every platform's syntax, which differs across Catalyst, IE and Nexus switches. Here is a representative Catalyst 9000 IOS XE config:

```
ptp mode boundary delay-req         ! act as a boundary clock, end-to-end delay mechanism
ptp domain 0                        ! PTP domain: clocks only listen within one domain
ptp priority1 128                   ! default; lower it to make this switch preferred master
ptp priority2 128
!
interface TenGigabitEthernet1/0/1   ! uplink toward the grandmaster
 ptp announce interval 1            ! log2 seconds: 2^1 = one Announce every 2 s
 ptp sync interval -3               ! 2^-3 = 8 Sync messages per second
 ptp delay-req interval -3
```

How to read it:

- **`mode boundary`** means the switch will sync to whatever its uplink hears and serve time downstream. `e2etransparent` or `p2ptransparent` would mean "correct packets in flight, don't sync". **`delay-req`** names the delay mechanism: the four-message exchange above, as opposed to peer-delay.
- **`domain`** is a namespace. A slave in domain 0 ignores a grandmaster in domain 24, so a mismatch looks like "no master found".
- **Intervals are logarithmic (base 2).** `-3` means 2^{-3} s, or eight messages per second. **Negative numbers mean** faster **than one per second.** Reading `-3` as "every 3 seconds" is the classic interpretation trap.
- **`priority1`** works like STP root priority. Lowering it on the grandmaster-facing switch (or the grandmaster itself) pins the election.

```
SW1# show ptp parent
 Parent Clock:
  Parent Clock Identity: 0x0:1D:C1:FF:FE:12:34:56
 Grandmaster Clock:
  Grandmaster Clock Identity: 0x0:1D:C1:FF:FE:12:34:56
  Grandmaster Clock Quality:
        Class: 6                        ! 6 = locked to a primary reference (GPS)
        Accuracy: Within 100ns
```

Use `show ptp clock` (my own clock and mode), `show ptp port` (each port's state: MASTER, SLAVE or PASSIVE) and `show ptp parent` (who won BMCA). Port states again mirror STP: one upstream "root" port (SLAVE), downstream "designated" ports (MASTER), and PASSIVE ports that hear a better master but aren't used, like a blocking port.

PTP Profiles

IEEE 1588 is a toolkit, and industries publish **profiles** that fix the options (message rates, delay mechanism, transport, domain numbers) for their use case. Clocks only interoperate if they run the same profile:

Profile	Where it's used	Notable choices
Default (1588)	General enterprise / data center	E2E delay-request, Layer 3 (UDP 319/320) or Layer 2
Power (IEEE C37.238 / IEC 61850-9-3)	Substations, grid protection	Peer-delay, Layer 2 Ethernet transport
802.1AS (gPTP)	AVB/TSN audio-video, automotive	Peer-delay, Layer 2 only, every bridge must be time-aware
AES67 / SMPTE ST 2059-2	Broadcast audio and video over IP	Faster message rates, specific domain numbers
Telecom (G.8275.x)	Mobile fronthaul/backhaul	Strict accuracy budgets, boundary clocks at every hop

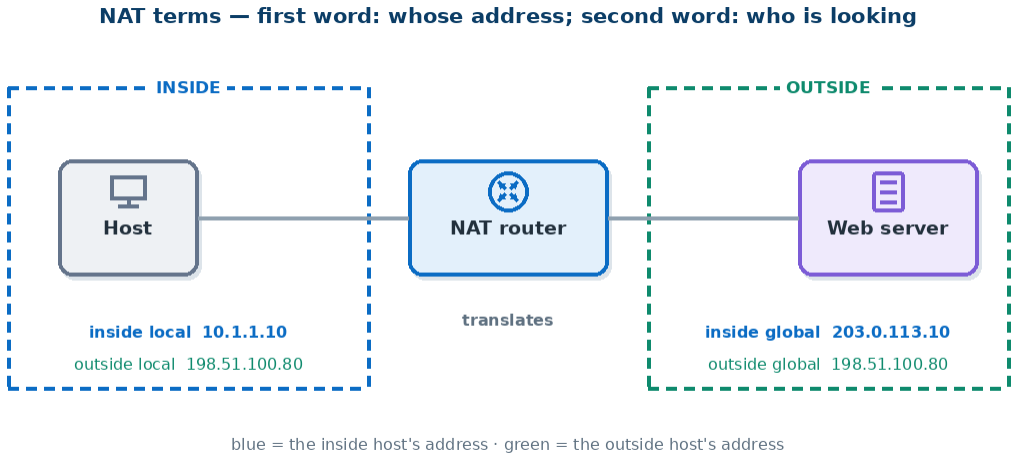
On the exam, recognizing that a mismatched profile (or domain) means "no master found" matters more than memorizing every profile's numbers.

8.5 NAT/PAT at CCNP Depth

The problem NAT solves: your inside hosts use private addresses (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) that the internet won't route, and public IPv4 addresses are scarce and expensive. **NAT (Network Address Translation)** rewrites addresses as packets cross the edge router, so private hosts can borrow a public address. **PAT (Port Address Translation**, also called NAT overload) goes further and lets hundreds of hosts share **one** public address by also rewriting the port numbers.

Story Time : A company has one public phone number and a receptionist. When Priya at extension 57 calls a supplier, the receptionist dials out from the main number and writes in a logbook: "outgoing line 51344 = extension 57". When the supplier calls back, the receptionist reads the log and puts the call through to extension 57. That logbook is the **NAT translation table**, and the main number is the **inside global** address. Now a stranger calls the main number asking for "the web server". There's no log entry, so the call goes nowhere, unless there's a standing instruction ("calls for the web server always go to extension 10"). That standing instruction is a **static NAT** entry. Where it breaks: a real receptionist can ask the caller who they want. A router can't: if no entry matches, the packet is simply dropped.

The Four Address Terms



blue = the inside host's address · green = the outside host's address

Term	Means	Example
Inside local	An inside host's address as seen inside: the real, usually private address	10.1.1.10
Inside global	That same inside host as seen from outside: the translated public address	203.0.113.10
Outside global	An outside host's real address	198.51.100.80
Outside local	That outside host as seen from inside. Usually identical to outside global unless you translate outside addresses too	198.51.100.80

How to never mix them up: the first word (inside/outside) says **whose address it is**. The second word (local/global) says **which side is looking at it**. "Inside global" = an inside host's address, as the global side sees it.

Order of Operations: Where NAT Happens Relative to Routing and ACLs

Direction	Order on IOS / IOS XE
Inside → outside	Inbound ACL on the inside interface → **routing** → **NAT** (inside source translated) → outbound ACL on the outside interface
Outside → inside	Inbound ACL on the outside interface → **NAT** (inside global translated back to inside local) → **routing** → outbound ACL on the inside interface

Why does this order matter in practice? Three consequences:

1. **An inbound ACL on the** outside **interface sees the** public **(inside global) destination**, because NAT hasn't untranslated it yet. Write `permit tcp any host 203.0.113.10 eq 443`, not the

10.x address. This is a top troubleshooting trap.

2. **An outbound ACL on the outside interface sees the already-translated source.** Matching on 10.1.1.0/24 there matches nothing.
3. **For traffic coming in, NAT happens before routing,** so the router routes on the real inside address. That's how a reply to 203.0.113.10 gets routed to 10.1.1.10 at all. Going out, routing happens first, so the egress interface must be marked `ip nat outside` for translation to trigger.

Rule of thumb: the outside world, including outside-facing ACLs, only ever sees global addresses.

Walk-through — one web request through PAT, step by step (using the config in the next section):

1. Host 10.1.1.57 sends a packet from **10.1.1.57:51344** to **142.250.72.14:443**. It arrives on Gi0/0/0 (`ip nat inside`).
2. Any inbound ACL on Gi0/0/0 is checked against the **private** source, 10.1.1.57.
3. The router does a **routing** lookup for 142.250.72.14. The default route points out Gi0/0/1, which is `ip nat outside`, so translation is needed.
4. **NAT:** the source matches ACL NAT-USERS, so the router picks an address from pool USERS (203.0.113.20), keeps port 51344 because it's free, rewrites the source to **203.0.113.20:51344**, and records the mapping in the translation table.
5. Any outbound ACL on Gi0/0/1 now sees source **203.0.113.20**, not 10.1.1.57.
6. The reply comes back from 142.250.72.14:443 to **203.0.113.20:51344**. An inbound ACL on Gi0/0/1 sees that **public** destination.
7. **NAT before routing on the way in:** the table says 203.0.113.20:51344 → 10.1.1.57:51344, so the destination is rewritten.
8. **Routing** looks up 10.1.1.57 (connected on Gi0/0/0), and the reply is delivered.

Worked example — the ACL that blocks a working server. You publish the web server with `ip nat inside source static 10.1.1.10 203.0.113.10` and put this inbound ACL on the outside interface:

```
ip access-list extended FROM-ISP
 permit tcp any host 10.1.1.10 eq 443        ! wrong: NAT hasn't run yet
```

A client sends to 203.0.113.10:443. The ACL runs **before** NAT, sees destination 203.0.113.10, finds no match, and the implicit deny drops the packet. `show ip nat translations` shows the static entry is fine, which is why this is so confusing. The fix is to match what the ACL actually sees: `permit tcp any host 203.0.113.10 eq 443`.

Try It — Every Flavor in One Router

```
interface GigabitEthernet0/0/0
 description LAN
 ip address 10.1.1.1 255.255.255.0
 ip nat inside
interface GigabitEthernet0/0/1
 description ISP - 203.0.113.0/27 assigned
 ip address 203.0.113.2 255.255.255.224
 ip nat outside
!
! 1) Static one-to-one: web server always 203.0.113.10, both directions
ip nat inside source static 10.1.1.10 203.0.113.10
!
! 2) Static PAT (port forwarding): outside :2222 -> inside SSH bastion :22
ip nat inside source static tcp 10.1.1.20 22 203.0.113.11 2222 extendable
!
! 3) Dynamic PAT from a pool for the users subnet
```

Why does the static server still work when the dynamic rule also matches 10.1.1.10? Static translations take precedence over dynamic ones for the hosts they cover. The server gets 203.0.113.10 both ways, and every other host in the ACL is overloaded onto the pool.

Why `extendable`? It lets the same inside local address appear in more than one static translation. For example, one server can be published on two public addresses or ports. IOS adds it automatically when you configure overlapping static port entries. Read it as "this static entry is allowed to share its local address".

Why a pool with `overload` instead of just the interface? A single public IP gives about 64,000 ports per protocol. That's plenty for an office, but not for a large campus or a carrier, and some services rate-limit per source IP. A pool spreads the sessions across several addresses. PAT on the **interface** wins when the ISP assigns the address via DHCP, because the rule follows whatever address the interface currently holds.

Dynamic NAT Without Overload — One Public Address per Host

Drop the `overload` keyword and the same pool behaves completely differently:

```
ip access-list standard NAT-LAB
 permit 10.1.2.0 0.0.0.255
ip nat pool LAB-POOL 203.0.113.24 203.0.113.27 netmask 255.255.255.224
ip nat inside source list NAT-LAB pool LAB-POOL      ! no overload = one-to-one
```

Behaviour	Dynamic NAT (no)	PAT ( )
Mapping	One inside host ↔ one pool address	Many hosts share one address, told apart by port
Translation entry	A simple entry with **no ports**, held until the timeout (24 h default)	Per-flow entry with ports, short timeouts (TCP 24 h, UDP 5 min, DNS 1 min)
Capacity	**4 addresses = 4 concurrent hosts**	Roughly 64,000 sessions per address
Failure mode	Host number 5 gets **no translation** until an entry expires	Rarely runs out
Inbound reachability	The outside can reach the host on its current mapped address while the entry exists	Only for sessions the inside started (or static port forwards)

Worked example — the fifth user. Hosts 10.1.2.11, .12, .13 and .14 open sessions and receive 203.0.113.24, .25, .26 and .27. The pool is now empty. Host 10.1.2.15 sends a packet: no address is free, so **no translation is created**, the pool `misses` counter goes up by one, and the packet is dropped. The four existing entries stay until they time out (24 hours by default), even if those users went home an hour ago. Add `overload` and all five (or five hundred) users share the pool by port instead.

Why would anyone choose the one-to-one version? For applications that break under port translation. Some protocols embed addresses or expect a fixed source port, and partner firewalls may whitelist one address per host. It also makes logs simple: every public address maps to

exactly one internal host at a time. The price is that the pool must be as big as the number of simultaneous users, and idle entries hold addresses hostage until they time out. That's why shorter timeouts (below) and `clear ip nat translation *` are the usual fixes when a one-to-one pool runs dry.

NAT for Overlapping Networks (briefly)

After a merger, both companies may use 10.1.0.0/16. Translating only one side isn't enough, because each side needs to see the other side under a non-overlapping address. The fix is to translate **both** directions: `ip nat inside source` for your hosts, plus `ip nat outside source static 10.1.5.5 172.31.5.5` so that your hosts reach the partner's server as 172.31.5.5 (its **outside local** address). DNS replies must also hand out the translated address. This is the one case where outside local ≠ outside global.

Worked example. Your host 10.1.5.20 wants the partner's server, which really is 10.1.5.5 (the same address space as yours, so a packet to 10.1.5.5 would stay on your own LAN). With the outside static rule in place:

1. Your host sends to **172.31.5.5** (outside local), the address DNS gave it. That's a destination your network routes toward the partner link.
2. On the way out, the router rewrites the destination 172.31.5.5 → **10.1.5.5** (outside global). Your inside rule also rewrites your source to an address the partner doesn't use.
3. The partner's server sees a request from a non-overlapping address and replies. On the way back, both translations are reversed.

Term	Value
Outside global (partner server's real address)	10.1.5.5
Outside local (how your hosts see it)	172.31.5.5

Verifying

```
R1# show ip nat translations
Pro  Inside global         Inside local       Outside local      Outside global
---  203.0.113.10          10.1.1.10          ---                ---
tcp  203.0.113.11:2222     10.1.1.20:22       ---                ---
tcp  203.0.113.20:51344    10.1.1.57:51344    142.250.72.14:443  142.250.72.14:443
udp  203.0.113.20:60211    10.1.1.61:60211    8.8.8.8:53         8.8.8.8:53
```

- Lines with `---` on the outside are **static entries**. They exist with no traffic flowing.
- Lines with protocol and ports are **live PAT sessions**, created by traffic.
- Outside local = outside global here. Nothing is translating the outside.
- **Port preservation:** IOS PAT keeps the inside source port (51344 → 51344) whenever that port is free on the public address, and only picks a different port when two inside hosts collide on the same one. Don't read an unchanged port as "PAT isn't working".

Worked example — two hosts pick the same port. 10.1.1.57 and 10.1.1.61 both happen to open a connection from source port **51344**. Both are translated to the pool's first address, 203.0.113.20:

1. 10.1.1.57:51344 arrives first. 203.0.113.20:51344 is free, so it gets **203.0.113.20:51344**.
2. 10.1.1.61:51344 arrives next. 203.0.113.20:51344 is taken, and two flows can't share it or the replies couldn't be told apart. IOS gives this flow a **different free port** on the same address (for example 203.0.113.20:1024), or the next pool address if the first is full.
3. Replies to :51344 go to .57 and replies to the other port go to .61. The table entries keep them apart, the same way the receptionist's logbook does.

The exact replacement port IOS picks doesn't matter for the exam. What matters is that the **(address, port)** pair on the outside is unique for each flow.

```
R1# show ip nat statistics
Total active translations: 214 (2 static, 212 dynamic; 213 extended)
Outside interfaces:
  GigabitEthernet0/0/1
Inside interfaces:
  GigabitEthernet0/0/0
Hits: 1882301  Misses: 4410
Expired translations: 39821
Dynamic mappings:
-- Inside Source
[Id: 1] access-list NAT-USERS pool USERS refcount 212
 pool USERS: id 1, netmask 255.255.255.224
        start 203.0.113.20 end 203.0.113.23
        type generic, total addresses 4, allocated 1 (25%), misses 0
```

Hits means a packet matched an existing translation. **Misses** means a packet needed a new entry to be created. A **misses** counter on the pool line that keeps climbing means the pool can't hand out addresses. That points to a non-overload pool that is exhausted, the classic dynamic-NAT failure.

Timeouts

Entry type	Default timeout
Simple dynamic translation (no ports)	**86,400 s** (24 h)
TCP PAT entry	**86,400 s** (24 h)
UDP PAT entry	**300 s**
DNS	**60 s**
TCP after FIN/RST	**60 s** (`finrst-timeout`)

Tune them with `ip nat translation timeout <sec>` and per-protocol variants such as `ip nat translation udp-timeout 120`. **Why would you shorten them?** Because a dynamic pool without overload is held by an entry until it expires. With a 24-hour default, ten users can exhaust a ten-address pool for a whole day after they've gone home.

NAT Troubleshooting

Symptom	Likely cause	Command that proves it
No translations created at all	Missing `ip nat inside`/`outside`, or interfaces swapped	`show ip nat statistics` (interface lists)
Some hosts translate, others don't	NAT ACL doesn't match their subnet	`show ip access-lists NAT-USERS` (hit counts)
First N users work, the rest fail	Pool without `overload` exhausted	Pool `misses` counter, `show ip nat translations`
Server unreachable from the internet	Inbound ACL written for the private address; or missing static entry	`show ip access-lists` on the outside ACL; `show ip nat translations`
Return traffic never arrives	ISP not routing the pool's addresses to you; pool not in the outside subnet and no route	`debug ip nat` (limit it with an ACL), traceroute from outside
Translation works only after `clear`	Stale entries after a rule change. Existing sessions keep their old mapping	`clear ip nat translation *`

Exam trap: changing a NAT rule doesn't touch translations that already exist. If you fix the config and nothing changes, clear the table, then retest.

8.6 Chapter Summary

Topic	The one-line idea
NTP stratum	Lower = closer to the truth; 16 = unsynchronized
NTP auth	Key + **trusted-key** + **authenticate** + `key` on the server line
`show ntp associations`	`*` = synced to; `reach 377` = 8/8 replies; offset in ms
PTP vs NTP	Hardware timestamps + on-path correction → sub-µs
BC vs TC	Boundary re-generates time; transparent corrects residence time
BMCA	Priority1 → class → accuracy → variance → priority2 → identity
PTP intervals	$\log_2$ seconds. **Negative = faster than 1/s**
NAT terms	First word = whose address; second = who's looking
NAT order	In→out: route then NAT. Out→in: NAT then route
Outside ACL	Matches **global** (public) addresses

Chapter 9: First Hop Redundancy — HSRP & VRRP

> **Blueprint:** ENCOR 3.3.c — Configure first hop redundancy protocols, such as HSRP, VRRP; ENCOR 1.1.b — High availability techniques such as redundancy, FHRP (the FHRP part; SSO and stacking are in Chapter 2)

The CCNA guide (14.7) explained why a first-hop redundancy protocol exists. A PC has exactly one default gateway, so two physical routers must pretend to be one virtual router with a shared IP and MAC. At CCNP you have to build it properly. That means choosing the right protocol and version, making failover depend on what actually matters (the uplink, not just the LAN port), splitting the load across both routers, and aligning all of it with Spanning Tree so traffic doesn't take a detour. Chapter 2 places FHRPs among the other high-availability tools. This chapter is the engineering.

9.1 What Every FHRP Has in Common

The problem: a PC is configured (or told by DHCP) with **one** default gateway address, say 10.10.10.1. If the router holding that address dies, the PC can't reach anything off its subnet, even though a second healthy router sits on the same VLAN. The PC has no idea the second router exists and no way to switch to it. A **first hop redundancy protocol (FHRP)** fixes this on the router side, so nothing has to change on the PCs.

Story Time : A shop has one front door with the shop's name above it. Customers only know "go through that door". Behind it, two staff members take turns standing at the counter. If the one on duty collapses, the other steps up to the same counter, wearing the same name badge. Customers walk through the same door, see the same name, and never notice the swap. The door's address is the **virtual IP**, the name badge is the **virtual MAC**, and the staff member on duty is the **Active** (HSRP) or **Master** (VRRP) router. Where it breaks: in a real shop the customers would see a different face. In the network they truly can't tell, because the frames they send look identical before and after.

All three protocols (HSRP, VRRP and GLBP) share one mechanism:

1. A group of routers on a segment agrees on a **virtual IP (VIP)**. Hosts use the VIP as their default gateway.
2. The VIP maps to a **virtual MAC**, so a host's ARP cache never has to change during failover.
3. One router **actively forwards** for that VIP. The others listen to its hellos.
4. When the hellos stop, a backup takes over the VIP and the virtual MAC, and sends a **gratuitous ARP** so the switches relearn which port the MAC lives on. (A gratuitous ARP is an ARP reply nobody asked for: "10.10.10.1 is at 0000.0c9f.f00a". Switches see the source MAC arrive on a new port and update their MAC address tables.)

Why does the virtual MAC matter as much as the virtual IP? A host doesn't send frames to an IP address. It sends them to whatever MAC its ARP cache holds for the gateway, and it may keep that entry for hours. If failover moved only the IP, every host would keep sending frames to the dead router's MAC until its ARP entry aged out. Moving the MAC too makes failover invisible to hosts. The host never learns that anything happened, which is the whole point.

9.2 HSRP Mechanics

HSRP (Hot Standby Router Protocol) is Cisco-proprietary and runs over **UDP 1985**. One router is **Active** (forwards traffic), one is **Standby** (next in line), and any others are in **Listen**. It answers three questions: who forwards (the election), how everyone knows they're still alive (hellos and timers), and when the role moves (failure or preemption).

Version 1 vs Version 2

Property	HSRPv1	HSRPv2
Group numbers	0–255	**0–4095**, so the group can match the VLAN number
Virtual MAC	`0000.0c07.acXX` (XX = group in hex)	`0000.0c9f.fXXX` (XXX = group in hex)
Hello destination	**224.0.0.2** (all routers)	**224.0.0.102** (HSRP-specific)
IPv6 support	No	**Yes** (FF02::66, vMAC `0005.73a0.0XXX`)
Millisecond timers	Yes	Yes, plus better timer advertisement
Default	**Yes** (on most IOS)	Enable with `standby version 2`

Why did v2 move off 224.0.0.2? Because 224.0.0.2 means "all routers on this subnet", so every router has to process HSRPv1 hellos even if it doesn't run HSRP. A dedicated group address (224.0.0.102) means only HSRP speakers care. v2 also makes **group = VLAN** possible across a 4,000-VLAN campus. Group 100 on VLAN 100 is self-documenting.

Exam trap: v1 and v2 **don't interoperate**. If one side runs v1 and the other v2, both routers go **Active**, because each one's hellos are invisible to the other.

Quick check: HSRPv1 group 10 → vMAC `0000.0c07.ac0a` (10 = 0x0a). HSRPv2 group 100 → `0000.0c9f.f064` (100 = 0x064).

States

State	What's happening
Initial	HSRP starting, interface just came up
Learn	Waiting to hear the VIP from the Active router (if the VIP isn't configured locally)
Listen	Knows the VIP; neither Active nor Standby. **Silent**: listens for hellos but sends none
Speak	Sending hellos, taking part in the election
Standby	Next in line. Sends hellos
Active	Forwards for the VIP and answers ARP with the vMAC

The Election

1. **Highest priority wins.** The range is 0–255 and the default is **100**.
2. **Tie: highest interface IP address wins.**
3. **Preemption is OFF by default.** A higher-priority router that comes up later does **not** take over until you configure `standby X preempt`.

Worked example — three routers, one group.

Router	Interface IP	Priority	Preempt
R1	10.10.10.2	110	yes
R2	10.10.10.3	100	no
R3	10.10.10.4	100	no

1. All three boot together. R1 has the highest priority (110), so **R1 becomes Active**.
2. For Standby, R2 and R3 tie at 100, so the higher IP wins: **R3 (10.10.10.4) becomes Standby**.
3. R2 goes to **Listen**: it knows the VIP and listens to hellos, but stays silent.
4. Now R1 reloads. R3 stops hearing R1's hellos, waits out the hold time, and becomes **Active**. R2 moves up to **Standby**.
5. R1 comes back with priority 110. Because R1 has `preempt`, it takes the Active role back from R3 (after any preempt delay) by sending a **coup** message. R3 gives up the Active role, and the routers that aren't Active settle again into one Standby and one Listen.

Change one thing: remove `preempt` from R1. In step 5, R1 comes back, sees an Active router already (R3), and does **not** take over, even though its priority is higher. R3 stays Active until it fails. Priority decides who wins an election; preemption decides whether an election is re-held when someone better arrives.

Why is preempt off by default? Because every takeover is a small disruption: a gratuitous ARP, a MAC move and possibly lost packets. If the "better" router is flapping, preemption makes every flap cause two outages, one when it dies and one when it grabs the role back. Cisco's default

favors stability, and you opt in when you have a reason. When you do enable it, add a **delay** (`preempt delay minimum 60`) so a rebooting router finishes building its routing table before it attracts traffic it can't yet route.

Timers

The defaults are **hello 3 s, hold 10 s**. The hold time is how long a router waits without hearing hellos before declaring the Active router dead. You can tune them down to sub-second (`standby 10 timers msec 250 msec 750`) for faster failover, at the cost of more CPU and more risk of false failovers on a busy control plane. For true sub-second detection, BFD (Chapter 25) is often the better tool.

Walk-through — what happens second by second when the Active router loses power (default timers, DSW1 Active, DSW2 Standby):

1. **t = 0 s:** DSW1 sends a hello, then loses power immediately afterwards.
2. **t = 0–10 s:** DSW2 hears nothing. Hosts keep sending frames to vMAC `0000.0c9f.f00a`. The access switch still forwards them toward DSW1's port, and they're lost. This gap is the outage your users feel.
3. **t = 10 s:** DSW2's hold timer expires. DSW2 becomes **Active**, starts answering for 10.10.10.1, and sends a **gratuitous ARP** from the vMAC.
4. **t ≈ 10 s:** the access switch sees the vMAC arrive on DSW2's port and moves its MAC table entry. Frames now reach DSW2.
5. The hosts changed nothing: same gateway IP, same gateway MAC in their ARP caches.

With `timers msec 250 msec 750`, step 3 happens at about 0.75 s instead of 10 s. That's the whole reason people tune the timers.

Why hold = about 3 × hello? So that one or two lost hellos (a busy CPU, a dropped multicast) don't trigger a failover. The backup only acts after missing roughly three in a row.

9.3 Try It — HSRP Done Properly

```
track 1 interface GigabitEthernet1/0/48 line-protocol   ! the uplink toward the core
!
interface Vlan10
 ip address 10.10.10.2 255.255.255.0
 standby version 2
 standby 10 ip 10.10.10.1                     ! the VIP hosts use as gateway
 standby 10 priority 110                      ! I should be Active for VLAN 10
 standby 10 preempt delay minimum 60          ! take it back, but only after 60 s
 standby 10 authentication md5 key-string Hsrp!Key
 standby 10 track 1 decrement 20              ! uplink down -> 110-20 = 90 < 100
```

Why Tracking Is Not Optional

Picture DSW1 as Active for VLAN 10. Its **uplink** to the core fails, but its VLAN 10 interface is still up. HSRP hellos on VLAN 10 keep flowing, so DSW1 stays Active and keeps attracting every host's traffic, which it can no longer send anywhere useful. HSRP only watches the LAN segment, and it has no idea the uplink is gone.

Tracking links HSRP priority to something that does matter. When track object 1 goes down, priority drops by 20, from 110 to 90. DSW2 is at 100, so it now has the higher priority. **But DSW2 only takes over if DSW2 has `preempt` configured.** Tracking lowers your priority. Preemption is what lets the other router act on that.

Worked example — the uplink failure, step by step (DSW1 priority 110 with `track 1 decrement 20`, DSW2 priority 100 with `preempt`):

1. Gi1/0/48 on DSW1 goes down. Track object 1 changes to **Down**.
2. DSW1's HSRP priority drops: 110 − 20 = **90**. Its next hello advertises 90.
3. DSW2 hears an Active router advertising 90, which is lower than its own 100, and DSW2 has preempt. DSW2 sends a **coup** message and becomes **Active**. DSW1 drops to **Standby**.
4. Hosts' traffic now goes to DSW2, whose uplink works.
5. The uplink comes back. Priority returns to 110. DSW1 has preempt with a 60 s delay, so 60 s later it takes the Active role back.

`show standby` on DSW1 during the failure would show `Priority 90 (configured 110)` and `Track object 1 state Down decrement 20`.

Default decrement: if you write `standby 10 track 1` with no `decrement` value, the priority drops by **10**. **Exam trap:** the decrement has to be big enough to cross the other router's priority. 110 − 5 = 105 is still higher than 100, so nothing happens. Do the arithmetic every time.

Tracking More Than Interfaces

An uplink can be up/up while the path beyond it is broken. For example, the link to the core is fine, but the core's own link to the data center is down. Track **reachability** instead, using IP SLA (Chapter 15). IP SLA sends a probe (here a ping every 5 s), the track object follows whether the probe succeeds, and HSRP follows the track object:

```
ip sla 1
 icmp-echo 10.255.255.1 source-interface Vlan10   ! ping something beyond the uplink
 frequency 5
ip sla schedule 1 life forever start-time now
!
track 2 ip sla 1 reachability
 delay down 10 up 30                              ! damp flaps: wait before acting
!
interface Vlan10
 standby 10 track 2 decrement 20
```

`track X ip route 0.0.0.0 0.0.0.0 reachability` is another common choice: "Am I still learning a default route?"

Why `delay down 10 up 30`? If the ping fails once, the track object waits 10 s before going Down, so one lost probe doesn't move the gateway. When pings recover, it waits 30 s before going Up, so a path that is flapping has to prove it is stable before the priority is restored and traffic moves back.

Authentication

`standby 10 authentication md5 key-string …` stops a rogue device from sending priority-255 hellos and hijacking the gateway. Plain-text authentication (`standby 10 authentication text`) only prevents accidents, because the password is visible in a capture. A mismatch means each router ignores the other's hellos, which again produces **two Actives**.

The **key-chain** form lets you rotate keys without an outage, because each key has its own lifetime (the time must be right for that to work, which is one more reason NTP from Chapter 8 matters):

```
key chain HSRP-KEYS
 key 1
  key-string Hsrp!Key2026
!
interface Vlan10
 standby 10 authentication md5 key-chain HSRP-KEYS
```

IPv6 HSRP

```
interface Vlan10
 standby version 2                          ! required for IPv6
 standby 110 ipv6 autoconfig                ! VIP = link-local derived from the vMAC
 standby 110 priority 110
 standby 110 preempt
```

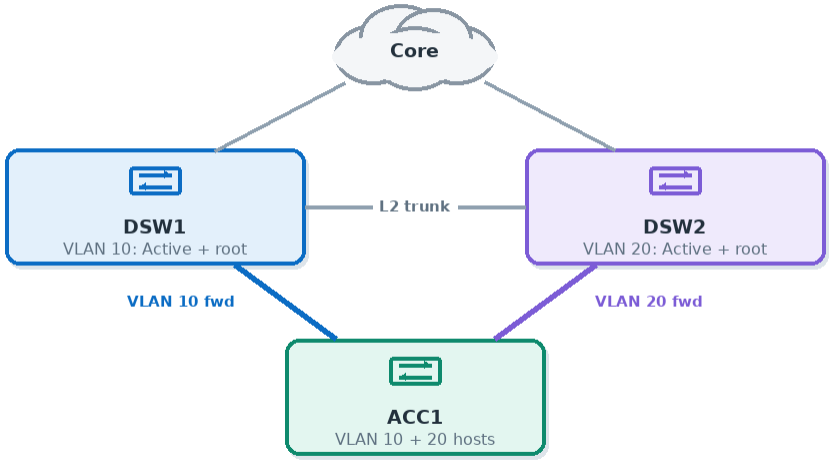
Why is the IPv6 VIP a link-local address? Because IPv6 hosts learn their default gateway from **Router Advertisements**, and the router in an RA is identified by its **link-local** address (CCNA guide 13.8). The HSRP Active router sends RAs from the virtual link-local address, so hosts install the VIP as their gateway automatically, without any static configuration.

9.4 Load Sharing — Using Both Routers

The problem: you paid for two distribution switches, but a single HSRP group leaves the Standby router idle, holding only a backup role. Half your uplink capacity does nothing until something breaks. To use both:

Run a separate group per VLAN, and alternate which router is Active.

Per-VLAN load sharing — HSRP Active and STP root on the same switch



each VLAN's forwarding uplink leads straight to its Active gateway

VLAN	HSRP Active	[redacted] (Chapter 4)
10	DSW1 (priority 110)	DSW1
20	DSW2 (priority 110)	DSW2

Why must the STP root and the HSRP Active router be the same switch? Picture them misaligned. For VLAN 10, DSW2 is the STP root, so the access switch's uplink to DSW1 is blocking for VLAN 10. HSRP Active is DSW1. A host's frame to the gateway now goes up to DSW2 (the only forwarding path), across the inter-distribution link, and then to DSW1. Every packet takes an extra hop across a link that was never sized for it. Aligning them means the forwarding uplink leads directly to the Active gateway. Layer 2 and Layer 3 redundancy have to agree on which box is "primary", or traffic zig-zags between them.

The matching configs, side by side. Each switch is primary for one VLAN and backup for the other, and both preempt so the design restores itself after a failure:

```
! --- DSW1: Active for VLAN 10, Standby for VLAN 20
spanning-tree vlan 10 root primary
spanning-tree vlan 20 root secondary
interface Vlan10
 ip address 10.10.10.2 255.255.255.0
 standby version 2
 standby 10 ip 10.10.10.1
 standby 10 priority 110
 standby 10 preempt delay minimum 60
 standby 10 track 1 decrement 20
interface Vlan20
 ip address 10.10.20.2 255.255.255.0
 standby version 2
 standby 20 ip 10.10.20.1
 standby 20 preempt                          ! default priority 100
!
! --- DSW2: Active for VLAN 20, Standby for VLAN 10
spanning-tree vlan 20 root primary
spanning-tree vlan 10 root secondary
interface Vlan10
 ip address 10.10.10.3 255.255.255.0
 standby version 2
 standby 10 ip 10.10.10.1
 standby 10 preempt                          ! takes over if DSW1 drops to 90
interface Vlan20
 ip address 10.10.20.3 255.255.255.0
 standby version 2
 standby 20 ip 10.10.20.1
 standby 20 priority 110
 standby 20 preempt delay minimum 60
 standby 20 track 1 decrement 20
```

Why does the backup **side need** `preempt` **too?** Tracking on DSW1 only lowers DSW1's priority to 90. DSW2 must be allowed to seize the role from a lower-priority Active, and that's exactly what `preempt` permits. Without it, DSW1 stays Active at priority 90 while its uplink is dead.

This is the per-VLAN load sharing pattern. It balances **per VLAN**, not per host, so it only splits traffic well if the VLANs carry similar loads.

Worked example — what each switch does in normal running and after DSW1's uplink fails.

Situation	VLAN 10 Active	VLAN 20 Active	Why
Normal	DSW1 (110 vs 100)	DSW2 (110 vs 100)	Each switch has the higher priority in "its" VLAN
DSW1 uplink down	DSW2 (100 vs 90)	DSW2 (unchanged)	DSW1 drops to 90 in VLAN 10; DSW2 preempts
Uplink restored	DSW1, 60 s later	DSW2	DSW1 is back at 110; its preempt delay expires

During the failure DSW2 carries both VLANs, which is exactly the redundancy you wanted. Once the uplink returns, the split restores itself without anyone touching a console.

9.5 VRRP — the Open Standard

Why a second protocol? HSRP only runs on Cisco gear. If your distribution pair is Cisco and a firewall cluster or another vendor's router must share the gateway role, you need an open standard that everyone implements.

VRRP (Virtual Router Redundancy Protocol) does the same job, standardized as RFC 3768 (v2) and RFC 5798 (v3). It is **IP protocol 112** (no UDP), and advertisements go to **224.0.0.18** (IPv4) or **FF02::12** (IPv6).

Property	VRRP
Roles	**Master** and **Backup** (no separate "standby": all non-masters are backups)
Virtual MAC	`0000.5e00.01XX` (IPv4), `0000.5e00.02XX` (IPv6), XX = group in hex
Group range	1–255
Priority	Default **100**; **255 = address owner**
Preemption	**ON by default**
Advertisement	Master only, every **1 s** (v3 counts in centiseconds: 100 cs)
Versions	v2: IPv4 only, optional authentication. v3: IPv4 + IPv6, sub-second, **no authentication**

What is the "address owner"? VRRP lets the VIP be one of the router's real interface addresses. The router that owns that address automatically gets priority **255** and always becomes Master. That's a sensible default: if the real owner is alive, it should answer for its own address.

Why does only the Master advertise in VRRP, while HSRP's Active and **Standby both send hellos?** Different design choices. VRRP keeps the segment quiet: backups listen, and if they hear nothing for the **master down interval** (3 × advertisement interval + skew time, where skew = (256 − priority) ÷ 256 s), the best backup takes over. With the defaults that's 3 × 1 + 156/256 ≈ **3.6 s**, and the skew means a priority-200 backup times out slightly sooner than a priority-100 one, so the best backup naturally wins. HSRP's Standby announces itself, so everyone knows the succession in advance; any further HSRP routers sit silently in **Listen**.

Worked example — which backup takes over? The Master dies. Two backups remain: B1 at priority 200 and B2 at priority 100. The advertisement interval is 1 s.

Backup	Skew = (256 − priority) ÷ 256	Master down interval = 3 × 1 + skew
B1 (200)	56 ÷ 256 ≈ 0.22 s	≈ **3.22 s**
B2 (100)	156 ÷ 256 ≈ 0.61 s	≈ **3.61 s**

1. Both backups stop hearing advertisements at the same moment.
2. B1's timer runs out first (3.22 s), so B1 becomes Master and starts advertising immediately.
3. B2's timer would have expired at 3.61 s, but it hears B1's advertisement first and resets. B2 stays Backup.

No election messages are needed: the higher priority simply gives a shorter fuse.

Story Time : HSRP is a team where the captain and vice-captain both keep calling out "still here!", so everyone always knows who's next. VRRP is a team where only the captain calls out. Everyone else stays quiet and has a stopwatch, and the most senior player's stopwatch is set slightly shorter, so if the captain goes quiet, the most senior player speaks up first.

Try It — VRRPv3 on IOS XE

```
fhrp version vrrp v3                        ! enable VRRPv3 syntax globally
!
interface Vlan20
 ip address 10.10.20.2 255.255.255.0
 vrrp 20 address-family ipv4
  address 10.10.20.1 primary                ! the VIP
  priority 110
  track 1 decrement 20                      ! same tracking idea as HSRP
  ! preempt is already on by default
 exit-vrrp
```

Older IOS uses VRRPv2 syntax at interface level (`vrrp 20 ip 10.10.20.1`, `vrrp 20 priority 110`). Know both when reading configs.

VRRPv2 vs VRRPv3 — Versions Must Match

VRRPv2 and VRRPv3 use different packet formats, so a v2 router and a v3 router on the same segment **don't understand each other's advertisements**. Both become Master, the same symptom as an HSRP version mismatch. Two ways to fix it:

- Run the same version on both. On IOS XE, `fhrp version vrrp v3` switches the whole box to v3 syntax.
- During a migration, put **`vrrpv2`** under the v3 IPv4 address family. The v3 router then also sends v2-format advertisements that the old router understands.

```
interface Vlan20
 vrrp 20 address-family ipv4
  vrrpv2                                  ! interoperate with VRRPv2 neighbours
```

VRRPv2 authentication exists (v3 removed it, per RFC 5798):

```
interface Vlan20
 vrrp 20 ip 10.10.20.1
 vrrp 20 authentication md5 key-string Vrrp!Key   ! or: authentication text <pw>
```

As with HSRP, a key mismatch makes each router ignore the other's advertisements, so both claim Master.

```
DSW1# show vrrp brief
Interface      Grp  A-F Pri  Time Own Pre State   Master addr/Group addr
Vl20            20 IPv4 110     0  N   Y  MASTER  10.10.20.2(local) 10.10.20.1
```

9.6 GLBP — Briefly, for Comparison

The problem GLBP aims at: with HSRP or VRRP, one group means one forwarding router per VLAN. GLBP lets every router in a single group forward at once.

GLBP (Gateway Load Balancing Protocol), Cisco-proprietary on **UDP 3222** to **224.0.0.102**, solves the "idle standby" problem inside one group. One router is the **AVG** (Active Virtual Gateway). It hands out **different virtual MACs** to different hosts in its ARP replies, and up to **four AVFs** (Active Virtual Forwarders) each forward for their own vMAC (`0007.b400.XXYY`). Everyone shares one VIP, yet hosts spread across all routers. Load balancing is **round-robin** by default, or weighted, or host-dependent.

Worked example. R1 is the AVG and AVF 1 (vMAC `0007.b400.0a01`), and R2 is AVF 2 (`0007.b400.0a02`), in group 10 with VIP 10.10.10.1. PC-A ARPs for 10.10.10.1 and the AVG answers with `.0a01`, so PC-A uses R1. PC-B ARPs next and the AVG answers with `.0a02`, so PC-B uses R2. Both PCs have the same gateway IP, but their frames go to different routers. If R2 dies, R1 takes over forwarding for `.0a02` too, so PC-B keeps working without re-ARPing.

It's elegant, but it fits poorly in modern designs. Spreading one VLAN's hosts across two gateways conflicts with STP alignment (9.4), and Catalyst 9000 switches often don't support it at all. You'll meet it in comparison questions far more than in production. It isn't named in the ENCOR v1.2 blueprint, which says "HSRP, VRRP".

9.7 Comparison Table (Exam Gold)

Feature	HSRP	VRRP	GLBP
Standard	Cisco	**IETF** (RFC 5798)	Cisco
Transport	UDP 1985	**IP protocol 112**	UDP 3222
Multicast	224.0.0.2 (v1) / .102 (v2)	**224.0.0.18**	224.0.0.102
Roles	Active / Standby	Master / Backup	AVG / AVF
Virtual MAC	0000.0c07.acXX / 0000.0c9f.fXXX	0000.5e00.01XX	0007.b400.XXYY
Default priority	100	100 (255 = owner)	100
Preempt default	**Off**	**On**	Off (AVG) / On (AVF)
Default timers	3 s / 10 s	1 s advert, ~3.6 s master down	3 s / 10 s
VIP can be a real IP?	No	**Yes**	No
Load balancing in one group	No	No	**Yes**

9.8 Troubleshooting FHRPs

Symptom	Likely cause	Command that proves it
Both routers Active/Master	Hellos not arriving: VLAN not allowed on the trunk, ACL dropping 224.0.0.2/.102/.18, **version** or **group number** mismatch, **authentication** mismatch	`show standby` (look for "Standby router is unknown"), `debug standby packets`
Both VRRP routers Master	VRRPv2 on one router, v3 on the other; or a v2 auth mismatch	`show vrrp` on both (version, auth), `debug vrrp packets`
Higher-priority router never takes over	`preempt` not configured	`show standby brief` (`P` column missing)
Uplink failed but no failover	No tracking, or the decrement is too small to cross the peer's priority	`show standby` (shows tracked objects and current priority), `show track`
Failover flaps repeatedly	Tracked object flapping; timers too aggressive	`show track` (change counters), add `delay down/up`, raise preempt delay
Duplicate address messages for the VIP	VIP configured as a real interface address on another device (HSRP)	`show ip arp`, syslog `%IP-4-DUPADDR`
Hosts lose connectivity for ~minutes after failover	Switch CAM didn't update; gratuitous ARP lost	`show mac address-table address <vMAC>`
Traffic hairpins between distribution switches	STP root and FHRP Active misaligned	`show spanning-tree vlan X` vs `show standby brief`

```
DSW1# show standby vlan 10
Vlan10 - Group 10 (version 2)
  State is Active
    5 state changes, last state change 00:12:41
  Virtual IP address is 10.10.10.1
  Active virtual MAC address is 0000.0c9f.f00a (MAC In Use)
  Hello time 3 sec, hold time 10 sec
  Preemption enabled, delay min 60 secs
  Active router is local
  Standby router is 10.10.10.3, priority 100 (expires in 9.168 sec)
  Priority 110 (configured 110)
    Track object 1 state Up decrement 20
```

Worked example — diagnosing "two Actives". Both DSW1 and DSW2 show `State is Active` for group 10, and both say `Standby router is unknown`. Work down the list:

1. `show standby` on both: DSW1 says `Group 10 (version 2)`, DSW2 says `(version 1)`. **Version mismatch found.** v1 hellos go to 224.0.0.2 and v2 to 224.0.0.102, so neither hears the other.
2. If the versions matched, the next checks would be: the same group number on both (10 vs 11 is a different group), the VLAN allowed on the trunk between them (`show interfaces trunk`), and matching authentication.

3. Fix: `standby version 2` on DSW2. Within one hello interval DSW2 hears DSW1, the election runs, and one of them drops to Standby.

Read it from the top: this router is Active, and it lists a real **Standby router**, so hellos are flowing both ways. Preemption is on, with a delay. Priority equals configured priority, which means no tracked object has fired. If you saw `Standby router is unknown` on both switches, they can't hear each other. Go straight to the first row of the table.

9.9 Chapter Summary

Concept	Remember
Virtual MAC	Moves with the VIP so hosts' ARP caches stay valid
HSRP election	Priority (default 100) → highest IP; **preempt off**
HSRPv2	Groups 0–4095, 224.0.0.102, needed for IPv6
Tracking	Lowers priority; the **peer's preempt** makes it matter
Load sharing	One group per VLAN, alternate Active, **align with STP root**
VRRP	Open, protocol 112, 224.0.0.18, **preempt on**, owner = 255
Two Actives	Always means the hellos aren't getting through

Chapter 10: Multicast

> **Blueprint:** ENCOR 3.3.d — Describe multicast protocols, such as RPF check, PIM SM, IGMP v2/v3, SSM, bidir, and MSDP

Multicast is the topic most CCNA graduates have seen the least of. CCNA mentions it only as a type of address. ENCOR's verb is **describe**, so you don't need to build a multicast network from memory. You do need to explain every moving part precisely: how hosts ask for a stream, how routers build a delivery tree toward them, and why a router will drop a perfectly good multicast packet that arrives on the "wrong" interface. Each protocol answers one question. Learn the question first, and the protocol follows.

10.1 Why Multicast Exists

Suppose one video server must send a 5 Mbps stream to 200 viewers.

Method	What the source sends	What the network carries	Problem
Unicast	200 copies (1 Gbps)	200 copies on the first links	Source and nearby links melt; doesn't scale
Broadcast	1 copy	Every host on the subnet gets it	Doesn't cross routers; bothers everyone
Multicast	**1 copy**	Copies made **only where paths split**, delivered **only to subscribers**	Needs protocols to know who wants what

Story Time : A magazine publisher could post every subscriber their own copy straight from the printing press. That's unicast: the press's loading dock is jammed with 200 vans. Or it could drop copies out of a plane over the whole city, and that's broadcast: everyone gets one whether they want it or not. Multicast is the sensible way: one bulk shipment goes out, each regional depot splits it only for the towns that have subscribers, and each local post office delivers only to the homes on its subscriber list. The subscription form is **IGMP**, and the depots arranging the shipping routes are **PIM**. Where it breaks: a real depot knows every subscriber's name. A multicast router only knows "someone on this interface wants group G", never who.

Worked example — counting copies. The video server sits behind R1. R1 has two downstream branches: R2 with 150 viewers and R3 with 50 viewers.

Link	Unicast copies (5 Mbps each)	Multicast copies
Server → R1	200 (1,000 Mbps)	**1** (5 Mbps)
R1 → R2	150 (750 Mbps)	**1**
R1 → R3	50 (250 Mbps)	**1**
R2 → its LAN	150	**1** (every viewer's NIC on that LAN picks it up)

R1 is where the paths split, so R1 makes the second copy. Nothing else is duplicated.

Why can't the source just send unicast and let the network cope? Because the cost grows with the number of receivers, and it's paid on the links nearest the source. With multicast, the source sends once, and each router replicates the packet only onto branches that lead to interested receivers. One copy per link, no matter how many viewers are downstream.

Multicast is almost always **UDP**, which means no handshake, no retransmission and best-effort delivery. Typical uses are IPTV and video distribution, market-data feeds, software imaging (one image pushed to 500 PCs), music-on-hold, and routing protocols themselves (OSPF's 224.0.0.5).

The two halves of every multicast design:

1. **Host ↔ first router:** a receiver says "I want group G". That's **IGMP**.
2. **Router ↔ router:** routers build a tree from the source to every router with interested receivers. That's **PIM**.

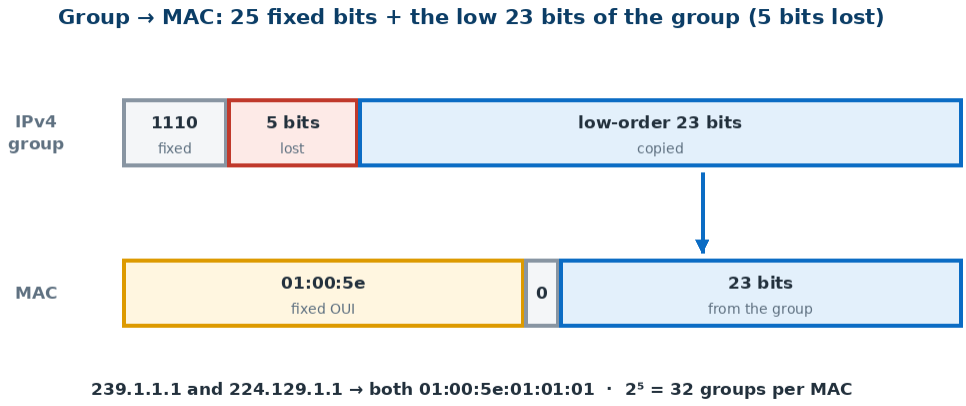
10.2 Multicast Addressing

Why special addresses? A multicast packet is addressed to a group, a "channel" that any number of hosts can tune into, rather than to one host. The network needs a reserved address block so every device can tell at a glance "this is a channel, not a person".

IPv4 multicast is **224.0.0.0/4** (224.0.0.0 – 239.255.255.255, the old Class D). A group address is a **destination only**: a packet is never sourced from a multicast address. The source is always a normal unicast address, such as the video server's 10.5.5.10.

Range	Name	Behavior
224.0.0.0/24	Local network control	**Link-local, TTL 1, never routed.** 224.0.0.1 all hosts, .2 all routers, .5/.6 OSPF, .9 RIPv2, .10 EIGRP, .13 PIM, .18 VRRP, .22 IGMPv3, .102 HSRPv2/GLBP
224.0.1.0/24	Internetwork control	Routable control: 224.0.1.1 NTP, .39/.40 Auto-RP, .129 PTP
232.0.0.0/8	**Source-Specific Multicast (SSM)**	Reserved for (S,G) delivery (10.6)
233.0.0.0/8	GLOP	Static ranges mapped from 16-bit AS numbers
239.0.0.0/8	**Administratively scoped**	The "private addresses" of multicast; use inside your organization

Mapping a Group to a MAC Address



An Ethernet NIC needs a **MAC** to filter on. IPv4 multicast MACs are built like this:

- The first 25 bits are fixed: **01:00:5e** plus one 0 bit.
- The **low-order 23 bits** of the group address are copied in.

A group address has 28 significant bits (32 minus the fixed `1110` prefix). Only 23 fit, so **5 bits are lost**, and **2^5 = 32 different groups map to the same MAC**.

Worked example: 239.1.1.1 → `01:00:5e:01:01:01`. And 224.**129**.1.1? The second octet, 129, is `1000 0001`. Its top bit falls into the 5 lost bits, leaving `000 0001` = 01, so this group also becomes `01:00:5e:01:01:01`.

Why does this overlap matter? A host that joined 239.1.1.1 will have its NIC accept frames for 224.129.1.1 too. The IP stack discards them, but only after the CPU has looked at them. It's mostly harmless, but it's why addressing plans avoid groups that differ only in those 5 bits. Exam favorite: "how many IP groups share one multicast MAC?" The answer is 32.

(IPv6 multicast is FF00::/8, mapped to MAC **33:33** plus the low 32 bits. IPv6 uses **MLD** instead of IGMP: MLDv1 ≈ IGMPv2, MLDv2 ≈ IGMPv3.)

10.3 IGMP — Hosts Asking for Groups

The problem: the router at the edge of a subnet has to know whether anyone on that subnet wants a group. If nobody does, it shouldn't waste the link by forwarding the stream there. **IGMP (Internet Group Management Protocol)** is how hosts tell their local router "I want group G" and "I'm finished with G".

IGMP runs between hosts and their **local router** (IP protocol 2). It never crosses a router.

IGMPv2

Message	Sent to	Purpose
Membership Report (join)	The group itself	"I want group G". Sent unsolicited when joining, and in answer to queries
General Query	224.0.0.1	Router asks: "Does anyone still want any groups?" Every **60 s** by default on IOS (RFC default 125 s). Hosts must answer within the **max response time, 10 s** by default
Group-Specific Query	The group	"Does anyone still want G?" Sent after a Leave
Leave Group	**224.0.0.2**	"I'm done with G" (new in v2)

Why does a router send a group-specific query after a Leave, instead of just pruning? Because IGMP tracks whether anyone on the subnet wants a group, not who. One host leaving tells the router nothing about the other 30 hosts on the VLAN. So the router asks once more, using a short **max response time** (the last member query interval, 1 s by default on IOS, sent twice). If nobody answers, the router stops forwarding. This **low leave latency** is what v2 added over v1, where a router only noticed silence after several minutes.

Worked example — two viewers on VLAN 10, one leaves (IGMPv2, IOS defaults).

1. PC-A (10.10.10.21) opens the video app and sends an unsolicited **Membership Report** for 239.1.1.1. R1 starts forwarding the stream onto VLAN 10.

2. PC-B (10.10.10.22) opens the same stream. It sends its own report. R1 is already forwarding, so nothing changes.
3. Every 60 s R1 sends a **General Query** to 224.0.0.1. Each PC sets a random timer within 10 s. PC-B's fires first and it reports. PC-A hears that report and **suppresses** its own. R1 only needed one "yes".
4. PC-B closes the app and sends **Leave** for 239.1.1.1 to 224.0.0.2.
5. R1 sends a **Group-Specific Query** for 239.1.1.1 ("anyone still want this?"), with a 1 s response time, and repeats it once.
6. PC-A is still watching, so it **reports**. R1 keeps forwarding. Had PC-A also left, nobody would answer, and about 2 s after the Leave R1 would stop forwarding onto VLAN 10.

Don't confuse this with the switch feature called immediate-leave (sometimes "fast-leave") in IGMP snooping, below. That one removes a port the instant a Leave arrives, with no query at all, and is only safe when a single host sits behind each port.

The querier: if a subnet has several routers, only one should send queries. In IGMPv2 the **lowest IP address** wins the querier election. (In IGMPv1 there was no election, so the PIM DR did it.)

Report suppression (v2): when a host hears another host's report for the same group, it suppresses its own. The router only needs one "yes" per group.

IGMPv3

Change	Why it matters
Reports go to **224.0.0.22**	Routers and snooping switches hear every report
No report suppression	Every host reports, so the router can track individual members (precise, fast leaves)
No separate Leave message	Leaving is a **state-change report** (for example "change G to INCLUDE {}", or BLOCK a source). The router then sends a group-and-source-specific query
Source filtering: INCLUDE / EXCLUDE source lists	A host can say "group G, but **only from source S**". This is the foundation of **SSM** (10.6)

IGMPv2 says "I want G". IGMPv3 says "I want G from S", and that one extra word removes the need for an RP.

Worked example — what an IGMPv3 report says. A set-top box wants channel 232.1.1.1, but only the copy sent by the head-end server 10.5.5.10. It sends a report to 224.0.0.22 meaning **"232.1.1.1, INCLUDE {10.5.5.10}"**. If some other machine, say 10.9.9.9, starts sending to 232.1.1.1, the router won't forward it to this box, because 10.9.9.9 isn't on the include list. When the box changes channel, it sends a state-change report meaning "232.1.1.1, INCLUDE {}" (an empty list, so nothing wanted), and that is how it leaves.

IGMP Snooping (Layer 2)

A switch treats multicast frames like broadcasts by default and **floods** them to every port in the VLAN. A 20 Mbps stream then hits every PC whether or not anyone is watching. **IGMP snooping** has the switch listen to IGMP reports passing through it and build a table mapping each group to the ports that asked for it. It then forwards each group only to those ports, plus the **mrouter port** facing the router. Snooping is on by default on Catalyst switches.

Worked example. SW1 has the router on Gi1/0/48, PC-A on Gi1/0/1, PC-B on Gi1/0/2 and 20 other PCs on the rest of VLAN 10.

1. PC-A sends a report for 239.1.1.1. SW1 notices it and adds Gi1/0/1 to the group table entry for 239.1.1.1.
2. SW1 already knows Gi1/0/48 is the mrouter port, because the router's queries and PIM hellos arrive there.
3. The stream arrives on Gi1/0/48. SW1 sends it out **only Gi1/0/1**. The other 21 PCs never see it.
4. Without snooping, all 22 ports in VLAN 10 would receive the 20 Mbps stream.

Why must the switch know the mrouter port? Because routers don't send IGMP reports. They send queries and PIM hellos, and the switch identifies the router port from those. If no querier exists on a VLAN (a VLAN with no router), reports eventually time out and snooping breaks. The fix is `ip igmp snooping querier`.

Immediate-leave: `ip igmp snooping vlan 10 immediate-leave` makes the switch drop a port from the group the moment it sees a Leave, skipping the query. It's great for a VLAN where each port has exactly one receiver (an IPTV set-top box, say), and wrong anywhere a port leads to several hosts, because one host leaving would cut off the rest.

Verifying IGMP and snooping:

```
R1# show ip igmp groups                        ! router: groups with members, per interface
R1# show ip igmp interface Gi0/1               ! version, querier, query/max-response timers
SW1# show ip igmp snooping                     ! global and per-VLAN snooping state
SW1# show ip igmp snooping groups              ! group -> ports table the switch built
SW1# show ip igmp snooping mrouter             ! which port leads to the multicast router
SW1# show ip igmp snooping querier             ! who is querying in each VLAN
```

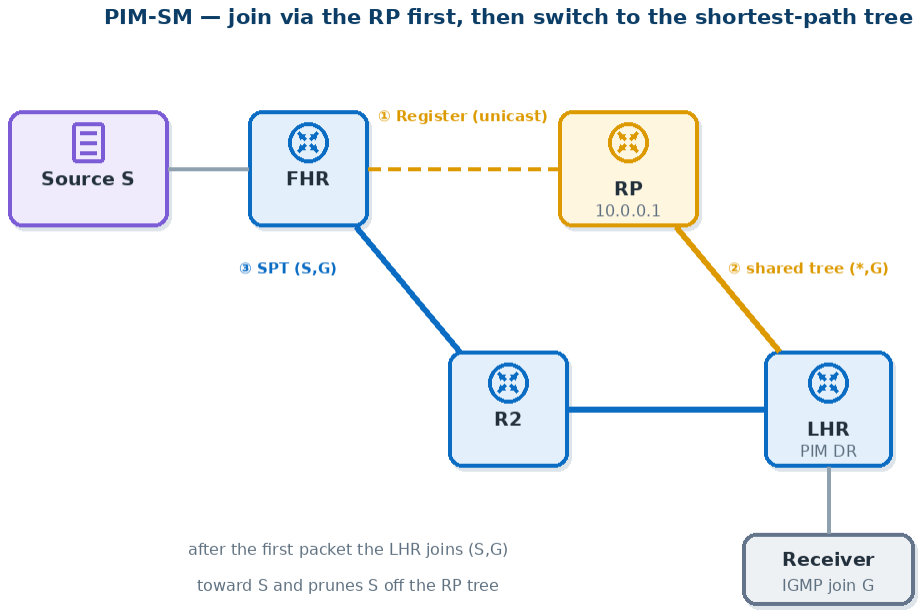
If `show ip igmp snooping groups` lists a group but no ports, or `mrouter` shows nothing, the stream either floods or never reaches the router. Those two outputs pin down most Layer 2 multicast faults.

10.4 PIM — Building the Tree Between Routers

The problem: IGMP only tells the last router that a receiver exists. Somewhere else, maybe five routers away, sits the source. The routers in between must build a path that carries the stream from the source to every router with a receiver, without loops and without sending it where nobody wants it. That's PIM's job.

PIM (Protocol Independent Multicast) is IP protocol **103**, with hellos to **224.0.0.13** every **30 s**. "Protocol independent" means PIM has **no routing table of its own**. It borrows the **unicast** routing table (whatever OSPF, EIGRP, BGP or static routes put there) to work out which direction a source or RP lies in.

Two Kinds of Tree



Tree	Notation	Root	Pros / cons
Source tree / shortest-path tree (SPT)	**(S,G)**	The source	Optimal path; state for every source × group
Shared tree (RPT)	**(*,G)**	The **Rendezvous Point (RP)**	One tree per group regardless of source; path may detour via the RP

How to read the notation: (S,G) = "traffic from source S to group G", for example (10.5.5.10, 239.1.1.1). **(*,G)** = "traffic to group G from any source", where the `*` is a wildcard.

Dense Mode vs Sparse Mode

Mode	Assumption	Mechanism
PIM Dense Mode (DM)	"Receivers are everywhere"	**Flood and prune**: send everywhere, routers with no receivers send Prunes. Prunes expire (≈3 min) and the flood repeats
PIM Sparse Mode (SM)	"Receivers are few; nobody gets traffic unless they ask"	**Explicit join** toward an RP; traffic flows only down branches that joined

Why is dense mode essentially obsolete? Because flood-and-prune puts every stream on every link at least every few minutes, and keeps state on routers that never wanted it. Sparse mode's "pull, don't push" model scales, which is why it's the enterprise default and the one ENCOR names.

PIM Neighbors, the DR and the Assert

PIM routers find each other with hellos to 224.0.0.13 (`show ip pim neighbor`). On a **multi-access** segment with several PIM routers, two elections decide who does what:

The Designated Router (DR). One router per segment is elected **DR**:

1. Highest **DR priority** wins (`ip pim dr-priority`, default **1**).
2. Tie: highest IP address.

On a **receiver** segment, the DR is the router that turns IGMP reports into PIM Joins (so only one Join goes upstream). On a **source** segment, the DR is the one that sends PIM Register messages to the RP. `show ip pim interface` shows the DR for each interface.

Worked example. VLAN 10 has two PIM routers: R1 (10.10.10.2) and R2 (10.10.10.3), both at the default DR priority 1. The tie goes to the higher IP, so **R2 is DR**. Receivers on VLAN 10 send IGMP reports, both routers hear them, but only R2 sends the PIM Join upstream. To make R1 the DR (perhaps it has the better uplink), set `ip pim dr-priority 10` on R1's VLAN 10 interface. 10 beats 1, so R1 becomes DR.

The Assert. Sometimes two routers both forward the same (S,G) onto one LAN, for example two routers each with a downstream receiver segment that is the same VLAN. Each sees the other's copies arriving on its own outgoing interface, which means the LAN is getting every packet twice. They exchange **PIM Assert** messages, and the winner is:

1. Lowest **administrative distance** of the route back to the source (or RP).
2. Tie: lowest **metric** to the source.
3. Tie: highest IP address.

The loser stops forwarding onto that LAN. The DR decides who asks for traffic; the Assert decides who delivers it when two routers end up delivering.

Worked example. R1 and R2 both forward (10.5.5.10, 239.1.1.1) onto VLAN 30. R1's route to 10.5.5.10 is OSPF (AD 110). R2's is EIGRP (AD 90). They exchange Asserts. Step 1 already decides

it: **R2 wins** with the lower AD, so R1 removes VLAN 30 from its outgoing list for that (S,G). VLAN 30 now receives one copy of each packet instead of two.

10.5 PIM Sparse Mode Step by Step

Picture a source S, its **first-hop router (FHR)**, an **RP**, and a receiver behind its **last-hop router (LHR)**.

Story Time : The RP is a noticeboard in a town square. People who want a club (receivers) pin up "interested in chess club" there. People starting a club (sources) also go to the noticeboard to announce "chess club, meeting at my house". The noticeboard introduces them. Once a member has the organiser's address, they go straight to the organiser's house instead of passing through the square every week. That's the **SPT switchover**. Where it breaks: at first, the RP actually carries the traffic itself (the shared tree), not just the introduction. The direct route only takes over after the first packets have arrived.

Concrete addresses for the walk-through: source S = 10.5.5.10 behind FHR R1, the RP = R2 (10.0.0.1), the receiver PC = 10.20.20.50 behind LHR R4, group G = 239.1.1.1.

1. **Receiver joins.** The host sends an IGMP report for G. The LHR (the PIM **DR** on that segment) sends a **PIM (*,G) Join** hop by hop **toward the RP**. Every router along the way adds (*,G) state, which builds the **shared tree**.
2. **Source starts sending.** The FHR doesn't yet know if anyone wants the traffic. It wraps the multicast packets in unicast **PIM Register** messages and sends them straight to the RP.
3. **RP joins toward the source.** The RP has receivers for G, so it de-encapsulates the packets, forwards them down the shared tree, and sends an **(S,G) Join** toward S. Once native multicast arrives along that path, the RP sends **Register-Stop** to the FHR, which ends the tunnelled phase.
4. **SPT switchover.** When the LHR receives the first packet, it now knows S's address. It sends an **(S,G) Join directly toward S** and, once traffic arrives on that path, prunes S off the shared tree (an **(S,G) RPT-bit prune** toward the RP). On IOS the default **SPT threshold is 0**: switch immediately on the first packet.

The same four steps with the addresses above:

Step	Who	Sends / builds	State created
1	PC 10.20.20.50 → R4	IGMP report for 239.1.1.1; R4 sends (*, 239.1.1.1) Join toward 10.0.0.1	(*, 239.1.1.1) on R4 and every router up to R2
2	10.5.5.10 → R1	R1 tunnels packets in unicast Register messages to 10.0.0.1	(10.5.5.10, 239.1.1.1) on R1

Step	Who	Sends / builds	State created
3	R2 (RP)	Forwards packets down the shared tree; sends (10.5.5.10, 239.1.1.1) Join toward R1; then Register-Stop	(S,G) on R2 and the routers toward R1
4	R4	First packet reveals S = 10.5.5.10. Sends (S,G) Join straight toward R1; prunes S off the RP tree	(10.5.5.10, 239.1.1.1) with flag **T** on R4

Why bother with a shared tree at all if everyone switches to the SPT immediately? Because of **source discovery**. A receiver knows the group but not the sources, and the LHR can't join a tree toward a source it has never heard of. The RP is the **meeting place** where sources register and receivers join. Once the first packet has introduced the source, the LHR builds the shortest path straight to it. The RP is the introduction, not the long-term delivery path.

The RPF Check — Multicast's Loop Prevention

Unicast forwarding asks "where is the destination?" Multicast can't ask that, because the destination is a group spread across the network. It asks the reverse question:

Reverse Path Forwarding (RPF) check: when a multicast packet arrives, look up the **source** address (or the **RP** address, for shared-tree traffic) in the **unicast** routing table. If the packet arrived on the interface this router would use to send traffic back toward that source, it **passes** and is replicated downstream. If it arrived on any other interface, it is **dropped**.

Why does this prevent loops? In a meshed network, copies of the same packet can arrive over several paths. Only the copy arriving on the shortest path back to the source passes, and every duplicate or looping copy fails the check. That guarantees a tree, which has no loops by definition. Multicast flows away from the source along paths the unicast table already agrees on.

Story Time : Imagine water flowing downhill from a spring. If you find water in a channel flowing uphill toward the spring, something is wrong: it has looped around. RPF is the router checking "did this packet arrive from the direction of its spring (the source)?" If not, it's looping water, so it's thrown away.

Worked example — RPF check on R3. R3 has two interfaces that could receive (10.5.5.10, 239.1.1.1): Gi0/0 and Gi0/1. Its unicast table says:

```
R3# show ip route 10.5.5.10
Routing entry for 10.5.5.0/24
  Known via "ospf 1", distance 110, metric 20, type intra area
  Routing Descriptor Blocks:
  * 10.1.23.2, from 2.2.2.2, 00:12:40 ago, via GigabitEthernet0/0
```

1. A packet from 10.5.5.10 arrives on **Gi0/0**. R3 looks up the source, 10.5.5.10. The route back to it uses Gi0/0, so the arrival interface matches and the **RPF check passes**. R3 replicates the packet to its outgoing interfaces.
2. A copy of the same packet arrives on **Gi0/1**, having gone round a loop in the mesh. The route back to 10.5.5.10 is not via Gi0/1, so the **RPF check fails** and the packet is dropped.

`show ip rpf 10.5.5.10` shows the same decision directly: `RPF interface: GigabitEthernet0/0`, with RPF neighbor 10.1.23.2.

The classic RPF failure: the unicast table and the multicast path disagree. A typical case is a stream coming in on a link where PIM is enabled, while the unicast route to the source points out a different link where PIM isn't. The packets fail RPF and are silently dropped. The fixes are to enable PIM on the RPF interface, fix the unicast routing, or add a static multicast route (`ip mroute`) to override RPF.

Reading `show ip mroute`

```
R3# show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:04:12/00:02:51, RP 10.0.0.1, flags: SJC
  Incoming interface: GigabitEthernet0/1, RPF nbr 10.1.13.1
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:04:12/00:02:51

(10.5.5.10, 239.1.1.1), 00:03:58/00:02:59, flags: JT
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.1.23.2
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:03:58/00:02:51
```

Item	Meaning
`(*, G)` + `RP 10.0.0.1`	Shared-tree entry, rooted at that RP
`(S, G)`	Source-tree entry for one specific source
Incoming interface / RPF nbr	The interface that passes the RPF check, and the upstream neighbor
Outgoing interface list (OIL)	Where copies are replicated. Empty = nothing downstream wants it
Flags **S** / **C** / **J** / **T**	**S**parse; **C**onnected receiver; **J**oin SPT; **T** = traffic is flowing on the SPT

Seeing an (S,G) entry with the **T** flag at the LHR means the switchover in step 4 happened.

Reading this exact output as a story: R3 has a receiver directly connected (**C**) on Gi0/2. It first joined the shared tree toward RP 10.0.0.1 through Gi0/1. Then it learned source 10.5.5.10 and joined the SPT (**J**). Traffic now arrives on Gi0/0 straight from the source side (**T**) and goes out Gi0/2 to the receiver. The incoming interfaces differ (Gi0/1 toward the RP, Gi0/0 toward the source) because the shortest path to the source doesn't pass through the RP.

10.6 SSM, Bidir and MSDP — Three Variations on PIM-SM

Why variations? Standard PIM-SM is a good general tool, but it has three weak spots. It needs an RP even when receivers already know the source (fixed by **SSM**). It creates per-source state that explodes when everyone is a source (fixed by **Bidir**). And its RP can only introduce sources it knows about, which stops at a domain boundary or at a single RP (fixed by **MSDP**).

Source-Specific Multicast (SSM)

If the receiver already knows the source, which IGMPv3 makes possible, the LHR can **join (S,G) toward the source immediately**. There's **no RP, no shared tree, no Register** step. The SSM range is **232.0.0.0/8**.

```
ip multicast-routing                ! some platforms: ip multicast-routing distributed
ip pim ssm default                  ! treat 232/8 as SSM
interface Vlan10
 ip pim sparse-mode
 ip igmp version 3                  ! receivers must send (S,G) reports
```

Walk-through — SSM with the earlier addresses (group now 232.1.1.1, since SSM uses 232/8):

1. The PC sends an IGMPv3 report: "232.1.1.1, INCLUDE {10.5.5.10}".
2. R4 sees a group in the SSM range, so it doesn't look for an RP. It sends a **(10.5.5.10, 232.1.1.1) Join** hop by hop toward 10.5.5.10, following the unicast route (and the RPF check).
3. Every router on that path creates (S,G) state. When the Join reaches R1, R1 starts forwarding the source's packets down the tree.
4. That's the whole process: no Register, no shared tree, no switchover. Three steps instead of four, and none of them depend on an RP.

Why is SSM increasingly the default for one-to-many services (IPTV, market data)? Because the RP is the most complex and fragile part of PIM-SM. It's a single meeting point you have to place, make redundant and keep reachable. SSM removes it entirely. As a bonus, a host can't receive a stream from an unexpected source sending to the same group, because the subscription names the source.

Bidirectional PIM (Bidir)

For **many-to-many** applications (each participant is both source and receiver), PIM-SM would create (S,G) state for every participant. **Bidir PIM** keeps **only (*,G)** state. Traffic flows **up** the shared tree toward the RP and **down** to receivers, with no Register messages and no SPT switchover. Since packets travel in both directions, the normal RPF check can't apply. Instead, each link elects a **Designated Forwarder (DF)**, the router allowed to forward traffic toward the RP on that link. Bidir trades optimal paths for very small state, which is the right trade when there are thousands of sources.

Worked example — counting state. A trading floor has 500 workstations, and each one both sends and receives on group 239.10.10.10. With PIM-SM, every router in the tree could hold **1 (*,G) + 500 (S,G) = 501 entries** for that one group. With Bidir it holds **1 (*,G) entry**, whatever the number of senders. Each link's DF (the router with the best unicast route to the RP on that link) is the only one allowed to forward packets upstream toward the RP, which keeps the tree loop-free without the per-source RPF check.

MSDP — Connecting RPs

Each PIM-SM domain has its own RP, so a receiver in domain A can't discover a source in domain B: they register with different RPs. **MSDP (Multicast Source Discovery Protocol)** runs between RPs over **TCP 639**. When a source registers with one RP, that RP sends **Source-Active (SA)** messages to its MSDP peers. A peer RP with receivers for that group then joins toward the source across the domain boundary.

Walk-through — two domains. Domain A's RP is RP-A, and domain B's RP is RP-B. They are MSDP peers.

1. Source 10.5.5.10 in domain A starts sending to 239.1.1.1. Its FHR registers with **RP-A**.
2. RP-A sends an **SA message** to RP-B: "source 10.5.5.10 is active for 239.1.1.1, and I'm its RP".
3. RP-B already has (*, 239.1.1.1) state, because receivers in domain B joined its shared tree. It sends an **(10.5.5.10, 239.1.1.1) Join** toward the source, across the boundary.
4. Traffic flows into domain B and down RP-B's shared tree. Receivers' LHRs can then switch to the SPT as usual.

Without step 2, RP-B would never learn the source exists, and domain B's receivers would get nothing.

Its other big use is inside **one** domain: **Anycast RP**. Several routers share the same RP address (a /32 loopback), so sources and receivers reach the nearest one through normal unicast routing. MSDP between those RPs makes sure each one learns sources that registered with the others. That gives you RP redundancy and load sharing, built from ordinary routing.

Worked example — Anycast RP. RP1 (in the east) and RP2 (in the west) both have Loopback1 = **10.0.0.1/32**, and every router uses `ip pim rp-address 10.0.0.1`. An eastern source registers with RP1, because that's the nearest 10.0.0.1. A western receiver joins toward RP2. Without MSDP, RP2 would never hear about the eastern source. With an MSDP peering between their unique loopbacks (for example 10.0.0.11 and 10.0.0.12), RP1 sends an SA to RP2 and RP2 joins toward the source. If RP1 dies, its 10.0.0.1 route disappears, and unicast routing sends everyone to RP2. That failover takes no PIM changes at all.

How Routers Learn the RP

Every router in a PIM-SM domain must agree on which router is the RP for each group, or Joins and Registers head to different places and never meet. There are three ways to tell them:

Method	How	Notes
Static	`ip pim rp-address 10.0.0.1` on every router	Simple and predictable; manual everywhere
Auto-RP (Cisco)	Candidate RPs announce to **224.0.1.39**; a **mapping agent** picks one per group and announces to **224.0.1.40**	Chicken-and-egg: those groups need flooding before an RP is known (`ip pim autorp listener`)

Method	How	Notes
BSR (standard, PIMv2)	A **bootstrap router** is elected; candidate RPs unicast to it; it floods the RP set hop by hop in PIM messages to 224.0.0.13	Standards-based; IPv6 too

---

10.7 Chapter Summary

Question	Protocol / answer
How does a host ask for a group?	**IGMP** (v2: group; v3: group + source)
How does a switch avoid flooding?	**IGMP snooping**
How do routers build delivery trees?	**PIM** (sparse mode; uses the unicast table)
How are loops prevented?	**RPF check**: arrive on the interface back toward the source
How do receivers find unknown sources?	Via the **RP** (shared tree), then SPT switchover
How do you skip the RP?	**SSM** (232/8 + IGMPv3)
Many-to-many with tiny state?	**Bidir PIM** (*,G) only, DF per link
How do RPs share sources?	**MSDP** (TCP 639), also used for Anycast RP
Groups per multicast MAC?	**32** (5 bits lost)

Chapter 11: QoS — Quality of Service

> **Blueprint:** ENCOR 1.4 — Interpret QoS configurations

The CCNA guide (17.7) explained why QoS exists: voice can't tolerate delay or jitter, a backup job will happily eat every bit of a link, and tail drop causes TCP global synchronization. ENCOR's verb is **interpret**. You'll be shown a QoS configuration, or the `show policy-map interface` output that comes with it, and asked what it does to a given packet. This chapter builds that skill. It first fills in the full marking vocabulary and the toolkit at CCNP depth, then reads a realistic WAN-edge policy line by line.

11.1 The Three QoS Models

The problem: a link can only send one packet at a time. When more traffic arrives than the link can carry, packets have to wait in a queue, and when the queue is full they are dropped. **QoS (Quality of Service)** is how you decide who waits and who gets dropped, so a phone call stays clear while a backup job slows down a little. A QoS model is the overall strategy for making those decisions across a whole network.

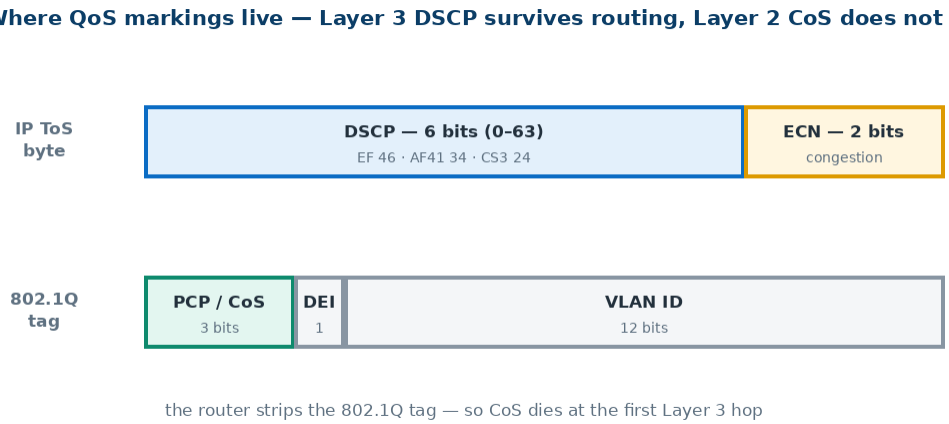
Story Time : Think of an airport. **Best effort** is one queue for everyone at every gate. **IntServ** is like booking a reserved slot at every security checkpoint, gate and runway along your whole route before you're allowed to leave home: perfect service, but every airport must keep a list of every passenger, and that's impossible at scale. **DiffServ** is printing a boarding group on each ticket at check-in ("Group 1", "Group 4"). Every gate just reads the ticket and follows the same simple rule: Group 1 boards first. No gate needs to know anything about you except the number on your ticket. Where it breaks: boarding group 1 doesn't guarantee you a seat; it only guarantees you go before group 4. DiffServ guarantees are **relative**, as the next paragraph explains.

Model	Idea	Reality
Best effort	No QoS. First in, first out	The default; fine until a link congests
IntServ (Integrated Services)	Each application **reserves** bandwidth end to end with **RSVP** before sending; every router keeps per-flow state	Strict guarantees, but per-flow state on every hop doesn't scale to an enterprise or the internet
DiffServ (Differentiated Services)	Traffic is **classified and marked** at the edge into a handful of classes; each hop applies a **per-hop behavior (PHB)** based on the marking	Scales because routers keep **no per-flow state**. It's the model behind everything below

Why did DiffServ win? Because state is what kills scale. IntServ asks every router in the path to remember every conversation. DiffServ asks every router to remember about **eight classes** and read one field in each packet. The trade-off is that DiffServ gives **relative** guarantees ("voice goes first") rather than absolute ones ("this call gets 80 kbps end to end"). For an enterprise, that's almost always the right trade.

11.2 Classification and Marking — the Vocabulary

The problem: working out what a packet is (a voice call? a backup? YouTube?) takes effort: looking at ports, addresses, even payloads. Doing that on every router along the path would waste CPU and could give different answers at different hops. So you do it **once**, at the edge (**classification**), and write the answer into the packet itself (**marking**). Every later hop just reads the mark. That's the "ticket" from the Story Time above.



Field	Where	Size	Values
CoS / PCP	802.1Q tag (Layer 2)	3 bits	0–7. **Disappears at the first router**, since the tag doesn't survive routing
IP Precedence	ToS byte, top 3 bits (legacy)	3 bits	0–7
DSCP	ToS byte, top **6** bits (Layer 3)	6 bits	0–63. Survives end to end across routed hops
ECN	ToS byte, bottom 2 bits	2 bits	Congestion signalling without dropping (11.6)

Why mark at Layer 3 rather than relying on CoS? CoS lives in the 802.1Q tag, and a router strips the tag when it routes the packet. So CoS can't describe a packet's class past the first Layer 3 hop. DSCP is part of the IP header, so it survives routing, which makes it the marking that matters end to end. Switches commonly map one to the other at the boundary.

The Standard PHBs

PHB	DSCP name	Decimal	Binary	Meaning / typical use
Default (BE)	DF / CS0	0	000000	Best effort
EF — Expedited Forwarding	EF	**46**	101110	Low delay, low jitter, low loss: **voice**
AF — Assured Forwarding	AFxy	8x + 2y	—	4 classes × 3 drop precedences
CS — Class Selector	CS1–CS7	**8 × n**	nnn000	Backward-compatible with IP Precedence (CS3 = 24 = Precedence 3)

The AF Table (Memorize the Pattern, Not the Numbers)

Class	Low drop (y=1)	Medium drop (y=2)	High drop (y=3)	Common use
AF1x	AF11 = **10**	AF12 = 12	AF13 = 14	Bulk data
AF2x	AF21 = **18**	AF22 = 20	AF23 = 22	Transactional data
AF3x	AF31 = **26**	AF32 = 28	AF33 = 30	Multimedia streaming
AF4x	AF41 = **34**	AF42 = 36	AF43 = 38	Interactive video

The formula is DSCP = 8x + 2y. For AF32: 8×3 + 2×2 = 28. The **first digit (x) is the class**, which decides the queue. The **second digit (y) is drop precedence** within **that class**, which decides who gets dropped first when that queue fills. **Exam trap:** AF13 isn't "better" than AF11. Same queue, but AF13 is dropped **first**. Within an AF class, a higher y means a higher drop probability.

Worked example — decoding markings both ways.

You see	Work it out	Answer
DSCP 34	34 = 8×4 + 2×1, so x = 4 and y = 1	**AF41** (interactive video, low drop)
DSCP 24	24 = 8 × 3	**CS3** (call signalling), same as IP Precedence 3
AF22	8×2 + 2×2	**20**
EF in a packet capture	DSCP 46 = `101110`; add the 2 ECN bits (`00`) and the whole ToS byte is `10111000`	ToS byte **0xB8** (184). Wireshark often shows this hex value

The last row is a common source of confusion: a capture tool may show "ToS 0xB8" or "DS field 0xb8", and that is EF. Divide the ToS byte by 4 (drop the two ECN bits) to get DSCP: 184 ÷ 4 = 46.

Why have drop precedence at all? It lets you be gentle with traffic that exceeds its share instead of brutal. A policer can **remark** out-of-contract traffic from AF11 to AF13 rather than dropping it. It still travels in the same queue, but if that queue congests, the excess traffic is first in line to be dropped (11.6). Over your limit, you still get on the bus, but you'll be the first one asked to leave.

Classification Tools

- **ACLs** and header fields (ports, addresses) for traffic you can identify by port.
- **Incoming markings** (`match dscp ef`) when the source is trusted.
- **NBAR / NBAR2** (Network-Based Application Recognition): deep packet inspection that identifies **applications** (Webex, Teams, YouTube, BitTorrent) even when they share port 443. Use `match protocol webex-meeting`. Run `ip nbar protocol-discovery` on an interface to see what's actually on the link.

Trust Boundaries

The trust boundary is the point where you start believing markings. Everything outside it is remarked.

Where	Typical policy
Cisco IP phone port	Trust the phone (often conditionally: only if CDP/LLDP sees a phone); the phone remarks the attached PC's traffic to 0
User PC port	Don't trust; classify and mark yourself (or set to 0)
Uplinks between your own switches/routers	Trust (the edge already did the work)
Internet / partner links	Don't trust; remark

Why not trust PCs? Because any user or piece of software can set DSCP 46 on a file transfer and jump the voice queue. Marking is an honor system. You trust only devices you control.

Walk-through — a phone with a PC plugged into it (the switch port is configured for conditional trust):

1. The IP phone sends voice packets already marked **EF (46)** and CoS 5. The switch sees the phone via CDP/LLDP, so it trusts the phone's marking and keeps EF.
2. The PC behind the phone sends a file upload marked EF, perhaps by misbehaving software. The phone's built-in switch **re-marks the PC's frames to CoS 0** before passing them on (Cisco phones don't trust the attached PC by default), so a switch that trusts CoS on this port sees 0, not 5. If the port trusts DSCP instead, the PC's DSCP passes through the phone unchanged, which is why many designs classify PC traffic with an ingress policy (11.3) rather than trusting it.
3. Someone unplugs the phone and plugs a laptop straight into the port. CDP no longer sees a phone, so the switch **stops trusting**, and the laptop's markings are not believed.

The goal in every case: only the real phone's voice reaches the priority queue. **Platform note:** older Catalyst switches with `mls qos` enabled treat every port as untrusted until you say otherwise. Catalyst 9000 switches preserve (trust) incoming markings by default, so on those you must explicitly remark untrusted ports.

11.3 MQC — the Three-Part Structure Every Config Uses

Why a structure at all? Older IOS had a different command for every QoS tool, each with its own syntax. **MQC (Modular QoS CLI)** replaced them with one pattern that separates which traffic, what to do and where to do it, so the same class can be reused in several policies and the same policy on many interfaces.

Every modern Cisco QoS configuration uses the **Modular QoS CLI**. Learn its three parts and you can read any of them:

Part	Command	Answers
1. **Class map**	`class-map [match-any|match-all] NAME` + `match …`	**Which** traffic?
2. **Policy map**	`policy-map NAME` + `class NAME` + actions	**What** happens to it?
3. **Service policy**	`service-policy {input|output} NAME` on an interface	**Where** and in which direction?

- **`match-all`** (the default!) requires **every** `match` line to be true (logical AND). **`match-any`** needs just one (OR). **Exam trap:** a `match-all` class with `match dscp ef` and `match dscp cs3` matches nothing, because one packet can't carry two DSCP values.
- Classes in a policy are checked **top to bottom; first match wins**.
- Anything unmatched falls into **`class-default`**, which always exists, even if you don't configure it.

Marking on Ingress — Reading a Classification Policy

Queuing policies (11.5, 11.7) act on markings, but someone has to set them first, at the trust boundary, **inbound** on the access port. ENCOR shows you policies like this one and asks what happens to a given packet:

```
class-map match-any VOICE-IN
 match protocol rtp audio                    ! NBAR: identify by application
class-map match-any WEBEX-IN
 match protocol webex-meeting
class-map match-all BULK-IN
 match access-group name BACKUP-SERVERS      ! ACL: identify by address/port
!
policy-map MARK-ACCESS-IN
 class VOICE-IN
  set dscp ef                                ! Layer 3 marking, survives routing
 class WEBEX-IN
  set dscp af41
 class BULK-IN
  set dscp af11
  set qos-group 10                           ! internal label, never leaves this box
 class class-default
  set dscp default                           ! untrusted: wipe whatever the PC sent
!
interface GigabitEthernet1/0/10
 description user port
 service-policy input MARK-ACCESS-IN
```

[redacted] action	What it changes	Lifetime
`set dscp X`	The 6-bit DSCP in the IP header	End to end, until someone re-marks it
`set cos X`	The 3-bit PCP in the 802.1Q tag	Only while the frame stays tagged. Gone at the first router or access port

action	What it changes	Lifetime
`set qos-group N`	An **internal** label inside this device	Never leaves the box. Lets an output policy match on a decision made at input (`match qos-group 10`) without changing the packet
`set precedence X`	The top 3 bits of the ToS byte (legacy)	End to end

Worked example — four packets arrive on Gi1/0/10. The policy checks the classes top to bottom and stops at the first match:

Packet	VOICE-IN?	WEBEX-IN?	BULK-IN?	Result
RTP audio from a softphone, PC marked 0	**yes**	(not checked)	(not checked)	Marked **EF**
Webex meeting traffic, PC marked EF	no	**yes**	(not checked)	Re-marked **AF41**
Backup to a server in BACKUP-SERVERS, marked 0	no	no	**yes**	Marked **AF11**, internal qos-group 10
Web browsing that the PC marked EF	no	no	no	class-default: reset to **0**

Reading it: a Webex packet the PC marked EF is re-marked to AF41, because the policy classifies by **application** and ignores what the PC claimed. An unknown packet marked EF is reset to 0 by `class-default`. An ingress marking policy is how an untrusted port is "re-colored" to match your design.

11.4 Policing vs Shaping at CCNP Depth

The problem: sometimes the issue isn't who goes first but how much is allowed through. An ISP sells you 50 Mbps on a 1 Gbps port, or you want guests limited to 10 Mbps. You need a tool that measures a traffic rate and acts on whatever goes over it.

The CCNA guide gave the core difference: **policing drops (or remarks) the excess; shaping buffers it.** CCNP adds the mechanics.

Story Time : A nightclub allows 100 people in per hour. A **policer** is a bouncer: once the hour's quota is used, anyone else arriving is turned away (dropped), or let in with a stamp saying "first out if it gets crowded" (re-marked). A **shaper** is a velvet rope with a queue: extra people wait outside and are let in steadily as space frees up. Nobody is turned away unless the queue outside gets too long, but everybody in the queue waits. That's why shaping adds **delay** and policing adds **loss**.

The Token Bucket

Both tools use a **token bucket**. Tokens drip in at the **CIR** (committed information rate), sending traffic spends tokens, and the bucket's size is the **Bc** (committed burst). With tokens available, traffic **conforms**. Without them, it **exceeds**.

Units trap: IOS counts the buckets differently for the two tools. On `shape`, Bc and Be are in **bits**. On `police`, `bc` and `be` are in **bytes**. The CIR is in bits per second for both. So `shape average 50000000 200000` means a 200,000-bit burst, while `police cir 10000000 bc 312500` means a 312,500-byte (2.5 Mbit) burst. Read the command before you do the arithmetic.

Term	Means
CIR	The average rate being enforced (bps)
Bc	Committed burst: the amount allowed in one interval (**bits** on `shape`, **bytes** on `police`)
Be	Excess burst: extra allowed if previous intervals were under-used (same units as Bc)
Tc	Time interval = **Bc (bits) ÷ CIR (bps)**. A shaper releases Bc bits every Tc

Worked example — a shaper's rhythm. `shape average 50000000` with Bc = 200,000 bits:

1. Tc = 200,000 ÷ 50,000,000 = **0.004 s = 4 ms**.
2. Every 4 ms, the shaper may send 200,000 bits = 25,000 bytes, which is about **16 full-size (1,500-byte) packets**.
3. The physical port runs at 1 Gbps, so those 16 packets leave in about 0.2 ms. The shaper then stays quiet for the rest of the 4 ms.
4. Over one second that's 250 intervals × 200,000 bits = **50,000,000 bits**, exactly the CIR. Anything more waits in the shaping queue for the next interval.

Worked example — a three-color policer handling a burst. `police cir 10000000 bc 312500 be 312500` (bytes). Both buckets start full, and refill is ignored for this instantaneous burst:

1. A burst of **700,000 bytes** arrives at once.
2. The first **312,500 bytes** take tokens from the Bc bucket → **conform** → transmitted.
3. The Bc bucket is empty, so the next **312,500 bytes** take tokens from the Be bucket → **exceed** → sent, re-marked AF13.
4. Both buckets are empty, so the last **75,000 bytes** → **violate** → dropped.
5. Tokens drip back into Bc at 10 Mbps (1,250,000 bytes per second). Tokens that overflow a full Bc bucket fill Be, so the policer allows a burst again only after a quiet spell.

Why Bc = 312,500 bytes? It equals 2.5 Mbit, which is **250 ms** worth of traffic at 10 Mbps (2.5 ÷ 10). That's the comment on the config line.

Policer Flavors

Policer	Rates / buckets	Results
Single-rate, two-color	CIR, Bc	conform / exceed
Single-rate, three-color (RFC 2697)	CIR, Bc, Be	conform / exceed / **violate**
Two-rate, three-color (RFC 2698)	CIR **and PIR**	conform / exceed / violate

```
policy-map POLICE-GUEST
 class class-default
  police cir 10000000 bc 312500 be 312500   ! 10 Mbps; bc/be in BYTES (250 ms)
   conform-action transmit
   exceed-action set-dscp-transmit af13      ! remark, don't drop: drop first later
   violate-action drop
```

Choosing Between Them

Situation	Tool	Why
ISP enforcing your contract **inbound** at their edge	Policing	They don't want to buffer your excess
Your router sending into a 50 Mbps circuit delivered on a 1 Gbps port	**Shaping** (outbound)	Smooth to the contract rate yourself, or the ISP's policer drops it (and drops it blindly)
Limiting a guest VLAN or a scavenger class	Policing	Cheap, no buffering, no added delay
Voice/video	Neither on the voice itself, beyond the LLQ's built-in policer	Shaping adds delay. Voice is protected by the priority queue instead

Why does shaping only work outbound? Because buffering means holding packets you haven't sent yet, and you can only hold what's waiting to leave. Traffic arriving inbound has already crossed the wire. You can drop or remark it, but you can't make it arrive later.

11.5 Queuing — Who Leaves First

The problem: when packets pile up, the order they leave in decides the experience. A voice packet stuck behind 50 big file-transfer packets arrives late, and late voice is as bad as lost voice. Queuing sorts waiting packets into separate lines and decides which line is served next.

Queuing only matters during **congestion**, when the interface's hardware transmit ring is full and packets wait in software queues. With no congestion, every QoS queuing policy is idle.

Story Time : A supermarket with several checkout lanes and one cashier who moves between them. **CBWFQ** is a rule like "serve the business lane 30% of the time, the regular lane 25%...": every lane is guaranteed its share of the cashier. **LLQ** adds an express lane with a special rule: whenever anyone is in it, they're served next. But the express lane is only allowed a limited

number of customers per minute, so it can never take the cashier away from everyone else for good.

Walk-through — how LLQ serves packets on a congested link:

1. Packets are sorted into their class queues as they arrive (voice → priority queue; video, data and default → their own CBWFQ queues).
2. Each time the interface is ready to send, the scheduler **checks the priority queue first**. If a voice packet is waiting and the voice class is within its cap, it goes next.
3. If the priority queue is empty, the scheduler serves the other queues in proportion to their `bandwidth` guarantees.
4. If voice exceeds its cap while the link is congested, the extra voice packets are **dropped**, not queued.
5. A voice packet therefore waits only for the packet already being transmitted (and any voice packets ahead of it), never for the backlog in the other queues. That's why LLQ voice delay stays tiny.

Mechanism	How it works	Problem it solves / leaves
FIFO	One queue	No differentiation
WFQ	Automatic per-flow fairness	Fair, but can't prioritize voice
CBWFQ	You define classes; each gets a **guaranteed minimum bandwidth** (`bandwidth`)	Guarantees bandwidth, but voice can still wait behind a full-size packet in another queue: **jitter**
LLQ	CBWFQ **plus one strict-priority queue** (`priority`) served before all others	Voice gets near-zero queueing delay

Why does the LLQ have a built-in policer? A strict-priority queue is always served first. If it could take unlimited bandwidth, a misbehaving "voice" class could starve every other class completely. So `priority 10000` (or `priority percent 10`) is both a promise and a **cap**: up to that rate during congestion, and any excess in the priority class is **dropped**. The fast lane is fast because it's small. Cisco's guidance is to keep the total priority (real-time) traffic at or below about **33%** of the link.

`bandwidth` vs `bandwidth remaining`: `bandwidth percent 30` guarantees 30% of the **interface (or shaped) rate**. `bandwidth remaining percent 30` divides what's **left after the priority queue** is served. A single policy can't mix the two styles across its classes.

Worked example. A 100 Mbps link has `priority percent 20` for voice, leaving 80 Mbps. Then:

- `bandwidth percent 30` for data guarantees 30% of **100** = **30 Mbps**.
- `bandwidth remaining percent 30` for data instead guarantees 30% of the **80** left = **24 Mbps**.

Same number, different answer. Read which keyword is used before doing the arithmetic.

Queuing on Catalyst 9000 Switches

Routers queue in software; Catalyst switches queue in **hardware** (the UADP ASIC), so the model is fixed. A Catalyst 9000 port supports up to **2P6Q3T**: **8 egress queues**, of which up to **2 are strict priority**, each with **3 drop thresholds** (WTD, weighted tail drop). An egress policy maps classes onto those queues:

```
policy-map C9K-EGRESS
 class VOICE
  priority level 1                          ! first strict-priority queue
  police rate percent 10                    ! cap it, as with router LLQ
 class VIDEO
  priority level 2                          ! second priority queue, served after level 1
  police rate percent 20
 class SIGNALING
  bandwidth remaining ratio 5               ! share of what the PQs leave
 class CRITICAL-DATA
  bandwidth remaining ratio 25
  queue-buffers ratio 20                    ! give this queue a bigger buffer slice
 class class-default
  bandwidth remaining ratio 25
!
interface GigabitEthernet1/0/49
 service-policy output C9K-EGRESS
```

Command	Meaning
`priority level 1` / `level 2`	Two strict-priority queues; level 1 is always served before level 2. Voice in 1, video in 2
`police rate percent N`	Caps a priority queue so it can't starve the others (Catalyst doesn't add an implicit LLQ policer the way routers do, so configure it)
`bandwidth remaining ratio N`	Weights for sharing the bandwidth left after the priority queues. Ratios are relative to each other, not percentages of the link
`queue-buffers ratio N`	Share of the port's buffer memory given to this queue. Deeper buffers absorb bursts (good for data), at the cost of delay

Worked example — turning ratios into bandwidth on a 1 Gbps port, assuming voice and video are both using their full caps (10% + 20% = 300 Mbps). That leaves 700 Mbps for the other classes. The ratios are 5 + 25 + 25 = **55** shares:

Class	Ratio	Share of the 700 Mbps
SIGNALING	5	5 ÷ 55 ≈ 9% ≈ **64 Mbps**
CRITICAL-DATA	25	25 ÷ 55 ≈ 45% ≈ **318 Mbps**
class-default	25	25 ÷ 55 ≈ 45% ≈ **318 Mbps**

If voice and video are quiet, the same ratios divide the whole 1 Gbps instead. A ratio is a share of whatever is left, never a fixed rate.

Why two priority queues? Because voice and interactive video are both real-time, but video is bursty and far bigger. In one shared priority queue, a video burst would sit in front of voice

packets. Level 1 for voice and level 2 for video keeps voice ahead of everything, including video.

AutoQoS — Reading What It Generates

Why it exists: a full campus QoS design is dozens of lines per port, and typing it by hand on hundreds of access ports invites mistakes. **AutoQoS** is a macro: one interface command expands into Cisco's recommended trust, classification, marking and queuing config. Exam questions show the command (or the config it produced) and ask what it does:

Command (interface)	Use it on	What it does
`auto qos voip cisco-phone`	Access port with a Cisco IP phone	**Conditional trust**: trusts markings only while CDP detects a Cisco phone (it adds `trust device cisco-phone`), plus ingress and egress policies for voice
`auto qos trust dscp`	Uplinks between your own switches	Trusts incoming DSCP; adds the egress queuing policy
`auto qos voip cisco-softphone`	PCs running softphones	Classifies and polices voice/signalling from an untrusted PC
`auto qos classify [police]`	Untrusted user ports	Classifies by ACL/NBAR and marks; optionally polices

`trust device cisco-phone` is the key line to recognize. If the phone is unplugged and a PC is connected directly, CDP no longer sees a phone, so the switch stops trusting and the PC's self-marked EF is ignored. The generated class-maps and policy-maps have names beginning `AutoQos-4.0-` (for example an input policy for Cisco phones and a shared output queuing policy). `show auto qos interface Gi1/0/10` lists what was applied, and `show running-config` shows the full expansion. AutoQoS doesn't do anything you couldn't type yourself: it's the Cisco-validated design typed for you, and you still need to read it.

11.6 Congestion Avoidance — Dropping Smartly

The problem: queuing decides who waits, but a queue still has a fixed size. Once it's full, something must be dropped, and how you drop matters a lot for TCP.

When a queue fills, new packets are **tail-dropped**. Tail drop hits every TCP flow in the queue at once, all of them back off together, the link goes idle, then all of them ramp up together again. That's **TCP global synchronization**, a sawtooth that wastes capacity. The CCNA guide covered this.

WRED (Weighted Random Early Detection) starts dropping some packets **before** the queue is full, randomly, so individual TCP flows slow down at different times:

WRED parameter	Meaning
Minimum threshold	Queue depth where random drops start
Maximum threshold	Depth where drop probability reaches its maximum; beyond it, everything is dropped (tail drop)
Mark probability denominator	Drop probability at the max threshold: 10 → 1 in 10

"Weighted" means each **DSCP** gets its own thresholds. With `random-detect dscp-based`, AF13 starts being dropped at a shallower queue depth than AF11. That's drop precedence (11.2) put into action.

Story Time : A classroom is getting loud. One approach is to wait until it's deafening and then shout at everyone at once. They all go silent, then all start talking again together, and the cycle repeats. That's tail drop and global synchronization. A smarter teacher starts tapping a few random students on the shoulder as the noise rises, so the volume eases off gradually and never hits the ceiling. That's WRED. "Weighted" means the students already on a warning (AF13) get tapped sooner than the well-behaved ones (AF11).

Worked example — WRED drop probability, using the thresholds from the 11.7 output (both maximum 40 packets, mark probability 1/10 = 10%):

Average queue depth	AF21 (min 32)	AF23 (min 24)
20 packets	0% (below min)	0% (below min)
32 packets	0% (just at min)	$(32 - 24) \div (40 - 24) \times 10\% =$ **5%**
36 packets	$(36 - 32) \div (40 - 32) \times 10\% =$ **5%**	$(36 - 24) \div 16 \times 10\% =$ **7.5%**
above 40	100%: tail drop	100%: tail drop

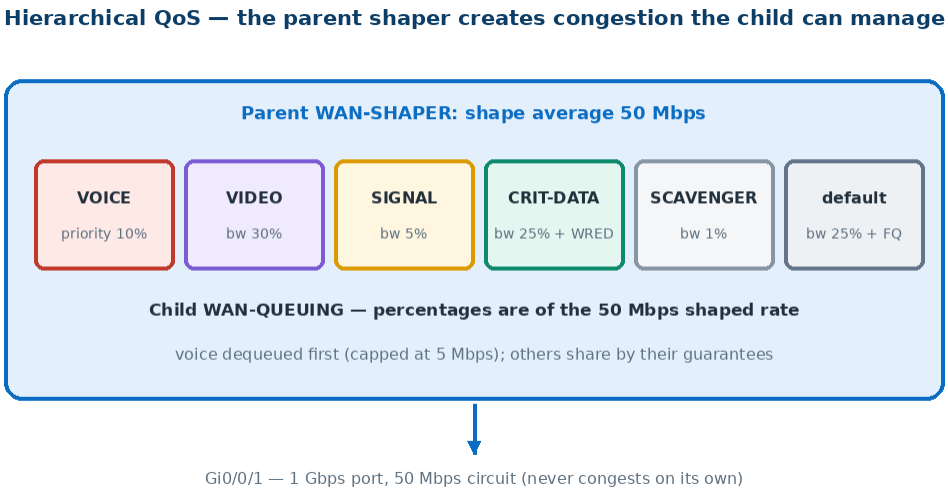
Between the two thresholds the drop chance climbs in a straight line from 0 to the mark probability. WRED uses the **average** queue depth, not the instantaneous one (the `Exp-weight-constant` in the output sets how fast that average moves), so a short burst doesn't trigger drops on its own.

ECN lets WRED **mark** packets (set the ECN bits to "congestion experienced") instead of dropping them, if both endpoints support ECN. The receiver echoes the signal and the sender slows down. The result is the same signal with no lost packet (`random-detect ecn`).

Why not use WRED on the voice class? Voice is UDP and has no congestion response, so dropping voice packets early just degrades the call for no benefit. WRED belongs on **TCP-heavy** classes.

11.7 Interpreting a Real Policy — Line by Line

This is the skill ENCOR tests: given a policy, say what happens to a specific packet and why. Here's a typical **WAN edge**: a 1 Gbps interface handing off to a **50 Mbps** circuit.




```
class-map match-any VOICE
 match dscp ef
class-map match-any VIDEO
 match dscp af41 af42 af43
class-map match-any SIGNALING
 match dscp cs3
class-map match-any CRITICAL-DATA
 match dscp af21 af22 af23
class-map match-all SCAVENGER
 match dscp cs1
!
policy-map WAN-QUEUING                      ! child: the queuing decisions
 class VOICE
  priority percent 10                       ! LLQ: strict priority, capped at 5 Mbps
 class VIDEO
  bandwidth percent 30                      ! guaranteed 15 Mbps during congestion
 class SIGNALING
  bandwidth percent 5
 class CRITICAL-DATA
  bandwidth percent 25
  random-detect dscp-based                  ! WRED: AF23 dropped before AF21
 class SCAVENGER
  bandwidth percent 1                       ! "less than best effort"
 class class-default
  bandwidth percent 25
  fair-queue                                ! flow fairness inside the default class
  random-detect dscp-based
!
policy-map WAN-SHAPER                       ! parent: create congestion at 50 Mbps
 class class-default
  shape average 50000000
  service-policy WAN-QUEUING                ! nest the child inside the shaper
!
interface GigabitEthernet0/0/1
 description 50M MPLS handoff on a 1G port
 service-policy output WAN-SHAPER
```

Reading it like an exam question:

1. **Where and which way?** Outbound on Gi0/0/1, the only direction where queuing and shaping can work.
2. **Why the parent shaper?** This is the crucial insight. The interface is 1 Gbps, so it never congests at 50 Mbps of traffic, and the queuing policy alone would **never activate**. The ISP's policer at 50 Mbps would then drop the excess blindly, voice included. The parent **shapes to 50 Mbps**, which creates congestion locally, where your child policy decides who waits. This

parent/child arrangement is **Hierarchical QoS (HQoS)**. You can only prioritize traffic in a queue you control, so you make the queue happen on your own router.

3. **What does each percentage mean?** They're percentages of the **parent's shaped rate** (50 Mbps), not of 1 Gbps. Voice gets strict priority up to 5 Mbps, video 15 Mbps guaranteed, and so on.
4. **What happens to a packet marked AF22?** `match-any` on CRITICAL-DATA matches it, so it goes into a queue guaranteed 12.5 Mbps, with WRED dropping it ahead of AF21 if the queue builds.
5. **And a packet marked AF31?** No class matches it, so it lands in **class-default**.
6. **What happens to a voice burst at 7 Mbps during congestion?** 5 Mbps goes via the priority queue. The other 2 Mbps is **dropped** by the LLQ's implicit policer, not queued.
7. **When there's no congestion?** Every class may use any spare bandwidth. Guarantees are minimums, enforced only when traffic is contending.

Reading `show policy-map interface`

```
R1# show policy-map interface GigabitEthernet0/0/1 output
 GigabitEthernet0/0/1
  Service-policy output: WAN-SHAPER
    Class-map: class-default (match-any)
      18723401 packets, 9811625540 bytes
      30 second offered rate 48211000 bps, drop rate 212000 bps
      Match: any
      Queueing
      queue limit 208 packets
      (queue depth/total drops/no-buffer drops) 64/30211/0
      shape (average) cir 50000000, bc 200000, be 200000
      target shape rate 50000000

      Service-policy : WAN-QUEUING
        Class-map: VOICE (match-any)
          4011201 packets, 802240200 bytes
          30 second offered rate 3980000 bps, drop rate 0000 bps
          Match: dscp ef (46)
          Priority: 10% (5000 kbps), burst bytes 125000, b/w exceed drops: 0

        Class-map: CRITICAL-DATA (match-any)
          2210334 packets, 1489102776 bytes
          30 second offered rate 14870000 bps, drop rate 96000 bps
          Match: dscp af21 (18) af22 (20) af23 (22)
          Queueing
          (queue depth/total drops/no-buffer drops) 22/8120/0
          bandwidth 25% (12500 kbps)
            Exp-weight-constant: 9 (1/512)
            class  Transmitted  Random drop  Tail drop  Minimum  Maximum  Mark
                                                        thresh   thresh   prob
            af21   1.9M         120          0          32       40       1/10
            af23   0.2M         7990         10         24       40       1/10
```

Line	What it tells you
Parent `offered rate 48211000 bps`	About 48 Mbps offered against a 50 Mbps shaper. The circuit is nearly full
`queue depth 64`, `total drops 30211`	Congestion is real right now. The policy is earning its keep
`shape (average) cir 50000000`, `bc 200000`	Tc = 200,000 bits ÷ 50 Mbps = **4 ms** intervals

Line	What it tells you
VOICE `Priority: 10% (5000 kbps)`	Percent resolved against the **shaped** rate: 5 Mbps, not 100 Mbps
VOICE `b/w exceed drops: 0`	The LLQ policer has never fired. Voice fits its cap
CRITICAL-DATA `offered 14.87 Mbps` vs `bandwidth 25% (12500 kbps)`	Offered above its guarantee, so drops in this class are expected when the link is full
WRED table: `af23` minimum threshold 24 vs `af21` 32	AF23 starts random drops at a shallower depth. Its drop count (7,990) dwarfs AF21's (120): **drop precedence working as designed**

Reading a Policer's Counters

The guest policer from 11.4, applied inbound, under load:

```
R1# show policy-map interface GigabitEthernet0/0/2 input
 GigabitEthernet0/0/2
  Service-policy input: POLICE-GUEST
    Class-map: class-default (match-any)
      812334 packets, 498221010 bytes
      30 second offered rate 13200000 bps, drop rate 1100000 bps
      Match: any
      police:
          cir 10000000 bps, bc 312500 bytes, be 312500 bytes
        conformed 702113 packets, 421268000 bytes; actions:
          transmit
        exceeded 98112 packets, 66710000 bytes; actions:
          set-dscp-transmit af13
        violated 12109 packets, 10243010 bytes; actions:
          drop
        conformed 9980000 bps, exceeded 2100000 bps, violated 1100000 bps
```

Line	What it tells you
`offered rate 13200000 bps`	Guests are trying to send 13.2 Mbps into a 10 Mbps policy
`cir 10000000 bps, bc 312500 bytes`	The contract: 10 Mbps, with bursts measured in **bytes** (it's a policer)
`conformed … 9980000 bps`	Just under 10 Mbps passes untouched: the CIR is being used fully
`exceeded … set-dscp-transmit af13`	2.1 Mbps of burst above CIR is **sent but re-marked** AF13, so it will be dropped first if a queue downstream fills
`violated … drop`	1.1 Mbps (matching the class `drop rate`) is discarded outright

Conformed + exceeded + violated = offered. If the drop rate equals the violated rate, the policer is the only thing dropping in this class.

Exam-style reasoning: "Users complain voice is choppy. Which counter do you check?" Check VOICE `b/w exceed drops`. If it's climbing, the priority queue is too small, or something non-voice is marked EF. "Video is fine but bulk transfers are slow during business hours." Bulk drops in

class-default are expected behavior under contention, not a fault.

QoS Troubleshooting

Symptom	Likely cause	Command that proves it
All counters stay at 0	Policy applied in the wrong direction or on the wrong interface	`show policy-map interface` / `show running-config interface`
Everything lands in class-default	Markings not arriving (untrusted port reset them) or `match-all` misuse	Class-map `Match:` counters, `show class-map`
Queuing configured but never activates on a fast port	No parent shaper; the interface never congests	Offered rate far below interface speed, queue depth 0
Voice choppy despite LLQ	Priority policer drops; or jitter introduced elsewhere (no QoS on another hop)	`b/w exceed drops`, then check each hop's policy
Policy rejected when applied	Mixing `bandwidth` and `bandwidth remaining`, or total over 100%	The CLI error on `service-policy`

11.8 Chapter Summary

Concept	Remember
DiffServ	Mark at the edge, PHB per hop, no per-flow state
DSCP	6 bits; EF = 46; AFxy = 8x + 2y; CSn = 8n
AF drop precedence	Same queue, **higher y dropped first**
CoS	3 bits in the 802.1Q tag; lost at the first router
MQC	class-map (which) → policy-map (what) → service-policy (where)
`match-all`	Default; AND logic
LLQ	Strict priority **with an implicit policer**
Shaping	Outbound only, buffers, adds delay; Tc = Bc ÷ CIR
HQoS	Parent shaper creates the congestion the child queues manage
WRED	Early random drops per DSCP; prevents global sync; not for voice

---

Chapter 12: Device & Data-Path Virtualization: Hypervisors, VRF, GRE & IPsec

> **Blueprint:** ENCOR 2.1 — Describe device virtualization technologies (2.1.a hypervisor type 1 and 2, 2.1.b virtual machine, 2.1.c virtual switching); ENCOR 2.2 — Configure and verify data path virtualization technologies (2.2.a VRF, 2.2.b GRE and IPsec tunneling)

Virtualization is one idea used over and over: **make one physical thing behave like several independent ones**, or **make several separate things behave like one**. A server becomes many servers. One router becomes several routing tables. A path across someone else's network becomes a private wire. This chapter covers the three forms ENCOR asks you to configure (VRF, GRE, IPsec) and the device-level form it asks you to describe (hypervisors, VMs, virtual switches).

You met the idea in the CCNA guide (1.9). Here we go from "what is it" to "how does it work, and how do I build and prove it".

12.1 Device Virtualization — Why Bother?

A physical server in 2005 typically ran **one** application at 5–15% CPU. The rest of the box (power, cooling, rack space, licences) was wasted. Virtualization lets one physical host run many isolated **virtual machines (VMs)**, each believing it owns real hardware.

Why does this matter to a network engineer and not just a server admin? Because the moment servers became VMs, **the access layer moved inside the server**. Traffic between two VMs on the same host may never touch a physical switch, so your VLANs, ACLs and monitoring have to reach into that host too. Virtualization didn't remove the network; it hid part of it inside a box.

12.2 Hypervisors — Type 1 vs Type 2

The problem it solves: several operating systems each expect to own the CPU, the memory and the network card. Something has to sit underneath them, hand out slices of the real hardware, and stop one guest from touching another's slice.

The **hypervisor** (or virtual machine monitor) is the software layer that creates VMs and shares physical CPU, memory, storage and NICs between them.

Story Time : think of a hotel. The building (the physical server) has a fixed number of rooms, a kitchen and a laundry. The hotel manager (the hypervisor) gives every guest (VM) a private room and a key, and shares the kitchen and laundry between them on a schedule. Guests never see each other's rooms. A **type 1** hypervisor is a manager who owns the building and runs it directly. A **type 2** hypervisor is a guest who rented a big suite and then sub-lets parts of it: every request for the kitchen goes through the real building owner first (the host OS), so it's slower.

Feature	Type 1 — bare metal	Type 2 — hosted
Runs on	Directly on the hardware	On top of a normal OS (Windows, macOS, Linux)
Examples	VMware ESXi, Microsoft Hyper-V, KVM, Xen	VMware Workstation/Fusion, Oracle VirtualBox, Parallels
Performance	Near-native — no host OS in the way	Lower — every call passes through the host OS
Typical use	Data centers, production, NFV	Labs, desktops, testing
Failure domain	Hypervisor only	Host OS and hypervisor

Why is type 1 faster? Because a type 2 hypervisor is just another application. When a VM asks for disk or network I/O, the request goes VM → hypervisor → **host OS** → hardware, and the host OS schedules it among everything else it's doing. A type 1 hypervisor is the operating system: VM → hypervisor → hardware. One fewer layer to wait on, and nothing else competing for the CPU.

> **Exam trap:** KVM is the one that confuses people. It's a Linux kernel module, so it "runs on Linux". It's still classed as **type 1**, because the kernel itself becomes the hypervisor; there's no separate host OS underneath it. Your EVE-NG server is built on exactly this (QEMU/KVM).

12.3 What a Virtual Machine Actually Is

The problem it solves: you want the isolation of a separate physical server (its own OS, its own crash domain, its own IP and MAC) without buying, racking and powering a separate box.

A VM is a set of files (a config file plus virtual disks) and a share of the host's resources. The hypervisor presents **virtual hardware** to it:

Virtual component	What the VM sees	What's really happening
vCPU	A processor core	A time-slice of a physical core, scheduled by the hypervisor
vRAM	Memory	A portion of physical RAM (can be overcommitted)
Virtual disk	A hard drive	A file (`.vmdk`, `.vhdx`, `.qcow2`) on host storage
vNIC	An Ethernet card with a MAC	A software port on a **virtual switch**

Each VM runs its **own guest OS** with its own kernel. That's the key difference from a **container**, which shares the host's kernel and packages only the application and its libraries (CCNA guide 1.9). VMs give stronger isolation; containers are lighter and start in seconds.

Why can a VM be moved to another host while it's running (vMotion / live migration)? Because the VM is just state: memory pages, CPU registers and a disk file on shared storage. The hypervisor copies memory to the new host, pauses the VM for a few milliseconds to copy the last

changes, and resumes it there. The VM keeps its MAC and IP, which is exactly why the **same VLAN must exist on both hosts**.

How a live migration works, step by step (VM `WEB01`, IP `10.20.0.15`, MAC `0050.56aa.0001`, VLAN 20, moving from Host A to Host B):

1. Host B reserves the same vCPU and vRAM for WEB01. The virtual disk stays where it is, on shared storage both hosts can reach.
2. Host A copies WEB01's memory to Host B over the migration network while WEB01 keeps running.
3. Pages that changed during that copy are sent again. Each round is smaller than the last.
4. When the remaining changes are tiny, Host A pauses WEB01 for a few milliseconds, sends the last pages and the CPU state, and Host B resumes it.
5. Host B sends a frame from `0050.56aa.0001` out its uplink (VMware sends a RARP frame for exactly this purpose). Every physical switch updates its MAC table: that MAC now lives on Host B's port.
6. Users typically notice nothing, or lose a packet or two during the pause. WEB01 still has `10.20.0.15` and its default gateway, because VLAN 20 exists on Host B's trunk too.

Worked example — what breaks if VLAN 20 is missing: Host B's uplink trunk only allows VLANs 10 and 30. The migration in steps 1–4 still succeeds (it uses the separate migration network), so the server team sees "migration complete". But the frame in step 5 is dropped at the physical switch port, because VLAN 20 isn't allowed there. WEB01 is running and unreachable. `show interfaces trunk` on the switch shows VLAN 20 missing from the "allowed" list for Host B's port. The fix is on your switch, not the server: `switchport trunk allowed vlan add 20`.

Live migration is one of the main reasons data centers wanted Layer 2 stretched everywhere — and one of the main reasons VXLAN exists (Chapter 13).

12.4 Virtual Switching

The problem it solves: twenty VMs on one host each have a vNIC, but the host has maybe two physical network cards. Something has to connect the VMs to each other and share those two cards among them, keeping VLANs separate, exactly like an access switch does for physical PCs.

Every vNIC plugs into a **virtual switch (vSwitch)**: a software Layer 2 switch inside the hypervisor. It learns MACs, supports VLANs (port groups) and connects to the outside world through the host's physical NICs (**uplinks**), which are normally configured as **802.1Q trunks** on the physical switch.

Type	Scope	Example
Standard vSwitch	One host — configured host by host	VMware vSphere Standard Switch, Linux bridge
Distributed vSwitch	Many hosts, managed as one switch from a central console	VMware vSphere Distributed Switch (VDS)
Open vSwitch (OVS)	Programmable, multi-host, open source	KVM/OpenStack environments

How a frame moves through a vSwitch, step by step. Host A has a vSwitch with port group "VLAN 20 – Finance" and two uplinks (`vmnic0`, `vmnic1`) trunked to the physical switch. VM `FIN01` (`10.20.0.11`) talks to VM `FIN02` (`10.20.0.12`, same host) and then to the gateway `10.20.0.1` (on the physical core switch):

1. FIN01 sends a frame to FIN02's MAC. The vSwitch looks up its MAC table, finds FIN02 on a local virtual port, and delivers it in software. **The frame never leaves the host**, so the physical switch never sees it, and a SPAN session on the physical switch won't capture it.
2. FIN01 sends a frame to the gateway's MAC. The vSwitch doesn't have that MAC on a local port, so it sends the frame out an uplink, **adding an 802.1Q tag of 20** because the port group is VLAN 20.
3. The physical switch receives a tagged frame on a trunk, exactly as it would from another switch, and forwards it to the gateway in VLAN 20.

That's the whole mental model: a vSwitch is an access switch for the VMs and a trunk neighbour for your physical switch. (One difference: a VMware vSwitch doesn't run STP with your switch. It avoids loops a simpler way: it never forwards a frame that came in one uplink back out of another uplink.)

Why does a distributed switch matter? Consistency. With standard vSwitches, an admin must create "VLAN 20 – Finance" identically on 40 hosts; miss one and a VM that migrates there loses its network. A distributed switch defines the port group **once** and every host gets it, and a VM's port settings and statistics follow it when it moves.

Getting Packets Out Faster

Software switching costs CPU. Two techniques reduce that:

- **SR-IOV (Single Root I/O Virtualization)**: a PCIe feature that lets one physical NIC present many lightweight **virtual functions (VFs)**. A VM is given a VF directly and **bypasses the vSwitch**, getting near line-rate performance. The trade-off: the hypervisor can no longer see or police that traffic, and live migration becomes harder.
- **DPDK**: a set of user-space libraries that let a virtual switch or virtual router poll the NIC directly instead of waiting for kernel interrupts.

Cisco's Place in This: NFV

Network Functions Virtualization (NFV) runs network functions (routers, firewalls, WAN optimizers) as VMs instead of dedicated appliances. Cisco examples: **Catalyst 8000V** (virtual IOS XE router, successor to CSR 1000v), **Secure Firewall Threat Defense Virtual** (virtual NGFW), and the **Catalyst 9800-CL** (cloud/virtual wireless controller, which replaced the AireOS vWLC). Catalyst 9000 switches can also host containers and small apps directly (application hosting) for tools such as packet capture or monitoring agents.

The network engineer's takeaway: VMs connect to vSwitches, vSwitches connect to your physical switch via trunks, and NFV means some of your routers are now VMs too.

12.5 VRF — Many Routing Tables in One Router

Device virtualization splits a server. **VRF (Virtual Routing and Forwarding)** splits a router. Each VRF is a separate routing table and forwarding table (CEF FIB) inside the same box. An interface belongs to exactly one VRF, and by default **packets in one VRF can never reach another**.

The problem it solves: you want two (or ten) groups of users to share one router, but you need them kept apart as if they had separate routers, even when they use the same IP addresses.

Story Time : a VRF is like a block of flats sharing one building. Every flat can have a "Room 1"; the same address in different flats is no conflict. And there's no internal door between flats: to get from one to another you must go out through a deliberate exit.

How the router decides which table to use, step by step:

1. A packet arrives on an interface.
2. The router checks which VRF that **interface** belongs to. That is the only thing that picks the table. The packet's addresses don't.
3. It looks up the destination in **that VRF's** routing table (FIB) only.
4. If there's no match in that table, the packet is dropped. The router does **not** try the global table or another VRF.

Worked example. R1 has Gi0/1 in VRF CUST-A and Gi0/2 in VRF CUST-B. Both customers use `10.1.1.0/24`, and both send a packet to `10.9.9.9`:

Arrives on	VRF	Table consulted	Route found	Result
Gi0/1	CUST-A	CUST-A only	`10.9.9.0/24` via 172.16.100.2 (OSPF)	Forwarded out Gi0/3.100
Gi0/2	CUST-B	CUST-B only	none	Dropped, even though CUST-A has a route

Same destination, same router, two different answers. That's the isolation.

Why use VRFs?

- **Overlapping addresses.** Two customers (or two merged companies) both use `10.0.0.0/8`. In separate VRFs, that's fine.
- **Segmentation.** Guest Wi-Fi, IoT, PCI payment systems and corporate users share hardware but cannot reach each other unless you route between VRFs on purpose (often through a firewall).
- **Management isolation.** Many devices put the management port in its own VRF (e.g. `Mgmt-vrf` on Catalyst 9000) so management traffic never mixes with user traffic.

VRF-Lite vs MPLS VPN. Using VRFs on their own, hop by hop, with each link carrying one VRF (usually one 802.1Q subinterface per VRF), is called **VRF-Lite**. When VRFs are carried across an MPLS core using MP-BGP, it's an **MPLS L3VPN** (Chapter 29). Chapter 25 covers VRF-Lite troubleshooting and route leaking for ENARSI. ENCOR wants you to configure and verify the

basics.

12.6 Configuring a VRF on IOS XE (Try It!)

Modern IOS XE uses the **multiprotocol** syntax `vrf definition` (the older `ip vrf` is IPv4-only):

```
R1(config)# vrf definition CUST-A
R1(config-vrf)#  rd 65000:1                  ! route distinguisher - optional for VRF-Lite,
                                             ! required once BGP/MPLS carries the VRF
R1(config-vrf)#  address-family ipv4         ! enable IPv4 in this VRF
R1(config-vrf-af)#  exit-address-family
R1(config-vrf)#  address-family ipv6         ! and IPv6, if you need it
R1(config-vrf-af)#  exit-address-family
R1(config)# vrf definition CUST-B
R1(config-vrf)#  address-family ipv4
R1(config-vrf-af)#  exit-address-family
```

Now put interfaces into the VRFs. **Order matters:**

```
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ip address 10.1.1.1 255.255.255.0
R1(config-if)# vrf forwarding CUST-A
% Interface GigabitEthernet0/1 IPv4 disabled and address(es) removed due to enabling VRF CUST-A
R1(config-if)# ip address 10.1.1.1 255.255.255.0   ! re-enter it - it was wiped
```

Why does `vrf forwarding` delete the IP address? Because an IP address is installed as a connected route in a specific routing table. Moving the interface to a different table means the old connected route is now in the wrong place, so IOS removes the address rather than leave it half-attached. Always apply `vrf forwarding` first, then the IP address. Forgetting this is the number one VRF lab mistake: the interface looks configured but has no address.

The same address can now be reused in the other VRF:

```
R1(config)# interface GigabitEthernet0/2
R1(config-if)# vrf forwarding CUST-B
R1(config-if)# ip address 10.1.1.1 255.255.255.0   ! identical IP - legal, different table
```

Routing is per-VRF too:

```
R1(config)# ip route vrf CUST-A 0.0.0.0 0.0.0.0 10.1.1.254   ! static route inside CUST-A
R1(config)# router ospf 10 vrf CUST-A                        ! an OSPF process for CUST-A only
R1(config-router)#  network 10.1.1.0 0.0.0.255 area 0
```

VRF-Lite Across a Link: One Subinterface per VRF

A VRF only exists inside one router. To carry two VRFs to a neighbour without MPLS, give each VRF its **own logical link**, normally an 802.1Q subinterface on a shared trunk. This is VRF-Lite:

```
! ---- R1: one physical link to R2, two VRFs riding it
interface GigabitEthernet0/3
 no ip address
interface GigabitEthernet0/3.100
 encapsulation dot1Q 100
 vrf forwarding CUST-A                       ! vrf first, then the address
 ip address 172.16.100.1 255.255.255.252
interface GigabitEthernet0/3.200
 encapsulation dot1Q 200
 vrf forwarding CUST-B
 ip address 172.16.100.1 255.255.255.252     ! same IP again: different table
!
router ospf 10 vrf CUST-A
 network 172.16.100.0 0.0.0.3 area 0
router ospf 20 vrf CUST-B
```

R2 mirrors it with `.2` on each subinterface. Each VRF now has its own adjacency over its own VLAN, and nothing in CUST-A can ever be routed into CUST-B, because they never share a table or a link. **The scaling cost is obvious:** every VRF needs a subinterface, an address and a routing process on every hop. That's the problem MPLS L3VPN solves (Chapter 29), and Chapter 25 covers VRF-Lite troubleshooting.

Verifying — Everything Needs the VRF Name

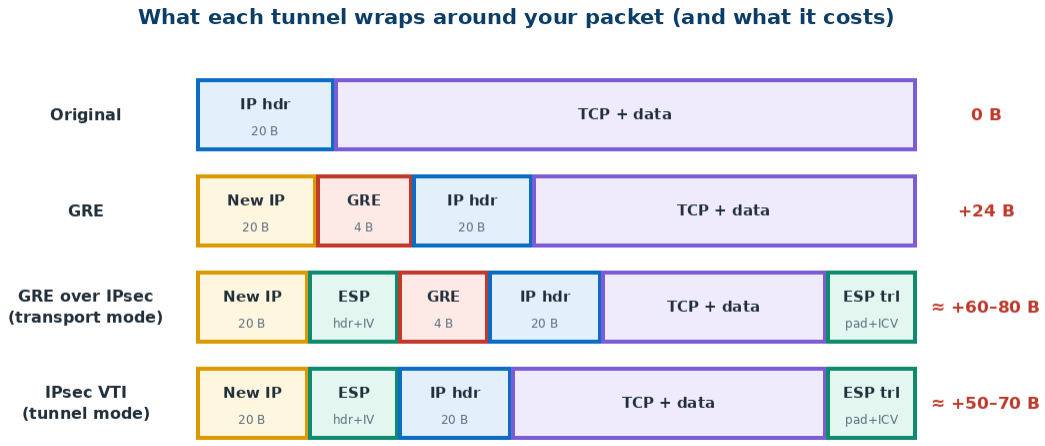
```
R1# show vrf
  Name                 Default RD          Protocols   Interfaces
  CUST-A               65000:1             ipv4,ipv6   Gi0/1
  CUST-B               <not set>           ipv4        Gi0/2

R1# show ip route vrf CUST-A
R1# show ip vrf interfaces CUST-A          ! which interfaces + IPs are in the VRF
R1# ping vrf CUST-A 10.1.1.254
R1# traceroute vrf CUST-A 10.1.1.254
R1# show ip arp vrf CUST-A
```

The trap that catches everyone: `ping 10.1.1.254` with no VRF keyword uses the **global** table. If the address only exists in CUST-A, the ping fails and it looks like the network is broken. Every VRF verification command needs `vrf NAME`.

12.7 GRE — A Virtual Wire Across Any IP Network

Generic Routing Encapsulation (GRE, IP protocol 47) wraps one packet inside another IP packet. The outer packet travels between two public addresses (the **tunnel source** and **destination**); the inner packet can be anything: IPv4, IPv6, even multicast and routing protocol hellos.



Why tunnel at all? Because the network in the middle (the internet, a provider) only knows how to route between the public addresses. It has never heard of your `10.x` networks and won't carry OSPF multicast. GRE makes two distant routers look **directly connected** over a point-to-point link, so you can run a routing protocol over it just as you would over a cable.

Story Time : you want to post a letter addressed to "Room 12, Finance" to a colleague in another office. The public postal service has no idea where "Room 12, Finance" is. So you put your letter inside a second envelope addressed to the other office's street address. The postal service delivers the outer envelope; the mailroom at the other end opens it and hands the inner letter to Room 12. GRE is that outer envelope. Where the analogy breaks: a sealed paper envelope hides the letter, but GRE hides nothing. Anyone along the way can read the inner packet. (That's IPsec's job, 12.8.)

How a packet crosses a GRE tunnel, step by step. PC `10.1.0.10` (behind R1) pings server `10.2.0.20` (behind R2). R1's public address is `198.51.100.1`, R2's is `203.0.113.2`, and the tunnel uses `172.16.0.0/30`:

1. R1 receives the packet `10.1.0.10 → 10.2.0.20`. Its routing table says `10.2.0.0/16 via 172.16.0.2, Tunnel0` (learned by OSPF over the tunnel).
2. Tunnel0 encapsulates: it adds a 4-byte GRE header (protocol type `0x0800` = "IPv4 inside") and a new outer IP header `198.51.100.1 → 203.0.113.2`, protocol **47**.
3. R1 now routes the outer packet. It looks up `203.0.113.2` and sends it out Gi0/0 towards the internet via its default route.
4. Internet routers only read the outer header. To them it's an ordinary packet between two public addresses.
5. R2 receives a protocol-47 packet addressed to itself, matches it to Tunnel0 (source `198.51.100.1` = its configured tunnel destination), strips the outer IP and GRE headers, and finds `10.1.0.10 → 10.2.0.20` inside.
6. R2 routes the inner packet to its LAN as if it had arrived on a directly connected link.

Notice that R1 does **two** routing lookups to send one packet into the tunnel (inner destination, then outer destination). That's the root of the recursive-routing trap below.

The Cost: 24 Bytes

GRE adds a **new IP header (20 bytes) + a GRE header (4 bytes minimum)** = **24 bytes**. The tunnel interface's default IP MTU is therefore **1476** (1500 − 24).

Worked example. A host sends a full 1500-byte packet into the tunnel. 1500 + 24 = **1524 bytes**, which won't fit a 1500-byte Ethernet link on the way out. The router must either fragment it (IOS splits the inner packet to fit the tunnel's IP MTU before encapsulating, which costs CPU, and the destination host must reassemble the pieces) or, if the inner packet has the Don't Fragment (DF) bit set, drop it and send back ICMP "fragmentation needed" quoting the tunnel MTU. Add IPsec on top (roughly 50–70 more bytes, depending on the algorithms) and you see why the configs below set the tunnel's `ip mtu` to **1400**: it leaves a safe margin for every header and stops the problem before it starts.

Configuring a GRE Tunnel

```
! R1 - public address 198.51.100.1 on Gi0/0
R1(config)# interface Tunnel0
R1(config-if)#  ip address 172.16.0.1 255.255.255.252    ! the "inside" of the tunnel
R1(config-if)#  tunnel source GigabitEthernet0/0         ! or the public IP
R1(config-if)#  tunnel destination 203.0.113.2           ! R2's public IP
R1(config-if)#  ! tunnel mode gre ip is the default - no need to type it
R1(config-if)#  ip mtu 1400                              ! leave room for IPsec later
R1(config-if)#  ip tcp adjust-mss 1360                   ! = 1400 - 40 (IP + TCP headers)
R1(config-if)#  keepalive 10 3                           ! detect a dead far end
R1(config)# router ospf 1
R1(config-router)#  network 172.16.0.0 0.0.0.3 area 0    ! routing runs OVER the tunnel
R1(config-router)#  network 10.1.0.0 0.0.255.255 area 0  ! advertise the LAN
```

R2 mirrors it (source and destination swapped, `172.16.0.2`).

Why `ip tcp adjust-mss`? Hosts assume a 1500-byte MTU and send TCP segments of 1460 bytes. After GRE (and IPsec) is added, those packets are too big; the router must fragment them or, if DF is set and the ICMP "fragmentation needed" message is blocked somewhere, **silently drop them**. The classic symptom: ping works, small web pages load, big downloads and HTTPS hang. `adjust-mss` rewrites the MSS value in TCP SYNs passing through the tunnel, so both hosts agree to use smaller segments from the start (CCNA guide 2.7 explains MSS). Fix the MTU problem at the handshake instead of fragmenting every packet.

Tunnel Line Protocol and Keepalives

A GRE tunnel interface is **up/up as soon as there is a route to the tunnel destination**, even if the far router is switched off. GRE itself has no handshake.

Why add keepalives then? Because without them the tunnel stays up/up while forwarding into a black hole. With `keepalive 10 3`, the router sends a keepalive every 10 seconds and brings line protocol **down** after 3 missed replies, so static routes pointing into the tunnel are withdrawn and a backup path can take over. (A routing protocol running over the tunnel detects failure on its own, via its hellos.)

The Recursive Routing Trap

```
%TUN-5-RECURDOWN: Tunnel0 temporarily disabled due to recursive routing
```

The problem behind the message: the router needs a route to the tunnel destination to send the outer packet (step 3 of the walk-through above). If the best route to that destination points into the tunnel, the router can never send anything.

This happens when the router learns the route **to the tunnel destination through the tunnel itself**. Example: R2 advertises its public `203.0.113.0/24` into OSPF over the tunnel. R1 now has a "better" route to 203.0.113.2 via Tunnel0, so to send a tunnel packet it must first send it through the tunnel... which needs a tunnel packet... The router detects the loop and shuts the tunnel down. It comes back, relearns the route, and fails again, so the tunnel flaps.

Step by step, with the numbers:

1. R1 reaches `203.0.113.2` through a static default route out Gi0/0 (AD 1, `0.0.0.0/0`). The tunnel comes up.
2. OSPF forms over Tunnel0. R2 has `network 203.0.113.0 0.0.0.255 area 0`, so it advertises `203.0.113.0/24`.
3. R1 installs `O 203.0.113.0/24 via 172.16.0.2, Tunnel0`. A /24 is **more specific** than the /0 default, so longest match picks the OSPF route, even though OSPF's AD (110) is worse than the static's (1). AD only compares routes for the same prefix.
4. To send any tunnel packet, R1 looks up `203.0.113.2` and gets "out Tunnel0". Loop. `%TUN-5-RECURDOWN` is logged and Tunnel0 goes down.
5. With the tunnel down, OSPF loses the neighbour and the /24 disappears. The default route is used again, the tunnel comes up, and you're back at step 2. The tunnel flaps every minute or so.

The fix: never advertise the tunnel endpoints (the public/transport addresses) into the routing protocol that runs over the tunnel. Reach them through the underlay (a static or default route out the physical interface) and advertise only the inside networks. Underlay addresses go in the underlay; overlay routes go in the overlay. Mixing them is how the tunnel ends up routing through itself.

Verifying GRE

```
R1# show interface Tunnel0
Tunnel0 is up, line protocol is up
  Internet address is 172.16.0.1/30
  BW 100 Kbit/sec, DLY 50000 usec,
  Tunnel source 198.51.100.1 (GigabitEthernet0/0), destination 203.0.113.2
  Tunnel protocol/transport GRE/IP
    Key disabled, sequencing disabled
  Keepalive set (10 sec), retries 3
  Tunnel transport MTU 1476 bytes
```

Read three things: **line protocol up**, the **source/destination** are what you intended, and **transport GRE/IP**. Note the default tunnel **bandwidth of 100 kbps**: OSPF and EIGRP compute metrics from it, so set `bandwidth` on the tunnel if path selection matters.

GRE's big weakness: it is **not encrypted**. Anyone on the path can read the inner packets. That's what IPsec is for.

12.8 IPsec — Making the Tunnel Private

The problem it solves: a GRE tunnel or a plain internet path is readable by anyone in between, and anyone can forge or replay packets. IPsec makes the traffic unreadable, proves who sent it, and proves nobody changed it, even though it crosses a network you don't trust.

Story Time : IPsec is an armoured courier service. Before the first parcel, the two offices meet and check each other's ID (authentication), agree on the lock type (the algorithms) and secretly agree on a key (Diffie-Hellman). After that, every parcel goes in a locked box (encryption) with a tamper seal (integrity) and a numbered ticket (anti-replay), so a forged, opened or repeated box is spotted and thrown away.

IPsec is a framework of protocols that provides:

Service	What it means	Provided by
Confidentiality	Nobody can read it	Encryption — AES (CBC or GCM)
Integrity	Nobody changed it	HMAC — SHA-256/384/512 (or GCM's built-in check)
Authentication	It really came from the peer	Pre-shared keys or certificates (RSA/ECDSA)
Anti-replay	Old packets can't be re-sent	Sequence numbers

ESP vs AH

Protocol	IP protocol #	Encrypts?	Authenticates outer IP header?	Works through NAT?
ESP	**50**	**Yes**	No	**Yes** (with NAT-T)
AH	**51**	**No**	Yes	**No**

Why does nearly everyone use ESP? Because AH doesn't encrypt, and by authenticating the outer IP header, AH breaks the moment NAT rewrites that header (the integrity check fails). ESP protects the payload and leaves the outer header alone, so NAT can change it. When NAT is detected, IKE switches to **NAT Traversal (NAT-T)**, wrapping ESP in **UDP 4500**.

Tunnel Mode vs Transport Mode

- **Tunnel mode** encrypts the **entire original packet** and adds a **new IP header**. Used site-to-site, when the routers are protecting traffic for others. The default.
- **Transport mode** encrypts only the **payload**, keeping the original IP header. Used when the endpoints of the traffic are the IPsec peers themselves. The classic case being **GRE over IPsec**, where the GRE packet is already between the two routers. Transport mode saves 20 bytes.

IKE — Agreeing on Keys

Before any data is encrypted, the peers use **IKE (Internet Key Exchange, UDP 500)** to authenticate each other and agree on algorithms and keys. The result is a set of **Security Associations (SAs)**: one-way agreements, so every tunnel has an inbound and an outbound IPsec SA.

Version	Phase / exchange	What it builds	Messages
IKEv1	**Phase 1** — Main Mode (or Aggressive Mode)	An authenticated, encrypted management channel: the **ISAKMP SA**	6 (Main) / 3 (Aggressive)
IKEv1	**Phase 2** — Quick Mode	The **IPsec SAs** that carry data	3
IKEv2	**IKE_SA_INIT**	Negotiates crypto, Diffie-Hellman exchange	2
IKEv2	**IKE_AUTH**	Authenticates peers + builds the first **Child SA** (IPsec SA)	2

How an IKEv2 tunnel comes up, step by step (R1 `198.51.100.1` initiates to R2 `203.0.113.2`, the first time traffic is routed into the tunnel or as soon as the VTI comes up):

1. **IKE_SA_INIT request** (UDP 500): R1 offers its proposals (for example AES-CBC-256, SHA-256, DH group 19), its DH public value and a random nonce.

2. **IKE_SA_INIT response**: R2 picks one proposal it also has, and sends its own DH value and nonce. Both sides now compute the same DH secret and derive keys. Everything from here on is encrypted. If NAT was detected, they move to UDP 4500 now.
3. **IKE_AUTH request**: R1 proves its identity (here, a hash computed with the pre-shared key) and proposes the Child SA: transform set and the traffic it should cover.
4. **IKE_AUTH response**: R2 checks R1's proof, sends its own, and accepts the Child SA.
5. Result: one IKE SA (`show crypto ikev2 sa` → READY) and a pair of IPsec SAs, one inbound and one outbound, each with its own SPI (`show crypto ipsec sa`). Data now flows as ESP, IP protocol 50.

If step 2 fails you'll see `NO_PROPOSAL_CHOSEN` (no common proposal). If step 4 fails it's usually `AUTH_FAILED` (wrong key or identity). That's why knowing the four messages makes debugs readable.

Why did IKEv2 replace IKEv1? Four messages instead of nine; built-in NAT-T, dead peer detection and EAP support; asymmetric authentication (each side can use a different method); and far fewer ambiguous options, so interoperability is better. IKEv2 is the default choice for anything new, and FlexVPN on IOS XE is built on it.

Why Diffie-Hellman? Because the two routers need a shared secret key, but every packet between them crosses an untrusted network. DH lets each side combine a private value with the other side's public value so that **both compute the same secret without ever sending it**.

Story Time (the paint trick): both sides publicly agree on a base colour, say yellow. R1 secretly picks red, mixes it with yellow and sends the orange result. R2 secretly picks blue, mixes it with yellow and sends the green result. R1 adds its secret red to R2's green; R2 adds its secret blue to R1's orange. Both end up with the same brown, but an eavesdropper who saw yellow, orange and green can't un-mix the paint to find red or blue. In real DH the "mixing" is maths that is easy one way and practically impossible to reverse. The DH **group** sets its strength: prefer groups **19, 20** (elliptic curve) or **14+**; groups 1, 2 and 5 are considered broken.

Crypto Maps vs Virtual Tunnel Interfaces

Feature	Crypto map (legacy, policy-based)	VTI / tunnel protection (route-based)
What gets encrypted	Traffic matching a crypto ACL ("interesting traffic")	Anything **routed into** the tunnel interface
Applied to	The physical outside interface	A tunnel interface
Routing protocols over it	Not on its own (no multicast); only by putting GRE inside it (12.10)	**Yes**
QoS, ACLs, NetFlow per tunnel	Awkward	Natural — it's an interface
Add a new subnet	Edit ACLs on **both** sides	Just route it

Why is VTI the modern answer? Because it turns VPN design back into routing. With a crypto map, the routing table says "send it out Gi0/0" and a hidden ACL decides whether it's encrypted, so two separate policies must agree. With a VTI, encryption is simply a property of an interface: if

the route points into the tunnel, it's encrypted. One decision instead of two, and you can run OSPF across it.

12.9 Configuring a Static VTI with IKEv2 (Try It!)

What you're building and why: a routable, encrypted point-to-point link between two sites over the internet. Because it's an interface, you can point routes (or run OSPF) into it, and everything that goes in is encrypted automatically.

Site A (R1, public `198.51.100.1`) to Site B (R2, public `203.0.113.2`). A **static VTI** uses `tunnel mode ipsec ipv4`: no GRE header, pure IPsec tunnel mode.

```
! ---- Phase "IKE_SA_INIT": what crypto to offer
crypto ikev2 proposal PROP-AES256
 encryption aes-cbc-256
 integrity sha256
 group 19
crypto ikev2 policy POL-1
 proposal PROP-AES256
!
! ---- Who the peer is and the shared secret
crypto ikev2 keyring KR-SITEB
 peer R2
  address 203.0.113.2
  pre-shared-key Str0ng-Secret!          ! same key configured on both ends
!
! ---- "IKE_AUTH": how we identify and authenticate the peer
crypto ikev2 profile IKEV2-PROF
 match identity remote address 203.0.113.2 255.255.255.255
 authentication remote pre-share
 authentication local pre-share
 keyring local KR-SITEB
!
! ---- The Child SA: how data is protected
crypto ipsec transform-set TS-AES256 esp-aes 256 esp-sha256-hmac
 mode tunnel
crypto ipsec profile IPSEC-PROF
 set transform-set TS-AES256
 set ikev2-profile IKEV2-PROF
!
! ---- The routable interface
interface Tunnel1
 ip address 172.16.1.1 255.255.255.252
 ip mtu 1400
 ip tcp adjust-mss 1360
 tunnel source GigabitEthernet0/0
 tunnel mode ipsec ipv4                  ! static VTI - no GRE header
 tunnel destination 203.0.113.2
 tunnel protection ipsec profile IPSEC-PROF
!
ip route 10.2.0.0 255.255.0.0 Tunnel1     ! or run OSPF/EIGRP over Tunnel1
```

R2 is the mirror image: keyring peer address `198.51.100.1`, match identity `198.51.100.1`, its own tunnel IP `172.16.1.2`.

Read the config as a chain of references, because that's how to troubleshoot it. The tunnel points to the **IPsec profile**; the IPsec profile points to the **transform set** and the **IKEv2 profile**; the IKEv2 profile points to the **keyring**. A typo in any name breaks the chain, usually without an obvious error.

IKEv2 Smart Defaults

IOS XE ships with **built-in IKEv2 and IPsec defaults** (FlexVPN smart defaults): a default IKEv2 proposal and policy, a default IPsec transform set, and an IPsec profile named `default`. The only things you must supply are the parts that can't have a default, namely **who the peer is and how to authenticate it**:

```
crypto ikev2 keyring KR-SITEB
 peer R2
  address 203.0.113.2
  pre-shared-key Str0ng-Secret!
crypto ikev2 profile IKEV2-PROF
 match identity remote address 203.0.113.2 255.255.255.255
 authentication remote pre-share
 authentication local pre-share
 keyring local KR-SITEB
!
crypto ipsec profile default                  ! the built-in profile...
 set ikev2-profile IKEV2-PROF                 ! ...just needs to know the IKEv2 profile
!
interface Tunnel1
 tunnel protection ipsec profile default
```

`show crypto ikev2 proposal default` and `show crypto ipsec transform-set default` reveal what the defaults actually are on your release. **Why does this matter for the exam?** Because a config with no `crypto ikev2 proposal` or `transform-set` is not incomplete. It's using the defaults, and both sides will agree as long as both use them. Defining your own proposal (as in the full config above) overrides the default for that policy.

12.10 GRE over IPsec

The problem it solves: GRE can carry routing protocols and multicast but isn't encrypted; IPsec encrypts but (in its pure form) carries only unicast IP. Put one inside the other and you get both.

How the layers stack (tunnel `172.16.0.1 → .2`, publics `198.51.100.1 → 203.0.113.2`):

1. Original packet: `10.1.0.10 → 10.2.0.20`.
2. GRE wraps it: new IP header `198.51.100.1 → 203.0.113.2`, protocol 47.
3. IPsec in **transport mode** encrypts everything after that outer header (GRE header plus original packet) and inserts an ESP header. The outer IP header stays, its protocol field now says **50** (ESP).
4. The internet sees only ESP between two public addresses. It can't even tell that GRE is inside.

Why put GRE inside IPsec when a VTI already supports routing? Because GRE can carry things a pure IPsec VTI can't: **non-IP protocols, multiple protocols over one tunnel**, and it's the basis for **mGRE in DMVPN** (Chapter 30). The config is the same, except the tunnel stays in its default GRE mode, and transport mode saves 20 bytes:

```
crypto ipsec transform-set TS-GRE esp-aes 256 esp-sha256-hmac
 mode transport                          ! GRE already supplies the outer header
crypto ipsec profile GRE-PROT
 set transform-set TS-GRE
 set ikev2-profile IKEV2-PROF
!
interface Tunnel0
 ip address 172.16.0.1 255.255.255.252
 tunnel source GigabitEthernet0/0
 tunnel destination 203.0.113.2
 tunnel protection ipsec profile GRE-PROT   ! the only line that adds encryption
```

`tunnel protection` automatically protects exactly the GRE traffic between tunnel source and destination. No crypto ACL is needed.

The Legacy Way: GRE over IPsec With IKEv1 and a Crypto Map

You'll still meet this form in production and in exam exhibits, so read it fluently. The same GRE tunnel is protected not by `tunnel protection`, but by a **crypto map** on the physical interface, whose ACL selects the GRE packets:

```
! ---- IKEv1 Phase 1: the ISAKMP policy and the PSK
crypto isakmp policy 10
 encryption aes 256
 hash sha256
 authentication pre-share
 group 14
crypto isakmp key Str0ng-Secret! address 203.0.113.2
!
! ---- Phase 2: how data is protected
crypto ipsec transform-set TS-LEGACY esp-aes 256 esp-sha256-hmac
 mode transport                                ! GRE supplies the outer header
!
! ---- "Interesting traffic": the GRE packets between the two public IPs
ip access-list extended GRE-TO-R2
 permit gre host 198.51.100.1 host 203.0.113.2
!
crypto map VPN-MAP 10 ipsec-isakmp
 set peer 203.0.113.2
 set transform-set TS-LEGACY
 match address GRE-TO-R2
!
interface GigabitEthernet0/0                   ! the PHYSICAL outside interface
 crypto map VPN-MAP
!
interface Tunnel0                              ! plain GRE, no tunnel protection
 ip address 172.16.0.1 255.255.255.252
 tunnel source GigabitEthernet0/0
 tunnel destination 203.0.113.2
```

Reading it: sequence 10 of crypto map VPN-MAP says "anything matching ACL GRE-TO-R2, leaving Gi0/0, gets encrypted to peer 203.0.113.2 with TS-LEGACY". The peer's ACL must be the **mirror image** (`permit gre host 203.0.113.2 host 198.51.100.1`). Mismatched ACLs are the classic crypto-map failure: Phase 1 comes up but Phase 2 is rejected.

Verify Phase 1 with `show crypto isakmp sa`:

```
R1# show crypto isakmp sa
IPv4 Crypto ISAKMP SA
dst             src             state          conn-id status
203.0.113.2     198.51.100.1    QM_IDLE           1001 ACTIVE
```

State	Meaning
QM_IDLE	**Healthy.** Phase 1 is complete and the ISAKMP SA is idle, ready to build or rekey Phase 2 SAs
MM_NO_STATE	Phase 1 stuck at the first exchange: no reply from the peer, or no matching ISAKMP policy
MM_KEY_EXCH	Keys exchanged but authentication failed: usually a pre-shared key mismatch

IKEv1 crypto maps and IKEv2 VTIs protect traffic equally well. The difference is that a crypto map ties encryption to an ACL, while a VTI ties it to routing (12.8).

12.11 Verifying and Troubleshooting Tunnels

Why a method matters here: a tunnel has three layers that can each fail on their own: the underlay (public path), the tunnel and crypto (GRE/IKE/IPsec), and the overlay (routes into it). The commands below each look at one layer, so read them in that order.

```
R1# show crypto ikev2 sa
 IPv4 Crypto IKEv2  SA
Tunnel-id Local                 Remote                fvrf/ivrf            Status
1         198.51.100.1/500      203.0.113.2/500       none/none            READY
      Encr: AES-CBC, keysize: 256, PRF: SHA256, Hash: SHA256, DH Grp:19, Auth sign: PSK

R1# show crypto ipsec sa interface Tunnel1
   local  ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
   remote ident (addr/mask/prot/port): (0.0.0.0/0.0.0.0/0/0)
    #pkts encaps: 1520, #pkts encrypt: 1520, #pkts digest: 1520
    #pkts decaps: 1498, #pkts decrypt: 1498, #pkts verify: 1498
     inbound esp sas:
      spi: 0x8C4F21A3(2354004387)
        transform: esp-256-aes esp-sha256-hmac ,
        in use settings ={Tunnel, }

R1# show crypto session
Interface: Tunnel1
Session status: UP-ACTIVE
Peer: 203.0.113.2 port 500
```

How to read it:

- IKEv2 SA **READY** means the peers authenticated each other. Without READY, nothing else matters.
- **encaps rising, decaps stuck at 0** means you're sending but nothing comes back. Look at the far side: its routing back into the tunnel, or an ACL/firewall in the path dropping ESP (protocol 50) or UDP 4500.
- **Both counters at 0** means no traffic is being routed into the tunnel. Check the routing table, not the crypto.
- `in use settings ={Tunnel, }` versus `{Transport, }` confirms the mode.

Symptom	Likely cause	Command that proves it
Tunnel up/up, but nothing gets through (plain GRE)	Far end down, no keepalives configured	`show interface Tunnel0` (no keepalive line); `ping` the tunnel IP
`%TUN-5-RECURDOWN`, tunnel flapping	Tunnel destination learned via the tunnel	`show ip route 203.0.113.2` points out Tunnel0
IKEv2 SA never reaches READY	PSK mismatch, proposal mismatch, identity mismatch	`debug crypto ikev2` ("AUTH_FAILED", "NO_PROPOSAL_CHOSEN")
SA up, encaps grow, decaps stay 0	Return routing on the peer, or ESP/UDP 4500 blocked	`show crypto ipsec sa` on **both** ends

Symptom	Likely cause	Command that proves it
Pings work, big transfers hang	MTU/MSS — oversized packets dropped	`ping 10.2.1.1 size 1500 df-bit` fails; `ip tcp adjust-mss` missing
Interface IP missing after config	`vrf forwarding` applied after `ip address`	`show run interface` — no address line
`ping` to a VRF host fails, routes look fine	Pinging from the global table	`ping vrf NAME …` succeeds

Worked example. Users at Site A can't reach `10.2.0.0/16` over Tunnel1. You check in order:

1. Underlay: `ping 203.0.113.2 source GigabitEthernet0/0` succeeds. The public path is fine.
2. Tunnel: `show crypto ikev2 sa` shows READY, so authentication worked.
3. Crypto counters: `show crypto ipsec sa` shows `#pkts encaps: 240`, `#pkts decaps: 0`. R1 is sending, nothing comes back.
4. On R2, `show ip route 10.1.0.0` returns % Subnet not in table. R2 has no route back to Site A through Tunnel1, so it sends the replies out its default route, unencrypted, and R1's tunnel never sees them.
5. Fix: add `ip route 10.1.0.0 255.255.0.0 Tunnel1` on R2 (or bring the tunnel into OSPF). Decaps start rising on R1 within seconds.

The underlying principle for every tunnel problem: check the **underlay** first (can I reach the far public IP?), then the **tunnel** (up, SA ready?), then the **overlay** (routes pointing into it?). Fix them in that order. An overlay problem can't be solved while the underlay is broken.

12.12 Chapter Summary

Topic	What to be able to say
Type 1 vs type 2 hypervisor	Type 1 runs on bare metal (ESXi, Hyper-V, KVM); type 2 runs on a host OS (Workstation, VirtualBox)
Virtual machine	Virtual hardware (vCPU, vRAM, vNIC, vDisk) as files plus state; can live-migrate if the VLAN exists on both hosts
Virtual switching	vSwitch/distributed switch inside the hypervisor; SR-IOV and DPDK to go faster
VRF	Separate routing table per VRF; `vrf forwarding` **before** `ip address`; every command needs the VRF name
VRF-Lite	One subinterface (or link) per VRF per hop; no MPLS
GRE	IP protocol 47, 24 bytes overhead, carries multicast/routing protocols, no encryption, up/up without keepalives
Recursive routing	Never let the tunnel destination be learned through the tunnel
IPsec	ESP (protocol 50) encrypts; tunnel vs transport mode; IKE on UDP 500/4500
IKEv1 vs IKEv2	Main/Aggressive + Quick Mode (9 or 6 messages) vs IKE_SA_INIT + IKE_AUTH (4)
Crypto map vs VTI	ACL-driven, on the physical interface vs route-driven, on a tunnel interface
Verification	`show crypto isakmp sa` (QM_IDLE), `show crypto ikev2 sa` (READY), `show crypto ipsec sa` (encaps/decaps)

Chapter 13: Network Virtualization: LISP & VXLAN

> **Blueprint:** ENCOR 2.3 — Describe network virtualization concepts (2.3.a LISP, 2.3.b VXLAN)

Chapter 12 virtualized devices and paths. This chapter virtualizes the **network itself**: a logical network (an overlay) built on top of a plain routed IP network (the underlay). Two protocols do the heavy lifting, and they're worth learning together because **SD-Access uses both** (Chapter 14): **LISP** answers where is this endpoint? and **VXLAN** carries the traffic once you know.

ENCOR asks you to describe them, so you won't configure these. You do need to know every role, message and number precisely.

13.1 The Problem Overlays Solve

Traditional IP addressing does two jobs with one number. `10.2.3.4` says **who** the device is (its identity) and **where** it is (its location, because the prefix ties it to one subnet in one place).

Why is that a problem? Because the two jobs conflict as soon as something **moves**:

- A VM live-migrates to a host in another building (12.3). Its identity must stay the same, so applications and sessions survive, but its location just changed.
- A laptop roams from floor 1 to floor 4, onto a different switch and different subnet.
- A branch with two ISPs (multihoming) wants one stable set of addresses while choosing between providers.

If the address is the location, moving means renumbering, which breaks sessions. The old workaround was **stretching Layer 2** (the same VLAN everywhere), which brings back huge broadcast domains and spanning tree across the campus or data center.

The overlay idea: give each endpoint a stable identity, and keep a separate, changeable record of where it currently is.

13.2 Underlay vs Overlay

Term	What it is	Example
Underlay	The physical, routed IP network. Only needs to get packets between tunnel endpoints	OSPF or IS-IS between switches, all links routed
Overlay	The logical network built from tunnels on top of the underlay	VXLAN segments, LISP-routed endpoint prefixes
Encapsulation	How overlay packets ride the underlay	VXLAN (UDP 4789), LISP data plane (UDP 4341)
Control plane	How tunnel endpoints learn who is behind which endpoint	LISP mapping system, BGP EVPN, or flood-and-learn

Story Time : the underlay is the motorway network. It only knows how to get a lorry from one depot to another, and it doesn't care what's inside. The overlay is the parcel company's own delivery network that runs on top of it: parcels are addressed to people, sorted at the depot, loaded into lorries addressed depot-to-depot, and unpacked at the other end. You can reorganise parcel routes (move a customer, add a new delivery zone) without building a single new road. Where it breaks: real roads can jam, but a well-built routed underlay mostly doesn't, because ECMP spreads the load over every link.

Why is a simple routed underlay so attractive? Because routed links have **no spanning tree**, use **every link at once with ECMP**, and converge in milliseconds. The overlay provides

the flexibility endpoints need (mobility, segmentation, stretched subnets) without turning the physical network back into one giant Layer 2 domain.

13.3 LISP — Locator/ID Separation Protocol

LISP (RFC 9300/9301, originally RFC 6830) splits the address into two namespaces:

Name	Stands for	Answers	Example
EID	Endpoint Identifier	**Who** — the address of the host or prefix	`10.2.3.4`, `10.2.0.0/16`
RLOC	Routing Locator	**Where** — the address of the LISP router that the EID currently sits behind	`198.51.100.2`

The underlay only routes **RLOCs**. It never needs to know any EID. EID-to-RLOC mappings live in a **mapping system**, which works rather like DNS for locations.

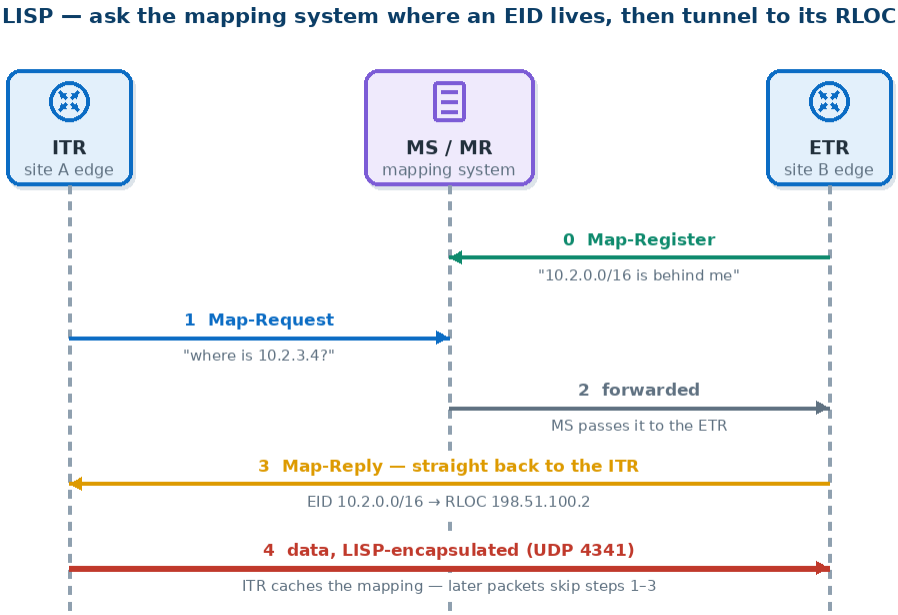
Story Time : an EID is your name and an RLOC is the address of the hotel you're staying at tonight. When you change hotels you update one directory entry ("Alex is now at the Riverside"). Your friends still look you up by name, and the postal service only ever needs to know how to reach hotels.

The LISP Roles

Role	Job
ITR (Ingress Tunnel Router)	Receives packets from local hosts, looks up the destination EID's RLOC, **encapsulates** and sends it
ETR (Egress Tunnel Router)	Receives LISP-encapsulated packets, **decapsulates** and delivers them to local EIDs. Also **registers** its EIDs with the mapping system
xTR	A router doing both ITR and ETR — the normal case at a site edge
MS (Map Server)	Accepts **Map-Registers** from ETRs; holds the authoritative EID-to-RLOC database
MR (Map Resolver)	Accepts **Map-Requests** from ITRs and passes them into the mapping system
MS/MR	Both functions on one box — the usual deployment (and SD-Access's control plane node)
PITR (Proxy ITR)	Attracts traffic **from non-LISP sites** to LISP EIDs, and encapsulates it on their behalf
PETR (Proxy ETR)	Receives traffic **from LISP sites to non-LISP destinations** (e.g. the internet) and decapsulates it
PxTR	A router doing both proxy roles — the bridge between the LISP and non-LISP worlds

Why have proxy roles at all? Because most of the world doesn't speak LISP. The internet has no idea that `10.2.0.0/16` sits behind RLOC `198.51.100.2`. The PxTR stands at the border and translates: traffic coming in from the non-LISP world is encapsulated toward the right RLOC (PITR), and traffic going out is decapsulated and routed normally (PETR).

The Conversation, Step by Step



1. **Map-Register.** Each ETR tells the Map Server which EID prefixes are behind it ("10.2.0.0/16 is reachable at RLOC 198.51.100.2"). This repeats periodically (about every 60 seconds), authenticated with a shared key. If the ETR asks for it, the Map Server acknowledges each registration with a **Map-Notify**, so the ETR knows it was accepted.
2. **Map-Request.** A host at site A sends a packet to `10.2.3.4`. The ITR has no mapping in its cache, so it sends a Map-Request to the **Map Resolver**.
3. **Forwarding.** The MR/MS finds which ETR registered that prefix and **forwards the request to that ETR**. (Alternatively, the MS can be configured to answer on the ETR's behalf — proxy reply.)
4. **Map-Reply.** The ETR replies **directly to the ITR** with the mapping: EID prefix, RLOC(s), plus a **priority and weight** for each RLOC (used for multihoming and load sharing) and a TTL.
5. **Data.** The ITR caches the mapping in its **map-cache** and encapsulates packets to the RLOC. Every later packet to that prefix uses the cache with no further lookups until the TTL expires. The default map-cache TTL in a Map-Reply is **24 hours** (1,440 minutes); mobility events refresh entries long before that (see below).

Worked example. Site A's xTR (RLOC `198.51.100.1`) has just sent its first packet to `10.2.3.4` and received a Map-Reply from Site B's xTR. Its map-cache (output trimmed) now shows:

```
xTR-A# show ip lisp map-cache
LISP IPv4 Mapping Cache for EID-table default (IID 0), 2 entries

0.0.0.0/0, uptime: 00:05:12, expires: never, via static-send-map-request
  Negative cache entry, action: send-map-request
10.2.0.0/16, uptime: 00:00:05, expires: 23:59:54, via map-reply, complete
  Locator         Uptime    State      Pri/Wgt
  198.51.100.2    00:00:05  up           1/100
```

Reading it: the first line is the catch-all: "for anything I don't know, send a Map-Request". The second entry was learned from a Map-Reply 5 seconds ago and expires in just under 24 hours. Its

single locator is up with priority 1, weight 100. Every packet to anything in `10.2.0.0/16` now gets an outer IP header `198.51.100.1 → 198.51.100.2`, UDP destination port 4341, and an 8-byte LISP header carrying IID 0. If Site B had two RLOCs with priority 1/weight 50 each, the ITR would share flows between them equally; an RLOC with a higher priority number is used only as a backup.

If the mapping system answers with a **negative Map-Reply** (the destination isn't a LISP EID), the ITR forwards natively or sends traffic to a **PETR**.

Why does the ETR, not the Map Server, send the reply? Because the ETR is the **authority on its own site**. It knows which of its RLOCs are currently up and what priorities and weights to advertise. Answering itself guarantees the ITR gets the freshest possible locator information. The mapping system is a directory that routes questions to the right person; it isn't the person.

Why is this "pull" model scalable? An ITR only learns mappings for destinations it actually talks to, on demand. A branch xTR talking to five sites holds five cache entries, not a full table of every EID in the company. Compare BGP or OSPF, which **push** every route to every router whether it's needed or not.

Instance IDs — VRFs Inside LISP

A LISP EID is really a pair: **(Instance ID, EID prefix)**. The **Instance ID (IID)** is a 24-bit number carried in the LISP header and in every register, request and reply, and each IID maps to one VRF on the xTR. Two customers (or two SD-Access virtual networks) can therefore both use 10.1.0.0/16 without colliding in the mapping system, because the Map Server stores them under different IIDs. The IID does for LISP exactly what the VNI does for VXLAN and the route distinguisher does for MPLS VPNs: it keeps overlapping address spaces apart. In SD-Access, every virtual network (VN) gets its own IID.

LISP on the Wire

Plane	Transport	Notes
Control (Map-Register/Request/Reply)	**UDP 4342**	
Data (encapsulated packets)	**UDP 4341**	Outer IP (20) + UDP (8) + LISP header (8) = **36 bytes** for IPv4

Mobility, the payoff. When an endpoint moves to a new xTR, that xTR registers the EID from its own RLOC. The Map Server updates the mapping and sends a **Map-Notify** to the old xTR, which tells any ITR still sending to the old location (a Solicit-Map-Request) to refresh its cache. The host keeps its IP address and the underlay never changes. This is the mechanism SD-Access uses for wired and wireless roaming.

The same move, step by step. Laptop `10.50.1.25` (a /32 host EID) roams from Edge 1 (RLOC `192.168.255.1`) to Edge 2 (RLOC `192.168.255.2`). Edge 3 (RLOC `192.168.255.3`) has a printer that is talking to the laptop:

1. Edge 2 sees traffic from `10.50.1.25` on a local port and sends a **Map-Register**: `10.50.1.25/32 → 192.168.255.2`.

2. The Map Server overwrites the old entry (which pointed at Edge 1) and sends a **Map-Notify** to Edge 1: "this EID has left you".
3. Edge 3 doesn't know yet. It still encapsulates the printer's packets to Edge 1, using its cached mapping.
4. Edge 1 receives them, sees the EID is no longer local, and sends Edge 3 a **Solicit-Map-Request (SMR)**: "your cache is stale, ask again".
5. Edge 3 sends a fresh Map-Request, gets `→ 192.168.255.2`, updates its cache and sends to Edge 2 from then on.

The laptop kept `10.50.1.25`, its TCP sessions survived, and no underlay route changed. Only one directory entry and one cache entry did.

13.4 VXLAN — Layer 2 Over Layer 3

VXLAN (Virtual Extensible LAN, RFC 7348) encapsulates a **complete Ethernet frame** inside a UDP packet so it can cross a routed network. The two endpoints of a VXLAN tunnel are **VTEPs**.

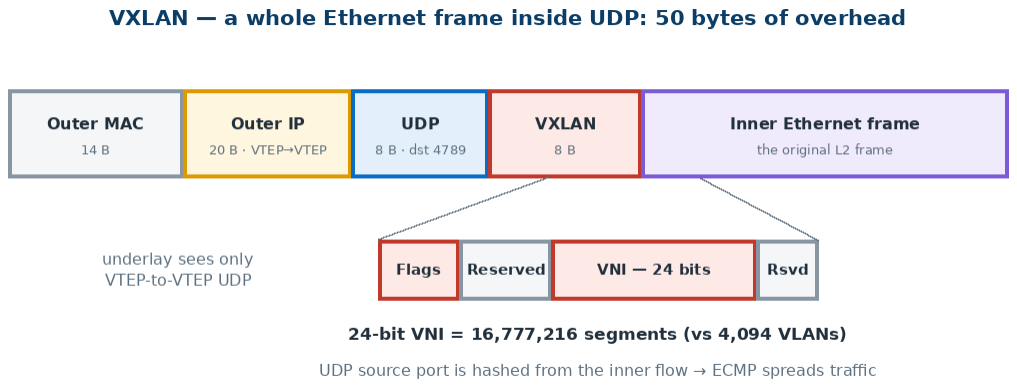
Story Time : a shipping container. A container (the VXLAN packet) can carry anything that fits, here a whole Ethernet frame with its own MACs and VLAN-ish label. Lorries, trains and ships (underlay routers) only read the label on the outside of the container (the outer IP header) and never open it. The container's number (the VNI) tells the receiving port which customer the goods belong to.

Why VXLAN Exists

1. **The VLAN ID is only 12 bits** — 4,094 usable segments. A multi-tenant data center or cloud runs out fast.
2. **Workloads need Layer 2 adjacency across a routed fabric** — live migration (12.3), clusters, and legacy apps that assume a shared subnet — without stretching spanning tree across the data center.
3. **Routed underlays are better** — ECMP across all links, no blocked ports, fast convergence (13.2).

VXLAN gives you Layer 2 service over a Layer 3 network.

The Header



Field	Size	Purpose
Outer Ethernet	14 B	Hop-by-hop MACs across the underlay
Outer IP	20 B	Source VTEP → destination VTEP
Outer UDP	8 B	**Destination port 4789**; source port is a hash of the inner frame
VXLAN header	8 B	Flags + **24-bit VNI**
Inner frame	—	The original Ethernet frame, untouched

Total overhead: 50 bytes (54 if the outer frame carries an 802.1Q tag). Underlay links therefore need a larger MTU, typically **9000 or 9216 jumbo**, or at minimum 1550, otherwise full-size 1500-byte frames from hosts get dropped.

Why is the VNI 24 bits? Because 2^{24} = **16,777,216** segments ends the scarcity problem for good. Each **VNI (VXLAN Network Identifier)** is one Layer 2 segment, just as each VLAN ID was. VTEPs map local VLANs to VNIs: VLAN 10 on leaf 1 and VLAN 210 on leaf 2 can both be VNI 10010, so VLAN numbers become merely local.

Why put it in UDP rather than straight into IP? For **load balancing**. Underlay routers spread traffic over ECMP paths by hashing the 5-tuple. If every VXLAN packet between two VTEPs had the same outer header, all their flows would hash onto one path. By setting the **UDP source port from a hash of the inner frame's headers**, each inner flow gets a different outer 5-tuple, so it spreads across the fabric, while packets of a single flow stay in order on one path. UDP is the cheapest header that carries a field routers already hash on.

VTEPs

A **VTEP (VXLAN Tunnel Endpoint)** encapsulates and decapsulates. It has a local IP in the underlay (usually a loopback) and one or more VNIs. VTEPs can be hardware (a leaf switch such as a Nexus 9000 or Catalyst 9000) or software (a hypervisor vSwitch).

How a frame crosses the fabric, step by step. VM-A (MAC `00aa.bb01.0001`, IP `10.10.0.11`) on Leaf 1 sends to VM-B (MAC `00aa.bb01.0002`, IP `10.10.0.12`) on Leaf 2. Both are in VLAN 10, mapped

to **VNI 10010**. Leaf 1's VTEP address is `10.0.0.11`, Leaf 2's is `10.0.0.12`. Leaf 1 already knows where VM-B's MAC is (how it knows is the next topic):

1. VM-A sends an ordinary 1500-byte Ethernet frame, `00aa.bb01.0001 → 00aa.bb01.0002`, VLAN 10, into Leaf 1.
2. Leaf 1 maps VLAN 10 → VNI 10010 and looks up the destination MAC in VNI 10010: "behind VTEP `10.0.0.12`".
3. Leaf 1 adds an 8-byte VXLAN header (VNI 10010), a UDP header (destination port **4789**, source port from a hash of the inner frame), and an outer IP header `10.0.0.11 → 10.0.0.12`. The original VLAN tag is not carried; the VNI replaces it.
4. The spine switches route the outer packet using the underlay routing table (OSPF or IS-IS learned the two loopbacks), picking one of several equal-cost paths from the UDP source-port hash.
5. Leaf 2 sees UDP 4789 to its own VTEP address, strips the outer headers, reads VNI 10010, maps it to its local VLAN for that VNI, and delivers the original frame to VM-B. VM-B can't tell it ever left the switch.

Worked example — the MTU sum. The frame from step 1 is 1514 bytes on the wire (1500 payload + 14 Ethernet header). After step 3 the outer IP packet is 1500 (inner payload) + 14 (inner Ethernet) + 8 (VXLAN) + 8 (UDP) + 20 (outer IP) = **1550 bytes**. A spine link left at the default 1500 MTU can't carry it. VTEPs must not fragment VXLAN packets (RFC 7348), and switch hardware drops oversize packets rather than fragmenting them, so the frame is lost. Small pings work and big transfers fail, the same symptom as GRE in Chapter 12. Hence jumbo MTU on every underlay link.

The Control-Plane Question: How Does a VTEP Learn Where MACs Are?

VXLAN itself only defines the encapsulation. A VTEP still needs to know "MAC 00aa.bb01.0203 in VNI 10010 is behind VTEP 10.0.0.12", and to handle broadcasts, unknown unicast and multicast (**BUM** traffic).

Approach	How BUM traffic is handled	How MACs are learned	Trade-off
Flood and learn — multicast underlay	Each VNI maps to a multicast group; BUM is sent to the group	Data plane: like a switch, learn the source MAC + source VTEP from received packets	Simple, but needs PIM in the underlay (Chapter 10) and still floods
Flood and learn — ingress replication	The VTEP unicasts a copy to every other VTEP in a configured list	Data plane learning	No multicast needed, but more replication load
BGP EVPN	Ingress replication or multicast, with peers discovered via BGP (route type 3)	**Control plane:** VTEPs advertise MAC and IP addresses in MP-BGP (route type 2); prefixes as type 5	Scales, little flooding, ARP suppression, distributed anycast gateway. The data-center standard

Approach	How BUM traffic is handled	How MACs are learned	Trade-off
LISP (SD-Access)	Fabric handles it; flooding is optional per pool	Edges register endpoints with the LISP control plane node	Campus-optimized; pull-based

How EVPN fills the table, step by step (same VMs as above):

1. VM-B sends its first frame (a gratuitous ARP when it boots, for example). Leaf 2 learns `00aa.bb01.0002 / 10.10.0.12` on a local port.
2. Leaf 2 advertises an EVPN **route type 2** in BGP: "MAC `00aa.bb01.0002`, IP `10.10.0.12`, VNI 10010, next hop `10.0.0.12`".
3. The route reflector (usually the spines) passes it to every leaf.
4. Leaf 1 installs the MAC against remote VTEP `10.0.0.12` before VM-A has sent a single frame to it.
5. When VM-A ARPs for `10.10.0.12`, Leaf 1 already knows the answer and replies itself (ARP suppression), so the broadcast never crosses the fabric.

With flood-and-learn, step 4 only happens after a frame from VM-B has actually reached Leaf 1 through the tunnel, typically the reply to an ARP that Leaf 1 first flooded to every VTEP in VNI 10010.

Why is BGP EVPN preferred over flood-and-learn? Because flood-and-learn repeats the oldest Layer 2 behaviour (flood to everyone until you learn) across a whole fabric, and it needs every VTEP to observe traffic before it knows anything. EVPN **tells** every VTEP where every MAC and IP is as soon as it appears, so unknown-unicast flooding mostly disappears, VTEPs can answer ARP locally (**ARP suppression**), and the same default gateway can exist on every leaf (**anycast gateway**). Control-plane learning replaces guessing with knowing.

VXLAN-GPO — How SD-Access Carries Policy

SD-Access uses a VXLAN variant with the **Group Policy Option (VXLAN-GPO)**, which uses some reserved header bits to carry a **16-bit Scalable Group Tag (SGT)**. The destination edge can then enforce group-based policy (Chapter 14) without looking anything up. One header carries the segment (VNI = virtual network) and the identity group (SGT).

13.5 LISP vs VXLAN at a Glance

Why compare them: the two are easy to blur because both are overlays, both use a 24-bit segment ID and both ride UDP. The difference is the job. LISP mainly answers where is it? (a lookup service), and VXLAN mainly answers how do I carry it? (an envelope). SD-Access uses LISP for the first and VXLAN for the second.

Topic	LISP	VXLAN
Main job	**Control plane** — the EID-to-RLOC mapping system (it also defines its own data plane)	**Data plane** — the encapsulation
Encapsulates	IP packets (Layer 3)	Ethernet frames (Layer 2)
UDP port	4342 control / 4341 data	**4789**
Overhead (IPv4 underlay)	36 bytes	**50 bytes**
Segment ID	Instance ID (24-bit)	VNI (24-bit)
Learning model	Pull, on demand	Flood-and-learn, or a control plane (BGP EVPN / LISP)
Where you'll meet it	SD-Access control plane, multihomed branches, mobility	Data-center fabrics (EVPN), SD-Access data plane

Exam trap: "Which protocol does SD-Access use for its data plane?" — **VXLAN** (with GPO). "Control plane?" — **LISP**. "Underlay routing?" — **IS-IS** (when Catalyst Center's LAN automation builds it). "Policy plane?" — **TrustSec/SGT**. Learn the four as a set; questions usually test whether you can keep them apart.

Chapter 14: Catalyst SD-WAN & SD-Access

> **Blueprint:** ENCOR 1.2 — Explain the working principles of the Cisco Catalyst SD-WAN solution (1.2.a control and data plane elements, 1.2.b benefits and limitations); ENCOR 1.3 — Explain the working principles of the Cisco SD-Access solution (1.3.a control and data plane elements, 1.3.b traditional campus interoperating with SD-Access)

These are Cisco's two big **controller-based architectures** for the enterprise: **SD-WAN** for connecting sites across any transport, and **SD-Access** for the campus. Both apply the same idea from the CCNA guide's SDN chapter (22.2): separate the brain (management and control) from the muscle (forwarding), centralize the brain, and express what you want as **policy** rather than box-by-box CLI.

They're also where Chapters 12 and 13 come together. SD-WAN is built on **IPsec tunnels** and **VRF-style segmentation**; SD-Access is built on **LISP + VXLAN + VRFs**.

> **Names changed in 2023.** Cisco renamed the SD-WAN components. Learn the new names, but recognise the old ones, because documentation and many exam-prep sources still use them: **SD-WAN Manager = vManage**, **SD-WAN Controller = vSmart**, **SD-WAN Validator = vBond**. **Catalyst Center = DNA Center.**

14.1 Why SD-WAN? The Problem With the Traditional WAN

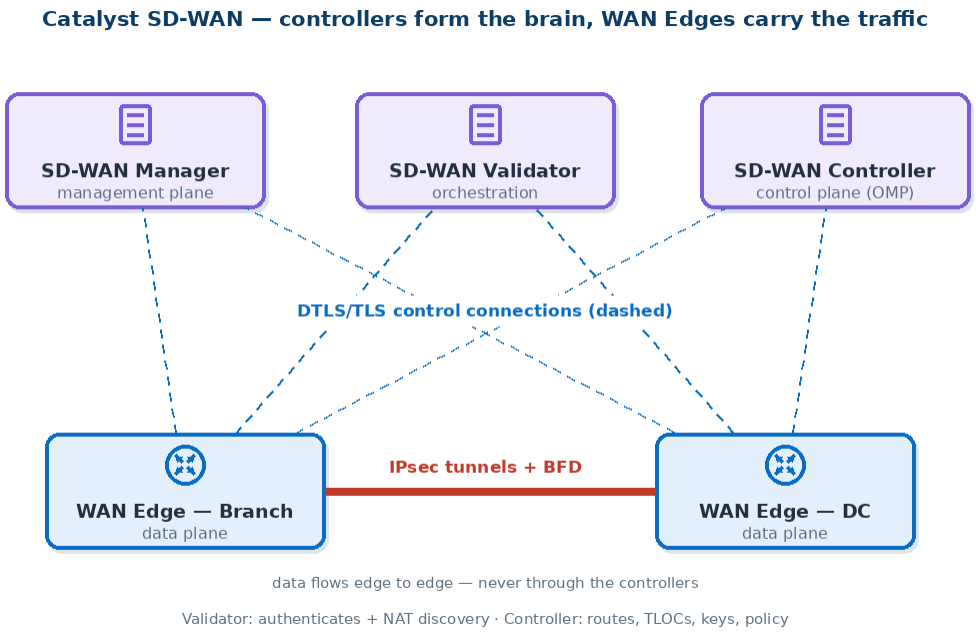
The classic enterprise WAN was **MPLS** circuits from one provider to every branch: reliable, but expensive, slow to order and rigid. Meanwhile:

- Applications moved **to the cloud** (Microsoft 365, SaaS, IaaS), but branch traffic was still backhauled to the data center first, adding latency.
- **Broadband and 4G/5G** became cheap and fast, but each is unreliable on its own.
- Every branch router was configured by hand, so policy drifted from site to site.

SD-WAN's answer: treat every transport — MPLS, internet, LTE — as a pool of interchangeable paths, build **encrypted tunnels over all of them**, measure each path continuously, and let a **central policy** decide which application uses which path.

Story Time : a traditional WAN is a company that sends every parcel by one expensive courier on one fixed route, and every branch office writes its own shipping rules on a notepad. SD-WAN is a modern logistics firm. It has contracts with several carriers (MPLS, broadband, 4G), tracks every carrier's delivery times live, writes the shipping rules once at head office, and for each parcel picks whichever carrier is meeting the promised delivery time right now. Urgent medical supplies (voice) always get the carrier that's on time today; bulk catalogues (backups) take the cheapest.

14.2 Catalyst SD-WAN Components



Component (former name)	Plane	Job
SD-WAN Manager (vManage)	**Management**	Single GUI and REST API: configuration templates/configuration groups, policy authoring, monitoring, software upgrades, certificates. Pushes config to devices over NETCONF
SD-WAN Validator (vBond)	**Orchestration**	The **first point of contact**. Authenticates every new device, tells it where the Controllers and Managers are, and helps with **NAT traversal** (it sees each device's public, post-NAT address). Must have a public, reachable IP
SD-WAN Controller (vSmart)	**Control**	The routing brain. Runs **OMP** with every WAN Edge, acting like a BGP route reflector: receives routes and TLOCs, applies **centralized control policy**, and redistributes them. Also distributes data-plane encryption keys
WAN Edge (cEdge — Catalyst 8000/8300/8500, ISR 1000/4000 on IOS XE; vEdge on legacy Viptela OS)	**Data**	The branch/DC routers. Build IPsec tunnels to each other, forward traffic, enforce data policy, measure paths

Story Time (the four roles): think of an airline. The **Validator** is the passport desk at the entrance: it checks your identity and tells you which gate to go to, and you only go back to it if you have to rejoin. The **Controller** is air traffic control: it knows where every plane is and tells each pilot which routes are open. The **Manager** is head office: it writes the timetable and the rules, and watches the dashboards. The **WAN Edges** are the planes: they actually carry the passengers. If air traffic control's radio goes quiet for a while, planes already in the air keep flying their filed routes (graceful restart, 14.4), but no new routes can be issued.

Why split it into four roles? Each one has a different scaling and security need. The Validator must sit on the public internet so devices behind NAT can reach it, but it holds no routes. The Controllers need to be highly available and scale with the number of edges. The Manager is a heavyweight database and GUI that people log in to. Separating them means a busy GUI never slows routing, and an exposed orchestrator never holds the routing table. Separate planes, separate failure domains.

Where the controllers run: on-premises (VMs in your data center), in a private cloud, or **Cisco-hosted (cloud-delivered)**, where Cisco runs the Manager, Controllers and Validators for you and you only own the WAN Edges. The architecture and roles are identical in every case; only who operates the controller VMs changes.

How a New WAN Edge Joins (Zero-Touch Provisioning)

1. The router boots with no configuration, gets an IP via DHCP on its WAN port, and contacts the **PnP Connect** cloud service (or a configured Validator address).
2. PnP Connect redirects it to the organisation's **Validator**.
3. The Validator authenticates the device by **certificate and serial number** (it must be on the authorised device list uploaded from Cisco's Plug and Play portal), then tells it the addresses of the **Controllers** and **Manager**.
4. The edge builds **permanent control connections** to the Controllers and Manager. The connection to the Validator is torn down once onboarding is done (it's transient).
5. The Manager pushes the device's configuration; the Controller starts sending routes and policy via OMP.

Onboarding prerequisites, the usual reasons a device never joins:

- **Organization name** must match exactly (case-sensitive) on every component, because it is embedded in the certificates each side checks.
- **The device must be on the authorised WAN Edge list** (its chassis/serial number), uploaded to the SD-WAN Manager and synced to the Validators and Controllers. An unknown serial is rejected at the Validator.
- **A trusted root CA chain** must be installed, so each side can validate the others' certificates (Cisco's PKI by default, or an enterprise CA).
- The edge needs a **unique system IP and site ID**, the **Validator's address** (or reachability to PnP Connect), and working DNS/NTP: certificate checks fail if the clock is wrong.

Control Connections: DTLS or TLS

All control traffic runs inside **DTLS** (Datagram TLS, over UDP, the default) or **TLS** (over TCP) tunnels, authenticated with certificates in both directions. Nothing is trusted just because it's on the right subnet. The default base port is **12346** (UDP for DTLS). Devices with several control connections use **port offsets** above it, and **port hopping** moves to 12366, 12386, 12406 and 12426 if a NAT or firewall blocks the base port. Firewalls in front of controllers must allow this range.

14.3 OMP and TLOCs — How Routes and Paths Are Shared

OMP (Overlay Management Protocol) runs between each WAN Edge and the Controllers, never edge to edge. It carries three kinds of route:

OMP route type	What it advertises
OMP routes (vRoutes)	The service-side prefixes a site can reach — its LANs, learned via connected, static, OSPF, EIGRP or BGP and redistributed into OMP
TLOC routes	The site's transport attachment points, and how to reach them

OMP route type	What it advertises
Service routes	Network services (firewall, IPS) available at a site, for service chaining

A **TLOC (Transport Locator)** identifies one WAN attachment point of one router. It is the tuple:

TLOC = System IP + Color + Encapsulation

- **System IP** — a unique router ID, like an OSPF router ID (not necessarily routable).
- **Color** — a label for the transport type: `mpls`, `biz-internet`, `public-internet`, `lte`, `private1`... Colors decide which TLOCs try to build tunnels to each other and whether to use public (post-NAT) or private addresses.
- **Encapsulation** — IPsec (normal) or GRE.

Worked example — two branches, two transports each. Branch 20 has an MPLS circuit and a broadband circuit; Branch 30 has broadband and LTE:

Router	System IP	TLOC (color)	Transport address
Branch 20	1.1.1.20	`mpls`	172.31.20.2 (provider private)
Branch 20	1.1.1.20	`biz-internet`	203.0.113.20 (public)
Branch 30	1.1.1.30	`biz-internet`	198.51.100.30 (public)
Branch 30	1.1.1.30	`lte`	100.64.7.9 (behind carrier NAT)

By default each TLOC tries to build an IPsec tunnel to every remote TLOC, so Branch 20 attempts four tunnels: mpls↔biz-internet, mpls↔lte, biz-internet↔biz-internet and biz-internet↔lte. The two from the MPLS TLOC stay down, because a private MPLS circuit has no route to internet addresses. Engineers usually add the `restrict` keyword to the `mpls` color so it only tries to build tunnels to other `mpls` TLOCs and doesn't waste time on attempts that can never come up. BFD then shows the two working internet-side tunnels as up.

Why TLOCs instead of just IP addresses? Because a WAN interface's IP can change (DHCP on broadband, NAT, a new circuit), and the overlay shouldn't care. An OMP route says "10.20.0.0/16 is reachable via TLOC {System IP 1.1.1.20, color biz-internet}". If that circuit's public address changes, only the TLOC route changes; the prefixes stay valid. Notice that's exactly LISP's split from 13.3 — identity (the prefix) is separated from location (the TLOC).

14.4 The Data Plane

The problem it solves: OMP has told every edge what is reachable and through which TLOCs. The data plane now has to carry user traffic securely between sites, notice when a path dies or gets worse, and move the right applications off it.

How a route becomes a forwarding path, step by step (Branch 20 LAN `10.20.0.0/16`, Branch 30 wants to reach it):

1. Branch 20 learns `10.20.0.0/16` from its LAN-side OSPF in service VPN 1 and redistributes it into OMP.
2. It sends an OMP route to the Controller: "VPN 1, `10.20.0.0/16`, reachable via TLOCs {1.1.1.20, mpls, ipsec} and {1.1.1.20, biz-internet, ipsec}". It also sends TLOC routes saying how to reach each TLOC (its public and private addresses) and its encryption keys.
3. The Controller applies any centralized control policy and reflects the routes to Branch 30.
4. Branch 30 installs `10.20.0.0/16` in VPN 1 with the next hops being the TLOCs it has a working (BFD-up) tunnel to.
5. A user at Branch 30 sends to `10.20.5.5`. Branch 30 encrypts the packet with Branch 20's key and sends it through the chosen IPsec tunnel. The VPN ID (1) is carried in the packet, so Branch 20 delivers it into VPN 1 and nowhere else.

- WAN Edges build **IPsec tunnels directly to each other**, one per pair of TLOCs that policy allows. By default this is a **full mesh** of tunnels across all colors; policy can restrict it to hub-and-spoke or regional topologies.
- **Encryption keys are distributed by the Controller over OMP** rather than negotiated pairwise with IKE. Each edge generates its keys, sends them to the Controller, and the Controller reflects them to the edges that need them. Why? Pairwise IKE between 1,000 sites would mean about 500,000 negotiations; one channel to the Controller scales linearly.
- **BFD** runs inside every tunnel, both to detect failure and to **measure loss, latency and jitter** continuously. The defaults are a **1 s hello** with a **multiplier of 7**, so a dead tunnel is declared down after about **7 seconds**. Tune the hello interval down per color if you need faster failover.
- **Application-Aware Routing (AAR)** uses those measurements. You define **SLA classes** ("voice: loss < 1%, latency < 150 ms, jitter < 30 ms") and map applications (identified by DPI, NBAR2) to them. If the current path breaks the SLA, traffic moves to one that meets it.

Worked example — Application-Aware Routing in action. Branch 30 has two tunnels to the data center: biz-internet and LTE. The voice SLA class is loss < 1%, latency < 150 ms, jitter < 30 ms:

Measurement window	biz-internet	LTE	Voice uses
09:00	0.1% loss, 35 ms, 4 ms jitter	0.3% loss, 70 ms, 12 ms jitter	biz-internet (both meet the SLA; policy prefers it)
09:20	3% loss, 40 ms, 25 ms jitter	0.3% loss, 72 ms, 11 ms jitter	**LTE** (biz-internet breaks the loss limit)
09:50	0.1% loss, 36 ms, 5 ms jitter	0.4% loss, 71 ms, 13 ms jitter	biz-internet again

Meanwhile email (no strict SLA) can stay on biz-internet the whole time. Note the time scale. AAR acts on **averages over polling windows** (several minutes by default), so it moves traffic away from a path that is degrading. A path that dies outright is caught much faster by BFD (about 7 seconds), and traffic simply fails over to the remaining tunnels.

Segmentation: VPNs in SD-WAN are VRFs. **VPN 0** is the transport (underlay) VPN facing the WAN; **VPN 512** is out-of-band management; **service VPNs** (e.g. 1–511) carry user traffic. The

VPN ID travels in the data packets, so segments stay separate end to end.

What if the Controllers are unreachable? The data plane keeps forwarding. OMP **graceful restart** lets edges keep using their last-known routes and keys (by default for up to 12 hours), so a controller outage isn't a site outage. New changes simply can't be made until the controllers come back.

Policy: Centralized vs Localized

SD-WAN policy comes in two families, defined in SD-WAN Manager:

Policy type	Family	Delivered to	Acts on	Example
Control policy	Centralized	The **Controllers**	OMP routes and TLOCs (the topology)	Build hub-and-spoke instead of full mesh; prefer a data-center exit
Data policy	Centralized	Controllers, then to edges via OMP	Packets at the edges	Direct internet access for guest traffic; send a subnet through a firewall (service chaining)
App-aware routing policy	Centralized	Controllers, then to edges via OMP	Applications vs SLA classes	Voice must use a path with < 150 ms latency
Localized policy	Localized	**Directly to one device** by the Manager (NETCONF)	That device's interfaces and routing protocols	QoS shaping, ACLs, route filtering on the LAN-side OSPF

Why the split? Centralized policy expresses network-wide intent ("branches talk only through hubs"), so it belongs on the Controller that sees every route. Localized policy is about one box's interfaces (queues, ACLs), so it's pushed straight to that box. A classic exam question: "Which policy changes the overlay topology?" Centralized **control** policy, because it manipulates OMP routes and TLOCs on the Controller.

14.5 SD-WAN Benefits and Limitations

Benefits	Limitations
Transport independence — MPLS, broadband and LTE are all usable, all encrypted	**Controller dependency** — the overlay needs Controllers and Validators for onboarding and changes; they must be designed for HA
Application-aware path selection based on measured SLAs	**Overhead** — IPsec + extra headers reduce usable MTU; BFD probes add a little traffic on every tunnel
Centralized policy — one place to define routing, QoS, security	**Complexity and new skills** — OMP, TLOCs, colors, templates are a new model to learn and troubleshoot
Zero-touch provisioning — ship a router, plug it in	**Licensing and cost** — subscription licences; savings depend on replacing MPLS
Direct cloud / internet access from branches (Cloud OnRamp for SaaS/IaaS)	**Internet transports have no SLA** — SD-WAN steers around problems; it can't create bandwidth that isn't there
Built-in segmentation (VPNs) and encryption	**Full-mesh scale** — tunnel counts grow as roughly n^2; large designs need regional/hub topologies

---

14.6 SD-Access — The Campus Fabric

The problem it solves: in a traditional campus, who you are is tied to where you plugged in. Your VLAN and subnet depend on the switch port, so your IP address changes as you move, and security rules written as IP ACLs must be updated on many switches whenever anything changes.

Traditional campus networks segment with VLANs and ACLs: every new group means new VLANs, new subnets, new ACL lines on many switches, and policy tied to IP addresses that change as users move. **SD-Access** replaces this with a **fabric** in which:

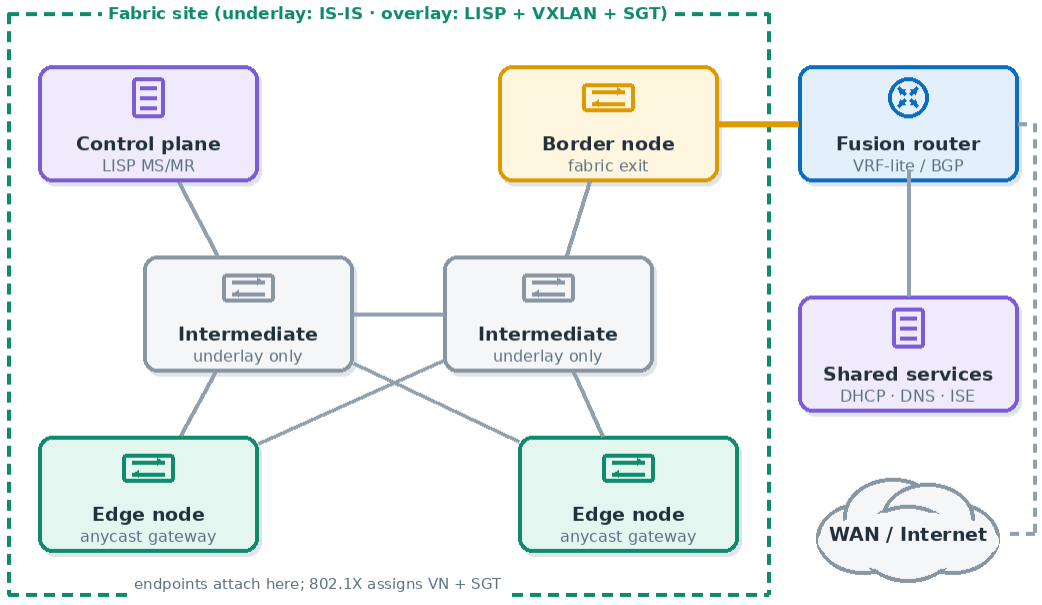
- **Identity drives policy.** A user or device authenticates (802.1X/MAB via **ISE**) and is placed in a **virtual network** and a **group** automatically, wherever it plugs in.
- **The same subnet can exist on every access switch**, so users keep their IP addresses while roaming.
- **Everything is automated** by **Catalyst Center** (Chapter 16).

The Planes of the Fabric

Plane	Technology	Role
Underlay	Routed IP, normally **IS-IS** (built by Catalyst Center LAN automation; OSPF possible for manual underlays)	Connects all fabric nodes' loopbacks. No VLANs and no STP between switches
Overlay — control plane	**LISP** (13.3)	Tracks which endpoint (EID) sits behind which edge switch (RLOC)
Overlay — data plane	**VXLAN-GPO** (13.4)	Carries endpoint traffic between fabric nodes, including the VN and the SGT
Policy plane	**Cisco TrustSec** — **SGTs** and SGACLs, from **ISE**	Decides who may talk to whom, by group
Management plane	**Catalyst Center**	Design, automation, assurance

14.7 SD-Access Fabric Roles

SD-Access fabric roles — and where the traditional network plugs in



Role	What it does
Control plane node	The **LISP Map Server/Map Resolver**. Holds the host tracking database: every endpoint's identity and which edge it's behind. Often co-located with the border

Role	What it does
Edge node	The access switch endpoints connect to. A LISP **xTR** and VXLAN VTEP. Authenticates endpoints, registers them with the control plane node, and provides the **anycast gateway**: the same gateway IP and MAC for a subnet on every edge
Border node	Connects the fabric to everything outside it: the data center, WAN, internet or a traditional network. Advertises external prefixes into the fabric and fabric prefixes out. Types: **internal** (known internal networks), **external** (default exit, e.g. to the internet), and **anywhere/internal + external**
Intermediate node	Any underlay switch between edges and borders. Just routes IP. It **knows nothing about the overlay** — it only sees VXLAN packets between loopbacks
Fabric WLC	Wireless controller integrated with the fabric. It handles the **control plane** for wireless clients (registering them with the control plane node), while AP **data** traffic is VXLAN-encapsulated straight to the edge switch. Wireless and wired then share the same policy model
Extended node	A small Layer 2 switch (e.g. Catalyst IE or compact switches) hanging off an edge, extending the fabric to places like factory floors

Story Time (anycast gateway): imagine a company where every floor of every building has a reception desk with the same phone extension, 1000. Wherever you sit, dialling 1000 reaches the desk on your floor. You never need to learn a new number when you move desks. The anycast gateway is that: the same gateway IP and MAC (say `10.10.5.1`) on every edge switch, answered locally.

How a packet crosses the fabric, step by step. A laptop plugs into Edge 1 (RLOC loopback `192.168.255.1`) and sends to a printer on Edge 2 (RLOC `192.168.255.2`):

1. **Authenticate.** The laptop does 802.1X. ISE checks the user and returns: VN Corporate, SGT **10** (Employees). Edge 1 puts the port in the matching VLAN/VN.
2. **Get an address.** The laptop gets `10.10.5.25/24` by DHCP. Its default gateway is the anycast gateway `10.10.5.1`, which Edge 1 itself answers.
3. **Register.** Edge 1 sends a LISP Map-Register to the control plane node: "`10.10.5.25/32` (in the Corporate VN's instance ID) is behind `192.168.255.1`".
4. **Look up.** The laptop sends a packet to the printer at `10.10.7.40`. Edge 1 doesn't have it in its map-cache, so it asks the control plane node, which answers: "behind `192.168.255.2`". The printer authenticated on Edge 2 earlier and was assigned SGT **20** (Printers). Edge 2 knows that tag; Edge 1 doesn't need to.
5. **Encapsulate.** Edge 1 wraps the packet in VXLAN-GPO: the VNI for Corporate, **source SGT 10**, outer IP `192.168.255.1 → 192.168.255.2`.

6. **Carry.** Intermediate nodes route on the outer header using IS-IS routes to the loopbacks. They know nothing about the laptop or the printer.
7. **Enforce.** Edge 2 decapsulates, reads SGT 10, sees the destination is SGT 20, and checks the SGACL for Employees → Printers that it downloaded from ISE. Permit, so the printer receives the packet. Had the source been SGT 30 (Guests) with a deny rule, Edge 2 would drop it right there.

Nowhere in those steps did anyone write an ACL with an IP address in it.

Why the anycast gateway? Because in a traditional campus, a user's gateway lives on one distribution switch (with HSRP, Chapter 9). Roam to a different building and the subnet, and possibly the IP address, changes. In SD-Access, **every edge node owns the same gateway** for every subnet, so wherever an endpoint lands, its gateway is one hop away and its address stays valid. LISP tells the fabric where that endpoint now lives. Mobility without stretching VLANs.

More Fabric Building Blocks

Term	Meaning
Fabric in a box (FiaB)	Border, control plane and edge roles all on **one switch** (or stack). For a small site that still wants the fabric's policy model
INFRA_VN	The VN that carries **fabric APs and extended nodes**. It maps to the **global routing table**, not a VRF, because APs need to reach the WLC and the underlay
DEFAULT_VN	A VN created by default for user endpoints when you don't need macro segmentation
Extended node	Layer 2 switch connected to an edge node. Ports are placed in VLANs that map to fabric VNs, but it **can't enforce SGACLs**; the edge node enforces for it
Policy extended node	A more capable extended node (selected Catalyst IE/compact models) that **can carry SGTs inline and enforce SGACLs** itself

Fabric APs. Access points connect to an **edge node** and run in **local mode**. They sit in the **INFRA_VN**, so they reach the WLC in the global table. Control traffic (CAPWAP) goes to the fabric WLC as normal, but **client data does not**: the AP encapsulates each client's frames in **VXLAN straight to its edge switch**, which treats the wireless client like a wired one (same VN, same SGT enforcement, same anycast gateway). Wireless becomes just another access port of the fabric.

Macro vs Micro Segmentation

Level	Mechanism	Separates	Example
Macro	**Virtual Network (VN) = a VRF**	Whole routing domains; no communication between VNs unless deliberately routed outside the fabric	Corporate VN, Guest VN, IoT VN
Micro	**Scalable Group Tag (SGT)** within a VN	Groups inside the same VN, enforced by SGACLs	In the Corporate VN, Contractors can't reach Finance-Servers

Why two levels? VRFs are strong walls but blunt. Every extra VN needs its own routing tables, and crossing between them requires a fusion device. SGTs are lightweight and plentiful, so you can write fine-grained "group A to group B" rules without new subnets or IP-based ACLs. Use VNs for "these must never meet", and SGTs for "these share a network but have different permissions".

Why tag-based policy beats IP ACLs: an IP ACL says "10.1.5.0/24 cannot reach 10.9.0.0/16". When users move, change subnet, or DHCP hands out new addresses, the rule is wrong. An SGT policy says "Contractors cannot reach Finance-Servers". The tag follows the authenticated identity, so the rule stays correct wherever the user connects. The ingress edge stamps the SGT into the VXLAN-GPO header, and the egress edge enforces the SGACL.

14.8 Traditional Campus Interoperating With SD-Access

Very few organisations rip out the whole campus at once. The blueprint asks how the fabric connects to what's already there.

1. Layer 3 handoff at the border, through a fusion router. The border node hands each VN out as a separate VRF, usually over **802.1Q subinterfaces with eBGP per VRF (VRF-Lite)**, to an external **fusion router** (or firewall).

Worked example — a fabric user gets DHCP through the fusion router. The DHCP server `10.99.0.10` sits in the shared-services network, outside the fabric, in the global routing table:

1. A laptop in the Corporate VN broadcasts DHCPDISCOVER. Edge 1 (acting as DHCP relay) unicasts it towards `10.99.0.10` inside the Corporate VRF.
2. The border's Corporate VRF has a route to `10.99.0.0/24` only because the fusion router leaked that prefix into its own Corporate VRF and advertised it to the border over eBGP. So the border sends the packet out its Corporate subinterface (VLAN 3001) to the fusion router.
3. The fusion router has leaked `10.99.0.0/24` into its Corporate VRF and `10.10.5.0/24` into the global table, so it routes the request across to the DHCP server and the reply back again.
4. A laptop in the Guest VN gets DHCP through the same fusion router, but its VRF has no leaked route to the Corporate subnets, so Guests still can't reach Corporate users.

Why is it called a "fusion" router? Because the VNs are deliberately isolated, yet they all need **shared services** — DHCP, DNS, ISE, Catalyst Center, the internet. The fusion device **leaks**

routes between VRFs (or a firewall routes between them under security policy), fusing the separate VNs with the shared-services and global routing tables at one controlled point. Isolation inside the fabric, controlled sharing at the edge of it.

2. Layer 2 handoff (Layer 2 border). A border node can bridge a fabric subnet to a **traditional VLAN** outside the fabric. Endpoints in the old network and in the fabric then share one subnet with one gateway (in the fabric). This is the migration tool: move users switch by switch, without readdressing hosts that are still on the old network.

3. Routed connection to the existing core. The traditional distribution/core simply becomes "outside" the fabric. Fabric prefixes are advertised to it from the border via BGP or redistributed into the existing IGP, and the legacy network's prefixes are imported into the fabric.

4. Underlay coexistence. Existing switches that support the fabric roles can be added; those that don't can act as intermediate nodes (pure IP routing) or stay outside the fabric entirely.

Interop need	Tool	Typical design
Fabric users reach DHCP/DNS/ISE	Fusion router / firewall	Route leaking between VN VRFs and the shared-services VRF
Fabric reaches DC, WAN, internet	External border	VRF-Lite + eBGP per VN to fusion / WAN edge
Migrate a subnet without readdressing	Layer 2 border handoff	Fabric subnet bridged to legacy VLAN during the cutover
Legacy core keeps running	Border advertises fabric prefixes	BGP or IGP redistribution at the border

Connecting Fabric Sites: Transits

A large organisation runs several **fabric sites** (a campus, a branch, a data center), and something must join them. That joining network is a **transit**:

Transit	How sites connect	What survives between sites
SD-Access transit	Native fabric: VXLAN between site borders over an IP network, with **transit control plane nodes** acting as a LISP map server between sites	VN **and SGT** stay in the VXLAN header end to end. Single policy domain
IP-based transit	Borders hand off with **VRF-Lite + BGP** (or across an SD-WAN/MPLS network)	VNs survive as VRFs, but the VXLAN header is removed, so SGTs must be re-carried another way (inline tagging on the links, or SXP)

Why does SD-Access transit need its own control plane node? Because each site's control plane node only knows its own endpoints. The **transit control plane node** aggregates the sites' prefixes, so a border can ask "which site has 10.9.4.0/24?" and tunnel straight there. It's LISP again, one level up.

Exam trap: the **control plane node** is not "the controller". The controller (management plane) is **Catalyst Center**. The control plane node is a switch running LISP MS/MR. Don't mix them up.

14.9 SD-WAN and SD-Access Side by Side

Why compare them: exam questions love swapping one architecture's terms into the other ("Which protocol does the SD-WAN Controller use?" with LISP as a distractor). Seeing them side by side shows they solve the same problem in two places.

Topic	Catalyst SD-WAN	SD-Access
Domain	WAN — connecting sites	Campus / branch LAN
Management	SD-WAN Manager	Catalyst Center
Control plane	OMP via SD-WAN Controllers	LISP via control plane nodes
Data plane	IPsec tunnels between WAN Edges	VXLAN-GPO between fabric nodes
Segmentation	VPNs (VRFs)	VNs (VRFs) + SGTs
Path / location abstraction	TLOC (System IP + color + encap)	RLOC (fabric node loopback)
Onboarding	Validator + PnP, certificates	Catalyst Center PnP / LAN automation
Policy	Centralized control and data policy, AAR	Group-based access control via ISE

The pattern repeats: a central controller holds intent, a control protocol maps identities to locations, tunnels carry the data across a simple underlay, and VRFs plus tags keep the segments apart. Learn the pattern once and both architectures make sense.

Chapter 15: Network Assurance Tools: Debugs, SNMP, Syslog, Flexible NetFlow, SPAN & IP SLA

> **Blueprint:** ENCOR 4.1 — Diagnose network problems using tools such as debugs, conditional debugs, traceroute, ping, SNMP, and syslog; ENCOR 4.2 — Configure and verify Flexible NetFlow; ENCOR 4.3 — Configure SPAN/RSPAN/ERSPAN; ENCOR 4.4 — Configure and verify IP SLA

Network assurance means being able to prove the network is doing what you intended, and to find out quickly when it isn't. The CCNA guide gave you ping, traceroute, syslog and SNMP as concepts (21.6, 17.5, 17.6). ENCOR expects you to use each one as a working diagnostic tool and

to add four more: conditional debugs, Flexible NetFlow, traffic mirroring and IP SLA.

Each tool answers a different question:

Question	Tool
Can I reach it, and by which path?	**ping, traceroute** (extended)
What is the device doing internally, packet by packet?	**debug** — ideally conditional
What has the device been telling me happened?	**syslog**
How are its counters and health trending?	**SNMP**
Who is talking to whom, how much, using what?	**Flexible NetFlow**
What exactly is in the packets?	**SPAN / RSPAN / ERSPAN** + a packet analyzer
Is the path performing well right now, continuously?	**IP SLA**

Chapter 32 returns to SNMP, logging, IP SLA and NetFlow at ENARSI **troubleshooting** depth. This chapter gets them configured and working.

15.1 Ping and Traceroute — Extended Versions

The problem these solve: someone says "I can't reach the server". Before you look at anything else, you need two facts: does traffic get there and back? and if not, where does it stop? Ping answers the first and traceroute answers the second. Both use **ICMP** (Internet Control Message Protocol), the small "status message" protocol that rides inside IP.

A basic `ping` sources from the outgoing interface with 5 × 100-byte packets. That is a very blunt question ("does something get there?"). The **extended** options let you ask much more precise questions:

```
R1# ping 10.2.2.2 source Loopback0 repeat 10 size 1500 df-bit
Type escape sequence to abort.
Sending 10, 1500-byte ICMP Echos to 10.2.2.2, timeout is 2 seconds:
Packet sent with a source address of 1.1.1.1
Packet sent with the DF bit set
MMMMMMMMMM
Success rate is 0 percent (0/10)
```

Option	Question it answers
`source Loopback0`	Does the **far end have a route back** to this network? (A default ping sources from the exit interface, which may be reachable when your LANs aren't)
`repeat 1000`	Is there **intermittent** loss? Five packets can't show 1% loss
`size 1500 df-bit`	What MTU does the path support? `M` = "fragmentation needed but DF set"
`vrf NAME`	Test inside a VRF (12.6)

Ping result codes: `!` reply · `.` timeout · `U` destination unreachable · `M` could not fragment · `Q` source quench · `&` TTL exceeded · `?` unknown packet type.

Why does sourcing from a loopback catch so many problems? Because routing is a two-way problem. A plain ping from R1 to R2 uses the link's own subnet, which R2 knows as directly connected, so it always works. Real user traffic comes from R1's LANs. If R2 has no route back to those, users fail while your basic ping succeeds. Always test the way the traffic actually flows.

Worked example — finding the path MTU. Users at a branch can open small web pages but large downloads stall. The branch reaches HQ over a GRE tunnel. **MTU** (Maximum Transmission Unit) is the largest packet a link can carry without splitting it, and the **DF** (Don't Fragment) bit tells routers "drop this rather than split it". So pinging with DF set and a growing size finds the biggest packet the whole path allows:

```
R1# ping 10.2.2.2 source Loopback0 size 1500 df-bit
MMMMM                                    ! 1500 is too big somewhere on the path
R1# ping 10.2.2.2 source Loopback0 size 1476 df-bit
!!!!!                                    ! 1476 fits
R1# ping 10.2.2.2 source Loopback0 size 1477 df-bit
MMMMM                                    ! 1477 does not: the path MTU is 1476
```

What the router did, and why: a router on the path had a 1476-byte limit (a 1500-byte link minus the 24 bytes of GRE plus outer IP header, Chapter 12). It could not split a DF packet, so it dropped it and sent back ICMP "fragmentation needed", which ping shows as `M`. The fix is on the tunnel (`ip tcp adjust-mss 1436`, so TCP never builds packets bigger than 1476 bytes, with `ip mtu 1476` set to match), not on the users' PCs. One sweep of `size` with `df-bit` turns "large downloads stall" into a number.

Traceroute (IOS sends UDP probes with increasing TTL) shows the path hop by hop, with these markers: `*` no reply within the timeout · `!H` host unreachable · `!N` network unreachable · `!P` protocol unreachable · `!A` administratively prohibited (an ACL). A trace that stops at one hop while the next hop's own pings work usually points at an ACL, or at a device configured not to send ICMP unreachables.

How does traceroute know it has arrived? Each intermediate router whose TTL expires answers with ICMP **time exceeded**. The destination itself isn't expiring anything; it receives a UDP probe aimed at a high port nobody listens on (33434 and up) and answers with ICMP **port unreachable**. That different reply is the signal that the trace is complete. If the destination filters UDP or doesn't send unreachables, the last line is `* * *` repeated to the 30-hop maximum, even though the host is reachable.

Story Time : Every IP packet carries a **TTL** (Time To Live), a counter that each router lowers by 1. When it hits 0 the router throws the packet away and sends a note back saying "your packet died here". Traceroute abuses that on purpose. It's like sending a letter with the instruction "the 1st post office to touch this, send me a postcard", then another saying "the 2nd post office...", and so on. The postcards arrive from each post office in turn, and together they draw you the route.

How it works, step by step (R1 tracing to 10.3.3.3, three routers away):

1. R1 sends three UDP probes with **TTL 1**. R2 lowers TTL to 0, drops them, and replies with ICMP time exceeded from its own address. R1 prints line 1: R2's address and three round-trip times.

2. R1 sends three probes with **TTL 2**. R2 forwards them (TTL now 1), R3 drops them and replies. Line 2: R3.
3. TTL 3 reaches 10.3.3.3 itself. Nothing expires; the host finds no application on port 33434+ and replies **port unreachable**. Line 3, and the trace stops.

```
R1# traceroute 10.3.3.3 source Loopback0
  1 10.1.12.2 1 msec 1 msec 1 msec       ! R2 answered "time exceeded"
  2 10.1.23.3 2 msec 1 msec 2 msec       ! R3 answered "time exceeded"
  3 10.3.3.3 3 msec *  2 msec            ! destination answered "port unreachable"
```

The single `*` in line 3 is normal: many devices rate-limit ICMP replies, so one probe of three goes unanswered. A whole line of `* * *` followed by more hops that do answer usually means one router simply doesn't send time-exceeded messages; the path still works.

15.2 Debugs — and Doing Them Safely

The problem debugs solve: `show` commands give you the result (the neighbor isn't there). They don't tell you why. To see why, you need to watch the protocol's own decisions as they happen: "I received a hello, I rejected it, here's the reason."

`debug` makes IOS print events from a process as they happen: `debug ip ospf adj`, `debug ip bgp updates`, `debug crypto ikev2`. It is the most detailed tool you have, and the most dangerous.

Story Time : A debug is like asking a busy shop cashier to also say out loud, and write down, every single thing they do. With three customers that's helpful. With three hundred, the cashier spends all their time talking and writing, and the queue stops moving. The router's CPU is that cashier.

Why are debugs dangerous? Debug output is generated by the **CPU**. A debug that fires per packet (`debug ip packet` on a busy router) can drive the CPU to 100%, and the router may then drop routing protocol hellos, lose neighbors and lock you out of the console. A debug can cause a bigger outage than the one you were investigating.

Safe debugging habits:

```
R1(config)# no logging console                ! don't send debug output to the slow console
R1(config)# logging buffered 1000000 debugging ! to RAM instead - read: show logging
R1(config)# service timestamps debug datetime msec   ! so you can correlate events
R1# terminal monitor                          ! see log output on an SSH/VTY session
R1# undebug all                               ! (or: no debug all) - the stop button
```

Why does `terminal monitor` matter? Log and debug messages go to the console by default. If you're connected over SSH, you see **nothing** until you enter `terminal monitor`. It applies only to your current session, so you must repeat it on each login. "I turned on debug and nothing happened" is usually a missing `terminal monitor`.

Conditional Debugs — Narrowing the Firehose

A **debug condition** limits debug output to traffic that matches a filter, such as an interface, an IP address or a peer:

```
R1# debug condition interface GigabitEthernet0/1   ! only events on this interface
R1# debug condition ip 10.1.1.50                    ! only events involving this host
R1# debug ip ospf adj
R1# show debug condition                           ! lists the active conditions
R1# undebug all
R1# no debug condition all                          ! conditions persist - clear them too
```

Many feature debugs also accept their own filters, for example an ACL with `debug ip packet 101 detail`, or a neighbor with `debug ip bgp 10.0.0.2 updates`.

How a conditional debug works, step by step:

1. You set one or more conditions (`debug condition interface Gi0/1`). Nothing prints yet; a condition is only a filter.
2. You turn on a debug (`debug ip ospf hello`). The process now generates its event messages.
3. Before each message is printed, IOS checks it against the conditions. Messages about Gi0/1 pass; messages about the other interfaces are thrown away.
4. Only the matching lines reach the log buffer or your session.

Not every debug honours every condition type, so check `show debug condition` and confirm the output really is narrowed before you trust it on a busy box.

Worked example. A core router has 40 OSPF neighbors. One of them, on Gi0/1, never forms an adjacency. Debugging OSPF hellos on all 40 would produce a line per hello per neighbor, so you narrow it to the one interface:

```
R1# debug condition interface GigabitEthernet0/1
R1# debug ip ospf hello
OSPF-1 HELLO Gi0/1: Rcv hello from 2.2.2.2 area 0 10.1.12.2
OSPF-1 HELLO Gi0/1: Mismatched hello parameters from 10.1.12.2
OSPF-1 HELLO Gi0/1: Dead R 120 C 40, Hello R 30 C 10
R1# undebug all
R1# no debug condition all
```

Reading it: `R` is what was **received** from the neighbor and `C` is what is **configured** here. The neighbor sends hellos every 30 s with a 120 s dead time (the OSPF non-broadcast defaults), and R1 expects 10 and 40. So the neighbor's interface is probably set to a different OSPF network type. The exact prefix on each line varies a little by release, but the `R`/`C` comparison is the part to look for. Three lines of output, one clear answer, and the CPU barely noticed.

Exam trap: debug conditions **stay in place** after `undebug all`. If you forget to clear them, the next engineer's debug silently shows only part of the picture.

15.3 Syslog

The problem syslog solves: problems often happen when nobody is watching: an interface flaps at 03:00, a BGP session resets, a fan fails. **Syslog** is how a device writes down what happened, so you can read it later and so a central server can collect it from every device in one place.

Story Time : Syslog is a ship's logbook. Every notable event gets a line with the time, who reported it and how serious it was. The copy kept on board (the local buffer) is handy but goes down with the ship (a reload). The copy radioed to shore (the syslog server) survives. And the captain can decide that only serious entries get radioed, to keep the airwaves quiet.

Every IOS log message has a **severity** from 0 to 7. Lower numbers are more severe, and choosing a level includes **that level and everything more severe**.

Level	Keyword	Example
0	emergencies	System unusable
1	alerts	Immediate action needed (e.g. temperature)
2	critical	Critical condition (hardware failure)
3	errors	Interface errors, failed operations
4	warnings	Config warnings, approaching limits
5	notifications	Normal but significant: interface up/down, neighbor changes (`%LINEPROTO-5-UPDOWN`)
6	informational	ACL log hits, general info
7	debugging	Debug output

Mnemonic: Every Awesome Cisco Engineer Will Need Ice-cream Daily.

Reading a message. Every IOS log line has the same shape:

```
*Oct 26 14:02:11.318 EST: %LINK-3-UPDOWN: Interface Gi0/1, changed state to down
```

Part	Here	Meaning
Timestamp	`Oct 26 14:02:11.318 EST`	From `service timestamps` (a leading `*` means the clock isn't NTP-synchronized)
FACILITY	`LINK`	The subsystem that generated it (`LINEPROTO`, `OSPF`, `BGP`, `SYS`…)
SEVERITY	`3`	0–7 as in the table above (here, errors)
MNEMONIC	`UPDOWN`	Short code for the event type
Description	`Interface Gi0/1, …`	The human-readable detail

So `%FACILITY-SEVERITY-MNEMONIC` tells you, before you read a word of the description, who is talking and how serious it is.

Where messages go, and each destination's own level:

Destination	Command	Default level	Notes
Console	`logging console <level>`	debugging (7)	Slow serial line; heavy logging here can hurt CPU
VTY sessions	`logging monitor <level>`	debugging (7)	Only shown in a session that ran `terminal monitor`

Destination	Command	Default level	Notes
Local buffer	`logging buffered <size> <level>`	debugging (7)	Read with `show logging`; lost on reload
Syslog server	`logging trap <level>` + `logging host`	informational (6)	The only destination that survives a reload

Each destination filters independently. Sending debugs to the buffer but only warnings to the server is a normal design.

Worked example — where does each message go? R1 has this logging setup:

```
logging console warnings          ! console: levels 0-4
logging buffered 64000 informational  ! buffer: levels 0-6
logging trap notifications        ! server: levels 0-5
logging host 10.0.0.50
```

Gi0/1 goes down, and IOS generates two messages:

Message	Severity	Console (0–4)	Buffer (0–6)	Server (0–5)
`%LINK-3-UPDOWN: Interface Gi0/1, changed state to down`	3	**Yes**	**Yes**	**Yes**
`%LINEPROTO-5-UPDOWN: Line protocol on Interface Gi0/1, changed state to down`	5	No	**Yes**	**Yes**

How the router decides: for each destination, it compares the message's severity number with that destination's level. If the message number is **less than or equal to** the configured level, it is sent. 3 ≤ 4, so the LINK message reaches the console; 5 > 4, so the LINEPROTO message doesn't. Remember "lower number = more serious, and a level includes everything more serious than itself".

```
R1(config)# service timestamps log datetime msec localtime show-timezone
R1(config)# logging buffered 64000 informational  ! local RAM buffer, levels 0-6
R1(config)# logging host 10.0.0.50                ! syslog server, UDP 514
R1(config)# logging trap warnings                 ! send levels 0-4 to the server
R1(config)# logging source-interface Loopback0    ! consistent source IP
R1# show logging                                  ! settings + the local buffer
```

Why set `logging source-interface`? The syslog server identifies devices by source IP. Without it, a router's messages come from whichever interface faces the server, which can change after a reroute, so its logs appear under a different "device". A loopback is always the same.

Why are timestamps essential? Because troubleshooting is **correlating events across devices**: "BGP dropped on R1 at 14:02:11.318 — what happened on R2 at that moment?" Without accurate time (NTP, Chapter 8) and millisecond timestamps, logs from different devices can't be lined up. Logs without synchronised timestamps are anecdotes.

15.4 SNMP

The problem SNMP solves: you have 500 devices and you want graphs of every interface's traffic, CPU and memory, plus an alert the moment a link drops. Logging into each one to type `show` commands can't do that. **SNMP** (Simple Network Management Protocol) gives a central server a standard way to ask every device for numbers, and gives devices a way to tell the server when something happens.

SNMP lets a **manager** (the **NMS**, Network Management Station) read and write **MIB** variables on an **agent** (the SNMP software on the device). Agents listen on **UDP 161**; traps and informs go to the manager on **UDP 162**.

Story Time : Think of the device as a filing cabinet with thousands of numbered drawers, each holding one value (a counter, a temperature, an interface name). The MIB is the cabinet's index. The manager is a clerk who phones up and says "read me drawer 1.3.6.1.2.1.2.2.1.10.3" (a **poll**). A **trap** is the cabinet phoning the clerk unprompted: "drawer 'link status' just changed!" A trap is a shout across the room; an **inform** is a shout that waits for "got it" and repeats itself until it hears it.

MIBs and OIDs. A **MIB** (Management Information Base) is a structured definition of what can be read or set: interface counters, CPU load, routing table entries. Every individual variable has an **OID** (Object Identifier), a dotted path through a global tree. For example `1.3.6.1.2.1.2.2.1.10` is `ifInOctets` (under iso.org.dod.internet.mgmt.mib-2.interfaces). Standard MIBs live under `1.3.6.1.2.1` (mib-2); vendor MIBs under `1.3.6.1.4.1` (enterprises; Cisco is `.9`). Appending an index selects one instance: `…10.3` is the input octet counter of interface index 3.

Operation	Direction	What it does
Get	Manager → agent	Read one OID
GetNext	Manager → agent	Read the next OID in the tree (how tables are walked)
GetBulk (v2c+)	Manager → agent	Read many next OIDs in one request
Walk	Manager tool	Repeated GetNext/GetBulk to dump a whole subtree (`snmpwalk`)
Set	Manager → agent	Write a value (needs RW access)
Trap / Inform	Agent → manager	Unsolicited notification of an event

How a poll works, step by step (SNMPv2c, community `R3ad0nly`, the config shown below):

1. The NMS at 10.0.0.50 sends a **GetRequest** to R1 on UDP 161, carrying the community string and the OID `1.3.6.1.2.1.2.2.1.10.3` (ifInOctets for interface index 3).
2. R1's agent looks up the community string, then checks the request's source address against the ACL attached to that community (20). If either check fails, it silently drops the request, so the NMS just sees a timeout.
3. The agent reads the live counter and sends a **GetResponse** back with the value.

Worked example — turning counters into a graph. SNMP counters only ever count up. The NMS polls `ifInOctets` on Gi0/0 every 300 seconds:

Poll time	ifInOctets value
10:00:00	1,000,000,000
10:05:00	1,075,000,000

Difference: 75,000,000 bytes in 300 s. × 8 bits = 600,000,000 bits ÷ 300 s = **2,000,000 bit/s, or 2 Mbps** average inbound. That's every traffic graph you've ever seen: two readings, a subtraction, a division. SNMP gives raw counters; the NMS does the maths. It's also why a 5-minute poll can't show a 10-second burst: the burst is averaged away. (On fast links, NMSs poll the 64-bit `ifHCInOctets` instead, because a 32-bit counter wraps back to 0 after about 4.3 GB, which a 10 Gbps link fills in under 4 seconds.)

How a trap works, step by step: Gi0/1 goes down → the agent builds a `linkDown` notification naming the interface → it sends it once to 10.0.0.50 on UDP 162 → done. If that one UDP packet is lost, the NMS never hears about it, which is why NMSs still poll to catch missed events.

Version	Security	Notes
v1 / v2c	**Community string**, sent in clear text	v2c added GetBulk and informs. Read-only (RO) or read-write (RW) communities
v3	**Users and groups**, with authentication and encryption	The only version you should deploy today

SNMPv3 Security Levels

Why v3 exists: in v1/v2c the community string is the password, and it crosses the network in clear text in every packet. Anyone who captures one poll can read the device, and an RW community lets them change it. v3 replaces communities with named **users**, proves each message really came from that user (**auth**, a keyed hash) and can encrypt the contents (**priv**). The level names say which of those you get:

Level	Authentication	Encryption	Keyword
noAuthNoPriv	Username only	No	`noauth`
authNoPriv	HMAC-SHA (MD5 is legacy)	No	`auth`
authPriv	HMAC-SHA (MD5 is legacy)	**Yes** (AES; DES/3DES are legacy)	`priv`

Traps vs Informs

- **Trap** — the agent sends a notification and forgets it. If the UDP packet is lost, the event is lost.
- **Inform** — the manager must **acknowledge** it, and the agent retransmits until it does. Reliable, but uses more memory and traffic.

How an inform works, step by step: the link goes down → the agent sends an InformRequest to the NMS on UDP 162 and **keeps a copy in memory** → the NMS replies with a Response (the acknowledgement) → the agent deletes its copy. If no Response arrives before the timeout, the agent sends the same inform again, up to its retry limit, and only then gives up. `show snmp` shows counters of informs sent, retried and dropped.

Why not always use informs? Because during a big failure, when hundreds of events fire, the device must hold every unacknowledged inform in memory and keep retransmitting, possibly toward a manager it can no longer reach. Informs are for events you can't afford to lose; traps are fine for high-volume, lower-value ones.

Configuring SNMPv2c (Read-Only, Restricted)

You'll still see v2c on older estates and in exam exhibits. The only protection a community string has is the ACL attached to it:

```
R1(config)# access-list 20 permit host 10.0.0.50                  ! the NMS only
R1(config)# snmp-server community R3ad0nly RO 20                  ! RO + ACL
R1(config)# snmp-server host 10.0.0.50 version 2c R3ad0nly        ! trap destination
R1(config)# snmp-server enable traps
```

Why the ACL matters: the community string crosses the network in clear text. Anyone who captures it can poll the device, and with an RW community, change its configuration. Restricting the community to the NMS address is the minimum; v3 is the real fix.

Configuring SNMPv3 (authPriv)

```
R1(config)# snmp-server view ALL-MIB iso included             ! what the group can see
R1(config)# snmp-server group NMS-GRP v3 priv read ALL-MIB    ! group requires authPriv
R1(config)# snmp-server user nms-admin NMS-GRP v3 auth sha AuthPass#1 priv aes 128 PrivPass#1
R1(config)# snmp-server host 10.0.0.50 version 3 priv nms-admin   ! where to send traps
R1(config)# snmp-server enable traps                          ! or enable specific ones
R1# show snmp user
R1# show snmp group
```

Informs over v3 need one more step. For a trap, the sending router is the authoritative SNMP engine, so the user above (keyed to the router's own engine ID) works. For an **inform**, the receiver is authoritative, because it must acknowledge. The router therefore needs the manager's engine ID and a copy of the user localized to it:

```
! (global configuration mode)
snmp-server engineID remote 10.0.0.51 800000090300AABBCCDDEE01
snmp-server user nms-admin NMS-GRP remote 10.0.0.51 v3 auth sha Auth#1 priv aes 128 Priv#1
snmp-server host 10.0.0.51 informs version 3 priv nms-admin
```

Configure the `engineID remote` **before** the remote user: the user's keys are computed from that engine ID. Skip it, and the informs are sent but never acknowledged, so the router keeps retransmitting them and the manager never logs a single event.

A quirk worth knowing: SNMPv3 users **don't appear in** `show running-config`. Their keys are localized and hashed, so you verify them with `show snmp user`. This surprises people the first time they check a config backup.

15.5 Flexible NetFlow

SNMP tells you an interface is 90% busy. **NetFlow** tells you **who** is using it: which hosts, which applications, how many bytes. A **flow** is a set of packets sharing the same **key fields**, classically the 7-tuple: source/destination IP, source/destination port, protocol, ToS and input interface.

Flexible NetFlow (FNF) is the modern IOS XE implementation. You choose exactly which fields define a flow and which are collected. It is built from four parts:

Component	Job	Command
Flow record	Which fields are **keys** (`match`, define a flow) and which are **non-key** (`collect`, measured per flow)	`flow record`
Flow exporter	Where to send flow data, how, and in what format	`flow exporter`
Flow monitor	Ties a record to one or more exporters; holds the **cache**	`flow monitor`
Interface	Applies the monitor, input and/or output	`ip flow monitor … input`

Story Time : NetFlow is your phone bill, not a wiretap. The itemised bill lists every call: who called whom, when, for how long. It never records what was said. That's exactly enough to spot "someone called an overseas number for six hours at 3 a.m.", and it costs almost nothing to keep. When you need to hear the actual words, you need a recording: that's SPAN (15.6).

Why the key/non-key distinction? Key fields decide what counts as a separate flow. Add `match transport destination-port` and HTTPS and SSH between the same hosts become separate flows. Non-key fields are just measurements recorded about each flow: bytes, packets, timestamps. Keys determine the granularity of the answer; non-keys determine what you learn about each flow.

How the flow cache works, step by step:

1. A packet arrives on an interface with a flow monitor applied.
2. The router reads the **key fields** from it (with the record below: source/destination IP, protocol, source/destination port, input interface).
3. It looks those keys up in the **cache**, a table held in memory.

- **No entry yet:** a new flow entry is created, with packets = 1, bytes = this packet's size, first and last timestamps = now.
- **Entry exists:** packets +1, bytes + this packet's size, last timestamp = now. 4. An entry is **expired** (removed from the cache and exported) when the flow ends (TCP FIN or RST), when no packet has matched it for the **inactive timeout** (15 s default), when it has existed for the **active timeout** (1800 s default), or when the cache is full. 5. The exporter packs expired entries into NetFlow v9 or IPFIX packets and sends them to the collector over UDP. Because v9/IPFIX are template-based, it first sends a **template** saying which fields each record contains, so the collector knows how to read them.

Worked example. PC 10.1.1.25 opens one HTTPS session to 52.96.10.4 and sends three packets of 1500, 1500 and 600 bytes, all from source port 51544:

After packet	Cache entry (10.1.1.25:51544 → 52.96.10.4:443, TCP, Gi0/0)
1	new entry: pkts 1, bytes 1500
2	same keys, update: pkts 2, bytes 3000
3	same keys, update: pkts 3, bytes 3600

Now the PC opens a **second** session (new source port 51545) to the same server. One key differs, so it becomes a **second** entry. If your record had not matched on source port, both sessions would merge into one entry. That's the key-field choice in action.

Configuring It (Try It!)

```
flow record REC-WAN
 match ipv4 source address             ! key fields - define the flow
 match ipv4 destination address
 match ipv4 protocol
 match transport source-port
 match transport destination-port
 match interface input
 collect counter bytes long            ! non-key fields - measured per flow
 collect counter packets long
 collect timestamp absolute first
 collect timestamp absolute last
!
flow exporter EXP-COLLECTOR
 destination 10.0.0.60                 ! the NetFlow collector
 source Loopback0
 transport udp 2055                    ! collector's port (2055, 9995, 9996 are common)
 export-protocol netflow-v9            ! or ipfix
!
flow monitor MON-WAN
 record REC-WAN
 exporter EXP-COLLECTOR
 cache timeout active 60               ! export long-lived flows every 60 s
!
interface GigabitEthernet0/0
 ip flow monitor MON-WAN input
 ip flow monitor MON-WAN output
```

Verifying

```
R1# show flow monitor MON-WAN cache format table
IPV4 SRC ADDR  IPV4 DST ADDR  TRNS SRC PORT  TRNS DST PORT  IP PROT  bytes long  pkts long
=============  =============  ==============  ==============  =======  ==========  =========
10.1.1.25      52.96.10.4              51544             443        6    18422190      13021
10.1.1.40      10.2.0.10               62011              22        6       41250        310

R1# show flow exporter EXP-COLLECTOR statistics    ! are packets actually being sent?
R1# show flow interface GigabitEthernet0/0         ! which monitors are applied where
R1# show flow monitor MON-WAN statistics           ! cache size, active/added/aged flows
```

Why `cache timeout active`? A flow is normally exported when it **ends** (TCP FIN/RST, or inactive for 15 seconds by default). A two-hour file transfer would then appear on the collector only after it finished, so for two hours the graphs would show nothing. The active timeout forces long flows to

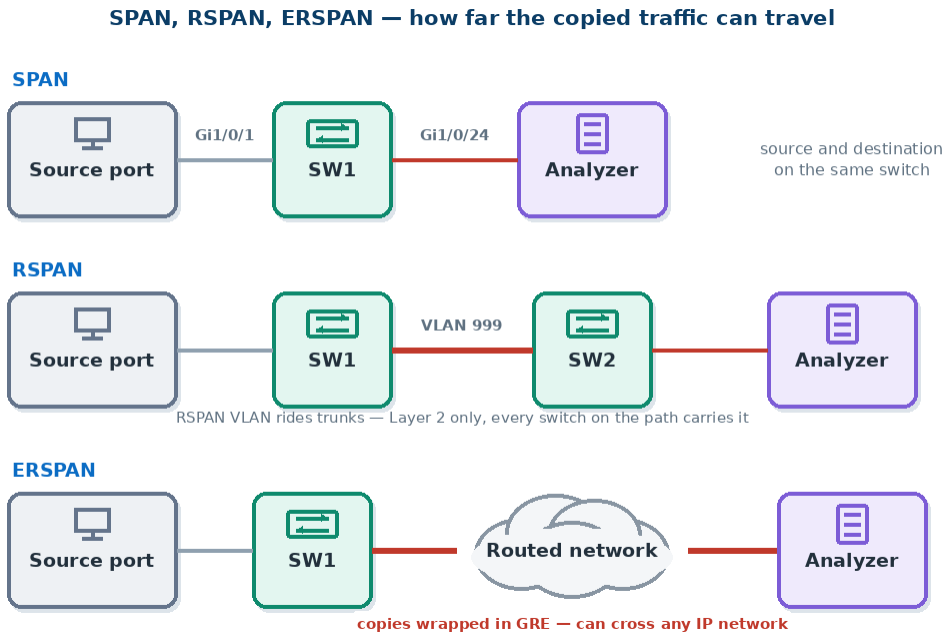
be exported in slices (default 1800 s; 60 s is common), so the collector sees traffic as it happens.

Common FNF problems: monitor applied in only one direction (half the conversation is missing); the exporter's source interface has no route to the collector; a collector expecting a different port or format (v5 vs v9 vs IPFIX). **NetFlow v5** has a fixed format (IPv4 only, no templates); **v9 and IPFIX** are template-based, which is what makes flexible records possible.

15.6 SPAN, RSPAN and ERSPAN — Copying Traffic to an Analyzer

NetFlow summarizes traffic; sometimes you need the **actual packets**, for Wireshark, an IDS (Intrusion Detection System) or a recorder. A switch normally forwards a frame only to its destination port, so a laptop plugged into a spare port sees almost nothing. You have to tell the switch to **mirror** traffic to a monitoring port. **SPAN** stands for Switched Port ANalyzer.

Story Time : SPAN is a photocopier in the mail room. Every letter going to or from one desk is still delivered normally, but a photocopy of it is also dropped into your tray. RSPAN sends the photocopies to another building through a private internal mail route (a dedicated VLAN). ERSPAN puts them in a sealed envelope and posts them through the public mail (GRE over IP) to anywhere. The analogy breaks in one place: a real photocopier waits for you, but a SPAN port that's too slow for the volume of copies just throws the extras away.



Type	Source and destination	Transport for the copies
SPAN (local)	Same switch	Internal
RSPAN (remote)	Different switches, same Layer 2 domain	A dedicated **RSPAN VLAN** across trunks
ERSPAN (encapsulated remote)	Anywhere reachable by IP	**GRE** across a routed network

Local SPAN

```
SW1(config)# monitor session 1 source interface Gi1/0/1 both    ! rx, tx, or both
! VLANs can be sources too (monitor session 2 source vlan 10 rx), but a session's
! sources must be all ports or all VLANs, never a mix
SW1(config)# monitor session 1 destination interface Gi1/0/24   ! the analyzer port
SW1# show monitor session 1
Session 1
---------
Type                        : Local Session
Source Ports                :
    Both                    : Gi1/0/1
Destination Ports           : Gi1/0/24
    Encapsulation           : Native
          Ingress           : Disabled
```

How the copy path works: a frame arrives on Gi1/0/1 (an **rx**, received, copy is taken) → the switch forwards the original normally → when a frame leaves Gi1/0/1 a **tx**, transmitted, copy is taken → the hardware queues every copy to Gi1/0/24 → the analyzer receives them. `Encapsulation: Native` means the copies go out untagged (add `encapsulation replicate` to keep the original VLAN tags), and `Ingress: Disabled` means Gi1/0/24 ignores anything the analyzer sends.

Worked example — will the capture be complete? Gi1/0/1 is a 1 Gbps server port. During the problem it receives 700 Mbps and sends 600 Mbps. With source `both`, the copies total 700 + 600 = **1.3 Gbps**, but Gi1/0/24 is also 1 Gbps. About 300 Mbps of copies are dropped, with no error message. Wireshark then flags "previous segment not captured" for packets that were actually delivered fine, and you can waste a day chasing loss that only exists in the capture. Fixes: mirror only `rx` (or only `tx`), limit a trunk source to the VLANs you care about (`monitor session 1 filter vlan 10`), or use a 10 Gbps destination port.

RSPAN

```
! On EVERY switch the copies cross:
vlan 999
 remote-span                            ! marks it as RSPAN - no MAC learning in this VLAN
!
! Source switch
SW1(config)# monitor session 1 source interface Gi1/0/1 both
SW1(config)# monitor session 1 destination remote vlan 999
!
! Destination switch
SW2(config)# monitor session 1 source remote vlan 999
SW2(config)# monitor session 1 destination interface Gi1/0/24
```

Why must the VLAN be flagged `remote-span`? Because mirrored frames still carry their **original** MAC addresses. In a normal VLAN, intermediate switches would learn those MACs on the wrong port (the mirror path) and could start forwarding real traffic for those hosts along it. In an RSPAN VLAN, MAC learning is disabled, so the copies simply flood along to the destination. The RSPAN VLAN must also be **allowed on every trunk** in the path.

RSPAN walk-through: SW1 copies Gi1/0/1's frames → it puts the copies into VLAN 999 and sends them up its trunk → the middle switch receives them in VLAN 999, learns no MACs, and floods them out its other trunks allowed for VLAN 999 → SW2 takes everything arriving in VLAN 999 and delivers it to Gi1/0/24. If any trunk on the way doesn't allow VLAN 999 (`switchport trunk`

`allowed vlan`), the copies stop there and the analyzer sees nothing.

ERSPAN (IOS XE)

Why ERSPAN exists: RSPAN only works where VLAN 999 can stretch, so the source and analyzer must sit in the same Layer 2 domain. Modern campuses route between distribution blocks, and the analyzer may be in a data centre across a WAN. ERSPAN wraps each copied frame in **GRE** (Generic Routing Encapsulation, the same tunnel header as Chapter 12) inside an ordinary IP packet, so the copies can be routed anywhere the source can reach.

```
! Source router/switch
monitor session 10 type erspan-source
 source interface GigabitEthernet1/0/1 both
 no shutdown                          ! ERSPAN sessions are shut by default
 destination
  erspan-id 100                       ! must match on the receiver
  ip address 10.9.9.9                 ! the receiving device or analyzer
  origin ip address 10.1.1.1          ! source of the GRE packets
!
! Destination switch (if a switch, not an analyzer, decapsulates)
monitor session 20 type erspan-destination
 destination interface GigabitEthernet1/0/24
 no shutdown
 source
  erspan-id 100
  ip address 10.9.9.9
```

Many analyzers (Wireshark, commercial probes) can receive and decode ERSPAN directly, so a destination session isn't always needed.

SPAN Rules and Limits

- A **destination port** stops being a normal switch port: it doesn't forward ordinary traffic or take part in STP, and by default it ignores incoming traffic.
- **Oversubscription:** mirroring `both` directions of a 1 Gbps port can produce up to 2 Gbps of copies, more than a 1 Gbps destination can carry. The excess is **silently dropped**, so your capture may be missing packets.
- The number of monitor sessions is limited by platform, often to just a few per switch. Check the platform's limits before you rely on another one being free.
- Mirroring adds load; ERSPAN adds GRE overhead to the network it crosses (so mind the MTU).
- A port can't be both a source and a destination, and an EtherChannel member can't be a destination.

15.7 IP SLA — Measuring the Path Continuously

Ping tells you about **now**, when you happen to run it. **IP SLA** (Service Level Agreement) makes the router run probes **continuously, on a schedule**, keep statistics, and (combined with tracking) **react** when performance degrades.

Why that matters: your ISP's link can stay physically up while the ISP's network behind it is broken. The interface says "up/up", the default route stays in the table, and every user is cut off. Only a test that actually crosses the ISP's network, repeated all day, notices.

Story Time : IP SLA is a nurse doing hourly observations. Nobody waits for the patient to complain; the nurse takes a pulse on a timetable, writes it on the chart, and calls the doctor when a reading crosses a line. The probe is the pulse check, `frequency` is the timetable, `show ip sla statistics` is the chart, and the **track** object (below) is the call to the doctor.

How an operation runs, step by step (operation 10 below):

1. Every `frequency` seconds (10), R1 sends one ICMP echo to 8.8.8.8 out of Gi0/0.
2. It waits up to `timeout` (1000 ms) for the reply.
3. It sets the **return code**: **OK** if the reply came back within `threshold` (300 ms); **Over threshold** if it came back but took longer; **Timeout** if it never came.
4. It updates the statistics (latest RTT, success and failure counts).
5. Anything tracking the operation reads the new return code and decides whether it is up or down.

```
R1(config)# ip sla 10
R1(config-ip-sla)#  icmp-echo 8.8.8.8 source-interface GigabitEthernet0/0
R1(config-ip-sla-echo)#  frequency 10            ! probe every 10 s (default 60)
R1(config-ip-sla-echo)#  threshold 300           ! ms - above this, "over threshold"
R1(config-ip-sla-echo)#  timeout 1000            ! ms to wait for a reply
R1(config)# ip sla schedule 10 life forever start-time now
```

```
R1# show ip sla statistics 10
IPSLA operation id: 10
        Latest RTT: 14 milliseconds
Latest operation start time: 10:42:31 UTC Mon Oct 26 2026
Latest operation return code: OK
Number of successes: 412
Number of failures: 3
Operation time to live: Forever

R1# show ip sla configuration 10
R1# show ip sla summary
```

Why `life forever`? An operation that's configured but not **scheduled** never runs, and the default life is 3600 seconds, after which it silently stops. Forgetting the schedule, or leaving the default life, is the classic "my IP SLA does nothing" problem.

Operation Types You Should Recognise

Operation	Measures	Needs a responder?
`icmp-echo`	Reachability and round-trip time	No — any device that answers ping
`udp-jitter`	Delay, **jitter**, loss in each direction, MOS for voice	**Yes** — `ip sla responder` on the far Cisco device
`tcp-connect`	Can a TCP session be opened to a port (application up?)	No for well-known ports; yes otherwise
`http`	Web response time	No
`udp-echo`	UDP round trip	Usually

Why does `udp-jitter` need a responder? Because measuring **one-way** delay and jitter requires timestamps from both ends. A plain ping only shows the round trip. The **IP SLA responder** timestamps each probe on arrival and departure, and subtracts its own processing time, so the measurements reflect the network rather than the far router's CPU.

Jitter is the variation in delay between packets. Voice copes with steady delay but sounds broken when delay jumps around. Example: four packets take 20, 22, 40 and 21 ms one way. The average delay (about 26 ms) is fine, but the jumps (2, 18 and 19 ms) are what the far phone's jitter buffer must absorb. A one-way jitter of 30 ms or more is a common warning level for voice.

How a udp-jitter operation runs: R1 first sends a small **control message** to R2's responder on **UDP 1967**, asking it to listen on UDP 16384 for the next few seconds → the responder opens that port → R1 sends a train of timestamped probes (with `codec g711ulaw`, 1000 packets of 172 bytes, 20 ms apart, by default) → the responder stamps each one with its arrival and departure times and returns it → R1 computes one-way delay, jitter and loss in each direction. One-way delay figures are only accurate if both routers' clocks are synchronized with NTP (Chapter 8).

Try It — udp-jitter for Voice Quality

```
R2(config)# ip sla responder                          ! far end: answer and timestamp probes
!
R1(config)# ip sla 20
R1(config-ip-sla)#  udp-jitter 10.2.2.2 16384 codec g711ulaw source-ip 10.1.1.1
R1(config-ip-sla-jitter)#  frequency 30
R1(config)# ip sla schedule 20 life forever start-time now
!
R1# show ip sla statistics 20
```

The `codec g711ulaw` keyword makes the probe mimic a G.711 call (packet size and interval), so the statistics include a **MOS** (Mean Opinion Score) and **ICPIF** voice-quality estimate alongside one-way latency, jitter and loss in each direction. If R2's responder is missing, every operation fails with a **timeout** return code, even though R2 answers pings.

From Measurement to Action

On its own, IP SLA only measures. Linked to a **track object**, it can change routing:

```
track 1 ip sla 10 reachability
ip route 0.0.0.0 0.0.0.0 198.51.100.254 track 1   ! primary default - removed if probe fails
ip route 0.0.0.0 0.0.0.0 203.0.113.254 10         ! floating backup (CCNA guide 15.5)
```

`state` or `reachability`? Two ways to turn a probe into up/down, and the difference is the **threshold**:

Track keyword	Up when the return code is	Down when
`track 1 ip sla 10 reachability`	OK **or Over threshold**	Timeout or other failure
`track 1 ip sla 10 state`	**OK only**	Over threshold, timeout, failure

So with `threshold 300`, a path answering in 450 ms keeps a `reachability` track **up** (it's slow but reachable) but takes a `state` track **down** (it's failing its SLA). Pick `state` when "too slow" should trigger failover, and `reachability` when only "dead" should.

```
R1# show track 1
Track 1
  IP SLA 10 reachability
  Reachability is Up
    3 changes, last change 00:41:12
  Latest operation return code: OK
  Latest RTT (millisecs) 14
  Tracked by:
    Static IP Routing 0
```

Read it as: what it tracks, its current state, **how often it has flapped** (the change counter), the latest probe result, and **what depends on it** (here, a static route).

Worked example — the failover, second by second. Using operation 10 (`frequency 10`), track 1 (`reachability`) and the two default routes above:

1. **Normal:** probes return OK. Track 1 is Up, so the primary default (AD 1) is in the routing table and the backup (AD 10) waits.
2. **The ISP's core fails at 10:00:03.** Gi0/0 stays up/up. The 10:00:10 probe gets no reply within 1000 ms, so its return code is **Timeout**.
3. Track 1 sees the failure and goes **Down**. IOS logs `%TRACK-6-STATE: 1 ip sla 10 reachability Up -> Down`.
4. The static route tied to `track 1` is withdrawn, so the floating route is installed.
5. **The ISP recovers.** The next OK probe brings track 1 Up, and the primary route (AD 1 beats AD 10) returns.

```
R1# show ip route static | include 0.0.0.0        ! step 1: primary in use
S*    0.0.0.0/0 [1/0] via 198.51.100.254
R1# show ip route static | include 0.0.0.0        ! step 4: backup in use
S*    0.0.0.0/0 [10/0] via 203.0.113.254
```

Users lost about 10 seconds of connectivity instead of hours. To stop a flapping path bouncing the routes back and forth, add `delay down 5 up 30` under the track: it must stay down 5 s before acting, and stay up 30 s before failing back.

This is how a router fails over when an ISP's link stays up but the ISP's network is broken, which an interface-down check would never notice. Chapter 9 uses the same tracking to lower HSRP priority, and Chapter 32 covers troubleshooting tracking and jitter operations at ENARSI depth.

15.8 Choosing the Right Tool

Why this matters: ENCOR 4.1 says diagnose, so exam questions give you a symptom and ask which tool finds the cause. Each tool has a cost (CPU, setup time, disk space) and answers one kind of question. Pick by the question, not by habit.

Symptom	First tool	Then
"Site B is unreachable"	Extended ping **sourced from the LAN**, traceroute	Routing table on each hop; ACLs at the hop where the trace stops
"OSPF neighbor keeps flapping"	`show logging` (look for `%OSPF-5-ADJCHG`)	Conditional `debug ip ospf adj` on that interface
"The WAN link is full — by what?"	**Flexible NetFlow** cache / collector	QoS (Chapter 11) or a conversation with the top talker
"VoIP calls are choppy"	**IP SLA udp-jitter**	QoS marking/queuing on the path
"The app vendor wants a packet capture"	**SPAN / ERSPAN** to Wireshark	Analyze the handshake and retransmissions
"CPU and interface trends over a month"	**SNMP** polling to the NMS	Thresholds and alerts
"Failover didn't happen when the ISP broke"	`show track`, `show ip sla statistics`	Check the schedule, life and the target used

Worked example — one complaint, four tools. "The ERP app at Site B is slow every morning around 09:00."

1. **Syslog first** (cheap, already collected): `show logging` on Site B's router shows no interface flaps or neighbor changes at 09:00. So it isn't an outage.
2. **SNMP graphs** (already polled): the WAN interface's inbound traffic sits at 98% of the 100 Mbps link from 08:55 to 09:40 every weekday. So it's congestion.
3. **Flexible NetFlow** (who is using it?): the cache during that window shows 10.20.5.14 → 10.1.9.30 on TCP 445 carrying 85% of the bytes. That's a file server copying to a PC, probably a scheduled sync.
4. **IP SLA udp-jitter** (prove the impact and the fix): jitter and loss during the window confirm the damage. After the sync is moved to 22:00, or put in a low-priority QoS class (Chapter 11), the next morning's statistics are clean.

No debug and no packet capture were needed. Broad tools found the right place; narrow tools would only have been needed if the broad ones had pointed at a single misbehaving device.

The habit to build: go from broad and cheap tools (logs, counters, flows) to narrow and expensive ones (debugs, captures). Start the expensive ones only once you know where to point them.

Chapter 16: Cisco Catalyst Center: Traditional & AI-Powered Workflows

> **Blueprint:** ENCOR 4.5 — Describe how Cisco Catalyst Center (formerly Cisco DNA Center) is used to apply network configuration, monitoring, and management using traditional and AI-powered workflows

Cisco Catalyst Center (called **DNA Center** until 2023) is Cisco's controller for the enterprise campus, branch and wireless network. It's the management plane of SD-Access (Chapter 14), but it manages traditional networks too. You met it in the CCNA guide as "the controller" (22.2). ENCOR wants you to describe how it is used: how intent becomes configuration, how monitoring becomes **assurance**, and what its AI features actually do.

> **A note on product claims.** Catalyst Center's features, especially the AI ones, change with every release and depend on licensing (most AI features require the **Advantage** tier, and some need cloud connectivity). This chapter describes the capabilities and the ideas behind them. Check Cisco's current release notes before you rely on a specific feature in production.

16.1 From Box-by-Box to Intent

The traditional way: an engineer works out what is needed ("a new floor needs a voice VLAN with QoS"), translates it into CLI, then logs in to each switch and types it, hoping every box ends up consistent. Monitoring is a separate set of tools (SNMP graphs, syslog server, NetFlow collector) that each show one slice, and correlating them is a manual job.

Intent-based networking (IBN) turns that around:

1. **Express the intent** (what you want: "voice gets priority on every access port at this site").
2. **Translate and activate.** The controller works out the per-device configuration and pushes it to every device consistently.
3. **Assure.** Continuously collect telemetry and verify the network is behaving as intended. When it isn't, explain why and suggest a fix.

Story Time : traditional management is giving a driver turn-by-turn directions from memory and hoping they arrive. Intent-based networking is a satnav. You state the destination (the intent), it works out every turn (the configuration), and then it keeps watching your position. If you drift off route or hit traffic, it notices and suggests a new route (assurance). Where it breaks: a satnav re-routes on its own, while Catalyst Center mostly suggests fixes and leaves the decision to change the network with you.

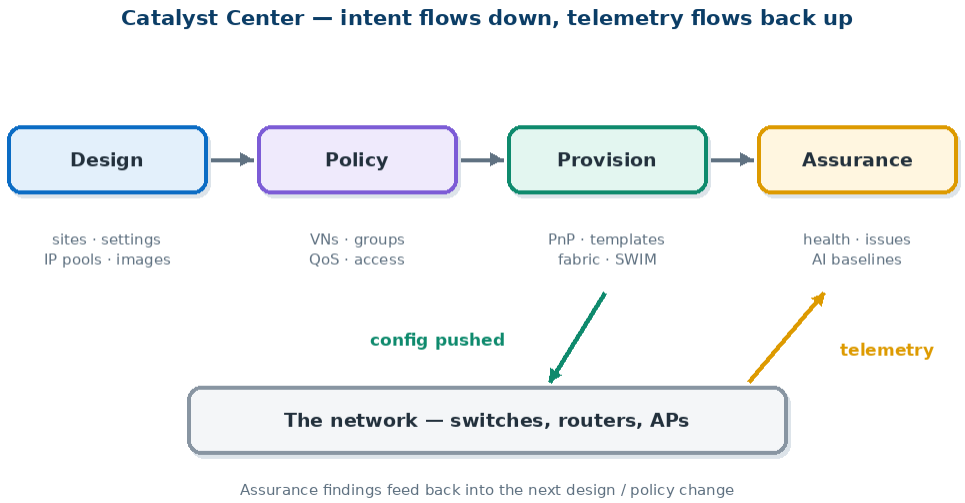
Why does the "assure" step matter so much? Because pushing configuration was never the hard part; scripts could already do that. The hard part is knowing whether the network is actually delivering what you asked for, right now, for every user. A config that was pushed successfully

can still produce terrible voice quality. IBN closes the loop: intent → config → verification → back to intent.

Aspect	Traditional CLI management	Catalyst Center (controller-based)
Unit of work	One device	A site, a group, a policy
Consistency	Depends on the engineer's discipline	Templates and intent enforced everywhere; drift is flagged
New device	Console cable, manual config	**Plug and Play** — zero-touch
Software upgrades	Device by device, manual checks	**SWIM** — golden images, automated pre/post checks
Monitoring	Separate SNMP, syslog, NetFlow tools	**Assurance** — correlated health scores per device, client, application
Troubleshooting	Engineer's experience + `show` commands	Guided issues with suggested actions, path trace, AI baselines
Integration	Screen-scraping CLI	**Intent API** (REST), webhooks, ITSM integration (Chapter 20)

16.2 The Four Workflows

Why workflows: every network change has the same life: decide what you want, decide who may do what, roll it out, then check it works. Catalyst Center gives each stage its own area, so the menus follow the job: **Design → Policy → Provision → Assurance**, plus a platform layer for integration.



Discovery and Inventory — Getting Existing Devices In

PnP (below) handles new devices. An existing network gets into Catalyst Center through **Discovery**:

1. **Choose how to find devices:** a **CDP** or **LLDP** crawl outward from a seed device (with a hop limit), or an **IP address range/subnet** scan.
2. **Supply credentials**, because Catalyst Center must log in to and poll what it finds: **CLI** (SSH username, password, enable secret), **SNMP** (v2c read/write communities or v3 users), and optionally **NETCONF** (port 830) and HTTP(S) credentials.
3. **Discovered devices land in the Inventory** with their reachability status ("Reachable", "Unreachable", "Authentication failed"), software version, platform and role. From there you **assign them to a site** in the hierarchy, which applies that site's settings.

Why do so many discoveries end in "Authentication failed" or "Unreachable"? Because Discovery is just SSH and SNMP under the hood. Wrong CLI credentials, an SNMP community that doesn't match (or an SNMP ACL that doesn't permit the Catalyst Center address), or SSH not enabled on the device all stop it. Catalyst Center manages a device only as well as it can log in to it.

Design — Describe the Network You Want

- **Network hierarchy:** Areas → Buildings → Floors (with floor maps for wireless). Everything else (settings, policies, reports) attaches to a place in this hierarchy, and child sites inherit from parents.
- **Network settings:** AAA servers, DHCP, DNS, NTP, syslog, SNMP and NetFlow collectors, time zone, banners, all defined once per site.
- **IP address pools:** IP Address Management (IPAM) built in, or integrated with an external IPAM.
- **Image repository:** which software version is the **golden image** for each device family.
- **Wireless:** SSIDs, RF profiles.

Why define settings once at the site level? So a new switch dropped into "Building 3" automatically gets the right NTP server, syslog collector and AAA configuration. Settings become a property of the place, not something someone has to remember to type.

Policy — Describe Who Can Do What

- **Group-based access control** with **ISE**: define scalable groups (SGTs) and the policy matrix between them (14.7).
- **Virtual networks** for SD-Access macro-segmentation.
- **Application policy (QoS):** classify applications as business-relevant, default or business-irrelevant, and Catalyst Center generates the matching QoS configuration (classification, marking, queuing) per platform (Chapter 11).

Provision — Make It Real

- **Plug and Play (PnP):** a new device boots with no configuration and finds Catalyst Center through **DHCP option 43**, a DNS lookup of `pnpserver.<domain>`, or Cisco's cloud **PnP Connect** redirect. It then receives its image and Day-0 configuration automatically.
- **Templates:** Day-N configuration written in **Velocity** or **Jinja2**, with variables filled in per device or site, and pushed consistently.

- **SWIM (Software Image Management):** mark a golden image, and Catalyst Center flags non-compliant devices, distributes the image, runs **pre-checks** (flash space, config saved, startup config valid), activates it in a maintenance window, and runs **post-checks**.
- **LAN automation and fabric provisioning** for SD-Access: build the IS-IS underlay automatically and assign fabric roles (14.7).
- **Compliance:** compares running configurations, images and security advisories against the intended state, and flags drift.

How PnP finds Catalyst Center, step by step (option 43 method). A new switch is cabled to an access port in VLAN 99, whose DHCP server is a router:

```
ip dhcp pool PNP-VLAN99
 network 10.99.0.0 255.255.255.0
 default-router 10.99.0.1
 option 43 ascii "5A1N;B2;K4;I10.10.10.50;J80"
```

1. The switch boots with no startup configuration, so the PnP agent starts and sends DHCPDISCOVER on its uplink.
2. It gets `10.99.0.x` plus option 43. Reading the string: `5A1N` = PnP, active, version 1, no debug; `B2` = the server address is IPv4; `K4` = use HTTP; `I10.10.10.50` = Catalyst Center's address; `J80` = port 80.
3. The switch contacts `10.10.10.50` and appears in Catalyst Center as an Unclaimed device, identified by its serial number.
4. An engineer (or a pre-staged rule matching that serial) **claims** it to a site, for example Leeds / Building 1 / Floor 2, and picks a template and image.
5. Catalyst Center upgrades the image if needed, pushes the Day-0 configuration, and the switch moves into the Inventory as managed.

If option 43 is missing, the switch tries the DNS name `pnpserver.<domain>` (using the domain name from DHCP), and then the PnP Connect cloud service.

Worked example — one template, two switches. A Day-N template written in Velocity:

```
hostname $hostname
interface Vlan$mgmt_vlan
 ip address $mgmt_ip 255.255.255.0
#foreach ($port in [1..24])
interface GigabitEthernet1/0/$port
 switchport access vlan $data_vlan
 switchport voice vlan $voice_vlan
#end
```

Provisioned to two switches with different variable values:

Variable	LEEDS-AS-01	YORK-AS-01
`$mgmt_vlan` / `$mgmt_ip`	99 / 10.99.1.11	99 / 10.99.2.11
`$data_vlan` / `$voice_vlan`	10 / 110	20 / 120

Each switch gets 24 identically structured port configurations; only the numbers differ. Nobody types 48 interface blocks by hand, so nobody mistypes one.

Why do PnP and templates together matter so much? Because the costliest part of rolling out 300 branch switches isn't buying them. It's the engineer time spent consoling into each one, and the configuration errors that slip in. With PnP and templates, a non-technical person at the site plugs the switch in, and it gets exactly the same validated configuration as every other site.

Assurance — Prove It Works

Catalyst Center continuously collects telemetry — **streaming telemetry**, **SNMP**, **syslog**, **NetFlow**, wireless data from controllers and APs, and data from ISE and other integrations — and correlates it into **health scores** and **issues**.

Health score	Meaning
8–10	Good
4–7	Fair
1–3	Poor
—	No data (shown grey, not scored)

- **Device health** combines KPIs such as CPU, memory, interface errors, reachability and (for APs) radio utilization and interference.
- **Client health** combines onboarding (association, authentication, DHCP) and connectivity (RSSI, SNR, data rate) for each wired and wireless client.
- **Network health** is the **percentage of devices that are healthy** (score 8–10) — overall and per site and category. Example: the York site has 20 devices; 18 score 8–10, one scores 6 (high CPU) and one scores 2 (unreachable uplink). Network health for York is 18 ÷ 20 = **90%**, and the two unhealthy devices appear as issues you can click into.
- **Application health** uses NBAR/AVC data: throughput, latency, packet loss and jitter per application.

Issues are detected problems with context: what happened, where, when, the impact (how many clients), and **suggested actions** (often a guided set of checks that Catalyst Center can run for you).

Tools inside Assurance:

- **Path Trace** — pick a source and destination, and Catalyst Center computes the hop-by-hop path from its knowledge of the topology, routing and configuration, including ACLs that would block the flow.
- **Intelligent Capture** — packet-level and RF data for a specific wireless client, captured by the AP.
- **Sensor tests** — dedicated sensors (or APs in sensor mode) that act as synthetic clients, testing onboarding, DNS, DHCP, web access and more, on a schedule.
- **Time travel** — scroll health back in time to see the state when the user complained, not just now.

Why is "time travel" so valuable? Because most user complaints arrive after the fact: "the Wi-Fi was terrible during the 10 o'clock meeting". With traditional tools you'd see a healthy network at 2 p.m. and close the ticket. With stored, correlated history you can look at that client,

AP and moment directly. Assurance turns "can't reproduce" into evidence.

Platform — Talk to Everything Else

The **Intent API** (northbound REST) exposes most of Catalyst Center's functions, so scripts, ITSM tools (e.g. ServiceNow) and orchestrators can drive and query it. Event notifications can be pushed out by webhook, email or syslog. Chapter 20 covers the API and its response codes.

How Catalyst Center Talks to Devices (Southbound)

Intent is turned into device changes, and telemetry collected, over ordinary management protocols:

Protocol	Used for
SSH / CLI	Pushing configuration (templates, provisioning) and running show commands for inventory and compliance. The workhorse
NETCONF (SSH, port 830)	Model-driven configuration on IOS XE devices that support it (Chapter 20)
SNMP	Polling device status and counters for inventory and assurance; receiving traps
Syslog, NetFlow, streaming telemetry	Event and flow data feeding Assurance
HTTPS / RESTCONF and controller APIs	Talking to wireless controllers, ISE (pxGrid/ERS) and other integrated systems

Northbound, Catalyst Center offers the Intent API (REST) to your scripts (Chapter 20). Southbound, it is still mostly SSH and SNMP to the devices, which is why device credentials and reachability are the first things to check when provisioning fails.

16.3 AI-Powered Workflows

The v1.2 blueprint specifically asks about **AI-powered workflows**. The core idea: the network already produces vastly more telemetry than people can read, so machine learning and reasoning engines turn that data into **baselines, anomalies, root causes and natural-language answers**.

AI Network Analytics — Learned Baselines and Anomaly Detection

A traditional alert uses a **static threshold**: "alert when DHCP response time > 500 ms". That threshold is always wrong somewhere. Too low for a busy Monday morning at headquarters, too high for a quiet branch.

AI Network Analytics sends anonymized telemetry to Cisco's cloud ML service, which learns a **dynamic baseline** for your network: what is normal for this KPI, at this site, at this time of day and week. Then it:

- **Detects anomalies** — deviations from the learned normal (for example, onboarding failures at one site far above what's usual for a Tuesday at 9 a.m.), rather than crossings of a fixed line.
- **Peer comparison** — compares a site or AP with similar ones ("this building's roaming times are worse than your other buildings").
- **Trends and insights** — longer-term patterns, such as recurring interference or growing capacity pressure.

Story Time : a static threshold is a doctor who flags every patient with a resting heart rate above 90. A baseline is your own doctor, who knows your normal is 55, so a reading of 80 is worth a look for you, while 85 is normal for someone else. Same number, different meaning, because "normal" depends on who and when.

Why do learned baselines beat static thresholds? A static threshold produces two kinds of noise: false alarms where "high" is normal, and missed problems where a large deviation still sits below the line. A baseline asks "is this unusual **here and now**?", which is the question an experienced engineer actually asks. It cuts alert fatigue while catching subtle problems earlier.

Machine Reasoning Engine (MRE) — Automated Root-Cause Analysis

The **Machine Reasoning Engine** encodes troubleshooting knowledge from Cisco experts (TAC, engineering) as **reasoning workflows**. Given an issue, it gathers the relevant data from devices (configs, `show` output, state) and works through the logical steps an expert would, to reach a **root cause and a recommended fix**.

Typical uses include:

- **Network Reasoner** tools — guided checks such as diagnosing high CPU, an interface that is down, or a failed ping between devices.
- **Security advisories** — checking whether a published Cisco PSIRT vulnerability actually applies to each device's configuration, not just its software version.
- **Compliance and best-practice checks** across the network.

Why "reasoning" rather than ML? Because many problems have a known logical diagnosis ("BGP is down → is the neighbor reachable? → are the AS numbers right? → is there an ACL on TCP 179?"). Learning that from data would be pointless when experts can write it down. MRE automates **known expertise**; ML finds **unknown patterns**. Catalyst Center uses both.

AI Endpoint Analytics

Identifies and classifies endpoints (IP phones, cameras, medical devices, printers, IoT) using deep packet inspection, network probes and ML clustering. It suggests labels for unknown devices so they can be placed in the correct **group** for policy (14.7, with ISE).

Natural-Language Operations — the AI Assistant

Cisco has added a generative-AI **AI Assistant** to Catalyst Center and other management platforms. Operators can ask questions in plain language ("which sites had the most client onboarding failures this week?", "why is this AP unhealthy?") and receive summarized answers, explanations of issues and suggested next steps. Cisco has also introduced **AI Canvas**, a collaborative workspace in which AI agents help with troubleshooting across Cisco's platforms ("AgenticOps"). Availability, scope and licensing vary by release.

What's different from traditional workflows? The engineer no longer needs to know which page or command holds the answer. They state the question, and the system finds and correlates the data. The engineer's judgement still matters: AI output is a starting point that must be verified, especially before any change is made.

16.4 Traditional vs AI-Powered — Same Problem, Two Workflows

Scenario: users at the Leeds office report that Wi-Fi "keeps dropping" every morning.

Step	Traditional workflow	AI-powered Catalyst Center workflow
Detect	Wait for complaints	**AI baseline** flags abnormal disconnects at that site at 08:30–09:30, before most tickets arrive
Scope	Ask users which floor; check WLC logs	**Issue** lists the affected clients, APs and floor; client health over time
Correlate	Compare syslog times with RF graphs by hand	Peer comparison shows only this building; insights show co-channel interference rising at the same times
Root cause	Engineer's experience, maybe a site visit	Guided checks / **MRE** reasoning plus RF data point to channel planning on two APs
Fix	Change channels on the WLC by hand	Apply an RF profile change in **Design**, provision it to the site
Verify	Wait for complaints to stop	Assurance shows client health back to 8–10 the next morning (**time travel** to compare)

The point: the network and the engineer haven't changed. What's changed is how quickly the evidence is assembled, and that the system can see a problem before users report it.

16.5 Exam-Ready Summary

Term	One-line meaning
Catalyst Center	Cisco's enterprise controller (formerly DNA Center); management plane for SD-Access and traditional networks
Design / Policy / Provision / Assurance	The four lifecycle workflows: describe the network, define access/QoS policy, push it, prove it works
PnP	Zero-touch onboarding via DHCP option 43, DNS `pnpserver`, or PnP Connect
Templates	Velocity/Jinja2 configuration with variables, applied consistently
SWIM	Golden images, automated distribution and activation with pre/post checks
Health score	1–10 per device/client/application; 8–10 good, 4–7 fair, 1–3 poor
Path Trace	Computed hop-by-hop path between two endpoints, including ACL checks
AI Network Analytics	Cloud ML — learned baselines, anomaly detection, peer comparison, trends
Machine Reasoning Engine	Expert knowledge encoded as automated root-cause workflows
AI Endpoint Analytics	ML-assisted endpoint classification for group policy
AI Assistant	Natural-language questions and guided answers about the network
Intent API	Northbound REST API (Chapter 20)

Chapter 17: Device Access Control, AAA, ACLs & CoPP

> **Blueprint:** ENCOR 5.1.a — Lines and local user authentication; ENCOR 5.1.b — Authentication and authorization using AAA; ENCOR 5.2.a — ACLs; ENCOR 5.2.b — CoPP

At CCNA you learned that a router should have passwords, that AAA has three A's, and that ACLs filter traffic (CCNA guide 18.4, Chapter 19). At CCNP the question changes from "what is it?" to "how do I lock down a device so that the right people get exactly the right access, it keeps working when the AAA server dies, and nobody can knock it over by flooding its CPU?" This chapter builds that answer in four layers: **the lines** you log in through, **who you are** (local users

and AAA), **what traffic is allowed through** the box (ACLs), and **what traffic is allowed to the box's own brain** (CoPP).

Chapter 31 takes the same features into troubleshooting depth for ENARSI. Here we build them correctly.

17.1 The Doors Into a Device — Console, AUX and VTY Lines

The problem: a router's CLI can change everything the network does. So the first security question is how can someone even reach a command prompt? If any of those paths is left open, every other control in this chapter can be walked around.

Every Cisco device has **lines**: the doors an administrator can walk through to get a CLI (Command-Line Interface).

Line	What it is	Typical numbering	Reach
Console (`line con 0`)	Physical serial/USB port on the device	Exactly one	You must be standing there (or on a terminal server)
AUX (`line aux 0`)	Auxiliary serial port, historically for a modem	One (many platforms no longer have it)	Out-of-band dial-in
VTY (`line vty 0 4`, `0 15`, ...)	Virtual terminals for Telnet/SSH sessions	5 or 16+ — one per simultaneous remote session	Anywhere with IP reachability

Why are VTYs numbered as a range? Because each one is a slot for one remote session. `line vty 0 4` is five slots, so five admins can be connected at once; a sixth is refused. Newer IOS XE images have `vty 0 15` or more. **The trap:** if you configure `line vty 0 4` but the device also has `line vty 5 15`, those extra eleven lines keep their old settings — possibly Telnet with a weak password. Always check `show running-config | section line vty` to see every VTY block, not just the first one.

Hardening the lines

```
R1(config)# line con 0
R1(config-line)# exec-timeout 10 0          ! log out an idle session after 10 min 0 s
R1(config-line)# logging synchronous        ! stop log messages chopping your typing
R1(config-line)# login local                ! authenticate against the local user database
!
R1(config)# line aux 0
R1(config-line)# transport input none       ! nothing can open a session through AUX
R1(config-line)# no exec                    ! and no EXEC shell even if something connects
!
R1(config)# ip access-list standard MGMT-HOSTS
R1(config-std-nacl)# permit 10.99.0.0 0.0.0.255   ! the management subnet only
!
R1(config)# line vty 0 15
R1(config-line)# transport input ssh        ! SSH only — Telnet sends passwords in clear text
R1(config-line)# exec-timeout 10 0
R1(config-line)# login local
R1(config-line)# access-class MGMT-HOSTS in ! only these sources may even attempt to log in
```

Why `access-class` rather than an ACL on an interface? Because the VTY is the thing you want to protect, no matter which interface the session arrives on. An interface ACL protects one door in the building; `access-class` protects the room itself. A router with ten interfaces and a loopback would need the same entry on all eleven; `access-class` does it once. Note that `access-class ... in` checks the source of incoming sessions, and `access-class ... out` restricts where a user may connect to from the router (via `telnet`/`ssh` out of it). If management sessions arrive in a VRF, add `vrf-also` (`access-class MGMT-HOSTS in vrf-also`), or the ACL silently doesn't apply to them.

How an SSH login meets this config, step by step:

1. A TCP connection arrives for port 22 on any of R1's addresses.
2. R1 picks a **free VTY** line (0–15). If all 16 are busy, the connection is refused.
3. `transport input ssh` is checked: SSH is allowed on this line (a Telnet attempt to port 23 would be refused here).
4. `access-class MGMT-HOSTS in` is checked against the **source** IP. Not permitted → the connection is closed before any login prompt appears.
5. `login local` asks for a username and password and checks them against the `username` lines.
6. The session starts. If it sits idle for 10 minutes (`exec-timeout 10 0`), R1 logs it out and frees the line.

Worked example. Two people try to SSH to R1:

Source	Step 4 ( )	Step 5 ( )	Result
10.99.0.15 (management PC)	permitted by `10.99.0.0 0.0.0.255`	`admin` + correct password	Logged in
198.51.100.7 (internet)	no match → implicit deny	never reached	Connection closed; the attacker never even sees a `Username:` prompt to guess against

Why `exec-timeout` matters for security, not just tidiness. An abandoned privileged session on a console is a logged-in root shell for whoever walks past next. `exec-timeout 0 0` (never time out) is a lab convenience you should never see in production.

Three ways a line can authenticate (before AAA)

Without AAA, each line decides for itself how to check who you are. There are exactly three options, and the exam expects you to read them:

```
R1(config)# line con 0
R1(config-line)# password C0ns0le-Pw     ! the line has its own password...
R1(config-line)# login                   ! ...and "login" says "ask for it"
!
R1(config)# line vty 0 15
R1(config-line)# login local             ! ask for a username + password from "username" lines
!
R1(config)# line aux 0
R1(config-line)# no login                ! no prompt at all — whoever connects gets a shell
```

Line config	Prompt you see	Checked against
`password X` + `login`	`Password:` only	The line's own password
`login local`	`Username:` then `Password:`	The local `username ... secret` database
`no login`	Nothing	Nobody — open door

The two traps. First, `login` **without** a `password` on a VTY line refuses every session with "Password required, but none set". VTYs (unlike the console) won't let you in with no password configured. Second, `login local` with **no** `username` lines locks everyone out, because there is nothing to match. `password` is a shared secret per door; `login local` is per-person identity, which is what accounting and audit need.

Landing straight in privileged mode. By default a line session starts at level 1 (`R1>`). `privilege level 15` under a line drops everyone who logs in through that line straight into `R1#`:

```
R1(config)# line con 0
R1(config-line)# privilege level 15      ! console users start at level 15
```

It's convenient on a console in a locked room, and dangerous on VTYs with a shared line password, because then knowing one password means full control. With `login local`, the **user's** privilege (`username X privilege N`) decides the level instead.

SSH prerequisites (recap from CCNA guide 6.6, with the CCNP extras)

```
R1(config)# hostname R1
R1(config)# ip domain name corp.example
R1(config)# crypto key generate rsa modulus 3072   ! RSA keys need hostname + domain first
R1(config)# ip ssh version 2
R1(config)# ip ssh time-out 60                     ! seconds to finish logging in
R1(config)# ip ssh authentication-retries 3
R1(config)# login block-for 120 attempts 5 within 60  ! brute-force brake (see below)
```

`login block-for 120 attempts 5 within 60` means: if 5 logins fail within 60 seconds, refuse all logins for 120 seconds ("quiet mode"). Pair it with `login quiet-mode access-class MGMT-HOSTS` so your management subnet can still get in while the device is shrugging off an attack.

Worked example. A password-guessing script hits R1. Its 1st to 5th wrong guesses arrive at 10:00:02, :09, :15, :21 and :30, all inside one 60-second window. On the 5th failure R1 logs `%SEC_LOGIN-1-QUIET_MODE_ON` and refuses every login for 120 seconds, until 10:02:30. The script now gets about 5 guesses every two minutes or so instead of hundreds per minute. Meanwhile an admin at 10.99.0.15 can still log in, because the quiet-mode ACL exempts the management subnet. `show login` displays the current state and the blocked time remaining.

17.2 Local Users, Privilege Levels and Password Hashes

The problem: not everyone who logs in should be able to do everything. A NOC (Network Operations Centre) operator who watches interfaces shouldn't be able to erase the config. And the passwords themselves sit in the config file, which gets backed up, emailed and pasted into tickets, so how they're stored matters as much as how strong they are.

Story Time : Privilege levels are the floors of an office building with a keycard lift. Level 1 is the lobby: you can look around (`show` a few things) but not touch anything. Level 15 is the top floor with every key. Levels 2–14 are empty floors you can fit out yourself, moving chosen commands onto them, then giving someone a card that stops at floor 5.

Privilege levels

IOS has **16 privilege levels, 0–15**:

Level	Meaning
0	Almost nothing — `logout`, `enable`, `disable`, `help`, `exit`
1	User EXEC (`R1>`) — the default after login
2–14	Unused by default — you can move commands into them
15	Privileged EXEC (`R1#`) — everything

```
R1(config)# username noc privilege 5 algorithm-type scrypt secret N0c-Viewer!
R1(config)# privilege exec level 5 show running-config   ! level 5 may RUN the command...
R1(config)# username admin privilege 15 algorithm-type scrypt secret Adm1n-Str0ng!
```

A user created at `privilege 15` lands straight in `R1#` on login. A level-5 user gets `R1#` too (any level above 1 shows `#`) but can only run commands assigned to levels 0–5. Check where you are with `show privilege`.

Worked example — the `noc` user's session (with the config above and `login local` on the VTYs):

```
R1# show privilege
Current privilege level is 5
R1# show running-config               ! allowed: moved to level 5 above
R1# configure terminal
               ^
% Invalid input detected at '^' marker.   ! level 15 command - not on this user's floor
R1# reload
% Unknown command or computer name, or unable to find computer address
```

IOS doesn't say "access denied". Commands above your level behave as if they **don't exist**, which is why the error is `Invalid input`, or, for a single unknown word like `reload`, `Unknown command or computer name` (IOS tries an unknown single word as a host name to Telnet to; with `ip domain-lookup` on, you first see a `Translating "reload"` delay). How it decides: every command has a level (most `show` commands are 1, `configure terminal` and `reload` are 15). You may run a command only if its level is **less than or equal to** your current level.

> **Exam trap — the near-empty running config.** Moving `show running-config` to level 5 lets the NOC user run it, but below level 15 the output only includes the commands that user is **allowed to configure**. A level-5 user with no configuration commands sees little more than the `version` line, `end`, and a few headers. It's a classic "why can't my NOC see the config?" question. The fixes: assign `show running-config view full` to the level (the `view full` option shows the whole config regardless of the user's level), or grant `file privilege 5` so a level-5 user can read the config as a file with `more system:running-config`. With AAA, command authorization (17.4) is the cleaner answer.

Separate enable passwords per level. `enable secret` can be set per level, and `enable N` asks for that level's password:

```
R1(config)# enable algorithm-type scrypt secret level 5 Lvl5-Pw!   ! the level-5 enable secret
R1> enable 5                  ! prompts for the level-5 secret; lands at level 5
R1# show privilege
Current privilege level is 5
```

Plain `enable` with no number means `enable 15`. This is how small shops gave a NOC "a bit more than user mode" before AAA, and it still shows up in exam exhibits.

Why do local custom privilege levels fall out of favour at scale? Because they're per-device. Maintaining `privilege exec level 5 ...` lines identically on 800 switches is exactly the problem AAA command authorization (17.4) solves centrally. Local levels are a fine fallback; they are a poor primary policy.

Password types — what the number in the config means

When you view the config, every password carries a **type** number that tells you how it's stored:

Type	How it's stored	Command that produces it	Verdict
0	Clear text	`password X` (no service password-encryption)	Never
7	Vigenère-style reversible encoding	`service password-encryption`	Hides from shoulder-surfers only — reversible in seconds
4	SHA-256, but **unsalted with a single iteration** (a flawed implementation)	Old `enable secret` on some 15.x releases	Deprecated and removed — weaker than type 5
5	MD5 hash (salted)	`secret X` on older IOS	Deprecated — too fast to brute-force
8	PBKDF2 with SHA-256	`algorithm-type sha256 secret X`	Good
9	scrypt	`algorithm-type scrypt secret X`	Best — memory-hard

```
R1(config)# enable algorithm-type scrypt secret En4ble-Str0ng!
R1# show running-config | include secret
enable secret 9 $9$nhEmQVczB7dqsO$X.HsgL6x1il0RxkOSSvyQYwucySCt7qFm4v7pqCxkKM
username admin privilege 15 secret 9 $9$7Mbz3XS/W/tKpe$ue8JC8/kk4lCSUbsZB3CKS5U7PmzKX...
```

Why is a hash (types 5/8/9) fundamentally different from type 7? Type 7 is encoding: the device has to turn it back into the original to use it, so anyone with the algorithm (it's public) can too. A hash is one-way: the device never needs the original. At login it hashes what you typed and compares hashes. To "crack" a hash an attacker has to guess passwords and hash every guess.
Why is scrypt (type 9) better than MD5 (type 5) when both are one-way? Because cracking is a speed race. MD5 was designed to be fast, so a GPU can try billions of guesses per second. scrypt is deliberately slow and memory-hungry, so every guess costs real RAM, which makes massively parallel GPU cracking expensive. The goal of a password hash is to be slow on purpose.

How a type 9 login check works, step by step: you type `Adm1n-Str0ng!` → the router takes the **salt** stored in the config (the random characters between `$9$` and the next `$`) → it runs scrypt over salt + what you typed → it compares the result with the stored hash → equal means correct. The original password is never stored and never recovered. The salt means two users with the same password get different hashes, so an attacker can't crack one and recognise the other.

Story Time : Type 7 is a note written in a simple code that comes with its own decoder ring, and everyone owns a copy of the ring. A hash is a fingerprint: you can check whether a finger matches it, but you can't grow a finger from the print. MD5 lets an attacker test billions of fingers a second; scrypt makes each test slow and expensive.

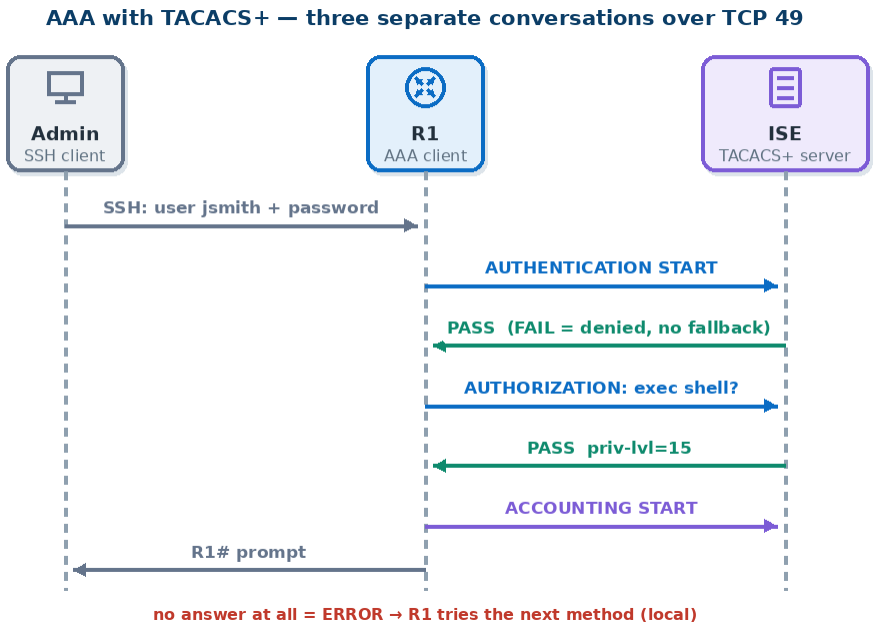
> **Exam trap:** `enable password` and `enable secret` can both exist; if they do, **`enable secret` wins** and `enable password` is ignored. And `service password-encryption` does **not** touch `secret` values. It only converts type 0 to type 7.

17.3 AAA — Turning It On and Building Method Lists

The problem AAA solves: local usernames live on each device. With 800 switches, adding a new engineer means 800 config changes, and removing one who leaves means 800 more, and the one you miss is a back door. **AAA** (Authentication, Authorization and Accounting) moves the decision to a central server, usually **Cisco ISE** (Identity Services Engine), so you change a person's access once and every device obeys.

- **Authentication** — who are you? (username and password, checked centrally)
- **Authorization** — what may you do? (privilege level, allowed commands)
- **Accounting** — what did you do? (a record of sessions and commands)

Story Time : The router is a nightclub bouncer, and the AAA server is head office. When you arrive, the bouncer phones head office: "Is Jane on the list?" (authentication). Head office says yes, and that she's VIP (authorization: level 15). The bouncer writes in a book "Jane in at 22:04, out at 01:30" (accounting). If the phone line is dead, the bouncer falls back to a short paper list kept at the door (the **local** users). But if head office answers "Jane is banned", the bouncer does not then check the paper list. That last rule is ERROR vs FAIL, below, and it's the most-tested idea in this section.



How a TACACS+ login works, step by step (admin `jsmith` SSHs to R1, which uses the config built below):

1. jsmith's SSH session reaches a VTY. The VTY uses the `default` login list: `group ADMIN-TACACS local`.
2. R1 opens a **TCP 49** connection from Loopback0 to the first server, ISE-1 (10.99.0.21). The whole TACACS+ payload is encrypted using the shared key.
3. R1 sends an authentication **START** with the username; ISE replies asking for the password (**GETPASS**); R1 relays what jsmith types in a **CONTINUE**.
4. ISE checks its identity store and replies **PASS** (or **FAIL**).
5. On PASS, if exec authorization is configured (17.4), R1 asks "may jsmith have a shell, at what level?" and ISE answers with `priv-lvl=15`.
6. If accounting is configured, R1 sends an accounting **START** record. When jsmith logs out, a **STOP** record follows with the session duration.

`aaa new-model` changes everything at once

```
R1(config)# aaa new-model
```

The moment you enter this, **line-level `login` and `password` commands stop being used**. Every line that doesn't name its own list (console and VTYs alike) now authenticates through the `default` method list, and if you haven't defined one, the implicit default is **local**. The VTYs are where this bites, because that's where a remote admin is when it happens. Why does that matter? Because if you type `aaa new-model` on a remote router that has no local usernames, your next SSH session has nothing to authenticate against. **Always create a local privilege-15 user before `aaa new-model`.** That's the exam-style "locked yourself out" scenario, and in real life it's the one where you drive to the data centre.

Defining servers and server groups (modern IOS XE syntax)

```
R1(config)# tacacs server ISE-1
R1(config-server-tacacs)# address ipv4 10.99.0.21
R1(config-server-tacacs)# key 0 T@cacsSharedKey
R1(config-server-tacacs)# single-connection          ! reuse one TCP session for many requests
!
R1(config)# tacacs server ISE-2
```

Why put the source interface on a loopback? The AAA server only answers devices it has been told about, identified by **source IP**. If the request leaves from whichever physical interface faces the server, a routing change can switch the source IP and the server will silently ignore the device. A loopback never goes down, so the device's identity never moves. This is the same reason OSPF prefers a loopback for its router ID (CCNA guide 16.7).

Method lists — the ordered "how do I check this?" list

A **method list** is a named, ordered list of places to check:

```
R1(config)# aaa authentication login default group ADMIN-TACACS local
R1(config)# aaa authentication login CONSOLE local
R1(config)# aaa authentication enable default group ADMIN-TACACS enable
!
R1(config)# line con 0
R1(config-line)# login authentication CONSOLE      ! console uses the named list
! (VTYs use "default" automatically — no command needed)
```

Part	Meaning
`login`	This list authenticates **logins** (as opposed to `enable`, `dot1x`, `ppp`)
`default`	Applied to **every line automatically** unless a line names another list
`CONSOLE`	A **named** list — only used where you apply it with `login authentication CONSOLE`
`group ADMIN-TACACS local`	Try the TACACS+ group first; then the local user database

The methods you can put in a list, and what each one checks:

Method keyword	Checks	Notes
`group tacacs+`	Every TACACS+ server defined globally	Generic — all servers, in config order
`group radius`	Every RADIUS server defined globally	Generic
`group NAME`	Only the servers in that named server group	Preferred: you control membership and order
`local`	Local `username` database, username matched **case-insensitively**	Password is always case-sensitive
`local-case`	Local database, username **case-sensitive**	`Admin` ≠ `admin`
`enable`	The enable secret	Common last resort in `aaa authentication enable` lists
`line`	The line's own `password`	Lets old per-line passwords act as a fallback

Method keyword	Checks	Notes
`none`	Nothing — always succeeds	Only as a deliberate last resort (e.g. console of a lab box)

Methods are tried left to right, up to four, and the ERROR-vs-FAIL rule below decides when the router moves on.

Why give the console its own `local`-only list? Because the console is the door you use when the network is broken, and a broken network often means the AAA server is unreachable. If the console tries TACACS+ first, every login waits for timeouts before falling back. Some designs deliberately keep the console independent of the network.

The rule everyone gets wrong — ERROR vs FAIL

The next method in the list is tried only when the current one returns an ERROR, never when it returns a FAIL.

What the TACACS+ server does	Result	Does the router try [redacted] next?
Doesn't answer (down, unreachable, wrong key → no valid reply)	**ERROR**	**Yes**
Answers "this user/password is wrong"	**FAIL** (reject)	**No** — login denied, list stops

Why is it designed this way? Think about what fallback is for: keeping you in when the central server is **unavailable**. If a rejection from the server also fell through to `local`, then any account the security team disabled centrally could still log in with a stale local password. That would turn the backup door into a bypass. The fallback is for an absent server, never for a server that said no. This is also why the local fallback account should have a long, vaulted password and be audited: it's the one account that works when ISE is down.

Worked example — three logins, one method list (`aaa authentication login default group ADMIN-TACACS local`; local user `admin` exists; ISE-1 and ISE-2 in the group):

Situation	What R1 does	Result
ISE-1 up; jsmith types the right password	ISE-1 replies PASS	Logged in via TACACS+
ISE-1 up; jsmith types the wrong password	ISE-1 replies FAIL	**Denied.** `local` is never tried
ISE-1 and ISE-2 both unreachable (WAN down); `admin` types the local password	ISE-1 times out (5 s default) → ERROR → tries ISE-2 → times out → ERROR → moves on to `local`	Logged in via the local account, after roughly a 10-second wait
Both unreachable; jsmith (a TACACS+-only account) tries	Both ERROR → `local` has no `jsmith`	Denied: only accounts that exist locally work during the outage

The 10-second pause in row 3 is normal and worth expecting: it's the timeout for each server in turn. That's also why the console in the example above uses a `local`-only list.

> **Exam trap:** "The admin's TACACS+ account was disabled but they can still log in using the local account" is **impossible** while the TACACS+ server is reachable. If they can, the server must be unreachable (ERROR), so check reachability, keys and source interface (Chapter 31).

17.4 Authorization and Accounting

The problem: after login, everyone authenticated by the server would otherwise get the same access. You want the help desk to run `show` commands, the network team to configure, and contractors to touch only their own switches, all decided centrally. And when something breaks at 03:12, you want to know who typed what.

Authentication proves who you are. **Authorization** decides what you may do. On a device you authorize two things:

```
R1(config)# aaa authorization exec default group ADMIN-TACACS local if-authenticated
R1(config)# aaa authorization commands 15 default group ADMIN-TACACS local if-authenticated
R1(config)# aaa authorization console        ! without this, the console skips authorization
R1(config)# aaa authorization config-commands
```

Command	What gets authorized
`aaa authorization exec`	Whether you get a shell at all, **and at which privilege level** (the server returns e.g. `priv-lvl=15`)
`aaa authorization commands 15`	**Every single level-15 command**, one by one, as you type it (TACACS+ only — see why below)
`if-authenticated`	Last-resort method: "if they authenticated, allow it" — keeps you working if the server dies

Why does per-command authorization need TACACS+ rather than RADIUS? Because RADIUS combines authentication and authorization into one exchange at login. It hands back a bundle of attributes and the conversation is over. TACACS+ keeps the three A's **separate**, so the device can ask a fresh question ("may `admin` run `configure terminal`?") for every command, long after login. That separation is the whole reason TACACS+ is the device-administration protocol.

How command authorization works, step by step. jsmith is logged in at level 15 and types `interface Gi0/1` then `shutdown`:

1. Before running `interface GigabitEthernet0/1`, R1 sends ISE an authorization **REQUEST**: user `jsmith`, `cmd=interface`, `cmd-arg=GigabitEthernet0/1`.
2. ISE checks jsmith's **command set** (for example, "help desk: permit `show *`, permit `interface *`, permit `shutdown`, permit `no shutdown`, deny everything else") and replies **PASS** or **FAIL**.
3. On PASS the command runs. On FAIL, R1 prints `Command authorization failed.` and nothing changes.

4. The same check repeats for `shutdown`. With command accounting on, R1 also sends a record of each command that ran.

Worked example — what the accounting server records:

Time	User	Device	Source	Command
03:11:48	jsmith	R1 (10.255.0.1)	10.99.0.15	`interface GigabitEthernet0/1`
03:12:02	jsmith	R1 (10.255.0.1)	10.99.0.15	`shutdown`

That's the 03:12 outage solved in one search. (`if-authenticated` as the last method means that if every earlier method errors out, for example ISE becomes unreachable, a user who has already authenticated may keep working rather than being frozen out mid-change.)

Accounting records what happened:

```
R1(config)# aaa accounting exec default start-stop group ADMIN-TACACS
R1(config)# aaa accounting commands 15 default start-stop group ADMIN-TACACS
```

`start-stop` sends a record when something begins and when it ends (session duration). `stop-only` sends just the end. With `commands 15` accounting, the server keeps a line for every config command and who typed it. That's the answer to "who shut that interface at 03:12?"

TACACS+ vs RADIUS — the CCNP version of the table

Property		
Transport	**TCP 49**	**UDP 1812** (auth) / **1813** (accounting); legacy 1645/1646
Encryption	**Entire payload**	**Only the password** attribute
AAA separation	Authentication, authorization, accounting **separate**	Authentication + authorization **combined**
Per-command authorization	**Yes**	No
Origin	Cisco (now RFC 8907, informational)	IETF standard (RFC 2865/2866)
Primary job	**Device administration**	**Network access** — 802.1X, VPN, Wi-Fi (18.3, Chapter 31)

Why TCP for TACACS+ and UDP for RADIUS? TACACS+ authorizes command by command, so a reliable, long-lived session (`single-connection`) suits it. RADIUS handles thousands of short, independent access requests (every laptop joining the network), and connectionless UDP with its own retries is lighter for that.

Verifying

```
R1# test aaa group ADMIN-TACACS jsmith Secr3t! legacy
Sending password
User successfully authenticated
R1# show tacacs                  ! per-server: connection state, packets sent/received, errors
R1# show aaa servers             ! UP/DEAD state, request counts, response times
R1# show aaa sessions
R1# debug aaa authentication     ! Chapter 31 walks through reading this output
```

17.5 ACLs at CCNP Depth

The problem ACLs solve: a router forwards anything it has a route for. That's its job, but it means that without a filter, every host that can reach your network can reach every service on it. An **ACL** (Access Control List) is the router's list of "who may send what to where".

Story Time : An ACL is a bouncer with a numbered checklist. For each packet he reads from the top: "Line 10, does this match? No. Line 20? No. Line 30? Yes: deny." He acts on the **first** line that matches and never reads further. If he reaches the bottom without a match, the unwritten last line says "turn them away" (the implicit deny).

CCNA taught standard vs extended, wildcards, placement and the implicit deny (CCNA guide Chapter 19). CCNP adds four practical skills: **editing in place**, **object-groups**, `established`, and **IPv6 ACLs**. It also adds the habit of reading hit counters as a troubleshooting tool.

Number ranges, for completeness

Type	Numbers
Standard IPv4	1–99, 1300–1999
Extended IPv4	100–199, 2000–2699

On IOS XE, a numbered ACL and a named ACL are the same thing internally. `ip access-list extended 110` opens numbered ACL 110 in named-ACL mode, so you get sequence numbers for it too.

Sequence numbers — editing without rewriting

```
R1# show access-lists EDGE-IN
Extended IP access list EDGE-IN
    10 permit tcp any host 203.0.113.10 eq 443 (18234 matches)
    20 permit tcp any host 203.0.113.10 eq 80 (512 matches)
    30 deny ip any any log (97 matches)
!
R1(config)# ip access-list extended EDGE-IN
R1(config-ext-nacl)# 25 permit udp any host 203.0.113.53 eq domain   ! insert between 20 and 30
R1(config-ext-nacl)# no 20                                          ! remove one line only
R1(config)# ip access-list resequence EDGE-IN 10 10                 ! renumber 10, 20, 30 ...
```

Why does this matter so much in production? Before sequence numbers, changing one line of a numbered ACL meant removing the whole ACL and re-entering it. For the few seconds in between, an interface ACL that's been deleted either permits everything or doesn't exist. Inserting by sequence number changes **one rule atomically**. The `(N matches)` counters are your first troubleshooting tool: a rule with 0 matches after a test either isn't being reached or doesn't match what you think it does. Clear them with `clear ip access-list counters EDGE-IN` before you test.

Worked example — walking packets through EDGE-IN (after adding 25 and removing 20, before the resequence: lines 10, 25, 30):

Packet arriving inbound	Line 10 (tcp → .10:443)	Line 25 (udp → .53:53)	Line 30 (deny ip any any log)	Result
TCP 198.51.100.7 → 203.0.113.10:443	**match**	—	—	permitted, line 10 counter +1
UDP 198.51.100.7 → 203.0.113.53:53	no	**match**	—	permitted, line 25 +1
TCP 198.51.100.7 → 203.0.113.10:**80**	no	no	**match**	**denied** (line 20 was removed), logged
TCP 198.51.100.7 → 203.0.113.10:22	no	no	**match**	denied, logged

The third row is how you'd prove a change worked: clear the counters, test from outside, and see line 30's counter (and a `%SEC-6-IPACCESSLOGP` log message) increment for port 80.

Object-groups — naming sets of things

When the same servers and ports appear in many rules, object-groups turn an ACL into readable policy:

```
R1(config)# object-group network WEB-SERVERS
R1(config-network-group)# host 10.20.0.10
R1(config-network-group)# host 10.20.0.11
R1(config-network-group)# 10.20.1.0 255.255.255.0          ! note: subnet MASK, not wildcard
!
R1(config)# object-group service WEB-PORTS
R1(config-service-group)# tcp eq 80
R1(config-service-group)# tcp eq 443
!
R1(config)# ip access-list extended TO-WEB
R1(config-ext-nacl)# 10 permit object-group WEB-PORTS any object-group WEB-SERVERS
R1(config-ext-nacl)# 20 deny ip any object-group WEB-SERVERS log
R1(config-ext-nacl)# 30 permit ip any any
```

Why use object-groups if the device expands them into the same rules anyway?
Because humans maintain ACLs. Adding a third web server is now one line in `WEB-SERVERS`, and every rule that references the group picks it up. Without groups you'd hunt through the ACL for every rule mentioning the old servers and inevitably miss one. Object-groups move the "what" (which hosts) away from the "policy" (what they may do).

Worked example — what line 10 really means. `WEB-PORTS` holds 2 services and `WEB-SERVERS` holds 3 destinations, so line 10 stands for 2 × 3 = **6** ordinary rules: TCP to port 80 or 443 on 10.20.0.10, 10.20.0.11, or anything in 10.20.1.0/24. Add `host 10.20.0.12` to `WEB-SERVERS` and line 10 now covers 8 combinations, while line 20 also starts protecting the new server, without either ACL line being touched.

`established` — the cheap return-traffic trick

```
R1(config)# ip access-list extended FROM-INTERNET
R1(config-ext-nacl)# permit tcp any 10.0.0.0 0.255.255.255 established
R1(config-ext-nacl)# deny ip any any log
```

`established` matches TCP segments with the **ACK or RST** bit set. Every TCP segment except the very first SYN of a connection has ACK set. So this lets replies to inside-initiated sessions back in, while blocking new connections initiated from outside.

Worked example. Inside PC 10.1.1.25 browses to 52.96.10.4:

Packet arriving from the internet	Flags	[redacted] matches?	Result
52.96.10.4:443 → 10.1.1.25:51544 (the server's reply)	SYN+ACK	yes (ACK set)	permitted
52.96.10.4:443 → 10.1.1.25:51544 (data)	ACK	yes	permitted
198.51.100.7:40000 → 10.1.1.25:3389 (someone trying RDP)	SYN only	no	denied by the next line
198.51.100.7:40000 → 10.1.1.25:3389 (crafted probe)	ACK	**yes**	**permitted** — the weakness below

Why isn't this a firewall? Because it's **stateless**. The router doesn't remember which connections actually exist; it just looks at flag bits, and an attacker can set ACK on a crafted packet to probe through. It also does nothing for UDP or ICMP replies. It's a filter that trusts what the packet says about itself. A stateful firewall (18.4) remembers what really happened.

Placement at CCNP level

The CCNA placement rules still hold (extended near the source, standard near the destination). CCNP adds three refinements:

- **Infrastructure ACLs (iACLs)** at the network edge deny traffic to your own device addresses (loopbacks, link IPs) from the outside, except for what must reach them (BGP from your peer, for example). They protect the control plane before CoPP ever sees the packet.
- **Inbound beats outbound** where you have the choice: a packet denied inbound never consumes a route lookup.
- `ip access-group` **applies to transit traffic.** Outbound interface ACLs do **not** filter traffic the router itself generates. Protecting the router's own management uses `access-class` (17.1) and CoPP (17.6).

IPv6 ACLs — the same idea with three differences

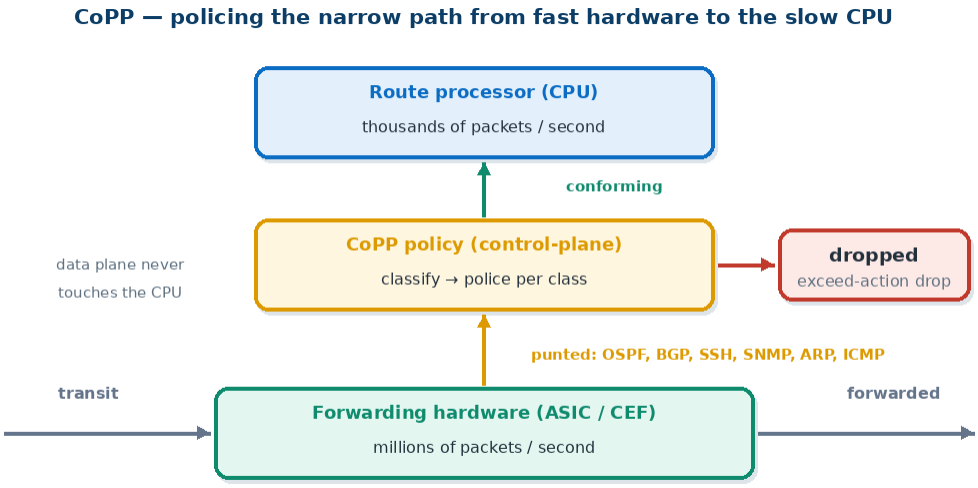
```
R1(config)# ipv6 access-list V6-EDGE-IN
R1(config-ipv6-acl)# permit tcp any host 2001:db8:20::10 eq 443
R1(config-ipv6-acl)# permit icmp any any nd-na              ! keep neighbor discovery working
R1(config-ipv6-acl)# permit icmp any any nd-ns
R1(config-ipv6-acl)# deny ipv6 any any log
R1(config)# interface GigabitEthernet0/0/0
R1(config-if)# ipv6 traffic-filter V6-EDGE-IN in           ! not "access-group"
```

Difference	IPv4 ACL	IPv6 ACL
Types	Standard and extended	**Named extended only**
Address match	Wildcard masks	**Prefix lengths** (`2001:db8::/32`)
Applied with	`ip access-group`	**`ipv6 traffic-filter`**
Hidden lines at the end	`deny ip any any`	`permit icmp any any nd-na`, `permit icmp any any nd-ns`, **then** `deny ipv6 any any`

Why does IPv6 hide two permits before the implicit deny? Because IPv6 has no ARP. Neighbor Discovery (NS/NA) is how neighbours find each other's MACs (CCNA guide 13.8), and it runs over ICMPv6. If the implicit deny blocked ND, any IPv6 ACL would break basic reachability on the link. **The trap:** those hidden permits sit after your last line. If you write your own explicit `deny ipv6 any any` at the end (to log it, say), you've placed it above the hidden ND permits, and you break neighbor discovery. That's why the example above permits `nd-na` and `nd-ns` explicitly before its own deny.

Worked example of the trap. An engineer writes `permit tcp any host 2001:db8:20::10 eq 443` then `deny ipv6 any any log`, and applies it inbound on the link to the upstream router. Web traffic works for a few minutes, then everything stops. Why: the upstream router's Neighbor Solicitations and Advertisements now hit the explicit deny. Once R1's neighbor cache entry for the upstream router expires and can't be refreshed, R1 can no longer resolve its MAC address, so nothing can be sent to it. `show ipv6 neighbors` shows the entry stuck at `INCMP` (incomplete), and the deny line's counter is climbing with ICMPv6 packets. Adding the two `nd-` permits above the deny fixes it. Chapter 31 covers IPv6 filtering troubleshooting.

17.6 Control Plane Policing (CoPP)



The problem CoPP solves: a router can forward a flood of traffic through itself without breaking a sweat, yet be knocked over by a much smaller flood sent to itself. CoPP (Control Plane Policing) protects the router's own CPU, so that its routing protocols keep running whatever is thrown at it.

Three planes, one weak spot

Plane	What it does	Example traffic	Handled by
Data plane	Forwards through the device	User packets	Forwarding hardware (ASIC/CEF) — fast
Control plane	Builds the forwarding tables	OSPF hellos, BGP updates, ARP, STP BPDUs	**The CPU**
Management plane	Lets people manage the device	SSH, SNMP, NTP, syslog	**The CPU**

Why does the CPU need its own protection? A switch ASIC can forward tens or hundreds of gigabits per second. The CPU behind it can process only a few thousand packets per second. Any traffic addressed to the device, or that needs CPU handling (TTL expiry, ARP, IP options), is **punted** from the fast hardware up to that slow CPU. If an attacker, or just a loop or a misbehaving host, floods the box with SSH SYNs, pings or ARP, the CPU drowns. **When the CPU drowns, OSPF hellos go unanswered, neighbours time out, and the router drops off the network even though its forwarding hardware is idle.**

Story Time : Picture a huge warehouse with conveyor belts (the forwarding hardware) moving thousands of parcels a second, and one manager in a small office upstairs (the CPU). Parcels addressed to the manager have to be carried up a narrow staircase. If someone posts ten thousand junk letters to the manager, the staircase jams, and the manager never receives the phone calls from the other warehouses (the routing protocol hellos) that keep the business running. CoPP is a bouncer at the foot of the staircase with a rule for each kind of visitor: "routing partners, come right up; staff from the management office, a steady stream; strangers with pings, one at a time; anyone else, a trickle."

Building a CoPP policy with MQC

CoPP uses the same **Modular QoS CLI** (MQC) you'll use for QoS in Chapter 11: **class-map** (what), **policy-map** (what to do), **service-policy** (where). The "where" is the special `control-plane` configuration mode.

Step 1 — ACLs that identify the traffic (a `permit` means "belongs in this class", not "allow"):

```
ip access-list extended COPP-ROUTING
 permit ospf any any
 permit tcp any any eq bgp             ! BGP sessions in either direction:
 permit tcp any eq bgp any             !   to port 179 and from port 179
ip access-list extended COPP-MGMT
 permit tcp 10.99.0.0 0.0.0.255 any eq 22
 permit udp 10.99.0.0 0.0.0.255 any eq snmp
 permit udp any eq ntp any eq ntp
ip access-list extended COPP-ICMP
 permit icmp any any echo
 permit icmp any any echo-reply
 permit icmp any any ttl-exceeded
 permit icmp any any unreachable
```

Step 2 — classes:

```
class-map match-any CM-COPP-ROUTING
 match access-group name COPP-ROUTING
class-map match-any CM-COPP-MGMT
 match access-group name COPP-MGMT
class-map match-any CM-COPP-ICMP
 match access-group name COPP-ICMP
```

Step 3 — policy: a rate per class:

```
policy-map PM-COPP
 class CM-COPP-ROUTING
  police cir 1000000 conform-action transmit exceed-action transmit  ! measure, never drop
 class CM-COPP-MGMT
  police cir 512000 conform-action transmit exceed-action drop
 class CM-COPP-ICMP
  police cir 64000 conform-action transmit exceed-action drop       ! pings get a trickle
 class class-default
  police cir 128000 conform-action transmit exceed-action drop      ! everything unexpected
```

The order of classes matters. A policy-map is evaluated **top-down, and the first class that matches wins**. A packet that would match two classes is policed only by the first. So put the most specific classes first (routing, then management from known sources, then ICMP) and broader ones after them. `class-default` is always evaluated **last**, whatever position you type it in, and catches everything nothing else matched. If a broad class like "all TCP" sat above the BGP class, BGP would be policed at the generic rate, and the routing class above would never see a packet.

Step 4 — attach it to the control plane:

```
R1(config)# control-plane
R1(config-cp)# service-policy input PM-COPP
```

The design reasoning behind each class

- **Routing gets generous treatment, often `exceed-action transmit`.** Dropping your own OSPF or BGP packets causes the very outage CoPP exists to prevent. Many designs police routing only to measure it at first, then tighten.
- **Management is restricted by source.** SSH from the management subnet is in `CM-COPP-MGMT`. SSH from anywhere else doesn't match that ACL, so it falls into `class-default` and gets the small rate.
- **ICMP gets a trickle.** You want ping and traceroute to work for troubleshooting, but a ping flood shouldn't eat the CPU.
- **`class-default` is the safety net**, and it must be policed, not left unlimited. Anything you didn't anticipate lands here. An unpoliced class-default means CoPP protects you from everything except what you didn't think of, which is exactly what attackers use.

How a policer decides, step by step. `police cir 64000` sets a **CIR** (Committed Information Rate) of 64,000 bits per second, and IOS picks a burst size (**Bc**, here 2,000 bytes) for it. Think of a bucket of tokens, each token worth one byte:

1. Tokens drip into the bucket at the CIR: 64,000 bps = 8,000 bytes' worth per second. The bucket holds at most Bc (2,000 bytes); extra tokens spill away.
2. A punted ICMP packet of 100 bytes arrives. If the bucket holds at least 100 tokens, the packet **conforms**: 100 tokens are removed and `conform-action transmit` sends it to the CPU.
3. If the bucket holds fewer than 100 tokens, the packet **exceeds**, and `exceed-action drop` throws it away. No tokens are taken.

So short bursts up to 2,000 bytes pass at once, and anything sustained above 64 kbps is trimmed down to 64 kbps.

Worked example — a ping flood meets CoPP. A compromised host sends R1 **500 pings per second** of 100 bytes each: 500 × 100 × 8 = **400,000 bps**. The ICMP class only allows 64,000 bps, so about 64,000 ÷ 400,000 = **16%** of the pings reach the CPU (about 80 per second) and the other ~84% are dropped before they ever reach the CPU. The CPU barely notices, OSPF hellos (in the routing class) are untouched, and `show policy-map control-plane` shows the `exceeded` counter on CM-COPP-ICMP climbing, which tells you exactly what's happening.

Worked example — the same SSH, two outcomes:

Packet	First class it matches	Policed at
TCP SYN to port 22 from 10.99.0.15	CM-COPP-MGMT (source in 10.99.0.0/24)	512 kbps
TCP SYN to port 22 from 198.51.100.7	none of the ACLs → class-default	128 kbps, shared with every other unexpected packet

The outsider isn't blocked by CoPP (that's the job of `access-class` and the iACL); they're just squeezed into the small shared pipe, so a flood from them can't starve the management class.

Why `permit` in a CoPP ACL never means "allow". The ACL is only a classifier. `permit` puts a packet into the class, and the policer decides its fate. A `deny` line just means "not this class, try the next one." Nothing is ever dropped by the ACL. Read CoPP ACLs as membership lists.

Verifying

```
R1# show policy-map control-plane
 Control Plane
  Service-policy input: PM-COPP
    Class-map: CM-COPP-ICMP (match-any)
      14520 packets, 1393920 bytes
      5 minute offered rate 61000 bps, drop rate 12000 bps
      Match: access-group name COPP-ICMP
      police:
          cir 64000 bps, bc 2000 bytes
        conformed 11201 packets, 1075296 bytes; actions:
          transmit
        exceeded 3319 packets, 318624 bytes; actions:
          drop
```

Read it as: ICMP is arriving at ~61 kbps against a 64 kbps contract, and 3,319 packets have been dropped. That's either an attack or a monitoring system pinging too aggressively, and now you know which class to investigate. A climbing `exceeded` count on the **routing** class is a design error: the rate is too low for your network.

> **Platform note.** On Catalyst 9000 switches, CoPP is **on by default** as a system-defined policy (`system-cpp-policy`). You tune its per-queue rates (`show policy-map system-cpp-policy`) rather than attaching your own policy to `control-plane`. Routers (ISR/ASR/Catalyst 8000) use the user-defined MQC policy shown above. Both follow the same idea: classify punted traffic and rate-limit it per class.

17.7 Putting It Together — A Hardening Checklist

Layer	Control	Section
Doors	SSH only, `access-class`, `exec-timeout`, AUX disabled, all VTY ranges covered	17.1
Identity	scrypt secrets, local fallback admin, `login block-for`	17.1, 17.2
Central policy	`aaa new-model`, TACACS+ group with loopback source, local fallback	17.3
Least privilege	exec + command authorization, command accounting	17.4
Transit filtering	iACLs at the edge, object-group based policy ACLs, IPv6 equivalents	17.5
CPU protection	CoPP with routing / management / ICMP / default classes	17.6

Each layer assumes the one before it might fail. That's the defense-in-depth idea Chapter 18 applies to the whole network.

Chapter 18: Security Design & REST API Security

> **Blueprint:** ENCOR 5.3 — Describe REST API security; ENCOR 5.4.a — Threat defense; ENCOR 5.4.b — Endpoint security; ENCOR 5.4.c — Next-generation firewall; ENCOR 5.4.d — TrustSec and MACsec

Chapter 17 hardened **one device**. This chapter zooms out to the **whole network**: which security components exist, where each one sits, and what each catches that the others miss. Then it zooms back in on something you'll use daily as automation grows (Chapters 19–21): the **APIs** that controllers and devices expose, and how they're secured.

The blueprint verb here is **describe**, so the exam wants you to know what each piece is for and how the pieces fit together. You won't be configuring a firewall policy. The exception is TrustSec and MACsec, where you should know the mechanics well enough to explain a packet's journey.

18.1 Defense in Depth — Why One Wall Is Never Enough

Story Time : A medieval castle didn't rely on its outer wall. It had a moat, the wall, a gatehouse, guards inside, a keep, and locked rooms in the keep. An attacker who climbed the wall still had five more problems. Each layer assumes the one before it has already failed.

Modern networks work the same way, for a reason the castle builders never faced: **the attacker is often already inside.** A phishing email lands on a laptop, a contractor plugs in an infected device, a cloud app is compromised. The old model of a hard perimeter firewall around a soft trusted inside fails the moment anything inside is compromised, because everything inside trusts everything else.

Stage of an attack	What defends against it	Component (this chapter)
Before — reaching you	Block known-bad destinations and senders	DNS-layer security, email security, threat intelligence
At the edge	Inspect and control traffic crossing boundaries	Next-generation firewall (18.4)
Getting onto the network	Prove identity and health before granting access	ISE, 802.1X, posture (18.3)
Moving inside	Limit who can talk to whom	TrustSec segmentation (18.5)
On the host	Detect and stop malicious behaviour on the device	Endpoint security (18.3)
On the wire	Stop eavesdropping and tampering on links	MACsec (18.6)
After — already in	Spot abnormal behaviour and contain it	Network analytics (NetFlow-based), retrospective file analysis

Why the "before, during, after" framing? No single control catches everything, so the design question isn't "is it blocked?" but "how quickly do we see it and how far can it spread?" Prevention fails sometimes; detection and containment are what limit the damage when it does.

This is the thinking behind **zero trust**: never trust because of location, always verify identity and posture, and grant the least access needed. TrustSec (18.5) and ISE are Cisco's main tools for building it into the campus.

18.2 Threat Defense — The Cisco Secure Portfolio

The problem: attacks arrive by email, web, DNS, USB stick and stolen password, and no single box sees all of those. **Threat defense** is the set of products that each watch one route in, all fed by the same threat intelligence, so that something spotted in one place gets blocked everywhere.

The exam expects you to recognise the main products and what each one contributes. Several were renamed, so you'll see both names in study material:

Component	Older name	What it does
Cisco Talos	—	Cisco's threat-intelligence organisation. It researches threats and feeds verdicts (bad IPs, domains, file hashes, IPS signatures) into **every** product below.
Cisco Secure Firewall	Firepower / FTD	Next-generation firewall and IPS (18.4). Managed by Firewall Management Center (FMC) or cloud-delivered management.
Cisco Umbrella / **Cisco Secure Access**	OpenDNS	**DNS-layer security**: blocks lookups to malicious domains before a connection exists. Secure Access is the broader cloud **SSE** (Security Service Edge) service: secure web gateway, cloud firewall, CASB and zero-trust access.
Cisco Secure Endpoint	AMP for Endpoints	Endpoint detection and response on laptops and servers (18.3)
Cisco Secure Malware Analytics	Threat Grid	**Sandbox**: detonates suspicious files in an isolated VM and reports what they do
Cisco Secure Network Analytics	Stealthwatch	Collects **NetFlow** (Chapter 15) from the network and uses behaviour modelling to spot anomalies: data exfiltration, scanning, unusual peers
Cisco Secure Email	ESA	Filters phishing, spam and malicious attachments
Cisco ISE	—	**Identity Services Engine**: the policy brain for network access. It handles 802.1X/MAB authentication, posture, guest access, profiling, and assigns TrustSec tags (18.5).
Cisco Secure Web Appliance	WSA (Web Security Appliance)	On-premises **web proxy**: URL filtering, malware scanning of downloads, application visibility for HTTP/HTTPS
Cisco Secure Client	AnyConnect	The endpoint **VPN client** and agent bundle: remote-access VPN, ISE posture module, Umbrella roaming module, Secure Endpoint connector

Component	Older name	What it does
Cisco Duo	—	**Multi-factor authentication** and device-trust checks for applications, VPN and admin logins — the "something you have" factor
Cisco XDR	SecureX	**Extended detection and response**: correlates telemetry from endpoint, network, firewall, email and cloud into one incident view, with automated response actions

Why is DNS-layer security such an effective early control? Almost every connection starts with a DNS lookup, including malware calling home to its command-and-control server. If the resolver refuses to answer for a known-bad domain, the connection never happens. This works for every port and protocol, and it covers devices that can't run an agent. Blocking at DNS stops the attack before any packet of it exists.

Why does Secure Network Analytics use NetFlow instead of inspecting packets? Because NetFlow records exist on every router and switch already, and they're cheap. Inspecting full packets everywhere is impossible at campus scale, and encryption hides payloads anyway. Flow records show who talked to whom, how much, and when. That's enough to spot an internal host suddenly sending 40 GB to an unknown country at 3 a.m. Metadata catches behaviour that payload inspection misses, including inside encrypted traffic.

Worked example — one phishing attack, every layer getting a chance. An attacker emails finance staff a "late invoice" link:

1. **Secure Email** scores the message. The sending domain was registered yesterday, so it's quarantined for most recipients. One copy gets through.
2. A user clicks. The laptop asks DNS for `invoice-portal-pay.example`. **Umbrella** knows from **Talos** that the domain hosts malware and answers with a block page instead of the real address. No connection is ever made.
3. Suppose the user is at home, outside both Umbrella and the office firewall, and the page loads. It offers `invoice.pdf.exe` for download. The **Secure Endpoint** agent on the laptop checks the file's hash against the cloud. The hash has never been seen, so it isn't known-bad yet; the file is sent to **Secure Malware Analytics** (the sandbox), which runs it, watches it start encrypting files, and convicts it. (In the office, the **Secure Firewall** would have done the same check on the download as it crossed the edge.)
4. Three hours later the verdict reaches every product. Secure Endpoint's **retrospective** feature finds the file on that one laptop and quarantines it.
5. If the malware had already started scanning the network, **Secure Network Analytics** would flag the laptop's sudden connections to hundreds of internal hosts on TCP 445, and **TrustSec** (18.5) would have limited which of them it could reach at all.
6. **XDR** shows all of this as one incident with one timeline, instead of five separate alerts in five consoles.

No single step had to be perfect. That's defense in depth working as designed.

18.3 Endpoint Security and Posture

The problem: the firewall sees traffic, but the attack actually runs on a laptop, phone or server: the **endpoint**. Laptops also leave the building, so they spend much of their life outside every network control you own. Endpoint security protects the device itself, and checks that it's healthy before the network lets it in.

The endpoint is where most attacks actually execute, so it needs its own defence:

- **Endpoint protection / EDR (Cisco Secure Endpoint):** a lightweight agent on the host checks files against cloud verdicts (from Talos) at execution, watches behaviour (a Word document spawning PowerShell, for example), and can **quarantine** files or isolate the host. Its standout feature is **retrospective security**. If a file that was considered clean last week is later convicted, the system knows every machine where it ran and can act on all of them.
- **Posture assessment (ISE + Secure Client agent):** before granting full network access, ISE checks the device's **health**: Is the OS patched? Is antivirus running and current? Is the disk encrypted? A non-compliant device can be placed in a **remediation** VLAN or given a restricted policy until it's fixed.
- **Profiling:** ISE identifies what a device is (IP phone, printer, camera) from DHCP, CDP/LLDP, HTTP and similar clues. Devices that can't do 802.1X still get an appropriate policy, usually via MAB (MAC Authentication Bypass).

802.1X — the identity check at the switch port

The CCNA guide (18.6) introduced 802.1X as "the bouncer at the port". CCNP expects the moving parts by name:

Role	Who plays it	Does what
Supplicant	Software on the endpoint (Windows/macOS native, Secure Client)	Presents credentials
Authenticator	The access switch (or WLC)	Relays between the two — never checks credentials itself
Authentication server	A RADIUS server, usually **ISE**	Decides, and returns the policy (VLAN, downloadable ACL, SGT)

Two protocols carry the conversation: **EAPOL** (EAP over LAN, EtherType 0x888E) between supplicant and switch, and **RADIUS** (UDP 1812/1813) between switch and ISE. The switch unwraps EAP from EAPOL and re-wraps it in RADIUS. Until authentication succeeds, an 802.1X port passes **only EAPOL** (plus whatever the deployment mode allows, below). **EAP** is the Extensible Authentication Protocol: a common envelope that can carry many different login methods.

Story Time : 802.1X is a hotel check-in. You (the **supplicant**) show your booking to the receptionist (the **authenticator**, the switch). The receptionist doesn't decide anything; they pass your details to the central booking system (**ISE**) and wait. The booking system says "yes, room

10, gym access, no pool" (VLAN, ACL, SGT), and only then does the receptionist hand you a key card that opens those doors. Until then you can stand in the lobby and talk to the receptionist (EAPOL), and that's all.

How an 802.1X login works, step by step (a laptop plugs into Gi1/0/10, PEAP with a username and password):

1. The port comes up in the **unauthorized** state. The switch sends **EAP-Request/Identity** over EAPOL: "who are you?"
2. The supplicant answers **EAP-Response/Identity**: `CORP\jsmith`.
3. The switch wraps that EAP message in a **RADIUS Access-Request** and sends it to ISE on UDP 1812.
4. ISE answers with **Access-Challenge** messages, which the switch relays to the laptop and back. In PEAP this is where a TLS tunnel is built using ISE's certificate, and the password exchange runs inside it.
5. ISE checks the credentials and its policy, then sends **Access-Accept** with the authorization result: for example, VLAN 10, a downloadable ACL, and SGT 10.
6. The switch sends **EAP-Success** to the laptop, applies the VLAN, ACL and SGT, and moves the port to **authorized**. Normal traffic (DHCP first) now flows.
7. With accounting on, the switch sends ISE an **Accounting-Request (Start)** on UDP 1813.

```
SW1# show access-session interface GigabitEthernet1/0/10 details
            Interface:  GigabitEthernet1/0/10
          MAC Address:  0050.5600.0a1b
            User-Name:  CORP\jsmith
               Status:  Authorized
               Domain:  DATA
       Oper host mode:  multi-auth
Server Policies:
              Vlan Group:  Vlan: 10
                  SGT Value:  10
Method status list:
       Method           State
        dot1x           Authc Success
```

Read it as: who (`User-Name`), the result (`Authorized`), what ISE pushed (VLAN 10, SGT 10), and which method won (`dot1x`). On older IOS XE the same information is under `show authentication sessions interface Gi1/0/10 details`.

The **EAP method** decides what credential is proven, and how:

EAP method	Credentials	Notes
EAP-TLS	Certificates on **both** client and server	Strongest; no passwords at all; needs a PKI to issue client certs
PEAP (usually with MSCHAPv2 inside)	Server certificate + username/password	Builds a TLS tunnel using only the server's cert, then runs the password exchange inside it
EAP-FAST	Protected Access Credential (PAC) or certs	Cisco-designed tunnel method; supports **EAP chaining**

EAP method	Credentials	Notes
EAP chaining (EAP-FAST, or the standard **TEAP**)	Machine **and** user credentials in one session	Lets ISE require "a corporate laptop and a valid employee", not just one of them

Not everything can run a supplicant. Printers, cameras and badge readers fall back to **MAB (MAC Authentication Bypass)**: the switch sends the device's MAC address to ISE as the identity, and ISE checks it against its profiling database. MAB is weak on its own because MACs are easily spoofed, which is why it's paired with profiling and a restricted policy. Guests and BYOD users get **WebAuth**, where the switch redirects the browser to a login or acceptable-use portal (usually ISE's central WebAuth).

```
R1(config)# aaa new-model
R1(config)# aaa authentication dot1x default group ISE-RADIUS
R1(config)# aaa authorization network default group ISE-RADIUS  ! lets ISE push VLAN/ACL/SGT
R1(config)# aaa accounting dot1x default start-stop group ISE-RADIUS
R1(config)# dot1x system-auth-control                          ! global on-switch for 802.1X
!
R1(config)# interface GigabitEthernet1/0/10
R1(config-if)# switchport mode access
R1(config-if)# switchport access vlan 10
R1(config-if)# authentication host-mode multi-auth     ! each MAC behind the port authenticates
R1(config-if)# authentication order dot1x mab          ! try 802.1X first, then MAB
R1(config-if)# authentication priority dot1x mab       ! if both succeed, 802.1X wins
R1(config-if)# authentication port-control auto        ! actually enforce
R1(config-if)# dot1x pae authenticator
R1(config-if)# mab
```

Why separate `order` from `priority`? `order` is the sequence the switch tries methods in. `priority` decides which result wins if a later attempt succeeds too. With `order mab dot1x` (MAB first, for faster printer onboarding) plus `priority dot1x mab`, a laptop that MAB'd onto a restricted policy can still be upgraded the moment its supplicant completes 802.1X. Order is about speed; priority is about trust.

Worked example — why the printer takes a minute and a half. With `order dot1x mab` and default timers, a printer (no supplicant) plugs into Gi1/0/11:

1. The switch sends EAP-Request/Identity. No answer.
2. It waits the `dot1x timeout tx-period` (30 s by default) and asks again, up to `dot1x max-reauth-req` (2) more times. Still nothing, because the printer doesn't speak EAP.
3. After about **90 seconds** the switch gives up on 802.1X and falls back to **MAB**. It learns the printer's MAC from its first frame and sends it to ISE as both username and password (by default as 12 lowercase hex digits, for example `001ba9123456`).
4. ISE's profiling recognises a printer, and returns a restricted policy (printer VLAN, an ACL allowing print traffic only). The port is authorized.

Ninety seconds can be long enough for a printer's DHCP client to give up and fall back to a self-assigned address. Common fixes: lower `tx-period` (for example to 10 s, so the fallback happens after about 30 s), or use `order mab dot1x` with `priority dot1x mab` as described above.

Deployments roll out in three stages, because turning on enforcement on day one locks out every device you forgot about:

Mode	Config idea	Effect
Monitor mode	`authentication open`	Authenticate and log everything, but **forward all traffic** whether it passes or fails. Pure visibility.
Low-impact mode	`authentication open` + a **pre-auth port ACL**	Limited access (DHCP, DNS, ISE) before authentication; full access after
Closed mode	Default 802.1X behaviour	Nothing but EAPOL until authenticated

Newer IOS XE uses the **IBNS 2.0** style (`access-session` commands and C3PL policy maps) for the same functions. The concepts are identical; only the syntax differs.

Posture builds on this: after 802.1X or MAB authentication, ISE can hold the device in a restricted state, have the Secure Client posture module report its health, and then issue a **Change of Authorization (CoA)** that moves it to full access, or to remediation if it fails.

Worked example — posture in action. jsmith's laptop authenticates by 802.1X at 08:30. ISE's first answer is a **posture-pending** policy: an ACL allowing only DHCP, DNS, ISE and the patch server. The posture module reports: Windows patched, disk encrypted, but antivirus signatures are 9 days old against a policy limit of 3. ISE marks the laptop **non-compliant** and keeps it restricted, and the user sees a "please update" pop-up. The antivirus updates at 08:34, the agent reports again, and ISE sends a **CoA** to the switch (RADIUS CoA, UDP 1700 on Cisco switches by default). The switch re-authorizes the session, and the laptop gets the full employee policy. The user never unplugged anything.

Why check posture rather than just identity? Because a valid user on an infected or unpatched laptop is still a threat. Authentication proves who is connecting, and posture proves the device is in a safe state. Zero trust verifies both, every time.

18.4 Next-Generation Firewalls

What "next-generation" adds

A **traditional stateful firewall** tracks connections. It knows that a returning packet belongs to a session an inside host started, which is something the stateless ACL `established` trick (17.5) cannot do. But it decides purely on **IP addresses, protocols and ports** (Layers 3–4).

Why isn't that enough anymore? Because ports stopped meaning anything. Almost everything — web apps, file sharing, video, malware command-and-control — runs over **TCP 443**. A rule that says "permit 443" permits all of it. A port-based firewall can't distinguish your CRM from a cloud file-sharing service leaking data, or from a botnet's encrypted check-in.

Story Time : A traditional firewall is a mail room that sorts by what's written on the envelope: sender, recipient, which department (IP addresses and ports). An NGFW also checks what's inside the envelope: "this says it's for Sales, but it's actually a parcel of stolen customer records." The catch, below, is that many envelopes are now sealed (encrypted), and opening them takes extra work and permission.

A **next-generation firewall (NGFW)** keeps the stateful engine and adds layers of understanding:

Capability	What it adds	Example policy it enables
Stateful inspection	Connection tracking (the baseline)	Return traffic allowed automatically
Application visibility and control (AVC)	Identifies the application regardless of port (thousands of app signatures)	"Allow Webex, block BitTorrent, even on 443"
Intrusion prevention (IPS)	Signature and anomaly matching for exploits (Snort engine on Secure Firewall)	Drop an exploit against an unpatched server
URL filtering	Categorises and scores websites	Block gambling and newly registered domains
Advanced malware protection	File-hash reputation, sandboxing of unknown files	Block a known-bad PDF mid-download
Identity awareness	Rules by user or group (from ISE or AD)	"Finance can reach the ERP; contractors cannot"
TLS decryption (optional)	Inspect inside encrypted sessions	Required for IPS or malware inspection of HTTPS content

Why is TLS decryption the uncomfortable trade-off? Without it, the NGFW sees only the server name and certificate of most traffic, so IPS and file inspection are mostly blind. With it, the firewall becomes a deliberate man-in-the-middle, with real costs in performance, privacy (banking and health sites are usually exempted), and certificate management on every endpoint. Designs usually decrypt selectively, by risk category.

How an NGFW handles a new connection, step by step:

1. The first packet (a TCP SYN) arrives. The **stateful** engine checks whether it belongs to an existing session. It doesn't, so the access policy is evaluated.
2. At this point the firewall knows only addresses and ports, so it can't yet name the application. It lets the handshake and the first few data packets through while the **application detector** examines them (for TLS, the requested server name and certificate).
3. Once the application is identified, say "Dropbox", the full rule set is applied: is Dropbox allowed for this user's group, at this URL category?
4. If allowed, file and IPS inspection continue for the rest of the session. If blocked, the session is reset.
5. The session is added to the state table, so return traffic is matched without being evaluated again.

Worked example — two sessions on the same port. Both cross the firewall as TCP 443 from 10.1.1.25:

Session	Old stateful firewall (rule: permit tcp any any eq 443)	NGFW (rules by application and user)
Webex meeting	permitted	identified as **Webex** → permitted, prioritised
Upload to a personal Dropbox account	permitted	identified as **Dropbox**; user is in Finance; policy "no personal file-sharing for Finance" → **blocked**

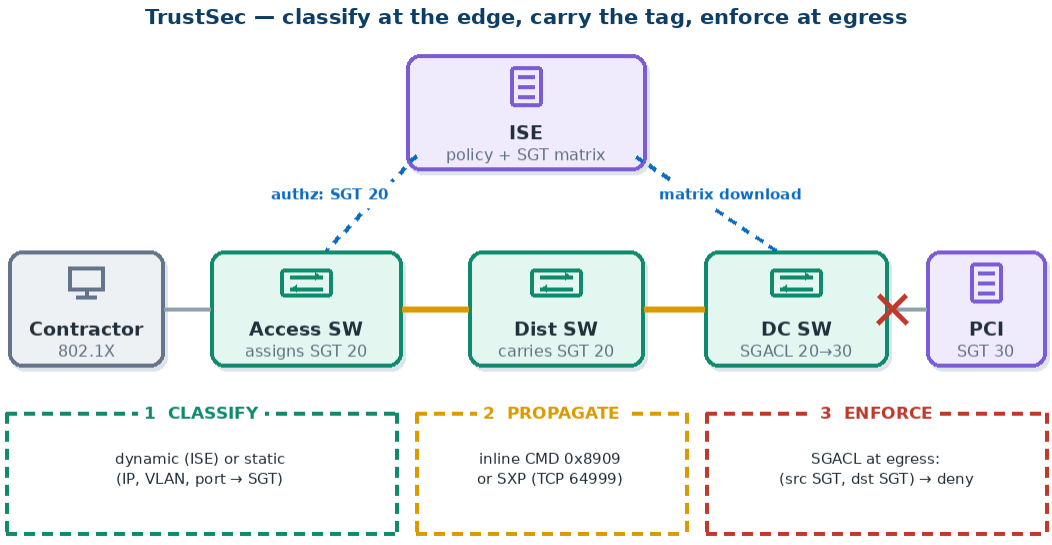
To the port-based firewall the two are identical. Only the application and identity layers can tell them apart.

Where firewalls sit

Placement	Protects
Internet edge	The classic north–south boundary
Data-centre edge / internal segmentation	East–west traffic between zones (users ↔ servers, PCI zone)
Branch (often inside SD-WAN routers, Chapter 14)	Direct internet access without backhauling to HQ
Cloud (virtual NGFW or cloud-delivered firewall)	Workloads and users that never touch your hardware

---

18.5 Cisco TrustSec — Segmentation by Role, Not by IP



The problem it solves

Traditional segmentation ties policy to **IP addresses**: "10.1.20.0/24 may reach 10.50.0.0/24 on 443". That works until users move, subnets are reused, and the rule list grows to thousands of lines on dozens of devices, each a copy that can drift. IP addresses describe where something is, but policy is really about what something is.

Story Time : TrustSec works like coloured wristbands at a music festival. At the entrance gate (the access switch), security checks your ticket once (802.1X) and gives you a wristband: green for general admission, gold for VIP, red for crew. After that, nobody checks your ticket again. The guard at the backstage door doesn't care which gate you came through or where you're standing; they only look at the colour. Policy is "red can go backstage, green cannot", not a list of seat numbers.

TrustSec gives every packet a label describing the role of its sender. That label is a **Security Group Tag (SGT)**, a **16-bit** number such as 10 = Employees, 20 = Contractors, 30 = PCI-Servers. Policy is then written between **groups**: "Contractors → PCI-Servers: deny." It doesn't matter which IP, VLAN or building the contractor is in today.

Three functions: classify, propagate, enforce

1. Classification — the tag is assigned where traffic enters

Method	How
Dynamic	ISE returns the SGT as an authorization result after 802.1X or MAB. The usual method for users and endpoints.
Static	Mapped by configuration: IP/subnet → SGT, VLAN → SGT, or port → SGT. For servers and devices that never authenticate.

2. Propagation — the tag must reach the enforcement point

Method	How it carries the tag	Requirement
Inline tagging	The SGT rides in the frame itself, in a **Cisco Meta Data (CMD)** field inserted in the Ethernet header (EtherType **0x8909**)	Every hop must support TrustSec hardware. Often protected by MACsec.
SXP (SGT Exchange Protocol)	The IP-to-SGT mappings are sent over a **TCP (port 64999)** control-plane connection from devices that can't tag to devices that can enforce	Works across non-TrustSec hardware. Scales less well.

In SD-Access (Chapter 14), the SGT is carried in the **VXLAN header's group policy field** (Chapter 13). That's the same concept on an overlay.

3. Enforcement — usually at egress

The switch or firewall nearest the **destination** looks up (source SGT, destination SGT) in a matrix downloaded from ISE and applies a **Security Group ACL (SGACL)**:

Source ↓ / Destination →	Employees (10)	PCI-Servers (30)	Printers (40)
Employees (10)	permit	deny	permit
Contractors (20)	deny	deny	permit

Why enforce at egress rather than ingress? Because the ingress switch knows the source's group but not necessarily the destination's. The egress device, close to the destination, knows both. It also means the matrix is pushed only to devices that protect a given group, not to every access switch. Classify where you know the source, enforce where you know the destination.

Why is this simpler than ACLs at scale? An IP-based policy for 10 groups across 50 subnets could need thousands of ACL entries copied to many devices. The SGT matrix for 10 groups has at most 100 cells, defined once in ISE. A new subnet, a moved user or a re-IP'd server needs **no policy change**, because their group didn't change.

Worked example — a contractor's packet, hop by hop. A contractor's laptop (10.10.20.57) tries to reach a PCI database server (10.50.0.10). The access switch, distribution and core all support inline tagging; the data-centre switch in front of the PCI servers enforces.

1. **Classify (access switch):** the laptop authenticates with 802.1X. ISE's Access-Accept includes **SGT 20** (Contractors). The access switch now binds 10.10.20.57 → 20.
2. **Tag:** every frame the laptop sends is forwarded up the uplink with a CMD field carrying **20**. The laptop itself knows nothing about this; the switch adds the tag.
3. **Carry:** distribution and core forward the frame and keep the tag. The uplinks run MACsec (18.6), so nobody can rewrite it on the wire.
4. **Destination group (data-centre switch):** the PCI server never authenticates, so it has a **static** mapping configured: `cts role-based sgt-map 10.50.0.10 sgt 30`.
5. **Enforce:** the data-centre switch looks up the cell (source 20, destination 30) in the matrix from ISE: **deny**. The packet is dropped, and the SGACL's counter increments.

```
DC-SW# show cts role-based sgt-map all
Active IPv4-SGT Bindings Information
IP Address              SGT     Source
============================================
10.50.0.10              30      CLI
DC-SW# show cts role-based permissions from 20 to 30
```

The contractor's address isn't in this table, and doesn't need to be: with inline tagging, the source group arrives **inside each frame**.

Next week the contractor moves to another building and gets a different IP address. They authenticate, get SGT 20 again, and the same cell of the matrix blocks them. Nobody touched a policy.

SXP in the same story: if the contractor's access switch is too old to insert CMD tags, it sends the binding "10.10.20.57 = SGT 20" over an SXP TCP connection to a device that can use it, such as the data-centre switch or a firewall. That device then knows the source group from the packet's **IP address** rather than from a tag, and the binding appears in its table with `SXP` as the source:

```
DC-SW# show cts role-based sgt-map all
IP Address              SGT     Source
========================================
10.10.20.57             20      SXP
10.50.0.10              30      CLI
```

18.6 MACsec — Encrypting the Wire

The problem: inside your buildings, traffic crosses cables in ceilings, risers and shared closets, and fibre between sites may run through someone else's ducts. Anyone who can tap a cable can read or alter what's on it, and most internal traffic isn't encrypted by the applications. MACsec encrypts each cable.

MACsec (IEEE 802.1AE) encrypts Ethernet frames at **Layer 2**, link by link, at line rate in switch hardware.

Story Time : MACsec is a chain of armoured vans between depots. Goods travel locked in a van from depot A to depot B. At B they're unloaded, checked and sorted (the switch reads headers, tags and QoS markings), then locked into a new van to depot C. Nobody can open a van on the road, but every depot sees the goods. IPsec is different: it's a locked safe that travels the whole way, and no depot can look inside.

Property	Detail
Standard	IEEE **802.1AE** (encryption); keys agreed by **MKA** — MACsec Key Agreement, IEEE **802.1X-2010**
Cipher	AES-GCM, 128- or 256-bit — confidentiality **and** integrity
Scope	**Hop by hop**: encrypted on each link, decrypted inside each device, re-encrypted on the next link
What's protected	Everything after the source MAC, including the 802.1Q tag and the CMD/SGT field. The MAC addresses stay visible so switching works.
Overhead	A SecTAG header (8–16 bytes) plus a 16-byte integrity check value (ICV): up to 32 bytes per frame. Mind your MTU.

Two deployment shapes

- **Host-to-switch (downlink):** the endpoint authenticates with **802.1X**, ISE indicates MACsec is required, and the derived keys feed **MKA**. The cable from desk to closet, the easiest place to tap, is now encrypted.
- **Switch-to-switch (uplink):** between network devices, typically with **MKA using a pre-shared key** (or 802.1X between switches). Protects trunks running through shared risers, between buildings, or across a provider's dark fibre.

How MKA brings up a link, step by step (switch-to-switch with a pre-shared key):

1. Both switches are configured with the same **CAK** (Connectivity Association Key), the long-term pre-shared secret, and its name (CKN).
2. They exchange MKA messages (EAPOL frames) proving that each holds the same CAK. A switch with a different key is simply never accepted as a peer.
3. One switch is elected **key server** (the lowest key-server priority value wins; a tie goes to the lowest SCI, the Secure Channel Identifier).
4. The key server generates a random **SAK** (Secure Association Key), the key that actually encrypts data, and sends it to its peer protected by a key derived from the CAK.
5. Both install the SAK in hardware. Frames are now encrypted with AES-GCM. The SAK is replaced periodically, so the long-term CAK is never used to encrypt traffic itself.

Worked example — what a tapped cable reveals. An attacker splices into the fibre between building A and building B and captures a frame:

Field	Without MACsec	With MACsec
Destination / source MAC	visible	visible
802.1Q VLAN tag, CMD/SGT	visible (and could be rewritten)	encrypted and integrity-protected
IP addresses, ports, payload	visible	encrypted
A modified frame re-injected	accepted by the next switch	**dropped**: its ICV (integrity check) fails

The attacker learns which two MAC addresses are talking, and nothing else.

Why hop-by-hop instead of end-to-end like IPsec? Because the network needs to look inside frames at every hop: to read the SGT for enforcement, apply QoS, run NetFlow and do security inspection. IPsec (Chapter 12) hides the payload from everything in the middle, which is right across an untrusted WAN but wrong inside your own campus. MACsec protects each **wire** while every **device** still sees the traffic. IPsec protects a conversation across someone else's network; MACsec protects the cables in your own.

Why MACsec and TrustSec go together. An inline SGT in clear text could be forged by anyone who taps the link and rewrites the tag. MACsec's integrity check makes the tag tamper-proof between trusted devices.

18.7 REST API Security

Controllers like Catalyst Center and SD-WAN Manager (Chapter 20), and devices running RESTCONF, expose **REST APIs** that can reconfigure the entire network. An API is an administrator's CLI without the human. It needs every protection an admin login gets, plus some that only matter when the caller is a program.

The building blocks

Control	What it does	Why it matters for APIs
Transport encryption — HTTPS/TLS	Encrypts and integrity-protects every request and response	Credentials and tokens travel in headers on every call; over plain HTTP they're readable by anyone on the path
Authentication	Proves who is calling (methods below)	Anonymous APIs that change config are a gift to attackers
Authorization (RBAC)	Decides what that caller may do: read-only, specific sites, specific APIs	A monitoring script should never hold credentials that can delete devices
Rate limiting / throttling	Caps requests per client per time window (often returning **429 Too Many Requests**)	Protects the controller from runaway scripts and brute force
Input validation	Rejects malformed or unexpected payloads (returning **400**)	Blocks injection attacks and stops a bad automation run corrupting config
Logging and auditing	Records who called what, when, from where	APIs are invisible otherwise, and accountability matters as much as for CLI changes (17.4)
Certificate validation	The client verifies the server's certificate	Without it the client can be tricked into sending credentials to an impostor. `verify=False` in a lab script is fine; in production it's a vulnerability.

Authentication methods compared

Method	How it works	Strengths	Weaknesses
HTTP Basic	`Authorization: Basic <base64(user:pass)>` on every request	Simple; used by RESTCONF on IOS XE	Base64 is **encoding, not encryption**. Safe only inside TLS, and the real password travels on every call.
API key	A long static secret in a header or query string	Easy for service-to-service use	Usually long-lived. A leaked key works until someone notices and rotates it.
Token (session) based	Authenticate once (usually Basic), receive a **token**, send the token on later calls, e.g. Catalyst Center's `X-Auth-Token` (Chapter 20)	The password crosses the wire once. Tokens expire.	You must handle token expiry and refresh.
OAuth 2.0	An authorization server issues **scoped**, short-lived **access tokens** (often JWT bearer tokens) after the user or app authenticates; **refresh tokens** obtain new ones	Delegation without sharing passwords, fine-grained scopes, central revocation	More moving parts

Story Time : Basic auth is showing your passport at every door in a building. Token auth is showing it once at reception and getting a visitor badge that opens the right doors and stops working at 6 p.m. An API key is a permanent badge: handy, but if you drop it in the car park it works for whoever picks it up until someone cancels it. OAuth 2.0 is a valet key: the owner lets the hotel issue a key that can drive the car and park it but can't open the boot (trunk), and the owner never hands over their own key.

How a token-based API session works, step by step (Catalyst Center; Chapter 20 shows the Python code):

1. The script sends `POST /dna/system/api/v1/auth/token` over **HTTPS**, with an HTTP Basic header carrying the service account's username and password.
2. Catalyst Center checks the credentials (locally or against its AAA server) and replies `200 OK` with `{"Token": "eyJ0eXAi..."}`.
3. Every later call carries `X-Auth-Token: eyJ0eXAi...` instead of the password. The controller checks that the token is valid and unexpired, and checks the account's **role** (RBAC, Role-Based Access Control) for that particular API.
4. After about 60 minutes the token expires and calls return **401 Unauthorized**. The script requests a new token (step 1) and carries on.

Worked example — one request, every control. A monitoring script calls `GET /dna/intent/api/v1/network-device` with an observer-role token:

Check	What happens	If it fails
TLS	The script validates the controller's certificate, then the request travels encrypted	Script refuses to connect (the impostor case)
Authentication	Token is valid and unexpired	**401 Unauthorized**
Authorization	Observer role may read devices	A `DELETE` from the same token gets **403 Forbidden**
Rate limit	Under the per-client limit	**429 Too Many Requests** (back off and retry later)
Input validation	No body needed for a GET; a malformed body on a POST is rejected	**400 Bad Request**
Audit	The call, account and source IP are logged	—

Reading the error code tells you which control stopped you. That's exactly how ENCOR 6.5 (Chapter 20) expects you to read API responses.

OAuth 2.0 in three steps (the simplest flow, used by one program calling another): the client program authenticates to an **authorization server** with its own client ID and secret → it receives an **access token** limited to certain **scopes** (for example `devices:read`) and valid for a short time → it sends that token as `Authorization: Bearer <token>` to the API, which checks the signature, expiry and scope. Revoking the client at the authorization server cuts off every API it could call, in one place.

Why do tokens expire? Because every credential eventually leaks: into a log file, a screenshot, a Git commit. A password or static key that leaks stays valid until someone notices. A token that expires after an hour (Catalyst Center's tokens are valid for about 60 minutes by default) limits the attacker's window to the remainder of that hour. Short-lived credentials turn a breach into an inconvenience.

Why is Base64 in Basic auth so often misread as encryption? Because the header looks scrambled: `YWRtaW46Q2lzY28xMjM=`. But anyone can decode it instantly to `admin:Cisco123`. Basic auth provides **no confidentiality on its own**; all of its safety comes from the TLS around it.

Secret hygiene for the caller. Most API breaches aren't clever attacks on the controller. They're credentials leaking from the client side:

- **Never hardcode credentials** in scripts. Read them from environment variables or a secrets vault (HashiCorp Vault, a cloud secrets manager, Ansible Vault), so the code can be shared and committed safely.
- **Keep keys out of URLs.** An API key in a query string (`?api_key=...`) gets written to web-server access logs, proxy logs and browser history. Send it in a header instead.
- **Least privilege per integration.** One service account per script, scoped to exactly what it needs, so a leak can be revoked without breaking everything else.

- **Rotate** keys and secrets on a schedule, and immediately when someone leaves or a leak is suspected.

> **Exam focus:** expect questions that pair a scenario with a control: "a script is overloading the controller" → **rate limiting**; "credentials visible in a packet capture" → **HTTPS/TLS**; "a read-only monitoring tool changed configuration" → **authorization / RBAC**; "a stolen credential remained usable for months" → **expiring tokens / OAuth 2.0**.

Chapter 19: Python & JSON for Network Engineers

> **Blueprint:** ENCOR 6.1 — Interpret basic Python components and scripts; ENCOR 6.2 — Construct valid JSON-encoded files

The verbs in this blueprint matter. **Interpret** Python means you'll be shown a script and asked what it does, what it prints, or why it fails. You won't have to write one from scratch. **Construct** JSON means you must be able to produce, or spot, **valid** JSON exactly. So this chapter teaches reading Python fluently and writing JSON without a single mistake.

You'll use both constantly in Chapter 20 (RESTCONF and controller APIs return JSON) and Chapter 21 (Ansible). Python is the language you automate in; JSON is the language the network answers in.

A few terms used throughout, defined once here:

Term	Plain meaning
Script	A text file of Python instructions, run with `python3 file.py`
CLI (Command-Line Interface)	The `R1#` prompt you type `show` commands into
API (Application Programming Interface)	A doorway built for programs rather than people: you send a structured request, you get structured data back
REST / **HTTP**	The style and protocol most web APIs use: a request to a URL, a numbered status code and a body back (Chapter 20)
JSON (JavaScript Object Notation)	The text format those APIs send data in (19.8)
SSH (Secure Shell)	The encrypted remote-login protocol you already use to reach the CLI
Library / **module**	Ready-made Python code you load with `import` instead of writing it yourself

19.1 Why Python, and How a Script Runs

The problem it solves: configuring 300 switches by hand is slow and error-prone, and every engineer types it slightly differently. A script does the same steps the same way every time, in seconds. To read scripts (which is what the exam asks), you first need to know the order Python runs things in.

Python is **interpreted**: you run `python3 script.py`, and the interpreter reads the file **top to bottom**, executing each line as it reaches it. Two consequences matter when you read scripts:

- **A function's body runs only when it's** called**,** not when the interpreter reaches its `def` line. A `def` just defines it.
- **Indentation is syntax.** Python has no `{ }` braces. The lines that belong to a loop, `if` or function are the ones indented beneath it. Four spaces is the convention. Get it wrong and you get an `IndentationError`, or worse, code that runs in the wrong block.

Story Time : A script is like a recipe card that a very literal cook reads from the top. A `def` block is a sub-recipe written in the margin ("Gravy: melt butter, add flour..."). The cook doesn't make gravy when reading past the margin note. Gravy gets made only when a later step says "now make the gravy." That later step is the **function call**.

Worked example — predict the order.

```
print("start")
def greet(name):
    print(f"hello {name}")
print("middle")
greet("R1")
```

Walk through it the way the interpreter does:

1. Line 1 runs: prints `start`.
2. Lines 2–3: Python remembers a function called `greet`. Nothing prints. The indented line 3 belongs to the function, so it is skipped for now.
3. Line 4 runs: prints `middle`.
4. Line 5 **calls** `greet`, with `name` set to `"R1"`. Now line 3 runs: prints `hello R1`.

Output:

```
start
middle
hello R1
```

The common wrong answer puts `hello R1` second, because it appears second in the file. Definition order isn't run order: follow the calls.

Why did the network industry settle on Python? Readability (it reads close to English), a huge library ecosystem for networking (`requests`, `netmiko`, `ncclient`, `napalm`, Ansible itself), and it's preinstalled on most Linux systems, including the Guest Shell (a small Linux container that runs on the router itself) on IOS XE. The libraries are the real reason: someone has already written the hard part.

19.2 Variables and Data Types

The problem it solves: a script has to remember things while it runs (a hostname, a VLAN number, a list of interfaces) and reuse them. Variables are how it remembers. Data types matter because Python treats the text `"1500"` and the number `1500` completely differently.

A variable is a **name pointing at a value**. There's no declaration, and the type comes from the value:

```
hostname = "R1"            # str   — text, in single or double quotes
vlan_id = 20               # int   — whole number
cpu_load = 37.5            # float — decimal number
is_up = True               # bool  — True or False (capitalised!)
next_hop = None            # None  — "no value"
```

Type	Example	Notes
`str`	`"GigabitEthernet1"`	Immutable. `+` concatenates. `len()` gives the length.
`int`	`1500`	`"1500"` (str) is **not** the same as `1500` (int)
`float`	`99.9`	
`bool`	`True`, `False`	
`list`	`["R1", "R2", "R3"]`	Ordered, changeable, indexed from **0**
`dict`	`{"name": "R1", "asn": 65001}`	**Key → value** pairs; the workhorse for API data
`tuple`	`("10.0.0.1", 22)`	Like a list but immutable
`None`	`None`	Python's "nothing" — becomes `null` in JSON

The classic type trap:

```
mtu = "1500"
print(mtu + 100)       # TypeError: can only concatenate str (not "int") to str
print(int(mtu) + 100)  # 1600
```

Why does this come up constantly in network scripts? Because everything a device sends as CLI output is text. `"1500"` scraped from `show interface` is a string until you convert it with `int()`. JSON from an API, by contrast, often does carry real numbers. Always know whether you're holding text or a number.

Story Time : Think of a variable as a **sticky note** with a name written on it, stuck onto a value. `vlan_id = 20` sticks the note "vlan_id" onto the number 20. `vlan_id = 30` peels the note off and sticks it on 30. Where the "labelled box" picture you may have learned elsewhere breaks is the next example: two sticky notes can be on the **same** list.

```
a = [10, 20]
b = a              # a second sticky note on the SAME list, not a copy
b.append(30)
print(a)           # [10, 20, 30]  — changed "through" b
c = list(a)        # list() builds a NEW list with the same items
c.append(40)
print(a, c)        # [10, 20, 30] [10, 20, 30, 40]  — a is untouched
```

Assignment never copies a list or dict; it adds another name for the same one. That's why a script that "saves the old VLAN list" with `old = vlans` and then changes `vlans` finds that `old` changed too.

Arithmetic — three kinds of division

```
print(7 / 2)     # 3.5   — "/" ALWAYS returns a float, even 8 / 2 gives 4.0
print(7 // 2)    # 3     — floor division: round DOWN to a whole number
print(7 % 2)     # 1     — modulo: the remainder
print(-7 // 2)   # -4    — floor goes down, toward negative infinity, not toward zero
print(2 ** 10)   # 1024  — exponent
```

Exam trap: `print(8 / 2)` prints `4.0`, not `4`. If the question's answer choices include both, the float is the right one.

Sets — unique values only

```
vlans = {10, 20, 20, 30}        # duplicates vanish
print(len(vlans))               # 3
a = {10, 20, 30}
b = {20, 30, 40}
print(a & b)                    # {20, 30}  — in both (intersection)
print(a - b)                    # {10}      — in a but not b
```

A set is unordered and holds each value once. Scripts use it to answer "which VLANs exist on SW1 but not SW2?" in one line. `{}` on its own is an empty **dict**, not a set; an empty set is `set()`.

19.3 Strings and f-strings

The problem it solves: network automation is full of text. CLI output arrives as text, and the commands and URLs a script sends are text it has to build. String methods take text apart; f-strings put it together.

```
intf = "GigabitEthernet1"
print(intf.lower())               # gigabitethernet1
print(intf.startswith("Gig"))     # True
print(intf[0:3])                  # Gig   (slice: index 0 up to, not including, 3)
print("10.1.1.1/24".split("/"))   # ['10.1.1.1', '24']
print(" R1 \n".strip())           # R1    (removes surrounding whitespace)

vlan = 20
name = "USERS"
print(f"vlan {vlan}\n name {name}")   # f-string: {} evaluates the expression inside
```

Output of the last line:

```
vlan 20
 name USERS
```

f-strings (the `f` before the quote) are how scripts build CLI commands and URLs:

```
host = "10.0.0.1"
url = f"https://{host}/restconf/data/ietf-interfaces:interfaces"
```

Worked example — taking a CLI line apart. A script has read one line of `show ip interface brief`:

```
line = "GigabitEthernet1       10.0.0.1        YES manual up                    up      "
fields = line.split()
print(fields)
print(fields[0], fields[1], fields[-1])
```

1. `split()` with no argument splits on **any run of spaces** and throws away the spaces at both ends.
2. `fields` becomes `['GigabitEthernet1', '10.0.0.1', 'YES', 'manual', 'up', 'up']`, six items.
3. `fields[0]` is the name, `fields[1]` the address, and `fields[-1]` (the **last** item) the protocol status.

Output:

```
['GigabitEthernet1', '10.0.0.1', 'YES', 'manual', 'up', 'up']
GigabitEthernet1 10.0.0.1 up
```

The catch: a shut-down interface's status is **two words**, `administratively down`, so that line splits into **seven** fields and `fields[4]` is `'administratively'`. That's why the example uses `fields[-1]` for the last column: counting from the end still works when a middle column grows. Text parsing breaks at exactly these edge cases (19.7).

The two older formatting styles

f-strings arrived in Python 3.6. Older scripts, and plenty of exam snippets, use `.format()` or the `%` operator. All three produce the same text:

```
sw, count = "SW1", 3
print(f"{sw} has {count} uplinks")
print("{} has {} uplinks".format(sw, count))     # {} filled left to right
print("{0} has {1} uplinks, {0}!".format(sw, count))  # numbered: reuse an argument
print("%s has %d uplinks" % (sw, count))         # %s = string, %d = integer
```

Every line prints `SW1 has 3 uplinks` (the third adds `, SW1!`). Format specs work in all of them: `f"{port:>4}"` right-aligns in 4 characters, and `f"{util:.1f}"` shows one decimal place (`37.46` → `37.5`).

19.4 Lists and Dictionaries

The problem it solves: one variable per interface doesn't scale to a 48-port switch. Lists hold many things in order; dictionaries hold named facts about one thing. Nest them and you can describe a whole network, which is exactly what API responses do.

Story Time : A **list** is a row of numbered lockers, and the first locker is number **0**. You fetch by position: "what's in locker 2?" A **dictionary** is your phone's contact list. You don't care what position "Alice" is in; you look her up **by name** and get her number. In Python terms, `routers[2]` asks for a position and `device["hostname"]` asks for a name (a **key**).

Lists — ordered collections

```
routers = ["R1", "R2", "R3"]
print(routers[0])       # R1   — first item is index 0
print(routers[-1])      # R3   — negative counts from the end
routers.append("R4")    # add to the end
print(len(routers))     # 4
print("R2" in routers)  # True
```

Dictionaries — key/value lookups

```
device = {
    "hostname": "R1",
    "mgmt_ip": "10.0.0.1",
    "interfaces": ["Gi1", "Gi2"],
}
print(device["hostname"])             # R1
print(device["interfaces"][1])        # Gi2   — dict, then list index
device["os"] = "IOS-XE"               # add a key
print(device.get("serial"))           # None — .get() doesn't crash on a missing key
print(device["serial"])               # KeyError: 'serial'
```

Three methods walk a dict:

```
device = {"hostname": "R1", "mgmt_ip": "10.0.0.1"}
print(list(device.keys()))     # ['hostname', 'mgmt_ip']
print(list(device.values()))   # ['R1', '10.0.0.1']
for key, value in device.items():
    print(f"{key}={value}")    # hostname=R1  then  mgmt_ip=10.0.0.1
```

Dicts keep **insertion order** (guaranteed since Python 3.7), so `items()` returns keys in the order they were added.

List comprehensions — a loop in one line

```
intfs = [{"name": "Gi1", "status": "up"},
         {"name": "Gi2", "status": "down"},
         {"name": "Gi3", "status": "up"}]
up = [i["name"] for i in intfs if i["status"] == "up"]
print(up)                      # ['Gi1', 'Gi3']
```

Read it right to left: for each `i` in `intfs`, if its status is up, keep `i["name"]`. It's exactly equivalent to a four-line `for` loop with `append()`.

Why are dictionaries the **data structure of network automation?** Because API responses are nested key/value structures, and JSON maps one-to-one onto Python dicts and lists (19.9). Reading API output in Python is mostly chaining lookups: `data["response"][0]["hostname"]`. When you see a chain of square brackets in a script, read it left to right as "open this, then open that inside it."

Worked example — unwrapping a chain one bracket at a time.

```
data = {"response": [{"hostname": "SW1", "upTime": "10 days"},
                     {"hostname": "SW2", "upTime": "3 days"}]}
print(data["response"][0]["hostname"])
```

Step	Expression so far	What you're holding
1	`data`	A **dict** with one key, `"response"`
2	`data["response"]`	A **list** of 2 dicts (square brackets in the data mean list)
3	`data["response"][0]`	The **first** dict: `{'hostname': 'SW1', 'upTime': '10 days'}`
4	`data["response"][0]["hostname"]`	The string `'SW1'` — this is what prints

The rule that unlocks it: **a word in the brackets means "look up this key in a dict"; a number means "take this position from a list."** If the data at that step isn't the kind the bracket expects, Python stops with an error: a key on a list raises `TypeError`, and a missing key raises `KeyError`.

19.5 Conditionals and Loops

The problem it solves: a script has to make decisions ("is this interface down?") and repeat work ("do this for every switch"). `if` makes the decisions; `for` and `while` do the repeating. Most exam "what does this print?" questions are really about following these two correctly.

```
cpu = 85
if cpu > 90:
    print("critical")
elif cpu > 75:
    print("warning")          # this one runs — the first true branch wins
else:
    print("ok")
```

Comparison operators: `==` (equal; `=` is assignment), `!=`, `>`, `<`, `>=`, `<=`. Logical: `and`, `or`, `not`. Membership: `in`.

`for` loops — once per item

```
for vlan in [10, 20, 30]:
    print(f"vlan {vlan}")

for name, ip in {"R1": "10.0.0.1", "R2": "10.0.0.2"}.items():
    print(name, ip)            # R1 10.0.0.1 / R2 10.0.0.2

for i in range(3):             # 0, 1, 2 — range stops BEFORE its argument
    print(i)
```

`range()` takes up to three arguments: `range(stop)`, `range(start, stop)`, `range(start, stop, step)`. The stop value is **never** included:

```
print(list(range(1, 4)))         # [1, 2, 3]
print(list(range(10, 31, 10)))   # [10, 20, 30]
print(list(range(5, 0, -1)))     # [5, 4, 3, 2, 1]  — counting down
```

`enumerate()` gives you a counter alongside each item, so you don't keep one by hand:

```
for index, sw in enumerate(["SW1", "SW2"], start=1):
    print(index, sw)              # 1 SW1   then   2 SW2
```

Nested loops — predict the output

```
for sw in ["SW1", "SW2"]:
    for port in range(1, 3):
        print(f"{sw}-Gi1/0/{port}")
```

The **inner** loop runs completely for each turn of the outer loop:

```
SW1-Gi1/0/1
SW1-Gi1/0/2
SW2-Gi1/0/1
SW2-Gi1/0/2
```

Count lines before reading them: 2 switches × 2 ports = 4 lines. If an answer choice has a different number of lines, eliminate it.

`while` loops — repeat while a condition holds

```
retries = 0
while retries < 3:
    retries += 1                  # same as retries = retries + 1
print(retries)                    # 3
```

`break` leaves a loop immediately. `continue` skips to the next iteration.

Worked example — `continue` and `break` in one loop.

```
intfs = {"Gi1": "up", "Gi2": "down", "Gi3": "admin-down", "Gi4": "up"}
down = 0
for name, status in intfs.items():
    if status == "up":
        continue
    if status == "admin-down":
        print(f"{name} shut on purpose - stopping")
        break
    down += 1
    print(f"{name} is down")
print(f"down count: {down}")
```

Trace it one turn at a time:

Turn	[redacted], [redacted]	What happens
1	`Gi1`, `up`	First `if` is true → `continue`: skip the rest, go to the next item
2	`Gi2`, `down`	Neither `if` matches → `down` becomes 1, prints `Gi2 is down`
3	`Gi3`, `admin-down`	Second `if` is true → prints the stop message, `break` leaves the loop

Turn	[redacted], [redacted]	What happens
—	`Gi4`	**Never visited**: the loop already ended

Then the line after the loop (not indented under it) runs once. Output:

```
Gi2 is down
Gi3 shut on purpose - stopping
down count: 1
```

19.6 Functions, Imports and Exceptions

The problem it solves: without functions, a script repeats the same twenty lines for every device. Without imports, you'd write your own SSH and HTTP code. And without exception handling, the first unreachable device crashes the whole run. These three are what turn a quick hack into a tool you can trust.

Functions

Story Time : A function is like a vending machine. You put things in (the **arguments**: `30`, `"VOICE"`), it does its work out of sight, and it hands one thing back out of the slot (the **return value**). A function that only `print`s is a machine that shows the snack on a screen but never drops it in the slot: you saw it, but your hands are empty, and what you're holding is `None`.

```
def build_vlan(vlan_id, name="UNNAMED"):     # name has a default value
    return f"vlan {vlan_id}\n name {name}"

print(build_vlan(30, "VOICE"))
print(build_vlan(40))                        # uses the default: name UNNAMED
```

A function **returns** a value with `return`. If it has no `return`, it returns `None`. That's a classic "predict the output" trap when someone writes `result = print_something()`:

```
def show(x):
    print(x)          # prints, but returns nothing

result = show("hi")   # the call prints "hi"; result gets None
print(result)
```

Output: `hi` on the first line, then `None`. When the first call runs, the `print` inside it shows `hi`. The function then ends without `return`, so `result` is `None`, and the last line prints that.

Worked example — `build_vlan` step by step. For `build_vlan(40)`:

1. Python matches arguments to parameters: `vlan_id = 40`. No second argument was given, so `name` takes its default, `"UNNAMED"`.
2. The f-string builds `"vlan 40\n name UNNAMED"` (`\n` is a newline character).
3. `return` hands that string back to the caller, and `print` shows it on two lines:

```
vlan 40
 name UNNAMED
```

Imports

```
import json                                 # whole module: use as json.loads(...)
import requests
from netmiko import ConnectHandler          # one name from a module
from ncclient import manager
```

Library	What it does
`requests`	HTTP client for REST APIs (RESTCONF, Catalyst Center, SD-WAN Manager)
`json`	Convert between JSON text and Python objects (standard library)
`netmiko`	SSH to devices and send CLI commands (screen-scraping, made robust)
`ncclient`	NETCONF client (Chapter 20)

Exceptions — handling failure without crashing

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()      # turns 4xx/5xx status codes into an exception
except requests.exceptions.Timeout:
    print("device did not answer")
except requests.exceptions.HTTPError as err:
    print(f"API error: {err}")
finally:
    print("done")                    # runs whether or not there was an error
```

How Python runs that block, step by step, if the device doesn't answer within 5 seconds:

1. `requests.get(...)` waits, and at 5 seconds **raises** a `Timeout` exception: Python abandons the rest of the `try` block immediately (`raise_for_status()` never runs).
2. Python checks each `except` from the top for a matching type. The first one, `Timeout`, matches, so it prints `device did not answer`.
3. The `HTTPError` clause is skipped. Only one `except` ever runs.
4. `finally` runs: prints `done`. The script carries on after the block instead of crashing.

If the device instead answers with **401**, `get` succeeds (a reply arrived) and `raise_for_status()` raises `HTTPError`. The output is then `API error: 401 Client Error: ...` followed by `done`. With no `try` at all, either failure would stop the script with a red traceback.

Story Time : `try`/`except` is the safety net under a trapeze. The act (the `try` block) runs as normal. If the performer falls (an exception), the net (`except`) catches them and the show goes on. `finally` is the crew sweeping the stage afterwards, which happens whether or not anyone fell.

The full form has four clauses, and each runs at a different time:

Clause	Runs when
`try:`	Always — the code that might fail
`except X:`	Only if an exception of type X was raised in `try`

Clause	Runs when
`else:`	Only if `try` finished **without** an exception
`finally:`	**Always**, last, error or not — used for cleanup like closing sessions

```
try:
    vlan = int("20")
except ValueError:
    print("not a number")
else:
    print(f"vlan {vlan} ok")     # runs: no exception happened
finally:
    print("checked")             # always runs
```

Output: `vlan 20 ok` then `checked`. Change `"20"` to `"twenty"` and the output becomes `not a number` then `checked`.

Why does production automation wrap every network call in `try`? Because networks fail: a device is down, a password expired, an API returns 503 while it upgrades. A script pushing a change to 300 devices must not die at device 47 and leave you guessing which ones changed. Catch the failure, record it, move on, and report at the end.

`if __name__ == "__main__":`

You'll see this at the bottom of almost every real script:

```
def main():
    print("collecting inventory")

if __name__ == "__main__":
    main()
```

Python sets the special variable `__name__` to `"__main__"` when a file is **run directly** (`python inventory.py`), and to the module's name when the file is **imported** by another script. So the guarded block runs only when you run the file, not when someone imports its functions to reuse them. It means "if this file is the program, start here."

19.7 Reading a Real Script — Line by Line

Why this section exists: the exam doesn't ask about Python features one at a time. It shows a whole script that combines them and asks what happens. The skill is to follow the run order (19.1) and keep track of what each variable holds.

This is the shape of exam question you should expect. Read it, then predict the output:

```
import requests

requests.packages.urllib3.disable_warnings()     # lab: silence self-signed cert warnings

HOST = "10.0.0.1"
URL = f"https://{HOST}/restconf/data/ietf-interfaces:interfaces"
HEADERS = {"Accept": "application/yang-data+json"}

def get_interfaces():
    resp = requests.get(URL, headers=HEADERS, auth=("admin", "Cisco123"), verify=False)
    if resp.status_code == 200:
        return resp.json()["ietf-interfaces:interfaces"]["interface"]
    print(f"Error {resp.status_code}")
    return []
```

Line(s)	What it does
`import requests`	Loads the HTTP library
`disable_warnings()`	Suppresses the warning that the certificate isn't verified (because `verify=False` below). Fine in a lab, a security gap in production (18.7).
`URL = f"..."`	Builds the RESTCONF URL for the `ietf-interfaces` YANG model (Chapter 20)
`HEADERS`	Asks for **JSON** encoding of the YANG data
`def get_interfaces():`	Defines a function; nothing runs yet
`requests.get(...)`	Sends an HTTPS GET with Basic auth. `resp` holds status code, headers and body.
`resp.status_code == 200`	200 = OK (the full code list is in 20.7)
`resp.json()[...]["interface"]`	Parses the JSON body into a dict, then digs down to the **list** of interfaces
`for intf in get_interfaces():`	Now the function runs; the loop walks the returned list
`"up" if ... else "down"`	Conditional expression: a one-line if/else
`{intf["name"]:<20}`	Format spec: left-align in a 20-character column

The run order, step by step. The table above goes top to bottom through the file; this is the order things actually happen:

1. The `import`, `disable_warnings()` and the three `HOST`/`URL`/`HEADERS` assignments run. `URL` now holds `"https://10.0.0.1/restconf/data/ietf-interfaces:interfaces"`.
2. `def get_interfaces():` is stored. Nothing is sent yet.
3. The `for` line calls `get_interfaces()`. **Now** the function body runs.
4. `requests.get` sends the request and waits. Say the device answers `200` with this body: `{"ietf-interfaces:interfaces": {"interface": [{"name": "GigabitEthernet1", "enabled": true}, {"name": "GigabitEthernet2", "enabled": false}]}}`
5. `resp.json()` turns the body into a dict. `["ietf-interfaces:interfaces"]` opens the outer object, and `["interface"]` takes the list of two dicts. `return` hands that list to the `for` loop.
6. Turn 1: `intf` is the Gi1 dict. `intf["enabled"]` is `True`, so `state = "up"`. The print pads `GigabitEthernet1` to 20 characters, then adds `up`.
7. Turn 2: `intf` is the Gi2 dict, `enabled` is `False`, so `state = "down"`, and it prints.
8. The list is exhausted, so the loop and the script end.

If the device returns two interfaces, GigabitEthernet1 enabled and GigabitEthernet2 disabled, the output is:

```
GigabitEthernet1    up
GigabitEthernet2    down
```

If the password is wrong, the device returns **401**: the script prints `Error 401`, the function returns an empty list, the loop runs zero times, and nothing else prints. **Interpreting the failure path is**

as examinable as the success path.

A second example — netmiko over SSH

```
from netmiko import ConnectHandler

device = {
    "device_type": "cisco_ios",
    "host": "10.0.0.2",
    "username": "admin",
    "password": "Cisco123",
}

with ConnectHandler(**device) as conn:          # opens SSH; closes it when the block ends
    output = conn.send_command("show ip interface brief")
    commands = ["interface Loopback100", "description Managed by Python"]
    conn.send_config_set(commands)              # enters config mode, sends, exits

for line in output.splitlines():
    if "up" in line and "Loopback" not in line:
        print(line.split()[0])                  # first column = interface name
```

The `**device` syntax **unpacks** the dict into keyword arguments: `host="10.0.0.2", username=...`. The `with` block guarantees the SSH session is closed even if an error occurs. The final loop prints the name of every non-loopback interface whose line contains "up".

Worked example — what it prints. Suppose `send_command` returned this text (it's captured before Loopback100 is created, so that loopback isn't in it):

```
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet1       10.0.0.2        YES NVRAM  up                    up
GigabitEthernet2       unassigned      YES NVRAM  administratively down down
Loopback0              2.2.2.2         YES NVRAM  up                    up
```

`output.splitlines()` breaks the text into four lines, and the loop tests each one:

Line	Contains [redacted]?	Contains [redacted]?	Printed?
Header	No	No	No
GigabitEthernet1	Yes	No	**Yes**: `GigabitEthernet1`
GigabitEthernet2	No (`down` and `administratively` don't contain `up`)	No	No
Loopback0	Yes	**Yes**	No: excluded

Output: `GigabitEthernet1`. Note what `"up" in line` really tests: the two letters u-p anywhere in the line. An interface description such as `backup-link` would also match. That fragility is the point of the next paragraph.

Why is netmiko considered a fallback compared with RESTCONF? Because it scrapes **human-formatted text**. If a software upgrade changes a column width, `split()[0]` can silently pick the wrong field. An API returns **structured data** whose keys don't move. Parse text when you must; consume structured data when you can.

19.8 JSON — The Rules, Exactly

The problem it solves: a Python script, a Java-based controller and a router's web server all need to swap data, and none of them speak each other's language. JSON is the neutral text format they all agree on. Because every program must read it the same way, its rules are strict and you must know them exactly.

JSON (JavaScript Object Notation) is a text format for structured data. It has exactly **six value types**:

JSON type	Example	Python equivalent
object	`{"name": "R1"}`	`dict`
array	`[1, 2, 3]`	`list`
string	`"GigabitEthernet1"`	`str`
number	`1500`, `99.5`, `-3`	`int` / `float`
boolean	`true`, `false`	`True`, `False`
null	`null`	`None`

The rules that make or break validity

1. **Keys must be strings in double quotes.** `{"vlan": 10}` is valid. `{vlan: 10}` and `{'vlan': 10}` are both **invalid**.
2. **Strings use double quotes only.** Single quotes are never valid JSON.
3. **No trailing commas.** `[1, 2, 3,]` and `{"a": 1,}` are invalid.
4. **Commas** between **items, colons between key and value.** Missing either is invalid.
5. **`true`, `false`, `null` are lowercase.** `True`, `False`, `None` (Python spellings) are invalid.
6. **No comments.** `//` and `#` are invalid in JSON.
7. **Numbers aren't quoted unless you mean a string.** `"mtu": 1500` is a number; `"mtu": "1500"` is a string. Both are valid, but they're different.
8. **Whitespace between tokens doesn't matter.** One line or pretty-printed, it's the same document.
9. **A document has exactly one top-level value.** RFC 8259 allows any value there (`42`, `"hello"` and `true` are each a complete, valid JSON text), but APIs almost always send one object `{...}` or one array `[...]`.
10. **Numbers have a strict grammar.** No leading zeros (`010`), no leading `+`, no hex (`0x1F`), no bare decimal point (`.5` must be `0.5`), and no `NaN` or `Infinity`. Exponents (`1e3`) and negatives (`-3`) are fine.
11. **Some characters in strings must be escaped with a backslash:** `\"` for a double quote, `\\` for a backslash, `\n` for a newline.

Spot the error

Snippet	Valid?	Why
`{"hostname": "R1", "asn": 65001}`	**Yes**	
`{'hostname': 'R1'}`	No	Single quotes
`{"vlans": [10, 20, 30,]}`	No	Trailing comma in the array
`{"enabled": True}`	No	`True` must be `true`
`{"enabled": "true"}`	Yes	Valid, but the value is the string "true", not a boolean
`{"name": "R1" "asn": 65001}`	No	Missing comma between pairs
`{"interfaces": {"name": "Gi1"}, {"name": "Gi2"}}`	No	Two objects need to be inside an **array**: `[{...}, {...}]`
`{"ip": 10.0.0.1}`	No	`10.0.0.1` isn't a number; it must be the string `"10.0.0.1"`
`{"desc": null}`	Yes	
`{"vlan": 010}`	No	Leading zero — numbers may not start with 0 (except `0` and `0.x`)
`{"loss": NaN}`	No	`NaN` and `Infinity` aren't JSON; use `null` or a string. (Python's `json` accepts them as a non-standard extension, but other parsers and APIs reject them.)
`{"mask": 0xFF}`	No	No hex in JSON
`{"offset": +1}`	No	A leading `+` is not allowed (`-1` is fine)
`{"path": "C:\\cfg"}`	Yes	The backslash is escaped as `\\`
`{"desc": "say "hi""}`	No	Inner quotes must be escaped: `"say \"hi\""`
`{"a": 1, "a": 2}`	Parses, but don't	RFC 8259 says names **should** be unique; behaviour then varies by parser. Python's `json` keeps the **last** value.
`[1, "two", null]`	Yes	Arrays may mix types
`"just a string"`	Yes	Any value is a valid JSON text on its own

Why is JSON this strict when Python is more forgiving? Because JSON is an exchange format between programs written in many languages, so it has to be unambiguous to every parser. A single malformed character makes the whole document unreadable. The API answers **400 Bad Request** and changes nothing. The strictness is the feature: a document either means

exactly one thing or it's rejected.

Story Time : JSON is a customs declaration form. The officer doesn't guess what you meant. A date in the wrong box or a missing signature, and the whole form comes back stamped REJECTED, even though everything else was perfect. That's exactly what a parser does with one stray comma.

How to check a JSON document in four passes

When an exam shows a snippet and asks whether it's valid, check it in this order. Each pass looks for one kind of mistake, so you don't miss one while hunting another.

1. **Brackets.** Every `{` has a `}` and every `[` has a `]`, closed in the reverse order they were opened.
2. **Quotes.** Every key is in double quotes. Every string value is in double quotes. Nothing uses single quotes.
3. **Punctuation.** There's a colon between each key and its value, a comma between items, and no comma after the last item.
4. **Values.** Each value is a double-quoted string, a valid number (no leading zero, `+` or hex), `true`/`false`/`null` in lowercase, an object or an array. Anything else (an unquoted IP address, `True`, `None`) fails.

Worked example — build it from a requirement. "Describe VLAN 30, named VOICE, active, on ports Gi1/0/1 and Gi1/0/2, with no SVI address yet."

1. It's one thing with named facts, so it's an **object**: `{ }`.
2. The VLAN ID is a number: `"id": 30`. The name is text: `"name": "VOICE"`.
3. "Active" is yes/no, so a boolean: `"active": true`.
4. "Ports" is a list of text: `"ports": ["Gi1/0/1", "Gi1/0/2"]`.
5. "No address yet" is the absence of a value: `"svi_ip": null`.
6. Join the pairs with commas, and put no comma after the last one:

```
{"id": 30, "name": "VOICE", "active": true, "ports": ["Gi1/0/1", "Gi1/0/2"], "svi_ip": null}
```

Now run the four passes on it: brackets balanced, all keys and strings double-quoted, commas only between items, and every value one of the six types. It's valid.

19.9 JSON ↔ Python

The problem it solves: JSON arrives as one long piece of text. You can't ask text "what's in the second VLAN?" Converting it into Python dicts and lists (and back again for sending) is what makes the data usable.

```
import json

text = '{"hostname": "R1", "enabled": true, "vlans": [10, 20], "desc": null}'
data = json.loads(text)              # loads = load from String → Python dict
print(data["enabled"])               # True   (JSON true became Python True)
print(data["vlans"][1])              # 20
print(data["desc"])                  # None

back = json.dumps(data, indent=2)    # dumps = dump to String → JSON text
```

`json.load(f)` and `json.dump(obj, f)` (no `s`) do the same with **files**. Memory hook: the `s` stands for string. `requests` gives you `resp.json()`, which is just `json.loads(resp.text)` for you.

Worked example — the round trip, value by value. What `json.loads(text)` did to each piece of the string above:

In the JSON text	Became in Python
`{ ... }`	`dict`
`"hostname": "R1"`	key `'hostname'`, value `'R1'` (str)
`true`	`True`
`[10, 20]`	`[10, 20]` (list of int)
`null`	`None`

So `print(data)` shows Python's spelling: `{'hostname': 'R1', 'enabled': True, 'vlans': [10, 20], 'desc': None}`. Going back, `print(back)` shows JSON's spelling again, pretty-printed by `indent=2`:

```
{
  "hostname": "R1",
  "enabled": true,
  "vlans": [
    10,
    20
  ],
  "desc": null
}
```

Two things to notice. Python's printout uses **single** quotes and `True`/`None`, which is not valid JSON, so never copy a printed dict into an API body. Always use `json.dumps()`. And invalid text fails loudly: `json.loads("{'hostname': 'R1'}")` raises `json.JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)`. The parser is telling you rule 1 from 19.8.

19.10 Construct It — Practice With Answers

Why practise this way: reading about JSON and loops feels easy; predicting output under exam pressure isn't. Work each one on paper before opening the answer, and trace variables in a small table as in 19.5.

Exercise 1. Write valid JSON describing router R1 with ASN 65001, BGP enabled, two neighbours (10.0.0.2 in remote AS 65002, and 10.0.0.6 in remote AS 65003), and no description.

> **Answer**

```
{
  "hostname": "R1",
  "bgp": {
    "asn": 65001,
    "enabled": true,
    "neighbors": [
      {"address": "10.0.0.2", "remote_as": 65002},
      {"address": "10.0.0.6", "remote_as": 65003}
    ]
  },
  "description": null
}
```

Check yourself: the IPs are **strings**, the ASNs are **numbers**, `true` and `null` are lowercase, the two neighbours are objects **inside an array**, and there's **no comma** after the last item at any level.

Exercise 2. Fix this so it is valid JSON (there are five errors):

```
{'interface': "Loopback0", "ipv4": 1.1.1.1, "shutdown": False, "tags": ["core", "mgmt",]
```

Answer

```
{"interface": "Loopback0", "ipv4": "1.1.1.1", "shutdown": false, "tags": ["core", "mgmt"]}
```

1. `'interface'` → `"interface"` (double quotes)
2. `1.1.1.1` → `"1.1.1.1"` (an IP is a string)
3. `False` → `false`
4. Trailing comma after `"mgmt"` removed
5. Missing closing `}` added

Exercise 3. Given `data = {"response": [{"hostname": "SW1", "upTime": "10 days"}, {"hostname": "SW2", "upTime": "3 days"}]}`, what does this print?

```
for dev in data["response"]:
    if "10" in dev["upTime"]:
        print(dev["hostname"].lower())
```

Answer

`sw1`. The loop visits both devices. Only SW1's `upTime` string contains the substring `"10"`, and `.lower()` turns `SW1` into `sw1`.

Exercise 4. What does this print?

```
ports = ["Gi1/0/1", "Gi1/0/2", "Gi1/0/3"]
for n, p in enumerate(ports):
    if n % 2 == 0:
        print(p)
```

Answer

```
Gi1/0/1
Gi1/0/3
```

`enumerate` starts at 0 by default, so `n` is 0, 1, 2. `n % 2 == 0` is true for 0 and 2: the first and third ports.

Exercise 5. What does this print?

```
vlans = {"SW1": [10, 20, 30], "SW2": [20, 30, 40]}
common = set(vlans["SW1"]) & set(vlans["SW2"])
print(len(common), 30 // 4, 30 / 4)
```

Answer

`2 7 7.5`. The intersection is {20, 30} (two items). `30 // 4` floors to 7, and `30 / 4` is always a float: 7.5.

Exercise 6. Is each of these valid JSON? `{"mtu": 1500.0}` · `{"id": 007}` · `[]` · `{"up": TRUE}` · `null`

Answer

Valid, **invalid** (leading zeros), valid (an empty array), **invalid** (`true` must be lowercase), valid (any single value is a valid JSON text).

Exercise 7. What does this print, and what would it print if `"8"` were `"eight"`?

```
def check(v):
    try:
        n = int(v)
    except ValueError:
        return "bad"
    else:
        return "%s x2 = %d" % (v, n * 2)
    finally:
        print("checked", end=" ")

print(check("8"))
```

Answer

`checked 8 x2 = 16`. `finally` runs before the function actually returns, so its `print` (with `end=" "` instead of a newline) appears first, then the returned string is printed. With `"eight"`, `int()` raises `ValueError` and the output is `checked bad`.

Chapter 20: YANG, NETCONF, RESTCONF & Controller APIs

> **Blueprint:** ENCOR 4.6 — Configure and verify NETCONF and RESTCONF; ENCOR 6.3 — Describe the high-level principles and benefits of a data modeling language, such as YANG; ENCOR 6.4 — Describe APIs for Cisco Catalyst Center and SD-WAN Manager; ENCOR 6.5 — Interpret REST API response codes and results in payload using Cisco Catalyst Center and RESTCONF

Chapter 19 gave you the languages: Python to drive, JSON to read. This chapter is about **what you talk to**. There are two layers:

- **Devices**, configured through **model-driven programmability**: YANG models, carried by NETCONF or RESTCONF.
- **Controllers** (Catalyst Center, SD-WAN Manager), which manage hundreds of devices and expose their own **REST APIs**.

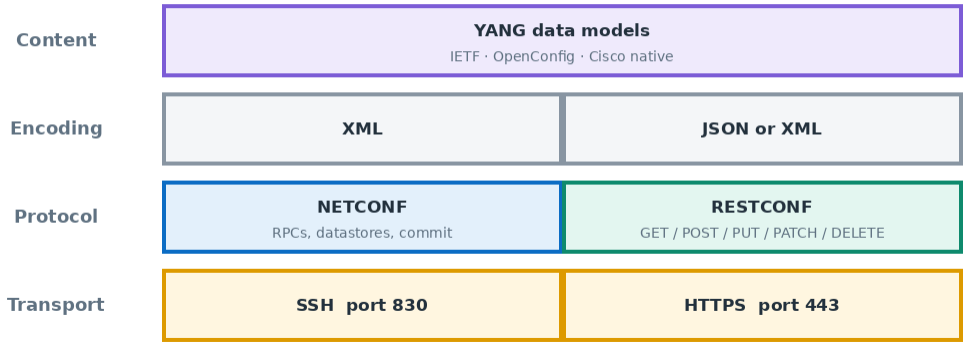
Why does CCNP care so much about this? Because the CLI was built for humans, and its output is formatted for eyes, not programs. Every automation tool that screen-scrapes CLI output (19.7) is one software upgrade away from breaking. Model-driven programmability gives programs what the CLI gives people: a **stable, structured, documented** way to read and change a device.

The chapter is dense with acronyms. Here they all are in one place:

Term	Stands for	Plain meaning
YANG	Yet Another Next Generation	The language that describes what data a device has (20.1)
NETCONF	Network Configuration Protocol	Reads and changes YANG data as XML over SSH (20.2)
RESTCONF	REST + NETCONF	The same YANG data over HTTPS, in JSON or XML (20.3)
XML	eXtensible Markup Language	Text format with `<tags>` around values, like HTML
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)	The web's request/response protocol; HTTPS adds TLS encryption
URI / URL	Uniform Resource Identifier / Locator	The "address" of the thing you're asking for
REST	Representational State Transfer	API style: each thing has a URL, and HTTP methods (GET, PUT…) act on it
RFC	Request for Comments	The numbered documents that define internet standards

Term	Stands for	Plain meaning
RPC	Remote Procedure Call	"Run this operation over there and send me the answer"
Datastore	—	A complete copy of the configuration (running, startup, candidate)
Token	—	A temporary pass a server issues after you log in, so you don't resend the password

Model-driven programmability — one model, two protocols



20.1 YANG — A Data Model for Network Devices

The problem it solves: before a program can read or change "the description on GigabitEthernet1", it needs to know exactly where that item lives, what it's called and what values are allowed. On the CLI that knowledge sits in an engineer's head. YANG writes it down in a form that both programs and devices can check.

YANG (Yet Another Next Generation, RFC 7950) is a **data modelling language**. It doesn't configure anything itself. It **describes** what configuration and state data exists, its structure, its data types and its constraints.

Story Time : A YANG model is like the blank **form** at the DMV. It says there's a "name" field (text, required), a "date of birth" field (a date, must be in the past), and a list of "vehicles", each with make, model and plate. The form isn't your data; it's the agreed **shape** of the data. Once everyone uses the same form, any clerk can process any application without guessing.

The building blocks

YANG node	What it holds	JSON shape it becomes	Example
module	The whole model, with a namespace and prefix	Prefix on top-level keys	`ietf-interfaces`
container	A group of related nodes (one instance)	Object `{}`	`interfaces`
list	Many entries, each identified by a **key**	Array of objects `[{}, {}]`	`interface`, keyed by `name`
leaf	One value of a defined type	`"key": value`	`enabled` (boolean), `mtu` (uint16)
leaf-list	Many values of one type	Array of values	DNS servers `["8.8.8.8", "1.1.1.1"]`

A trimmed piece of the real `ietf-interfaces` module:

```
module ietf-interfaces {
  namespace "urn:ietf:params:xml:ns:yang:ietf-interfaces";
  prefix if;
  container interfaces {
    list interface {
      key "name";
      leaf name        { type string; }
      leaf description { type string; }
      leaf type        { type identityref { base interface-type; } mandatory true; }
      leaf enabled     { type boolean; default "true"; }
    }
  }
}
```

The same data as **JSON** (what RESTCONF returns):

```
{
  "ietf-interfaces:interfaces": {
    "interface": [
      {
        "name": "GigabitEthernet1",
        "description": "Uplink to core",
        "type": "iana-if-type:ethernetCsmacd",
        "enabled": true
      }
    ]
  }
}
```

Read the mapping: **container → object, list → array of objects, leaf → key/value**. The module name prefixes the top-level key (`ietf-interfaces:interfaces`). Once you can map a YANG tree to its JSON shape, you can predict any RESTCONF payload without ever seeing one.

Reading a YANG tree

Nobody reads raw YANG to find a path. Engineers (and exam exhibits) use the **tree view** that the `pyang` tool prints with `pyang -f tree`. An excerpt of `ietf-interfaces` (RFC 8343):

```
module: ietf-interfaces
  +--rw interfaces
     +--rw interface* [name]
        +--rw name                            string
        +--rw description?                    string
        +--rw type                            identityref
        +--rw enabled?                        boolean
        +--rw link-up-down-trap-enable?       enumeration {if-mib}?
        +--ro admin-status                    enumeration {if-mib}?
        +--ro oper-status                     enumeration
        +--ro last-change?                    yang:date-and-time
        +--ro phys-address?                   yang:phys-address
        +--ro speed?                          yang:gauge64
        +--ro statistics
           +--ro discontinuity-time    yang:date-and-time
           +--ro in-octets?            yang:counter64
           +--ro in-errors?            yang:counter32
```

Read it symbol by symbol:

Symbol	Meaning	In this tree
`+--rw`	**Configuration** data: readable and writable	`name`, `description`, `enabled` — things you set
`+--ro`	**State** (operational) data: read-only, declared `config false` in the model	`oper-status`, `speed`, the `statistics` counters
`*`	A **list** (or leaf-list): many entries	`interface*` — one entry per interface
`[name]`	The list's **key**: what identifies each entry	Why the RESTCONF URI says `interface=GigabitEthernet1`
`?`	**Optional** leaf (may be absent)	`description?`
No `?` on a leaf	Mandatory, or always present	`type` is mandatory
`{if-mib}?`	Exists only if the device supports that **feature**	`admin-status`
Indentation	Nesting: each level is a container or list inside the one above	`statistics` is a container inside each `interface`
Right-hand column	The leaf's **data type**	`boolean`, `string`, `yang:counter64`

Why does the rw/ro split matter in practice? Because it tells you what you can **send** versus what you can only **read**. A RESTCONF PATCH that tries to set `oper-status` is rejected, because it's state, not configuration. And `?content=config` (20.3) returns only the `rw` nodes, while `?content=nonconfig` returns only the `ro` ones. `rw` is what you intend; `ro` is what actually happened. Older models (RFC 7223) kept state in a separate `interfaces-state` container; the current model merges them into one tree, as shown.

Worked example — from the tree to a request and its answer. Task: find out whether GigabitEthernet1 is actually up.

1. **Find the leaf in the tree.** "Actually up" is operational state, so look for an `ro` leaf: `oper-status`. Its path is `interfaces` → `interface*` → `oper-status`.
2. **Handle the list.** `interface*` is a list keyed by `[name]`, so you must pick an entry: `interface=GigabitEthernet1`.
3. **Add the module prefix** to the top node, and the RESTCONF prefix in front of that (20.3): `https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1/oper-status`
4. **Predict the answer.** The target is one leaf of type `enumeration`, so the JSON is a single key/value whose key carries the module prefix:

```
{"ietf-interfaces:oper-status": "up"}
```

The same interface data over **NETCONF** is XML. The module is named by its **namespace** (`xmlns`) instead of a prefix, each node becomes a pair of tags, and the list key (`name`) appears as an ordinary child element:

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  <interface>
    <name>GigabitEthernet1</name>
    <oper-status>up</oper-status>
  </interface>
</interfaces>
```

Same model, same path, two encodings. That's the "one model, many protocols" benefit below, made concrete.

Who writes the models

Model family	Written by	Strength	Weakness
IETF (`ietf-interfaces`, `ietf-ip`, ...)	Standards body	Vendor-neutral	Covers only common features
OpenConfig (`openconfig-interfaces`, ...)	Operators (Google, Microsoft, AT&T, ...)	Vendor-neutral, operationally focused, including streaming telemetry	Still not every feature
Native (`Cisco-IOS-XE-native`, ...)	The vendor	Covers **everything** the platform can do	Cisco-only; your code won't work on another vendor

Why use YANG at all — what problem does it solve? Three things:

1. **Structure instead of text.** A program reads `"enabled": true` directly. It doesn't need a regex to find "administratively down" in CLI output.
2. **Validation before change.** The model defines types and constraints, so the device can reject `mtu = 99999` or a missing mandatory leaf before touching the configuration.
3. **One model, many protocols.** The same YANG model is served over NETCONF (XML), RESTCONF (JSON or XML) and gNMI. Learn the model once and use it everywhere.

YANG is the contract; NETCONF and RESTCONF are just two delivery trucks that carry data shaped by it.

20.2 NETCONF — The Transactional Protocol

The problem it solves: pushing a change by sending CLI lines one at a time means a failure halfway through leaves the device half-changed. Nobody else is stopped from editing at the same moment, either. NETCONF gives programs a proper way to read and write YANG data, with locks and all-or-nothing commits.

NETCONF (RFC 6241) is a protocol for reading and changing device configuration using **XML-encoded** YANG data over **SSH, TCP port 830**.

NETCONF's four layers

Layer	Contents
Content	The configuration or state data itself — XML shaped by a YANG model
Operations	`<get>`, `<get-config>`, `<edit-config>`, …
Messages	`<rpc>` (client request) and `<rpc-reply>` (server answer), each with a `message-id`; plus `<notification>`
Transport	SSH (port 830), securely carrying it all

Operations

Operation	What it does
`<get>`	Retrieve **configuration and operational state** (counters, status)
`<get-config>`	Retrieve **configuration only**, from a named datastore
`<edit-config>`	Change configuration (merge, replace, create, delete, remove)
`<copy-config>`	Replace a whole datastore with another
`<delete-config>`	Delete a datastore (not running)
`<lock>` / `<unlock>`	Take exclusive control of a datastore while you change it
`<commit>`	Make the **candidate** datastore become running
`<close-session>` / `<kill-session>`	End your session / forcibly end another

Datastores

Datastore	Meaning
running	The live configuration (always exists)
startup	Loaded at boot (optional capability)
candidate	A scratch copy you edit, validate, then `<commit>` all at once (optional capability)

Why is NETCONF "transactional", and why does that matter? With the CLI, a 40-line change is 40 separate commands, each applied immediately. If line 23 fails, lines 1–22 are already live and you have a half-configured device. NETCONF with the **candidate** datastore stages the entire change, validates it as a whole, and commits it **all or nothing**. Add `<lock>` and nobody else can change the device mid-edit. Across many devices this makes network-wide changes safe in a way CLI scripts never are. A transaction is the difference between "the change worked" and "most of the change worked."

Story Time : The candidate datastore is an online **shopping cart**. You add items one by one (each `<edit-config>` to candidate), and nothing is charged yet. At checkout (`<commit>`) the whole order goes through, or, if the card is declined, nothing is bought. You never end up with half an order. `<lock>` is like holding the last item in stock while you check out, so nobody else can grab it. Where the analogy stops: a shop checks each item on its own, whereas a commit checks the configuration **as a whole**. A route-map that refers to a prefix-list added in the same cart is valid, because both arrive together.

A full NETCONF session, step by step

This is everything that happens when a script makes one transactional change on a device with the candidate datastore enabled:

1. **TCP and SSH.** The client opens TCP to port **830** and authenticates over SSH with the privilege-15 user (20.4).
2. **The NETCONF subsystem.** Inside SSH, the client asks for the `netconf` **subsystem**, not a shell. You get no `R1#` prompt; the device starts speaking XML.
3. **Server `<hello>`.** The device sends its capabilities and a **session-id** (shown below).
4. **Client `<hello>`.** The client replies with its own capabilities. If both list `base:1.1`, they switch to chunked framing.
5. **`<lock>` candidate,** then **`<lock>` running**, so no other session can change either one mid-edit. Each request gets `<ok/>`.
6. **`<edit-config>` with target `candidate`.** The change is staged; running is untouched.
7. **`<validate>`** (optional): the device checks the whole candidate against the YANG models and reports any error now, before anything goes live.
8. **`<commit>`.** The candidate becomes the running configuration in one step. If it fails, running is left exactly as it was.
9. **`<unlock>`** both datastores, then **`<close-session>`**.

Every request is an `<rpc message-id="N">`, and every answer is an `<rpc-reply message-id="N">` containing `<ok/>`, `<data>` or `<rpc-error>`. Matching IDs is how the client knows which reply answers which request.

A NETCONF request in practice

```
from ncclient import manager

with manager.connect(host="10.0.0.1", port=830, username="admin", password="Cisco123",
                     hostkey_verify=False) as m:
    for cap in m.server_capabilities:            # the <hello> exchange lists capabilities
        if "candidate" in cap:
            print(cap)
    flt = """<filter><interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
             </filter>"""
    reply = m.get_config(source="running", filter=flt)
    print(reply.xml)                             # XML: <rpc-reply><data>...</data></rpc-reply>
```

When a session opens, both sides send a **`<hello>`** listing their **capabilities**: the operations, datastores and YANG models they support. That's how a client discovers what the device can do.

Reading the script as it runs:

1. `manager.connect(...)` performs steps 1–4 above. `hostkey_verify=False` skips checking the device's SSH key, which is acceptable in a lab only. The `with` block closes the session at the end.
2. The loop walks every capability string the device sent. On IOS XE, where the candidate datastore is **off by default** (20.4), it prints nothing. With it enabled, it prints `urn:ietf:params:netconf:capability:candidate:1.0`.
3. `flt` is a **subtree filter**: an empty `<interfaces>` element in the right namespace means "give me everything under interfaces, and nothing else."
4. `get_config(source="running", ...)` sends `<get-config>` and waits for the `<rpc-reply>`.
5. `reply.xml` is the reply as text: the interfaces configuration, inside `<rpc-reply><data>`.

What actually goes over the wire

`ncclient` hides the protocol, but you can speak NETCONF by hand. Opening the NETCONF **SSH subsystem** on port 830 shows exactly what the library does:

```
$ ssh -p 830 admin@10.0.0.1 -s netconf
<?xml version="1.0" encoding="UTF-8"?>
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <capabilities>
    <capability>urn:ietf:params:netconf:base:1.0</capability>
    <capability>urn:ietf:params:netconf:base:1.1</capability>
    <capability>urn:ietf:params:netconf:capability:candidate:1.0</capability>
    ...hundreds of YANG module capabilities...
  </capabilities>
  <session-id>2871</session-id>
</hello>]]>]]>
```

The device speaks first with its `<hello>` and a **session-id**. The client must answer with its own `<hello>` (without a session-id) before sending any RPC. A minimal client hello:

```
<?xml version="1.0" encoding="UTF-8"?>
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <capabilities>
    <capability>urn:ietf:params:netconf:base:1.0</capability>
  </capabilities>
</hello>]]>]]>
```

This client advertises only `base:1.0`, so both sides stay on the simple `]]>]]>` framing, which is the easiest way to type a session by hand. Each message is then followed by a **framing marker** so the receiver knows where it ends:

NETCONF version	Framing	How a message ends
1.0 (`base:1.0`)	End-of-message marker	The literal sequence `]]>]]>`
1.1 (`base:1.1`)	**Chunked** framing	Each chunk starts `#<length>` on its own line; the message ends with `##`

If **both** sides advertise `base:1.1`, they switch to chunked framing right after the hello exchange. Chunked framing exists because `]]>]]>` could legally appear inside XML data and cut a message short; a length prefix can't be fooled. That's why a hand-typed session works until 1.1 kicks in: from then on, every message needs a correct byte count.

An `<edit-config>` that creates a loopback looks like this (the same data a RESTCONF PATCH would carry as JSON):

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <edit-config>
    <target><running/></target>
    <config>
      <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
        <interface xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-type">
          <name>Loopback100</name>
          <description>Created by NETCONF</description>
          <type>ianaift:softwareLoopback</type>
          <enabled>true</enabled>
        </interface>
      </interfaces>
    </config>
  </edit-config>
</rpc>]]>]]>
```

The device answers `<rpc-reply message-id="101" ...><ok/></rpc-reply>`. The **message-id** ties each reply to its request. Read the structure from the outside in: **message** (`<rpc>`) → **operation** (`<edit-config>`) → **datastore** (`<target><running/>`) → **content** (YANG-shaped data). Those are exactly the four layers in the table above. To delete instead of merge, mark the node with an operation attribute: `<interface nc:operation="delete">`, with `xmlns:nc` bound to the base NETCONF namespace. Use `<target><candidate/></target>` followed by `<commit/>` for the transactional version.

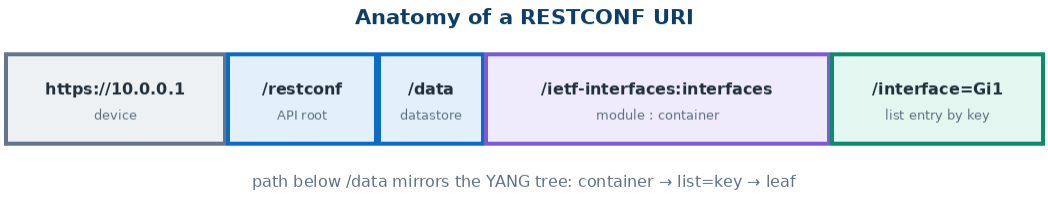
20.3 RESTCONF — YANG over HTTP

The problem it solves: NETCONF is powerful, but it needs an SSH client, XML and a NETCONF library. Most programmers, web tools and quick scripts already speak HTTP and JSON. RESTCONF serves the same YANG data in that familiar way, so `curl` or a few lines of Python is enough.

RESTCONF (RFC 8040) exposes the **same YANG data** through a **REST-style HTTPS API**. If you know how to use a web API, you know how to use RESTCONF.

Story Time : Picture the device's data as a big **filing cabinet** laid out exactly as the YANG tree says: drawer `interfaces`, a folder per `interface` labelled with its name, and a sheet per leaf. A RESTCONF URL is the **exact address of a drawer, folder or sheet**. The HTTP method says what to do there: **GET** reads it, **POST** adds a new folder (refused if one with that label exists), **PUT** swaps the folder's contents for exactly what you hand over, **PATCH** updates only the sheets you hand over, and **DELETE** throws the folder out.

URI structure




```
https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1
```

Part	Meaning
`https://10.0.0.1`	The device, over HTTPS (port 443)
`/restconf`	The API root (discoverable via `GET /.well-known/host-meta`)
`/data`	The datastore resource: configuration and state. `/operations` is for RPCs.
`ietf-interfaces:interfaces`	**module:top-level-container**
`/interface=GigabitEthernet1`	A **list** entry selected by its **key** value

A slash in a key value must be percent-encoded: `GigabitEthernet1%2F0%2F1` for `GigabitEthernet1/0/1`.

Building URIs — beyond one key

The path always follows the YANG tree: **container names are path segments, a list entry is `list=key`**. A few patterns cover nearly everything:

You want	URI after
One leaf from the **native** model	`Cisco-IOS-XE-native:native/hostname`
A native-model interface (keyed by number only)	`Cisco-IOS-XE-native:native/interface/GigabitEthernet=1`
A list with a **two-part key**	`openconfig-acl:acl/acl-sets/acl-set=MGMT,ACL_IPV4` (keys `name` and `type`, comma-separated, in the order the model declares them)
One leaf inside a list entry	`ietf-interfaces:interfaces/interface=Loopback100/descrip`

RFC 8040 **query parameters** shape what comes back:

Parameter	Effect	Example
`?depth=N`	Only N levels of nesting below the target	`.../interfaces?depth=2` lists interfaces without their contents
`?content=config`	Configuration (`rw`) nodes only	Back up just the intent
`?content=nonconfig`	State (`ro`) nodes only	Just counters and status
`?content=all`	Both (the default)	
`?fields=...`	Only the named children	`.../interfaces?fields=interface(name;ena`

Notice the response's top-level key follows the **target**: GET `.../native/hostname` returns `{"Cisco-IOS-XE-native:hostname": "R1"}`, not the whole native tree.

A GET, step by step — what crosses the wire

Take `curl -k -u admin:Cisco123 -H "Accept: application/yang-data+json" https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=Loopback0`:

1. **TLS handshake** with 10.0.0.1 on port 443. `-k` tells curl to accept the device's self-signed certificate (a lab shortcut; 18.7).
2. **The request goes out.** `-u admin:Cisco123` becomes an `Authorization` header: the word `Basic` plus `admin:Cisco123` in Base64, which is an encoding, not encryption. Only TLS keeps it secret.

```
GET /restconf/data/ietf-interfaces:interfaces/interface=Loopback0 HTTP/1.1
Host: 10.0.0.1
Authorization: Basic YWRtaW46Q2lzY28xMjM=
Accept: application/yang-data+json
```

3. **The device checks the user** through AAA (20.4). A wrong password or a user below privilege 15 gets **401**.
4. **It follows the path** through the YANG tree: module `ietf-interfaces`, container `interfaces`, list entry with key `Loopback0`. If there's no such entry, the answer is **404**.
5. **It encodes the result** in the format `Accept` asked for, and replies:

```
HTTP/1.1 200 OK
Content-Type: application/yang-data+json

{
  "ietf-interfaces:interface": {
    "name": "Loopback0",
    "type": "iana-if-type:softwareLoopback",
    "enabled": true,
    "ietf-ip:ipv4": {
      "address": [{"ip": "1.1.1.1", "netmask": "255.255.255.255"}]
    }
  }
}
```

Read the body against the tree: the top key is the **target** node, `interface`, prefixed with its module. The IPv4 address sits under `ietf-ip:ipv4`, a container that the separate `ietf-ip` module **augments** (adds) into each interface, which is why it carries a different prefix. Different IOS XE releases wrap a single list entry slightly differently (as an object, or as an array of one), so read

whichever you're given rather than assuming.

HTTP methods → operations

HTTP method	RESTCONF meaning	NETCONF equivalent	Success code
GET	Read	`<get>` / `<get-config>`	200 OK
POST	**Create** a new child resource (fails if it already exists)	`<edit-config>` create	**201 Created**
PUT	**Create or replace** the target resource entirely	`<edit-config>` replace	201 Created or **204 No Content**
PATCH	**Merge** into the existing resource	`<edit-config>` merge	**204 No Content**
DELETE	Remove the resource	`<edit-config>` delete	**204 No Content**

Worked example — POST creates. POST goes to the **parent** (the `interfaces` container), and the body is the new child:

```
$ curl -k -u admin:Cisco123 -X POST \
    -H "Content-Type: application/yang-data+json" \
    https://10.0.0.1/restconf/data/ietf-interfaces:interfaces \
    -d '{"ietf-interfaces:interface": [{"name": "Loopback100",
                                        "type": "iana-if-type:softwareLoopback"}]}'
HTTP/1.1 201 Created
Location: https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=Loopback100
```

201 plus a `Location` header that gives the new resource's own URI. That's the address the PATCH and PUT below use. Send the same POST again and the answer is **409 Conflict** (worked through in 20.7), because POST never overwrites.

Here are PATCH and PUT, in full, against the same loopback:

```
$ curl -k -u admin:Cisco123 -X PATCH \
    -H "Content-Type: application/yang-data+json" \
    -H "Accept: application/yang-data+json" \
    https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=Loopback100 \
    -d '{"ietf-interfaces:interface": [{"name": "Loopback100",
                                        "description": "PATCHed"}]}'
HTTP/1.1 204 No Content
```

PATCH merged the description in; the loopback's IP and enabled state are untouched. Now the PUT:

```
$ curl -k -u admin:Cisco123 -X PUT \
    -H "Content-Type: application/yang-data+json" \
    https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=Loopback100 \
    -d '{"ietf-interfaces:interface": [{"name": "Loopback100",
                                        "type": "iana-if-type:softwareLoopback",
                                        "description": "Replaced",
                                        "enabled": true}]}'
HTTP/1.1 204 No Content
```

204 because the resource already existed and was replaced; the same PUT to a new name returns **201 Created**. After this PUT the loopback has only what the body listed: any IPv4 address

configured under it in this model is gone. Note the body's top-level key is the **list name** (`interface`, as an array with one entry), and its `name` must match the key in the URI. If they differ, the device returns **400**.

Why does the PUT vs PATCH distinction matter so much? Because **PUT replaces**. PUT an interface body containing only a description, and every leaf you didn't include (IP address, enabled state) is removed from that resource. **PATCH merges**: it changes only the leaves you send. PUT is "make it exactly this", PATCH is "change just these bits." Mixing them up is how a one-line description change takes an interface's IP away.

Media types — the `Accept` and `Content-Type` headers

Header value	Meaning
`application/yang-data+json`	YANG data encoded as JSON
`application/yang-data+xml`	YANG data encoded as XML

`Accept` says what format you want **back**. `Content-Type` says what format you're **sending** (needed on POST, PUT and PATCH). Send a body with the wrong or missing `Content-Type` and you get **415 Unsupported Media Type**.

NETCONF vs RESTCONF

Property	NETCONF	RESTCONF
Transport	**SSH, port 830**	**HTTPS, port 443**
Encoding	XML	**JSON or XML**
Operations	RPCs (`get-config`, `edit-config`, …)	HTTP methods (GET, POST, PUT, PATCH, DELETE)
Datastores	running, candidate, startup; locking; commit	Effectively a single unified datastore view; no candidate/lock
Transactions	Yes — all-or-nothing with candidate and commit	Each request stands alone
Best for	Multi-step, multi-device transactional change	Simple reads and edits, web tooling, quick scripts

---

20.4 Enabling and Verifying NETCONF and RESTCONF on IOS XE

Why this section matters: neither API is on by default. The blueprint verb is configure and verify, so you need to know the handful of commands that turn them on, and how to prove each piece is working when a script can't connect.

Both need a **privilege-15 user** (they authenticate through **AAA**, Authentication, Authorization and Accounting, Chapter 17) and **AAA** enabled. In plain terms: the API checks the username and password the same way an SSH login does, and only a fully privileged user may use it.

```
R1(config)# username admin privilege 15 algorithm-type scrypt secret Cisco123
R1(config)# aaa new-model
R1(config)# aaa authentication login default local
R1(config)# aaa authorization exec default local       ! the API user needs priv 15 granted
!
! --- NETCONF ---
R1(config)# netconf-yang                               ! starts the NETCONF server on port 830
!
! --- RESTCONF ---
R1(config)# ip http secure-server                      ! RESTCONF rides on the HTTPS server
R1(config)# ip http authentication local
R1(config)# restconf
```

Verify:

```
R1# show netconf-yang status
netconf-yang: enabled
netconf-yang candidate-datastore: disabled
netconf-yang side-effect-sync: enabled
netconf-yang ssh port: 830
...
R1# show platform software yang-management process
confd            : Running
nesd             : Running
syncfd           : Running
ncsshd           : Running               ! NETCONF SSH daemon
dmiauthd         : Running
nginx            : Running               ! web server that fronts RESTCONF
ndbmand          : Running
pubd             : Running
R1# show netconf-yang sessions
R1# show ip http server secure status
```

The candidate datastore is off by default on IOS XE. `netconf-yang feature candidate-datastore` turns it on (with a brief restart of the NETCONF server).

A quick end-to-end test of RESTCONF from any workstation:

```
curl -k -u admin:Cisco123 -H "Accept: application/yang-data+json" \
  https://10.0.0.1/restconf/data/ietf-interfaces:interfaces/interface=Loopback0
```

Symptom	Likely cause	Proof
NETCONF connection refused on 830	`netconf-yang` not configured, or the processes are still starting (they take a minute or two after enabling)	`show platform software yang-management process`
RESTCONF returns **401**	Wrong credentials, or the user isn't privilege 15	`show running-config | include username`, `debug aaa authentication`
RESTCONF connection refused on 443	`ip http secure-server` missing	`show ip http server secure status`
RESTCONF returns **404** for a valid-looking URI	Wrong module prefix or key, or the resource doesn't exist	Try a `GET` on the parent container

Worked example — "the password is right, but I still get 401." A colleague created `username netops secret Cisco123` and RESTCONF answers 401 every time.

1. **Is the server even reachable?** A 401 is an HTTP answer, so HTTPS, `restconf` and nginx are all working. Rule out the 443 and 404 rows.
2. **Is the password checked locally?** `show running-config | include aaa authentication|ip http auth` shows `aaa authentication login default local` and `ip http authentication local`. So it is.
3. **What privilege does the user have?** `show running-config | include username netops` shows no `privilege 15`, so the user is at the default level, privilege 1.
4. **Fix:** `username netops privilege 15 secret Cisco123`. The same curl now returns **200**.

A 401 with a correct password almost always means a privilege or AAA-method problem, not a typo.

20.5 Catalyst Center APIs

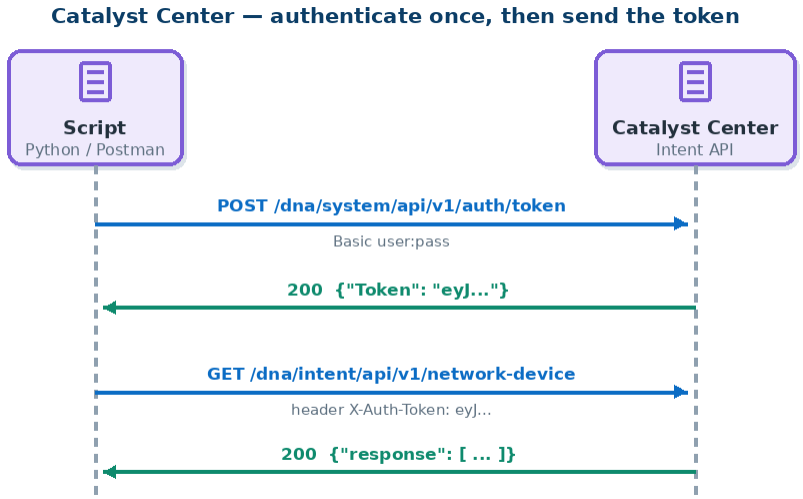
The problem it solves: RESTCONF talks to one device at a time. A question like "which of my 600 switches are unreachable?" or "push this template to every branch" needs one place that already knows the whole network. Catalyst Center is that place, and its API lets your scripts ask it directly instead of logging in to 600 boxes.

Cisco **Catalyst Center** (formerly DNA Center, Chapter 16) manages the campus. It exposes APIs in four directions. Northbound means "up toward you and your apps", southbound "down toward the devices", and east/westbound "sideways to other systems":

Direction	API	Purpose
Northbound	**Intent API** (REST)	You (scripts, apps) tell Catalyst Center **what** you want: device inventory, sites, health, templates, path trace
Southbound	**Multivendor SDK**	Catalyst Center manages devices, including third-party ones via device packages
Eastbound	**Events and notifications**	Catalyst Center pushes events (device down, compliance failure) to webhooks, email, syslog
Westbound	**Integration API**	Connects to IT systems: ITSM (ServiceNow), IPAM, reporting

Why is it called the "Intent" API? Because you describe outcomes ("give me client health for this site", "deploy this template to these switches"), not device commands. Catalyst Center works out the per-device configuration. That's the abstraction a controller buys you: one API call for what would be a hundred CLI sessions.

Authentication flow — get a token, then use it



Story Time : It works like a music festival **wristband**. You show your ID once at the gate (username and password), and you get a wristband (the **token**). For the rest of the day you flash the wristband at each stage (each API call) instead of showing your ID again. Wristbands expire, so tomorrow you queue at the gate again (the token lasts about an hour). And anyone who steals your wristband gets in as you, which is exactly why tokens must only ever travel over HTTPS.

Step 1 — request a token with HTTP Basic credentials:

```
POST https://cc.corp.example/dna/system/api/v1/auth/token
Authorization: Basic YWRtaW46Q2lzY28xMjM=
```

```
{"Token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiI1ZTk..."}
```

Step 2 — send the token in the `X-Auth-Token` header on every later call:

```
GET https://cc.corp.example/dna/intent/api/v1/network-device
X-Auth-Token: eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiOiI1ZTk...
```

The token is valid for **about one hour**. After that, calls return **401** and the script must request a new one (18.7 explains why tokens expire).

In Python:

```
import requests
BASE = "https://cc.corp.example"
tok = requests.post(f"{BASE}/dna/system/api/v1/auth/token",
                    auth=("admin", "Cisco123"), verify=False).json()["Token"]
hdrs = {"X-Auth-Token": tok, "Accept": "application/json"}
devices = requests.get(f"{BASE}/dna/intent/api/v1/network-device",
                       headers=hdrs, verify=False).json()["response"]
for d in devices:
    print(d["hostname"], d["managementIpAddress"], d["reachabilityStatus"])
```

What happens, line by line:

1. `requests.post(..., auth=("admin", "Cisco123"))` sends the Basic `Authorization` header (`YWRtaW46Q2lzY28xMjM=` is `admin:Cisco123` in Base64). Catalyst Center answers `200` with `{"Token": "eyJ0eXAi..."}`.
2. `.json()["Token"]` pulls the token string out of that JSON, and `tok` now holds it.
3. `hdrs` builds the header dict that goes with every later call. The password is never sent again.
4. `requests.get(...)` asks for the inventory. The response JSON is `{"response": [ {...}, {...} ], "version": "1.0"}`, and `["response"]` keeps just the list of devices.
5. The loop prints three fields per device. With two devices, the output looks like:

```
edge-sw1 10.10.20.11 Reachable
edge-sw2 10.10.20.12 Unreachable
```

Run this script 70 minutes later with the same `tok` and step 4 returns **401**, because the wristband expired. Real scripts catch the 401, repeat step 1 and retry.

Common Intent API paths you should recognise:

Path	Returns or does
`/dna/intent/api/v1/network-device`	Device inventory
`/dna/intent/api/v1/site`	Site hierarchy
`/dna/intent/api/v1/network-health`	Overall network health score
`/dna/intent/api/v1/client-health`	Client health by category
`/dna/intent/api/v1/template-programmer/...`	Configuration templates
`/dna/intent/api/v1/task/{taskId}`	Status of an asynchronous job

Asynchronous calls — the 202 pattern

Long-running operations (deploying a template, running a command across 200 devices) don't return the result straight away. They return **202 Accepted** with a **task ID**:

```
{
  "response": {
    "taskId": "6f2a-4b1c-9d0e-81a7",
    "url": "/api/v1/task/6f2a-4b1c-9d0e-81a7"
  },
  "version": "1.0"
}
```

The script then **polls** `GET /dna/intent/api/v1/task/{taskId}` until the task reports it has finished (and whether it ended in error). **Why asynchronous?** An HTTP request that waited ten minutes for 200 switches would time out. The API says "accepted, here's your ticket" and lets you check back. 202 means "I'll do it", not "it's done".

Story Time : It's a coat check. You hand over your coat (the job) and get a numbered ticket straight away (**202** and the `taskId`). You don't stand at the counter while they find a hanger. Later you come back with the ticket and ask "is it ready?" (poll the task URL).

Worked example — polling a task.

```
import time
task_id = resp.json()["response"]["taskId"]           # resp = the 202 response
while True:
    task = requests.get(f"{BASE}/dna/intent/api/v1/task/{task_id}",
                        headers=hdrs, verify=False).json()["response"]
    if task.get("endTime"):                           # the key appears once it's finished
        break
```

1. Take the `taskId` from the 202 body above (`"6f2a-4b1c-9d0e-81a7"`).
2. GET the task. While the job runs, the task object has no `endTime`, so the loop sleeps 5 seconds and asks again.
3. Once the job is finished, `endTime` appears and the loop breaks.
4. `isError` (true/false) says how it ended, and `progress` (or `failureReason` on error) says what happened. The script prints, for example, `OK` followed by the progress text.

The exact fields in the task object vary a little between Catalyst Center releases, but the pattern of 202 → `taskId` → poll until finished → check for error is constant.

20.6 Catalyst SD-WAN Manager APIs

The problem it solves: the same as Catalyst Center's, but for the WAN. One API answers "which branch routers are down, and how healthy is each tunnel?" for the whole SD-WAN fabric. What's different here is the login method, and that's what exam questions probe.

SD-WAN Manager (formerly vManage, Chapter 14) is the management plane of Catalyst SD-WAN. Its REST API lives under **`/dataservice/`** and covers device inventory, monitoring (interface stats, tunnel health, alarms), configuration templates and policies.

Authentication flow — session cookie plus XSRF token

Step 1 — log in with a **form POST** (not Basic auth):

```
POST https://vmanage.corp.example/j_security_check
Content-Type: application/x-www-form-urlencoded

j_username=admin&j_password=Cisco123
```

A successful login sets a **`JSESSIONID`** session **cookie**. Gotcha: a failed login usually still returns **200**, with an HTML login page as the body instead of a cookie. Scripts must check for the cookie, not just the status code.

Step 2 — fetch the XSRF token (needed for any request that changes something):

```
GET https://vmanage.corp.example/dataservice/client/token
Cookie: JSESSIONID=...
```

The response body is the token string.

Step 3 — call the API with the cookie, plus the **`X-XSRF-TOKEN`** header on POST, PUT and DELETE:

```
GET https://vmanage.corp.example/dataservice/device
Cookie: JSESSIONID=...
```

```
{
  "header": {"title": "Devices"},
  "data": [
    {"host-name": "branch1-cedge", "system-ip": "10.255.1.1", "reachability": "reachable",
     "device-type": "vedge", "version": "17.12.03"}
  ]
}
```

Why does SD-WAN Manager need an XSRF token on top of the cookie? A browser automatically attaches cookies to every request to that site, including requests triggered by a malicious page in another tab (**cross-site request forgery**). The XSRF token must be read and deliberately copied into a header, and a forged cross-site request can't do that. The cookie proves who you are; the XSRF token proves the request came from your own code.

Worked example — the three steps in Python. A `requests.Session` stores cookies for you, just as a browser does:

```
import requests
BASE = "https://vmanage.corp.example"
s = requests.Session()
s.verify = False                                        # lab only: self-signed certificate
r = s.post(f"{BASE}/j_security_check",
           data={"j_username": "admin", "j_password": "Cisco123"})   # data= sends form fields
if "<html>" in r.text:                                  # login page came back = login failed
    raise SystemExit("login failed")
s.headers["X-XSRF-TOKEN"] = s.get(f"{BASE}/dataservice/client/token").text
for dev in s.get(f"{BASE}/dataservice/device").json()["data"]:
    print(dev["host-name"], dev["system-ip"], dev["reachability"])
```

1. `s.post(..., data=...)` sends the username and password as **form fields** (not Basic auth). On success the reply sets `JSESSIONID`, and the session stores it and sends it on every later request automatically.
2. The `if` handles the gotcha above: a failed login still returns 200, but the body is the HTML login page.
3. The token GET returns the token as **plain text**, so it's `.text`, not `.json()`. It's added to the session's headers for every later request.
4. The device GET returns the JSON shown above, and `["data"]` is the list. Output: `branch1-cedge 10.255.1.1 reachable`.

Property	Catalyst Center	SD-WAN Manager
API base	`/dna/intent/api/v1/...`	`/dataservice/...`
Login	`POST /dna/system/api/v1/auth/token` with Basic auth	`POST /j_security_check` with form fields
Credential on later calls	**`X-Auth-Token`** header	**`JSESSIONID`** cookie (+ **`X-XSRF-TOKEN`** header for changes)
Data wrapper in responses	`"response": ...`	`"data": [...]`

20.7 HTTP Response Codes — Reading the Answer

The problem it solves: an automation script can't look at a screen and see that something went wrong. It has to decide from the response itself. The status code is the server's one-number verdict, and the body explains the details. ENCOR 6.5 asks you to interpret both.

Every HTTP response starts with a **status code**. The first digit is the category:

Class	Meaning	Who's at fault
1xx	Informational	—
2xx	**Success**	—
3xx	**Redirection** — look elsewhere	—
4xx	**Client error** — your request is wrong	You / your script
5xx	**Server error** — the request may be fine, the server failed	The device or controller

The codes you must know:

Code	Name	Typical network-API meaning
200	OK	GET succeeded; data in the body
201	Created	POST (or PUT) created a new resource
202	Accepted	Async job queued; poll the task ID (Catalyst Center)
204	No Content	Success with no body: RESTCONF PATCH, PUT-replace, DELETE
301 / 302	Moved / Found	Redirect, e.g. HTTP → HTTPS
304	Not Modified	Cached copy still valid
400	Bad Request	Malformed JSON/XML or invalid values (19.8)
401	Unauthorized	**Not authenticated**: missing, wrong or expired credentials or token
403	Forbidden	**Authenticated but not allowed** (RBAC, 18.7)
404	Not Found	Wrong URI, module, key, or the resource doesn't exist
405	Method Not Allowed	e.g. POST to a resource that only supports GET
409	Conflict	RESTCONF POST of something that **already exists**
415	Unsupported Media Type	Wrong or missing `Content-Type` (e.g. `application/json` where `application/yang-data+json` was required)
429	Too Many Requests	Rate limit hit (18.7)
500	Internal Server Error	Server-side failure processing a valid-looking request

Code	Name	Typical network-API meaning
503	Service Unavailable	Server overloaded, starting up, or under maintenance; retry later

Why is 401 vs 403 a favourite exam distinction? Because the fixes are completely different. **401** says "I don't know who you are": fix the credentials or get a fresh token. **403** says "I know exactly who you are, and you may not do this": changing the password won't help; the account needs a different role. 401 is identity; 403 is permission.

Story Time : At an office building's front desk, **401** is "I don't recognise you, show me your badge." **403** is "Yes, I know you, but your badge doesn't open the server room." **404** is "There's no room 9 on this floor." And **500** is "Sorry, the lift is broken": nothing you did, and asking again later may work.

Reading any response in three steps

1. **First digit.** 2 means it worked, so stop worrying. 4 means your request needs fixing. 5 means the server failed, so retry later or escalate.
2. **The exact code** narrows it down (the table above). With 4xx in particular, it usually names the fix: credentials (401), role (403), URI (404), method (405), a clash with existing data (409), header (415), or slowing down (429).
3. **The body** gives the precise reason: RESTCONF's `errors` block, or the controller's `response` and `error` fields. On success, the body is the data you asked for, and you read it with the same bracket-chaining as 19.4.

Interpreting a success payload — worked example

A Catalyst Center inventory call returns **200** with this body (trimmed to four fields per device):

```
{
  "response": [
    {"hostname": "core-1", "managementIpAddress": "10.10.0.1",
     "reachabilityStatus": "Reachable", "softwareVersion": "17.12.4"},
    {"hostname": "edge-7", "managementIpAddress": "10.10.7.1",
     "reachabilityStatus": "Unreachable", "softwareVersion": "17.9.5"}
  ],
  "version": "1.0"
}
```

Questions the exam asks about a payload like this, with the reasoning:

Question	Answer	How you know
Did the call succeed?	Yes	**200**, and the data is under `response`
How many devices are listed?	2	`response` is an array with two objects
Which device needs attention?	`edge-7`	Its `reachabilityStatus` is `Unreachable`

Question	Answer	How you know
Python to get `edge-7`'s IP?	`data["response"][1]["managementIpAddress"]`	dict key → list position 1 → dict key
Is `softwareVersion` a number?	No, a **string**	It's in double quotes; `"17.12.4"` couldn't be a JSON number anyway (two dots)

A 200 means the request worked. It doesn't mean the network is healthy. Here the call succeeded and is reporting a problem device.

Interpreting an error payload — worked example

A script sends this RESTCONF request:

```
POST https://10.0.0.1/restconf/data/ietf-interfaces:interfaces
Content-Type: application/yang-data+json
{"ietf-interfaces:interface": [{"name": "Loopback10",
                                "type": "iana-if-type:softwareLoopback"}]}
```

The response:

```
HTTP/1.1 409 Conflict
Content-Type: application/yang-data+json
{
 "errors": {
  "error": [
   {
    "error-type": "application",
    "error-tag": "data-exists",
    "error-message": "object already exists: /if:interfaces/if:interface[if:name='Loopback10']"
   }
  ]
 }
}
```

Read it: **409** plus `error-tag: data-exists` means Loopback10 **already exists**. POST only creates. To change the existing interface, use **PATCH** (merge) or **PUT** (replace). RESTCONF error bodies always follow this `errors → error[] → error-type / error-tag / error-message` shape. The status code tells you the category; the error-tag and message tell you the exact cause.

Chapter 21: EEM, Orchestration Tools & AI in Operations

> **Blueprint:** ENCOR 6.6 — Construct an EEM applet to automate configuration, troubleshooting, or data collection; ENCOR 6.7 — Compare agent vs. agentless orchestration tools; ENCOR 6.0 — Automation and Artificial Intelligence (domain context)

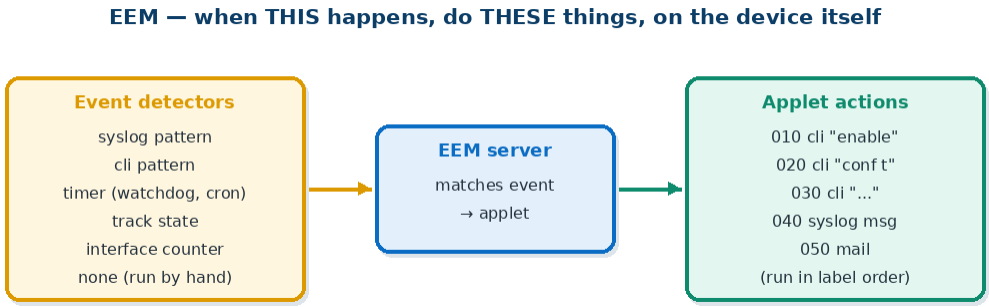
Chapters 19 and 20 automated the network **from outside**: a script or controller reaches in through an API. This chapter covers the other two places automation lives:

- **On the box itself**: the **Embedded Event Manager (EEM)**, which reacts to events locally, even when the network that would carry a script's commands is broken.
- **Across the fleet**: **orchestration tools** (Ansible, Puppet, Chef, SaltStack) that keep hundreds of devices in a declared state.

It finishes with the newest part of the v1.2 blueprint's domain title, **AI in network operations**: what it is genuinely good for, and the guardrails that keep it safe.

Note the verb for EEM: **construct**. You must be able to write a working applet, which means getting the syntax exactly right.

21.1 EEM — Automation That Lives on the Device



EEM is an event-driven automation engine built into IOS and IOS XE. Its logic is one sentence: **when** this **happens, do** these **things.**

Why put automation on the device rather than in an external script? Three reasons:

1. **It works when the network doesn't.** If an uplink fails, a central server can't reach the device to collect diagnostics. EEM is already there.
2. **It reacts in the moment.** The CPU spikes for 20 seconds at 3 a.m. By the time a human logs in, it's gone. An applet triggered by the spike captures `show processes cpu sorted` during the event.
3. **No infrastructure.** No server, no credentials stored elsewhere, no Python environment. It's just configuration.

EEM is your on-site technician who never sleeps and does exactly what the runbook says.

Story Time : EEM is a smart-home routine: "when the front door opens after 11 p.m., turn on the hall light and text me." The **event** is the door sensor, and the **actions** are the light and the text. The routine runs in the house itself, so it still works when your phone has no signal. Where the analogy stops: an EEM action is a CLI command run with full authority over the device, so a careless applet can do as much damage as a careless engineer.

Anatomy of an applet

```
event manager applet <NAME>
 event <detector> <parameters>          ! one trigger (several only with event tags)
 action <label> <action-type> <args>    ! one or more actions, run in label order
```

Typed in global configuration mode, the prompt changes to `(config-applet)` while you're inside the applet:

```
R1(config)# event manager applet HELLO
R1(config-applet)# event none
R1(config-applet)# action 010 puts "Hello from EEM"
R1(config-applet)# end
R1# event manager run HELLO
Hello from EEM
```

The applet is stored in the running configuration like any other feature, so `copy run start` saves it.

Event detectors (the "when")

Detector	Fires when	Example
`syslog`	A log message matches a regular expression	`event syslog pattern "%LINEPROTO-5-UPDOWN.*GigabitEthernet0/1`
`cli`	Someone types a command matching a pattern	`event cli pattern "^reload" sync no skip yes`
`timer`	Time-based: `watchdog` (every N s), `cron`, `countdown`, `absolute`	`event timer cron cron-entry "0 2 * * *"`
`track`	A tracked object (IP SLA, interface state, Chapter 15) changes state	`event track 10 state down`
`interface`	An interface counter crosses a threshold	`event interface name GigabitEthernet0/1 parameter input_errors entry-op ge entry-val 100 entry-type increment poll-interval 60` — `entry-type` is required: `value` (the raw counter), `increment` (change since the last poll) or `rate` (change per second)
`snmp`	An SNMP OID value crosses a threshold	`event snmp oid <oid> get-type exact entry-op ge entry-val 90 poll-interval 30`
`none`	Never automatically — only when run by hand	`event none` + `event manager run <NAME>`

Actions (the "do")

Action	Purpose
`cli command "<cmd>"`	Run a CLI command, exactly as if typed
`syslog msg "<text>"`	Generate a log message (optionally `priority`)
`mail server ... to ... from ... subject ... body ...`	Send an email
`wait <seconds>`	Pause
`set <var> <value>`, `if` / `else` / `end`, `foreach`	Variables and flow control
`puts "<text>"`	Print to the terminal (useful with `event none`)
`reload`	Reload the device

The output of the most recent `cli` action is stored in the built-in variable **`$_cli_result`**.

Variables, regexp and decisions

EEM fills in built-in variables depending on what triggered the applet, and you can create your own with `set`:

Variable	Holds	Available when
`$_cli_result`	Output of the most recent `cli` action	Any applet that runs a `cli` action
`$_syslog_msg`	The full syslog message that matched	`event syslog`
`$_cli_msg`	The command the user typed	`event cli`
`$_event_type_string`	The type of event that fired (e.g. `syslog`, `timer watchdog`)	Any event
`$_event_pub_time`	When the event was published	Any event
`$_regexp_result`	`1` if the last `regexp` matched, `0` if not	After a `regexp` action

The `regexp` action pulls a value out of command output, and `if` / `else` / `end` act on it:

```
event manager applet ERR-CHECK
 event timer watchdog time 600
 action 010 cli command "enable"
 action 020 cli command "show interface GigabitEthernet0/1 | include input errors"
 action 030 regexp "([0-9]+) input errors" "$_cli_result" match errs
 action 040 if $errs gt "1000"
 action 050  syslog priority warnings msg "Gi0/1 has $errs input errors"
 action 060 else
 action 070  syslog msg "Gi0/1 input errors OK ($errs)"
 action 080 end
```

How to read action 030: apply the pattern to `$_cli_result`, put the **whole match** in `match` and the **first bracketed group** (the number) in `errs`. `if` compares numerically when both sides are numbers (`eq`, `ne`, `gt`, `ge`, `lt`, `le`). Two more actions round out the toolkit: `set count 0` creates a

variable, and `increment count` adds 1 to it (or `increment count 5` adds 5), which is how an applet counts loop iterations or retries.

Worked example — one firing of ERR-CHECK, step by step.

1. The watchdog timer counts down 600 seconds (10 minutes) and fires. EEM starts the applet.
2. **010** `enable` moves the applet's private CLI session from user EXEC to privileged EXEC.
3. **020** runs the `show` command. The `| include` keeps one line, so `$_cli_result` holds: `1234 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored` followed by the next prompt (`R1#`), which EEM captures too. The regular expression below ignores it.
4. **030** applies the regular expression `([0-9]+) input errors`. `[0-9]+` means "one or more digits", and the brackets capture them. So `match` = `1234 input errors` and `errs` = `1234`.
5. **040** checks `1234 gt 1000`. It's true, so **050** runs, and **060–070** (the `else` branch) are skipped.
6. **050** logs at severity warnings (4). EEM syslog actions appear under the `HA_EM` facility with the applet name in front of your text: `%HA_EM-4-LOG: ERR-CHECK: Gi0/1 has 1234 input errors`
7. **080** closes the `if`. The applet ends, and the timer starts the next 10-minute countdown.

With 12 errors instead, step 5 takes the `else` branch, and the log reads `%HA_EM-6-LOG: ERR-CHECK: Gi0/1 input errors OK (12)` (no `priority` given, so the default, informational, which is 6).

More than one trigger — event tags and correlation

An applet normally has one event. To fire on a **combination**, give each event a tag and add a `trigger` block that correlates them:

```
event manager applet BOTH-UPLINKS-DOWN
 event tag e1 track 10 state down
 event tag e2 track 20 state down
 trigger
  correlate event e1 and event e2
 action 010 syslog priority critical msg "Both WAN uplinks are down"
```

This fires only when **both** tracked objects are down, which one-event applets can't express. `or` works too, and `trigger occurs 3 period 60` makes an applet fire only if its event happens three times within 60 seconds, which is handy for ignoring a single flap.

The two rules everyone gets wrong

Rule 1 — an applet starts at user EXEC. A `cli` action runs in a fresh session with **no privilege**. So the **first** CLI action is almost always `enable`, and `configure terminal` must appear explicitly before any config command:

```
 action 1.0 cli command "enable"
 action 2.0 cli command "configure terminal"
 action 3.0 cli command "interface Gi0/1"
```

Rule 2 — action labels sort as strings**, not numbers.** EEM runs actions in **lexical (alphabetical) order** of their labels:

Labels you wrote	Order EEM runs them
`1.0`, `2.0`, … `9.0`, `10.0`	**`1.0`, `10.0`, `2.0`**, `3.0` … `9.0`
`010`, `020`, … `100`	`010`, `020`, … `100`
`1.0`, `1.1`, … `1.9`, `2.0`	(all same length before the dot)

Why does lexical ordering cause real bugs? Compare `"10.0"` and `"2.0"` character by character: `"1"` sorts before `"2"`, so `10.0` runs before `2.0`. An applet with ten actions labelled `1.0` to `10.0` runs its tenth action second, perhaps saving the config before making the change. Pad labels to equal width (`010`, `020` …) or keep them under ten, and leave gaps so you can insert actions later.

> **Watch out for AAA.** If command authorization is on (17.4), applet commands are authorized too, as a user with no name, and get rejected. Fix it with `event manager session cli username "eem-svc"` pointing at an account that's authorized on the AAA server.

21.2 Three Applets You Should Be Able to Write

Why this section exists: the blueprint says construct, and it names three purposes: configuration, troubleshooting and data collection. Here is one applet for each, plus a repeatable method for writing any applet from a plain-English requirement.

How to construct any applet — the method

1. **Name it** after what it does (`WAN1-DOWN`, not `TEST1`). The name is what `show event manager` lists.
2. **Pick the detector** from the requirement's "when". A log message → `syslog`. A schedule → `timer`. A probe result → `track`. Someone typing a command → `cli`.
3. **Copy the real trigger text.** For `syslog`, copy the exact message from `show logging` and turn the variable parts into `.*`.
4. **Start the actions with `enable`**, and add `configure terminal` if anything changes the configuration (Rule 1, 21.1).
5. **Number the labels with equal width** (`010`, `020` …) so they sort correctly (Rule 2, 21.1).
6. **Raise `maxrun`** if the actions might take more than 20 seconds altogether, including any `wait`.
7. **Leave a trace:** finish with a `syslog` action, so there's a record that the applet ran.
8. **Verify** it with `show event manager policy registered`, and test it with `debug event manager action cli`.

Worked example — from requirement to applet. "When Gi0/2 is err-disabled by BPDU guard, log a critical message, wait 30 seconds, then bounce the port."

Step	Decision
1	Name: `GI02-ERRDISABLE`
2	"When … err-disabled" is a log message, so `syslog`

Step	Decision
3	The real message is `%PM-4-ERR_DISABLE: bpduguard error detected on Gi0/2, putting Gi0/2 in err-disable state`, so the pattern is `ERR_DISABLE: bpduguard.*Gi0/2,` (the comma stops it also matching Gi0/20 to Gi0/29)
4	It changes config (`shutdown` / `no shutdown`), so `enable` then `configure terminal`
5	Labels `010`–`080`
6	It contains `wait 30`, which is over the 20-second default, so `maxrun 60`
7	A `syslog` action comes first, before the wait

```
event manager applet GI02-ERRDISABLE
 event syslog pattern "ERR_DISABLE: bpduguard.*Gi0/2," maxrun 60
 action 010 syslog priority critical msg "Gi0/2 err-disabled by BPDU guard - bouncing in 30s"
 action 020 wait 30
 action 030 cli command "enable"
 action 040 cli command "configure terminal"
 action 050 cli command "interface GigabitEthernet0/2"
 action 060 cli command "shutdown"
 action 070 cli command "no shutdown"
 action 080 cli command "end"
```

In production, `errdisable recovery cause bpduguard` does the bounce natively. The value of the exercise is the method, which works for any requirement. Without `maxrun 60`, EEM would kill the applet during the `wait`, and the port would never come back.

1 — Configuration: restore a backup route when a tracked object fails

```
track 10 ip sla 1 reachability
!
event manager applet WAN1-DOWN
 event track 10 state down
 action 010 cli command "enable"
 action 020 cli command "configure terminal"
 action 030 cli command "ip route 0.0.0.0 0.0.0.0 192.0.2.254 name EEM-BACKUP"
 action 040 cli command "end"
 action 050 syslog priority warnings msg "WAN1 SLA failed - EEM installed backup default"
!
event manager applet WAN1-UP
 event track 10 state up
 action 010 cli command "enable"
 action 020 cli command "configure terminal"
 action 030 cli command "no ip route 0.0.0.0 0.0.0.0 192.0.2.254 name EEM-BACKUP"
 action 040 cli command "end"
 action 050 syslog msg "WAN1 SLA restored - EEM removed backup default"
```

What happens when WAN1 fails, in order:

1. IP SLA 1 (a probe that pings a target beyond WAN1 every few seconds) stops getting replies.
2. Track 10 follows the probe's reachability, so it changes from Up to Down and logs `%TRACK-6-STATE: 10 ip sla 1 reachability Up -> Down`.

3. That state change is the event `WAN1-DOWN` waits for. The applet runs its actions, adding the backup default route through 192.0.2.254, and logs the warning.
4. When the probe succeeds again, track 10 goes Up, and `WAN1-UP` removes the route.

Two applets, one for each direction, is the normal pattern: an applet reacts to one event, so "undo it when things recover" needs its own trigger.

In practice a tracked floating static (`ip route ... track 10`, Chapter 15) does this natively. The applet shows the pattern, which applies to any change a static route can't express: shutting a port, changing an HSRP priority, swapping an ACL.

2 — Troubleshooting: capture evidence the moment an uplink drops

```
event manager applet UPLINK-DOWN-CAPTURE
 event syslog pattern "LINEPROTO-5-UPDOWN.*GigabitEthernet0/1, changed state to down"
 action 010 cli command "enable"
 action 020 cli command "show clock | append flash:uplink-down.txt"
 action 030 cli command "show interface GigabitEthernet0/1 | append flash:uplink-down.txt"
 action 040 cli command "show logging | append flash:uplink-down.txt"
 action 050 cli command "show ip ospf neighbor | append flash:uplink-down.txt"
 action 060 syslog priority errors msg "Gi0/1 down - diagnostics in flash:uplink-down.txt"
```

The `| append flash:...` filter writes each output to a file instead of the screen, so the evidence is waiting when someone investigates. **Why include `show clock` first?** So every captured block is timestamped. Evidence without a time is a guess about when it happened.

3 — Data collection: sample CPU every 5 minutes

```
event manager applet CPU-SAMPLER
 event timer watchdog time 300 maxrun 60
 action 010 cli command "enable"
 action 020 cli command "show clock | append flash:cpu-log.txt"
 action 030 cli command "show processes cpu sorted 5min | append flash:cpu-log.txt"
```

`watchdog time 300` re-arms every 300 seconds. **`maxrun 60`** raises the applet's time limit. The default maximum runtime is **20 seconds**, after which EEM kills the applet mid-action, which matters when a command is slow or an applet collects many outputs.

Bonus — guard a dangerous command

```
event manager applet BLOCK-RELOAD
 event cli pattern "^reload" sync no skip yes
 action 010 syslog priority critical msg "Command '$_cli_msg' blocked by EEM"
 action 020 puts "Reload is disabled by policy. Use the change process."
```

`skip yes` means the matched command is **not executed**; `$_cli_msg` holds the command that was typed. `sync no` runs the applet asynchronously (the CLI doesn't wait for it).

Running, verifying and debugging

```
R1# event manager run CPU-SAMPLER            ! only works for applets with "event none"
R1# show event manager policy registered     ! every registered applet and its trigger
No.  Class   Type    Event Type          Trap  Time Registered          Name
1    applet  user    timer watchdog      Off   Mon Oct 26 14:02:11 2026  CPU-SAMPLER
 name {CPU-SAMPLER} time 300.000 maxrun 60.000
 action 010 cli command "enable"
 ...
R1# show event manager history events        ! when applets fired, and the result
R1# debug event manager action cli           ! watch each cli action run, and its output
```

Symptom	Cause	Proof / fix
Applet never fires	The syslog regex doesn't match the real message (spacing, interface name)	Compare against `show logging` exactly; simplify the pattern
Fires, but config commands fail	Missing `enable` or `configure terminal`	`debug event manager action cli` shows `% Invalid input`
Actions run in a strange order	Labels sorted lexically (`10.0` before `2.0`)	`show event manager policy registered` lists the order
Applet stops part-way through	Hit the 20-second `maxrun`	Add `maxrun` to the event line
`cli` commands rejected	AAA command authorization	`event manager session cli username`

21.3 Orchestration Tools — Agent vs Agentless

The problem it solves: 500 devices configured by hand drift apart. Someone adds an NTP server on one, forgets the banner on another, and nobody knows which device matches the standard. Orchestration tools hold the standard in one place and make every device match it.

EEM automates **one device reacting to itself**. Orchestration (configuration management) tools automate **the fleet**: you declare what every device should look like, and the tool makes it so, repeatably and idempotently.

Idempotent means running it twice has the same effect as running it once. If NTP is already configured correctly, the tool changes nothing and reports "ok". **Why is idempotency the property that makes these tools safe?** Because you can run the whole playbook against 500 devices every night. Devices that drifted get corrected, compliant devices are untouched, and the report shows exactly what changed. A script that blindly re-sends commands every time can't tell you what was different; an idempotent tool can.

Story Time : Idempotency is a **thermostat**. You set it to 21 °C. If the room is already 21 °C, it does nothing. If it's 18 °C, it heats until it reaches 21 and then stops. Setting it to 21 a second time changes nothing. A non-idempotent script is a person told to "turn the heating up two degrees" every hour: run it repeatedly and the room keeps getting hotter.

The two architectural questions

1. Agent or agentless?

- **Agent-based:** a software **agent** is installed on every managed node. It runs continuously, talks to a central server, and applies configuration locally.
- **Agentless:** nothing is installed. The control node connects with protocols the device already speaks: **SSH**, **NETCONF**, **REST APIs**.

2. Push or pull?

- **Push:** the central node **sends** changes to devices when you run it.

- **Pull:** each agent **periodically asks** the server "what should I look like?" and converges itself.

Story Time : An **agent** is a caretaker who lives in each building. Every 30 minutes the caretaker phones head office ("what should this building look like?") and fixes whatever differs. That's **pull**. **Agentless** is a contractor who lives at head office. When you send them out, they drive to each building with the plan and do the work through the front door (SSH) that every building already has. That's **push**. Where the analogy stops: Salt has resident agents (minions) and still pushes, over a fast message bus. "Agent" and "pull" usually go together, but they're two separate questions.

How a pull cycle works (Puppet as the example):

1. Every 30 minutes, the agent on the node wakes up.
2. It runs **Facter** to collect facts about the node (OS, hostname, interfaces) and sends them to the Puppet server over HTTPS (TCP 8140).
3. The server **compiles a catalog**: the desired state for this node, built from the manifests and the facts.
4. The agent compares the catalog with what's actually on the node and changes only what differs (idempotency again).
5. It sends a report back: what changed and what failed.

Nobody has to start the run. That's the strength of pull for servers, and also why it needs an agent that can run on the device.

Why does agentless fit network devices so well? Because you usually **can't install software** on a router or switch, and even where you can, you don't want a third-party daemon on your core. Every device already has SSH and, on IOS XE, NETCONF and RESTCONF (Chapter 20). An agentless tool uses what's there. Agents suit fleets of servers you fully control; network gear favours agentless.

Property				
Architecture	**Agentless**	**Agent-based** (agent + Puppet server)	**Agent-based** (chef-client + Chef Infra Server)	**Agent-based** (minions + master); agentless via `salt-ssh`
Model	**Push**	**Pull** (agents check in, every **30 min** by default)	**Pull** (chef-client runs periodically)	**Push** (fast, over an event bus); minions can also pull
Transport to devices	SSH, NETCONF, HTTP APIs	Agent ↔ server over HTTPS	Agent ↔ server over HTTPS	ZeroMQ message bus (minions); SSH with salt-ssh
Language you write	**YAML** playbooks (Jinja2 templates)	Puppet **DSL** (Ruby-like) — manifests	**Ruby DSL** — recipes in cookbooks	**YAML** states (Jinja2); written in Python
Tool written in	Python	Ruby (server in Clojure/JRuby)	Ruby (server in Erlang)	Python

Property				
Network-device fit	**Excellent** — the de facto standard for network automation	Limited (agentless device modules exist; designed for servers)	Limited	Good (proxy minions for devices that can't run an agent)
Well-known port	**TCP 22** (SSH) or **830** (NETCONF) to each device	**TCP 8140** (agent → Puppet server)	**TCP 443** (HTTPS to Chef Infra Server)	**TCP 4505** (master publishes) / **4506** (minions return)
Who's in charge	**Control node** + **managed nodes** listed in an **inventory**	**Puppet server** (formerly "master") + **agents**	**Workstation** (`knife`) → **Chef Infra Server** → **nodes** running chef-client	**Master** + **minions** (or **proxy minions** for network gear)
Units of work	**Playbook** → **plays** → **tasks**, each calling a **module**; reusable **roles**	**Manifests** grouped into **modules**; the server compiles a **catalog** per node	**Cookbooks** containing **recipes** made of **resources**	**States** (SLS files) organized in a **top file**
How it learns about a node	**Facts** gathered at run time	**Facter**	**Ohai**	**Grains** (node facts); **pillars** hold per-node secrets/data on the master

A **DSL** (domain-specific language) is a small language built for one job. **Jinja2** is a templating language that fills `{{ variables }}` into text. An **SLS** file (SaLt State) is Salt's YAML description of desired state.

Memory hooks: Ansible is agentless, push and YAML. Puppet and Chef are agent-based, pull and Ruby. Salt is agent-based, push and YAML, with a fast message bus.

> **Exam trap:** "Which tool requires no software on the managed device?" → **Ansible**. "Which uses a pull model with agents checking in to a master?" → **Puppet** (or Chef). "Which uses Ruby-based recipes and cookbooks?" → **Chef**.

21.4 A Small Ansible Example

Why this section exists: Ansible is the orchestration tool you're most likely to meet on network gear, and seeing one complete run makes "agentless, push, idempotent" concrete. Ansible needs two files: an **inventory** (which devices) and a **playbook** (what they should look like).

Reading YAML in 30 seconds

Playbooks are written in **YAML** (YAML Ain't Markup Language), a format that holds the same data as JSON (19.8) but uses indentation instead of brackets:

YAML	Means	JSON equivalent
`name: Standardize NTP`	Key and value	`"name": "Standardize NTP"`
A line starting `-`	One item in a list	An element of `[ ]`
Indented lines under a key	Belong inside that key, like Python blocks	A nested object `{ }`
`---` on the first line	Start of a YAML document	—
`false`, `true`	Booleans	`false`, `true`

As in Python, **indentation is the structure**. A task indented one level too little silently becomes part of a different block, or makes the file invalid.

Inventory — which devices, and how to reach them (`inventory.ini`):

```
[branch_routers]
br1 ansible_host=10.10.1.1
br2 ansible_host=10.10.2.1

[branch_routers:vars]
ansible_network_os=cisco.ios.ios
ansible_connection=ansible.netcommon.network_cli
ansible_user=automation
ansible_password=Vault-Stored-Secret
```

Playbook — the desired state (`ntp.yml`):

```
---
- name: Standardize NTP and login banner on branch routers
  hosts: branch_routers
  gather_facts: false

  tasks:
    - name: Ensure both NTP servers are configured
      cisco.ios.ios_config:
        lines:
          - ntp server 10.0.0.1
          - ntp server 10.0.0.2

    - name: Ensure the login banner is set
      cisco.ios.ios_banner:
        banner: login
        text: "Authorized access only"
        state: present

    - name: Save running-config if anything changed
      cisco.ios.ios_config:
        save_when: modified
```

Run it: `ansible-playbook -i inventory.ini ntp.yml`.

Line	Meaning
`hosts: branch_routers`	Target the inventory group
`gather_facts: false`	Skip server-style fact collection, which doesn't apply to routers

Line	Meaning
`cisco.ios.ios_config`	A **module** (fully qualified collection name) that compares `lines` with the running config and sends only what's missing
`ansible_connection=...network_cli`	Connect via SSH to the CLI. `ansible.netcommon.netconf` or `httpapi` would use NETCONF or REST instead.
`save_when: modified`	`write memory` only if a change was made

On the first run, each router reports **changed**. On the second run, they report **ok**, because nothing needed doing. **That's idempotency you can see.** Passwords in real use live in **Ansible Vault** (encrypted), not plain text.

What `ansible-playbook` does, step by step:

1. It reads `inventory.ini`, finds the group `branch_routers` (br1 and br2), and their variables: SSH via `network_cli`, IOS syntax, and the user `automation`.
2. It opens an SSH session to each router from the control node. Nothing is installed on the routers.
3. **Task 1:** `ios_config` reads the running configuration and looks for each line in `lines`. Any missing line is sent in configuration mode. If both lines were already there, it sends nothing.
4. **Task 2:** `ios_banner` compares the current login banner with the text, and sets it only if it differs.
5. **Task 3:** `save_when: modified` compares running with startup, and runs `write memory` only if they differ.
6. It prints a result per task per host, then a summary called the **PLAY RECAP**.

Worked example — the two runs side by side. The first run against routers that have neither NTP server nor a banner (the recap's long lines are trimmed at the right):

```
TASK [Ensure both NTP servers are configured] ***********************
changed: [br1]
changed: [br2]
...
PLAY RECAP *********************************************************
br1                        : ok=3    changed=3    unreachable=0    failed=0   ...
br2                        : ok=3    changed=3    unreachable=0    failed=0   ...
```

The second run, a minute later, with nothing changed in between:

```
TASK [Ensure both NTP servers are configured] ***********************
ok: [br1]
ok: [br2]
...
PLAY RECAP *********************************************************
br1                        : ok=3    changed=0    unreachable=0    failed=0   ...
br2                        : ok=3    changed=0    unreachable=0    failed=0   ...
```

Read the recap: **ok** counts every task that succeeded (whether or not it changed anything), **changed** counts the ones that actually altered the device, and **unreachable** counts hosts it couldn't reach at all (SSH down or wrong credentials). `changed=0` on the second run is the proof of idempotency. Add `--check --diff` to the command for a **dry run**: Ansible reports what it would

change, and shows the lines, without touching the devices.

21.5 AI in Network Operations

The problem it solves: modern networks produce more telemetry, logs and alarms than people can read. A single failure can raise hundreds of alerts, while a slow degradation raises none. AI helps in two ways: it finds the signal in that flood, and it lets engineers ask questions and draft work in plain language.

ENCOR v1.2 renamed this domain **"Automation and Artificial Intelligence"**, and Chapter 16 covers Catalyst Center's AI-powered workflows. Here's the general picture you need. Expect describe-level questions about purpose and benefits, not machine-learning mathematics. Three terms to know: **ML** (machine learning) is software that learns patterns from data instead of following hand-written rules. **AIOps** (AI for IT operations) is ML applied to monitoring and troubleshooting. An **LLM** (large language model) is the kind of generative AI behind chat assistants.

Two kinds of AI, two different jobs

Kind	What it does	Network examples
Predictive / analytical AI (ML, AIOps)	Learns **baselines** from telemetry and flags deviations; correlates many signals to a probable root cause; forecasts trends	Dynamic baselines for client onboarding times (not fixed thresholds); anomaly detection in flows (18.2); predicting link congestion or capacity exhaustion; grouping 400 alarms into one incident
Generative AI (LLM assistants)	Understands and produces natural language and code	"Why are clients at Building 4 failing to onboard?" answered from assurance data; summarising an incident; drafting a configuration, a template, an EEM applet or an Ansible playbook from a description; explaining unfamiliar `show` output

Why do baselines beat static thresholds? A fixed rule like "alert if DHCP takes over 5 seconds" fires constantly on a busy Monday morning and stays silent when a quiet site degrades from 0.2 to 2 seconds. A learned baseline knows what is normal for this site at this hour and flags what's genuinely unusual. AIOps turns alert floods into a short list of things that are actually wrong.

Story Time : A baseline is your doctor knowing your normal. A generic chart says a resting heart rate of 50 is low, but for a marathon runner it's perfectly healthy. And a jump from 55 to 85 is worth a question, even though 85 is "within the normal range." Static thresholds are the generic chart; a learned baseline is the doctor who has seen you every year.

Worked example — correlation. A distribution switch serving 40 access switches reboots. Within a minute the monitoring system receives about 400 alarms: 40 "device unreachable", 120 "uplink down", 200 "AP disconnected", plus a burst of client-onboarding failures. Without correlation, that's 400 tickets. AIOps sees that they all started within the same few seconds and that every affected device sits below that one switch in the topology. It raises **one incident**: "distribution switch DS-2 rebooted; 400 downstream alarms are symptoms." Engineers start at the cause instead of working through the symptoms.

Guardrails — how to use AI safely

A generated configuration is a **suggestion from a very fast junior engineer who has never seen your network**. It can be fluent, confident and wrong: a non-existent command, a plausible but wrong mask, a missing `permit` that blackholes a site. Good practice:

Guardrail	Why
Human-in-the-loop for any change	A person reviews and approves before anything touches production
Validate before deploy	Syntax checks, a lab or digital-twin test, pre/post-change checks (e.g. neighbour counts, route counts)
Least-privilege access for AI tools	An assistant that only reads telemetry can't break anything; write access goes through the normal change process
Change through the existing pipeline	AI output enters the same version control, review, idempotent deployment (Ansible) and rollback path as human changes
Protect sensitive data	Configs contain secrets, keys and topology; know where prompts and data go before sharing them with a service
Audit trail	Record what was suggested, who approved it and what was deployed (the AI version of 17.4 accounting)

Worked example — the guardrails catching a real mistake. An engineer asks an assistant: "Write an EEM applet that saves the config and logs a message whenever someone leaves configuration mode." It returns a tidy applet with ten actions labelled `1.0` to `10.0`, starting with `cli command "write memory"`.

1. **Human review** catches two problems from 21.1. There's no `enable` before `write memory`, so the command fails at user EXEC. And with labels `1.0`...`10.0`, action `10.0` would run second.
2. **Validation in the lab** confirms it: `debug event manager action cli` shows `% Invalid input`.
3. The engineer fixes both problems, and the corrected applet goes through **the normal change pipeline** (version control, review, Ansible push).
4. The **audit trail** records the prompt, the AI's draft, the human fixes and the approver.

The AI saved twenty minutes of typing. The guardrails stopped two bugs from reaching production.

The pattern that works: AI proposes and explains, automation executes idempotently, and humans approve. Each does what it's best at. This is the same layering as the rest of Part I: Python and APIs (Chapters 19–20) are the hands, orchestration (21.3) is the discipline, and AI is an increasingly capable advisor on top.

Part II — ENARSI 300-410: Advanced Routing & Services

Part I got you through **ENCOR**, the core exam that every CCNP Enterprise candidate takes. Part II prepares you for **ENARSI (300-410)**, the concentration exam. It's shorter, at **90 minutes**, but narrower and deeper, and its verbs change. ENCOR mostly says describe, configure and verify. ENARSI says **troubleshoot** again and again. Of its 25 numbered topics, 18 start with that word.

That changes how you study. ENARSI rarely asks "what does this command do?" It shows you a broken network — a route missing from the table, a neighbor stuck, a packet taking the wrong path — and asks why. The chapters in this part build on Part I's foundations (OSPF in Chapter 6, EIGRP in Chapter 5, BGP in Chapter 7, VRFs in Chapter 12). They add the configuration depth and, above all, the **diagnostic reasoning** that turns "I know OSPF" into "I can find the OSPF fault in under five minutes."

> **The ENARSI weights:** Layer 3 Technologies **35%** · VPN Technologies **20%** · Infrastructure Security **20%** · Infrastructure Services **25%**. Chapters 22–28 are the Layer 3 domain, 29–30 are VPN, 31 is security and 32 is services.

Chapter 22: The Troubleshooting Method & Administrative Distance

> **Blueprint:** ENARSI 1.1 — Troubleshoot administrative distance (all routing protocols).

22.1 Why a Method Beats Instinct

Every engineer has watched a colleague "troubleshoot" by changing things at random until the problem goes away. Sometimes it works. When it does, nobody knows why it worked. When it doesn't, the network is now broken in two ways.

A **structured method** is a way to shrink the search space on purpose. Each test you run should rule out a large chunk of the possible causes, whatever its result. Good troubleshooting is less

about knowing the answer and more about asking the question that eliminates the most wrong answers.

Story Time : Think of the game Guess Who?. A bad player asks "Is it Bob?" and rules out one face. A good player asks "Does your person wear glasses?" and, whatever the answer, half the board flips down. Troubleshooting works the same way. "Is it the ACL on R7?" is a Bob question. "Can VLAN 10 reach it when VLAN 20 can't?" is a glasses question: either answer removes a huge slice of the network from suspicion.

The general loop never changes:

1. **Define the problem precisely.** "The internet is down" is not a problem statement. "Hosts in VLAN 20 at Branch 3 cannot reach 10.50.0.0/16; VLAN 10 can" is one.
2. **Gather facts.** Use `show` commands, logs, recent changes, and the scope of the outage (who is affected and who isn't).
3. **Form a hypothesis** that explains all the facts, not just some.
4. **Test it**, ideally with a command that proves or disproves it rather than a config change.
5. **Fix, then verify** that the original symptom is gone and that nothing else broke.
6. **Document** what happened, so the next person doesn't start from zero.

Why is "who is not **affected" so valuable?** Because the difference between a working host and a broken one points straight at the cause. If VLAN 10 works and VLAN 20 doesn't, everything the two VLANs share is ruled out: the uplink, the WAN, the core and the destination. What's left is the handful of things that differ — VLAN 20's SVI, its ACL, its DHCP scope, its HSRP group. One comparison just eliminated most of the network.

Worked example — the loop in action. A ticket arrives: "Branch 3 can't reach the payroll server."

1. **Define.** You test and narrow it down: hosts in VLAN 20 (10.3.20.0/24) can't reach 10.50.1.10. Hosts in VLAN 10 (10.3.10.0/24) at the same branch can. Other branches can too.
2. **Gather facts.** `show ip route 10.50.1.10` on the branch router shows a good OSPF route. A ping from a VLAN 10 host succeeds; a ping from a VLAN 20 host fails. (A ping from the router itself, sourced from the VLAN 20 SVI, succeeds: router-generated traffic never passes that SVI's inbound ACL, which is itself a clue.) The change log shows a security change on the branch router last night.
3. **Hypothesis.** Something that applies only to VLAN 20 traffic is dropping it. An inbound ACL on the VLAN 20 SVI explains every fact: the route is fine, VLAN 10 is fine, and there was a recent change.
4. **Test without changing anything.** `show ip interface vlan 20 | include access list` shows `Inbound access list is SEC-V20`. `show access-lists SEC-V20` shows a `deny ip any 10.50.0.0 0.0.255.255` line whose hit counter climbs each time a VLAN 20 host pings.
5. **Fix and verify.** Correct the ACL line, then ping again from VLAN 20 and check that VLAN 10 and the other branches still work.
6. **Document.** Record the ACL change, who made it and why it broke payroll.

Notice what you never did: you never debugged OSPF, never touched the WAN and never rebooted anything. The VLAN 10 versus VLAN 20 comparison ruled all of that out in one step.

22.2 The Six Classic Approaches

A method tells you how to think. An approach tells you where to start looking. Picking the right starting point is the difference between a five-minute fix and an hour of checking things that were never broken. Cisco's troubleshooting material names these approaches, and ENARSI scenarios quietly reward knowing when to use each one:

Approach	How it works	Best when
Top-down	Start at the application (Layer 7) and work down	The problem looks app-specific: "web works, VoIP doesn't"
Bottom-up	Start at the physical layer and work up	You suspect hardware or cabling, or have no clue at all. Thorough but slow
Divide-and-conquer	Start in the middle (usually Layer 3: can I ping it?) and go up or down depending on the result	The most common default: one ping splits the OSI stack in half
Follow-the-path	Trace the packet hop by hop from source to destination, checking each device's forwarding decision	Routing problems: the best fit for most ENARSI questions
Compare configurations	Diff a broken device against a known-good one	Sites built from a template, or a change that "only touched one thing"
Swap components	Replace a suspect part (a cable, an SFP, a whole device) and see if the problem follows it	Suspected hardware; it isolates the fault but doesn't explain it

Why is follow-the-path the natural fit for routing problems? Because each router makes its forwarding decision **independently**, using only its own tables (CCNA guide 14.8). A packet that fails somewhere fails at one specific router, the one whose table holds the wrong answer. Following the path finds that router directly, without guessing. Routing faults are local decisions with global symptoms; follow-the-path turns the global symptom back into the one local decision that caused it.

Story Time : Follow-the-path is like tracking a lost parcel. The parcel went through five sorting depots. Each depot read the label and chose the next truck on its own. You don't need to inspect the whole postal system: you check each depot's scan log in order until you find the one that put the parcel on the wrong truck. That depot is your broken router.

Exam trap: don't treat these as competitors. Real troubleshooting chains them. You divide-and-conquer with a ping, follow the path with `traceroute` to find the failing hop, then compare that hop's config with its healthy twin.

22.3 A Repeatable Workflow for "The Route Is Wrong"

Most Layer 3 tickets reduce to one of three complaints: **the route is missing**, **the route points the wrong way**, or **traffic is dropped even though the route looks right**. The same set of questions answers all three. Ask them in this order on the router where traffic goes astray:

Step	Question	Command that answers it
1	Is there a route for the destination at all? Which prefix matches (longest match)?	`show ip route 10.50.1.1`
2	Which **source** installed it (connected, static, OSPF, EIGRP, BGP)?	The route code and `[AD/metric]` in that output
3	Did another source also offer this prefix and lose?	`show ip ospf database`, `show ip eigrp topology`, `show bgp ipv4 unicast <prefix>`
4	If the expected source lost, was it on **AD**, or did that protocol never learn it?	Compare ADs; check the protocol's own table
5	Is the next hop reachable, and what does the forwarding plane actually do?	`show ip cef 10.50.1.1 detail`, `show adjacency`
6	Is something overriding the table — PBR, an ACL, a VRF?	`show ip policy`, `show ip interface`, `show vrf`

`show ip route <address>` (with an address, not a prefix) is the single most useful routing command. It runs the router's real longest-match lookup and tells you everything about the winning route:

```
R2# show ip route 10.50.1.1
Routing entry for 10.50.0.0/16
  Known via "ospf 1", distance 110, metric 20, type extern 2, forward metric 2
  Last update from 10.0.23.3 on GigabitEthernet0/1, 00:12:41 ago
  Routing Descriptor Blocks:
  * 10.0.23.3, from 3.3.3.3, 00:12:41 ago, via GigabitEthernet0/1
      Route metric is 20, traffic share count is 1
```

Read it like a detective: this route came from OSPF (`Known via "ospf 1"`), it's an **E2 external** (so someone redistributed it — Chapter 24), its originator was router 3.3.3.3 (`from 3.3.3.3`), and the next hop is 10.0.23.3. If you expected this prefix to come from EIGRP, you now have your question: why did OSPF's 110 beat EIGRP? The answer is almost always administrative distance (22.5).

Why check the protocol's own table in step 3, not just the routing table? Because the routing table shows only **winners**. A prefix can sit perfectly healthy in the EIGRP topology table, or in the BGP table, and simply lose the AD contest. If you look only at `show ip route`, it looks as if EIGRP "doesn't know" the route, and you'll waste an hour debugging EIGRP neighbors that are fine. The RIB is the result of the election; the protocol tables are the ballots.

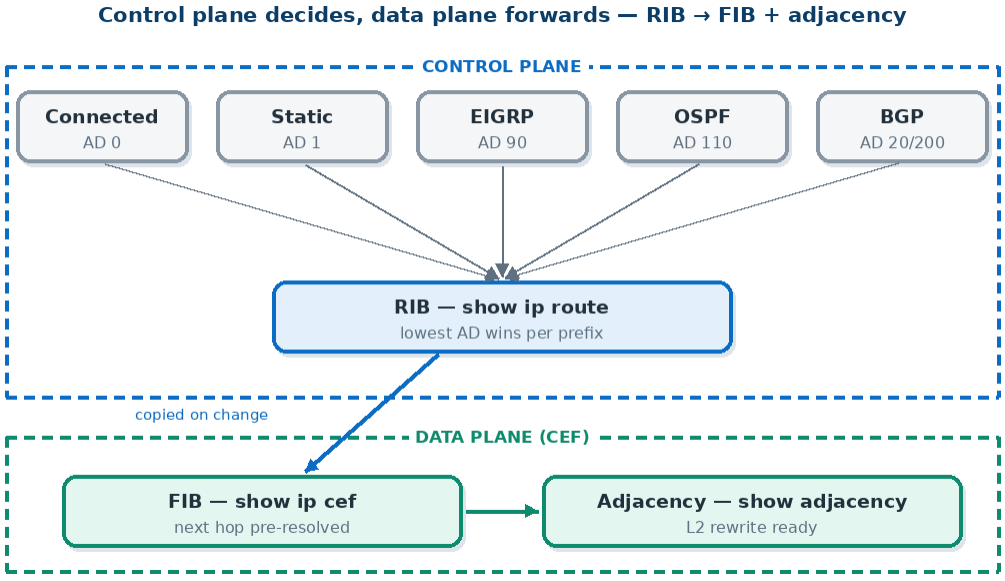
Worked example — a missing EIGRP route that isn't missing. The ticket says "R2 isn't using EIGRP for 10.50.0.0/16 any more." Walk the table:

1. `show ip route 10.50.1.1` → `Known via "ospf 1", distance 110`. So a route exists (step 1), and OSPF installed it (step 2).
2. Step 3: does EIGRP know it too?

```
R2# show ip eigrp topology 10.50.0.0/16
EIGRP-IPv4 Topology Entry for AS(100)/ID(2.2.2.2) for 10.50.0.0/16
  State is Passive, Query origin flag is 1, 1 Successor(s), FD is 3072
  Descriptor Blocks:
  10.0.12.1 (GigabitEthernet0/0), from 10.0.12.1, Send flag is 0x0
      ...
      External data:
        Originating router is 4.4.4.4
        AS number of route is 1
        External protocol is OSPF, external metric is 20
```

3. Step 4: EIGRP does know the prefix, so the neighbors are fine. But look at `External data`: this is an **EIGRP external** route (AD 170), redistributed from OSPF by router 4.4.4.4. OSPF's 110 beats 170, so the EIGRP copy loses the election and stays out of the RIB.
4. Conclusion: nothing is broken in EIGRP. Someone added redistribution at 4.4.4.4, and the prefix now reaches R2 by two protocols. The real question becomes "which path do we want?", answered with AD tuning (22.6) or by changing the redistribution (Chapter 24).

22.4 Control Plane vs Data Plane — the RIB and the FIB



The routing table is not what actually forwards packets on a modern Cisco router. There are two layers:

Structure	Built by	Contains	Used for
RIB — routing table (`show ip route`)	The control plane: every routing source competes, best AD wins	Best route per prefix, possibly a recursive next hop	Deciding **what** the best path is

Structure	Built by	Contains	Used for
FIB — CEF table (`show ip cef`)	CEF, copied from the RIB with recursion already resolved	Prefix → resolved next hop + outgoing interface	Forwarding every packet at speed
Adjacency table (`show adjacency`)	CEF, from ARP / neighbor discovery	Next hop → pre-built Layer 2 header (rewrite string)	Building the outgoing frame

Why split them at all — why not forward straight from the routing table? Because the routing table is built for deciding, not forwarding. A BGP route might say "next hop 192.0.2.1", which is three recursive lookups away from an actual interface. Doing that recursion for **every packet** would be painfully slow. CEF does it **once**, when the route changes, and stores the answer. It even pre-builds the Ethernet header in the adjacency table, so forwarding becomes a single lookup plus a header copy. The RIB thinks; the FIB just acts, quickly, on what the RIB already decided.

Story Time : The RIB is a travel planner who studies maps and decides "to get to Paris, go via Lyon." The FIB is a laminated card taped by the door: "Paris → Gate 4." The planner did the thinking once; the card lets anyone act in a second. The adjacency table is the pre-printed luggage tag already filled in for Gate 4. And if the card is wrong or Gate 4 is closed (ARP failed), the planner's map can still look perfect while every traveller gets stuck at the door. That is exactly the "route looks fine, traffic drops" ticket.

How a packet is forwarded with CEF, step by step. A packet for 10.50.1.1 arrives on R2:

1. R2 looks up 10.50.1.1 in the **FIB**. Longest match is 10.50.0.0/16 → next hop 10.0.23.3 out Gi0/1. Recursion was already resolved when the route was installed, so this is one lookup.
2. The FIB entry points to the **adjacency** for 10.0.23.3 on Gi0/1. That adjacency holds the ready-made 14-byte Ethernet header (destination MAC, source MAC, type 0x0800).
3. R2 decrements the TTL, rewrites the IP checksum, stamps the pre-built header on the front and queues the frame on Gi0/1.
4. If step 2 finds no complete adjacency (no ARP entry), the packet cannot be sent. On a connected subnet, a **glean** entry makes R2 send an ARP request and drop the triggering packet. If the next-hop address never answers ARP, the adjacency stays incomplete and traffic to it is dropped.

The RIB was never consulted for the packet itself. That is why `show ip route` alone can't prove the forwarding plane is healthy.

This matters in troubleshooting because the two can **disagree** for short periods, or when something has gone wrong. CEF also has forwarding states the RIB never shows:

```
R2# show ip cef 10.50.1.1 detail
10.50.0.0/16, epoch 2
  nexthop 10.0.23.3 GigabitEthernet0/1

R2# show adjacency GigabitEthernet0/1 detail
Protocol Interface                 Address
IP       GigabitEthernet0/1        10.0.23.3(10)
                                   0 packets, 0 bytes
                                   epoch 0
                                   sourced in sev-epoch 2
                                   Encap length 14
                                   5254001A2B3C5254004D5E6F0800
                                   ARP
```

That long hex string is the **rewrite**: destination MAC, source MAC and EtherType 0x0800, ready to stamp onto every packet. If ARP for the next hop has failed, you'll see the adjacency as **incomplete**, and packets matching that route are dropped — even though `show ip route` looks perfect.

CEF entry you see	What it means
`nexthop 10.0.23.3 Gi0/1`	Normal: forward out Gi0/1 using that adjacency
`attached`	Directly connected subnet: ARP for the destination itself
`receive`	Addressed to the router itself: punt to the CPU (control plane)
`drop` / `discard`	Deliberately dropped, e.g. a route to Null0 (a summary discard route, 24.6)
`glean`	Connected subnet, but no ARP entry yet for the host: the router ARPs first
`punt`	CEF can't switch it; hand it to process switching (slow path)

Two more CEF tools earn their place in an ENARSI troubleshooting kit:

```
R2# show ip cef exact-route 10.1.1.10 10.50.1.1   ! which ECMP link will THIS flow use?
R2# show ip cef 10.50.0.0 255.255.0.0 internal    ! deep detail incl. load-share buckets
```

`exact-route` answers the question that confuses people with equal-cost paths: "traceroute went one way, but the user's traffic goes another." CEF load-shares **per flow** by hashing source and destination (like EtherChannel, CCNA guide 10.4). Different flows can legitimately take different paths.

22.5 Administrative Distance — How the Router Chooses Between Sources

The problem AD solves: a router can run several routing protocols at once, and each one speaks a different "metric language". OSPF says cost 20. EIGRP says 3072. RIP says 2 hops. Those numbers can't be compared with each other, any more than 20 euros can be compared with 3072 yen without an exchange rate. So the router needs a separate rule to decide which source to believe. That rule is **administrative distance (AD)**: a trust score per source, where lower means more trusted.

You met AD at CCNA (CCNA guide 14.4): when two **different sources** offer the same prefix, the **lower AD wins**. Metric only compares routes within one protocol. ENARSI expects you to know the complete table and to recognize the problems it causes.

Story Time : You ask for directions in a strange city. Your phone's live map, a local shopkeeper, a tourist and a three-year-old leaflet all give answers. You don't compare how many streets each route has. You first decide whom you trust most and follow that person. AD is that ranking of sources. Only when two answers come from the same source (two routes from your phone's map)

do you compare the routes themselves. That second comparison is the metric.

How the router chooses, step by step. Suppose R1 hears about 10.20.0.0/16 from three sources at once:

Source	AD	Metric
OSPF	110	cost 30
EIGRP internal	90	28,160
RIP	120	2 hops

1. **Same prefix length?** Yes, all three are /16. (AD only compares routes for the identical prefix. A /24 inside it is a different route and is installed separately; longest match then decides at forwarding time.)
2. **Compare AD across sources.** 90 < 110 < 120, so EIGRP wins. The metrics 30, 28,160 and 2 are never compared with each other.
3. **Install the winner.** `D 10.20.0.0/16 [90/28160]` goes into the RIB, and CEF copies it into the FIB.
4. **Keep the losers in their own tables.** OSPF still holds its LSAs and RIP its route. If EIGRP withdraws the prefix, the router reruns the contest and OSPF (110) is installed next.

Worked example — longest match beats AD. R1 has `D 10.20.0.0/16 [90/28160]` and also learns `O 10.20.5.0/24 [110/40]` from OSPF. Which one carries a packet to 10.20.5.9? **The OSPF /24**, even though OSPF's AD is worse. These are two different prefixes, so both sit in the RIB, and forwarding always picks the longest match. AD never even got a vote. A packet to 10.20.9.9 doesn't match the /24, so it uses the EIGRP /16. Mixing up these two rules is one of the most common ENARSI mistakes.

Route source	Default AD	Note
Connected interface	**0**	Includes the local /32 (`L`) routes
Static route	**1**	Static pointing only to an interface is also 1 (old docs said 0)
EIGRP summary route	**5**	The local Null0 discard route a summary creates (24.6)
External BGP (eBGP)	**20**	Learned from a different AS
EIGRP internal	**90**	Routes native to the EIGRP AS
IGRP	**100**	Obsolete; still on the exam list
OSPF	**110**	All types: intra, inter, E1, E2, N1, N2
IS-IS	**115**	
RIP	**120**	
ODR (On-Demand Routing)	**160**	Legacy CDP-based hub-and-spoke

Route source	Default AD	Note
EIGRP external	**170**	Redistributed into EIGRP
Internal BGP (iBGP)	**200**	Learned from the same AS
BGP local (aggregates, `network` backdoor)	**200**	Locally generated BGP routes
NHRP	**250**	DMVPN shortcut routes (Chapter 30)
DHCP-learned default route	**254**	Default route installed by `ip address dhcp`; also OSPF's default for its external-summary discard route
Unknown / unreachable	**255**	**Never** installed

Why does eBGP get 20 but iBGP 200 — the same protocol with wildly different trust? Because the two are learned in completely different situations. An eBGP route comes from a directly attached neighbor in another AS, usually your ISP, and is often the only way to reach that prefix. It should beat your IGPs. An iBGP route was carried across your own AS, and your **IGP already knows how to reach anything inside your AS** more precisely. If iBGP routes beat the IGP, a BGP route could hijack your internal paths and create loops, since iBGP next hops themselves are resolved through the IGP. So iBGP is pushed below every IGP. eBGP is "news from outside, believe it"; iBGP is "rumor about our own house, let the IGP decide."

Why is EIGRP external (170) so much worse than EIGRP internal (90)? Because an external route was **redistributed** from somewhere else, and redistribution throws away the original protocol's loop information (23.6). EIGRP deliberately trusts those routes less, so if the same prefix is also learned natively by some protocol, the native path wins. This one design decision prevents many redistribution loops. Lower trust for second-hand information is a built-in safety margin.

Floating Static Routes

Why you need one: you want a backup path (an LTE modem, a second ISP) that carries traffic only when the primary fails, with no script or human involved. A static route with a **raised AD** sits idle as long as a better source offers the prefix, and "floats" into the RIB the moment that source withdraws it:

```
R1(config)# ip route 0.0.0.0 0.0.0.0 10.0.12.2        ! primary default, AD 1
R1(config)# ip route 0.0.0.0 0.0.0.0 172.16.0.1 250   ! backup via LTE, AD 250
```

How it floats, step by step:

1. Both statements are in the config, but the RIB can hold only one source per prefix. AD 1 beats AD 250, so `S* 0.0.0.0/0 [1/0] via 10.0.12.2` is installed. The AD 250 route waits.
2. The primary interface goes down. A static route is removed when its next hop is no longer resolvable (10.0.12.2 lived on the failed interface).
3. The router reruns the contest for 0.0.0.0/0. Only the AD 250 route remains, so `S* 0.0.0.0/0 [250/0] via 172.16.0.1` is installed.

4. The primary comes back, the AD 1 route returns and wins again, and the backup goes back to waiting.

One limit to know: step 2 only happens if the router can see the failure. If the primary next hop sits behind a switch, the interface can stay up while the ISP is dead, and the primary static never leaves the RIB. The fix is to tie the primary to an IP SLA probe with object tracking (Chapter 32).

When Two Sources Tie on AD

AD is designed to be unique per protocol, but `distance` lets you create a tie, for example two OSPF processes (both 110) offering the same prefix, or EIGRP raised to 110 to match OSPF. Know three facts:

- **There is no load sharing across protocols or processes.** Only **one** source is installed for a prefix. ECMP only happens between paths inside one routing process.
- **Which source wins a tie is implementation behaviour, not a rule you can design around.** IOS typically keeps the route that is already installed, and some releases fall back to the protocols' default ADs to break the tie. The result can therefore depend on which protocol converged first. That makes it non-deterministic after a reload. Never leave two protocols at the same AD on purpose.
- **Two static routes with the same AD to the same prefix** do **load-share.** They are the same source, so both go into the RIB and CEF splits flows across them. That's why a floating static must use a different AD from the primary.

Exam trap: a floating static must have an AD **higher** than the route it backs up. A backup for an OSPF-learned route needs an AD above 110 — set it at 111 or more. Leave the backup at the default AD of 1 and it beats OSPF immediately, so your "backup" link carries all the traffic.

22.6 Changing AD — the `distance` Command, Protocol by Protocol

Why change AD at all? The defaults suit a network that runs one protocol cleanly. Real networks merge companies, run two protocols during a migration, or learn the same prefix from a partner over BGP and over a private link. In those cases the default trust ranking can pick the wrong path, and changing AD is how you tell the router "in this case, trust that source more (or less)."

Each protocol tunes AD differently, and the syntax is a favorite exam detail:

```
! OSPF — per route type
R1(config)# router ospf 1
R1(config-router)# distance ospf intra-area 110 inter-area 110 external 175

! EIGRP — internal and external (classic mode)
R1(config)# router eigrp 100
R1(config-router)# distance eigrp 90 175

! EIGRP named mode — same command, under the topology
R1(config)# router eigrp CAMPUS
R1(config-router)# address-family ipv4 unicast autonomous-system 100
R1(config-router-af)# topology base
R1(config-router-af-topology)# distance eigrp 90 175

! BGP — external, internal, local
R1(config)# router bgp 65001
R1(config-router)# address-family ipv4 unicast
R1(config-router-af)# distance bgp 20 200 200

! RIP, or any protocol: one AD for routes from a given source
```

There is also a generic, **per-source** form that changes AD only for routes learned from particular neighbors, optionally only for prefixes matching an ACL:

```
R1(config)# access-list 10 permit 10.50.0.0 0.0.255.255
R1(config)# router ospf 1
R1(config-router)# distance 200 3.3.3.3 0.0.0.0 10
!               AD^   ^source  ^wildcard  ^ACL: only these prefixes
```

For EIGRP the per-source form works the same way (`distance 95 10.0.12.2 0.0.0.0 10` under `router eigrp 100`), but it is documented to affect **internal** EIGRP routes. External EIGRP routes keep the distance set by `distance eigrp <internal> <external>`. If a per-source change "does nothing" to a D EX route, that is why. Confirm the result with `show ip route <prefix>` rather than trusting the config.

Exam trap: for OSPF, the "source" in that command is the **Router ID of the router that originated the LSA** — not the neighbor's interface address. For EIGRP, RIP and BGP it's the **neighbor's IP address**. Getting this wrong means the command matches nothing and silently does nothing.

Why would you ever change AD instead of fixing the metric? Because metrics only compete inside one protocol. If OSPF and EIGRP both offer a prefix, no OSPF cost and no EIGRP delay will change the outcome. AD is the only lever that works between protocols. Metric picks the best road on one map; AD picks which map to trust.

Worked example — the per-source distance, step by step. R1 runs OSPF and EIGRP. It learns 10.50.0.0/16 from OSPF, originated by router 3.3.3.3, and also from EIGRP. You want R1 to prefer EIGRP for this one prefix only, and leave every other OSPF route alone. Using the config above (`distance 200 3.3.3.3 0.0.0.0 10`, ACL 10 permitting 10.50.0.0/16):

1. An LSA for 10.50.0.0/16 arrives, originated by RID 3.3.3.3. The source matches `3.3.3.3 0.0.0.0` (wildcard 0.0.0.0 means "exactly this RID").
2. The prefix is checked against ACL 10. It is permitted, so the route gets AD **200**.
3. The RIB contest: EIGRP internal 90 (or external 170) beats OSPF 200. EIGRP's route is installed.
4. A different prefix from 3.3.3.3, say 10.60.0.0/16, fails ACL 10, so it keeps AD 110. Routes from other originators are untouched.

Before: `O 10.50.0.0/16 [110/3] via 10.0.23.3`. After: `D 10.50.0.0/16 [90/...] via 10.0.12.1` (or `D EX ... [170/...]` if the EIGRP copy is external). The change is surgical: one prefix, one originator.

The BGP Backdoor

A special AD case: say you learn a partner's network over eBGP (AD 20), but you also have a private back-to-back link to that partner running OSPF (AD 110). eBGP wins and your traffic crosses the internet instead of the private link. The `backdoor` keyword fixes this:

```
R1(config-router-af)# network 172.30.0.0 mask 255.255.0.0 backdoor
```

This tells BGP: "treat my eBGP-learned copy of this prefix as AD 200 (local)." The IGP's 110 now wins. The prefix is **not** advertised by this `network` statement — `backdoor` only changes the distance.

Worked example. Before the change, R1 shows `B 172.30.0.0/16 [20/0] via 203.0.113.1` (the ISP) and the OSPF copy over the private link loses. After `network 172.30.0.0 mask 255.255.0.0 backdoor`, the eBGP copy is re-ranked at AD 200, so R1 installs `O 172.30.0.0/16 [110/2] via 10.99.0.2` and traffic uses the private link. If the private link fails, the OSPF route disappears and the BGP copy (now AD 200) is installed instead: the internet path becomes the automatic backup. Note that the prefix in the `backdoor` statement must match the eBGP-learned prefix exactly.

22.7 When AD Causes the Problem

AD is a blunt rule: it doesn't care about path quality, only about who said it. Three classic ENARSI scenarios come from that:

1. Suboptimal routing after redistribution. Router R3 redistributes EIGRP into OSPF. Another router, R4, runs **both** protocols and learns a distant EIGRP-native prefix two ways: as **EIGRP internal (90)** over a slow path, and as **OSPF E2 (110)** over a fast path. EIGRP wins on AD, so R4 takes the slow path. Nothing is "broken", but users complain. The fix is to lower OSPF's AD for exactly those prefixes, or change where redistribution happens. (A trickier case — EIGRP external routes at 170 losing to OSPF's 110 — is in 24.4.)

2. A backup that isn't. A floating static at AD 5 meant to back up EIGRP (90) is always preferred. The "backup" link carries everything — until somebody notices the bill.

3. Routing loops at two-point mutual redistribution. A prefix travels out of a domain at one boundary router and back into it at another, now wearing a better AD than the original. Routers start preferring the looped-back copy. This is the most important AD problem on the exam, and it gets its own section with the fix (24.4).

Symptom	Likely AD cause	Command that proves it
Traffic takes a slow path although a faster one exists	Faster path learned by a higher-AD protocol	`show ip route <dest>` — check Known via and distance
Backup link carries traffic while primary is up	Floating static AD too low	`show ip route static`, `show run | include ip route`
Route flaps between two next hops every few seconds	Mutual redistribution feedback	`show ip route <dest>` repeatedly; watch Known via change
Expected protocol's route missing from RIB	Lost on AD to another source	`show ip eigrp topology <prefix>` / `show ip ospf database` — present but not installed
`distance` command has no effect	Wrong source (OSPF RID vs neighbor IP) or ACL mismatch	`show ip protocols` shows the distance list; verify the ACL
Partner prefix goes via internet, not private link	eBGP (20) beating the IGP	`show ip route <prefix>` shows `bgp`; fix with `backdoor`

Verify what you set: `show ip protocols` lists every distance a protocol is using, including per-source entries:

```
R1# show ip protocols | section ospf
Routing Protocol is "ospf 1"
  ...
  Routing Information Sources:
    Gateway         Distance      Last Update
    3.3.3.3              200      00:03:12
    2.2.2.2              110      00:03:12
```

Read it from the bottom up. The **Address / Wild mask / Distance / List** block is the per-source rule: LSAs originated by RID 3.3.3.3, for prefixes permitted by ACL 10, get AD 200. The **Routing Information Sources** table shows it taking effect, with 3.3.3.3 at 200. If the block is there but the source still shows 110, the ACL doesn't match the prefixes you think it does.

Worked ticket — "the backup link is always busy."

- **Symptom:** the LTE bill is huge, although the MPLS primary is up. Users at the branch say nothing is wrong.
- **Command:** `show ip route 0.0.0.0`

```
R1# show ip route 0.0.0.0
Routing entry for 0.0.0.0/0, supernet
  Known via "static", distance 1, metric 0, candidate default path
  Routing Descriptor Blocks:
  * 172.16.0.1
      Route metric is 0, traffic share count is 1
```

- **Reading it:** the default route is static, AD 1, via 172.16.0.1, the LTE router. The primary default was supposed to come from OSPF (AD 110), learned over MPLS.
- **Second command:** `show running-config | include ip route 0.0.0.0` → `ip route 0.0.0.0 0.0.0.0 172.16.0.1`. No AD was given, so it defaulted to 1 and beats OSPF's 110.
- **Cause:** a floating static configured without its floating AD.
- **Fix:** `no ip route 0.0.0.0 0.0.0.0 172.16.0.1`, then `ip route 0.0.0.0 0.0.0.0 172.16.0.1 250`.
- **Verify:** `show ip route 0.0.0.0` now shows `Known via "ospf 1", distance 110` and an `O*E2` default. Shut the MPLS interface in a maintenance window and confirm the static at `[250/0]` takes over.

Chapter 23: Route Maps, Prefix Lists & Loop Prevention

> **Blueprint:** ENARSI 1.2 — Troubleshoot route map for any routing protocol (attributes, tagging, filtering); ENARSI 1.3 — Troubleshoot loop prevention mechanisms (filtering, tagging, split horizon, route poisoning).

23.1 Why Policy Tools Exist

Out of the box, a routing protocol advertises everything it knows, to everyone it can, and prefers paths by its own metric. Real networks need to say things like:

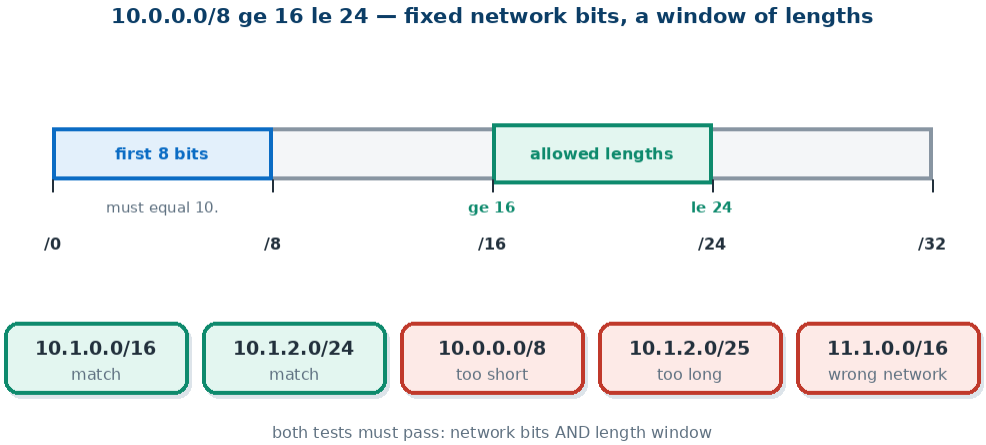
- "Advertise our customer subnets to the ISP, but **never** our infrastructure /30s."
- "Redistribute EIGRP into OSPF, but mark every route so it can't come back later."
- "Send web traffic from the guest VLAN out the cheap link, whatever the routing table says."

Saying that takes two kinds of tool. **Matching tools** — prefix lists and ACLs — pick out which routes or packets you mean. **Policy tools** — route maps — say what to do with them: permit, deny, or change an attribute. A prefix list is the noun; a route map is the sentence.

Story Time : Picture a post-room clerk sorting incoming parcels. The clerk has a stack of **description cards** ("anything from Germany", "anything heavier than 20 kg") — those are prefix lists and ACLs: they only describe. The clerk also has a **rulebook**: "Rule 10: if it matches the Germany card, put it in the bin. Rule 20: if it matches the heavy card, stick a red label on it and send it on. Rule 30: send everything else on." That rulebook is the route map. The cards never decide anything by themselves; the rulebook uses them to decide.

Term	Plain meaning
Prefix list	A list that matches routes by network bits and prefix length
ACL (access control list)	A list that matches packets (or, in older configs, routes) by address and wildcard
Route map	An ordered set of if/then clauses that uses match lists and can change route attributes
Attribute	A property carried with a route: metric, tag, next hop, or BGP values such as local preference
Tag	A number stuck onto a route as a label for your own policies (23.5)

23.2 Prefix Lists — Matching Routes Precisely



The problem it solves: you need to say "these routes, and only these" — for example, "our /24 customer blocks, but not the /30 links between routers." A route is a network address plus a length, so the matching tool must check both. That's exactly what a prefix list does.

A prefix list entry has two parts, and **both** must match for a route to match:

1. **The network bits.** The first N bits of the route must equal the first N bits of the entry, where N is the entry's prefix length.

2. **The prefix length.** The route's length must fall in the allowed range. With no `ge`/`le`, the range is exactly N. `ge` sets a minimum length, `le` sets a maximum.

```
ip prefix-list NAME [seq n] {permit | deny} network/len [ge min] [le max]
```

The rule IOS enforces is **len < ge ≤ le ≤ 32**. If you give only `ge`, the maximum becomes 32. If you give only `le`, the minimum stays len.

Story Time : Think of a prefix-list entry as a hotel's room-key rule: "Building 10 (the network bits), floors 24 to 26 only (the `ge`/`le` range)." A guest must be in the right building and on an allowed floor. Being in Building 10 on floor 30 fails, and so does being on floor 25 of Building 11. `ge` is the lowest allowed floor, `le` the highest; give neither and only the floor matching the entry's own length is allowed.

Entry	Matches	Doesn't match
`10.0.0.0/8`	10.0.0.0/8 **only**	10.1.0.0/16 (wrong length)
`10.0.0.0/8 le 24`	Anything inside 10/8 with length **8–24**: 10.0.0.0/8, 10.1.0.0/16, 10.1.2.0/24	10.1.2.0/25, 10.1.2.1/32
`10.0.0.0/8 ge 24`	Anything inside 10/8 with length **24–32**	10.1.0.0/16
`10.0.0.0/8 ge 24 le 24`	Only the **/24s** inside 10/8	10.1.2.0/23, 10.1.2.0/25
`172.16.0.0/16 ge 20 le 22`	172.16.32.0/20, 172.16.4.0/22	172.16.0.0/16, 172.17.0.0/20 (wrong network bits)
`0.0.0.0/0`	The **default route only**	Everything else
`0.0.0.0/0 le 32`	**Every** IPv4 route ("permit any")	—
`0.0.0.0/0 ge 32`	Every **host route** (/32)	Anything shorter
`0.0.0.0/0 ge 25`	Every route **longer than /24**	/24 and shorter

Try It — Does It Match?

The entry is `ip prefix-list P permit 192.168.0.0/16 ge 24 le 26`. For each route: do the first 16 bits equal 192.168? Is the length between 24 and 26?

Route	First 16 bits = 192.168?	Length 24–26?	Result
192.168.10.0/24	Yes	24 — yes	**Match**
192.168.10.128/25	Yes	25 — yes	**Match**
192.168.0.0/16	Yes	16 — no	No match
192.168.10.4/30	Yes	30 — no	No match
192.169.1.0/24	No (192.169)	—	No match

Why do prefix lists need a length range at all — isn't the network enough? Because in routing, **10.0.0.0/8 and 10.0.0.0/24 are different routes**, even though they start at the same

address. A summary and one of its components share network bits and differ only in length. An ACL can't easily express "the /8 but not the /24"; a prefix list does it in one line. The length is part of a route's identity, so a route-matching tool has to match it.

Sequence Numbers, Implicit Deny, and Order

Entries are checked in **sequence-number order** (default steps of 5), and the **first match wins**. Anything that matches no entry is **denied** by an invisible final entry. Sequence numbers let you insert a line later without rewriting the list:

```
ip prefix-list TO-ISP seq 5  deny   10.0.0.0/8 le 32        ! no internal space
ip prefix-list TO-ISP seq 10 permit 203.0.113.0/24          ! our public block
ip prefix-list TO-ISP seq 15 permit 198.51.100.0/24 le 25   ! customer block + halves
```

```
R1# show ip prefix-list detail TO-ISP
ip prefix-list TO-ISP:
   count: 3, range entries: 2, sequences: 5 - 15, refcount: 2
   seq 5 deny 10.0.0.0/8 le 32 (hit count: 14, refcount: 1)
   seq 10 permit 203.0.113.0/24 (hit count: 2, refcount: 1)
   seq 15 permit 198.51.100.0/24 le 25 (hit count: 0, refcount: 1)
```

Hit counts are your best troubleshooting friend. An entry with hit count 0 that should be matching is either written wrong or never consulted.

How the list is walked, step by step. Five routes are offered to `TO-ISP`:

Route	seq 5:	seq 10:	seq 15:	Result
10.1.1.0/24	**match** (inside 10/8, length 24 ≤ 32)	not checked	not checked	**Denied** by seq 5
203.0.113.0/24	no (not 10.x)	**match** (exact /24)	not checked	**Permitted** by seq 10
203.0.113.128/25	no	no (length 25, entry allows only 24)	no (wrong network bits)	**Denied** by implicit deny
198.51.100.128/25	no	no	**match** (inside 198.51.100/24, length 25 is within 24–25)	**Permitted** by seq 15
192.0.2.0/24	no	no	no	**Denied** by implicit deny

The router stops at the first match, so 10.1.1.0/24 never reaches seq 10. And 203.0.113.128/25 is a sneaky one: it sits inside the permitted /24, but seq 10 has no `le`, so only the exact /24 matches.

IPv6 works identically with `ipv6 prefix-list`, with lengths up to 128.

Exam trap: the silent rewrite. Type `ip prefix-list P permit 10.1.1.0/16` and IOS doesn't reject it. It **zeroes the host bits** and stores `10.1.0.0/16`. Because the entry was "fixed" for you, it may

match far more (or other) routes than you meant. Always check `show run | include prefix-list` or `show ip prefix-list` after typing one, and read what IOS actually stored.

23.3 ACLs for Route Matching — and Why Prefix Lists Usually Win

Why this section exists: ACLs were matching routes long before prefix lists were invented, so you will still meet them in older configs and exam exhibits. You need to read them correctly, and know why you shouldn't write new ones that way.

Older configs (and some exam questions) match routes with ACLs. Here an ACL tests the **route**, not a packet, so its fields mean something different:

ACL type	What it checks when matching routes
Standard	Only the **network address**, against the wildcard. It **cannot see the prefix length**: `permit 10.1.0.0 0.0.255.255` matches 10.1.0.0/16, 10.1.5.0/24 and 10.1.5.4/30
Extended — in BGP (route maps, `neighbor distribute-list`)	The "source" field matches the **network address**; the "destination" field matches the **subnet mask**
Extended — in an IGP distribute list (EIGRP, RIP)	The "source" field matches the **update source**, the neighbour that advertised the route; the "destination" field matches the **network address**

The two extended-ACL meanings are a favourite trick. In an EIGRP `distribute-list 110 in`, `access-list 110 deny ip host 10.0.12.2 10.50.0.0 0.0.255.255` means "ignore 10.50.x.x **when heard from 10.0.12.2**", which is per-neighbour filtering by prefix. The prefix-list form of the same idea is `distribute-list prefix NETS gateway NEIGHBORS in` (below). Behaviour for extended ACLs in other IGP contexts, such as redistribution route maps, is less consistent across releases. That's one more reason to use prefix lists for routes.

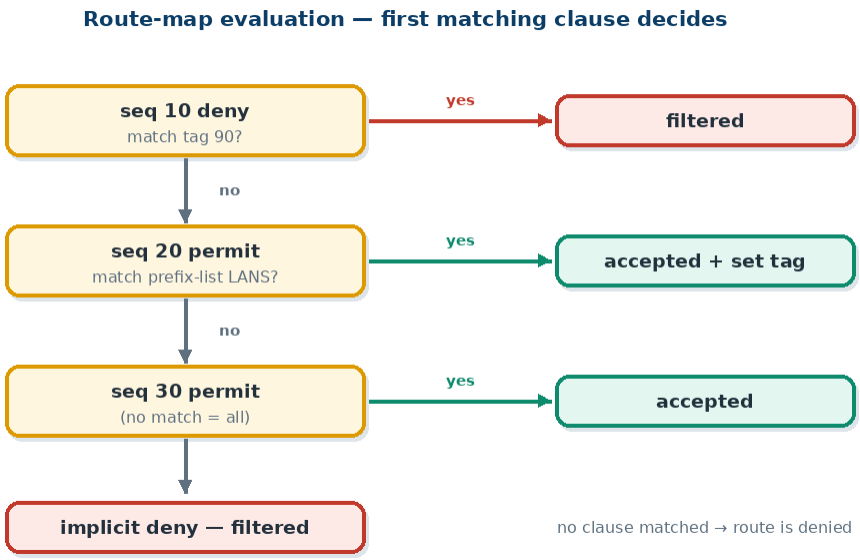
So to match exactly the /24s inside 10.1.0.0/16 with an extended ACL in **BGP**:

```
access-list 101 permit ip 10.1.0.0 0.0.255.0 host 255.255.255.0
!                         ^network: 10.1.x.0   ^mask: exactly /24
```

That works, but compare it with `ip prefix-list P permit 10.1.0.0/16 ge 24 le 24`. Prefix lists are clearer, they're processed more efficiently, they can be edited by sequence number, and they show hit counts. Use ACLs to match packets and prefix lists to match routes — the one big exception is PBR (Chapter 25), which matches **packets** and so uses ACLs.

Exam trap: the same extended ACL means three different things depending on where it's referenced. In **BGP** it's network/mask. In an **IGP distribute list** it's neighbour/network. In **PBR** it's an ordinary packet filter with source and destination addresses. Before reading an extended ACL, find out who is calling it.

23.4 Route Maps — If/Then Logic for Routing



The problem it solves: a prefix list can only say yes or no. Real policy needs more: "accept these routes and mark them", "drop those", "make this path look worse". A route map combines matching with actions, in a fixed order, so one object can express a whole policy.

A route map is an ordered list of **clauses** (sequence numbers). Each clause has an action (`permit` or `deny`), optional `match` statements (the if), and optional `set` statements (the then):

```
route-map RM-OSPF-TO-EIGRP deny 10
 match ip address prefix-list INFRA-P2P        ! if it's an infrastructure /30...
!                                               ...drop it (deny), stop here
route-map RM-OSPF-TO-EIGRP permit 20
 match ip address prefix-list BRANCH-LANS      ! if it's a branch LAN...
 set tag 110                                   ! ...mark it, and accept it
route-map RM-OSPF-TO-EIGRP permit 30           ! no match = matches everything
 set tag 111                                   ! accept the rest, marked differently
```

The evaluation rules, every one of which shows up in troubleshooting questions:

Rule	Detail
Top to bottom, first match wins	Clauses are tried in sequence order; once one matches, its action is applied and processing **stops** (unless `continue`, below)
`permit` / `deny` mean different things by context	In redistribution and BGP filtering: permit = accept the route, deny = filter it. In PBR: permit = policy-route the packet, deny = **route normally**
Implicit deny at the end	A route that matches no clause is denied: not redistributed, not advertised, not accepted
A clause with no `match` matches everything	That's how you write a final "permit the rest" clause
Different match types in one clause = AND	`match ip address prefix-list X` + `match tag 5` → both must be true

Rule	Detail
Several values in one match = OR	`match ip address prefix-list A B` → in A **or** in B. Two `match ip address` lines in one clause merge into one line, so they're OR too
`set` only runs on a permit match	`set` statements in a deny clause do nothing

Worked example — four routes through `RM-OSPF-TO-EIGRP`. Assume `INFRA-P2P` permits `10.0.0.0/8 ge 30 le 30` (all /30 links) and `BRANCH-LANS` permits `10.100.0.0/16 ge 24 le 24` (branch /24s). R1 runs `redistribute ospf 1 route-map RM-OSPF-TO-EIGRP` under EIGRP, with `default-metric 100000 100 255 1 1500` as the seed metric. Four OSPF routes arrive:

1. **10.0.12.0/30.** Clause 10 (deny): `INFRA-P2P` permits it, so the clause matches. The action is deny, so the route is **not redistributed**. Processing stops.
2. **10.100.7.0/24.** Clause 10: not a /30, so `INFRA-P2P` doesn't match; move on. Clause 20 (permit): `BRANCH-LANS` matches. The route is redistributed with **tag 110**. Stop.
3. **172.16.5.0/24.** Clause 10: no. Clause 20: no (not in 10.100/16). Clause 30 has no `match`, so it matches everything: redistributed with **tag 111**.
4. **10.100.8.0/25.** Clause 10: no (it's /25). Clause 20: `BRANCH-LANS` allows only /24s, so no. Clause 30: matches; redistributed with **tag 111**, probably not what the designer intended. A `le 25` on the prefix list would fix it.

The resulting EIGRP routes in a neighbour's routing table:

```
R5# show ip route eigrp | include D EX
D EX     10.100.7.0/24 [170/51456] via 10.0.15.1, 00:01:10, GigabitEthernet0/0
D EX     10.100.8.0/25 [170/51456] via 10.0.15.1, 00:01:10, GigabitEthernet0/0
D EX     172.16.5.0/24 [170/51456] via 10.0.15.1, 00:01:10, GigabitEthernet0/0
```

The /30 is absent. `show ip eigrp topology 10.100.8.0/25` would show `Administrator tag is 111`, which is the clue that it fell through to the catch-all clause.

Why does the meaning of "deny" change with context? Because the route map itself only answers one question: "does this item fall into this clause, and is the clause permit or deny?" What "deny" does belongs to the feature calling the route map. Redistribution asks "should I redistribute this?", so deny means no. PBR asks "should I apply special forwarding?", so deny means "no special treatment" — normal routing, not a drop. The route map is a classifier; the feature decides the consequence.

Prefix-List Deny Inside a Route-Map Permit

This combination trips up almost everyone:

```
ip prefix-list LANS seq 5 deny   10.9.0.0/16
ip prefix-list LANS seq 10 permit 10.0.0.0/8 le 24
route-map RM permit 10
 match ip address prefix-list LANS
```

10.9.0.0/16 hits the prefix list's **deny**. That does not mean "drop the route". It means **"this clause doesn't match."** The route falls through to the next clause, and with no next clause, the implicit deny filters it. Here the outcome is the same, but add a `route-map RM permit 20` with no

match and 10.9.0.0/16 is suddenly accepted by clause 20. Inside a route map, a match list's deny means "not me", never "drop".

Common `set` Clauses

Set clause	Used with	Effect
`set metric 100000 100 255 1 1500`	Redistribution into EIGRP	Seed metric: bandwidth, delay, reliability, load, MTU
`set metric 50`	Into OSPF / RIP / BGP (MED)	Seed cost / hop count / MED
`set metric-type type-1`	Into OSPF	E1 instead of the default E2
`set tag 110`	Redistribution	Marks the route (23.5)
`set local-preference 200`	BGP inbound	Prefer this exit for the whole AS
`set weight 500`	BGP inbound	Prefer on this router only
`set as-path prepend 65001 65001`	BGP outbound	Make our path look longer to the neighbor
`set community 65001:100`	BGP	Attach a community tag
`set ip next-hop 10.1.1.1`	PBR / BGP	Override the next hop

More `match` Options

`match ip address` is the most common, but troubleshooting questions use the others:

Match clause	Matches routes…	Typical use
`match ip address prefix-list NAME`	by prefix and length	Almost everything
`match ip route-source prefix-list NAME`	by the **advertising router's** address	"Accept this prefix only from R3"
`match ip next-hop prefix-list NAME`	by next-hop address	Policy by exit point
`match route-type internal` / `external type-1` / `external type-2` / `nssa-external` / `local`	by route type	`redistribute ospf 1 route-map X` taking only E2s, or only internals
`match interface Gi0/1`	whose next hop is out that interface	Redistributing only routes learned on one link
`match metric 20 [+-deviation]`	by metric value	Selecting OSPF externals seeded at a known metric
`match tag 90`	by route tag (23.5)	Loop prevention at redistribution points
`match source-protocol ospf 1`	by the protocol that installed the route	Mixed-source redistribution (IOS XE)

`continue` — Evaluating More Than One Clause

Normally the first matching clause ends processing. In **BGP** route maps you can add `continue` so that, after a clause matches and its `set` runs, evaluation carries on to a later clause. That lets independent policies stack — tag by region in one clause, set local preference by customer type in another:

```
route-map FROM-PEER permit 10
 match ip address prefix-list EU-ROUTES
 set community 65001:44 additive
 continue 20                                ! keep going to clause 20
route-map FROM-PEER permit 20
 match ip address prefix-list GOLD-CUSTOMERS
 set local-preference 200
route-map FROM-PEER permit 30               ! accept everything else unchanged
```

Where Route Maps Get Applied

Feature	Command	Chapter
Redistribution	`redistribute ospf 1 route-map NAME`	24
BGP neighbor policy	`neighbor 192.0.2.1 route-map NAME in|out`	28
Policy-based routing	`ip policy route-map NAME` (interface)	25
Distribute list (IGP filtering)	`distribute-list route-map NAME in`	below
Conditional default	`default-information originate route-map NAME`	24.8

Distribute Lists — Filtering an IGP

Why you need one: sometimes a neighbour advertises routes you must never use (a lab network leaking into production), or you must hide part of your network from a neighbour. A distribute list is the IGP's built-in filter for that.

A **distribute list** filters routes in or out of a routing protocol. It can reference an ACL, a prefix list or a route map:

```
router eigrp 100
 distribute-list prefix NO-LAB in GigabitEthernet0/1    ! ignore these updates from Gi0/1
 distribute-list prefix CORE-ONLY out GigabitEthernet0/2 ! advertise only these out Gi0/2
!
router ospf 1
 distribute-list route-map NOT-IN-RIB in                 ! keep these out of the RIB
```

You can also filter by **who** advertised a route, not just what it is. The `gateway` keyword adds a second prefix list that matches the **update source**:

```
ip prefix-list LAB-NETS  seq 5 permit 10.99.0.0/16 le 32
ip prefix-list FROM-R3   seq 5 permit 10.0.13.3/32
!
router eigrp 100
 distribute-list prefix LAB-NETS gateway FROM-R3 in    ! accept lab nets only from R3
```

A route must match **both** lists to be permitted: the right prefix and the right neighbour. Routes from other neighbours fail the gateway list and are denied, so add further entries if they should be allowed. `distribute-list gateway NAME in` on its own filters purely by neighbour.

Offset Lists — Nudging a Distance-Vector Metric

A distribute list is all-or-nothing. An **offset list** keeps the route but makes it look **worse** by adding to its metric, which steers path selection without removing anything. It exists for the distance-vector protocols, **EIGRP and RIP** (OSPF has `ip ospf cost` instead):

```
access-list 20 permit 10.50.0.0 0.0.255.255
!
router eigrp 100
 offset-list 20 in 50000 GigabitEthernet0/2   ! routes in ACL 20 heard on Gi0/2: +50000
!
router rip
 offset-list 0 out 3 GigabitEthernet0/1        ! ACL 0 = every route; RIP hop count +3
```

The syntax is `offset-list {acl | 0} {in | out} <offset> [interface]`. `in` adjusts routes as they're received, `out` adjusts what you advertise, and **ACL number 0 means "all routes"**. Leave off the interface and it applies on every interface. In EIGRP named mode the command lives under `topology base`. Chapter 26 shows what the offset does to EIGRP's FD and RD and therefore to the feasibility condition.

Why add metric instead of filtering? Because filtering removes the path entirely, while an offset only changes its rank. The worse path stays in the topology table and can still take over if the preferred one fails. A distribute list is a wall; an offset list is a toll — traffic avoids it, but it's still open.

Why can't an OSPF distribute list filter what neighbors learn? Because OSPF doesn't advertise routes. It floods **LSAs**, and every router in an area must hold an identical LSDB (CCNA guide 16.11). A distribute list can't edit the LSDB without breaking that rule, so OSPF's `distribute-list in` only stops the result of SPF from entering **this router's** RIB. The LSAs still flood onward. To really stop routes from spreading in OSPF you filter at the boundaries where routes are created: on an ABR with `area X filter-list prefix NAME in|out` or `area X range ... not-advertise` (Chapter 6, 6.6, and 24.6; a troubleshooting walk-through is in 27.10), or on an ASBR by filtering redistribution (where `distribute-list out` does work, on redistributed routes only). In OSPF, filter where information is summarized or born, not where it's flooded.

Exam trap: an OSPF `distribute-list in` can create a **black hole**. The filtering router keeps flooding the LSA, so its neighbors still compute a path through it — and it drops the traffic, because the route isn't in its own table.

Worked example — the black hole, step by step. R1 — R2 — R3 sit in a line in area 0. Network 10.77.0.0/24 lives behind R3. Someone puts `distribute-list prefix NO-77 in` (denying 10.77.0.0/24) under `router ospf 1` on R2.

1. R3 floods its router LSA containing 10.77.0.0/24. R2 receives it, stores it and floods it on to R1. The LSDB is untouched.
2. R2 runs SPF and finds a path to 10.77.0.0/24. The distribute list blocks that result from R2's RIB, so R2 has **no route** to it.
3. R1 runs SPF on the same LSDB and installs `O 10.77.0.0/24 via R2`.

4. A packet from R1 to 10.77.0.5 reaches R2. R2 has no matching route (and no default), so it drops it and sends ICMP unreachable.

The tell-tale signs: on R1, `show ip route 10.77.0.5` points at R2. On R2, the same command returns `% Network not in table`, yet `show ip ospf database router 3.3.3.3` on R2 still lists the prefix. Fix it by removing the distribute list. If the real goal is to hide the prefix from a whole area, filter the Type 3 LSA at an ABR with `area filter-list` instead, so no router in that area computes a path to it at all.

23.5 Route Tagging

The problem it solves: once a route is redistributed, the new protocol forgets where it came from. Later, at another router, you need to know "was this route born in EIGRP?" so you can stop it going back. A tag carries that answer with the route.

Story Time : A tag is like a "Made in Portugal" sticker on a crate. The shipping company doesn't care what the sticker says; it moves the crate the same way. But a customs officer at the next border can read it and say "this came from Portugal, don't send it back there." Routing protocols are the shipping company: they carry the tag untouched. Your route maps are the customs officers.

A **tag** is a 32-bit number attached to a route. Routing protocols carry it but never act on it themselves — it's a sticky note for **your** policies. OSPF carries it in external (Type 5/7) LSAs, EIGRP in external routes, and RIPv2 in every route. BGP has no tag attribute of its own (it uses communities), but tags can be mapped to and from BGP during redistribution.

```
route-map EIGRP-TO-OSPF permit 10
 set tag 90                          ! "this route was born in EIGRP"
!
route-map OSPF-TO-EIGRP deny 10
 match tag 90                        ! "it was born in EIGRP — don't send it back"
route-map OSPF-TO-EIGRP permit 20
```

Why tag at all, instead of filtering by prefix? Because prefix filters must be **maintained**: every new subnet means editing lists on every boundary router, and the one you forget becomes a loop. A tag records a route's origin, which stays true however many subnets get added. Prefix filters describe a network as it was; tags describe where a route came from, which never goes out of date. Chapter 24 uses exactly this to make two-point mutual redistribution safe.

```
R2# show ip route 10.20.0.0
Routing entry for 10.20.0.0/16
  Known via "ospf 1", distance 110, metric 20
  Tag 90, type extern 2, forward metric 1
```

How the tag travels, step by step (using the two route maps above on two boundary routers, R1 and R2, that both run EIGRP and OSPF):

1. EIGRP network 10.20.0.0/16 reaches R1. R1 redistributes it into OSPF through `EIGRP-TO-OSPF`, which sets **tag 90**. R1 creates a Type 5 LSA for 10.20.0.0/16 carrying tag 90.
2. The LSA floods through OSPF and reaches R2 unchanged — OSPF carries the tag but never looks at it. That's the `Tag 90` in R2's output above.

3. R2 redistributes OSPF into EIGRP through `OSPF-TO-EIGRP`. Clause 10 asks `match tag 90` — yes. The clause is deny, so 10.20.0.0/16 is **not** sent back into EIGRP.
4. An OSPF-native route such as 10.30.0.0/16 has no tag (tag 0). It fails clause 10, hits clause 20 and is redistributed into EIGRP normally.

Without step 3, the EIGRP route would re-enter EIGRP at R2 as an external, and the loop described in 24.4 could form.

23.6 Loop Prevention — Every Mechanism, and What It Defends

A routing loop happens when routers disagree about a path, so packets bounce between them until their TTL runs out. Each protocol family has its own defenses:

Mechanism	Protocol(s)	How it prevents loops
Split horizon	RIP, EIGRP (on by default)	Never advertise a route out of the interface you learned it on — the neighbor that told you doesn't need to hear it back
Route poisoning	RIP, EIGRP	When a route fails, advertise it as **unreachable** (RIP metric 16; EIGRP infinite delay) instead of just going silent
Poison reverse	RIP, EIGRP	Advertise a learned route back to its source with an infinite metric — an explicit "don't use me for this" that overrides split horizon
Hold-down timers	RIP (180 s)	After a route fails, ignore worse news about it for a while, so stale information can't resurrect it
Maximum hop count	RIP (15), EIGRP (100 by default, up to 255)	Counting to infinity stops at a ceiling
Feasibility condition (DUAL)	EIGRP	Only accept a backup path that's provably loop-free (Chapter 26)
Identical LSDB + SPF	OSPF, IS-IS	Every router computes a tree from the same map, so loops can't form inside an area
Area 0 hub rule	OSPF	Inter-area routes must pass through the backbone; ABRs ignore Type 3 LSAs received from non-backbone areas (Chapter 27)
AS_PATH check	eBGP	Reject any route whose AS_PATH already contains your own AS

Mechanism	Protocol(s)	How it prevents loops
iBGP split horizon	iBGP	Never pass iBGP-learned routes to other iBGP peers (hence full mesh or route reflectors with ORIGINATOR_ID and CLUSTER_LIST — Chapter 28)
TTL	IP itself	Even if all else fails, a looping packet dies after at most 255 hops
Tagging + filtering	Redistribution	Stop routes from re-entering the domain they came from (24.4)

Why did distance-vector protocols need so many add-ons? Because a distance-vector router believes its neighbors (CCNA guide 16.2). When a network fails, router A may hear from B that "I can reach it in 3", not knowing B's path runs back through A. The two routers then pass the dead route back and forth, each adding one, and **count to infinity**. Each mechanism closes one door: split horizon stops the most direct echo, poisoning spreads bad news actively, hold-down stops old good news from overriding new bad news, and the hop limit puts an end to the counting. Link-state needs fewer patches because no router trusts another's conclusions — only its raw facts.

Story Time : Distance-vector routing is like asking for directions by word of mouth. Alice tells Bob "the bakery is 1 street from me." Bob tells Carol "it's 2 streets from me." The bakery closes. Alice now knows, but before she can say so, she overhears Bob saying "2 streets from me" and thinks "great, then it's 3 from me via Bob" — not realising Bob's directions run through her. Link-state is everyone having the same printed town map: when a street closes, everyone crosses it out on their own map and recalculates.

How count-to-infinity happens, and how each fix stops it. Three RIP routers in a line: **A — B — C**. Network 10.9.0.0/24 is connected to C.

Step	Without split horizon	What's happening
1	C: 0 hops (connected). B: 1 via C. A: 2 via B	Normal, converged
2	C's link to 10.9.0.0 fails; C removes it	C now has no route
3	B advertises "10.9.0.0 in 1" out every interface, including toward C	No split horizon, so B echoes C's own route back to C
4	C believes B: installs 10.9.0.0 at 2 hops **via B**	A loop: B points to C, C points to B
5	B hears C say 2, and C is B's next hop, so B updates to 3. C then hears 3 and goes to 4…	Count to infinity

Step	Without split horizon	What's happening
6	The count reaches 16 (RIP's "unreachable") and the route is finally removed	Minutes of looping traffic

Now add the fixes one at a time:

- **Split horizon:** in step 3, B never advertises 10.9.0.0 toward C, because B learned it from C. Step 4 can't happen.
- **Route poisoning:** in step 2, C immediately advertises 10.9.0.0 with metric **16**. B and A learn "unreachable" straight away instead of waiting for the route to time out.
- **Poison reverse:** B advertises 10.9.0.0 back toward C with metric 16: an explicit "don't use me for this."
- **Hold-down:** after B marks 10.9.0.0 as down, it ignores any worse-or-equal news about it for 180 seconds, so a stale "3 hops" from A can't revive it.
- **Maximum hop count:** if all else fails, 16 ends the counting.

The A-to-B-to-C line is the simple case. Split horizon can't stop loops around a ring of routers, which is why the other mechanisms exist too.

RIP footnote: on legacy **Frame Relay physical (multipoint) interfaces**, IOS turned RIP split horizon **off by default**, because spokes behind one interface had to hear each other. Subinterfaces kept it on. You'll only see this in older exam scenarios, but it explains why the same RIP config behaved differently on `Serial0` and `Serial0.1`.

Exam trap — split horizon on a hub: on a DMVPN or Frame Relay-style hub, every spoke arrives on **one** multipoint interface. Split horizon stops the hub from advertising spoke A's routes out that same interface to spoke B. With EIGRP over DMVPN you must disable it on the hub (Chapter 30):

```
interface Tunnel0
 no ip split-horizon eigrp 100           ! classic mode
!
router eigrp DMVPN                        ! named mode equivalent
 address-family ipv4 unicast autonomous-system 100
  af-interface Tunnel0
   no split-horizon
```

Why can redistribution break all of these at once? Because every mechanism above works **inside one protocol**. When a route crosses into another protocol, it loses its history: its AS_PATH, its original metric, the fact that it was a feasible successor. To the new protocol it's a brand-new external route. Send it back into its original protocol at a second router and nothing remembers it has been there before. Redistribution is the one place where you have to supply the memory yourself — that's what tags are for.

23.7 Troubleshooting Route Maps and Filters

Why this needs its own method: a policy that's slightly wrong rarely breaks a network loudly. It quietly lets one extra route through, or drops one route that a branch needs. The approach is always the same: take one specific route, walk it through each clause yourself, and compare your answer with the counters and tables.

Symptom	Cause	Command that proves it
Nothing gets redistributed / advertised	No clause matches, so the implicit deny catches everything	`show route-map NAME` — check every clause; is there a final permit?
Everything gets through a filter meant to be selective	`match` references an ACL or prefix list that **doesn't exist** (typo, or wrong case); IOS treats it as match-all	`show ip prefix-list NAME` / `show access-lists NAME` returns nothing
Nothing redistributed, or a BGP neighbour sends/accepts nothing	The **route map itself** doesn't exist. `redistribute … route-map MISSING` redistributes nothing; a missing BGP neighbour route map denies everything on current IOS/IOS XE. That's the opposite of a missing ACL inside an existing route map	`show route-map MISSING` returns nothing; `show run | include route-map`
Prefix-list-denied route still accepted	A later permit clause catches it after the prefix list's deny meant "no match"	Walk the clauses; `show ip prefix-list detail` hit counts
`match ip address NAME` never matches	You meant `match ip address prefix-list NAME`; without the keyword, IOS looks for an **ACL** named NAME	`show route-map NAME` shows `ip address (access-lists): NAME`
Prefix-list entry never hits	Length range wrong (forgot `le`/`ge`), so the route's length falls outside it	`show ip prefix-list detail` — hit count 0
BGP policy change has no effect	Policy is applied only to new updates	`clear ip bgp <neighbor> soft in|out`
OSPF neighbor still learns a filtered route	`distribute-list in` affects only the local RIB	`show ip ospf database` on the neighbor
Traffic black-holed through one router	OSPF `distribute-list in` on a transit router	`show ip route <dest>` there (missing) while neighbors point to it
Two clauses' `set`s expected, only one applied	First match wins; no `continue`	Add `continue` (BGP), or merge the clauses

```
R1# show route-map RM-OSPF-TO-EIGRP
route-map RM-OSPF-TO-EIGRP, deny, sequence 10
  Match clauses:
    ip address prefix-lists: INFRA-P2P
  Set clauses:
  Policy routing matches: 0 packets, 0 bytes
route-map RM-OSPF-TO-EIGRP, permit, sequence 20
  Match clauses:
    ip address prefix-lists: BRANCH-LANS
  Set clauses:
```

The `Policy routing matches` counters only count **PBR** packets — for redistribution and BGP they stay at zero, so don't read zero as "not matching". For those uses, check the prefix list hit counts and the resulting routing tables instead.

Worked ticket 1 — "the ISP is receiving our internal routes."

- **Symptom:** the ISP calls: you are advertising 10.0.0.0/8 subnets to them. The outbound policy was supposed to stop that.
- **Command:** `show run | section route-map TO-ISP-RM` shows `match ip address prefix-list TO-ISP` in a `permit 10` clause, and the BGP neighbour uses `route-map TO-ISP-RM out`.
- **Command:** `show ip prefix-list TO-ISP` returns **nothing**. `show ip prefix-list` lists a list called `TO_ISP` (underscore).
- **Cause:** the route map references a prefix list that doesn't exist. A `match` on a missing prefix list matches everything, so clause 10 permits every route.
- **Fix:** correct the reference to `match ip address prefix-list TO_ISP` (or rename the list), then `clear ip bgp 192.0.2.1 soft out`.
- **Verify:** `show ip bgp neighbors 192.0.2.1 advertised-routes` now lists only 203.0.113.0/24 and the 198.51.100.0 block.

Worked ticket 2 — "branch 7's LAN vanished after redistribution."

- **Symptom:** 10.100.7.0/24 is in OSPF but missing from EIGRP everywhere.
- **Command:** `show ip prefix-list detail BRANCH-LANS` shows `seq 5 permit 10.100.0.0/16 (hit count: 0)`.
- **Reading it:** the entry has no `ge`/`le`, so it matches only the /16 itself. The /24 fails clause 20 and, with no catch-all clause in this version of the route map, hits the implicit deny.
- **Fix:** `ip prefix-list BRANCH-LANS seq 5 permit 10.100.0.0/16 ge 24 le 24` (remove the old seq 5 first).
- **Verify:** the hit count rises and `show ip route 10.100.7.0` on an EIGRP router shows `D EX` with `Tag 110` in `show ip eigrp topology 10.100.7.0/24`.

Chapter 24: Redistribution & Summarization

> **Blueprint:** ENARSI 1.4 — Troubleshoot redistribution between any routing protocols or routing sources; ENARSI 1.5 — Troubleshoot manual and auto-summarization with any routing protocol.

24.1 Why Redistribution Exists — and Why It's Dangerous

In a perfect world, one network runs one routing protocol. Real networks don't. Company A, running EIGRP, buys company B, running OSPF. A branch learns its WAN routes from the provider over BGP. A lab segment is reached through a static route. **Redistribution** is how routes from one source get advertised into another protocol.

It's also the most error-prone feature in routing, for one reason: **every routing protocol speaks a different metric language**, and none of them understands the others' loop prevention. EIGRP's composite metric means nothing to OSPF, OSPF's cost means nothing to RIP, and BGP's AS_PATH vanishes the moment a route enters an IGP. Redistribution is translation between languages that don't share words for "distance" or "I've been here before" — so you have to supply both.

Story Time : Two shops sit side by side, one pricing in euros and one in yen. A shopper carries an item from one to the other. The second shop can't use the old price tag — "20" means nothing in yen — so the shopkeeper must write a fresh price (the **seed metric**). The new tag also says nothing about which shop the item came from. If nobody adds a "came from the euro shop" note (a **tag**), the item can be carried back next door and resold as if it were new stock. That is the redistribution loop in 24.4.

Term	Plain meaning
Redistribution	Taking routes from one source (a protocol, static or connected) and advertising them in another protocol
ASBR (Autonomous System Boundary Router)	In OSPF, a router that redistributes routes into OSPF
Seed metric	The starting metric a redistributed route gets in its new protocol
External route	A route that entered a protocol through redistribution (`O E1/E2`, `D EX`)
Summarization	Advertising one short prefix that covers many longer ones

Two rules hold whenever you redistribute:

1. **Redistribution reads from the redistributing router's own RIB.** A router can only redistribute a route that is **in its own routing table**, installed by the source protocol (plus that protocol's connected interfaces — 24.2). A route that lost on AD, or that the source protocol knows but didn't install, can't be redistributed.
2. **The receiving protocol needs a starting metric** — a **seed metric** — because the original one is meaningless to it.

Why does rule 1 cause so many tickets? Because the route you want to redistribute may be sitting in the OSPF database, healthy, yet lose on AD to a static route or another protocol. `redistribute ospf 1` then quietly skips it: OSPF didn't win that prefix on this router, so it isn't OSPF's to give away. If it isn't in `show ip route` with the right code, it isn't going to be redistributed.

24.2 Seed Metrics and Defaults, Protocol by Protocol

The problem it solves: a route arriving from another protocol has a metric the new protocol can't read. Every protocol needs some number to compare paths, so the router must invent a starting value. Each protocol has a different default for that value, and two of them (EIGRP and RIP) default to "unreachable", which is the cause of many "redistribution does nothing" tickets.

Redistributing [redacted]	Default seed metric	What you need to know
EIGRP	**Infinite** — routes are not advertised	Always set one, with `metric` on the redistribute line, `default-metric`, or a route map. Exceptions: from another EIGRP AS (metric is kept), and connected/static routes (metric taken from the interface)
OSPF	Cost **20**, type **E2**	Routes from **BGP** get cost **1**. Use `metric-type 1` for E1
RIP	**Infinite** (16) for routes from another protocol	Set `metric` (a hop count) or `default-metric`. On IOS, redistributed **connected** and **static** routes are an exception and are advertised with a small default hop count (typically 1) even without a metric. Don't rely on it: set a metric explicitly so the result doesn't depend on the source
BGP	**MED = the IGP metric**; origin incomplete (`?`)	Weight 32768 locally. OSPF→BGP takes **internal routes only** by default

The EIGRP metric takes five values: **bandwidth** (kbps), **delay** (tens of microseconds), **reliability** (out of 255), **load** (out of 255) and **MTU**:

```
router eigrp 100
 redistribute ospf 1 metric 100000 100 255 1 1500     ! 100 Mbps, 1 ms delay
!                           bw(kbps) ^delay(10 us)  ^rel ^load ^MTU
! — or set a default for every redistributed source:
 default-metric 100000 100 255 1 1500
```

Why does EIGRP refuse to advertise a route with no seed metric, when OSPF happily uses 20? Because EIGRP's DUAL algorithm depends on metrics being meaningful: feasible-successor checks compare real distances (Chapter 26), and an invented number could make a looping path look feasible. EIGRP chooses "unreachable" over guessing. OSPF externals are just tagged numbers sitting outside SPF, so a flat 20 is harmless. EIGRP's metric is load-bearing; OSPF's external metric is a label.

Worked ticket — "we redistributed OSPF into EIGRP and nothing happened."

- **Symptom:** R1 has `redistribute ospf 1` under `router eigrp 100`. No EIGRP router learns any OSPF prefix.
- **Command:** `show ip eigrp topology all-links` on R1 shows no external entries for the OSPF prefixes, and `show ip protocols | section eigrp` shows `Redistributing: ospf 1` but no default metric.
- **Command:** `show run | section router eigrp` → `redistribute ospf 1` with no `metric` keyword, and no `default-metric` line.

- **Cause:** the seed metric into EIGRP defaults to infinite, so the routes are treated as unreachable and never advertised.
- **Fix:** `redistribute ospf 1 metric 100000 100 255 1 1500` (or add `default-metric 100000 100 255 1 1500`).
- **Verify:** a neighbour now shows `D EX 10.x.x.x [170/51456]`. The 51456 comes straight from the seed metric plus one GigabitEthernet hop, worked out in "Proving Where an EIGRP External Came From" below.

Connected and Static Routes into OSPF

```
router ospf 1
 redistribute connected subnets route-map LOOPBACKS-ONLY   ! E2, cost 20
 redistribute static subnets metric 50 metric-type 1
```

Two details trip people up:

- **An interface already covered by a `network` statement (or `ip ospf area`) is already an** internal **OSPF route.** Redistributing connected routes doesn't turn it into an external, because OSPF only redistributes connected networks it isn't already advertising. Seeing a subnet as `O` in one place and `O E2` in another means it's being advertised by two different routers in two different ways.
- **Redistributing connected routes pulls in** every **connected interface**, including management and lab subnets. Always pair it with a route map or prefix list naming exactly what you want.

EIGRP to EIGRP — Two Autonomous Systems

Two EIGRP processes (AS 100 and AS 200) on one router don't share routes; you have to redistribute between them. This is the one redistribution that needs **no seed metric**: EIGRP copies the original composite-metric components across. But the route becomes **external** in the receiving AS, with AD **170** instead of 90:

```
router eigrp 100
 redistribute eigrp 200          ! metric components carried over automatically
router eigrp 200
 redistribute eigrp 100
```

The AD contest this creates: the boundary router learns a prefix that lives in AS 200 natively as **internal (90)**. If it also hears it inside AS 100 as an **external (170)** reflected from a second boundary router, the internal copy wins, which is fine. Now reverse it: a router in AS 100 that hears an AS 100-native prefix as external from AS 200's redistribution still prefers its own internal 90. Trouble starts only when the native source is lost. The router then falls back to the reflected external, and the two boundary routers can point at each other. Tag on the way out of each AS and deny the tag on the way back in, exactly as in 24.4.

Proving Where an EIGRP External Came From

`show ip eigrp topology <prefix>` is the single best redistribution-forensics command. For an external, it shows who injected the route, from what, and with what tag:

```
R2# show ip eigrp topology 192.168.5.0/24
EIGRP-IPv4 Topology Entry for AS(100)/ID(2.2.2.2) for 192.168.5.0/24
  State is Passive, Query origin flag is 1, 1 Successor(s), FD is 51456
  Descriptor Blocks:
  10.0.12.1 (GigabitEthernet0/1), from 10.0.12.1, Send flag is 0x0
      Composite metric is (51456/51200), route is External
      Vector metric:
        Minimum bandwidth is 100000 Kbit
        Total delay is 1010 microseconds
        Reliability is 255/255
        Load is 1/255
        Minimum MTU is 1500
        Hop count is 1
        Originating router is 1.1.1.1
      External data:
        AS number of route is 0
        External protocol is RIP, external metric is 2
        Administrator tag is 120 (0x00000078)
```

Read the bottom half. **Originating router 1.1.1.1** is the router-ID of the ASBR that redistributed it. If that equals the local router's own EIGRP router-ID, the local router **discards** the external, which is the duplicate-RID fault in 26.11. **External protocol RIP, metric 2** is where it came from. **Administrator tag 120** proves the tagging route map ran. The numbers also tie back to the seed metric `100000 100 255 1 1500`: 100 Mbps gives $10^7/100{,}000 = 100$, and 100 tens of µs of delay gives 100. So RD = 256 × (100 + 100) = **51,200**. One more GigabitEthernet hop adds 10 µs (1 unit): FD = 256 × (100 + 101) = **51,456**.

Selecting Which OSPF Routes Get Redistributed

When OSPF is the **source**, `match` chooses route types. The default differs by target:

```
router eigrp 100
 redistribute ospf 1 match internal external 2 metric 100000 100 255 1 1500
! internal = intra- and inter-area; external 1/2 = E1/E2; nssa-external = N1/N2
```

Into EIGRP, RIP and OSPF the default is to take internal and external routes. Into **BGP** it's internal only (below). Leaving out `external` is a common way to stop externals bouncing between domains. The same thing can be done with `match route-type` in a route map (23.4).

Limiting What Redistribution Can Inject

A route map with no filter, or a misbehaving source, can flood an IGP with thousands of externals. OSPF can cap it:

```
router ospf 1
 redistribute maximum-prefix 2000 80            ! warn at 80%, stop injecting at 2000
! redistribute maximum-prefix 2000 80 warning-only   ! log only, never stop
```

It's cheap insurance on any ASBR that redistributes BGP.

Redistributing IPv6

IPv6 redistribution uses the same ideas with two differences: **connected networks aren't included unless you say so**, and matching uses IPv6 prefix lists.

```
ipv6 prefix-list BRANCH6 seq 5 permit 2001:db8:100::/40 le 64
route-map EIGRP6-TO-OSPF6 permit 10
 match ipv6 address prefix-list BRANCH6
!
ipv6 router ospf 1
 redistribute eigrp 100 include-connected route-map EIGRP6-TO-OSPF6
!
```

The OSPF `subnets` Keyword

On classic IOS, `redistribute eigrp 100` into OSPF redistributed only **classful** networks — 10.0.0.0/8 if it existed exactly, but not 10.1.1.0/24. You had to add `subnets`. Current IOS XE adds `subnets` automatically (you'll see it appear in the running config), but older platforms and exam questions still feature the trap: **"only some routes get redistributed into OSPF" → check for `subnets`.**

```
router ospf 1
 redistribute eigrp 100 subnets metric-type 1 route-map EIGRP-TO-OSPF
```

What Comes Along: Connected Networks and Route Types

When you redistribute protocol X, IPv4 includes not just X's learned routes but also the **connected networks of interfaces where X is enabled**. IPv6 does **not** — you have to add `include-connected`:

```
ipv6 router ospf 1
 redistribute eigrp 100 include-connected
```

Exam trap: OSPF into BGP takes only **intra- and inter-area** routes by default, not OSPF externals. To include them:

```
router bgp 65001
 address-family ipv4 unicast
  redistribute ospf 1 match internal external 1 external 2
```

BGP into an IGP — Deliberately Hard

By default a router redistributes only **eBGP**-learned routes into an IGP. **iBGP** routes are skipped unless you add `bgp redistribute-internal`.

Why the protection? Two reasons. First, a full internet table is over a million prefixes, and pushing it into OSPF or EIGRP would overwhelm every router in the domain. Second, iBGP routes usually came from your IGP edge to begin with, so redistributing them back in is a classic loop. In practice you redistribute only a few filtered prefixes, or better, originate a default route instead. BGP-to-IGP redistribution should always be filtered — the default only protects you from the worst mistake.

24.3 OSPF External Types — E1 vs E2

The problem it solves: when a route enters OSPF from outside, OSPF has to decide whether the distance inside OSPF to the exit point matters. Sometimes it doesn't (one exit to the internet). Sometimes it matters a lot (two exits, one next door and one across the country). E2 and E1 are the two answers.

Type	Metric as it travels	Best when
E2 (default)	Stays at the seed metric everywhere; internal cost to the ASBR is ignored (used only as a tie-breaker, the forward metric)	One exit point, or when the external cost is all that matters
E1	Seed metric **plus** the internal OSPF cost to reach the ASBR	Several ASBRs advertise the same prefix, and each router should pick its nearest exit

Story Time : You're choosing between two airports to fly to Rome. E2 is like comparing **only the ticket prices** (the external cost) and ignoring how far each airport is from your house. E1 is like comparing **ticket price plus the taxi fare to the airport**. If one airport is next door and the other is 300 km away, only E1 gives the sensible answer.

Worked example — same prefix, two ASBRs. R5 learns 172.20.0.0/16 from two ASBRs. ASBR-A seeds it at 20, ASBR-B at 30. R5's internal OSPF cost is **100** to reach ASBR-A and **5** to reach ASBR-B.

Route type	Via ASBR-A	Via ASBR-B	R5 picks
As E2	metric 20 (forward metric 100)	metric 30 (forward metric 5)	**A** — 20 < 30; internal cost ignored
As E1	20 + 100 = **120**	30 + 5 = **35**	**B** — 35 < 120

With E2, R5 sends traffic across 100 units of internal network to save 10 units outside. With E1 it takes the nearby exit. The forward metric matters for E2 only when the external metrics tie: if both ASBRs seeded 20, E2 would pick B, because its forward metric (5) is lower.

```
R5# show ip route 172.20.0.0
Routing entry for 172.20.0.0/16
  Known via "ospf 1", distance 110, metric 35, type extern 1
  Last update from 10.0.56.6 on GigabitEthernet0/1, 00:00:42 ago
```

For the same prefix, OSPF **always prefers E1 over E2**, whatever the metrics. N1 and N2 are the NSSA equivalents (Type 7, Chapter 27). In the rare case where a Type 5 and a Type 7 describe the same prefix with the same type and metric, the tie-break follows RFC 3101's rules (which changed the older RFC 1587 preference). Treat that as a sign the design is wrong rather than something to memorise.

The Forwarding Address — Why an External Can Be in the LSDB but Not the RIB

Why this matters: this is one of the few cases where the LSA is in the database, OSPF looks healthy, and the route is still missing. Knowing the forwarding-address rule turns a mystery into a two-command diagnosis.

Every Type 5 and Type 7 LSA carries a **Forwarding Address (FA)**. If it's `0.0.0.0`, traffic for the external goes **to the ASBR** that originated it. If it's non-zero, traffic goes **to that address** instead. This happens when the ASBR's next hop for the external sits on a shared segment that OSPF also runs on, so the other routers can skip the extra hop through the ASBR.

The rule ENARSI tests: **a router installs an external route only if it can reach the forwarding address through an OSPF intra-area or inter-area route.** If the FA is reachable only through a static route, another protocol, or another external, or not at all, the LSA sits in the database and the route never appears.

On IOS an ASBR sets a non-zero FA in a Type 5 only when the next-hop interface is OSPF-enabled, not passive, and of a broadcast-style network type (not point-to-point or point-to-multipoint). **Type 7 LSAs in an NSSA carry a non-zero FA**, normally an address of the NSSA ASBR, and the ABR keeps it when it translates them into Type 5. That creates a classic fault. Filter or summarize away the subnet containing the FA at the NSSA ABR (`area range … not-advertise`, `area filter-list`), and every router outside the NSSA loses the external, even though the Type 5 is right there.

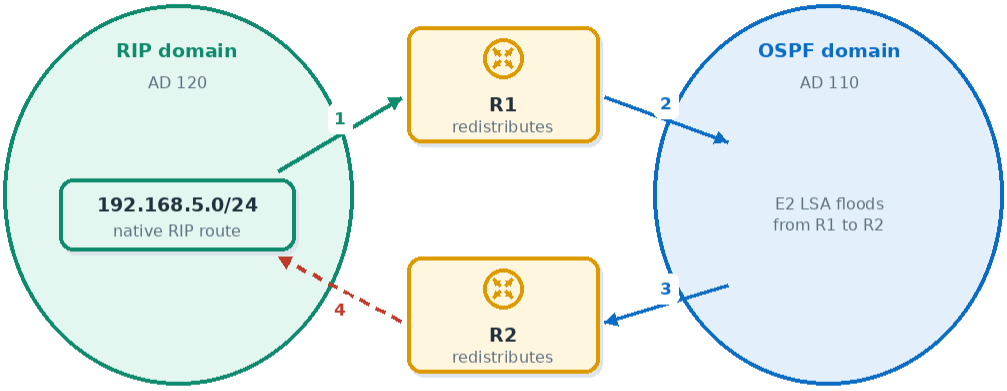
```
R5# show ip ospf database external 172.20.0.0
  Link State ID: 172.20.0.0 (External Network Number )
  Advertising Router: 4.4.4.4
        Metric Type: 2 (Larger than any link state path)
        Metric: 20
        Forward Address: 10.1.45.5          <- must be reachable as O or O IA
R5# show ip route 10.1.45.5
% Subnet not in table                       <- so 172.20.0.0/16 is never installed
```

An external route is a signpost. If the town it points to isn't on your OSPF map, you can't follow it.

Why would E2 ever be the default if it ignores internal distance? Because the typical external route is "the internet, via our one firewall", where internal distance doesn't change the decision. E2 is also stable: the external metric doesn't change as internal links flap, so less churn. E1 earns its place when there's a real choice of exits. E2 says "the cost is out there"; E1 says "the cost includes getting there."

24.4 Two-Point Mutual Redistribution — the Classic Loop

Two-point mutual redistribution — a route can come back wearing a better AD



1. R1 learns the prefix from RIP (AD 120)
2. R1 redistributes it into OSPF as an E2 route
3. R2 hears it twice: RIP 120 vs OSPF E2 110 — the reflected copy wins
4. R2 redistributes its OSPF entry back into RIP — feedback, then loops

The problem: one redistribution point is a single point of failure, so designs use two. But two boundary routers create a circle: a route can leave its domain at one router and come back in at the other. Picture a **RIP** domain and an **OSPF** domain joined by **R1** and **R2**, each redistributing **both ways**. Follow the RIP prefix 192.168.5.0/24:

1. R1 learns 192.168.5.0/24 from RIP (AD 120) and redistributes it into OSPF as an E2 route.
2. R2 now hears the prefix twice: natively from **RIP (AD 120)**, and from **OSPF E2 (AD 110)** via R1.
3. **OSPF wins on AD.** R2 installs the OSPF route, sending traffic for a RIP-domain network across the OSPF domain to R1 — a **suboptimal path**, even though R2 sits right next to the RIP domain.
4. R2 now redistributes its routing table entry — the OSPF one — back into RIP. If the original RIP source fails, RIP routers keep hearing 192.168.5.0/24 from R2, R2 points into OSPF, OSPF points at R1, R1 points into RIP... **A routing loop** that lives until a timer or TTL breaks it.

Why does this happen when both protocols work perfectly? Because AD tells R2 to trust OSPF (110) over RIP (120) **without knowing the OSPF route is just RIP's own route, reflected**. Redistribution erased its history. The same pattern appears with EIGRP **externals** (170 — they lose to OSPF's 110) and with anything else whose native AD is higher than the protocol it gets reflected through. The loop isn't a bug in either protocol; it's the missing memory at the boundary.

Fix 1 — Tag and Filter (stop the route returning home)

Configure the **same** policy on **both** R1 and R2:

```
route-map RIP-TO-OSPF deny 10
 match tag 110                      ! born in OSPF? never send it back to OSPF
route-map RIP-TO-OSPF permit 20
 set tag 120                        ! mark everything leaving RIP as "born in RIP"
!
route-map OSPF-TO-RIP deny 10
 match tag 120                      ! born in RIP? never send it back to RIP
route-map OSPF-TO-RIP permit 20
 set tag 110                        ! mark everything leaving OSPF as "born in OSPF"
!
router ospf 1
 redistribute rip subnets route-map RIP-TO-OSPF
router rip
 version 2
 redistribute ospf 1 metric 3 route-map OSPF-TO-RIP
```

Now a route can cross the boundary **once**. The tag travels with it (RIPv2 and OSPF externals both carry tags), and the other boundary router recognizes and refuses to send it back.

Fix 2 — Correct the AD (stop the suboptimal choice)

Tags stop the loop, but R2 still prefers the OSPF E2 copy (110) of a RIP route over its native RIP route (120). Making OSPF externals less trusted than RIP fixes that on both boundary routers:

```
router ospf 1
 distance ospf external 125         ! native RIP (120) now beats reflected E2s
```

This affects **every** OSPF external route on that router, so make sure that's acceptable. To be surgical, use the per-source `distance` command with an ACL listing just the RIP-domain prefixes (22.6).

Which fix does the exam want? Usually **both**: tags stop routes re-entering their origin domain (the loop), AD tuning makes boundary routers prefer the native path (the suboptimal routing). Tags fix the memory problem; AD fixes the trust problem.

Worked example — the same prefix, after both fixes. Replay 192.168.5.0/24 with Fix 1 and Fix 2 on R1 and R2:

1. R1 learns 192.168.5.0/24 from RIP (AD 120). `RIP-TO-OSPF` clause 10 asks "tag 110?" — no (RIP-native routes carry tag 0). Clause 20 permits it and sets **tag 120**. R1 floods a Type 5 with tag 120.
2. R2 hears the prefix from RIP (AD 120) and as OSPF E2 (now AD **125** because of `distance ospf external 125`). 120 < 125, so R2 installs the **native RIP** route. The suboptimal path is gone.
3. R2 redistributes OSPF into RIP through `OSPF-TO-RIP`. Even if R2 ever installs the OSPF copy (say its RIP neighbour fails), clause 10 matches **tag 120** and denies it. The route can never re-enter RIP. The loop is gone.
4. Now the RIP source of 192.168.5.0/24 really fails. RIP withdraws it, R1 stops redistributing it, the Type 5 is flushed, and the prefix disappears everywhere instead of circulating.

```
R2# show ip route 192.168.5.0
Routing entry for 192.168.5.0/24
  Known via "rip", distance 120, metric 1
  Redistributing via ospf 1
```

`Known via "rip"` on R2 proves the AD fix worked. `External Route Tag: 120` in R2's `show ip ospf database external 192.168.5.0` (shown below) proves the tag fix ran on R1.

Troubleshooting Redistribution

Symptom	Cause	Command that proves it
Routes redistributed into EIGRP or RIP never appear	No seed metric (infinite by default)	`show run | section router eigrp` — no `metric` / `default-metric`
Only classful networks reach OSPF	Missing `subnets` (older IOS)	`show ip ospf database external` — only /8, /16, /24 classful prefixes
Some source routes are missing after redistribution	They're not in the RIB from that source (lost on AD)	`show ip route <prefix>` — Known via a different source
Connected subnets missing (IPv6)	No `include-connected`	`show ipv6 route` on a neighbor
OSPF externals missing from BGP	Default `match internal` only	`show bgp ipv4 unicast` — compare with `show ip route ospf`
iBGP prefixes not redistributed into the IGP	Expected default behavior	Add `bgp redistribute-internal` — only with a filter
Route flaps between two sources every few seconds	Mutual redistribution feedback	Repeated `show ip route <prefix>`; `debug ip routing`
Boundary router uses a long path to a nearby prefix	Reflected copy wins on AD	`show ip route <prefix>` shows `ospf … type extern 2` and a `Tag` from the other domain
Route map filters everything	No final permit clause	`show route-map NAME` (23.7)

```
R2# show ip ospf database external 192.168.5.0
  LS Type: AS External Link
  Link State ID: 192.168.5.0 (External Network Number )
  Advertising Router: 1.1.1.1
  Network Mask: /24
        Metric Type: 2 (Larger than any link state path)
        Metric: 20
        Forward Address: 0.0.0.0
        External Route Tag: 120
```

`Advertising Router` is the ASBR that injected it, and `External Route Tag: 120` proves the tagging policy ran. If the tag is missing, the route map on that ASBR isn't being applied.

24.5 Summarization — Fewer Routes, More Stability

Summarization advertises one shorter prefix in place of many longer ones: 10.1.0.0/16 instead of 256 /24s. It shrinks routing tables, and it hides instability. When 10.1.37.0/24 flaps, routers outside the summary don't even notice, because the /16 stays up as long as any component exists.

Why does hiding detail make a network more stable? Because every advertised change forces work: EIGRP sends queries (Chapter 26), OSPF re-runs SPF. A summary is a **wall that**

stops churn spreading. A flapping link inside the summarized area is handled locally and never reaches the rest of the network. Summarization is to routing what areas are to OSPF: keep detail local, share only what others need.

Story Time : A courier in another country doesn't need to know every flat in your apartment block. "Deliver anything for 12 Oak Street to the front desk" is enough; the front desk knows the flats. If flat 37 is being repainted and closed for a week, the courier never hears about it. The summary is "12 Oak Street", the summarizing router is the front desk, and the repainting is a flapping /24 that stays a local problem. The analogy also shows the risk: if the front desk takes a parcel for a flat that doesn't exist, it has to bin it, not send it back to the courier. That bin is the Null0 route below.

How to work out a summary, step by step. You want one route covering 10.1.4.0/24, 10.1.5.0/24, 10.1.6.0/24 and 10.1.7.0/24.

1. The first two octets (10.1) are the same in all four, so the first 16 bits match.
2. Write the third octet in binary: 4 = 0000 01**00**, 5 = 0000 01**01**, 6 = 0000 01**10**, 7 = 0000 01**11**.
3. The first **6** bits of the third octet (000001) are the same in all four; the last 2 bits differ.
4. Matching bits: 16 + 6 = **22**. Set every non-matching bit to 0: the third octet becomes 0000 0100 = 4.
5. The summary is **10.1.4.0/22** (mask 255.255.252.0), which covers exactly 10.1.4.0 to 10.1.7.255 — no more, no less.

Try one that doesn't fit neatly: 10.1.5.0/24 to 10.1.8.0/24. In binary, 5 = 0000 0101 and 8 = 0000 1000 share only the first 4 bits, so the tightest summary is 10.1.0.0/20. That also covers 10.1.0.0 to 10.1.15.255, including twelve /24s you don't own. Either accept the wider claim (and its Null0 route), or advertise three exact routes instead: 10.1.5.0/24, 10.1.6.0/23 (which covers .6 and .7) and 10.1.8.0/24. Good addressing plans hand out blocks on power-of-two boundaries precisely so summaries come out clean.

It has costs, which ENARSI tests directly — covered in 24.7.

24.6 Summarization in Each Protocol

EIGRP — Auto and Manual

Auto-summary summarizes to the classful boundary whenever a route crosses a major-network boundary. It's **off by default since IOS 15.0** and should stay off: with **discontiguous networks** (subnets of 172.16.0.0 on both sides of a 10.x core), both edges advertise 172.16.0.0/16 and the core load-shares between them — sending half the traffic to the wrong site.

Manual summaries are configured **per interface**, so you can summarize differently toward different neighbors:

```
! Classic mode
interface GigabitEthernet0/1
 ip summary-address eigrp 100 10.1.0.0 255.255.0.0

! Named mode
router eigrp CAMPUS
 address-family ipv4 unicast autonomous-system 100
  af-interface GigabitEthernet0/1
   summary-address 10.1.0.0 255.255.0.0
```

Worked example — before and after an EIGRP summary. R1 has four branch /24s (10.1.4.0 to 10.1.7.0) and summarizes toward the core on Gi0/1 with `ip summary-address eigrp 100 10.1.4.0 255.255.252.0`.

```
! Core router BEFORE the summary
CORE# show ip route eigrp | include 10.1.
D        10.1.4.0/24 [90/3072] via 10.0.1.1, 00:20:01, GigabitEthernet0/0
D        10.1.5.0/24 [90/3072] via 10.0.1.1, 00:20:01, GigabitEthernet0/0
D        10.1.6.0/24 [90/3072] via 10.0.1.1, 00:20:01, GigabitEthernet0/0
D        10.1.7.0/24 [90/3328] via 10.0.1.1, 00:20:01, GigabitEthernet0/0

! Core router AFTER the summary
CORE# show ip route eigrp | include 10.1.
D        10.1.4.0/22 [90/3072] via 10.0.1.1, 00:00:12, GigabitEthernet0/0

! R1 itself
R1# show ip route 10.1.4.0 255.255.252.0 | include Known|Null
  Known via "eigrp 100", distance 5, metric 2816, type internal
   * directly connected, via Null0
```

Four routes became one, carrying the lowest component metric (3072). R1 gained the Null0 discard route at AD 5, with the lowest component metric as seen on R1 (2816).

The summary is advertised as long as **at least one** component route exists, and it carries the **lowest metric** of its components. Use `summary-metric` in named mode to pin it, so it doesn't change every time the best component changes. A **leak map** lets you advertise selected components alongside the summary:

```
route-map LEAK-VOICE permit 10
 match ip address prefix-list VOICE-SUBNET      ! components to advertise as well
!
! Classic mode
interface GigabitEthernet0/1
 ip summary-address eigrp 100 10.1.0.0 255.255.0.0 leak-map LEAK-VOICE
!
! Named mode — leak map on the interface, fixed metric under the topology
router eigrp CAMPUS
 address-family ipv4 unicast autonomous-system 100
  af-interface GigabitEthernet0/1
   summary-address 10.1.0.0 255.255.0.0 leak-map LEAK-VOICE
  exit-af-interface
  topology base
   summary-metric 10.1.0.0/16 100000 100 255 1 1500
```

Components matched by the leak map are advertised **in addition to** the summary, so a neighbour gets the /16 for everything plus the specific voice /24 for an optimal path. The `summary-metric` values are the same five components as a seed metric (24.2).

EIGRP and OSPF for IPv6

```
interface GigabitEthernet0/1
 ipv6 summary-address eigrp 100 2001:db8:10::/48     ! classic IPv6 EIGRP
!
router ospfv3 1
 address-family ipv6 unicast
  area 1 range 2001:db8:10::/48                       ! ABR, inter-area
  summary-prefix 2001:db8:200::/40                    ! ASBR, externals
```

The rules are the same as IPv4: per interface for EIGRP, ABR (`area range`) or ASBR (`summary-prefix`, the IPv6 name for `summary-address`) for OSPFv3.

RIP

RIPv2's **auto-summary** is still enabled by default on many IOS releases, unlike EIGRP's. Check `show ip protocols` for "Automatic network summarization is in effect" and turn it off with `no auto-summary` under `router rip`. Manual summaries go on the interface:

```
interface GigabitEthernet0/1
 ip summary-address rip 10.1.0.0 255.255.0.0
```

RIP restriction: a manual RIP summary **can't be shorter than the classful network** (no supernets such as 172.16.0.0/12). The command is rejected or ignored. Summarizing beyond the classful boundary needs another protocol.

```
R1# show ip route 10.1.0.0 255.255.0.0
Routing entry for 10.1.0.0/16
  Known via "eigrp 100", distance 5, metric 2816, type internal
  Routing Descriptor Blocks:
  * directly connected, via Null0
```

The Null0 Discard Route — Why Every Summary Creates One

Every summarizing router installs a route for the summary pointing to **Null0** — a virtual interface that silently discards packets. EIGRP gives it AD **5**; OSPF and BGP install their own Null0 routes too.

Why would a router deliberately drop traffic? Picture R1 advertising 10.1.0.0/16 upstream, with a default route pointing back upstream. A packet arrives for 10.1.99.0, a subnet that doesn't exist inside the summary. Step by step, without the discard route:

1. Upstream router U has `D 10.1.0.0/16 via R1`. A packet for 10.1.99.5 arrives at U; U sends it to R1.
2. R1 has no 10.1.99.x route. Its longest match is the **default** `0.0.0.0/0 via U`, so it sends the packet back to U.
3. U matches the /16 again and sends it to R1. The packet bounces until its TTL (often 64 or 128 at the start) reaches 0, using bandwidth on every pass.

With the discard route, step 2 changes: R1's longest match is now `10.1.0.0/16 via Null0`, which is longer than the default. R1 drops the packet at once (and can send ICMP unreachable). The discard route is the summarizer saying: "I claimed this whole block, so if I don't have the piece, nobody does."

Exam trap: don't delete the Null0 route because it "looks wrong" in the table. Removing it (OSPF: `no discard-route internal` or `no discard-route external`) brings the loop back.

OSPF — `area range` vs `summary-address`

OSPF can only summarize where routes are **created as summaries**, so there are exactly two places, with two commands (Chapter 6 covers the ENCOR basics):

Command	Where	Summarizes
`area 1 range 10.1.0.0 255.255.0.0`	**ABR**, in `router ospf`	Intra-area routes of area 1, advertised as one Type 3 into other areas
`summary-address 172.16.0.0 255.255.0.0`	**ASBR**, in `router ospf`	Redistributed external routes (Type 5/7)

```
router ospf 1
 area 1 range 10.1.0.0 255.255.0.0 cost 50            ! fixed cost, no flapping metric
 area 1 range 10.1.200.0 255.255.255.0 not-advertise  ! hide this block entirely
 summary-address 172.16.0.0 255.255.0.0               ! on the ASBR, for externals
```

Exam trap: `area range` on a router that isn't an ABR for that area does nothing. `summary-address` on a router that isn't redistributing anything does nothing. Neither produces an error — the summary just never appears. And `not-advertise` is a quiet **filtering** tool, so remember it when a prefix "mysteriously" never leaves its area.

BGP — `aggregate-address`

```
router bgp 65001
 address-family ipv4 unicast
  aggregate-address 10.0.0.0 255.0.0.0 summary-only    ! advertise the /8, suppress the rest
```

Option	Effect
(none)	Advertise the aggregate **and** all its components
`summary-only`	Advertise only the aggregate; components are suppressed (`s` in `show bgp`)
`as-set`	Include the components' AS numbers in the aggregate's AS_PATH as a set, preserving loop detection
`suppress-map` / `unsuppress-map`	Suppress (or leak) selected components

Two rules: the aggregate is only created if **at least one component exists in the BGP table** (not just the RIB), and the router installs a local **Null0** route for it (AD 200). Without `as-set`, the aggregate carries the **ATOMIC_AGGREGATE** attribute, a warning that path information was lost.

Worked example. R1 in AS 65001 has 10.1.0.0/16 and 10.2.0.0/16 in its BGP table. With `aggregate-address 10.0.0.0 255.0.0.0 summary-only`:

```
R1# show bgp ipv4 unicast | begin Network
     Network          Next Hop            Metric LocPrf Weight Path
 *>  10.0.0.0         0.0.0.0                            32768 i
 s>  10.1.0.0/16      10.0.12.2                0             0 65002 i
 s>  10.2.0.0/16      10.0.13.3                0             0 65003 i
```

The `s` marks the suppressed components: still best paths on R1, but not sent to neighbours. The aggregate has next hop 0.0.0.0 (locally created) and an empty AS_PATH, plus ATOMIC_AGGREGATE. With `as-set` added, its path would become `{65002,65003}`, and an AS 65002 router receiving it would reject it by the AS_PATH loop check.

Why is `as-set` optional, if it preserves loop detection? Because it makes the aggregate **unstable**. Every time a component from some AS appears or disappears, the AS_SET changes and the aggregate is re-advertised to the whole internet. Most operators skip `as-set` and accept the small loop risk, because their aggregates cover only their own address space. Accuracy versus stability — BGP almost always picks stability.

24.7 What Summarization Breaks

Problem	How it happens	Mitigation
Suboptimal routing	Two ABRs advertise the same summary; a remote router picks the closer ABR, which may be the farther one from the actual subnet	Leak specific prefixes (leak map, `summary-only` off), or accept it
Black hole	Two routers advertise a summary; one has lost a component but keeps advertising the summary (other components still exist) and drops that traffic to Null0	Summarize at one point, or leak the critical components; link the summarizers directly so one can reach the other's subnets
Discontiguous networks	Auto-summary at classful boundaries on both sides of a core	`no auto-summary` (the default since 15.0)
Metric churn	Summary metric follows its best component, flapping with it	`summary-metric` (EIGRP), `cost` on `area range` (OSPF)
Missing summary	No component in the RIB / BGP table, or wrong router	`show ip route` for components; check you're on the ABR/ASBR
Unwanted loops without discard	`no discard-route internal|external` (OSPF)	Restore the Null0 route

Why is the black hole so nasty? Because everything looks healthy. Both summarizers are up, both advertise the summary, and the routing tables all agree. Only the traffic that happens to choose the summarizer missing that one subnet disappears into Null0. Pings to other subnets work, so it looks like an application issue. Summaries are promises; a summarizer can keep promising after it's lost the ability to deliver. The test that exposes it is a `traceroute` that stops at the summarizing router, combined with `show ip route <specific>` on that router showing only the Null0 summary.

Worked ticket — "half the users can't reach 10.1.6.0/24."

- **Setup:** ABR1 and ABR2 both connect area 1 to area 0, and both have `area 1 range 10.1.4.0 255.255.252.0`. A link failure splits area 1 so that ABR2 can no longer reach 10.1.6.0/24 (ABR1 still can). Other components (10.1.4.0, 10.1.5.0, 10.1.7.0) remain reachable from both.
- **Symptom:** from some backbone routers 10.1.6.10 works; from others it fails.
- **Command, on a failing source:** `traceroute 10.1.6.10` stops at ABR2's address.

- **Command, on ABR2:** `show ip route 10.1.6.10` → `Routing entry for 10.1.4.0/22 ... via Null0`. ABR2 has no /24 for it.
- **Cause:** ABR2 still has three components, so it keeps advertising the /22. Backbone routers closer to ABR2 send 10.1.6.x traffic there, and ABR2 discards it.
- **Fix:** restore the failed area 1 link, and harden the design: give the two ABRs a direct link inside area 1 so each can reach the other's components, or leak 10.1.6.0/24 outside the summary if it is critical.
- **Verify:** `traceroute 10.1.6.10` from the failing source now completes through ABR2.

24.8 Originating a Default Route — Protocol by Protocol

The problem it solves: branch and campus routers rarely need the full list of internet or WAN prefixes. They just need "anything you don't know, send it toward the exit." The exit router has to advertise that default into the IGP, so every other router learns it automatically and learns when it goes away.

A default route is the most-redistributed "route" of all, and every protocol has its own way to originate one. Each has a precondition that the exam likes to break.

OSPF — `default-information originate`

```
router ospf 1
 default-information originate                          ! only if a default is in the RIB
 default-information originate always                   ! even with no default route
 default-information originate metric 10 metric-type 1  ! seed it as E1, cost 10
 default-information originate route-map ISP-ALIVE      ! conditional (below)
```

The router becomes an ASBR and floods a **Type 5 for 0.0.0.0/0** (default metric 1, type E2). The precondition: **without `always`, a default route must already be in this router's RIB from some other source**, typically a static to the ISP or one learned by BGP. No default in the RIB means nothing is originated and no error is shown. That's the classic ENARSI fault: the config looks right, but `show ip ospf database external 0.0.0.0` is empty.

Why not just always use `always`? Because then the router advertises a default even when its own exit is dead, and draws all traffic into a black hole. Without `always`, the advertisement disappears with the upstream route, and traffic shifts to another exit automatically. Use `always` only when the router is the sole exit and blackholing is acceptable, or pair it with a **conditional route map**:

```
ip prefix-list ISP-PROBE seq 5 permit 198.51.100.0/24   ! exists only while the ISP is up
route-map ISP-ALIVE permit 10
 match ip address prefix-list ISP-PROBE
!
router ospf 1
 default-information originate always route-map ISP-ALIVE
```

The route map is checked against the **RIB**. While 198.51.100.0/24 is present (learned from the ISP by BGP, say), the default is advertised. When it disappears, the default is withdrawn.

Worked example — what the rest of the area sees. Edge router R1 (RID 1.1.1.1) has `ip route 0.0.0.0 0.0.0.0 203.0.113.1` and `default-information originate metric 10 metric-type 1`.

Internal router R3, two hops away with internal cost 20 to R1, shows:

```
R3# show ip route 0.0.0.0
Routing entry for 0.0.0.0/0, supernet
  Known via "ospf 1", distance 110, metric 30, candidate default path
  Tag 1, type extern 1
  Routing Descriptor Blocks:
  * 10.0.23.2, from 1.1.1.1, 00:04:10 ago, via GigabitEthernet0/0
```

Metric 30 = seed 10 + internal cost 20, because it's E1. `from 1.1.1.1` names the originating ASBR. Remove R1's static route (and there's no `always`), and R1 flushes the Type 5: R3's default disappears within seconds, and if a second edge router also originates a default, R3 switches to it.

For NSSAs the command is different (`area X nssa default-information-originate`), and so are the rules. See 27.5.

EIGRP

EIGRP has no `default-information originate`. The two common methods:

```
ip route 0.0.0.0 0.0.0.0 203.0.113.1
router eigrp 100
 redistribute static metric 100000 100 255 1 1500   ! 0/0 becomes an external (D*EX)
!
! or: summarize to a default toward a stub branch
interface GigabitEthernet0/2
 ip summary-address eigrp 100 0.0.0.0 0.0.0.0
```

Exam trap 1: a 0.0.0.0/0 summary **suppresses every more-specific route** out of that interface. The branch receives only the default, which is often exactly the point.

Exam trap 2: like every EIGRP summary, it installs a local **Null0 discard route** for 0.0.0.0/0 at **AD 5**. A static default (AD 1) still beats it. But if this router learns its own default dynamically, for example from eBGP (20) or OSPF (110), the AD-5 Null0 route wins and the router blackholes everything it was meant to forward upstream. Give the summary a worse AD with the optional distance argument:

```
interface GigabitEthernet0/2
 ip summary-address eigrp 100 0.0.0.0 0.0.0.0 250   ! local discard route at AD 250
```

BGP

BGP won't advertise a default learned from redistribution unless told to, and has a per-neighbour shortcut:

```
router bgp 65001
 address-family ipv4 unicast
  neighbor 10.0.0.2 default-originate                    ! send 0/0 to this neighbour only
  neighbor 10.0.0.2 default-originate route-map HAVE-ISP  ! only if HAVE-ISP matches the RIB
  network 0.0.0.0                                        ! advertise a default from the RIB
  default-information originate                          ! needed if 0/0 is redistributed
```

`neighbor default-originate` doesn't need a default in the RIB or the BGP table. It generates one for that neighbour, which is why the conditional route-map form exists.

RIP

```
router rip
 default-information originate [route-map NAME]
```

Unlike OSPF, RIP's `default-information originate` advertises 0.0.0.0/0 to RIP neighbours **without** needing a default in the RIB. Add the `route-map` option to make it conditional, in the same way as the OSPF example above. Neighbours see it as `R* 0.0.0.0/0 [120/1]`.

Troubleshooting Default Origination

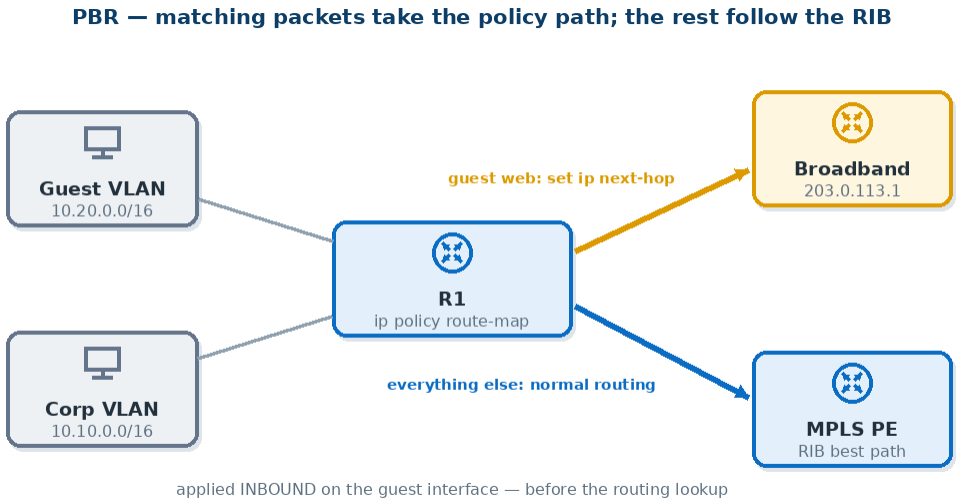
Symptom	Cause	Command that proves it
OSPF default not advertised	No default in the RIB and no `always`	`show ip route 0.0.0.0`; `show ip ospf database external 0.0.0.0` empty
OSPF default still advertised after ISP failure	`always` without a condition	Config review; add a route-map condition
Conditional default never appears	Route map's prefix not in the RIB (or prefix list typo)	`show ip route <probe prefix>`; `show route-map`
EIGRP branch learns nothing but 0/0	0/0 summary suppresses specifics	`show ip protocols` lists the summary; `show ip route` on the branch
Edge router blackholes upstream traffic after adding a 0/0 EIGRP summary	Local Null0 summary (AD 5) beats a dynamically learned default (eBGP 20, OSPF 110) or a floating static	`show ip route 0.0.0.0` shows `via Null0`, distance 5; fix with the summary's AD argument
NSSA routers get no default	`default-information-originate` missing on the NSSA ABR/ASBR, or no default on a non-ABR ASBR	27.5

Chapter 25: PBR, VRF-Lite & BFD

> **Blueprint:** ENARSI 1.6 — Configure and verify policy-based routing; ENARSI 1.7 — Configure and verify VRF-Lite; ENARSI 1.8 — Describe Bidirectional Forwarding Detection.

Three tools that change the normal rules of routing. **PBR** overrides where a packet goes, **VRF-Lite** gives one router several separate routing tables, and **BFD** changes how fast a router notices a path has died.

25.1 Policy-Based Routing — Routing by Something Other Than Destination



Normal routing looks at **one field**: the destination address. Chapter 5 introduced PBR as the exception. Here you configure and troubleshoot it. PBR lets a router decide using the **source address**, protocol, port, packet size or DSCP — anything an ACL can match. Typical uses:

- Send the guest VLAN out the cheap broadband link and corporate traffic over MPLS.
- Push backup or replication traffic onto a link that's idle during the day.
- Steer specific traffic through a firewall or WAN optimizer that isn't on the normal path.

Story Time : Normal routing is a sat-nav that only asks "where are you going?" PBR is a traffic officer standing at the junction before the sat-nav speaks. The officer looks at each car: "Delivery van from the guest car park? Take the side road." Every car the officer doesn't wave aside just follows the sat-nav as usual. The officer only stands at the **entrance** to the junction, which is why PBR goes on the inbound interface.

PBR is a **route map applied to an interface**. Every packet arriving on that interface is checked against the route map before the normal routing lookup:

```
ip access-list extended GUEST-WEB
 permit tcp 10.20.0.0 0.0.255.255 any eq 80
 permit tcp 10.20.0.0 0.0.255.255 any eq 443
!
route-map PBR-GUEST permit 10
 match ip address GUEST-WEB              ! guest web traffic...
 set ip next-hop 203.0.113.1              ! ...goes to the broadband router
!
interface GigabitEthernet0/2.20          ! the guest VLAN's gateway interface
 ip policy route-map PBR-GUEST           ! INBOUND — PBR only sees arriving packets
```

The rules that decide what happens to each packet:

Packet...	Result
Matches a **permit** clause with a usable `set`	Policy-routed as the `set` says

Packet...	Result
Matches a **deny** clause	**Routed normally** — deny means "no policy", not "drop" (23.4)
Matches **no** clause	Routed normally
Matches a permit clause, but the next hop is down / not adjacent	Falls through to the next `set` option, then to normal routing

How PBR decides, step by step. R1's routing table sends everything (`0.0.0.0/0`) to the MPLS router 10.0.0.2. Three packets arrive on Gi0/2.20 with the config above:

1. **10.20.4.7 → 93.184.216.34, TCP 443.** PBR checks clause 10: `GUEST-WEB` permits it (guest source, port 443). The clause is permit, and 203.0.113.1 is up and in ARP. R1 forwards to **203.0.113.1**, the broadband router. The routing table is never consulted.
2. **10.20.9.9 → 10.1.1.5, ICMP echo.** `GUEST-WEB` only permits TCP 80/443, so clause 10 doesn't match. No more clauses: the packet is **routed normally** — to 10.0.0.2 via the default route.
3. **10.30.1.1 → 93.184.216.34, TCP 443**, arriving on Gi0/2.30 (the corporate VLAN). There's no `ip policy` on Gi0/2.30, so PBR never runs. Normal routing sends it to 10.0.0.2.

Packet 3 shows the most-forgotten fact about PBR: it only affects traffic **arriving on the interfaces where you applied it**.

Why is PBR applied inbound only? Because by the time a packet would leave an interface, the routing decision has already been made — that's how the router chose the outbound interface. PBR has to run **before** the routing lookup to replace it, and the only place that happens is on ingress. To change the decision, you have to get in before it's made.

The `set` Options and Their Order

Set clause	Meaning
`set ip next-hop 203.0.113.1`	Forward to this **directly connected** next hop, ignoring the routing table
`set ip next-hop verify-availability 203.0.113.1 10 track 1`	Use it only while tracking object 1 is up (usually an IP SLA probe)
`set ip next-hop recursive 192.0.2.9`	The next hop is **not** directly connected; resolve it through the RIB
`set interface Tunnel0`	Send out this interface (best on point-to-point links)
`set ip default next-hop 203.0.113.1`	Use it **only if** the routing table has no explicit route for the destination (a default route doesn't count)
`set default interface Tunnel0`	Same idea, with an interface

The evaluation order. When one clause contains several `set` options, IOS tries them in a fixed order regardless of the order you typed them, and uses the first one that's usable:

1. `set ip next-hop` (including `verify-availability` / `recursive`)
2. `set interface`

3. `set ip default next-hop`, but only if the RIB has no explicit route
4. `set default interface`, likewise
5. Otherwise, normal routing

Within one `set ip next-hop` line you can list several addresses (`set ip next-hop 203.0.113.1 198.51.100.1`). The router uses the **first one that is reachable and adjacent**, and moves to the next only when it isn't. That gives you an ordered primary/backup in a single line.

Why do "default" versions exist? They flip the priority. Plain `set ip next-hop` says "policy first, routing table never." The `default` forms say "routing table first, policy only for what it can't route." A branch might use `set ip default next-hop` to send unknown (internet) destinations out a local breakout, while every known corporate prefix still follows the routing table. Plain = policy overrides routing; default = routing overrides policy.

Worked example — plain versus default. A branch router has `O 10.0.0.0/8 via 10.0.0.2` (corporate, over MPLS) and **no** default route. The clause matches all guest traffic.

Packet from the guest VLAN to...	With [redacted]	With [redacted]
10.1.1.5 (corporate server)	Sent to **203.0.113.1** — policy wins, even though the RIB has a route	Sent to **10.0.0.2** — the RIB has an explicit route, so the policy steps aside
93.184.216.34 (internet)	Sent to **203.0.113.1**	Sent to **203.0.113.1** — the RIB has no explicit route, so the policy applies

Now add a static default `ip route 0.0.0.0 0.0.0.0 10.0.0.2` to that router. With `set ip default next-hop`, internet traffic still goes to 203.0.113.1, because a **default route doesn't count** as an explicit route for this test. That rule is a favourite exam detail.

Exam trap: plain `set ip next-hop` to an address that's down doesn't drop traffic. It fails over to normal routing, so the policy silently stops applying. If the next hop is still in ARP and the interface is up, though, PBR keeps sending to a router that may be dead upstream. That's why production PBR uses `verify-availability` with tracking:

```
ip sla 1
 icmp-echo 203.0.113.1 source-interface GigabitEthernet0/0
 frequency 5
ip sla schedule 1 life forever start-time now
track 1 ip sla 1 reachability            ! up while the probe succeeds
!
route-map PBR-GUEST permit 10
 match ip address GUEST-WEB
 set ip next-hop verify-availability 203.0.113.1 10 track 1
```

Policy for Traffic the Router Generates

Why you'd need this: you may want the router's own management traffic — syslog to a collector, pings for IP SLA, SNMP traps — to leave by a specific link, such as an out-of-band management path. `ip policy` only sees packets **arriving** on an interface. Packets the router itself originates — pings, syslog, SNMP traps, routing protocol traffic — never arrive anywhere. For those, use **local policy**:

```
R1(config)# ip local policy route-map PBR-MGMT
```

IPv6 works the same way with `ipv6 policy route-map` and `ipv6 local policy route-map`, using IPv6 ACLs and `set ipv6 next-hop`.

Verifying and Troubleshooting PBR

```
R1# show ip policy
Interface      Route map
Gi0/2.20       PBR-GUEST

R1# show route-map PBR-GUEST
route-map PBR-GUEST, permit, sequence 10
  Match clauses:
    ip address (access-lists): GUEST-WEB
  Set clauses:
    ip next-hop verify-availability 203.0.113.1 10 track 1  [up]
  Policy routing matches: 18442 packets, 9871234 bytes

R1# debug ip policy
IP: s=10.20.4.7 (GigabitEthernet0/2.20), d=93.184.216.34, len 60, FIB policy match
IP: s=10.20.4.7 (GigabitEthernet0/2.20), d=93.184.216.34, g=203.0.113.1, len 60,
    FIB policy routed
IP: s=10.20.9.9 (GigabitEthernet0/2.20), d=10.1.1.5, len 84, FIB policy rejected(no match)
    - normal forwarding
```

Each debug line names the **ingress interface**, the source and destination, and the verdict. Policy match followed by policy routed with a `g=` gateway means the `set` worked. Policy rejected (no match) means the packet matched no permit clause (here the ICMP from 10.20.9.9 isn't in the web ACL) and got normal routing. If you see policy match but never policy routed, the next hop is unusable and the packet fell through. Run it with a **conditional debug** or an ACL on a busy router (32.3).

Unlike redistribution, PBR counters are real. `Policy routing matches` counts packets, so if it isn't increasing while users test, the packets aren't matching.

Symptom	Cause	Command that proves it
Counters stay at zero	Policy on the wrong interface or direction (it must be on the **ingress** interface)	`show ip policy`
Counters rise, but traffic still follows the routing table	Next hop unreachable or not adjacent, so it falls through	`show route-map` (track state), `show ip arp <next-hop>`
Wrong traffic is policy-routed	ACL too broad, or a deny clause missing	`show access-lists GUEST-WEB` hit counts
Router's own traffic ignores the policy	Needs `ip local policy`	`show ip policy` lists `local`
Policy keeps sending to a dead upstream	Plain `set ip next-hop` with no tracking	Add `verify-availability ... track`
Return traffic takes a different path	PBR is one-way; the return path follows normal routing (asymmetric, may break stateful firewalls/NAT)	`traceroute` from the far side

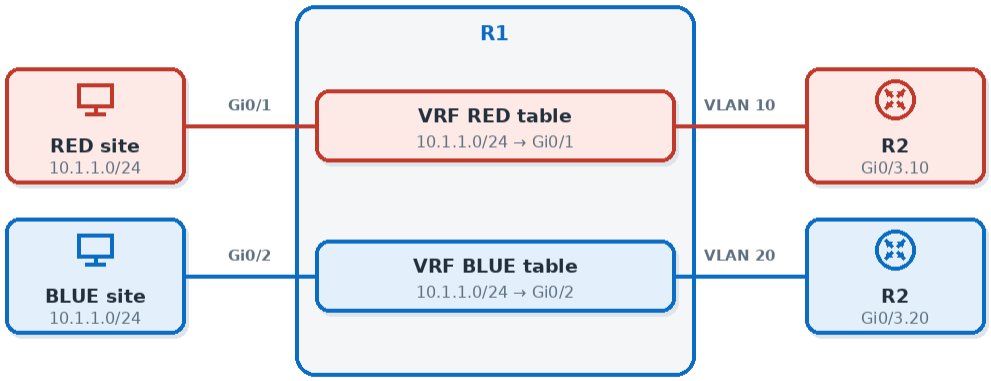
Worked ticket — "guest traffic still goes over MPLS."

- **Symptom:** after the change, guest web traffic still appears on the MPLS link.
- **Command:** `show route-map PBR-GUEST` → `Policy routing matches: 0 packets, 0 bytes`, and the count doesn't move while a guest browses.
- **Command:** `show ip policy` → `Gi0/0 PBR-GUEST`. Gi0/0 is the **outbound** WAN interface, not the guest VLAN's gateway.
- **Cause:** the policy was applied where guest packets leave the router. PBR only examines packets arriving on the interface, so it never sees them.
- **Fix:** `interface Gi0/0` → `no ip policy route-map PBR-GUEST`; `interface Gi0/2.20` → `ip policy route-map PBR-GUEST`.
- **Verify:** the match counter climbs while a guest browses, and `traceroute` from a guest host shows 203.0.113.1 as the first hop after R1.

Why is asymmetry such a common PBR side effect? Because PBR changes only **this router's** decision for **this direction**. The reply comes back through routers that know nothing about your policy, so it follows their routing tables. A firewall that saw only the outbound half of a TCP session will drop the reply. Every PBR design needs an answer to "and how does the reply come home?"

25.2 VRF-Lite — Several Routers Inside One

VRF-Lite — one router, separate tables, overlapping addresses allowed



a RED packet is only ever looked up in RED's table — no route, no leak

A **VRF** (Virtual Routing and Forwarding instance) gives a router a completely separate **routing table, CEF table and set of interfaces** for each customer, department or security zone. Chapter 12 introduced the concept. **VRF-Lite** means using VRFs without MPLS: each VRF is carried hop by hop over its own interfaces or 802.1Q subinterfaces, with its own routing protocol instance. (VRFs with MPLS — L3VPNs — are Chapter 29.)

Why not just use ACLs to separate departments? Because ACLs keep traffic apart by filtering a shared routing table, so one mistake in one line leaks everything. VRFs keep traffic apart by **construction**: a packet arriving on a RED interface can only be looked up in RED's table, and RED's table contains no route to BLUE. There's nothing to filter, because there's no path. VRFs

also allow **overlapping addresses** — RED and BLUE can both use 10.1.1.0/24 — which ACLs can never handle. ACLs say "you may not go there"; VRFs mean "there" doesn't exist.

Story Time : A VRF router is like an office building shared by two companies, each with its own **internal phone directory**. A call from a RED desk is looked up only in RED's directory. Both companies can have an extension 4001 without confusion, because nobody ever looks in the other company's book. Calls can only cross between companies if someone deliberately writes an entry in one book that points into the other — that's route leaking. Where the analogy stops: the building's shared lifts and power (the router's CPU and memory) are still shared, so VRFs separate forwarding, not the device itself.

How a VRF lookup works, step by step. Using the config below, a packet arrives on Gi0/3.10 with destination 10.1.1.5:

1. The router checks which VRF the **ingress interface** belongs to. Gi0/3.10 has `vrf forwarding RED`, so this packet belongs to RED. The packet itself carries no VRF label; the interface decides.
2. The router looks up 10.1.1.5 **only in RED's FIB**. It finds 10.1.1.0/24 connected on Gi0/1.
3. It ARPs (in RED's ARP table) and sends the packet out Gi0/1.
4. A packet with the same destination, 10.1.1.5, arriving on Gi0/3.20 is looked up in **BLUE's** table instead, and leaves by Gi0/2 to a completely different host.
5. A packet arriving on a global-table interface for 10.1.1.5 finds no route at all (the global table holds no 10.1.1.0/24 at all), and is dropped unless a default route sends it elsewhere.

Same router, same destination address, three different outcomes — decided entirely by the interface the packet came in on.

Configuring VRFs and Interfaces

```
vrf definition RED
 rd 65000:1                          ! optional for pure VRF-Lite; required for BGP in the VRF
 address-family ipv4
 exit-address-family
 address-family ipv6                 ! the multiprotocol syntax supports both families
 exit-address-family
!
vrf definition BLUE
 rd 65000:2
 address-family ipv4
 exit-address-family
!
interface GigabitEthernet0/1
 vrf forwarding RED                  ! assign FIRST...
 ip address 10.1.1.1 255.255.255.0 ! ...then re-add the IP (assigning wipes it)
!
interface GigabitEthernet0/2
 vrf forwarding BLUE
 ip address 10.1.1.1 255.255.255.0 ! the same address — legal, different table
!
interface GigabitEthernet0/3.10    ! one trunk to the next router carries both VRFs
 encapsulation dot1Q 10
 vrf forwarding RED
 ip address 172.16.0.1 255.255.255.252
interface GigabitEthernet0/3.20
 encapsulation dot1Q 20
 vrf forwarding BLUE
 ip address 172.16.0.5 255.255.255.252
```

Exam trap: putting an interface into a VRF **removes its IP address**. IOS warns you (`% Interface GigabitEthernet0/1 IPv4 disabled and address(es) removed due to enabling VRF RED`), but in a simulation it's easy to miss. An interface that "lost its IP" after a VRF change is behaving as designed.

You'll also meet the older IPv4-only syntax: `ip vrf RED` and `ip vrf forwarding RED` on the interface. It works the same for IPv4. `vrf definition` is the current form and supports IPv6.

Routing Protocols per VRF

Each VRF needs its own routing. Every protocol has a VRF-aware form:

```
! Static
ip route vrf RED 0.0.0.0 0.0.0.0 172.16.0.2

! OSPF — a separate process per VRF
router ospf 10 vrf RED
 router-id 1.1.1.10
 network 10.1.1.0 0.0.0.255 area 0
 network 172.16.0.0 0.0.0.3 area 0

! EIGRP classic — an address family per VRF under one process
router eigrp 100
 address-family ipv4 vrf BLUE autonomous-system 20
  network 10.1.1.0 0.0.0.255
  network 172.16.0.4 0.0.0.3
 exit-address-family

! EIGRP named mode
router eigrp CORE
 address-family ipv4 unicast vrf BLUE autonomous-system 20
  network 10.1.1.0 0.0.0.255

! BGP — an address family per VRF (the VRF must have an RD)
router bgp 65000
 address-family ipv4 vrf RED
  neighbor 172.16.0.2 remote-as 65001
  neighbor 172.16.0.2 activate
  redistribute connected
```

`capability vrf-lite`. When a VRF-Lite router's OSPF process connects to an MPLS provider edge, the PE sets the **DN (down) bit** on the Type 3, 5 and 7 LSAs it originates toward the customer. It also tags the Type 5s with a **domain tag** derived from the BGP AS. Both are loop-prevention marks for MPLS VPNs: a PE that receives an LSA with the DN bit set (or its own domain tag) must not use it. A VRF-Lite router behaving like a PE would ignore them too, and routes would go missing. `capability vrf-lite` under `router ospf 10 vrf RED` turns those PE checks off. Remember it as: "OSPF in a VRF, routes from the provider missing → `capability vrf-lite`."

VRF-Aware Verification — Everything Needs `vrf`

The most common VRF mistake isn't a config error. It's **testing in the wrong table**. A plain `ping 10.1.1.5` uses the **global** table, which has no route to anything in RED:

Task	Command
List VRFs and their interfaces	`show vrf` / `show vrf detail` / `show ip vrf interfaces`

Task	Command
Routing table of one VRF	`show ip route vrf RED`
Test reachability inside a VRF	`ping vrf RED 10.1.1.5` / `traceroute vrf RED 10.1.1.5`
ARP / CEF inside a VRF	`show ip arp vrf RED` / `show ip cef vrf RED`
Per-VRF protocol state	`show ip ospf 10 neighbor`, `show ip eigrp vrf BLUE neighbors`, `show ip bgp vpnv4 vrf RED summary` (or `show bgp vpnv4 unicast vrf RED summary`)
Management access into a VRF	`ssh -vrf MGMT -l admin 10.9.9.9`

```
R1# show vrf
  Name                             Default RD            Protocols   Interfaces
  BLUE                             65000:2               ipv4        Gi0/2
                                                                     Gi0/3.20
  RED                              65000:1               ipv4,ipv6   Gi0/1
                                                                     Gi0/3.10
```

Worked ticket — "RED's server is unreachable" (it isn't).

```
R1# ping 10.1.1.5
.....
Success rate is 0 percent (0/5)
R1# show ip route 10.1.1.5
% Network not in table
R1# show ip route vrf RED 10.1.1.5
Routing entry for 10.1.1.0/24
  Known via "connected", distance 0, metric 0 (connected, via interface)
R1# ping vrf RED 10.1.1.5
!!!!!
Success rate is 100 percent (5/5)
```

- **Cause:** the engineer tested in the global table. The route only exists in RED.
- **Lesson:** in a VRF network, every `ping`, `traceroute`, `show ip route`, `show ip arp` and `show ip cef` needs the `vrf` keyword, or it answers a different question from the one you asked.

Route Leaking — Controlled Holes Between VRFs

Sometimes separate VRFs must share something: every VRF needs the shared DNS servers or the internet edge. **Route leaking** imports specific routes from one table into another.

With static routes (simple, small scale): a static route in one table can point to a next hop in another, as long as you **name the exit interface**, or use the `global` keyword for a next hop in the global table:

```
! RED reaches shared services (192.168.100.0/24) in the global table
ip route vrf RED 192.168.100.0 255.255.255.0 192.168.0.1 global
! Global table returns traffic to RED's subnet via RED's interface
ip route 10.1.1.0 255.255.255.0 GigabitEthernet0/1 10.1.1.2
```

With BGP route targets (scalable): even on a single router, VRFs can export routes tagged with a **route target** and import other VRFs' targets. BGP does the copying, so it must be running, with

an address family for each VRF:

```
vrf definition RED
 address-family ipv4
  route-target export 65000:1
  route-target import 65000:1
  route-target import 65000:100        ! also import SHARED's routes
vrf definition SHARED
 rd 65000:100
 address-family ipv4
  route-target export 65000:100
  route-target import 65000:100
  route-target import 65000:1          ! and send RED's routes back the other way
!
router bgp 65000
 address-family ipv4 vrf RED
  redistribute connected
 address-family ipv4 vrf SHARED
  redistribute connected
```

How the route-target leak works, step by step:

1. BGP takes RED's connected routes (`redistribute connected` under `address-family ipv4 vrf RED`) and marks them with RED's export target, **65000:1**.
2. It does the same for SHARED's routes, marking them **65000:100**.
3. For each VRF, BGP copies in every route whose target matches one of that VRF's `import` lines. RED imports 65000:1 (its own) and **65000:100**, so SHARED's subnets appear in RED's table. SHARED imports 65000:100 and **65000:1**, so RED's subnets appear in SHARED's table.
4. The leaked routes appear in the importing VRF's table with the `B` code, pointing out the other VRF's interface. `show ip route vrf RED` therefore lists SHARED's subnets as `B` routes, and `show ip route vrf SHARED` lists RED's.

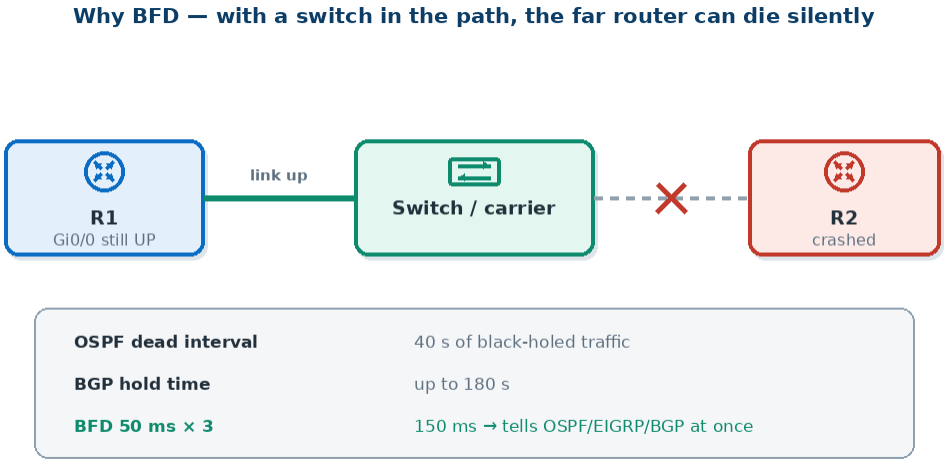
Remove the `import 65000:1` from SHARED and step 3 only works one way: RED can send to SHARED, but SHARED's replies have no route back.

Why must leaking always be configured in both directions? For the same reason PBR is one-way: each table answers only for its own lookups. If RED learns how to reach SHARED but SHARED has no route back to RED, requests arrive and replies vanish. **Leaking also reintroduces the overlap problem**: if RED and BLUE both use 10.1.1.0/24 and both leak into SHARED, SHARED cannot tell them apart. Every hole you punch between VRFs needs a return path and unique addresses on both sides.

Symptom	Cause	Command that proves it
Interface lost its IP	`vrf forwarding` applied after the address	`show ip interface brief`; re-add the IP
Ping fails, but routes look fine	Pinged in the global table	`ping vrf RED ...`
VRF neighbor never forms	Routing process not in the VRF, or neighbor's subinterface in a different VRF / VLAN	`show ip ospf 10 interface brief`, `show vrf` on both sides
BGP VRF address family rejected	VRF has no RD	`show vrf detail`
One-way connectivity across a leak	Return leak missing	`show ip route vrf SHARED <src-subnet>`

Symptom	Cause	Command that proves it
OSPF routes from an MPLS provider missing in VRF	DN-bit check	`capability vrf-lite`
EIGRP in a VRF sends no hellos at all	Classic-mode `address-family ipv4 vrf BLUE` entered **without** `autonomous-system 20` (or a different AS than the neighbour)	`show ip eigrp vrf BLUE interfaces` is empty; `show run | section router eigrp`

25.3 BFD — Detecting Failure in Milliseconds



Every routing protocol detects a dead neighbor by **missing hellos**: OSPF waits 40 seconds by default (the dead interval), EIGRP 15 seconds (hold time), BGP up to 180 seconds (hold time). For all that time, the router keeps forwarding into a black hole.

You might think a failed link would be noticed instantly, and if the cable is plugged straight into the router, it is: the interface goes down. But most real links run **through something**: a switch, a provider's Ethernet service, a DWDM system. When the far router dies, your interface to the switch **stays up**. Only the missing hellos reveal it.

Story Time : Imagine checking that a friend on a hike is okay by waiting for their full trip report every 10 minutes, and only worrying after 40 minutes of silence. That's routing-protocol hellos: rich messages, slow detection. BFD is a radio click every fifty milliseconds: "still here." It carries no news at all, just proof of life, so it can be sent constantly and cheaply. Miss three clicks in a row and you raise the alarm in 150 ms, then let the routing protocols deal with the consequences.

Bidirectional Forwarding Detection (BFD) is a tiny, dedicated liveness protocol (RFC 5880/5881). It runs **one** lightweight session between two neighbors, exchanging packets every few tens of milliseconds, and notifies any routing protocol that registered with it the moment the session fails.

Why not just tune each protocol's hello timers down? Three reasons. First, **CPU**: routing hellos are processed by the control plane, and sub-second hellos from several protocols on many interfaces add up. BFD is designed to be cheap, and many platforms run it in hardware or a distributed data plane. Second, **one session serves everyone**: OSPF, EIGRP, BGP, static routes and HSRP can all use the same BFD session, instead of each running its own fast timers. Third,

uniformity: BGP can't go below a 3-second hold time, but BFD gives BGP the same sub-second detection as everything else. BFD separates "is my neighbor alive?" from "what routes does my neighbor have?" — and answers the first question once, fast, for everyone.

How BFD Works

Item	Detail
Asynchronous mode	The standard mode: each side sends control packets at the negotiated interval; if multiplier packets in a row are missed, the session goes down
Echo mode	Each side sends echo packets **addressed to itself**; the neighbor's forwarding plane simply loops them back. Tests the real data path without the neighbor's CPU. On by default on Cisco when async mode runs; control packets then slow down (`bfd slow-timers`)
Transport	UDP **3784** (single-hop control), **3785** (echo), **4784** (multihop, e.g. eBGP multihop)
Detection time	interval × multiplier — e.g. 50 ms × 3 = **150 ms**
What BFD does on failure	Tells its **clients**; it never changes routing itself. The client tears down the adjacency and reconverges

How a failure is handled, step by step. R1 and R2 are OSPF neighbours through a provider switch, with BFD at 50 ms × 3 and OSPF registered as a client. R2 loses power at time 0.

Time	Without BFD	With BFD
0 ms	R2 dies. R1's Gi0/0 stays **up** (the switch port is fine)	Same
50, 100, 150 ms	Nothing noticed	R1 expects a BFD packet every 50 ms and gets none
~150 ms	Nothing noticed	3 misses → BFD session **Down**. BFD tells its registered clients: here OSPF (and, if configured, a BFD-tracked static route)
~150 ms +	Nothing noticed	OSPF immediately drops the R2 adjacency, floods a new router LSA, runs SPF and installs routes via the backup path
40 s	OSPF dead timer expires; only now does R1 drop the adjacency and reconverge	Long since rerouted

The traffic lost drops from about 40 seconds' worth to well under a second's.

Worked example — negotiated timers. The two ends don't have to be configured the same. (This example assumes echo mode is off, e.g. `no bfd echo` on the interface; with echo mode on,

detection runs on the echo packets as shown in the output below.) Say R1 has `bfd interval 50 min_rx 50 multiplier 3` and R2 has `bfd interval 100 min_rx 200 multiplier 3`.

1. Each router sends no faster than the other side is willing to receive. R1 wants to send every 50 ms, but R2 says "not faster than 200 ms" (`min_rx 200`), so **R1 sends every 200 ms**. R2 wants to send every 100 ms and R1 accepts 50, so **R2 sends every 100 ms**.
2. Each router's detection time is the **remote multiplier × the rate the remote is actually sending**. R1 detects R2 failing after 3 × 100 = **300 ms**. R2 detects R1 failing after 3 × 200 = **600 ms**.

So detection can differ in each direction, and the side with the slow `min_rx` slows down the other side's transmissions. When BFD detection looks slower than you configured, check the neighbour's `min_rx` in `show bfd neighbors details` (`Received MinRxInt`).

Configuring BFD

BFD timers go on the **interface**, and each routing protocol then **registers** as a client:

```
interface GigabitEthernet0/0
 bfd interval 50 min_rx 50 multiplier 3   ! send every 50 ms, expect every 50 ms, 3 misses
 no ip redirects                          ! echo mode requires this; IOS warns otherwise
!
router ospf 1
 bfd all-interfaces                       ! or per interface: ip ospf bfd
!
router eigrp 100                          ! classic mode
 bfd interface GigabitEthernet0/0         ! or: bfd all-interfaces
!
router eigrp CORE                         ! named mode
 address-family ipv4 unicast autonomous-system 100
  af-interface GigabitEthernet0/0
   bfd
!
router bgp 65001
 neighbor 10.0.12.2 fall-over bfd         ! BGP: per neighbor
!
ip route static bfd GigabitEthernet0/0 10.0.12.2      ! static: track the next hop...
ip route 10.50.0.0 255.255.0.0 GigabitEthernet0/0 10.0.12.2  ! ...for this route
```

HSRP can use the same session. On IOS releases that support it, HSRP registers with BFD automatically once the interface has BFD timers (`standby bfd` is on by default; `no standby bfd` turns it off), and `show standby neighbors` shows whether BFD is up toward each peer.

Both neighbors need BFD configured, or the session never comes up. Timers are **negotiated**: each side sends at the slower of its own `interval` and the other side's `min_rx`.

```
R1# show bfd neighbors

IPv4 Sessions
NeighAddr                       LD/RD         RH/RS     State     Int
10.0.12.2                        1/1          Up        Up        Gi0/0

R1# show bfd neighbors details
...
Session state is UP and using echo function with 50 ms interval.
OurAddr: 10.0.12.1
Local Diag: 0, Demand mode: 0, Poll bit: 0
MinTxInt: 1000000, MinRxInt: 1000000, Multiplier: 3
Received MinRxInt: 1000000, Received Multiplier: 3
Holddown (hits): 0(0), Hello (hits): 1000(84)
Rx Count: 84, Rx Interval (ms) min/max/avg: 1/1008/880 last: 236 ms ago
Registered protocols: OSPF EIGRP CEF
Uptime: 00:01:12
```

Notice what echo mode did. Detection is running on **echo packets every 50 ms**, so the **control** packets dropped to 1,000,000 µs (1 s, the `MinTxInt`/`MinRxInt` values are in microseconds), because they only need to keep the session alive. Without echo mode those fields would show 50000 (50 ms). `Holddown (hits)` counts how often the detection timer nearly expired. A climbing number means the timers are too tight for the path.

`LD/RD` are the local and remote **discriminators** (session IDs), `RH/RS` is the remote side's state, and `Registered protocols` shows which clients will be told when the session fails. **If your protocol isn't in that list, BFD failing won't help it.**

Symptom	Cause	Command that proves it
No BFD session at all	BFD configured on one side only, or no client registered	`show bfd neighbors` on both routers
Session up, but the protocol still waits for its dead timer	Protocol not registered (`bfd all-interfaces` / `fall-over bfd` missing)	`show bfd neighbors details` — Registered protocols
Session flaps constantly	Timers too aggressive for the platform or a busy CPU	`show bfd neighbors details` — Holddown hits; raise the interval
Echo mode not running	`ip redirects` on, or not supported by the peer	Warning in the log; `show bfd neighbors details`

Templates and Multihop BFD

Why templates and multihop exist: on a router with dozens of interfaces, typing the same timers everywhere invites mismatches, so a template defines them once. And some neighbours aren't on a shared link at all — eBGP between loopbacks, say — so there's no single interface to hang BFD on. On IOS XE, timers can be packaged in a **template** and reused. Multihop sessions (eBGP between loopbacks, for example) must use a template, and they're mapped by address pair instead of attached to an interface:

```
bfd-template single-hop FAST
 interval min-tx 50 min-rx 50 multiplier 3
!
interface GigabitEthernet0/0
 bfd template FAST                        ! replaces the per-interface bfd interval line
!
bfd-template multi-hop WAN-MH
 interval min-tx 300 min-rx 300 multiplier 3
bfd map ipv4 10.9.9.9/32 10.1.1.1/32 WAN-MH   ! destination, source, template
!
router bgp 65001
 neighbor 10.9.9.9 update-source Loopback0
 neighbor 10.9.9.9 ebgp-multihop 2
 neighbor 10.9.9.9 fall-over bfd multi-hop
```

Multihop BFD uses UDP **4784** and doesn't support echo mode. There's no single neighbour to loop the packets back, and the path can change underneath the session. Use more conservative timers than single-hop.

Exam framing: ENARSI 1.8 says **describe**, not configure. Expect questions on why BFD exists, async vs echo, how detection time is calculated, and which protocols can use it. The configuration above is what you'll type in a lab and what turns those descriptions into something you can verify.

Chapter 26: EIGRP In Depth

> **Blueprint:** ENARSI 1.9 — Troubleshoot EIGRP (classic and named mode; VRF and global): 1.9.a address families (IPv4, IPv6) · 1.9.b neighbor relationship and authentication · 1.9.c loop-free path selection (RD, FD, FC, successor, feasible successor, stuck in active) · 1.9.d stubs · 1.9.e load balancing (equal and unequal cost) · 1.9.f metrics

Chapter 5 compared EIGRP with OSPF at the concept level: what an "advanced distance vector" protocol is, and why EIGRP converges fast. This chapter is the ENARSI version. You'll configure EIGRP **both ways IOS XE allows**, for **IPv4 and IPv6**, in the **global table and inside a VRF**. You'll work DUAL through with real numbers, and you'll learn to find the one broken thing in an EIGRP network quickly.

The ENARSI mindset for this chapter: nearly every EIGRP question is a troubleshooting question in disguise. A neighbor won't form, a route is missing, a backup path isn't used, a router went stuck-in-active. Each of those has a short list of causes, and each cause has a command that proves it. That's the shape of everything below.

Words you'll meet in this chapter (each is explained properly where it first matters):

Term	Plain meaning
EIGRP	Enhanced Interior Gateway Routing Protocol: Cisco's advanced distance-vector protocol (now an open RFC, 7868)
AS number	in EIGRP, just a tag that says "these routers belong to the same EIGRP instance". It must match between neighbors
DUAL	Diffusing Update Algorithm: the maths EIGRP uses to pick a loop-free best path and backup
RTP	Reliable Transport Protocol: EIGRP's own way of making sure updates, queries and replies are acknowledged
K values	the five (or six) weights that decide which ingredients go into the metric
AD	administrative distance: how much the router trusts a source of routes (internal EIGRP 90, external 170)
RIB	Routing Information Base: the routing table you see in `show ip route`
VRF	Virtual Routing and Forwarding: a separate routing table inside the same router (Chapter 25)
SIA	stuck-in-active: a route whose recomputation never finishes because a reply never arrives

26.1 Two Ways to Configure the Same Protocol: Classic vs Named Mode

The problem: you'll inherit networks configured by different people in different decades. Some routers will use the old style, some the new, and an exam exhibit can show either. You need to read both fluently and know where each setting lives, or you'll miss the one wrong line.

IOS has two configuration styles for EIGRP. Both run the **same protocol on the wire**, so a classic-mode router and a named-mode router become neighbors happily. The exam expects you to read and write both.

Aspect	Classic mode	Named mode
Starts with	`router eigrp 100` (the AS number)	`router eigrp CORP` (any name)
AS number lives	in the `router` command	in `address-family … autonomous-system 100`
IPv6	separate `ipv6 router eigrp 100` process	another `address-family ipv6` in the **same** process
Per-interface settings	scattered across interfaces (`ip hello-interval eigrp 100 …`)	collected under `af-interface` inside the process
Per-topology settings	process level (`variance`, `maximum-paths`)	under `topology base`
Metrics	32-bit classic metric	**64-bit wide metrics** (26.10)
HMAC-SHA-256 auth	not available	available

Why did Cisco invent named mode if classic still works? Because classic EIGRP's configuration grew by accretion. Each new feature added another interface command with the AS number embedded in it, and IPv6 needed an entirely separate process. Troubleshooting meant hunting through every interface stanza to rebuild the picture in your head. Named mode puts **everything about one EIGRP instance in one hierarchical block**: the process, then the address family, then its interfaces and its topology. One `show run | section router eigrp` tells you the whole story. It's also where all new features land: wide metrics and SHA-256 authentication exist only there.

Story Time : classic mode is like a house where the instructions for each room are taped to that room's door. To know how the house works, you walk every room. Named mode is one binder with a tab per room. The house is identical; only the paperwork moved. That's why a classic router and a named router peer without any problem: the packets on the wire (the "house") are the same.

The same network, configured both ways

R1 runs EIGRP AS 100 on `10.0.0.0/8`. Gi0/0 faces a LAN with no routers on it, so it should be passive. Gi0/1 faces R2.

```
! ---- Classic mode ----
router eigrp 100
 eigrp router-id 1.1.1.1
 network 10.0.0.0 0.255.255.255          ! enable EIGRP on interfaces in 10/8
 passive-interface GigabitEthernet0/0    ! advertise the LAN, but no hellos there
 variance 2
!
interface GigabitEthernet0/1
```

```
! ---- Named mode ----
router eigrp CORP
 address-family ipv4 unicast autonomous-system 100
  eigrp router-id 1.1.1.1
  network 10.0.0.0 0.255.255.255
  af-interface default
   passive-interface                    ! passive everywhere by default...
  exit-af-interface
  af-interface GigabitEthernet0/1
   no passive-interface                 ! ...except where a neighbor lives
   hello-interval 5
   hold-time 15
  exit-af-interface
  topology base
   variance 2                           ! routing-table behavior lives here
  exit-af-topology
 exit-address-family
```

The three sub-modes of named mode are worth memorizing, because questions show you a config and ask what's wrong with it:

Sub-mode	What goes there	Examples
`address-family`	Things about the instance	`network`, `eigrp router-id`, `eigrp stub`, `metric weights`
`af-interface`	Things about one interface	`hello-interval`, `hold-time`, `passive-interface`, `authentication`, `split-horizon`, `summary-address`
`topology base`	Things about the routing table	`variance`, `maximum-paths`, `redistribute`, `distance`, `distribute-list`, `offset-list`

Exam trap: in named mode, `af-interface default` applies to every interface, and a specific `af-interface` overrides it. The "passive by default, unpassive the uplinks" pattern above is common. If an adjacency won't form, check whether that interface ever got its `no passive-interface`.

Verification commands differ slightly too. Classic uses `show ip eigrp …`. Named mode supports those and also the address-family form:

```
R1# show eigrp address-family ipv4 neighbors
R1# show eigrp address-family ipv4 topology
R1# show eigrp protocols                  ! K values, router-ID, stub status, AD
```

26.2 EIGRP for IPv6

The problem: most enterprises are dual-stack now, so the IPv6 prefixes need a routing protocol just as the IPv4 ones do. EIGRP can carry IPv6, but the way an interface joins, the addresses neighbors use and the router-ID rule all differ from IPv4. Each difference is a classic exam fault.

Classic mode runs IPv6 as a separate process, and it is enabled **per interface**. There's no `network` command:

```
ipv6 unicast-routing
!
ipv6 router eigrp 100
 eigrp router-id 1.1.1.1      ! REQUIRED if the router has no IPv4 address at all
 passive-interface GigabitEthernet0/0
!
```

Exam trap: the shut-down process. On older IOS releases a classic `ipv6 router eigrp 100` process was created in a **shutdown** state, and nothing happened until you entered `no shutdown` under it. The interfaces showed `ipv6 eigrp 100`, but there were no neighbours and no error. If a classic IPv6 EIGRP config "looks perfect" and nothing forms, check `show ipv6 protocols` and `show run | section ipv6 router eigrp` for a `shutdown` line.

```
ipv6 router eigrp 100
 no shutdown                  ! harmless where it's the default; essential where it isn't
```

Named mode adds an address family, and **every IPv6-enabled interface joins automatically**. You turn interfaces off rather than on:

```
router eigrp CORP
 address-family ipv6 unicast autonomous-system 100
  eigrp router-id 1.1.1.1
  af-interface GigabitEthernet0/2
   shutdown                    ! exclude this interface from EIGRP for IPv6
  exit-af-interface
 exit-address-family
```

Why does EIGRP for IPv6 still need a 32-bit router ID? Because the router ID is a protocol field, not an address. EIGRP uses it to tag external routes (so a router can recognize and reject its own redistributed routes). It's a 32-bit number in dotted-decimal form whatever address family is running. On a router with any IPv4 address, IOS picks one automatically. On an **IPv6-only** router there's nothing to pick from, so the process won't start until you set one.

Things that change for IPv6:

Detail	IPv4	IPv6
Hello destination	224.0.0.10	FF02::A
Hello source / next hop	interface IPv4 address	the **link-local** address
Neighbor must be on same subnet?	yes	no — link-local only
Show commands	`show ip eigrp …`	`show ipv6 eigrp …`

Why are IPv6 neighbors and next hops link-local? Because a link-local address always exists and is always on-link. That makes it the stable way to say "the router on the other end of this wire". It also explains a common puzzle: two IPv6 EIGRP routers can become neighbors even though their **global** prefixes don't match. The adjacency doesn't depend on those prefixes at all.

You can see the link-local addressing straight away in the neighbor table:

```
R1# show ipv6 eigrp neighbors
EIGRP-IPv6 Neighbors for AS(100)
H   Address                 Interface       Hold Uptime   SRTT   RTO  Q  Seq
                                            (sec)         (ms)       Cnt Num
0   Link-local address:     Gi0/1             12 00:05:11    5   100  0  8
    FE80::2
```

The neighbor is listed by its **FE80::** address, not by 2001:db8:12::2, and the IPv6 EIGRP routes in `show ipv6 route eigrp` point at `FE80::2` as their next hop.

Worked example — the IPv6-only branch. A new branch router, R7, has only IPv6 addresses. The engineer configures `ipv6 router eigrp 100` and `ipv6 eigrp 100` on the uplink, then waits. Nothing happens:

1. `show ipv6 eigrp neighbors` is empty on both R7 and the hub.
2. `show ipv6 eigrp interfaces` on R7 shows no interfaces, so the process isn't running at all; this isn't a neighbor problem.
3. `show run | section ipv6 router eigrp` shows no `eigrp router-id`, and `show ip interface brief` shows no IPv4 address anywhere on R7.
4. **Cause:** with no IPv4 address to borrow, there's no router ID, so the process can't start.
5. **Fix:** `eigrp router-id 7.7.7.7` under the process (and `no shutdown` on releases where the process starts shut). The neighbor comes up within a hello interval.

26.3 EIGRP Inside a VRF

The problem: one router often has to keep two groups of networks completely apart: two customers, a guest network and a corporate one, or two companies mid-merger. They might even use the same addresses. VRF-Lite (Chapter 25) gives a router several separate routing tables. Each table still needs its routes from somewhere, and EIGRP is one way to fill it. To run EIGRP inside one of them, you add a VRF address family.

Story Time : think of a building with two companies on different floors. Each floor has its own mailroom, its own address list and its own staff. Mail for "Room 101" on floor 2 never goes to "Room 101" on floor 5. A VRF is a floor; the EIGRP instance inside it is that floor's mailroom, talking only to the mailrooms of the same company in other buildings. **The syntax is nearly identical in classic and named mode.** That's a frequent source of exam confusion, so here are both together:

```
vrf definition CUST_A
 address-family ipv4
!
interface GigabitEthernet0/3
 vrf forwarding CUST_A                    ! (re-enter the IP afterwards — this clears it)
 ip address 172.16.1.1 255.255.255.0
```

```
! ---- Classic mode: VRF AF inside the numbered process ----
router eigrp 1
 address-family ipv4 vrf CUST_A autonomous-system 200
  network 172.16.0.0 0.0.255.255
 exit-address-family
```

```
! ---- Named mode ----
router eigrp CORP
 address-family ipv4 unicast vrf CUST_A autonomous-system 200
  network 172.16.0.0 0.0.255.255
 exit-address-family
```

Why does the VRF address family need its own `autonomous-system`? Because the classic process number (`router eigrp 1`) is only the global table's AS. The VRF instance is a separate EIGRP instance that sends its own hellos with its own AS number, and the router at the other end

must match **that** number. Leave it out and the address family never sends hellos at all. Process 1 and AS 200 coexisting on one router is normal: one table, one instance, one AS each.

Verify with the VRF keyword everywhere:

```
R1# show ip eigrp vrf CUST_A neighbors
R1# show ip eigrp vrf CUST_A topology
R1# show ip route vrf CUST_A eigrp
```

Worked example — the VRF neighbor that never appears. R1 and the customer edge router CE1 should peer in CUST_A. `show ip eigrp vrf CUST_A neighbors` is empty on R1.

1. **Is the interface in the VRF?** `show ip vrf interfaces` lists Gi0/3 under CUST_A with 172.16.1.1. Good.
2. **Can the two routers reach each other in that table?** `ping vrf CUST_A 172.16.1.2` succeeds, so Layer 1–3 is fine. (A plain `ping 172.16.1.2` would fail, because it looks in the global table. That's a trap of its own.)
3. **Does the instance match?** R1 has `address-family ipv4 vrf CUST_A autonomous-system 200`. CE1 has `router eigrp 100` with the 172.16 network under it.
4. **Cause:** AS 200 versus AS 100. EIGRP ignores hellos from a different AS without logging anything.
5. **Fix:** make the numbers agree (change CE1 to `router eigrp 200`, or R1's address family to `autonomous-system 100`). The neighbor appears in `show ip eigrp vrf CUST_A neighbors`.

26.4 Neighbor Relationships — Why Adjacencies Fail

The problem: no neighbor means no routes. Almost every "EIGRP is broken" ticket starts here, so you need to know exactly what two routers check before they trust each other, and which mismatches are fatal.

EIGRP neighbors find each other with hellos to **224.0.0.10** (IP protocol **88**). They then exchange the full topology using RTP, the Reliable Transport Protocol: updates, queries and replies are acknowledged. After that, they send only changes.

How an adjacency forms, step by step (R1 10.1.12.1 and R2 10.1.12.2 on Gi0/1):

1. Both routers multicast a **hello** to 224.0.0.10 every 5 seconds. Each hello carries the AS number, the K values, the sender's hold time and (if configured) an authentication digest.
2. R1 receives R2's hello and checks it: same AS? same K values? authentication valid? Is 10.1.12.2 in R1's own subnet on Gi0/1? If any check fails, the hello is dropped (some failures are logged, the AS mismatch isn't).
3. If everything passes, R1 adds R2 to its neighbor table and sends R2 an empty **update with the INIT flag**, by **unicast**. It's saying "I've seen you; let's sync".
4. The two routers exchange their **full topology tables** in unicast updates. Every update is **acknowledged** (that's RTP), and anything unacknowledged is retransmitted.
5. From then on they send only **changes** (partial, bounded updates), and hellos keep the neighbor alive. If no hello arrives within R2's advertised hold time (15 s), R1 declares R2 dead.

Step 2 is where most "no neighbor" faults live; step 4 is where the flapping-neighbor faults live (below).

Must match / must work	Doesn't have to match
AS number	Hello / hold timers
K values (metric weights)	Router ID (unique is best practice)
Authentication (mode, and the key in use)	Interface bandwidth / delay
Primary subnet on the shared link (IPv4)	Process name (named mode)
Interface **not passive** on either side	Classic vs named mode
Hellos actually delivered (ACLs, L2)	

Why don't EIGRP's hello and hold timers need to match, when OSPF's do? Because EIGRP puts its **hold time inside each hello**. The receiver then uses the number the sender chose. "Declare me dead if you hear nothing for 15 seconds" is a promise the speaker makes about itself, so neighbors can safely use different rhythms. OSPF instead treats the timers as a property of the shared link, and both ends must agree. Mismatched EIGRP timers don't block an adjacency, but they can make it flap. If R1 says hold 3 s but only sends a hello every 5 s, R2 drops R1 between hellos, over and over.

Story Time : two friends agree to check in by text. Alex says "I'll text every 5 minutes; if you hear nothing for 15, come looking for me". Sam says "I text every minute; worry after 3". Each friend judges the other by the other's own promise, so the different rhythms are fine. The only way it breaks is a promise the speaker can't keep: "worry after 3 minutes" from someone who texts every 5.

Default timers: **hello 5 s / hold 15 s** on most links, and **hello 60 s / hold 180 s** on NBMA interfaces at T1 speed or slower.

Symptom → cause → proof

Symptom	Likely cause	Command that proves it
No neighbor, no log message	AS mismatch (hellos silently ignored)	`show run | section eigrp` both sides; `debug eigrp packets hello`
`K-value mismatch` in the log	`metric weights` differ	`show ip protocols` — compare `K1=… K5=`
`Auth failure` / `authentication mode mismatch`	wrong key-string, key ID, or one side unauthenticated	`debug eigrp packets` shows `(invalid authentication)`
`not on common subnet` in the log	IPv4 addressing / mask error	`show ip int brief`, the log message itself
No neighbor; interface missing from list	`passive-interface`, or `network` didn't match	`show ip eigrp interfaces` (passive interfaces are absent)
Neighbor comes up, then is torn down after ~16 retransmissions (roughly a minute or more) and repeats	one-way path: ACL in one direction, unidirectional link, or unicast EIGRP blocked while multicast hellos pass	`retry limit exceeded` in the log; `show ip eigrp neighbors` — uptime keeps resetting, `Q Cnt` > 0; `show ip eigrp neighbors detail` (retransmit counts)
Neighbor on one side only, or no neighbour after adding a `neighbor` statement	**static `neighbor` command** on one end only (below)	`show run | section eigrp` both ends; `show ip eigrp neighbors detail` shows `Static neighbor`

Why does a one-way ACL produce a flapping **neighbor instead of no neighbor?** Because EIGRP hellos are multicast, but the reliable packets (the initial updates and their ACKs) are often **unicast**. If an ACL permits protocol 88 to 224.0.0.10 but blocks unicast EIGRP, or blocks traffic in one direction only, the hellos get through and the neighbor appears. The reliable exchange then fails, RTP retries **16 times**, and the neighbor is torn down with `retry limit exceeded`. Then it comes back, and the cycle repeats. A safe ACL entry is:

```
ip access-list extended EDGE_IN
 permit eigrp any any            ! protocol 88 — covers both multicast hellos and unicast
```

The Static `neighbor` Command — A Classic Self-Inflicted Fault

```
router eigrp 100
 neighbor 10.1.12.2 GigabitEthernet0/1      ! unicast hellos to this peer only
```

A static neighbour statement makes EIGRP send **unicast** hellos to that address, for links that don't carry multicast. The side effect is the trap: **on that interface, EIGRP stops sending and processing multicast hellos.** Configure it on R1 only, and R1 ignores R2's multicast hellos while R2 never hears from R1 at 224.0.0.10. Every other dynamic neighbour on R1's interface disappears too. **Both ends need matching `neighbor` statements**, and every router on that segment that should peer with R1 needs one.

Reading the neighbor table well is half the job:

```
R1# show ip eigrp neighbors
EIGRP-IPv4 Neighbors for AS(100)
H   Address         Interface   Hold Uptime    SRTT   RTO  Q  Seq
                                (sec)          (ms)       Cnt Num
0   10.1.12.2       Gi0/1         13 02:41:07    4   100  0  37
1   10.1.13.3       Gi0/2         11 00:00:42 1020  5000  4  12
```

- **H** is just the order the neighbors were learned (0 = first).
- **Hold** counting down from 15 and resetting is healthy.
- **Uptime** that keeps resetting to seconds is a flapping neighbor.
- **SRTT** (smooth round-trip time) is the average time, in ms, for this neighbor to acknowledge a reliable packet. **RTO** (retransmission timeout) is how long EIGRP waits for that ACK before resending; it's calculated from SRTT.
- A **Q Cnt** (queue count) that stays above 0 means packets are waiting for ACKs. Together with a huge **SRTT/RTO** (as on 10.1.13.3), that points to a lossy or one-way path.
- **Seq Num** is the sequence number of the last reliable packet received from that neighbor.

Worked example — the silent neighbor. After a change window, R1 has lost its neighbor R3 (10.1.13.3). There's nothing about it in R1's log except the original down message.

1. `show ip eigrp neighbors` on R1: 10.1.13.3 is missing. On R3: 10.1.13.1 is missing too. So both sides have rejected each other.
2. `show ip eigrp interfaces` on both: Gi0/2 is listed on both routers, so the interfaces are running EIGRP and aren't passive.
3. `ping 10.1.13.3` succeeds, so the link and addressing are fine.
4. No `K-value mismatch`, `Auth failure` or `not on common subnet` message in either log. That leaves the one mismatch that's silent: the **AS number**.
5. `show ip protocols` on R1: `EIGRP-IPv4 Protocol for AS(100)`. On R3: `EIGRP-IPv4 Protocol for AS(110)`. Someone retyped R3's process during the change.
6. **Fix:** put R3 back in AS 100. A `%DUAL-5-NBRCHANGE ... is up: new adjacency` message appears on both routers within seconds.

Method to remember: when an EIGRP neighbor is missing and nothing is logged, think AS number or passive interface first, because those two fail silently.

26.5 Authentication — MD5 and HMAC-SHA-256

The problem: EIGRP trusts any router that sends a valid hello. Anyone who plugs a router (or a laptop running routing software) into a LAN where EIGRP is active could become a neighbor and inject routes, pulling traffic toward themselves. Authentication makes a neighbor prove it knows a shared secret.

How it works: the sender runs each packet plus the shared key through a hash function (MD5 or SHA-256) and attaches the result, the **digest**. The receiver does the same calculation with its copy of the key. If the two digests match, the sender must have known the key, and the packet wasn't altered on the way. The key itself never crosses the wire.

EIGRP authentication means every packet carries a digest, so a rogue router can't inject routes. Both methods use a **key chain** (except the SHA password shortcut).

```
key chain EIGRP_KEYS
 key 1
  key-string S3cr3t!
  accept-lifetime 00:00:00 Jan 1 2026 infinite
  send-lifetime   00:00:00 Jan 1 2026 infinite
```

Classic mode — MD5 only:

```
interface GigabitEthernet0/1
 ip authentication mode eigrp 100 md5
 ip authentication key-chain eigrp 100 EIGRP_KEYS
```

Named mode — MD5 or HMAC-SHA-256, under the `af-interface`:

```
router eigrp CORP
 address-family ipv4 unicast autonomous-system 100
  af-interface GigabitEthernet0/1
   authentication mode md5
   authentication key-chain EIGRP_KEYS
  exit-af-interface
  af-interface GigabitEthernet0/2
   authentication mode hmac-sha-256 Str0ngP@ss    ! SHA with an inline password
   ! (or: authentication mode hmac-sha-256 + authentication key-chain NAME)
  exit-af-interface
```

Exam trap: the key ID matters for MD5. An EIGRP MD5 packet carries the **key number** of the key that signed it. The receiver looks up its own key with that number. If R1 sends with key 1 and R2's only key is key 2 with the same string, authentication fails. Matching strings aren't enough; the numbers must match too.

Why does a key chain have separate send and accept lifetimes? For key **rollover without an outage**. You add key 2 on every router with an accept lifetime that starts now, and a send lifetime that starts next week. For a while every router accepts both keys and sends key 1. Then, on schedule, every router switches to sending key 2, which everyone already accepts. Finally you remove key 1. If send and accept were the same window, there'd be a moment where one router signs with a key its neighbor won't accept yet. (Lifetimes depend on clocks, which is one more reason Chapter 8's NTP matters.)

Story Time : replacing the locks on an office. You don't change the lock first and hand out keys afterwards, because everyone would be locked out in between. You hand out the new key first, give the door a lock that opens with either key for a week, and only then retire the old key. Accept lifetime is "which keys the door opens for"; send lifetime is "which key people actually use".

Worked example — matching passwords, no neighbor. R1 and R2 both have the key string `S3cr3t!`, both use MD5, and the neighbor is down.

```
R1# show key chain
Key-chain EIGRP_KEYS:
    key 1 -- text "S3cr3t!"
        accept lifetime (always valid) - (always valid) [valid now]
        send lifetime (always valid) - (always valid) [valid now]

R2# show key chain
Key-chain EIGRP_KEYS:
    key 2 -- text "S3cr3t!"
        accept lifetime (always valid) - (always valid) [valid now]
        send lifetime (always valid) - (always valid) [valid now]
```

R1 signs with key **1**. R2 receives a packet saying "key 1", looks for its own key 1, finds none, and drops the packet. R2's packets say "key 2" and fail on R1 the same way. `debug eigrp packets` shows the authentication failures. **Fix:** make the key numbers agree on both routers (for example, renumber R2's key to 1). Adding key 2 to R1 alone isn't enough: R1 would accept R2's packets but

still sign its own with key 1, which R2 can't verify. The strings were never the problem.

Classic IPv6 EIGRP authenticates with the same key chain, configured with IPv6 interface commands:

```
interface GigabitEthernet0/1
 ipv6 authentication mode eigrp 100 md5
 ipv6 authentication key-chain eigrp 100 EIGRP_KEYS
```

In named mode the `af-interface` commands are identical under `address-family ipv6`.

Verify: `show key chain`, `show ip eigrp interfaces detail` (shows the auth mode and key chain per interface), and `debug eigrp packets` (look for `invalid authentication` or `key 1 not found`).

26.6 DUAL — Successors, Feasible Successors and the Feasibility Condition

The problem: a distance-vector router never sees the whole map. It only hears "I can reach X at cost N" from each neighbor. So how can it pick a backup path in advance and be certain that the backup doesn't lead straight back through itself? Old protocols like RIP couldn't; they waited, counted to infinity and looped in the meantime. DUAL (Diffusing Update Algorithm) solves this with one simple comparison.

DUAL is why EIGRP converges in milliseconds when a backup exists, and **guarantees loop-freedom** without a full map. It runs on four terms:

Term	Meaning
Reported Distance (RD)	The neighbor's own metric to the destination — what it tells you (also called advertised distance)
Feasible Distance (FD)	Your **best** total metric to the destination: neighbor's RD + your cost to reach that neighbor
Successor	The neighbor providing the FD — the route installed in the routing table
Feasible Successor (FS)	A backup neighbor whose **RD < FD** — a loop-free backup kept ready in the topology table

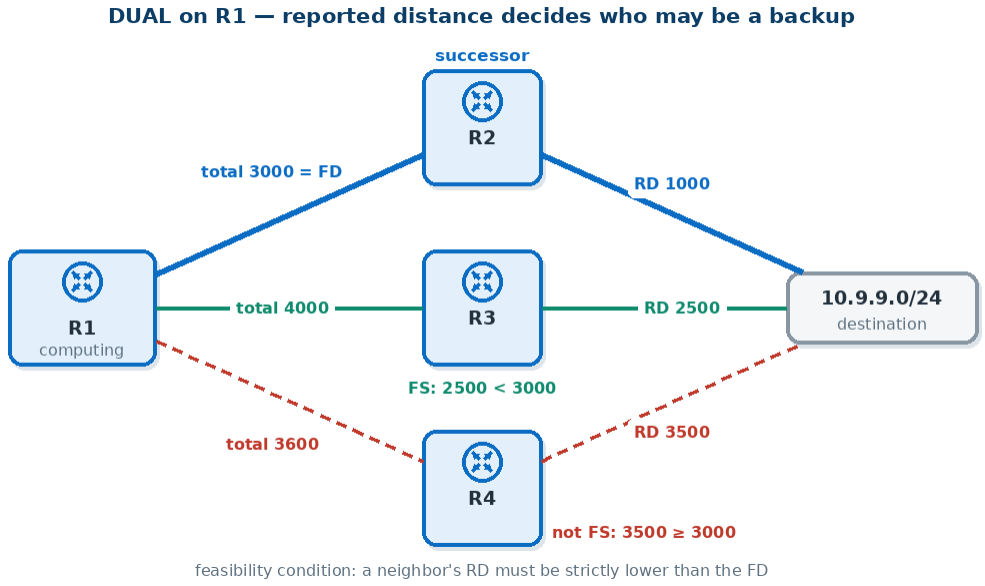
The rule in that last row is the **feasibility condition (FC)**: a neighbor qualifies as a backup only if its reported distance is strictly less than my current feasible distance.

Story Time : you're 30 minutes from the stadium and your usual road closes. Two friends phone in. Priya says "I'm 10 minutes from the stadium". Because she's **closer** to the stadium than you are, her route can't possibly come back past your house; if it did, she'd be at least 30 minutes away. Following her is guaranteed safe. Omar says "I'm 35 minutes away". Omar might have a fine route, but he's farther out than you, so his route could pass your house, and you'd be driving in a circle. You can't see his route, so you don't risk it without asking around first. Priya is a feasible successor; Omar isn't. (Where the analogy stops: EIGRP distance is a metric, not minutes, but the logic is identical.)

How DUAL picks the successor and backup, step by step:

1. For each neighbor, take the **RD** it advertised and add **your own cost to reach that neighbor**. That's the total via that neighbor.
2. The lowest total is the **FD**. The neighbor that gave it is the **successor**, and that route goes into the routing table.
3. For every other neighbor, compare its **RD** (not its total!) with the FD. RD < FD → **feasible successor**, kept ready. RD ≥ FD → not a backup, however good its total looks.
4. Ties: if several neighbors give the same lowest total, they're all successors and all are installed (equal-cost load balancing, 26.9).

Worked example



R1 has three paths to 10.9.9.0/24:

Via	RD (neighbor's metric)	R1's cost to neighbor	Total	Role
R2	1000	2000	**3000**	**Successor** — lowest total, so FD = 3000
R3	2500	1500	4000	**Feasible successor** — RD 2500 < FD 3000
R4	3500	100	3600	**Not** a feasible successor — RD 3500 ≥ 3000

Running the four steps on those numbers:

1. Totals: via R2 = 1000 + 2000 = **3000**; via R3 = 2500 + 1500 = **4000**; via R4 = 3500 + 100 = **3600**.
2. Lowest total is 3000, so **FD = 3000** and **R2 is the successor**.

3. R3: RD 2500 < 3000 → feasible successor. R4: RD 3500 is not < 3000 → rejected.

Look closely at R4. Its total (3600) is better than R3's (4000), yet R4 isn't a backup and R3 is. That's the whole point of the condition.

Why does "RD < FD" guarantee a loop-free backup? Because of what R4's RD means. If R4 is further from the destination than R1 is, R4's own best path **might run through R1**. R1 can't see R4's path; it only knows R4's distance. If that path does lead back through R1, then the moment R1 fails over to R4, packets bounce between them. A neighbor whose RD is smaller than my best distance is strictly **closer** to the destination than I am, so its path can't loop back through me. The condition is conservative: it rejects some paths that would actually have worked, like R4 here. In exchange, every path it accepts is provably safe, and that's what lets EIGRP fail over instantly with no checking.

What happens when the successor fails:

- **An FS exists** (R2 dies, R3 qualifies): R1 promotes R3 **immediately and locally**. There are no queries. The route stays **passive** and convergence takes well under a second.
- **No FS exists:** R1 marks the route **active** and **queries** every neighbor ("do you have a path to 10.9.9.0/24?"). It must wait for **every** reply before choosing a new successor.

Read the topology table to see all of this:

```
R1# show ip eigrp topology 10.9.9.0/24
EIGRP-IPv4 Topology Entry for AS(100)/ID(1.1.1.1) for 10.9.9.0/24
  State is Passive, Query origin flag is 1, 1 Successor(s), FD is 3000
  Descriptor Blocks:
  10.1.12.2 (GigabitEthernet0/1), from 10.1.12.2, Send flag is 0x0
      Composite metric is (3000/1000), route is Internal
  10.1.13.3 (GigabitEthernet0/2), from 10.1.13.3, Send flag is 0x0
      Composite metric is (4000/2500), route is Internal
```

The pair in parentheses is **(total metric / reported distance)**. Plain `show ip eigrp topology` lists **only successors and feasible successors**. To see R4's non-feasible path as well, add **`all-links`**:

```
R1# show ip eigrp topology all-links
P 10.9.9.0/24, 1 successors, FD is 3000, serno 41
        via 10.1.12.2 (3000/1000), GigabitEthernet0/1
        via 10.1.13.3 (4000/2500), GigabitEthernet0/2
        via 10.1.14.4 (3600/3500), GigabitEthernet0/3
```

If you're asked "why isn't EIGRP using the backup path?", the answer is usually in the `all-links` output: compare the path's RD with the FD.

26.7 Queries and Stuck-in-Active

The problem: sometimes a router loses its successor and has **no** feasible successor. It can't safely pick a new path on its own, because none of the remaining paths are proven loop-free. It has to ask. Asking is safe but slow, and in a big network one unanswered question can freeze a route for minutes. Understanding queries is understanding EIGRP's worst failure mode.

When a route goes **active**, the query **diffuses**. Each queried neighbor that has no feasible successor of its own goes active too and queries its neighbors, and so on. The original router can't

finish until every reply comes back up that tree.

Why must a router wait for every single reply? Because DUAL's loop-freedom depends on it. Choosing a new successor before everyone has answered could mean picking a neighbor whose information is already stale: one that's still routing through you. Waiting until everyone has recomputed is what keeps the new path safe. The cost of that safety is that one slow or broken router can hold up the whole computation.

Story Time : a phone tree. The teacher (R1) phones every parent to ask "does anyone know a way to the museum?". Parents who know answer at once. Parents who don't, phone their contacts before answering. The teacher can't announce the plan until every single parent has called back, even the one whose phone is in the washing machine. That last parent is a stuck-in-active router.

Worked example — a query diffusing, step by step. R1 has neighbors R2 and R3. R2 has a further neighbor R5. 10.9.9.0/24 lives behind R1's successor, which just failed, and R1 has no feasible successor.

1. **t = 0 s.** R1 marks 10.9.9.0/24 **active** and sends a query to R2 and R3.
2. R3's only path to 10.9.9.0/24 was through R1. It has nothing else, so it replies "unreachable" (infinite metric) **immediately**. R1 marks R3's reply received.
3. R2 also has no feasible successor, so R2 goes active itself and queries R5. R2 can't reply to R1 until R5 replies to R2.
4. R5 sits on an overloaded WAN link and its reply keeps getting lost.
5. **t = 90 s.** R1 sends an **SIA-query** to R2: "still alive?". R2 answers with an **SIA-reply**: "yes, I'm waiting on my own neighbor". R1 keeps waiting. At the same time, R2's own timer for R5 reaches 90 s, and R2 sends R5 an SIA-query. R5 doesn't answer.
6. **R2's timer for R5 reaches 180 s** with no answer. R2 declares R5 **SIA** and resets the R2–R5 adjacency. Now R2 can work out an answer and reply to R1.
7. R1 has every reply and finishes the computation: the route goes **passive** again, with a new successor or (as here) as unreachable.

The adjacency that was reset is **R2–R5**, right next to the real problem. The R1–R2 adjacency, which was healthy, survives. That's exactly what SIA-query/SIA-reply was designed to achieve.

That's **stuck-in-active (SIA)**. The protocol bounds the wait:

Time	Event
0 s	Route goes active; queries sent
90 s (half the active timer)	No reply yet → send an **SIA-query** to that neighbor ("are you still working on it?")
—	Neighbor answers with an **SIA-reply** ("still working, I'm waiting on my own neighbors") → timer resets; up to 3 SIA-queries
180 s with no reply at all	Neighbor declared **SIA** → that **adjacency is reset**

Why did Cisco add SIA-query/SIA-reply instead of just resetting at 3 minutes? Because in the old behavior, the router that got torn down was often innocent. A neighbor three hops away might be the one that's lost; your direct neighbor is just waiting on it, like you are. Resetting your direct neighbor flushes good routes and starts new queries, which can cascade. The SIA-query

asks the direct neighbor to prove it's alive and still working. Only a neighbor that **doesn't answer** gets reset, so the reset lands near the actual problem.

```
%DUAL-3-SIA: Route 10.9.9.0/24 stuck-in-active state in IP-EIGRP(0) 100.
             Cleaning up
%DUAL-5-NBRCHANGE: EIGRP-IPv4 100: Neighbor 10.1.13.3 (Gi0/2) is down: stuck in active
```

Finding the cause: while the route is active, `show ip eigrp topology active` lists which neighbors haven't replied (marked with `r`):

```
R1# show ip eigrp topology active
A 10.9.9.0/24, 1 successors, FD is Inaccessible
    1 replies, active 00:01:32, query-origin: Local origin
         via 10.1.13.3 (Infinity/Infinity), GigabitEthernet0/2
    Remaining replies:
         via 10.1.12.2, r, GigabitEthernet0/1
```

`A` = active; `active 00:01:32` = how long the route has been waiting; the `Remaining replies` list is who it's waiting for (R2 here). Log into R2 and run the same command: R2 will show it's waiting on R5. Follow that trail hop by hop until you find the router that's overloaded, flapping or on a lossy link. **Fix the cause by bounding the query domain**, not by raising `timers active-time`. The two tools for that are **stubs** (next section) and **summarization** (Chapter 24). A router that receives a query for a prefix it only knows as part of a summary replies "unreachable" immediately, so the query stops there.

26.8 Stub Routing — Keeping Spokes Out of Queries

A branch router with one or two uplinks to the hub can never be a useful transit path. Querying it is wasted time, and in a hub with hundreds of spokes, those queries are how SIA happens. **`eigrp stub`** tells neighbors this router is a dead end:

```
! Classic                                  ! Named mode
router eigrp 100                           router eigrp CORP
 eigrp stub connected summary               address-family ipv4 unicast autonomous-system 100
                                             eigrp stub connected summary
```

Story Time : a house at the end of a cul-de-sac. Nobody asks the people living there for a shortcut across town, because a dead-end street can't be a shortcut to anywhere. `eigrp stub` is the "No Through Road" sign the branch puts up, so the hub stops asking.

Worked example — counting the queries. A hub, HUB1, has 200 branch neighbors and one other hub, HUB2. HUB1 loses 10.50.0.0/16 and has no feasible successor.

Measure	Without stub on branches	With stub on branches
Queries HUB1 sends	201 (200 branches + HUB2)	**1** (HUB2 only)
Replies HUB1 must wait for	201	**1**
Chance one slow branch link causes SIA	real, on every outage	none: branches aren't asked

The routing result is the same in both cases, because no branch could ever have offered a path to 10.50.0.0/16 anyway. The only difference is 200 pointless questions and 200 chances to get stuck.

How it works, step by step:

1. You configure `eigrp stub` on the branch. It sends hellos that carry a **stub flag** and the list of route types it will advertise.
2. The hub reads the flag and records the branch as a stub peer (`show ip eigrp neighbors detail` shows it).
3. When the hub later goes active for a route, it **skips stub peers** when sending queries.
4. The branch still receives every update from the hub, and still advertises only the route types its keywords allow (table below).

Why does marking a spoke as a stub bound the query domain? Because the stub announces itself in its hello, and the hub then **never sends it queries**. The hub knows the stub has no alternate paths to offer, so it doesn't wait on spokes that can't help. A hub with 500 stub spokes can recompute a lost route by querying only the other hubs. The stub keyword is a promise: "I'm never a transit path, so don't ask me."

The keywords control **what the stub advertises** (it still receives everything):

Keyword	Stub advertises
`connected`	its connected routes (that are in EIGRP)
`summary`	its summary routes
`static`	static routes redistributed into EIGRP
`redistributed`	any redistributed routes
`receive-only`	**nothing** — learns only
`leak-map NAME`	routes learned from other neighbors that match the route-map (an exception to "never transit")
(no keyword)	defaults to **`connected summary`**

Exam trap: a stub **doesn't re-advertise routes it learned from one EIGRP neighbor to another**. Consider a spoke with two hubs, where Hub-1's LAN reaches Hub-2 only through the spoke. After `eigrp stub`, that path disappears, because transit through a stub is exactly what the stub feature forbids. That's what `leak-map` is for. In `show ip eigrp neighbors detail` on the hub, a stub neighbor shows `Stub Peer Advertising (CONNECTED SUMMARY) Routes` and `Suppressing queries`.

26.9 Load Balancing — Equal and Unequal Cost

The problem: you paid for two WAN links. If the routing protocol uses only the best one, the second sits idle until the first fails. Load balancing puts both to work. OSPF can only share between paths of exactly equal cost; EIGRP can also share across paths of different quality, in proportion to how good each one is.

Equal-cost: EIGRP installs up to **4** equal-metric paths by default (`maximum-paths`, whose ceiling is 16 or 32 depending on the platform; it goes under `topology base` in named mode).

Unequal-cost is EIGRP's unique trick, and it's done with `variance`. A path is installed if:

1. it is a **feasible successor** (passes the FC), **and**
2. its metric **< variance × FD**.

Using 26.6's numbers (FD 3000) with `variance 2`, so the limit is 6000:

Path	Metric	FS?	< 6000?	Installed?
via R2	3000	successor	yes	**yes**
via R3	4000	yes (RD 2500 < 3000)	yes	**yes**
via R4	3600	**no** (RD 3500 ≥ 3000)	yes	**no**

Why does variance only consider feasible successors? Because traffic actually flows over every installed path at the same time. If R1 sent part of its traffic to R4, and R4's best path ran back through R1, that traffic would loop. The FC that makes a path safe as a backup is the same guarantee that makes it safe to share load across. R4 has the better metric, but it can't be proven loop-free, so it gets no traffic.

How R1 decides, step by step:

1. Find the FD: **3000** (via R2).
2. Multiply by the variance: 2 × 3000 = **6000**. That's the ceiling. (`variance 1`, the default, means "only paths equal to the FD", i.e. equal-cost only.)
3. Take each path that is a **successor or feasible successor** and check whether its total is below the ceiling. R3: 4000 < 6000 → in. R4 isn't a feasible successor, so it's never even considered.
4. Install the surviving paths, up to `maximum-paths`.
5. Work out the **traffic share**: the better path gets more. Metrics 3000 and 4000 are in the ratio 3 : 4, so the shares are the inverse, **4 : 3**. Roughly 4 of every 7 flows use R2 and 3 use R3. (CEF shares per destination by default, so it's flows that are split, not individual packets.)

Exam trap: "what variance makes R3's path usable?" Divide and round up: 4000 ÷ 3000 = 1.33, so **variance 2** is the smallest that works. Variance must be a whole number (1–128).

Traffic is shared **in inverse proportion to the metric** (`traffic-share balanced`, the default), so R2 carries more than R3 here. `traffic-share min across-interfaces` installs both paths but uses only the best, which gives instant failover without sharing load.

```
R1# show ip route 10.9.9.0
  Routing Descriptor Blocks:
  * 10.1.12.2, from 10.1.12.2, via GigabitEthernet0/1
      Route metric is 3000, traffic share count is 4
    10.1.13.3, from 10.1.13.3, via GigabitEthernet0/2
      Route metric is 4000, traffic share count is 3
```

26.10 Metrics — Classic Composite and Wide

The problem: a metric has to turn "how good is this path?" into one number that every router computes the same way. If you can't calculate it, you can't predict which path EIGRP will choose, or change it on purpose.

The classic formula

Story Time : think of a road trip. How fast you can go is limited by the **slowest road** on the route; a motorway followed by a single-track lane is a single-track trip. But the **time** adds up along every stretch. EIGRP's default metric uses exactly those two ideas: the **minimum bandwidth** along the path (the slowest road) and the **sum of delays** (the total travel time).

EIGRP's **classic** metric combines five K values:

```
metric = 256 × [ K1·BW + (K2·BW)/(256 − load) + K3·DLY ] × [ K5/(reliability + K4) ]
```

The last bracket only applies if K5 ≠ 0; when K5 = 0 it's treated as 1.

Term	Meaning	Default K
BW	10^7 ÷ **lowest** bandwidth on the path in kbps	**K1 = 1**
load	link utilization (1–255)	K2 = 0
DLY	**sum** of interface delays on the path, in tens of µs	**K3 = 1**
reliability (K4, K5 terms)	1–255	K4 = 0, K5 = 0

With the defaults, it collapses to `metric = 256 × (10⁷ / min-BW-kbps + Σdelay/10)`.

Worked: a connected GigabitEthernet network: BW term = 10^7 / 1,000,000 = **10**; delay 10 µs → **1**. Metric = 256 × (10 + 1) = **2816**, the number you'll see on every Gi-connected EIGRP route. Add a second Gi hop and the delay sum becomes 2: 256 × 12 = **3072**.

Worked: a mixed path. R1 reaches 10.9.9.0/24 across one GigabitEthernet link (delay 10 µs) and then one FastEthernet link (100,000 kbps, delay 100 µs).

1. **Slowest link:** FastEthernet, 100,000 kbps. BW term = 10^7 ÷ 100,000 = **100**.
2. **Total delay:** 10 + 100 = 110 µs. In tens of µs: 110 ÷ 10 = **11**.
3. **Metric** = 256 × (100 + 11) = 256 × 111 = **28,416**.

Notice the Gigabit link only contributed its delay (1 of the 111). Its speed didn't matter, because the slowest link sets the BW term.

Worked: the famous T1 number. A serial T1 has bandwidth 1544 kbps and delay 20,000 µs:

1. BW term = 10^7 ÷ 1544 = 6476.68, and EIGRP drops the fraction: **6476**.
2. Delay term = 20,000 ÷ 10 = **2000**.
3. Metric = 256 × (6476 + 2000) = 256 × 8476 = **2,169,856**.

If you ever see 2169856 in old exhibits, it's a directly connected T1.

Why are load and reliability off by default? Because they **change by the second**. A metric that follows load means routes change as traffic changes. Traffic then moves to the "lighter" path, which makes it heavier, and the routes flip back. That's self-inflicted route flapping. Bandwidth and delay are configured values that stay put, so they make stable metrics. That's also why the exam steers you to change delay, not bandwidth, when you want to influence EIGRP. Delay is additive along the path and doesn't affect QoS or other features that read `bandwidth`.

Wide metrics (named mode)

Classic metrics have a scaling problem. The BW term is 10^7 / kbps, so 10 Gbps → 1, and 100 Gbps → 0.1, which truncates to 0. Delay can't separate them either. **Classic EIGRP can't distinguish links faster than about 10 Gbps.**

Named mode uses **wide (64-bit) metrics**. They're computed in picoseconds and with a 65,536 scaling factor, so every modern speed gets a distinct value. They also add a **K6** term (jitter/energy, default 0). The routing table still holds only 32-bit metrics, so the wide value is divided by `metric rib-scale` (default **128**) before installation. That's why the metric in `show ip route` doesn't equal the one in `show eigrp address-family ipv4 topology`. The topology table shows the full 64-bit number; the RIB shows it ÷ 128.

Named-mode and classic routers interoperate: a named-mode router detects classic neighbors and scales metrics to match. **K values, though, must still match.** In named mode `metric weights` takes seven numbers: the TOS (always 0) and then K1–K6, for example `metric weights 0 1 0 1 0 0 0`. Classic mode takes six: the TOS and then K1–K5.

Tuning the Path — Delay, Bandwidth and Offsets

Three knobs change which path EIGRP prefers, each with different side effects:

```
interface GigabitEthernet0/2
 delay 200                        ! tens of µs: 200 = 2,000 µs added for this link
 bandwidth 50000                  ! kbps — also changes QoS %, OSPF cost, EIGRP pacing
!
router eigrp 100
 offset-list 30 in 25600 GigabitEthernet0/2   ! +25600 to routes in ACL 30 heard here
```

Work the effect through R1's paths from 26.6. Suppose R1 wants R3 (total 4000) to be the successor instead of R2 (3000):

- **Raising delay on R1's link to R2** raises R1's cost to R2 and therefore the **total (FD candidate)** via R2. It does **not** change R2's RD, because RD is R2's own distance. Push R1's cost to R2 past 3000 (total > 4000) and R3 becomes the successor. R2 can stay a feasible successor as long as its RD (1000) is below the new FD (4000).
- **An offset list `in` on the R2 interface** adds to the metric as received from R2, so both the RD R1 records for R2 and the total via R2 rise. That can **remove R2 as a feasible successor**, if its adjusted RD reaches the FD. Offsetting a path makes it less preferred and can stop it qualifying as a backup.
- **Changing `bandwidth`** works only if it changes the **minimum** bandwidth along the path, and it has side effects. QoS percentages, OSPF cost and EIGRP's own pacing (below) all read it. That's why **delay is the preferred knob**: it's additive, predictable and used by nothing else.

EIGRP Bandwidth Pacing — `ip bandwidth-percent eigrp`

EIGRP limits its own packets to **50% of the interface's configured `bandwidth`** by default, so a burst of updates can't starve user traffic. That's usually invisible until `bandwidth` is set unrealistically low. The classic case is a **tunnel interface**, whose default bandwidth is very low (historically 100 kbps). There, EIGRP is paced to a few kbps, and a large topology takes minutes to converge, or SIA appears under load:

```
interface Tunnel0
 bandwidth 10000                          ! tell EIGRP the real capacity
 ip bandwidth-percent eigrp 100 25        ! classic: EIGRP may use 25% of that
!
router eigrp CORP                         ! named mode equivalent
 address-family ipv4 unicast autonomous-system 100
  af-interface Tunnel0
   bandwidth-percent 25
```

`show ip eigrp interfaces detail` shows the pacing interval. Values in the hundreds of milliseconds on a fast link mean the `bandwidth` is wrong.

Exam trap — K values and `show ip protocols`:

```
R1# show ip protocols
  EIGRP-IPv4 Protocol for AS(100)
    Metric weight K1=1, K2=0, K3=1, K4=0, K5=0
```

If one router has been changed to `metric weights 0 1 1 1 0 0`, the neighbor drops with `K-value mismatch`. That line is the first place to look.

Why must K values match, when timers don't? Because the K values define what the metric means. If R1 counts load and R2 doesn't, the same path gets two different numbers on the two routers, and DUAL's "RD < FD" comparison is comparing apples with oranges. The loop-freedom guarantee would be gone, so EIGRP refuses the adjacency instead. The log on either router says it plainly:

```
%DUAL-5-NBRCHANGE: EIGRP-IPv4 100: Neighbor 10.1.12.2 (GigabitEthernet0/1) is down:
                   K-value mismatch
```

Fix: `no metric weights` on the changed router puts it back to the defaults (K1 = K3 = 1, the others 0).

26.11 The EIGRP Troubleshooting Checklist

Work top-down: **neighbors, then the topology table, then the routing table.** Each layer depends on the one before it.

Question	Command	What you're looking for
Which interfaces run EIGRP?	`show ip eigrp interfaces [detail]`	Missing = `network` didn't match or passive; detail shows hello/hold, auth, split horizon

Question	Command	What you're looking for
Who are my neighbors?	`show ip eigrp neighbors [detail]`	Missing, flapping uptime, Q Cnt stuck, stub flags
What does the process think?	`show ip protocols`	AS, K values, router ID, `network`s, passive list, AD 90/170, variance
Is the route learned at all?	`show ip eigrp topology [prefix]`	Present? Passive or Active? FD and RD values
Why isn't a path used?	`show ip eigrp topology all-links`	RD vs FD → feasibility condition
Why isn't it in the RIB?	`show ip route [prefix]`	Better-AD source wins (Chapter 22); `D` vs `D EX`
Is something stuck active?	`show ip eigrp topology active`	Neighbors with outstanding replies (`r`)

And the classic route-level causes:

Symptom	Cause	Proof / fix
Route in topology table, not in RIB	another source with lower AD (static 1, OSPF 110 beats EIGRP external 170)	`show ip route` shows the winner; Chapter 22
Spoke routes missing between spokes on a hub	**split horizon** on the hub's multipoint interface	`no split-horizon` on the hub (Chapter 30)
Externals redistributed by one ASBR missing on **one particular router** (and anything that can only learn them through it)	**duplicate router ID**: that router shares the ASBR's EIGRP RID, sees its "own" RID as the originator and discards the externals. Other routers have them	On a router that has the route: `show ip eigrp topology <prefix>` → `Originating router is X`. On the router missing it: `show ip protocols` → `Router-ID: X`. Same value = proven. Fix `eigrp router-id`
Neighbour loss after adding unicast peering	`neighbor` statement on one side only	26.4
Discontiguous network reachability broken	`auto-summary` on (default **off** since IOS 15)	`show ip protocols` → `Automatic Summarization: disabled`
Backup path not used for load sharing	not a feasible successor	`all-links`: RD ≥ FD
Route flaps / SIA in a large network	unbounded query domain	configure stubs (26.8), summarize (Chapter 24)

Worked ticket — "the backup link never takes over"

Symptom: when R1's link to R2 fails, traffic to 10.9.9.0/24 stops briefly while the route goes active, even though the R4 link is up and "should be the backup".

1. **Neighbors?** `show ip eigrp neighbors`: R2 and R4 are both up (this time R1 has no R3 path). Not a neighbor problem.

2. **What does the topology table hold?** Plain `show ip eigrp topology` lists only R2 (the successor) for 10.9.9.0/24. So there's no feasible successor at all; that's why the route goes active when R2 is lost.
3. **Where's R4's path?** `show ip eigrp topology all-links` shows `via 10.1.14.4 (3600/3500)` against `FD is 3000`. R4's RD (3500) is not below the FD (3000), so it fails the feasibility condition.
4. **Why is R4's RD so high?** On R4, `show ip eigrp topology 10.9.9.0/24` shows its own best path. Suppose R4's path runs over a slow link with extra delay configured by an old change.
5. **Fix options:** lower R4's distance to the destination (remove the extra delay on R4's path, so its RD drops below 3000), or accept that R4 isn't a provable backup and let the query run. Making R4 a feasible successor turns a multi-second active computation into an instant local switchover.

If you can tell from `show ip eigrp topology all-links` alone why a path is or isn't used, you understand DUAL well enough for ENARSI.

Chapter 27: OSPF In Depth

> **Blueprint:** ENARSI 1.10 — Troubleshoot OSPF (v2/v3): 1.10.a address families (IPv4, IPv6) · 1.10.b neighbor relationship and authentication · 1.10.c network types (point-to-point, multipoint, broadcast, nonbroadcast), area types (backbone, normal, transit, stub, NSSA, totally stub), router types (internal, backbone, ABR, ASBR), virtual link · 1.10.d path preference

The CCNA guide (16.9–16.11) covered neighbor requirements, the DR/BDR election, the adjacency state machine and LSA types 1–7. Chapter 6 added multi-area design, summarization, filtering and OSPFv3 basics. This chapter covers the remaining ENARSI material: **every network type, every area type, virtual links, authentication, OSPFv3 address families and path preference**, all aimed at diagnosing what's broken.

Words you'll meet in this chapter:

Term	Plain meaning
LSA	Link-State Advertisement: one "fact card" a router floods ("I'm R2, I have these links at these costs")
LSDB	Link-State Database: every LSA a router holds for an area. All routers in an area must hold an identical copy
SPF	Shortest Path First (Dijkstra's algorithm): the calculation each router runs over its LSDB to build routes
RID	Router ID: the 32-bit name of an OSPF router, written like an IPv4 address

Term	Plain meaning
DR / BDR	Designated Router / Backup DR: the router that speaks for a shared segment, and its standby
ABR	Area Border Router: sits between Area 0 and another area
ASBR	Autonomous System Boundary Router: injects routes from outside OSPF (redistribution)
AF	address family: IPv4 or IPv6, carried as separate instances inside one OSPFv3 process

Story Time — the one picture that explains most of OSPF: every router in an area holds the **same map** (the LSDB), and works out its own best routes from it (SPF). Areas are like separate map books: inside your area you have the full street-level map; for other areas you only get a summary page saying "town X is about this far, go via this junction" (the ABR). Almost every OSPF fault in this chapter is one of two things: two routers that refuse to swap maps (a neighbor problem), or a page that never made it into someone's book (an LSA filtered or blocked).

27.1 OSPFv3 and Address Families

The problem: a dual-stack network running OSPFv2 for IPv4 and a separate OSPFv3 for IPv6 means two processes, two sets of adjacencies and two configurations to keep in step. Address families let one OSPFv3 process carry both, with one style of configuration and one set of commands.

OSPFv3 was written for IPv6. With **address families**, a single OSPFv3 process can carry **IPv4 routes too**:

```
ipv6 unicast-routing
!
router ospfv3 1
 router-id 1.1.1.1                      ! 32-bit RID, shared by both AFs
 address-family ipv6 unicast
  passive-interface GigabitEthernet0/0
 exit-address-family
 address-family ipv4 unicast
  area 1 stub
 exit-address-family
!
interface GigabitEthernet0/1
 ip address 10.1.12.1 255.255.255.0
 ipv6 address 2001:db8:12::1/64
 ospfv3 1 ipv6 area 0                   ! join the IPv6 AF
 ospfv3 1 ipv4 area 0                   ! join the IPv4 AF — same process, separate instance
```

Why does the IPv4 address family still need IPv6 on the interface? Because OSPFv3 always **talks over IPv6**. Its hellos come from the interface's **link-local** address to **FF02::5** (all OSPF routers) and **FF02::6** (DR/BDR), even when the routes it carries are IPv4. The IPv4 address family runs as a separate OSPFv3 **instance** (instance IDs 64–95, versus 0–31 for IPv6). Each instance has its own adjacencies and its own LSDB, but they all use the IPv6 transport. An interface with only an IPv4 address can't run OSPFv3 at all. Enable IPv6 on it (even just `ipv6`

`enable` for a link-local address) and the IPv4 AF comes up.

What's different inside OSPFv3, and why it matters for troubleshooting:

OSPFv2	OSPFv3
Router and network LSAs (1, 2) carry prefixes	Router/network LSAs carry **topology only**; prefixes move to new LSAs
—	**Type 8 Link LSA**: a router's link-local address and prefixes on one link (link scope)
—	**Type 9 Intra-Area-Prefix LSA**: the area's prefixes (replaces the prefix part of types 1/2)
Type 3 Summary, Type 4 ASBR Summary	renamed **Inter-Area-Prefix** and **Inter-Area-Router**
Neighbors must share an IPv4 subnet	Neighbors meet over link-local — global prefixes **don't** need to match
Authentication built into OSPF	Authentication uses **IPsec** (or, on newer IOS XE, a key-chain trailer)

Why move prefixes out of the router LSA? So that **renumbering doesn't trigger SPF**. In OSPFv2, changing a prefix on an interface changes the router LSA, which is the same object that describes the topology, so every router reruns the full SPF. In OSPFv3 the shape of the network (types 1/2) and the addresses on it (type 9) are separate. Adding a prefix only changes a type 9 LSA, and routers update their routes without recomputing the tree.

Verification uses `ospfv3`-prefixed commands, with an AF keyword where needed:

```
R1# show ospfv3 neighbor
R1# show ospfv3 ipv4 neighbor              ! just the IPv4 AF's adjacencies
R1# show ospfv3 database
R1# show ip route ospfv3                   ! IPv4 routes learned through OSPFv3
```

Worked example — the IPv4 AF that won't come up. R1 and R2 already run the OSPFv3 IPv6 AF happily. The engineer adds `ospfv3 1 ipv4 area 0` to a new link, Gi0/3, which has only an IPv4 address. `show ospfv3 ipv4 neighbor` shows nothing on Gi0/3.

1. `show ospfv3 ipv4 interface brief` doesn't list Gi0/3 as an active OSPFv3 interface.
2. `show ipv6 interface brief` shows Gi0/3 with **no IPv6 address at all**, not even a link-local one.
3. **Cause:** OSPFv3 sends its packets over IPv6 from the link-local address. With no IPv6 on the interface, there's nothing to send them from.
4. **Fix:** `ipv6 enable` on Gi0/3 (on both routers). That creates a link-local address, and the IPv4 AF adjacency comes up. No global IPv6 prefix is needed.

27.2 Neighbor Failures Beyond the CCNA List

The problem: OSPF can't share its map with a router it isn't fully adjacent to, so a neighbor fault means missing routes. And OSPF is strict: it checks many fields and stops at different states depending on which one disagrees. The **state where a neighbor gets stuck tells you which kind of fault you have**, so learn the states as a diagnostic tool.

How an adjacency forms, step by step (R1 RID 1.1.1.1 and R2 RID 2.2.2.2 on an Ethernet link):

1. **Down → Init.** R1 hears R2's hello but doesn't yet see its own RID (1.1.1.1) listed inside it. R1 has heard R2; R2 hasn't confirmed hearing R1. Stuck here = **one-way** communication.
2. **2-Way.** R1 sees 1.1.1.1 in R2's hello: both have heard each other. On a broadcast segment the DR/BDR election happens now. Two DROTHERs stay at 2-Way with each other permanently; that's **normal**.
3. **ExStart.** The pair decides who leads the database exchange (the higher RID becomes master). Stuck here = usually an **MTU mismatch**.
4. **Exchange.** They swap Database Description (DBD) packets: a table of contents of every LSA each one holds.
5. **Loading.** Each requests the LSAs it's missing (Link-State Requests) and receives them (Link-State Updates). Stuck here = an LSA that can't be delivered.
6. **Full.** Both LSDBs are identical. Routes can now be calculated.

So `INIT` points at one-way traffic, `EXSTART/EXCHANGE` at MTU, `2WAY` everywhere at priority 0, and no neighbor at all at a hello-field mismatch (the first table below).

The CCNA list (area, subnet, timers, authentication, MTU) still applies. Two more matching items come in at this level, plus failures that let an adjacency form but break the database.

Hello-packet items that must match: area ID, hello/dead interval, authentication, subnet mask (except on point-to-point links), and the **stub area flag** (the E/N option bits).

Symptom	Cause	Command that proves it
No neighbor; `Mismatched area ID`	interfaces in different areas	`debug ip ospf adj`; `show ip ospf interface brief`
No neighbor; `mismatched hello parameters`	hello/dead or mask differ	`show ip ospf interface Gi0/1` — compare `Timer intervals`
No neighbor; nothing obvious in the config	**stub flag mismatch** — one side `area 1 stub`, other not	`debug ip ospf hello` → `Hello from X with mismatched Stub/Transit area option bit`
No neighbor; `Mismatch Authentication type` / `key`	type or key differs	`debug ip ospf adj`
Stuck **EXSTART/EXCHANGE**	**MTU mismatch** (CCNA 16.11)	`show interface | i MTU`; fix the MTU, or `ip ospf mtu-ignore` as a last resort

Symptom	Cause	Command that proves it
No adjacency with a directly connected neighbour; `%OSPF-4-DUP_RTRID_NBR` logged	**duplicate RID between neighbours**: the two routers on the link share a router ID, so the adjacency is refused	the log message names the RID and interface; `show ip ospf` on both ends
Adjacencies FULL, but routes come and go and CPU climbs; `%OSPF-4-FLOOD_WAR` (or a duplicate-RID-in-area message)	**duplicate RID between non-adjacent routers** in the same area: each keeps "correcting" the other's router LSA (below)	`show ip ospf database router <RID>` — sequence number racing upward; `show ip ospf` on suspects
No adjacency in **OSPFv3**; both ends look right	**instance ID mismatch**: `ospfv3 1 ipv6 area 0 instance 1` on one end, default instance 0 on the other. Packets for another instance are ignored	`show ospfv3 interface` — compare `Instance ID`
No adjacency; interface missing from the neighbour's view	**passive interface** on one side (no hellos sent or processed), or the interface was put in the wrong area by a `network` statement / `ip ospf area`	`show ip ospf interface Gi0/1` — `No Hellos (Passive interface)`; area shown per interface
Neighbour stuck in **LOADING**	link-state requests not satisfied: an LSA the neighbour can't supply or keeps failing (often a corrupted or oversized LSA on an MTU-marginal path, or a bug)	`show ip ospf neighbor detail`; `debug ip ospf adj` for `BADSEQNUM` / retransmitted LS requests
Neighbor FULL, routes missing, odd next hops	**network type mismatch** (27.4)	`show ip ospf interface` — `Network Type` both ends
Stuck **2-WAY** with every neighbor	both routers priority 0 on a broadcast segment — no DR can be elected	`show ip ospf neighbor` → `2WAY/DROTHER` everywhere
Neighbor stuck **INIT**	one-way: my hellos reach them, theirs don't reach me (ACL, L2 issue)	`debug ip ospf hello`; check inbound ACLs for protocol 89 / 224.0.0.5

Why does a stub-flag mismatch block the adjacency? Because area type is a promise about what the LSDB contains. A stub area must never hold Type 5 LSAs. If a router that believes the area is normal became adjacent to one that believes it's stub, it would flood Type 5s into a database that's supposed to have none, and the two routers would disagree about what the area is. OSPF prevents that by refusing adjacencies when the stub (or NSSA) bit differs. Every router in an area must agree on its type, so the type is checked in every hello.

Why is a duplicate router ID so destructive? Because the RID is how every LSA identifies its author. Two routers using the same RID generate "the same" router LSA with different contents. Each sees the other's version as a newer copy of its own LSA and floods a correction, so the LSA flip-flops forever. That's the "flood war": CPU climbs, SPF runs constantly, and routes come and go. The fix is a unique RID followed by `clear ip ospf process`. The RID only changes when the

process restarts.

Story Time : two people with the same name editing the same shared document page signed with that name. Each opens the page, sees "their" page has changed, assumes it's out of date, and rewrites it. Neither ever stops. That's the flood war.

Worked example — stuck in EXSTART. A new link between R1 and R2 never reaches FULL:

```
R1# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   EXSTART/DR      00:00:36    10.1.12.2       GigabitEthernet0/1
```

1. **Read the state.** EXSTART means hellos match (otherwise there'd be no neighbor at all) but the database exchange can't start. Think MTU first.
2. **Compare IP MTUs:** `show ip interface Gi0/1 | include MTU` gives `MTU is 1500 bytes` on R1 and `MTU is 1400 bytes` on R2 (someone set `ip mtu 1400` there for a tunnel test). Use `show ip interface`, not `show interface`: `ip mtu` doesn't change the `MTU 1500 bytes` that `show interface` reports, and OSPF uses the IP MTU.
3. **Why that blocks it:** each DBD packet carries the sender's interface MTU. R2 sees R1 advertise 1500, larger than its own 1400, and refuses R1's DBDs, because R1 could send it LSA packets too big for R2 to receive. The exchange never progresses.
4. **Fix:** set the same MTU on both ends. `ip ospf mtu-ignore` would also bring the adjacency up, but it hides the mismatch rather than fixing it, so it's a last resort.

Worked example — stub flag mismatch. R4 (internal router in Area 1) has no neighbors after a change:

```
R4# debug ip ospf hello
OSPF-1 HELLO Gi0/0: Rcv hello from 2.2.2.2 area 1 10.1.24.2
OSPF-1 HELLO Gi0/0: Hello from 2.2.2.2 with mismatched Stub/Transit area option bit
```

The ABR (2.2.2.2) was changed to `area 1 stub`; R4 wasn't. **Fix:** add `area 1 stub` on R4 (and every router in Area 1), or remove it from the ABR.

27.3 Authentication

The problem: without authentication, any device that sends a valid OSPF hello on a link can become a neighbor and flood LSAs into your database, rerouting or black-holing traffic. Authentication makes every OSPF packet prove it came from a router that knows the shared key. It works the same way as EIGRP's (26.5): a hash of the packet plus the key is attached, and the receiver recomputes it.

OSPFv2 has three authentication types, carried in every packet header: **0 null**, **1 plain text**, **2 MD5** (plus HMAC-SHA via key chains). The type can be enabled per interface or for a whole area. The keys always live on the interfaces.

```
! ---- MD5, per interface ----
interface GigabitEthernet0/1
 ip ospf authentication message-digest           ! type 2 on this interface
 ip ospf message-digest-key 1 md5 S3cr3t!         ! key ID 1 must match the neighbor's
```

```
! ---- MD5, area-wide type, per-interface keys ----
router ospf 1
 area 0 authentication message-digest              ! every Area 0 interface uses type 2
interface GigabitEthernet0/1
 ip ospf message-digest-key 1 md5 S3cr3t!
```

```
! ---- HMAC-SHA-256 via a key chain (modern IOS XE) ----
key chain OSPF_KC
 key 1
  key-string Str0ngP@ss
  cryptographic-algorithm hmac-sha-256
interface GigabitEthernet0/1
 ip ospf authentication key-chain OSPF_KC
```

Plain text (type 1) is `ip ospf authentication` plus `ip ospf authentication-key PASS`, or `area 0 authentication` with no keyword. It keeps out mistakes, not attackers, since the password crosses the wire in the clear.

Why can you set the type per area but not the key? Because the **type** is a policy decision ("Area 0 must be authenticated"), and one line under the process enforces it everywhere. The **key** is a property of each link, shared by the two routers on it, and different links can use different keys. The interface command overrides the area setting, which is how one link can use a different type from the rest of its area.

Worked example — authentication that's "on" at one end only. R1 has `area 0 authentication message-digest` and a key on Gi0/1. R2 has the same key on its interface, but no `area 0 authentication` and no `ip ospf authentication` command, so R2 is still using type 0 (null).

1. `show ip ospf neighbor`: R1 and R2 don't see each other.
2. `show ip ospf interface Gi0/1` on R1 includes `Message digest authentication enabled` and `Youngest key id is 1`. On R2 the authentication line is missing.
3. `debug ip ospf adj` on R1: `Rcv pkt from 10.1.12.2, GigabitEthernet0/1 : Mismatch Authentication type. Input packet specified type 0, we use type 2`.
4. **Cause:** R2 has the key but was never told to use MD5. A key on its own does nothing.
5. **Fix:** `area 0 authentication message-digest` under R2's process (or `ip ospf authentication message-digest` on its interface).

Exam trap — virtual links inherit Area 0's authentication. A virtual link (27.6) is an Area 0 interface. If Area 0 has `area 0 authentication message-digest`, the virtual link needs a key too. Supply it on the `area X virtual-link` line (`area 1 virtual-link 3.3.3.3 message-digest-key 1 md5 S3cr3t!`), or the virtual link won't come up even though both ends look correct.

OSPFv3 authentication uses IPsec AH or ESP, per interface or per area:

```
interface GigabitEthernet0/1
 ospfv3 authentication ipsec spi 256 sha1 0123456789ABCDEF0123456789ABCDEF01234567
!
router ospfv3 1
 area 0 authentication ipsec spi 500 md5 0123456789ABCDEF0123456789ABCDEF
```

The **SPI and the key must match** on both ends. The SHA-1 key is 40 hex characters; the MD5 key is 32. A mismatch looks like an adjacency that never leaves DOWN. Check `show ospfv3 interface` (look for the `authentication` line) and `show crypto ipsec sa`.

27.4 Network Types

The problem: a link with exactly two routers on it, an Ethernet segment with ten, and a WAN cloud that can't carry multicast need different handling. On a shared segment, if every router formed a full adjacency with every other router, the number of adjacencies (and the flooding between them) would explode. The **network type** tells OSPF what kind of link it's on, so it can choose whether to elect a DR, how to find neighbors and which timers to use.

Story Time :

- **Broadcast** is a meeting room. Rather than everyone briefing everyone, the room picks a **chairperson** (the DR) and a deputy (the BDR). You tell the chair your news, and the chair tells the room.
- **Point-to-point** is a phone call between two people. There's nobody else to coordinate, so no chair is needed.
- **Non-broadcast** is an office where the intercom is broken: you can't announce to the room, so you have to phone each person by their number (`neighbor` statements). There's still a chair.
- **Point-to-multipoint** is a hub with a switchboard: every caller is connected individually, there's no chair, and messages between callers go through the switchboard (the hub).

Why a DR saves so much on a shared segment, in numbers: with 6 routers on one Ethernet, a full mesh of adjacencies would be $n(n-1)/2 = 6 \times 5 / 2 =$ **15**. With a DR and BDR, each of the 4 other routers is FULL only with the DR and the BDR ($4 \times 2 = 8$), plus the DR–BDR adjacency itself: **9**. The DROTHER pairs stay at 2-Way. With 20 routers the difference is 190 versus 37.

How the DR election works, step by step:

1. Routers on the segment exchange hellos, which carry each router's **priority** (0–255, default 1) and RID.
2. Highest priority wins DR; a tie is broken by the highest RID. The runner-up becomes BDR. Priority 0 means "never elect me".
3. The election is **not preemptive**: a better router that joins later doesn't take over. It waits until the DR fails (then the BDR is promoted and a new BDR is elected) or until the OSPF process is cleared.

That non-preemption is why "I set priority 255 on the hub but a spoke is still DR" happens: the priority only counts at the next election.

OSPF behaves differently depending on what it believes the link is. The network type is set per interface with `ip ospf network <type>`, or `ospfv3 network` in v3:

Network type	Default on	Hello/Dead	DR/BDR?	Neighbors	Next hop for routes across it
broadcast	Ethernet	10 / 40	**yes**	discovered (multicast)	the originating router's interface

Network type	Default on	Hello/Dead	DR/BDR?	Neighbors	Next hop for routes across it
point-to-point	serial (HDLC/PPP), GRE tunnels	10 / 40	no	discovered	the neighbor
non-broadcast (NBMA)	Frame Relay main interface	**30 / 120**	**yes**	**manual** `neighbor`	the originating router (needs L2 mapping to it)
point-to-multipoint	— (configure it)	**30 / 120**	no	discovered	the hub; advertises **/32 host routes** per endpoint
point-to-multipoint non-broadcast	—	30 / 120	no	**manual** `neighbor`	as point-to-multipoint
loopback	loopbacks	—	—	—	advertised as a **/32** unless `ip ospf network point-to-point`

Why does non-broadcast need `neighbor` statements? Because on an NBMA cloud the underlying network **can't deliver multicast**, so hellos to 224.0.0.5 never arrive. Instead, you list the neighbors and OSPF unicasts hellos to them:

```
router ospf 1
 neighbor 10.0.0.2                          ! unicast hellos to each spoke
 neighbor 10.0.0.3
interface Serial0/0
 ip ospf network non-broadcast
 ip ospf priority 255                       ! the hub must be DR...
! on each spoke:  ip ospf priority 0        ! ...because only the hub reaches everyone
```

Where do the `neighbor` statements go? Strictly, only on routers that are **eligible to become DR or BDR** (non-zero priority), which in a hub-and-spoke design means the **hub**. The hub unicasts hellos to each spoke, and a priority-0 spoke replies to the hello it receives, so the spokes need no `neighbor` lines. Adding them on spokes is harmless but not required.

Why must the hub be the DR on a hub-and-spoke NBMA network? Because the DR has to reach every router on the segment, and on a partial mesh **only the hub does**. A spoke that won the election couldn't flood to the other spokes, and the LSDB would never synchronize. Priority 0 on every spoke rules them out.

Why does point-to-multipoint advertise host routes and avoid a DR? It treats the shared subnet as a **collection of point-to-point links**. There's no DR, and each endpoint is advertised as a /32. Routes to spoke LANs then point through the **hub**, which can reach everyone. That fixes the partial-mesh problem without depending on spoke-to-spoke reachability or on a correct DR.

Network-type mismatches

Mismatch	Result
broadcast ↔ non-broadcast	**No adjacency** — timers differ (10/40 vs 30/120)
point-to-point ↔ point-to-multipoint	**No adjacency** — timers differ
broadcast ↔ point-to-point	**Adjacency forms (both 10/40) — but routes break.** One side expects a DR and a Type 2 LSA, the other describes a p2p link. SPF can't make the two descriptions agree, so routes across the link are missing
point-to-multipoint ↔ broadcast	adjacency may form after a timer fix; same inconsistency

The dangerous mismatch is the one where the timers match. The neighbor shows FULL and everything looks healthy, yet routes are missing. That's why `show ip ospf interface` (`Network Type BROADCAST` vs `POINT_TO_POINT`) belongs on your checklist even when the adjacency is up.

Worked example — FULL, but the remote LAN is missing. R1 and R2 share 10.1.12.0/24. R2's LAN, 10.2.2.0/24, is missing from R1's routing table, yet:

```
R1# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           0   FULL/  -        00:00:33    10.1.12.2       GigabitEthernet0/1
```

1. **Read the neighbor line.** `FULL/ -` means R1 thinks the link has no DR, which is a point-to-point view. Priority 0 is shown because p2p doesn't use it.
2. **Compare types:** `show ip ospf interface Gi0/1 | include Network Type` gives `Network Type POINT_TO_POINT` on R1 and `Network Type BROADCAST` on R2.
3. **Why routes break:** R2, believing it's on a broadcast segment, runs a DR election and the link gets described as a transit network with a Type 2 network LSA. R1 describes the same link as a point-to-point link with no Type 2. SPF needs both ends to describe the link the same way before it will use it (the "two-way check" in the LSDB), so the link isn't used and nothing beyond it is reachable.
4. **Fix:** same type on both ends: `ip ospf network point-to-point` on R2 (the usual choice for a two-router Ethernet link) or remove it from R1.

27.5 Area Types and Router Types

The problem: in a big network, every router holding every LSA costs memory and CPU, and every flap anywhere triggers work everywhere. A small branch area with one way out doesn't need to know the 5,000 external routes learned from the internet edge: whatever the destination, its answer is "go to the ABR". **Area types** let you stop whole categories of LSA at the area border and replace them with a default route.

Router types (one router can be several):

Type	Definition
Internal	all interfaces in one area
Backbone	at least one interface in Area 0
ABR	interfaces in Area 0 **and** another area; generates Type 3 (and Type 4) LSAs
ASBR	redistributes external routes into OSPF; generates Type 5 (or Type 7 in an NSSA)

Area types decide which LSAs get in. The table is the core of this section:

Area type	Config (all routers in the area)	Blocks	Default route
Backbone (Area 0)	—	nothing	—
Normal	—	nothing	only if originated
Stub	`area 1 stub`	Types **4, 5**	ABR injects default **(Type 3)** automatically
Totally stubby	`area 1 stub` + on the **ABR**: `area 1 stub no-summary`	Types **3** (except the default), **4, 5**	automatic Type 3 default
NSSA	`area 1 nssa`	Types 4, 5 — but **allows Type 7** from a local ASBR	**not automatic**: ABR needs `area 1 nssa default-information-originate` (Type 7 default)
Totally NSSA	`area 1 nssa` + on the **ABR**: `area 1 nssa no-summary`	Types 3 (except default), 4, 5; allows 7	automatic Type 3 default
Transit	(any normal area carrying a virtual link)	—	—

Story Time : think of a town with only one road out (a stub area). People there don't need a directory of every city in other countries (Type 5 externals). A single sign saying "everywhere else: take the highway" (the default route) gets them out just as well. A **totally stubby** town doesn't even need the list of other towns in the same country (Type 3 inter-area routes): the same sign covers those too. An **NSSA** is that same one-road town, except it also has a small local ferry to an island (a local ASBR with its own external routes). The town has to know about the ferry's destinations, so those are allowed in, as Type 7. The analogy stops where the "sign" is concerned: in a plain NSSA the default sign isn't put up automatically (below).

Worked example — what one internal router sees in each area type. The network: Area 0 contains R9, an ASBR redistributing 192.0.2.0/24 from the internet edge. R2 is the ABR between Area 0 and Area 1. R4 is an internal router in Area 1. Area 0 also holds 172.16.0.0/24. Here is R4's OSPF routing table as Area 1's type changes:

Area 1 type	R4's OSPF routes	Why
Normal	`O` Area 1 subnets, `O IA 172.16.0.0/24`, `O E2 192.0.2.0/24`	everything is flooded in
Stub	`O` Area 1 subnets, `O IA 172.16.0.0/24`, `O*IA 0.0.0.0/0`	R2 blocks the Type 5 (and Type 4) and injects a Type 3 default
Totally stubby	`O` Area 1 subnets, `O*IA 0.0.0.0/0`	R2 also blocks the Type 3s, except the default
NSSA, and R6 (another Area 1 router) redistributes 198.51.100.0/24	`O` Area 1 subnets, `O IA 172.16.0.0/24`, `O N2 198.51.100.0/24` (no default unless configured)	Type 5s blocked; R6's external is a Type 7 inside the area

And on R9 in Area 0, R6's 198.51.100.0/24 appears as **`O E2`**, because R2 translated the Type 7 into a Type 5 at the border. The **same prefix** has a different code depending on where you look: `N2` inside the NSSA, `E2` everywhere else.

How LSAs move at R2, step by step (NSSA case):

1. R6 redistributes 198.51.100.0/24 and floods a **Type 7** LSA, but only inside Area 1.
2. R2 (the ABR) receives the Type 7, and as the NSSA's translator it creates a **Type 5** for the same prefix and floods that into Area 0.
3. R9 redistributes 192.0.2.0/24 as a **Type 5** in Area 0. When R2 would flood it into Area 1, the NSSA rule stops it.
4. R2 sends Area 0's internal prefixes (172.16.0.0/24) into Area 1 as **Type 3** LSAs, because a plain NSSA blocks externals, not inter-area routes.

Why block Type 4 along with Type 5? A Type 4 LSA only says "here's how to reach ASBR X", and it exists so routers can use the Type 5s that ASBR originates. Block the Type 5s and the Type 4s are useless.

Why doesn't a plain NSSA get an automatic default route, when a stub area does? Because an NSSA, unlike a stub area, may have a **way out of its own**: the local ASBR's external connection. Injecting a default automatically could pull traffic toward the ABR that should leave through the local ASBR, or the reverse. Cisco makes you decide, with `default-information-originate`. **Totally NSSA** does get the automatic default, because `no-summary` has removed inter-area routes, and without a default nothing outside the area would be reachable.

What happens to Type 7s: the NSSA's ABR **translates Type 7 to Type 5** as it floods them into Area 0. If there are several ABRs, the one with the **highest router ID** does the translation. Elsewhere in the domain the routes appear as normal `O E1/E2`; inside the NSSA they show as **`O N1/N2`**.

Default Routes in Stub and NSSA Areas — the ENARSI Detail

Normal-area default origination (`default-information originate [always] [metric] [metric-type] [route-map]`, and the "no default in the RIB, nothing originated" fault) is covered in 24.8. NSSAs have their own command and their own rule:

```
router ospf 1
 area 1 nssa default-information-originate [metric 10] [metric-type 1]
```

Where it's configured	Behaviour
On the **NSSA ABR**	Originates a **Type 7 default** into the NSSA **unconditionally**. No default is needed in the ABR's RIB, because the ABR's way out is Area 0
On an **NSSA ASBR that is not an ABR**	Originates the Type 7 default **only if a default route exists in its RIB**, like `default-information originate` without `always`
`area 1 nssa no-summary` on the ABR	Totally NSSA: a **Type 3** default is injected automatically, with no `default-information-originate` needed

Two more NSSA knobs show up in troubleshooting questions:

```
router ospf 1
 area 1 nssa no-redistribution                ! ABR that is also an ASBR: redistribute
                                              !   into normal areas only, not into the NSSA
 area 1 nssa translate type7 suppress-fa      ! translated Type 5s carry FA 0.0.0.0
```

`no-redistribution` stops an ABR's own redistributed routes from being injected into the NSSA as Type 7s, which avoids duplicates when they also arrive through Area 0. `suppress-fa` is the fix for the **forwarding-address black hole** described in 24.3. If routers outside the NSSA can't reach the Type 7's forwarding address, perhaps because that subnet is filtered or summarized away at the ABR, the translated Type 5s are never installed. With the FA suppressed, traffic is sent to the translating ABR instead.

Rules that create exam questions:

- **Area 0 can't be stub or NSSA.**
- A **virtual link can't cross a stub or NSSA area** (27.6).
- A stub or totally stubby area **can't contain an ASBR**. Redistributing there means the area has to be an NSSA.
- `no-summary` only matters **on the ABR**. Internal routers just need `stub`/`nssa`, and that flag must still match everywhere (27.2).

27.6 Virtual Links

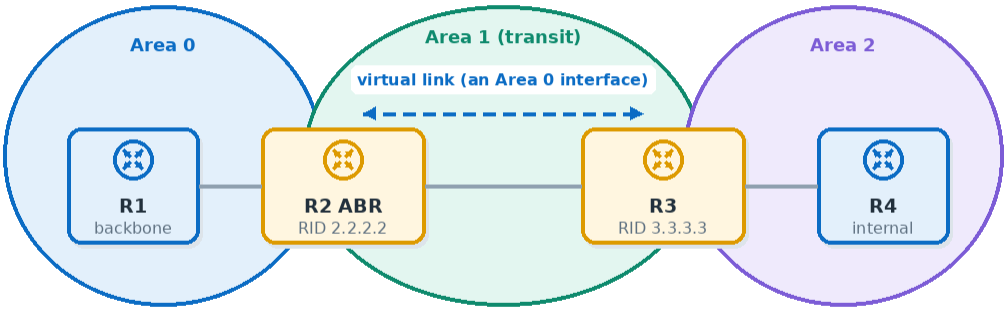
The problem: sometimes the physical network breaks OSPF's design rules. After a merger, a new area hangs off a non-backbone area. Or a single link failure cuts Area 0 into two halves. Re-cabling may take weeks. A virtual link is the software patch that makes OSPF behave as though the missing backbone connection exists.

Story Time : an office with no door onto the main corridor. Rather than knocking a hole in the wall, you agree with the neighboring office that people may walk through it to reach the corridor. The neighbor's office is the **transit area**, and the agreed walkway is the virtual link. It only works

if the neighbor's office is open-plan (a normal area). A stub area is like an office with locked internal doors: it doesn't hold the full information needed to pass traffic through.

OSPF's rule: **every area must attach to Area 0**, and Area 0 must be contiguous. A **virtual link** is the repair when that can't be done physically: a logical Area 0 adjacency tunneled through a **transit area**.

A virtual link stitches Area 2 to the backbone through transit Area 1



Area 2 doesn't touch Area 0 — R3 only becomes an ABR via the virtual link

transit area must be a normal area (never stub/NSSA)

```
! On R2 (ABR between Area 0 and Area 1)
router ospf 1
 area 1 virtual-link 3.3.3.3          ! the FAR end's ROUTER ID, not an IP address
! On R3 (Area 1 and Area 2)
router ospf 1
 area 1 virtual-link 2.2.2.2
```

Why does Area 2 need a backbone connection at all? R3 is connected to it. Because OSPF's inter-area design is a strict **hub and spoke around Area 0**. An ABR only advertises summaries into Area 0 and out of Area 0. Without an Area 0 interface, R3 isn't really an ABR: it won't generate Type 3s for Area 2 into Area 1, and the rest of the network never learns Area 2's prefixes. The virtual link gives R3 an Area 0 interface, so it becomes a true ABR. The "every area touches Area 0" rule is what prevents inter-area loops. Distance-vector-style summaries only flow through the backbone, so they can't circle around.

The two uses:

1. **An area not attached to Area 0** (as drawn), common after a merger or a quick fix.
2. **A partitioned (discontiguous) Area 0.** A link failure splits the backbone in two, and a virtual link through a normal area joins the halves.

The rules:

- Configured with the **remote router ID**, in the **transit area's** number.
- The transit area **must be normal**, never stub or NSSA, because the virtual link's traffic has to be routable across it with full information.
- The virtual link is **in Area 0**, so it inherits Area 0's authentication (27.3).
- Verify with `show ip ospf virtual-links` (look for `Virtual Link OSPF_VL0 to router 3.3.3.3 is up` and `Adjacency State FULL`). `show ip ospf neighbor` lists it on `OSPF_VL0`.

How the virtual link works, step by step:

1. R2 and R3 each find the other's RID in Area 1's SPF tree, using Area 1's router LSAs. (The RID doesn't need to be a routable address; SPF locates the router and supplies an interface address to send to.)
2. They send OSPF packets to each other as **unicast**, across Area 1, treating the path as a point-to-point link belonging to **Area 0**.
3. They form a FULL Area 0 adjacency over it and exchange Area 0's LSDB.
4. R3 now has an Area 0 interface, so it's a real ABR: it advertises Area 2's prefixes into Area 0 as Type 3s, and the rest of the network into Area 2.
5. The virtual link's cost is R2's (or R3's) intra-area cost through Area 1 to the other end. If Area 1's path changes, the virtual link's cost changes with it.

Worked example — verifying it:

```
R3# show ip ospf virtual-links
Virtual Link OSPF_VL0 to router 2.2.2.2 is up
  Run as demand circuit
  DoNotAge LSA allowed.
  Transit area 1, via interface GigabitEthernet0/0
  Transmit Delay is 1 sec, State POINT_TO_POINT,
  Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
    Hello due in 00:00:03
    Adjacency State FULL (Hello suppressed)
```

Reading it: `is up` and `Adjacency State FULL` are the two lines that matter. `Transit area 1` confirms the area it crosses. `Hello suppressed` and `Run as demand circuit` are normal: once the adjacency is up, the virtual link doesn't send periodic hellos and its LSAs don't age out.

Worked ticket — the virtual link that stays down. `show ip ospf virtual-links` on R3 says `Virtual Link OSPF_VL0 to router 10.1.23.2 is down`.

1. **Read the line.** The far end is named `10.1.23.2`, an interface address. A virtual link is addressed by **router ID**.
2. **Check the far end's RID:** `show ip ospf | include ID` on R2 gives `Routing Process "ospf 1" with ID 2.2.2.2`. No router in the network has RID 10.1.23.2, so R3 is trying to reach a router that doesn't exist.
3. **Fix:** `no area 1 virtual-link 10.1.23.2`, then `area 1 virtual-link 2.2.2.2`. The link comes up and Area 2's routes appear in Area 0.

The other causes to check, in order, when the RIDs are right: the **transit area number** must be the area R2 and R3 actually share, on both ends; the transit area must be **normal** (IOS won't accept a virtual link through a stub or NSSA); and if Area 0 is authenticated, the virtual link needs a key too (27.3).

A virtual link is a repair, not a design. If your topology needs one long-term, renumber the areas.

27.7 Path Preference

The problem: in a real network the same prefix can be learned several ways: from inside the area, from another area through an ABR, and from outside OSPF through one or more ASBRs. You need to predict which one OSPF installs, and know which knob changes it.

When OSPF knows several routes to the same prefix, it doesn't compare cost first. It compares **route type** first, and only uses cost within the same type:

Rank	Code	Route type	Cost compared
1	`O`	Intra-area	total cost
2	`O IA`	Inter-area	total cost
3	`O E1` / `O N1`	External type 1	external metric **+** internal cost to the ASBR
4	`O E2` / `O N2`	External type 2 (**default**)	external metric only; tie → lowest **forwarding metric** (cost to the ASBR)

Why does an intra-area route beat a cheaper inter-area one? For loop prevention, again. Intra-area routes come from the router's own SPF over a complete map. Inter-area routes are distance-vector-style summaries passed on by ABRs. Trusting the full map before the summary keeps the hub-and-spoke area model consistent. An `O` with cost 500 beats an `O IA` with cost 10, and the exam will test exactly that.

E1 vs E2 — when does it matter? E2 (the default, seed metric **20**; **1** for routes redistributed from BGP) keeps the external metric **constant** throughout the OSPF domain. Every router sees the same cost no matter how far away the ASBR is. That's fine with a single exit. **E1 adds** the internal path cost, so with two ASBRs advertising the same prefix, each router picks the **closer** exit. Use E1 when you have multiple exits and want each router to use its nearest one. With E2 and equal external metrics, the tiebreaker is the **forwarding metric** (the internal cost to the ASBR), which you can see in `show ip ospf database external` and `show ip route`.

```
R5# show ip route 192.0.2.0
Routing entry for 192.0.2.0/24
  Known via "ospf 1", distance 110, metric 20, type extern 2, forward metric 64
```

Worked example — E1 versus E2 with real numbers. R5 can leave the network through two ASBRs. Both advertise 192.0.2.0/24:

ASBR	External metric it sets	R5's internal cost to that ASBR
R3	20	10
R4	10	30

As E2: R5 compares only the external metrics. R4's 10 beats R3's 20, so R5 sends traffic to **R4**, even though R4 is three times farther away inside the network. The internal distance (the forwarding metric) is only used to break a tie, and there's no tie here.

As E1: R5 adds its internal cost. Via R3: 20 + 10 = **30**. Via R4: 10 + 30 = **40**. R5 sends traffic to **R3**, the exit that's actually closer overall.

Same metrics, tie case (E2): if both ASBRs set 20, E2 sees a tie and falls back to the forwarding metric: R3 (10) beats R4 (30). That's the `forward metric` value in `show ip route`.

One more rule: if R5 holds both an E1 and an E2 route for the same prefix, the **E1 wins regardless of the numbers**, because type 1 ranks above type 2 in the table.

27.8 Reading the LSDB

When routes are missing, the LSDB tells you whether the information **arrived** and the problem is in route calculation, or whether it **never arrived**.

```
R1# show ip ospf database

            OSPF Router with ID (1.1.1.1) (Process ID 1)

                Router Link States (Area 0)
Link ID         ADV Router      Age   Seq#       Checksum Link count
1.1.1.1         1.1.1.1         412   0x80000007 0x00A1B2 3
2.2.2.2         2.2.2.2         398   0x80000005 0x003C4D 2

                Net Link States (Area 0)
Link ID         ADV Router      Age   Seq#       Checksum
10.1.12.2       2.2.2.2         398   0x80000002 0x00E5F6

                Summary Net Link States (Area 0)
Link ID         ADV Router      Age   Seq#       Checksum
172.16.1.0      2.2.2.2         377   0x80000001 0x0012AB

                Type-5 AS External Link States
Link ID         ADV Router      Age   Seq#       Checksum Tag
192.0.2.0       3.3.3.3         210   0x80000001 0x0077CD 0
```

How to read it:

- **Router Link States** = Type 1, one per router in the area. **Net Link States** = Type 2, one per DR segment; the Link ID is the **DR's interface IP**.
- **Summary Net** = Type 3 from the ABR shown in ADV Router. **Summary ASB** = Type 4. **Type-5 AS External** = redistributed routes from the ASBR. **Type-7** appears only inside an NSSA.
- **Age** counts up to **3600 s** (MaxAge, then the LSA is flushed). Each LSA is refreshed every **1800 s**, with its **sequence number** incremented. A sequence number that keeps climbing quickly means a flapping LSA, often the duplicate-RID flood war from 27.2.

Drill-down commands: `show ip ospf database router 2.2.2.2` (that router's links and costs), `show ip ospf database external 192.0.2.0` (metric type, forward address), `show ip ospf border-routers` (the ABRs and ASBRs this router can reach). If a Type 5 is in the database but the route isn't in the table, the ASBR is probably unreachable. `show ip ospf border-routers` will be missing it.

Worked example — "the LSA is there, the route isn't". R1 has `192.0.2.0` in its Type-5 list, advertised by 3.3.3.3, but `show ip route 192.0.2.0` says `% Network not in table`.

1. **The information arrived** (it's in the LSDB), so the problem is in the calculation, not the flooding.
2. To use a Type 5, R1 must be able to reach its **advertising router**, 3.3.3.3. `show ip ospf border-routers` on R1 doesn't list 3.3.3.3.
3. 3.3.3.3 is in another area, so R1 needs a **Type 4** (`show ip ospf database asbr-summary`) from an ABR to find it. There isn't one: typically the ASBR's area has lost its connection to Area 0 (a

broken ABR link or a missing virtual link), so no ABR can describe 3.3.3.3 to R1's area.

4. **Fix the reachability of the ASBR** (restore the backbone connection), and the external route installs without touching the external LSA itself.

27.9 The OSPF Troubleshooting Checklist

Layer	Question	Command
Interfaces	Is OSPF on the right interfaces, in the right area, with the right type and timers?	`show ip ospf interface brief` / `show ip ospf interface Gi0/1`
Neighbors	Are they FULL (or 2-WAY between DROTHERs)?	`show ip ospf neighbor`
Process	RID, area types, reference bandwidth, `distribute-list`s?	`show ip ospf`, `show ip protocols`
Database	Did the LSA arrive? From whom?	`show ip ospf database …`
Reachability of the advertiser	ABR/ASBR reachable?	`show ip ospf border-routers`
RIB	Did OSPF's route lose to a lower AD?	`show ip route prefix` (Chapter 22)

Route-level causes to remember:

Symptom	Cause
Externals missing in one area only	area is stub/totally stubby (expected — check for the default)
Inter-area routes missing in one area	`no-summary` on its ABR, or an `area X filter-list` / `area X range … not-advertise` (Chapter 6)
Area's routes missing everywhere else	area not attached to Area 0 → virtual link needed (27.6)
Prefix in LSDB, not in RIB	`distribute-list in` filtering the RIB (LSDB unaffected — Chapter 23), or a lower-AD source
Route via the far exit instead of the near one	externals are E2 — use E1 (27.7)
Suboptimal path everywhere	`auto-cost reference-bandwidth` inconsistent between routers (CCNA 16.6)
Default route missing in a normal area	no default in the originator's RIB and no `always` (24.8)
Externals in the LSDB, not in the RIB, on routers outside an NSSA	forwarding address unreachable (24.3) — `suppress-fa` or stop filtering the FA subnet
Loopback advertised as /32 instead of /24	loopback network type — `ip ospf network point-to-point`

Most OSPF faults come down to two routers disagreeing about something: an area, a type, a timer, a key, an RID or a reference bandwidth. Find the disagreement and you've found the fault.

27.10 Walk-Through: Tracing an ABR Filter

ABR filtering (`area X filter-list`, `area X range … not-advertise`, Chapter 6, 6.6) is quiet. It produces no log message and no error, and the prefix simply never leaves its area. Here's the method on a real-looking fault.

Symptom: users in Area 2 can't reach 10.1.50.0/24, which lives in Area 1. Everyone else can.

Step 1 — Is the route missing, or just losing? On an Area 2 router:

```
R6# show ip route 10.1.50.0
% Subnet not in table
R6# show ip ospf database summary 10.1.50.0
            OSPF Router with ID (6.6.6.6) (Process ID 1)
R6#                                          <- no Type 3 at all
```

No Type 3 in Area 2's database means the problem is upstream, at an ABR, not in R6's calculation.

Step 2 — Does the Area 0 side have it? On a backbone router:

```
R2# show ip ospf database summary 10.1.50.0
                Summary Net Link States (Area 0)
  Link State ID: 10.1.50.0 (summary Network Number)
  Advertising Router: 3.3.3.3                  <- Area 1's ABR advertised it into Area 0
```

So Area 1's ABR (3.3.3.3) is fine, and the prefix reached the backbone. The loss is at the ABR **between Area 0 and Area 2**.

Step 3 — Inspect that ABR's filters:

```
R5# show run | section router ospf
router ospf 1
 area 2 filter-list prefix NO-LAB in
R5# show ip prefix-list NO-LAB
ip prefix-list NO-LAB: 2 entries
   seq 5 deny 10.1.0.0/16 le 24
   seq 10 permit 0.0.0.0/0 le 32
```

`area 2 filter-list … in` filters Type 3s going **into** Area 2. Seq 5 was meant to block a lab /16 but uses `le 24`, so it also catches 10.1.50.0/24. Fix the prefix list (for example `deny 10.1.0.0/16` with no `le`, to block only the /16 itself, or a deny for the real lab range) and check `show ip prefix-list detail NO-LAB` hit counts afterwards.

What to remember from the method: work outward from the router with the problem using **`show ip ospf database summary <prefix>`** in each area, and find the area boundary where the Type 3 stops existing. The ABR at that boundary is where the filter is. Remember the direction, too: `in` filters Type 3s entering the named area, `out` filters Type 3s generated from the named area's routes. And `distribute-list in` would have left the LSA in the database and removed only the route (Chapter 23). LSA present but route missing points at a distribute list or AD. LSA absent points at an ABR filter or range.

Chapter 28: BGP In Depth

> **Blueprint:** ENARSI 1.11 — Troubleshoot BGP (internal and external, unicast, and VRF-Lite): 1.11.a address families (IPv4, IPv6) · 1.11.b neighbor relationship and authentication (next-hop, multihop, 4-byte AS, private AS, route refresh, synchronization, operation, peer group, states and timers) · 1.11.c path preference (attributes and best-path) · 1.11.d route reflector (excluding multiple route reflectors, confederations, dynamic peer) · 1.11.e policies (inbound/outbound filtering, path manipulation)

Chapter 7 covered the BGP foundations: path-vector routing, eBGP between directly connected neighbors, the message types, and the best-path algorithm in outline. ENARSI adds **iBGP** and everything it brings with it: next-hop handling, loopback peering, route reflectors, address families, VRFs. It also adds the policy tools that make BGP useful, and the troubleshooting skill to see why a session isn't Established or why the "wrong" path won.

The mindset: BGP has no automatic discovery and almost no automatic behavior. Every neighbor is configured by hand, every policy is explicit, and the best-path algorithm is deterministic. So when BGP does something surprising, the reason is always visible in the output somewhere. Your job is to know where to look.

Words you'll meet in this chapter:

Term	Plain meaning
AS	Autonomous System: one organization's network under one routing policy, identified by an AS number (ASN)
eBGP / iBGP	BGP between routers in **different** ASes / in the **same** AS
IGP	Interior Gateway Protocol: the routing protocol inside your AS (OSPF, EIGRP). BGP relies on it to reach next hops and peers' loopbacks
Prefix / NLRI	a route BGP advertises (Network Layer Reachability Information is the RFC's name for it)
Path attribute	a property attached to a route: AS_PATH, NEXT_HOP, LOCAL_PREF, MED and so on. Best-path selection compares these
Next hop	the address BGP says to send traffic to for that prefix. It doesn't have to be directly connected; the router looks it up again in the routing table (a recursive lookup)
RR	Route Reflector: an iBGP router allowed to pass iBGP routes between other iBGP routers
AF	address family: IPv4 unicast, IPv6 unicast, VPNv4… each one activated separately per neighbor

Story Time — BGP in one picture: BGP is less like a road map and more like a **list of recommendations with notes attached**. Each neighbor says "you can reach 8.8.8.0/24 through me; here's the list of companies it passes through (AS_PATH), here's where to hand it over (NEXT_HOP)". Your router reads all the recommendations for a prefix, applies your company's rules (LOCAL_PREF, WEIGHT), and picks one. Nothing is discovered automatically; every friend you take recommendations from is configured by name.

28.1 iBGP vs eBGP — Different Rules for Different Neighbors

The problem: your border routers learn the internet's routes from your ISPs over eBGP. Your other routers need those routes too, so the border routers must pass them on inside the AS, over iBGP. But iBGP follows different rules from eBGP, and almost every "the route is in BGP but nothing works" ticket comes from one of those differences.

Behavior	eBGP (different AS)	iBGP (same AS)
Administrative distance	**20**	**200**
Default TTL of BGP packets	**1** — neighbor must be directly connected	255 — can be many hops away
Next hop when advertising	changed to **self**	**unchanged** (28.2)
AS_PATH	own AS **prepended**	unchanged
Re-advertising learned routes	to all peers	**never to another iBGP peer** (iBGP split horizon)
LOCAL_PREF	not sent	sent
Typical peering address	directly connected interface	**loopback**

Why can't an iBGP router pass an iBGP-learned route to another iBGP peer? Because inside one AS, BGP has **no loop detection**. eBGP spots loops by finding its own AS number in the AS_PATH, but iBGP doesn't add to the AS_PATH, so a route circling among iBGP peers would look the same every time around. The **iBGP split-horizon rule** removes the possibility: a route learned from an iBGP peer is never sent to another iBGP peer. The consequence is that every iBGP router must hear every route **directly** from the router that learned it. That means either a **full mesh** of iBGP sessions, n(n−1)/2 of them, or a **route reflector** (28.7) that's allowed to break the rule under controlled conditions.

Story Time : an office rule about news. "If you hear news from **outside** the company, tell all your colleagues. If you hear it from a **colleague**, don't repeat it to other colleagues." The rule stops the same rumour circling the office forever. The cost is that whoever hears outside news must tell **every** colleague personally (the full mesh), unless the office appoints a team lead who is allowed to repeat it (the route reflector).

Worked example — counting sessions. An AS has 10 iBGP routers.

- Full mesh: 10 × 9 / 2 = **45** iBGP sessions, and adding an 11th router means touching all 10 existing routers.

- One route reflector with 9 clients: **9** sessions, and adding a router means configuring only the new router and the RR.

Why peer iBGP on loopbacks? Because the session should survive any single link failure. A loopback never goes down, and the IGP finds some path to it as long as one exists. Peering on a physical interface IP ties the session to that one cable. Two pieces of configuration make it work:

```
router bgp 65001
 neighbor 2.2.2.2 remote-as 65001
 neighbor 2.2.2.2 update-source Loopback0    ! source my packets from my loopback
```

Why is `update-source` needed? Because BGP runs over TCP, and TCP checks addresses. R2 is configured with `neighbor 1.1.1.1`, so it only accepts a connection from 1.1.1.1. Without `update-source`, R1 sources the TCP SYN from its outgoing physical interface. R2 sees an unknown address and refuses it, so **R1's** connection attempts fail. Strictly, the session can still come up if R2 has `update-source Loopback0`: R2 opens the TCP connection from 2.2.2.2 to 1.1.1.1, R1 recognises 2.2.2.2 as a configured neighbor and accepts it. The session only stays stuck in **Active** when **neither** side can open a connection the other will accept. Configure `update-source` on both ends anyway, so either side can initiate and the behavior is predictable — and both ends need a route to the other's loopback.

eBGP between loopbacks (often used to load-share across two parallel links) runs into the TTL-1 rule, because the loopback is one hop beyond the directly connected interface:

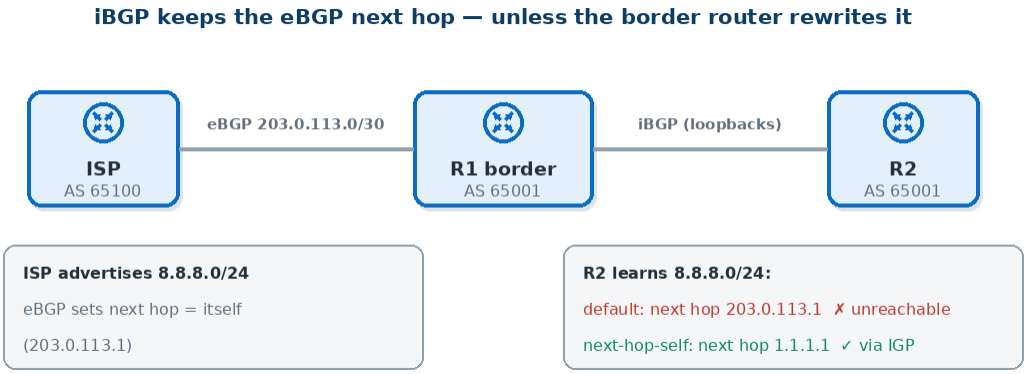
Command	What it does	Use when
`neighbor X ebgp-multihop 2`	sends eBGP packets with TTL 2 (or any number up to 255)	the eBGP peer really is more than one hop away
`neighbor X disable-connected-check`	keeps TTL 1, but skips the "peer address must be on a connected subnet" check	loopback peering between **directly connected** routers
`neighbor X ttl-security hops 1`	sends with TTL **255**, accepts only TTL ≥ 254 (GTSM)	protecting eBGP from spoofed remote packets; **can't be combined with `ebgp-multihop`**

Why is TTL security stronger than TTL 1? Because an attacker anywhere on the internet can craft a packet that **arrives** with TTL 1. They just set the starting TTL to the right number of hops. Nobody, though, can make a packet arrive with TTL 254 from more than one hop away, because every router along the way decrements it. Starting high and requiring a high value on arrival proves the packet came from a directly connected router.

Worked example — TTL security in numbers. R1 and its ISP peer are directly connected, both with `ttl-security hops 1`. The accepted minimum is 255 − 1 = **254**.

1. The ISP router sends its BGP packets with TTL **255**. They cross no other router, so they arrive with TTL **255**. 255 ≥ 254: accepted.
2. An attacker five routers away also sends with TTL 255. Each of the five routers decrements it, so it arrives with **250**. 250 < 254: dropped before BGP ever looks at it.
3. With plain eBGP (TTL 1 sent, no check on arrival), the attacker would simply send with TTL 6, so the packet arrives with TTL 1 and looks exactly like the real peer's.

28.2 Next Hop and `next-hop-self`



The problem: BGP tells a router where to hand traffic over, the next hop, but it doesn't tell the router how to get there. The router has to find the next hop in its own routing table. If it can't, the BGP route is useless, however good it looks.

When R1 learns 8.8.8.0/24 from the ISP, the next hop is the ISP's address, 203.0.113.1. When R1 passes the route to iBGP peer R2, **the next hop doesn't change**. R2 needs a route to 203.0.113.1, and the IGP usually doesn't carry the ISP link's subnet.

```
R2# show ip bgp 8.8.8.0
BGP routing table entry for 8.8.8.0/24, version 0
Paths: (1 available, no best path)
  65100
    203.0.113.1 (inaccessible) from 1.1.1.1 (1.1.1.1)
      Origin IGP, metric 0, localpref 100, valid, internal
```

`(inaccessible)` **and** `no best path` are the symptom. The route is in the BGP table but can't be used, because step 0 of the best-path algorithm (28.6) is "**is the next hop reachable?**". The two fixes:

```
router bgp 65001
 neighbor 2.2.2.2 next-hop-self      ! on R1: rewrite next hop to my update-source (1.1.1.1)
! — or — advertise the ISP link into the IGP (passive) so 203.0.113.1 is reachable
```

Why doesn't iBGP change the next hop by default? Because the original next hop is often the **better** forwarding answer. In a shared-segment design, or when the exit router isn't the BGP speaker, the true exit is the eBGP peer, and rewriting it would detour traffic through R1. BGP keeps the most specific forwarding information it has and leaves the choice to you. In a typical enterprise edge, `next-hop-self` on the border router toward its iBGP peers is the standard configuration.

Story Time : a parcel arrives at your company's reception with the note "deliver via Door 203.0.113.1". Reception (R1) knows where that door is, because it's right next to it. It forwards the parcel to the third floor (R2) with the note unchanged. The third floor has never heard of Door 203.0.113.1, so the parcel sits there. `next-hop-self` is reception crossing out the note and writing "bring it back to reception (1.1.1.1)", an address every floor knows how to reach.

How R2 resolves the route, step by step, before and after the fix:

1. R1 learns 8.8.8.0/24 from the ISP with next hop 203.0.113.1, and sends it to R2 over iBGP with the next hop **unchanged**.
2. R2 looks up 203.0.113.1 in its routing table. The ISP link's subnet isn't in the IGP, so there's no match. R2 marks the path **inaccessible**, and it can't be best.
3. Add `neighbor 2.2.2.2 next-hop-self` on R1, then `clear ip bgp 2.2.2.2 soft out` so R1 resends its routes.
4. R2 now receives the route with next hop **1.1.1.1**, R1's loopback. R2's OSPF already has 1.1.1.1/32, so the lookup succeeds and the path becomes best.
5. Traffic from R2 to 8.8.8.8 goes to 1.1.1.1 using OSPF, and R1 forwards it to the ISP.

R2's view after the fix:

```
R2# show ip bgp 8.8.8.0
BGP routing table entry for 8.8.8.0/24, version 15
Paths: (1 available, best #1, table default)
  65100
    1.1.1.1 (metric 2) from 1.1.1.1 (1.1.1.1)
      Origin IGP, metric 0, localpref 100, valid, internal, best
```

`(metric 2)` is R2's IGP cost to the next hop: the value best-path step 9 compares.

28.3 Sessions: States, Timers and Why They Get Stuck

The problem: a BGP session that isn't Established exchanges no routes at all. Unlike OSPF and EIGRP, BGP runs over **TCP port 179**, so a session can fail for TCP reasons (reachability, source address, TTL, ACLs, MD5) or for BGP reasons (wrong AS, wrong router ID). The state it's stuck in tells you which.

Story Time : setting up a business phone call. **Idle**: you haven't picked up the phone. **Connect**: it's ringing. **Active**: the call didn't connect, so you're waiting for them to call you, and you'll redial shortly. **OpenSent**: they answered and you've introduced yourself ("I'm AS 65001, router 1.1.1.1"). **OpenConfirm**: they've introduced themselves and you're waiting for "OK, let's talk". **Established**: business begins. A call that never connects is a phone problem (TCP). A call that connects and then gets hung up straight after introductions is a "wrong person" problem (the OPEN parameters).

How a session comes up, step by step (R1 1.1.1.1 AS 65001, R2 2.2.2.2 AS 65001, loopback peering):

1. **Idle → Connect.** R1 has a route to 2.2.2.2, so it starts a TCP connection: SYN from 1.1.1.1 (thanks to `update-source Loopback0`) to 2.2.2.2 port 179.
2. **Connect → OpenSent.** R2 recognises 1.1.1.1 as a configured neighbor and completes the TCP handshake. R1 sends its **OPEN** message: version 4, my AS 65001, my hold time 180, my BGP ID 1.1.1.1, my capabilities (route refresh, 4-byte AS, address families).
3. **OpenSent → OpenConfirm.** R1 receives R2's OPEN and checks it: is the AS what my `remote-as` says? Is the hold time acceptable? Is the BGP ID valid (an iBGP peer using my own router ID is rejected)? If all is well, R1 sends a **KEEPALIVE** to accept it.

4. **OpenConfirm → Established.** R1 receives R2's KEEPALIVE. Both sides now send **UPDATE** messages with their routes, then KEEPALIVEs every 60 s.
5. If TCP fails at step 1 or 2, R1 goes to **Active**, keeps listening for R2 to connect to it, and retries later. If a check fails at step 3, R1 sends a **NOTIFICATION** naming the error and closes the session.

BGP's finite-state machine:

State	Meaning	Stuck here usually means
Idle	not trying (or waiting before retrying)	no route to the neighbor; `neighbor shutdown` → `Idle (Admin)`; max-prefix exceeded → `Idle (PfxCt)`
Connect	TCP three-way handshake in progress	— (brief)
Active	the TCP attempt **failed**; retrying and listening	wrong neighbor IP, missing `update-source`, missing `ebgp-multihop`, ACL blocking **TCP 179**, peer not configured
OpenSent	TCP up, OPEN sent, waiting for theirs	—
OpenConfirm	OPENs exchanged, waiting for KEEPALIVE	—
Established	exchanging UPDATEs	healthy

The trap in the name: "Active" sounds healthy, but it means **failure**. The router is actively trying to connect and not succeeding. A session that **flips between Idle and Active** is a TCP problem. A session that reaches OpenSent and then drops back is a **parameter mismatch** detected in the OPEN message, and the router logs a NOTIFICATION that names it:

```
%BGP-3-NOTIFICATION: sent to neighbor 10.1.12.2 passive 2/2 (peer in wrong AS) 2 bytes FDE9
%BGP-5-ADJCHANGE: neighbor 10.1.12.2 Down BGP Notification sent
```

`2/2` is the OPEN error **bad peer AS**. The `FDE9` is the AS the peer actually sent (0xFDE9 = 65001).

Timers: keepalive **60 s**, hold **180 s** by default. They're **negotiated**: each side proposes a hold time in its OPEN, and both use the **lower** value. Set them per neighbor with `neighbor X timers 10 30` or globally with `timers bgp 10 30`. The timers don't have to match, unlike OSPF's. For sub-second failure detection, use **BFD** (`neighbor X fall-over bfd`, Chapter 25), not aggressive timers.

Worked example — negotiated hold time. R1 is configured with `neighbor 10.1.12.2 timers 10 30` (keepalive 10, hold 30). R2 keeps the defaults (60/180). In the OPEN messages R1 proposes hold 30 and R2 proposes 180. Both routers use the lower, **30 seconds**. `show ip bgp neighbors 10.1.12.1 | include hold` on R2 (10.1.12.1 being R1's address) shows the negotiated value, which is not the one R2 was configured with. If R1 had proposed a hold time of 0, both sides would agree to send no keepalives and never time the session out, which is why you'll rarely see 0 outside a lab.

Authentication uses a TCP MD5 signature on every segment (RFC 2385):

```
router bgp 65001
 neighbor 10.1.12.2 password BGPs3cr3t
```

A mismatched or missing password means the TCP session never establishes. The session sits in **Active/Idle**, and the log names the cause directly:

```
%TCP-6-BADAUTH: Invalid MD5 digest from 10.1.12.2(179) to 10.1.12.1(34521)
%TCP-6-BADAUTH: No MD5 digest from 10.1.12.2(179) to 10.1.12.1(34521)
```

"Invalid" means the passwords differ. "No digest" means the other side has none configured.

The summary command shows everything at a glance:

```
R1# show bgp ipv4 unicast summary
BGP router identifier 1.1.1.1, local AS number 65001
Neighbor     V    AS  MsgRcvd MsgSent  TblVer InQ OutQ Up/Down  State/PfxRcd
2.2.2.2      4 65001     1204    1199       88   0    0 18:22:41       14
10.1.12.2    4 65100       31      29       88   0    0 00:27:10      812
10.1.13.3    4 65200        0       0        1   0    0 never    Active
```

A **number** in State/PfxRcd means Established, and the number is how many prefixes were received. A **word** (Idle, Active) means the session is down.

Worked ticket — the loopback session stuck in Active. R1 (1.1.1.1) and R3 (3.3.3.3) are in AS 65001 and should peer between loopbacks. `show bgp ipv4 unicast summary` on R1 shows `3.3.3.3 ... never Active`.

1. **Is it a routing problem?** `ping 3.3.3.3 source Loopback0` from R1 succeeds, so each router can reach the other's loopback. That rules out the Idle-type causes.
2. **Active means TCP keeps failing.** Look at how each side connects:

```
R1# show run | section router bgp
router bgp 65001
 neighbor 3.3.3.3 remote-as 65001
R3# show run | section router bgp
router bgp 65001
 neighbor 1.1.1.1 remote-as 65001
```

3. **Reason it through.** Neither side has `update-source`. R1 opens its TCP connection from its outgoing interface, 10.1.13.1. R3 only accepts connections from a configured neighbor, 1.1.1.1, so it rejects it. R3 does the same thing in reverse, from 10.1.13.3, and R1 rejects that. Neither direction can ever succeed.
4. **Fix:** `neighbor 3.3.3.3 update-source Loopback0` on R1 and `neighbor 1.1.1.1 update-source Loopback0` on R3. Within seconds the summary shows a prefix count instead of `Active`.
5. **If it's still Active** after that, the next suspect is an ACL on the path. BGP needs TCP port 179 in **both** directions: to port 179 from the side that opens the connection, and from port 179 for the replies. The safest entries are `permit tcp host 1.1.1.1 host 3.3.3.3 eq bgp` and `permit tcp host 1.1.1.1 eq bgp host 3.3.3.3` (plus the mirror pair on the other router's inbound ACL).

28.4 Address Families, MP-BGP and IPv6

The problem: the original BGP could only carry IPv4 prefixes. Networks now need IPv6 routes, VPN routes and more, and running a separate protocol for each would be a mess. Multiprotocol extensions let one BGP process (and even one session) carry many kinds of routes, each kept in its own table.

Multiprotocol BGP (MP-BGP) carries any **address family** (IPv4 unicast, IPv6 unicast, VPNv4, …) over any session. One TCP session can carry several families, and each neighbor must be **activated** per family.

```
router bgp 65001
 bgp router-id 1.1.1.1
 no bgp default ipv4-unicast                  ! don't auto-activate IPv4 for every neighbor
 neighbor 10.1.12.2 remote-as 65100           ! IPv4 transport session
 neighbor 2001:DB8:12::2 remote-as 65100      ! IPv6 transport session
 !
 address-family ipv4 unicast
  neighbor 10.1.12.2 activate
  network 198.51.100.0 mask 255.255.255.0
 exit-address-family
 !
 address-family ipv6 unicast
  neighbor 2001:DB8:12::2 activate
  network 2001:DB8:100::/48
 exit-address-family
```

Why use `no bgp default ipv4-unicast`? By default, IOS activates **every** neighbor for IPv4 unicast, including IPv6-transport neighbors. You end up with IPv4 routes being exchanged over sessions you meant for IPv6 only. Turning the default off makes activation explicit: a neighbor carries exactly the families you `activate`. It's what you want in any dual-stack design, and if you forget an `activate`, the session comes up but no prefixes are exchanged for that family.

Carrying IPv6 prefixes over an IPv4 session works, with one catch. The next hop has to be an IPv6 address, so IOS builds an **IPv4-mapped** one (`::FFFF:10.1.12.2`). It isn't routable, so the routes show as inaccessible. Either run a native IPv6 session for the IPv6 family (as above, the clean design), or set a real next hop with an outbound route map (`set ipv6 next-hop 2001:DB8:12::1`).

Verify per family: `show bgp ipv6 unicast summary`, `show bgp ipv6 unicast`, and `show bgp all summary`.

4-byte AS numbers. ASNs were 16-bit (to 65535) and are now 32-bit. There are two notations for the same number:

asplain	asdot	Calculation
65546	1.10	1 × 65536 + 10
131072	2.0	2 × 65536

`router bgp 65546` and `router bgp 1.10` are the same AS. To convert asplain to asdot, divide by 65,536: the whole number goes before the dot and the remainder after it. For example, the first 4-byte private ASN, 4,200,000,000, is 64,086 × 65,536 + 59,904, so in asdot it's **64086.59904**. `bgp asnotation dot` changes only how IOS **displays** it. Older speakers that only understand 2-byte ASNs see **AS 23456** (AS_TRANS) standing in for any 4-byte AS. If you see 23456 in a path, an old

router is somewhere upstream.

Private ASNs: 64512–65534 (2-byte) and **4200000000–4294967294** (4-byte). They're fine inside an organization or between a customer and its ISP, but they must never reach the internet. The ISP (or your edge) strips them:

```
neighbor 203.0.113.1 remove-private-as              ! strip if the path is ALL private ASNs
neighbor 203.0.113.1 remove-private-as all          ! strip private ASNs wherever they appear
neighbor 203.0.113.1 remove-private-as all replace-as   ! replace them with my own AS
```

28.5 Operations: Refreshing Policy, Synchronization, Peer Groups

The problem: BGP sessions can carry hundreds of thousands of routes and stay up for months. Dropping one just to apply a new route map would be a self-inflicted outage. You also need ways to keep dozens of neighbor configurations consistent. This section is the day-to-day operational toolkit.

Applying a policy change. BGP doesn't re-evaluate routes it already holds when you change a route map. You have to make it reprocess them:

Method	How	Cost
Hard reset	`clear ip bgp 10.1.12.2`	drops the session — **avoid in production**
Route refresh (default, negotiated capability)	`clear ip bgp 10.1.12.2 soft in`	asks the peer to resend its routes; no session drop, no extra memory
Soft reconfiguration inbound	`neighbor X soft-reconfiguration inbound`, then `clear … soft in`	stores an **unfiltered copy** of everything received — memory-heavy
Outbound	`clear ip bgp 10.1.12.2 soft out`	re-sends my routes through the new outbound policy

Why does `soft-reconfiguration inbound` still exist when route refresh works? For **troubleshooting**. With it, `show ip bgp neighbors X received-routes` shows exactly what the peer sent **before** your inbound policy filtered it, next to `routes` (what survived the policy). Comparing the two tells you immediately whether a route is missing because the peer never sent it or because your filter dropped it. Without it, `received-routes` isn't available. (`advertised-routes` always works for the outbound direction.)

Story Time : route refresh is phoning a colleague and saying "please resend me that list, I've changed how I file things". Soft reconfiguration inbound is keeping a **photocopy** of every list you've ever received, before you filed it. The photocopy costs drawer space (memory), but it lets you prove exactly what you were sent.

Worked example — "the ISP says they sent it". The ISP insists they advertise 812 prefixes, including 198.18.0.0/15, but R1 doesn't have that route. R1 has `soft-reconfiguration inbound`

toward the ISP.

```
R1# show ip bgp neighbors 10.1.12.2 received-routes | include 198.18
*  198.18.0.0/15    10.1.12.2                0             0 65100 i
R1# show ip bgp neighbors 10.1.12.2 routes | include 198.18
R1#
```

The prefix **was received** (it's in `received-routes`) but **didn't survive the inbound policy** (it's missing from `routes`). So the ISP is right, and the cause is on R1: an inbound prefix list or route map denies it. (198.18.0.0/15 is a reserved benchmarking range, so a bogon filter is probably doing its job.) Had it been missing from `received-routes` too, the problem would have been on the ISP's side.

Synchronization was an old rule: don't use or advertise an iBGP-learned route unless the same prefix is also in the IGP. It existed for transit ASes whose internal routers didn't all run BGP. It made sure traffic sent across the AS wouldn't reach a non-BGP router that would drop it. Modern designs run BGP on every transit router (or use MPLS), so synchronization is **disabled by default** (since IOS 12.2(8)T) and is deprecated — many current IOS XE releases no longer accept the `synchronization` command at all. Know what it was and why it's off. Turning it on would also mean redistributing the full BGP table into the IGP.

Peer groups and templates keep large neighbor lists consistent:

```
! Peer group — neighbors share one outbound policy (and one update calculation)
router bgp 65001
 neighbor IBGP peer-group
 neighbor IBGP remote-as 65001
 neighbor IBGP update-source Loopback0
 neighbor IBGP next-hop-self
 neighbor 2.2.2.2 peer-group IBGP
 neighbor 3.3.3.3 peer-group IBGP
```

```
! Peer templates — session and policy separated, inheritable, more flexible
router bgp 65001
 template peer-session IBGP-SESSION
  remote-as 65001
  update-source Loopback0
 exit-peer-session
 template peer-policy IBGP-POLICY
  next-hop-self
  send-community both
 exit-peer-policy
 neighbor 2.2.2.2 inherit peer-session IBGP-SESSION
 address-family ipv4 unicast
  neighbor 2.2.2.2 inherit peer-policy IBGP-POLICY
```

Why were peer groups originally about CPU, not convenience? Because a peer group's members are guaranteed to share the **same outbound policy**, so BGP computes the update **once** and copies it to every member. On a route reflector with hundreds of clients, that saves a lot of CPU. Modern IOS builds **update groups** automatically from any neighbors with identical outbound policy, so the CPU benefit no longer depends on peer groups. Templates add flexibility: a neighbor can inherit a session template and a policy template, and templates can inherit from each other.

28.6 Path Preference — the Best-Path Algorithm in Full

The problem: for most internet prefixes, a multi-homed router hears several paths. It must pick exactly one to use and to advertise onward, it must pick the same way every time, and you must be able to steer the choice with business policy. The best-path algorithm is that deterministic recipe.

BGP picks **one** best path per prefix (unless multipath is on), comparing candidates **one attribute at a time** in a fixed order. The first difference decides:

Step	Prefer	Scope / default	How you manipulate it
0	Next hop **reachable**	—	`next-hop-self`, IGP (28.2)
1	Highest **WEIGHT**	Cisco-only, **this router only**; default 0 (32768 if locally originated)	`neighbor X weight 200` or `set weight` (inbound)
2	Highest **LOCAL_PREF**	whole AS (iBGP); default **100**	`set local-preference 200` (inbound), `bgp default local-preference`
3	**Locally originated**	`network`, `aggregate-address`, `redistribute`	originate it yourself
4	Highest **AIGP**	rarely used; ignored unless configured	—
5	Shortest **AS_PATH**	`AS_SET` counts as 1	`set as-path prepend 65001 65001` (outbound); `bgp bestpath as-path ignore` skips this step
6	Lowest **ORIGIN**	IGP (`i`) < EGP (`e`) < incomplete (`?`)	`set origin igp`
7	Lowest **MED**	compared only between paths from the **same neighbor AS**; default 0	`set metric 50` (outbound)
8	**eBGP over iBGP**	—	—
9	Lowest **IGP metric to the next hop**	—	IGP costs
10	(multipath check if `maximum-paths` configured)	—	`maximum-paths 2` (28.11)
11	**Oldest** eBGP path	stability	—

Step	Prefer	Scope / default	How you manipulate it
12	Lowest **BGP router ID** of the advertiser	(originator ID if reflected)	`bgp router-id`
13	Shortest **CLUSTER_LIST**	reflected paths	—
14	Lowest **neighbor address**	—	—

The numbering and mnemonic are the same as Chapter 7's (7.7): **"N WLLA OMNI"** — **N**ext hop reachable, **W**eight, **L**ocal pref, **L**ocally originated, **A**S path (with AIGP folded in), **O**rigin, **M**ED, **N**eighbor type (eBGP over iBGP), **I**GP metric. After those, the tie-breakers run oldest → router ID → cluster list → neighbor address.

The direction rule, and the logic that ties it together. Every attribute either influences **your own outbound traffic** or **asks your neighbors** to send traffic to you a certain way:

Goal	Tool	Applied
Choose which exit **my** traffic uses — this router only	WEIGHT	**inbound** on routes from the preferred peer
Choose which exit **my whole AS** uses	LOCAL_PREF	**inbound** at the preferred border router
Influence how **other ASes** send traffic **to me**	AS_PATH prepend, MED	**outbound** toward the less-preferred link

Why does LOCAL_PREF beat AS_PATH length? Because BGP is **policy first, distance second**. LOCAL_PREF is your decision about your own traffic: "we pay less on ISP-A, so leave through ISP-A". AS_PATH length is only a rough measure of distance, the number of networks the route crosses. An AS's own business policy has to override that, or BGP couldn't express commercial reality. That's also why weight and LOCAL_PREF come first: they're the attributes you control. Everything after them is information you received.

Story Time : choosing a flight. Your company's travel policy (LOCAL_PREF: "use Airline A, we have a contract") comes first, whatever the itinerary. Only if the policy doesn't decide do you prefer fewer stopovers (AS_PATH length). Only after that do you look at smaller details: which one the airline itself recommends (MED), whether it's a direct booking (eBGP over iBGP), how far the departure gate is from your desk (IGP metric to the next hop). And if it's still a tie, you keep the booking you already have (oldest path).

Worked example — walking the steps. R1 (AS 65001) has three paths to 172.20.0.0/16:

Path	Learned from	Weight	LOCAL_PREF	AS_PATH	Origin	MED	Type	IGP metric to NH
A	203.0.113.1 (AS 65100)	0	100	65100 65300	i	0	eBGP	0

Path	Learned from	Weight	LOCAL_PREF	AS_PATH	Origin	MED	Type	IGP metric to NH
B	2.2.2.2 (iBGP, R2 learned it from AS 65200)	0	100	65200 65300	i	0	iBGP	20
C	198.51.100.1 (AS 65200)	0	100	65200 65400 65300	i	0	eBGP	0

1. **Next hop reachable? (step 0)** All three: yes. Continue.
2. **Weight (step 1):** all 0. Tie.
3. **LOCAL_PREF (step 2):** all 100. Tie.
4. **Locally originated? (step 3)** None. (AIGP, step 4, isn't in use.) Tie.
5. **AS_PATH length (step 5):** A = 2, B = 2, C = 3. **C is eliminated**; A and B continue.
6. **Origin (step 6):** both `i`. Tie.
7. **MED (step 7):** A came from AS 65100, B from AS 65200: different neighbor ASes, so MED isn't compared. Tie.
8. **eBGP over iBGP (step 8):** A is eBGP, B is iBGP. **A wins.**

Now change the policy. On R2, add an inbound route map from AS 65200 with `set local-preference 150`. R2 sends path B to R1 with LOCAL_PREF 150. Walk again: weight (step 1) ties, and at LOCAL_PREF (step 2) B (150) beats A and C (100). **B wins** at step 2, and the AS_PATH and eBGP steps are never reached. That's "first difference decides" in action: a later step can never overturn an earlier one.

Why is MED compared only between paths from the same neighbor AS? Because MED values are **set by the neighbor AS to suit its own network**. AS 65100's MED 10 and AS 65200's MED 50 come from different scales with different meanings, so comparing them is meaningless. `bgp always-compare-med` overrides this when you know both neighbors use a consistent scheme (for example, two ISPs that both use IGP cost). Configure it **on every router in the AS**, or the routers will make inconsistent decisions. Related: a path with no MED counts as **0** (the best value), unless `bgp bestpath med missing-as-worst` is set.

Worked policy: dual-homed to two ISPs

Requirement: all outbound traffic uses ISP-A; inbound traffic should arrive via ISP-A too.

```
route-map FROM-ISPA permit 10
 set local-preference 200             ! my whole AS prefers ISP-A's routes
!
route-map TO-ISPB permit 10
 set as-path prepend 65001 65001 65001  ! make my prefixes look 3 hops farther via ISP-B
!
router bgp 65001
 address-family ipv4 unicast
  neighbor 203.0.113.1 route-map FROM-ISPA in
  neighbor 198.51.100.1 route-map TO-ISPB out
```

Then `clear ip bgp * soft`. Verify outbound with `show ip bgp` (look for `200` in the LocPrf column on ISP-A's paths). For inbound, check from outside, for example on a looking glass, that your prefix shows the prepended path via ISP-B.

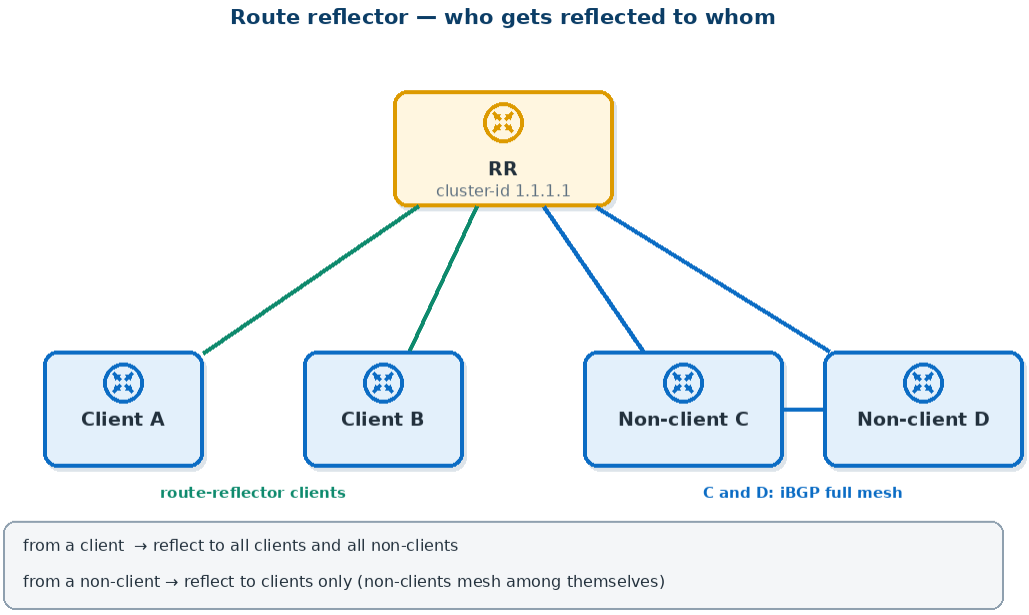
```
R1# show ip bgp 8.8.8.0
Paths: (2 available, best #1, table default)
  65100
    203.0.113.1 from 203.0.113.1 (203.0.113.1)
      Origin IGP, metric 0, localpref 200, valid, external, best
  65200 65100
    198.51.100.1 from 198.51.100.1 (198.51.100.1)
      Origin IGP, metric 0, localpref 100, valid, external
```

To explain why a path won, compare the two entries in the table's order and stop at the first difference. Here it's localpref 200 vs 100, at step 2.

28.7 Route Reflectors

The problem: iBGP split horizon (28.1) means every iBGP router must peer with every other one. At 10 routers that's 45 sessions; at 50 it's 1,225. Every new router means reconfiguring all the old ones. Route reflection cuts this down to one or two sessions per router.

A **route reflector (RR)** is the scalable alternative to an iBGP full mesh. Its **clients** peer only with the RR, and the RR is allowed to re-advertise iBGP routes, under rules that preserve loop-freedom.




```
router bgp 65001
 neighbor 2.2.2.2 remote-as 65001
 neighbor 2.2.2.2 update-source Loopback0
 neighbor 2.2.2.2 route-reflector-client     ! the ONLY extra line — configured on the RR
```

Clients need no special configuration. They peer with the RR as an ordinary iBGP neighbor.

Route received from	RR reflects it to
an **eBGP** peer	all clients and non-clients (normal behavior)
a **client**	all **other clients** and all **non-clients**
a **non-client**	**clients only** (non-clients must still be fully meshed with each other)

Loop prevention: two new attributes. Reflection removes the iBGP split-horizon protection, so the RR adds two attributes:

- **ORIGINATOR_ID**: the router ID of the router that first put the route into the AS. A router that receives a route carrying its **own** RID as originator drops it.
- **CLUSTER_LIST**: each RR **prepends its cluster ID** (by default its router ID; set with `bgp cluster-id`) as it reflects. An RR that sees its own cluster ID in the list drops the route. It's the iBGP equivalent of AS_PATH loop detection.

Why does a route from a non-client only go to clients? Because non-clients are, by definition, still in a full mesh with each other. The non-client that originated the route has already sent it to every other non-client directly, so reflecting it again would only create duplicates. Clients rely entirely on the RR, so they must receive it.

Story Time : back to the office news rule from 28.1. The route reflector is the team lead who's officially **allowed to pass on colleagues' news**. To stop the same news going round in circles, the team lead stamps each item with **who first reported it** (ORIGINATOR_ID) and **which team leads have already relayed it** (CLUSTER_LIST). Anyone who gets an item they reported themselves, or a team lead who sees their own stamp, bins it.

How reflection works, step by step. R2 (RID 2.2.2.2) is the RR. R1 (1.1.1.1) and R3 (3.3.3.3) are its clients. R1 is the border router and learns 8.8.8.0/24 from the ISP.

1. R1 sends 8.8.8.0/24 to R2 over iBGP, with `next-hop-self`, so the next hop is 1.1.1.1.
2. R2 received it from a **client**, so it may reflect it to all other clients and to non-clients. It adds **ORIGINATOR_ID 1.1.1.1** (R1 put the route into the AS) and prepends its cluster ID to the **CLUSTER_LIST**: `2.2.2.2`.
3. R2 sends it to R3. Nothing else changes: the next hop is still 1.1.1.1, LOCAL_PREF and AS_PATH are untouched.
4. R3 installs it, recursing to 1.1.1.1 through the IGP:

```
R3# show ip bgp 8.8.8.0
BGP routing table entry for 8.8.8.0/24, version 22
Paths: (1 available, best #1, table default)
  Not advertised to any peer
  65100
    1.1.1.1 (metric 3) from 2.2.2.2 (2.2.2.2)
      Origin IGP, metric 0, localpref 100, valid, internal, best
      Originator: 1.1.1.1, Cluster list: 2.2.2.2
```

Reading it: `from 2.2.2.2` says who **sent** the route (the RR); `1.1.1.1` before it is the **next hop**; `Originator: 1.1.1.1` says who **originated** it inside the AS. Three different routers, three different roles. `Not advertised to any peer` is iBGP split horizon in action: R3, a client, doesn't pass the iBGP route on.

The loop check in action: if the route were ever reflected back to R1 (for example by a second RR), R1 would see its own RID as the originator and discard it. A second RR in the same cluster would see its cluster ID already in the CLUSTER_LIST and discard it too.

Exam traps:

- An RR **doesn't change the next hop** or any other attribute when it reflects. Clients still need a route to the original next hop, so `next-hop-self` belongs on the **border** router.
- An RR only reflects its **best** path. Clients therefore see fewer alternatives than a full mesh would give them, which can make their path choice suboptimal.
- Reflected paths show `Originator: … Cluster list: …` in `show ip bgp <prefix>`. Use that to confirm a route came through the RR.

28.8 Policies — Filtering and Path Manipulation

The problem: by default, BGP accepts everything a neighbor sends and advertises everything it has to every neighbor. Neither is safe. You don't want a mistake at your ISP to flood you with junk, and you certainly don't want to advertise one ISP's routes to the other. Policies control what comes in, what goes out, and which path wins.

Four filter tools can be attached to a neighbor, `in` or `out`:

Tool	Matches on	Applied with
Prefix list	prefix + length (`ge`/`le`)	`neighbor X prefix-list NAME in|out`
AS-path access list	AS_PATH via regex	`neighbor X filter-list N in|out`
Route map	anything (prefix lists, AS-path lists, communities…) — and can **set** attributes	`neighbor X route-map NAME in|out`
Distribute list	standard/extended ACL (legacy)	`neighbor X distribute-list N in|out`

Prefix lists and route-map logic are Chapter 23's subject. What's specific to BGP is the **AS-path regex**:

Regex	Matches	Typical use
`^$`	empty AS_PATH — **routes I originated**	outbound to ISPs: advertise only my own prefixes, **never become transit**
`^65100_`	learned directly from AS 65100 (any route that neighbor sent)	
`^65100$`	**originated by** AS 65100 and learned directly from it — nothing behind it	accept only the neighbor's own prefixes

Regex	Matches	Typical use
`_65200$`	**originated** by AS 65200	
`_65300_`	passed **through** AS 65300 anywhere	block a distrusted AS
`^[0-9]+$`	exactly one AS — the neighbor's own routes	accept only the peer's customer-free routes
`.*`	anything	the final permit-all

`_` matches any delimiter (a space, the start or end of the string), `^` the start, `$` the end.

Worked example — reading regexes against a real path. A route arrives with AS_PATH `65100 65300` (learned from 65100, originated by 65300):

Regex	Match?	Why
`^65100_`	yes	the path starts with 65100 followed by a delimiter
`_65300$`	yes	the path ends with 65300
`^65100$`	**no**	it would need 65100 to be the only AS
`_65300_`	yes	65300 appears, bounded by a space and the end of the string
`^[0-9]+$`	**no**	there are two ASes, separated by a space
`^$`	**no**	the path isn't empty

Note the trap in plain `65100` without delimiters: it would also match AS 165100 or 651001. That's why real filters use `_`, `^` and `$`.

```
ip as-path access-list 1 permit ^$
!
router bgp 65001
 address-family ipv4 unicast
  neighbor 203.0.113.1 filter-list 1 out      ! advertise only locally originated routes
```

Why is `^$` outbound the most important enterprise BGP filter? Because without it, a dual-homed enterprise re-advertises ISP-A's routes to ISP-B. If ISP-B prefers that path, its traffic toward ISP-A starts flowing **through your network**, over your links, at your cost. That's the classic "accidental transit AS" outage. Allowing out only the paths with an empty AS_PATH, meaning routes you originated, stops it. Test a regex before you apply it: `show ip bgp regexp ^$` lists exactly what it matches.

Communities

A **community** is a 32-bit tag (written `AS:value` after `ip bgp-community new-format`) that groups routes so policy can be applied to the group. Standard communities aren't sent to neighbors unless you configure it:

Story Time : a community is a **luggage tag**. The bag (the route) is the same, but the colored tag tells every handler down the line how to treat it: "priority", "don't send abroad". Your router writes the tag; routers later in the path read it and act on it, as long as the tag isn't stripped on the way (which is exactly what a missing `send-community` does).

Worked example — tag at the branch, act at the hub. Branch routers tag their LAN prefixes with 65001:100. The hub gives everything carrying that tag LOCAL_PREF 150, so every router in the AS prefers the primary path to branches, without the hub needing a prefix list that names every branch subnet. Add a new branch, tag its routes, and the hub's policy already covers it. The configuration is below: `TAG-BRANCH` applied outbound on the branch, `send-community` toward the hub, and `MATCH-BRANCH` applied inbound on the hub.

```
router bgp 65001
 address-family ipv4 unicast
  neighbor 2.2.2.2 send-community              ! standard; "both" adds extended communities
!
route-map TAG-BRANCH permit 10
 set community 65001:100 additive              ! add to existing communities, don't replace
!
ip community-list standard BRANCH permit 65001:100
route-map MATCH-BRANCH permit 10
 match community BRANCH
 set local-preference 150
```

Well-known communities have built-in meanings that every BGP speaker honors:

Community	Meaning
no-export	don't advertise outside this AS (or confederation)
no-advertise	don't advertise to **any** peer, iBGP or eBGP
local-AS	don't advertise outside this sub-AS within a confederation (= no-export without confederations)
internet	advertise normally

Exam trap — the forgotten `send-community`. The route map sets the community correctly, `show ip bgp` on the local router shows it, but the neighbor never sees it, so its policy never matches. The tag is only sent with `send-community` on the **sending** router, under the right address family. Check with `show ip bgp <prefix>` on the receiver. If there's no `Community:` line, the sender didn't send it.

28.9 BGP in a VRF (VRF-Lite)

The problem: a router running VRF-Lite needs to exchange routes per VRF with customers, firewalls or partner networks, and some of those neighbors only speak BGP. Running one BGP process per VRF isn't allowed (a router has exactly one BGP process), so each VRF becomes an address family inside the single process.

VRF-Lite (Chapter 25) can use BGP as a per-VRF routing protocol, for example to peer with a customer or firewall context per VRF. Each VRF gets its own address family under the **one** BGP process:

```
vrf definition CUST_A
 rd 65001:1                                   ! REQUIRED for BGP — even without MPLS
 address-family ipv4
!
router bgp 65001
 address-family ipv4 vrf CUST_A
```

Why does BGP demand a route distinguisher when VRF-Lite doesn't otherwise need one? Because BGP stores every VRF's routes in **one shared BGP table**, internally as VPNv4 routes. If CUST_A and CUST_B both have 10.1.1.0/24, the RD is what keeps them apart: `65001:1:10.1.1.0/24` and `65001:2:10.1.1.0/24`. Without an RD, IOS refuses the address family with "VRF CUST_A does not have an RD configured". Chapter 29 explains RDs fully, since they're the core of MPLS L3VPN.

Story Time : two people called "John Smith" in one filing cabinet. To keep their files apart, you write a department code in front of each name: "Sales: John Smith", "Finance: John Smith". The RD is that department code, written in front of every prefix so identical customer addresses never collide in BGP's one shared table. The code says nothing about who may read the file; that's the job of route targets in Chapter 29.

Verify per VRF:

```
R1# show ip bgp vpnv4 vrf CUST_A summary
R1# show ip bgp vpnv4 vrf CUST_A
R1# show ip route vrf CUST_A bgp
```

28.10 The BGP Troubleshooting Checklist

First, the session:

Symptom	Cause	Proof
`Idle`	no route to the neighbor address	`show ip route <neighbor>`; `ping <neighbor> source lo0`
`Active` (loopback peering)	missing `update-source` or no route to the loopback	`show ip bgp neighbors X | i Local host` shows the wrong source
`Active` (eBGP to a loopback)	TTL 1 — missing `ebgp-multihop` / `disable-connected-check`	`show ip bgp neighbors X | i hop`
`Active`, `BADAUTH` log	MD5 password mismatch / missing	`%TCP-6-BADAUTH`
`Active`, nothing logged	ACL dropping TCP 179, or peer has no matching `neighbor`	`show access-lists`; check the other side's config
Reaches OpenSent, then NOTIFICATION `2/2`	wrong `remote-as`	log shows the AS received
`Idle (PfxCt)`	`maximum-prefix` exceeded (28.11)	`show ip bgp neighbors X` shows the reset reason; recover with `clear ip bgp X`
`Idle (Admin)`	`neighbor X shutdown`	config
Established, 0 prefixes for a family	neighbor not `activate`d in that AF	`show bgp all summary`

Then, the routes:

Symptom	Cause	Proof
`network` statement not advertised	no **exact** match (prefix **and** mask) in the RIB	`show ip route <prefix> <mask>`
Route present, no `>` (not best), `inaccessible`	next hop unreachable	`show ip bgp <prefix>`; fix with `next-hop-self` (28.2)
iBGP route not passed on to another iBGP peer	iBGP split horizon	needs full mesh or RR (28.7)
Route best in BGP, flagged `r`	**RIB failure** — a lower-AD source owns the prefix	`show ip bgp rib-failure`
Route never arrives at the peer	outbound filter / policy	`show ip bgp neighbors X advertised-routes`
Peer says it sent it, I don't have it	inbound filter	`received-routes` vs `routes` (28.5)
"Wrong" path chosen	best-path order (28.6)	`show ip bgp <prefix>` — first differing attribute
Routes from my other site (same AS) never arrive	AS_PATH loop check drops paths containing my own AS	`debug ip bgp updates` shows the denial; fix with `allowas-in` (CE) or `as-override` (PE) — 28.11
Only one path installed although two look equal	`maximum-paths` not set, or paths differ in an attribute multipath requires	`show ip bgp <prefix>` shows `multipath` only when it works (28.11)
Communities ignored downstream	no `send-community`	receiver's `show ip bgp <prefix>`
Policy change has no effect	routes not re-evaluated	`clear ip bgp X soft in/out`

Worked ticket — "the branch can't reach the internet"

Symptom: users behind R3 can't reach 8.8.8.8. Users behind the border router R1 can.

1. **Session?** On R3, `show bgp ipv4 unicast summary`: the session to the RR (2.2.2.2) shows `14` in State/PfxRcd. Established, routes received. The session is fine.
2. **Is the route received?** `show ip bgp 8.8.8.0` on R3 shows the path, but with `203.0.113.1 (inaccessible)` and `no best path`.
3. **Read it.** The next hop is the ISP's address. The route came through the RR, and an RR **never changes the next hop** (28.7). R3 has no IGP route to 203.0.113.0/30.
4. **Where should the fix go?** Not on the RR. On the **border router R1**, which learned the route from eBGP: `neighbor 2.2.2.2 next-hop-self` (R1's session to the RR), then `clear ip bgp 2.2.2.2 soft out`.
5. **Verify:** on R3, the path now shows next hop `1.1.1.1 (metric 3)` and `best`, and `show ip route 8.8.8.8` gives a `B` route via 1.1.1.1.

BGP troubleshooting is always two questions in order: is the session Established, and then is the route sent, received, accepted, and best? Each question has its own command, and the answers are always in the output.

28.11 More Knobs the Exam Expects You to Recognise

These don't change how BGP works; each one bends one specific rule. ENARSI questions show them in a config and ask what they do — or show the symptom of the one that's missing.

Protecting the router: `maximum-prefix`

```
router bgp 65001
 address-family ipv4 unicast
  neighbor 203.0.113.1 maximum-prefix 1000 80 restart 30
  ! 1000 = limit; 80 = warn (syslog) at 80% of it (default threshold is 75%)
  ! restart 30 = after tearing the session down, retry automatically after 30 minutes
```

Variant	What happens when the limit is exceeded
`maximum-prefix 1000`	Session torn down, state **`Idle (PfxCt)`**, and it **stays down** until you `clear ip bgp 203.0.113.1`
`maximum-prefix 1000 restart 30`	Torn down, then re-established automatically after 30 minutes
`maximum-prefix 1000 warning-only`	Session stays up; only a syslog message is generated

Why tear down a working session over a number? Because a peer that suddenly sends far more routes than agreed has usually made a mistake — leaking a full internet table, or redistributing an IGP into BGP — and accepting it could exhaust your memory or reroute your traffic. A dropped session is a contained failure; a router out of memory is not.

Same AS at two sites: `allowas-in` and `as-override`

BGP's loop prevention (23.6) drops any path whose AS_PATH already contains the receiver's own AS. That breaks a legitimate design: two branch sites that **share one private AS** (say 65010) connected through a provider AS 64500. Site B receives Site A's prefix with AS_PATH `64500 65010`, sees its own 65010 and silently drops it.

Fix	Configured on	How it works
`neighbor 192.0.2.1 allowas-in [count]`	the **customer** (CE) router, toward the provider	Accept paths containing my own AS, up to `count` times (1–10; default 3)
`neighbor 192.0.2.2 as-override`	the **provider** (PE) router, toward the customer	When advertising to that neighbor, replace every occurrence of the neighbor's AS in the AS_PATH with the provider's own AS, so `64500 65010` becomes `64500 64500`

`allowas-in` relaxes the check on the receiver; `as-override` rewrites the path so the check never fires. Both deliberately weaken loop prevention, so use them only where the topology makes a

loop impossible. Chapter 29 shows where this appears in MPLS L3VPNs.

Sending a default: `neighbor default-originate`

```
router bgp 64500
 address-family ipv4 unicast
  neighbor 198.51.100.2 default-originate                ! send 0.0.0.0/0 to this peer
  neighbor 198.51.100.3 default-originate route-map HAVE-UPLINK  ! only while the map matches
```

Unlike `network 0.0.0.0` (which needs an exact 0.0.0.0/0 in the RIB) or redistributing a default, `default-originate` sends a default to **that neighbor only** and does **not** require a default route in the local routing table. The optional route map makes it conditional — for example, advertise the default only while the upstream prefix is still present.

Changing the AS you present: `local-as`

```
router bgp 65001
 neighbor 203.0.113.9 remote-as 64999
 neighbor 203.0.113.9 local-as 65100 no-prepend replace-as
```

The neighbor sees this router as AS **65100** even though its real process is 65001. It's used for **migrations and mergers**: the peer's configuration doesn't have to change on the day you renumber. By default the local-as value is prepended to received paths; `no-prepend` and `replace-as` hide it so the old AS doesn't appear twice. If a peer expects one AS and the router's `router bgp` number is different, check for `local-as` before declaring a `remote-as` mismatch.

Using several paths at once: `maximum-paths`

```
router bgp 65001
 address-family ipv4 unicast
  maximum-paths 2          ! up to 2 equal eBGP paths into the RIB
  maximum-paths ibgp 2     ! up to 2 equal iBGP paths
```

By default BGP installs **one** best path. Multipath only combines paths that tie on everything up to the IGP-metric step — weight, local preference, AS_PATH **length and contents**, origin and MED — and, for eBGP, come from the same neighboring AS. One path is still marked best (`>`); the others show `multipath` in `show ip bgp <prefix>`. (`bgp bestpath as-path multipath-relax` relaxes the "same AS_PATH contents" requirement so paths from different ASes of equal length can be combined.)

Ignoring AS_PATH length: `bgp bestpath as-path ignore`

This skips step 5 of the best-path algorithm entirely, so paths are compared as if their AS_PATH lengths were equal. It's a hidden command on IOS — it works but isn't listed by `?` — and is rarely appropriate, because it throws away BGP's only distance measure. Recognise it if an exam config uses it to explain why a longer path won.

Chapter 29: MPLS & Layer 3 VPNs

> **Blueprint:** ENARSI 2.1 — Describe MPLS operations (LSR, LDP, label switching, LSP); ENARSI 2.2 — Describe MPLS Layer 3 VPN.

MPLS stands for **Multiprotocol Label Switching**: routers forward packets by reading a short number (a **label**) stuck on the front of the packet, instead of looking up the destination IP address at every hop. A **Layer 3 VPN** (L3VPN) is the service providers build on top of it: many customers' private networks carried across one shared provider network, kept completely separate.

Both blueprint items here say **describe**, not configure. You won't build a service-provider core in the exam, but you will be shown labels, LFIB entries, route targets and a PE configuration and asked what they mean. So this chapter does what the exam does: it teaches each moving part and then shows you real configuration and `show` output to read. Read the configs as you'd read code someone else wrote.

29.1 Why Label Switching Exists

The story usually told is "MPLS was invented because IP lookups were slow." That was true in the 1990s — longest-prefix matching against a big table was expensive, and an exact-match lookup on a short fixed label was cheap. Today, with CEF and hardware forwarding, IP lookups are just as fast. **MPLS survived because of what labels let you build, not because of speed.**

Why is a label more powerful than a destination IP address? Because a label is **decoupled from the packet's contents**. A router forwarding plain IP can only ask one question: "where does this destination address go?" A label-switching router asks "what does label 18 mean?" — and label 18 can mean anything the network agreed on: "the path to PE2's loopback", "customer A's network 10.2.2.0/24", "the traffic-engineered tunnel through the cheap links". That indirection gives you three things plain IP can't:

What labels enable	Why plain IP can't do it
A BGP-free core — P routers know only the provider's loopbacks, never the internet table or customer routes	An IP router must know the destination to forward; a label router only needs to know the label
L3VPNs — overlapping customer address spaces (two customers both using 10.0.0.0/8) across one core	The destination 10.1.1.1 is ambiguous; a label is not
Traffic engineering — steering traffic down a path that isn't the IGP's shortest one	IP forwarding always follows the destination's best route

Labels replace "where is this going?" with "which agreed path is this on?" — and that one change is the whole of MPLS.

Story Time : Think of a courier company. You write a full street address on your parcel. At the first depot, a clerk reads that address once and sticks on a coloured routing sticker: "Route 18".

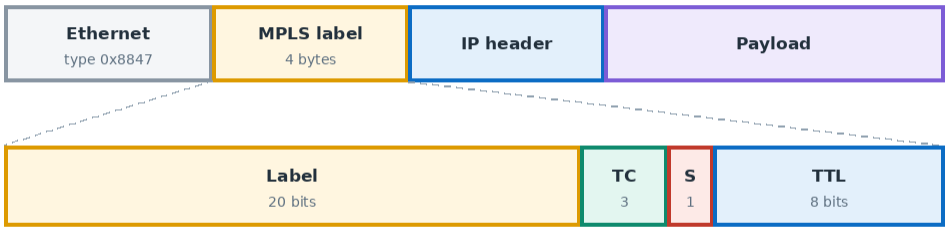
From then on, the sorting depots in the middle never read the address again — they glance at the sticker, put the parcel on the right conveyor, and swap the sticker for the one their next depot uses ("Route 21"). Near the end, the last depot before your house peels the sticker off, and the local van reads the street address again to deliver it. The middle depots don't need a map of every street in the country; they only need their own sticker list. That is MPLS: the **first router (the edge)** reads the IP address once, the **middle routers** only swap labels, and the **last hops** remove the label. Where the analogy breaks: a real courier sticker usually means the same thing everywhere; an MPLS label only means something to the one router that handed it out — which is exactly why every hop swaps it (29.4, 29.5).

29.2 The Label Itself

The problem it solves: routers need somewhere to put the label that doesn't disturb the original IP packet, so the edge can take it off later and hand the customer back exactly what they sent. The answer is a small extra header slotted in front of the IP packet.

An MPLS label is a **32-bit shim header** inserted **between the Layer 2 header and the IP header** — which is why MPLS is often called "Layer 2.5". Ethernet announces it with EtherType **0x8847** (MPLS unicast) instead of 0x0800 (IPv4).

The MPLS label — a 32-bit shim between Layer 2 and Layer 3



Label: what to do with me · TC: QoS (old EXP) · S = 1 on the bottom label · TTL: loop guard

labels 0–15 reserved — 3 = implicit null ("pop before sending to me")

Field	Bits	What it does
Label	20	The value routers switch on. 0–15 are reserved; Cisco allocates dynamic labels from **16** upward
TC (Traffic Class, formerly **EXP**)	3	QoS marking — copied from IP precedence by default so the core can still honour priority
S (Bottom of Stack)	1	**1** on the last label before the IP header, 0 on every label above it
TTL	8	Loop protection, exactly like IP's TTL — decremented at every label hop

Why is there an S bit — why would a packet carry more than one label? Because a **label stack** lets different parts of the network handle different questions independently. In an L3VPN (29.6), the outer label says "get me to PE2" and the inner label says "and once there, this belongs

to customer A". The core only ever reads the top label. The S bit tells the router reading a label whether there's another one underneath or the IP header comes next.

Worked example — decoding one label. A packet capture shows the 4-byte shim `00 01 51 40` (hex). Split the 32 bits by field:

Field	Bits from the shim	Value
Label (first 20 bits)	`0000 0000 0000 0001 0101` → 0x00015	**21**
TC (next 3 bits)	`000` (first three bits of the next hex digit, 0x1 = `0001`)	0 (best effort)
S (next 1 bit)	`1` (the last bit of that same digit)	**1** — this is the last label; the IP header comes next
TTL (last 8 bits)	`0100 0000` → 0x40	**64**

So this packet is carrying label 21 (the label P2 uses for PE2's loopback in this chapter's examples), it has only one label, and it has 64 hops of life left. If a second label sat underneath, this top label would have S = 0 and the bottom one S = 1.

The reserved labels you'll see in output:

Label	Name	Meaning
0	IPv4 explicit null	"Pop me at the last hop, but keep my TC bits until then"
2	IPv6 explicit null	Same for IPv6
3	**Implicit null**	Never appears in a packet. Advertised to say "**pop the label before sending to me**" — PHP (29.4)

29.3 The Cast: LSRs, LERs, PEs, Ps and CEs

Why name the roles at all? Because each router in an MPLS network does a different amount of work, and exam questions describe a router by its role ("what does the P router know about customer routes?"). Learn the names once and every later section reads easily.

Role	Generic name	Provider name	What it does
Edge router where labels are **added** or **removed**	**LER** (Label Edge Router)	**PE** (Provider Edge)	IP lookup on one side, label imposition/disposition on the other
Router in the middle that only **swaps** labels	**LSR** (Label Switch Router)	**P** (Provider)	Label in → label out; never looks at the IP header
Customer's router	—	**CE** (Customer Edge)	Runs plain IP; **doesn't know MPLS exists**

Strictly, every MPLS router is an LSR; "LER" just means one sitting at the edge. The **LSP** (Label Switched Path) is the one-directional sequence of labels a packet follows from ingress LER to egress LER. One-directional matters: the return traffic takes a separate LSP, built separately, possibly through different routers.

The topology used for every example in this chapter:

Router	Role	Loopback / LDP ID	Notes
CE1	Customer edge, site 1	—	Customer A, LAN 10.1.1.0/24
PE1	Provider edge (ingress LER)	1.1.1.1	Faces CE1
P1	Provider core (LSR)	3.3.3.3	Between PE1 and P2
P2	Provider core (LSR)	2.2.2.2	Between P1 and PE2
PE2	Provider edge (egress LER)	4.4.4.4	Faces CE2
CE2	Customer edge, site 2	—	Customer A, LAN 10.2.2.0/24

Traffic from site 1 to site 2 follows **CE1 → PE1 → P1 → P2 → PE2 → CE2**.

29.4 Push, Swap, Pop — and Penultimate Hop Popping

The problem it solves: a label is only useful if every router along the path knows exactly what to do with it. MPLS keeps that simple by allowing only three actions. Every label operation is one of three:

Operation	Where	What happens
Push (impose)	Ingress PE	IP packet arrives; FIB lookup says "send with label 18"; label pushed on
Swap	P routers	Label 18 in → LFIB says "out Gi0/2 with label 21"
Pop (dispose)	Penultimate or egress router	Top label removed

Penultimate Hop Popping (PHP). Without it, the egress PE would do two lookups for every packet: a label lookup ("label 21 is mine — pop it") and then an IP or VPN lookup on what's underneath. PHP removes the first one. The egress PE advertises **implicit-null (label 3)** for its own loopback, which tells its upstream neighbour "**don't bother sending me a label for this — pop it for me**". The penultimate router pops instead of swapping, and the egress PE receives a packet it can process with a single lookup.

Why let the second-to-last router do the egress router's job? Because the penultimate router is doing a label lookup anyway — popping instead of swapping costs it nothing extra. The egress PE, which would otherwise have done a label lookup purely to discover "this one's for me", is spared a lookup on every packet. The work moves to where it's free.

Story Time : Back at the courier company: the last sorting depot before your town knows the parcel is about to reach its final depot. If it left the routing sticker on, the final depot would have to read the sticker just to learn "oh, this one's ours — throw the sticker away", and only then read the street address. So the final depot tells its neighbour, "don't bother putting a sticker on parcels for me — peel it off before you send them." Implicit-null (label 3) is that message.

How it works — one plain packet along the LSP to 4.4.4.4. PE1 pings PE2's loopback (no VPN yet, just transport):

1. **PE1 (push).** Its FIB says 4.4.4.4/32 is reached via P1, and P1 advertised label **18** for it. PE1 pushes label 18 (S = 1, TTL copied from the IP TTL minus one) and sends the frame to P1 with EtherType 0x8847.
2. **P1 (swap).** P1 reads only the label. Its LFIB says "in 18 → out **21**, Gi0/2 toward P2". It swaps 18 for 21, decrements the label TTL and forwards. P1 never looks at the IP header.
3. **P2 (pop — PHP).** Its LFIB entry for in-label 21 says **Pop Label**, because PE2 advertised implicit-null for its own loopback. P2 removes the label and sends a plain IP packet to PE2.
4. **PE2.** Receives an ordinary IP packet addressed to 4.4.4.4 — its own loopback — and answers with one IP lookup.

The reply travels a different LSP (toward 1.1.1.1), built from the labels each router advertised for PE1's loopback.

In `show mpls forwarding-table`, PHP shows up as **`Pop Label`** (older IOS: `Pop tag`):

```
P2# show mpls forwarding-table
Local      Outgoing   Prefix           Bytes Label   Outgoing   Next Hop
Label      Label      or Tunnel Id     Switched      interface
17         16         1.1.1.1/32       1767110       Gi0/1      10.0.23.3
19         Pop Label  3.3.3.3/32       20416         Gi0/1      10.0.23.3   ! P1 is next door
21         Pop Label  4.4.4.4/32       1849302       Gi0/2      10.0.24.4   ! PHP toward PE2
```

Read the last line as: "a packet arriving **with label 21** is destined for 4.4.4.4/32; **pop** the label and send it out Gi0/2 to 10.0.24.4." P2 is the penultimate hop toward PE2, so PE2 asked it (via label 3) to pop. Label 21 is the one P2 advertised to P1 for 4.4.4.4 — the same 21 you'll see in P1's bindings below and in the packet walk (29.11). The middle line is PHP again: P1 owns 3.3.3.3, so it advertised implicit-null and P2 pops. The first line is an ordinary **swap**: label 17 in, label 16 (P1's label for PE1's loopback) out. P2's directly connected subnets have no entries at all — for those, P2 itself is the egress and advertises implicit-null to its neighbours.

> **Exam trap:** `Pop Label` and `No Label` are different. **Pop Label** = "remove the top label" (PHP — normal). **No Label** = "I have no outgoing label for this, so send it as plain IP" — and on a P router in a BGP-free core that's a **broken LSP**, because the next router may not have the IP route.

29.5 LDP — How Routers Agree on Labels

The problem it solves: labels are **locally significant** — each router picks its own label for each prefix, so label 21 on P2 and label 21 on P1 can mean completely different things. For a label path to work, every router must learn which label each neighbour expects for each destination. **LDP** (Label Distribution Protocol) is how they tell each other.

How labels spread — building the LSP to 4.4.4.4, step by step. Cisco's LDP hands out labels without being asked (downstream unsolicited) and each router does so independently as soon as the prefix is in its routing table:

1. **PE2** owns 4.4.4.4/32. It advertises **implicit-null (3)** for it to all its LDP neighbours: "pop before sending to me."
2. **P2** learns 4.4.4.4/32 from the IGP, picks a free local label (**21**) and advertises "4.4.4.4/32 → 21" to every LDP neighbour (P1 and PE2). In its LFIB it records: in 21 → pop, out toward PE2.
3. **P1** picks local label **18**, advertises "4.4.4.4/32 → 18" to PE1 and P2, and records: in 18 → out 21 toward P2 (because the IGP says P2 is the next hop).
4. **PE1** picks its own local label (**19**) and advertises it to P1 — useful only to routers that would forward to 4.4.4.4 through PE1. For its own traffic, PE1 uses P1's label 18, because P1 is its IGP next hop.

No router asked for anything and no router needed the whole picture: each just announced its own label and listened to its neighbours'.

Discovery and session

Step	Transport	Detail
1. Discovery	**UDP 646** hellos to **224.0.0.2** (all routers)	Sent on every interface with `mpls ip`; every 5 s, hold 15 s by default
2. Session	**TCP 646**	Opened to the neighbour's **transport address** (= its LDP router ID by default); the router with the **higher** LDP ID actively opens it
3. Label exchange	Over that TCP session	Each router advertises a label binding for every IGP prefix in its table

Why must the LDP router ID be reachable — and why is it usually a loopback? Because the TCP session goes to the transport address, which defaults to the LDP router ID. If R1's LDP ID is 1.1.1.1 but nobody has a route to 1.1.1.1, hellos still arrive (they're link-local multicast) yet the TCP session can never open — LDP sits forever in discovery. That's why you advertise loopbacks into the IGP and pin the ID: `mpls ldp router-id Loopback0 force`.

LIB versus LFIB

Table	Contains	Command
LIB (Label Information Base)	**Every** binding: my local label, plus the label **every** neighbour advertised, for each prefix	`show mpls ldp bindings`
LFIB (Label Forwarding Information Base)	Only the **one** outgoing label from the **IGP's best next hop**	`show mpls forwarding-table`

```
P1# show mpls ldp bindings 4.4.4.4 32
  lib entry: 4.4.4.4/32, rev 14
        local binding:  label: 18
        remote binding: lsr: 1.1.1.1:0, label: 19
        remote binding: lsr: 2.2.2.2:0, label: 21
```

P1 (LDP ID 3.3.3.3) has two LDP neighbours, PE1 (1.1.1.1) and P2 (2.2.2.2), so it holds two remote bindings — never one from itself. The IGP's next hop to 4.4.4.4 is P2, so the LFIB swaps P1's local 18 for P2's **21**; PE1's label 19 sits unused in the LIB. P1 keeps both remote bindings but the LFIB uses only the one from whichever neighbour the IGP says is the next hop to 4.4.4.4. **Why keep labels you aren't using?** Cisco runs LDP in **liberal retention** mode: if the IGP reconverges and the best next hop changes, the label from the new next hop is already in the LIB, so the LFIB updates instantly instead of waiting for a fresh label request. The IGP picks the path; LDP just makes sure a label is ready for whichever path it picks.

What the configuration looks like

```
mpls label protocol ldp                ! the default on IOS XE; shown for clarity
mpls ldp router-id Loopback0 force     ! stable, reachable LDP ID
!
interface GigabitEthernet0/1
 mpls ip                               ! run LDP + label switching on this core link
!
router ospf 1
 mpls ldp autoconfig area 0            ! alternative: enable LDP on every OSPF area-0 link
```

```
P1# show mpls ldp neighbor
    Peer LDP Ident: 2.2.2.2:0; Local LDP Ident 3.3.3.3:0
        TCP connection: 2.2.2.2.646 - 3.3.3.3.38211
        State: Oper; Msgs sent/rcvd: 41/40; Downstream
        Up time: 00:18:22
        LDP discovery sources:
          GigabitEthernet0/2, Src IP addr: 10.0.23.2
        Addresses bound to peer LDP Ident:
          10.0.23.2       10.0.24.2       2.2.2.2
```

What's the `:0` after every LDP ID? An LDP identifier is six bytes: the 4-byte router ID plus a 2-byte **label space** ID. **`:0` means the platform-wide label space** — the router uses one pool of labels for all interfaces, so a label means the same thing whichever interface it arrives on. That's the normal case for Ethernet core links (frame-mode MPLS). A non-zero value indicates a per-interface label space, which you'll essentially never see today.

Note the port pairing: the peer with the **higher LDP ID** (3.3.3.3) used an ephemeral port and connected to 646 on 2.2.2.2. The "Addresses bound" list matters too: it's how P1 maps "next hop 10.0.23.2 in my routing table" to "the LDP peer 2.2.2.2 whose labels I should use".

Symptom	Likely cause	Command that proves it
No LDP neighbour at all	`mpls ip` missing on one side, or UDP 646 filtered	`show mpls interfaces`, `show mpls ldp discovery`
Discovery works, session never forms	LDP router ID not reachable / TCP 646 blocked	`show mpls ldp discovery detail`, `ping` the ID from the transport source
`No Label` toward a PE loopback	That prefix is **summarized** in the IGP, or LDP down on the next hop	`show mpls forwarding-table <prefix>`, `show ip route <prefix>`
Large packets fail, pings work	Each label adds **4 bytes**; core MTU too small	`ping <pe> size 1500 df-bit`, `show mpls interfaces detail`

Worked example — ticket: "PE1 has no LDP neighbour to P1."

- **Symptom:** `show mpls ldp neighbor` on PE1 is empty, and VPN traffic from PE1 dies.
- **Step 1 — is discovery working?**

```
PE1# show mpls ldp discovery detail
 Local LDP Identifier:
    1.1.1.1:0
    Discovery Sources:
    Interfaces:
        GigabitEthernet0/1 (ldp): xmit/recv
            LDP Id: 3.3.3.3:0; no route to transport addr
```

- **Reading it:** `xmit/recv` means hellos go both ways, so `mpls ip` is on at both ends and UDP 646 gets through. The problem is the next line: PE1 has **no route to P1's transport address** 3.3.3.3, so the TCP session can't open.
- **Step 2 — prove it:** `show ip route 3.3.3.3` → `% Network not in table`.
- **Cause:** P1's Loopback0 was never advertised into OSPF (no `network 3.3.3.3 0.0.0.0 area 0`).
- **Fix:** advertise the loopback on P1. Within seconds `show mpls ldp neighbor` on PE1 shows `Peer LDP Ident: 3.3.3.3:0 ... State: Oper`.

> **Exam trap:** never summarize PE loopbacks in the core IGP. LDP binds labels to **exact prefixes**. If P routers only have 4.4.0.0/16, there's no binding for 4.4.4.4/32, the LSP breaks, and VPN traffic dies even though `ping 4.4.4.4` from a P router works fine.

LDP–IGP Synchronization

The IGP and LDP converge separately, and the IGP is usually faster. When a core link comes up (or LDP restarts on it), OSPF can start routing over the link **before** LDP has exchanged labels there. Plain IP traffic survives that, but labeled VPN traffic arriving at a router with no outgoing label is sent unlabeled (`No Label`) and dropped by a BGP-free core — a black hole lasting until LDP catches up.

```
router ospf 1
 mpls ldp sync                  ! on every OSPF link where LDP runs
!
P1# show mpls ldp igp sync
```

With synchronization on, OSPF advertises the link with its **maximum metric** until LDP is fully up on it, so traffic keeps using the old path until labels exist on the new one. The IGP waits for LDP instead of racing ahead of it. (IS-IS has the equivalent `mpls ldp sync` under its process.)

29.6 MPLS Layer 3 VPNs — The Problem

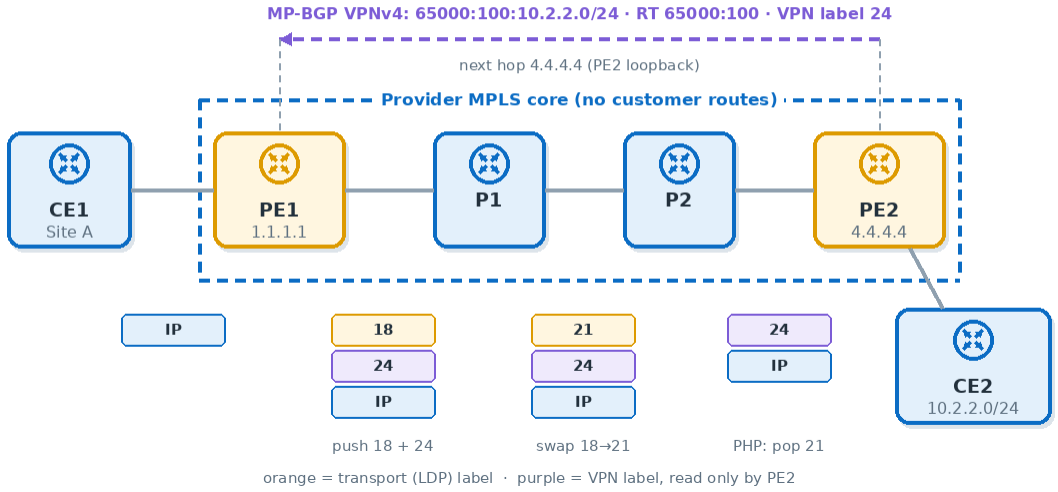
A provider sells "private WAN" service to many customers over one shared core. Two problems:

1. **Customers overlap.** Customer A and Customer B both use 10.1.0.0/16 internally. A single routing table can't hold both.
2. **The core shouldn't care.** The P routers must not carry thousands of customer routes.

MPLS L3VPN solves the first with **VRFs + route distinguishers**, the second with **labels**, and glues it together with **MP-BGP** (Multiprotocol BGP — BGP extended to carry more than plain IPv4 routes) between PEs.

Story Time : Picture a courier that serves two companies, Acme and Globex. Both have a "Room 101" in their offices. If the courier's computer just stored "Room 101", it couldn't tell them apart. So it stores **"Acme–Room 101"** and **"Globex–Room 101"** — that prefix is the **route distinguisher**. Separately, each parcel carries a **distribution stamp** saying which mailrooms should receive copies: "Acme mailrooms" (or "Acme and the shared reception desk"). That stamp is the **route target**. The trucks on the motorway (P routers) carry sealed bags labelled only with the destination depot — they never see any room numbers at all. The rest of this chapter is that story with real names.

MPLS L3VPN — routes ride MP-BGP, packets ride a two-label stack



29.7 VRFs on the PE

Each PE holds a separate **VRF** (a separate routing table and CEF table — see Chapter 12 for VRF basics and Chapter 25 for VRF-Lite) per customer, and each customer-facing interface is placed in its customer's VRF. CEs talk to their own VRF and never see anyone else's routes. P routers have no VRFs at all.

Why a VRF and not just ACLs? Because the problem isn't "who may talk to whom" — it's that two customers use the same addresses. A single routing table can hold only one route to 10.1.1.0/24. A VRF gives each customer a separate table, so both can exist at once.

Worked example — the same subnet twice on one PE. Customer A (VRF CUST_A, on Gi0/2) and Customer B (VRF CUST_B, on Gi0/3) both use 10.1.1.0/24:

Command on PE1	What it shows
`show ip route vrf CUST_A 10.1.1.0`	10.1.1.0/24 via CE1 (192.168.1.2), Gi0/2
`show ip route vrf CUST_B 10.1.1.0`	10.1.1.0/24 via Customer B's CE (192.168.1.6), Gi0/3

Command on PE1	What it shows
`show ip route 10.1.1.0`	`% Network not in table` — the global table holds only the provider's own routes

A packet arriving on Gi0/2 is looked up only in CUST_A's table; one arriving on Gi0/3 only in CUST_B's. The interface a packet arrives on decides which table is used.

29.8 Route Distinguisher vs Route Target — Why Both Exist

This is the single most-asked L3VPN concept. They sound alike and do completely different jobs.

Aspect		
Is	A 64-bit value **prepended** to the IPv4 prefix	A BGP **extended community** attached to the route
Job	Make overlapping prefixes **unique** inside BGP	Control which VRFs **import** the route
Result	65000:100:10.1.0.0/16 ≠ 65000:200:10.1.0.0/16 — a **VPNv4** prefix (96 bits)	"Any VRF importing 65000:100 takes this route"
How many per route	Exactly **one**	**One or many**
Policy?	None — just a uniqueness tag	All of it

Why isn't the RD enough on its own — couldn't the RD also say which VPN a route belongs to? Because a route can have only one RD, yet many real designs need a route to land in several VRFs. An **extranet** (Customer A's servers shared with Customer B), a **hub-and-spoke VPN** (spokes import only the hub's routes; the hub imports everyone's) and a **shared-services VRF** all need "this route belongs in VRFs X and Y". RTs are a list, so a route can carry several and each VRF picks what it wants. The RD's only job is to stop BGP treating two customers' 10.1.0.0/16 as the same prefix and keeping just one. The RD makes routes unique; the RT decides where they go.

How it works — export and import, step by step.

1. A route enters a VRF on a PE (from the CE).
2. On its way into MP-BGP, the PE **prepends the VRF's RD** (making a VPNv4 prefix) and **attaches every `route-target export` value** of that VRF as extended communities.
3. On every other PE, each local VRF compares the route's RTs with its `route-target import` list. **Any one match** is enough: the route is copied into that VRF.
4. Once inside a VRF the RD is dropped — the VRF holds a plain IPv4 route again.

Worked example — a shared-services extranet. A provider hosts a shared DNS/NTP service for two customers who must not see each other:

VRF	Exports	Imports	Ends up containing
CUST_A	65000:100	65000:100, **65000:999**	Its own sites + shared services

VRF	Exports	Imports	Ends up containing
CUST_B	65000:200	65000:200, **65000:999**	Its own sites + shared services
SHARED	**65000:999**	65000:100, 65000:200	Shared services + both customers' routes

Walk one route through: CUST_A's 10.1.1.0/24 leaves PE1 tagged RT 65000:100. CUST_A on other PEs imports it (match on 65000:100). SHARED imports it (match on 65000:100). CUST_B does **not** — none of its imports is 65000:100. So A reaches the services, the services can answer A, and A and B stay invisible to each other. The one design rule this exposes: the SHARED VRF now holds routes from both customers, so the customer subnets that meet there **must not overlap** — a VRF is still one routing table.

> **Exam trap:** the RD and RT are often configured with the **same number** (e.g. both 65000:100). That's a convention, not a requirement — they're unrelated fields. A question showing matching RDs but mismatched RTs is testing whether you know that **only the RTs decide import**.

29.9 MP-BGP Between PEs

The problem it solves: PE1 and PE2 each hold VRFs full of customer routes, but the core in between knows nothing about customers. The PEs need their own private conversation to swap those routes, and BGP is the protocol built for carrying huge numbers of routes with tags attached. So PEs exchange customer routes as **VPNv4** prefixes over an iBGP session (address family `vpnv4`), each update carrying:

- the **VPNv4 prefix** (RD + IPv4 prefix),
- the **RT** extended communities (so `send-community extended` must be on),
- a **VPN label** chosen by the advertising PE for that prefix,
- a **next hop** of the advertising PE's **loopback** — which must be a /32 with an LSP to it.

The receiving PE checks each route's RTs against every local VRF's `import` list, strips the RD and installs the plain IPv4 prefix into each matching VRF.

How a customer route travels from PE2 to PE1:

1. **CE2 → PE2.** CE2 advertises 10.2.2.0/24 to PE2 over eBGP (CE2 is AS 65102). The route lands in VRF CUST_A on PE2.
2. **Into BGP.** Routes learned by eBGP inside `address-family ipv4 vrf CUST_A` are already in BGP. (If the CE used OSPF, EIGRP or static instead, PE2 would need `redistribute ospf …` — or `redistribute static` — under `address-family ipv4 vrf CUST_A`, plus redistribution of BGP back into the CE protocol for the other direction. Forgetting that is a classic "routes never cross the cloud" fault.)
3. **Export.** PE2 prepends RD 65000:100 → `65000:100:10.2.2.0/24`, attaches RT 65000:100, allocates VPN label **24**, sets next hop **4.4.4.4** (its Loopback0) and sends the update to PE1 in the `vpnv4` address family.

4. **Receive.** PE1 checks the next hop: is there a route — and an LSP — to 4.4.4.4? Yes (LDP label 18 via P1).
5. **Import.** RT 65000:100 matches CUST_A's `route-target import`, so PE1 installs 10.2.2.0/24 in VRF CUST_A, remembering "VPN label 24, next hop 4.4.4.4".
6. **To CE1.** PE1 advertises 10.2.2.0/24 to CE1 over the CE–PE protocol. CE1 now has an ordinary route.

Worked example — reading the result on PE1:

```
PE1# show bgp vpnv4 unicast vrf CUST_A 10.2.2.0/24
BGP routing table entry for 65000:100:10.2.2.0/24, version 18
Paths: (1 available, best #1, table CUST_A)
  Advertised to update-groups:
     1
  Refresh Epoch 1
  65102
    4.4.4.4 (metric 3) (via default) from 4.4.4.4 (4.4.4.4)
      Origin IGP, metric 0, localpref 100, valid, internal, best
      Extended Community: RT:65000:100
      mpls labels in/out nolabel/24
```

Line by line: the entry name shows the **RD** glued to the prefix; `65102` is CE2's AS (the AS path); `4.4.4.4 ... from 4.4.4.4` is the next hop and the iBGP peer; `(via default)` means the next hop was resolved in the **global** table, where the LSP lives; the **RT** is listed as an extended community; and `Advertised to update-groups` is PE1 passing the route on to CE1 (its eBGP neighbour inside the VRF, step 6); `in/out nolabel/24` says PE1 must push **VPN label 24** when sending toward this prefix (PE1 allocated no label of its own for a remote route).

Worked example — ticket: "Site 2's LAN never appears at site 1."

- **Symptom:** `show ip route vrf CUST_A` on PE1 has no 10.2.2.0/24. The VPNv4 session between the PEs is up.
- **Check PE2:** `show bgp vpnv4 unicast all 10.2.2.0/24` shows the route with `Extended Community: RT:65000:101`.
- **Check PE1:** the route isn't in PE1's VPNv4 table at all. That's expected: by default a PE **discards VPNv4 routes whose RTs match none of its VRFs' imports** (automatic route-target filtering), so it doesn't waste memory on other customers' routes.
- **Cause:** PE2's CUST_A exports 65000:101 — a typo — while PE1's CUST_A imports 65000:100.
- **Fix:** correct the export RT on PE2. PE2 sends a fresh update, PE1 imports it, and CE1 learns the route.

29.10 The Two-Label Stack

Packets crossing the core carry **two** labels:

Label	Called	Allocated by	Read by	Means
Outer (top)	Transport / **LDP** label	LDP, hop by hop	P routers	"Get me to PE2's loopback"

Label	Called	Allocated by	Read by	Means
Inner (bottom, S=1)	**VPN** label	**Egress PE**, advertised in MP-BGP	Egress PE only	"This belongs to VRF CUST_A, prefix 10.2.2.0/24"

Why can't one label do both jobs? Because the two labels answer questions owned by different parts of the network. The transport label must be understood by **every P router**, so it's tied to something every P router knows — the PE loopback, learned through the IGP. The VPN label is meaningful only to **the egress PE**: no P router has the VRF, so it couldn't act on it anyway. Keeping them separate means the core never learns a single customer route, and adding a new customer changes nothing on any P router. The outer label is the envelope's street address; the inner label is the apartment number only the building's doorman reads.

Worked example — the stack on the wire between PE1 and P1:

Position	Label	S bit	Who reads it
Top (outer)	**18** — "to 4.4.4.4", from P1's LDP binding	0 (another label follows)	P1
Bottom (inner)	**24** — "VRF CUST_A, 10.2.2.0/24", from PE2's MP-BGP update	1 (IP header next)	PE2 only
Then	The customer's original IP packet, untouched	—	CE2

Each label adds 4 bytes, so this packet is **8 bytes bigger** than the customer sent — the reason core links need an MTU above 1500 (29.5's MTU symptom).

29.11 End-to-End Packet Walk

Topology: **CE1 → PE1 (1.1.1.1) → P1 → P2 → PE2 (4.4.4.4) → CE2**. CE2's LAN is 10.2.2.0/24 in VRF CUST_A.

Control plane first (routes and labels):

1. CE2 advertises 10.2.2.0/24 to PE2 (eBGP, OSPF, static — 29.12).
2. PE2 installs it in VRF CUST_A, assigns VPN label **24**, and sends it to PE1 by MP-BGP as `65000:100:10.2.2.0/24`, RT 65000:100, label 24, next hop 4.4.4.4.
3. PE1 imports it into CUST_A (RT matches) with next hop 4.4.4.4.
4. Independently, LDP has built an LSP to 4.4.4.4: PE2 advertised implicit-null to P2, P2 advertised label 21 to P1, P1 advertised label 18 to PE1.

Data plane (a packet from CE1 to 10.2.2.10):

Hop	Receives	Lookup	Sends
CE1	—	Plain IP route to PE1	IP packet

Hop	Receives	Lookup	Sends
PE1	IP on a CUST_A interface	VRF CUST_A FIB → next hop 4.4.4.4, VPN label 24, LDP label 18	labels **18, 24** + IP
P1	Label 18	LFIB: 18 → swap 21	labels **21, 24** + IP
P2	Label 21	LFIB: 21 → **Pop** (PHP)	label **24** + IP
PE2	Label 24	VPN label 24 → VRF CUST_A, out to CE2	IP packet
CE2	IP	Plain IP	Delivered

Notice the VPN label 24 is never touched by P1 or P2. And notice what P1 and P2 don't need: any route to 10.2.2.0/24, any VRF, any BGP.

How PE2 knows what to do with label 24 — its LFIB has an entry for the VPN label it allocated:

```
PE2# show mpls forwarding-table labels 24
Local      Outgoing   Prefix           Bytes Label   Outgoing   Next Hop
Label      Label      or Tunnel Id     Switched      interface
24         No Label   10.2.2.0/24[V]   48210         Gi0/2      192.168.2.2
```

`[V]` marks a VPN (VRF) prefix. `No Label` **is correct here**: PE2 is handing the packet to CE2, which doesn't run MPLS, so it must leave as plain IP. Compare the exam trap in 29.4 — `No Label` is only a fault when it appears on a core router for a PE loopback.

29.12 CE–PE Routing Options

The problem it solves: the customer's router has to tell the PE which networks live at its site, and learn the other sites' networks back. Any routing method works between CE and PE — the choice depends on what the customer already runs and how much control they want.

Option	Why choose it	Watch out for
Static	Single-homed small site; simplest	No failure detection without IP SLA (32.6)
eBGP	The most common — scales, policy-rich, PE already speaks BGP	Same customer AS at every site → AS-path loop check drops routes; fix with `as-override` on the PE or `allowas-in` on the CE — both explained in 28.11
OSPF	Customer already runs OSPF everywhere	Routes arrive as inter-area (Type 3) or external; sham-links and the **down bit** prevent loops and backdoor problems
EIGRP	Customer runs EIGRP	Multi-homed sites need **Site of Origin (SoO)** to stop routes looping back in

Worked example — why `as-override` is needed. Suppose Customer A used private AS 65101 at both sites (instead of 65101 and 65102 as elsewhere in this chapter). Site 1 advertises 10.1.1.0/24; by the time it reaches CE2 the AS path is `65000 65101`. CE2 sees its **own** AS (65101) in the path and drops the route as a loop — eBGP's standard loop check. With `neighbor <CE2> as-override` on PE2, PE2 replaces 65101 with its own AS before sending, so CE2 receives `65000 65000`, finds no 65101, and accepts it. (`allowas-in` on CE2 is the alternative: CE2 accepts its own AS in the path.)

29.13 Reading a PE Configuration

Why read a whole PE config? Because the exam shows you one and asks what a line does or what's missing. Every piece from this chapter appears in it once — read it top to bottom with the comments.

```
vrf definition CUST_A
 rd 65000:100                              ! makes CUST_A prefixes unique as VPNv4
 address-family ipv4
  route-target export 65000:100            ! tag routes leaving this VRF
  route-target import 65000:100            ! accept routes carrying this RT
 exit-address-family
!
interface Loopback0
 ip address 1.1.1.1 255.255.255.255        ! /32: the BGP next hop AND LDP ID
!
interface GigabitEthernet0/1
 description Core link to P1
 ip address 10.0.13.1 255.255.255.0
 mpls ip                                   ! LDP + labels on the core side only
!
interface GigabitEthernet0/2
 description CE1 - Customer A
 vrf forwarding CUST_A                     ! NOTE: wipes the IP address; re-enter it after
 ip address 192.168.1.1 255.255.255.252
!
mpls ldp router-id Loopback0 force
!
router ospf 1                              ! core IGP: carries loopbacks, no customer routes
 router-id 1.1.1.1
 network 1.1.1.1 0.0.0.0 area 0
 network 10.0.13.0 0.0.0.255 area 0
!
router bgp 65000
 bgp router-id 1.1.1.1
 no bgp default ipv4-unicast
 neighbor 4.4.4.4 remote-as 65000          ! iBGP to the other PE (often via route reflector)
 neighbor 4.4.4.4 update-source Loopback0
 !
 address-family vpnv4
  neighbor 4.4.4.4 activate
  neighbor 4.4.4.4 send-community extended ! RTs are extended communities
 exit-address-family
 !
 address-family ipv4 vrf CUST_A
  neighbor 192.168.1.2 remote-as 65101     ! eBGP to CE1, inside the VRF
  neighbor 192.168.1.2 activate
 exit-address-family
```

What each block tells you: the **VRF** defines the customer's table and its import/export policy; the **core interface** runs LDP while the **customer interface** doesn't; the **IGP** carries only provider loopbacks and links; **BGP** has two personalities — `vpnv4` to other PEs (customer routes in transit)

and `ipv4 vrf` to the CE (customer routes at the edge).

The verification set you should recognise:

Command	Answers
`show vrf` / `show ip vrf interfaces`	Which interfaces are in which VRF
`show ip route vrf CUST_A`	The customer's table as PE1 sees it (remote routes show next hop 4.4.4.4)
`show bgp vpnv4 unicast all`	Every VPNv4 route, grouped by RD
`show bgp vpnv4 unicast all labels`	The VPN label in/out for each prefix
`show mpls forwarding-table 4.4.4.4 32`	The transport label toward the remote PE
`ping vrf CUST_A 10.2.2.10 source Gi0/2`	End-to-end test from inside the VRF

> **Depth: why traceroute through a provider often shows nothing in the middle.** By default the ingress PE copies the IP TTL into the label TTL, so the core hops answer traceroute. Providers commonly configure `no mpls ip propagate-ttl forwarded`, which sets the label TTL to 255 — the core becomes invisible and the customer sees the whole MPLS cloud as a single hop.

Chapter 30: DMVPN

> **Blueprint:** ENARSI 2.3 — Configure and verify DMVPN (single hub): 2.3.a GRE/mGRE, 2.3.b NHRP, 2.3.c IPsec, 2.3.d Dynamic neighbor, 2.3.e Spoke-to-spoke.

Chapter 12 built a single GRE tunnel and protected it with IPsec: one tunnel, two routers, both addresses typed in by hand. That's fine for two sites. For fifty branches it falls apart. **DMVPN (Dynamic Multipoint VPN)** is how Cisco connects many branches over the internet with **one tunnel interface per router**, a hub that never needs reconfiguring when a branch is added, and branches that build **direct tunnels to each other on demand**.

30.1 The Problem DMVPN Solves

Picture a hub and 50 branches using point-to-point GRE:

Design	Tunnels on the hub	Branch-to-branch traffic	Adding branch 51
Hub-and-spoke p2p GRE	**50** interfaces, 50 configs	Hairpins through the hub (two internet crossings)	Edit the hub

Design	Tunnels on the hub	Branch-to-branch traffic	Adding branch 51
Full mesh p2p GRE	50 on the hub, **49 on every branch** — 1,225 tunnels	Direct	Edit **every** router
DMVPN	**1** mGRE interface	Direct, built **on demand**	Configure only the new branch

DMVPN gets there by combining three technologies, each answering one question:

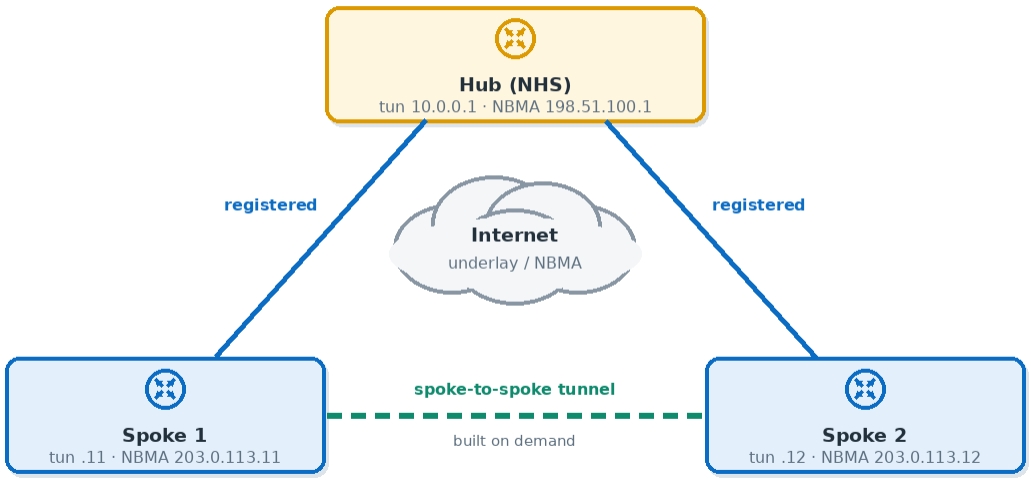
Piece	Answers
mGRE (multipoint GRE)	"How can one tunnel interface reach many destinations?"
NHRP (Next Hop Resolution Protocol)	"What's the real internet address behind this tunnel address?"
IPsec	"How do I keep it private across the internet?"

Plus a routing protocol running over the tunnel to exchange the sites' LANs.

Quick definitions you'll need all chapter: **GRE** (Generic Routing Encapsulation) wraps one packet inside another IP packet so it can cross a network that doesn't know its addresses. **IPsec** (IP Security) encrypts and authenticates packets. **NBMA** (Non-Broadcast Multi-Access) is the old name for a network — like the internet underneath DMVPN — where many routers can reach each other but you can't broadcast to find them. A **hub** is the central router; **spokes** are the branch routers.

Story Time : Think of a hotel. When guests arrive, each one checks in at **reception (the hub)** and leaves a note: "I'm in room 11, and my mobile number is 203.0.113.11." Reception never needs to know the guests in advance — they register themselves. When the guest in room 11 wants to talk to the guest in room 12, they could pass every message through reception (slow, and reception gets busy), or they can ask reception once, "what's room 12's mobile number?", and then call room 12 **directly**. In DMVPN the room numbers are **tunnel addresses**, the mobile numbers are **internet (NBMA) addresses**, the check-in is **NHRP registration**, asking reception is **NHRP resolution**, and the direct call is a **spoke-to-spoke tunnel**. Where it breaks: in a real hotel reception answers your question itself; in DMVPN (Phases 2 and 3) reception passes your question to room 12, and room 12 answers you directly (30.4, 30.5).

DMVPN — one mGRE tunnel subnet (10.0.0.0/24) laid over the internet



hub knows spokes from NHRP registration — adding Spoke 3 never touches the hub

30.2 mGRE — One Interface, Many Destinations

The problem it solves: with point-to-point GRE, every new branch means a new tunnel interface on the hub. mGRE lets one tunnel interface talk to any number of routers, so the hub's configuration stops growing with the number of branches.

A normal GRE tunnel has `tunnel destination` — exactly one far end. An **mGRE** tunnel (`tunnel mode gre multipoint`) has **no destination at all**. It's a tunnel interface that behaves like a shared LAN: every router's tunnel interface sits in **one subnet** (say 10.0.0.0/24), like hosts on an Ethernet segment.

That immediately raises the question an Ethernet segment answers with ARP: I want to send to 10.0.0.12 on this tunnel — what outer (internet) address do I put on the GRE packet? There's no broadcast on the internet to ask with. That's NHRP's job.

Two addresses per router, and you must keep them straight:

Address	DMVPN term	Example	Lives in
The tunnel interface IP	**Tunnel / overlay address**	10.0.0.11	The inside of the GRE packet
The physical internet-facing IP	**NBMA / underlay address**	203.0.113.11	The outer IP header

Worked example — what one packet looks like on the wire. A host behind Spoke 1 (10.11.0.5) sends to the hub's LAN (10.10.0.9). Spoke 1 routes it into Tunnel0 and builds:

Layer (outside → inside)	Source	Destination	Why
Outer IP header (underlay)	203.0.113.11	198.51.100.1	So the internet can deliver it to the hub

Layer (outside → inside)	Source	Destination	Why
GRE header (key 100)	—	—	Says "a packet is inside"; the key identifies the tunnel
Inner IP header (the original)	10.11.0.5	10.10.0.9	The real conversation, untouched

Spoke 1 knew to put **198.51.100.1** on the outside because it has a static NHRP mapping for the hub (10.0.0.1 → 198.51.100.1). For any other tunnel address, it has to ask — that's 30.3.

Story Time : A point-to-point GRE tunnel is a **dedicated phone line** to one friend: pick it up and you're connected to them and only them. An mGRE tunnel is **one phone with a contacts list**: one handset, any number of people — but you need to look up the number before you can dial. NHRP is the contacts list.

30.3 NHRP — ARP for the Internet

NHRP is a client/server protocol. The **hub** is the **NHS** (Next Hop Server); spokes are **NHCs** (clients).

Registration. When a spoke's tunnel comes up it sends an **NHRP Registration Request** to the hub: "I'm 10.0.0.11 and you can reach me at 203.0.113.11." The hub stores that as a **dynamic** mapping. The spoke only needs to know the hub's addresses in advance — configured statically — which is exactly why spokes can be added without touching the hub. Registrations are refreshed at one-third of the holdtime by default (`ip nhrp holdtime`, default 7200 s; commonly lowered to 300–600 so stale entries age out faster). The refresh interval can be set explicitly with `ip nhrp registration timeout <seconds>` on the spoke — useful when you want the hub to notice a returning spoke quickly without shortening how long its mappings live.

How registration works, step by step (Spoke 1: tunnel 10.0.0.11, NBMA 203.0.113.11; hub: tunnel 10.0.0.1, NBMA 198.51.100.1; holdtime 300 s):

1. Spoke 1's Tunnel0 comes up. Its config says "my NHS is 10.0.0.1, at NBMA 198.51.100.1", so it can reach the hub without asking anyone.
2. If `tunnel protection` is configured, IPsec to 198.51.100.1 is negotiated first (30.6) — NHRP messages travel inside the encrypted GRE tunnel.
3. Spoke 1 sends an **NHRP Registration Request** to 10.0.0.1: "tunnel 10.0.0.11 = NBMA 203.0.113.11, valid for 300 s." It carries the NHRP authentication string, which must match.
4. The hub stores a **dynamic** entry 10.0.0.11 → 203.0.113.11 and, because of `ip nhrp map multicast dynamic`, adds 203.0.113.11 to its list of places to send multicast (routing hellos).
5. The hub sends a **Registration Reply**. Spoke 1 now shows the hub as `UP`.
6. Spoke 1 re-registers every 100 s (one-third of 300), so the hub's entry never expires while the spoke is alive. If the spoke dies, the entry ages out after 300 s.

The result on the hub:

```
Hub# show ip nhrp
10.0.0.11/32 via 10.0.0.11
   Tunnel0 created 00:14:03, expire 00:04:12
   Type: dynamic, Flags: registered used nhop
   NBMA address: 203.0.113.11
10.0.0.12/32 via 10.0.0.12
   Tunnel0 created 00:13:50, expire 00:04:31
   Type: dynamic, Flags: registered used nhop
```

Read it as: "tunnel address 10.0.0.11 lives at internet address 203.0.113.11; the spoke registered it (`registered`, `dynamic`); it expires in 4 min 12 s unless refreshed."

Dynamic neighbors — why adding branch 51 needs no hub change. This registration is the "dynamic" in DMVPN (blueprint 2.3.d). The hub never lists its spokes: each spoke announces itself. To add a branch you configure only the branch — its tunnel address, the hub's NHS address, the shared keys. Within seconds the hub has an NHRP entry for it, starts sending it routing hellos, and the routing protocol forms an adjacency. With BGP, the hub's `bgp listen range` does the same job for BGP sessions (30.8).

Resolution. When a spoke needs to reach another spoke's tunnel address, it sends an **NHRP Resolution Request**; the answer (the other spoke's NBMA address) lets it build a direct GRE tunnel. Phase 3 adds **Redirect** messages to trigger this (30.4).

Command	On	Does
`ip nhrp network-id 1`	All	Enables NHRP on the interface. **Locally significant** — it identifies the NHRP domain on this router; it doesn't have to match peers (though by convention it does)
`ip nhrp authentication KEY`	All	Cleartext string (max 8 chars) that **must match**; NHRP packets with the wrong key are dropped
`ip nhrp map multicast dynamic`	Hub	Replicate multicast (routing hellos!) to every spoke that registers
`ip nhrp nhs 10.0.0.1`	Spoke	"My NHS is the hub's tunnel IP"
`ip nhrp map 10.0.0.1 198.51.100.1`	Spoke	Static mapping: hub tunnel IP → hub NBMA IP
`ip nhrp map multicast 198.51.100.1`	Spoke	Send multicast (routing hellos) to the hub
`ip nhrp nhs 10.0.0.1 nbma 198.51.100.1 multicast`	Spoke	**Newer one-line form** of the three lines above
`ip nhrp redirect`	Hub	Phase 3: tell spokes "there's a better path"
`ip nhrp shortcut`	Spoke	Phase 3: accept redirects and install shortcut routes

Why does the hub need `ip nhrp map multicast dynamic`? Because mGRE has no destination, so the router has no idea where to send a multicast packet like an EIGRP hello (224.0.0.10) or OSPF hello (224.0.0.5). This command says "send a copy of every multicast to each spoke that has registered". Without it the hub's hellos go nowhere, and **routing adjacencies never form even though `show dmvpn` looks perfect**. That mismatch — NHRP up, routing down — is the classic DMVPN troubleshooting question.

30.4 The Three Phases

"Phase" describes how spoke-to-spoke traffic is handled. ENARSI expects you to compare them and know the routing implications. **Why three phases?** Each one fixed the previous one's weakness: Phase 1 was simple but sent everything through the hub; Phase 2 allowed direct spoke-to-spoke tunnels but forced every spoke to know every route; Phase 3 kept the direct tunnels and let the hub summarize again.

Aspect			
Spoke tunnel type	**p2p GRE** (`tunnel destination` = hub)	mGRE	mGRE
Spoke-to-spoke traffic	Always through the hub	**Direct** tunnels	**Direct** tunnels
What triggers the direct tunnel	Nothing — never built	The spoke's **routing table**: next hop is the other spoke's tunnel IP	**NHRP Redirect** from the hub
Hub may summarize / send only a default	**Yes**	**No** — next hop must be preserved	**Yes**
Key NHRP commands	—	—	Hub `ip nhrp redirect`, spokes `ip nhrp shortcut`
Scales to	Small	Medium (every spoke needs every route)	**Large**

Why can't Phase 2 summarize at the hub? Because in Phase 2 the routing table is what triggers the spoke-to-spoke tunnel. Spoke 1 looks up 10.12.0.0/24, sees next hop **10.0.0.12** (Spoke 2's tunnel address), resolves it with NHRP and goes direct. That only works if the route reached Spoke 1 with **the original next hop intact** — and with **every specific route**. If the hub sent a summary or a default, the next hop would be the hub (10.0.0.1) and Spoke 1 would happily send everything through the hub forever.

How does Phase 3 get away with summarizing? It moves the decision from the routing table to NHRP. Spokes may know only "default via hub". The hub notices a packet entering and leaving **the same DMVPN tunnel interface** — a hairpin — and sends the source spoke an **NHRP Redirect**. The spoke then resolves the destination and installs an **NHRP shortcut route** that overrides the routing table for that destination. Phase 2 asks the routing protocol to know everything; Phase 3 lets the routing protocol be lazy and lets NHRP fix the path when traffic actually flows.

Worked example — the same packet in each phase. A host behind Spoke 1 sends to 10.12.0.10 (behind Spoke 2):

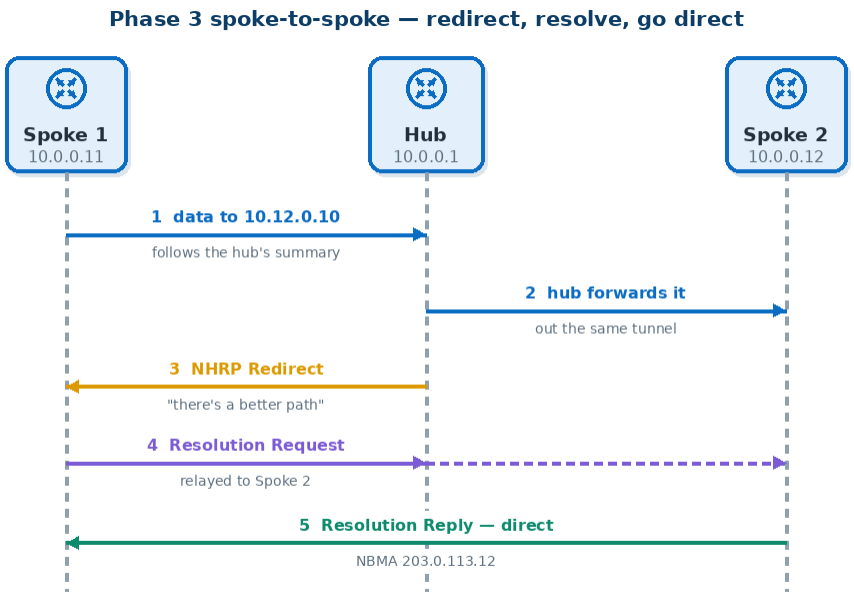
Phase	Spoke 1's routing table says	What happens to the packet
Phase 1	10.12.0.0/24 (or a summary) via **10.0.0.1** — the hub	Always to the hub; the hub forwards it to Spoke 2. Every packet crosses the internet twice.

Phase	Spoke 1's routing table says	What happens to the packet
Phase 2	10.12.0.0/24 via **10.0.0.12** — Spoke 2 itself	Spoke 1 has no NHRP entry for 10.0.0.12, so it sends a Resolution Request to the hub (the first packets go via the hub meanwhile). The hub forwards the request to Spoke 2, which replies directly: "10.0.0.12 = 203.0.113.12". Spoke 1 builds a direct tunnel; later packets go straight there.
Phase 3	Only 10.0.0.0/8 via **10.0.0.1** — the hub's summary	First packets go via the hub; the hub sends a Redirect; Spoke 1 resolves and installs an `H` shortcut route for 10.12.0.0/24 via 10.0.0.12 (30.5). Later packets go direct.

How Phase 2 resolution works, step by step:

1. Spoke 1 looks up 10.12.0.10 → route via next hop **10.0.0.12** on Tunnel0.
2. To build the GRE packet it needs 10.0.0.12's NBMA address. `show ip nhrp` has no entry, so Spoke 1 sends an **NHRP Resolution Request** for 10.0.0.12 to its NHS (the hub). While it waits, the packets themselves are forwarded through the hub so the user sees no outage.
3. The hub has Spoke 2's registration, so it forwards the request to Spoke 2.
4. Spoke 2 sends a **Resolution Reply** straight to Spoke 1 (negotiating IPsec with 203.0.113.11 first): "10.0.0.12 is at 203.0.113.12."
5. Spoke 1 caches 10.0.0.12 → 203.0.113.12. New packets are GRE-encapsulated with outer destination **203.0.113.12** — direct.
6. Spoke 2 does the same in reverse for its return traffic, and the entries age out with the holdtime when the traffic stops.

30.5 Spoke-to-Spoke in Phase 3, Step by Step



1. A host behind **Spoke 1** sends to 10.12.0.10 (behind Spoke 2). Spoke 1's route is the hub's summary, so the packet goes **to the hub**.
2. The hub forwards it on to Spoke 2 — out the **same** tunnel it came in on. Seeing that hairpin, the hub sends an **NHRP Redirect** (traffic indication) back to Spoke 1.
3. Spoke 1 sends an **NHRP Resolution Request** for 10.12.0.10. It travels through the hub to Spoke 2.
4. Spoke 2 answers **directly** to Spoke 1 (bringing up IPsec between them first): "10.12.0.0/24 is behind me, NBMA 203.0.113.12."
5. Spoke 1 installs an **NHRP shortcut**: 10.12.0.0/24 via 10.0.0.12. Later packets go spoke-to-spoke. Return traffic triggers the same process in reverse.

On the spoke, the shortcut appears in the routing table with an `H` code and a "next hop override":

```
Spoke1# show ip route nhrp
H     10.12.0.0/24 [250/1] via 10.0.0.12, 00:01:12, Tunnel0
```

Shortcuts age out with the NHRP holdtime when traffic stops — the direct tunnel is genuinely **on demand**.

30.6 Securing It: IPsec Profiles

The problem it solves: GRE on its own is not encrypted — anyone on the internet path can read the packets. IPsec adds encryption and proves that each packet really came from a member of the DMVPN.

Chapter 12's site-to-site tunnels always **named the peer**: a tunnel destination (and, for a crypto map, a `set peer`) typed in by hand, plus a pre-shared key bound to that peer's address. DMVPN

can't name its peers — spokes are discovered dynamically, and spoke-to-spoke peers aren't known until traffic needs them. So DMVPN uses an **IPsec profile** attached to the tunnel interface with `tunnel protection` (the same mechanism Chapter 12 used with a VTI), and authenticates with a **wildcard pre-shared key or certificates** instead of per-peer keys. Whenever GRE needs to reach a new NBMA address, IPsec negotiates with it automatically.

```
crypto isakmp policy 10
 encryption aes 256
 hash sha256
 authentication pre-share
 group 14
crypto isakmp key DMVPN-PSK address 0.0.0.0 0.0.0.0   ! wildcard: any peer may connect
!
crypto ipsec transform-set TS-DMVPN esp-aes 256 esp-sha256-hmac
 mode transport                       ! GRE already adds an outer header; tunnel mode wastes 20 B
!
crypto ipsec profile DMVPN-PROF
 set transform-set TS-DMVPN
!
interface Tunnel0
 tunnel protection ipsec profile DMVPN-PROF
```

How the protected tunnel comes up, step by step (Spoke 1 203.0.113.11 to the hub 198.51.100.1):

1. **Trigger.** GRE needs to send its first packet (the NHRP registration) to 198.51.100.1. `tunnel protection` says that traffic must be encrypted, and no IPsec security association (SA) exists yet.
2. **IKE phase 1** (UDP 500; the exchange moves to UDP 4500 if NAT is detected). The two routers agree on ISAKMP policy 10 — AES-256, SHA-256, pre-shared key, DH group 14 — do a Diffie-Hellman exchange to create shared secret keys, and prove they know the PSK. Result: one ISAKMP SA, shown as `QM_IDLE` in `show crypto isakmp sa`. `show dmvpn` shows the peer in state `IKE` until this finishes.
3. **IKE phase 2** (quick mode). Protected by the phase 1 SA, they agree on transform set TS-DMVPN — ESP with AES-256 and SHA-256 HMAC, **transport mode** — for GRE traffic between 203.0.113.11 and 198.51.100.1. Result: two IPsec SAs (one each direction, each with its own SPI number). State shows `IPsec` until this finishes.
4. **NHRP.** The registration now travels inside ESP. State `NHRP` until the hub's reply arrives.
5. `UP`**.** From here, `show crypto ipsec sa` shows `#pkts encaps` and `#pkts decaps` both rising.

For a spoke-to-spoke tunnel the same five steps happen between 203.0.113.11 and 203.0.113.12 — triggered automatically the moment NHRP needs to talk to that NBMA address.

Why transport mode? GRE already wraps the packet in a new IP header with the NBMA addresses. IPsec tunnel mode would add another IP header with the same addresses — 20 bytes of pure duplication. Transport mode encrypts the GRE packet and keeps its outer header. (Tunnel mode still works; it's just less efficient.)

Spokes behind NAT. When a spoke sits behind a NAT device, IKE detects it and switches to **NAT-T** (ESP inside UDP 4500). Add `crypto isakmp nat keepalive 20` (IKEv1; IKEv2 has `crypto ikev2 nat keepalive`) so the NAT device's translation for that UDP flow never times out between bursts of traffic. If the underlay interface lives in its own VRF — common when the internet link must be kept out of the global table — that's **front-door VRF (FVRF)**, covered in Chapter 12; on DMVPN

it's `tunnel vrf <name>` on the tunnel plus a VRF-aware keyring.

Why a wildcard pre-shared key? Because the hub can't list spokes it hasn't met. A wildcard PSK means every spoke shares one secret — acceptable in a lab, and the reason production DMVPN usually uses **IKEv2 with certificates (PKI)** so each router proves its own identity.

30.7 Complete Working Configuration (Phase 3, EIGRP)

Why study a full config? Because DMVPN faults are almost always one missing line, and you can only spot a missing line if you know what a complete set looks like. Every line below is commented with the job it does.

Hub NBMA 198.51.100.1, Spoke 1 NBMA 203.0.113.11, Spoke 2 NBMA 203.0.113.12. Tunnel subnet 10.0.0.0/24. LANs 10.10.0.0/24 (hub), 10.11.0.0/24, 10.12.0.0/24. The crypto block in 30.6 is on all three routers.

Hub

```
interface Tunnel0
 ip address 10.0.0.1 255.255.255.0
 no ip redirects
 ip mtu 1400                          ! leave room for GRE + IPsec overhead
 ip tcp adjust-mss 1360               ! 1400 - 20 (IP) - 20 (TCP)
 ip nhrp authentication NHRPKEY
 ip nhrp network-id 1
 ip nhrp holdtime 300
 ip nhrp map multicast dynamic        ! routing hellos to every registered spoke
 ip nhrp redirect                     ! Phase 3
 no ip split-horizon eigrp 100        ! see 30.8
 ip summary-address eigrp 100 10.0.0.0 255.0.0.0   ! Phase 3 allows summarizing
 tunnel source GigabitEthernet0/0
 tunnel mode gre multipoint
 tunnel key 100                       ! must match on every router
 tunnel protection ipsec profile DMVPN-PROF
!
router eigrp 100
 network 10.0.0.0 0.0.0.255
 network 10.10.0.0 0.0.0.255
```

Spoke 1 (Spoke 2 identical except its addresses)

```
interface Tunnel0
 ip address 10.0.0.11 255.255.255.0
 no ip redirects
 ip mtu 1400
 ip tcp adjust-mss 1360
 ip nhrp authentication NHRPKEY
 ip nhrp network-id 1
 ip nhrp holdtime 300
 ip nhrp nhs 10.0.0.1 nbma 198.51.100.1 multicast   ! hub: tunnel IP, NBMA IP, send multicast
 ip nhrp shortcut                     ! Phase 3: install shortcut routes
 tunnel source GigabitEthernet0/0
 tunnel mode gre multipoint
 tunnel key 100
 tunnel protection ipsec profile DMVPN-PROF
!
router eigrp 100
 network 10.0.0.0 0.0.0.255
 network 10.11.0.0 0.0.0.255
!
ip route 0.0.0.0 0.0.0.0 203.0.113.1  ! underlay default to the ISP
```

Why `ip summary-address` here and not on a spoke? The 10.0.0.0/8 summary from the hub is what lets each spoke carry one route instead of fifty; that's the scaling win of Phase 3. One thing to check whenever you summarize over a tunnel: the summary must never cover the **NBMA** (underlay) addresses. If the internet-facing addresses also fell inside 10.0.0.0/8, spokes would learn the route to the hub's NBMA address through the tunnel itself. That's recursive routing, and the tunnel shuts itself down (30.10).

30.8 Routing Protocols Over DMVPN

The problem it solves: NHRP only maps tunnel addresses to internet addresses. It doesn't tell anyone which LANs sit behind which spoke. A routing protocol over the tunnel does that — but the hub has one interface facing every spoke, which trips up rules designed for ordinary point-to-point links.

EIGRP

Two hub-side defaults get in the way, because the hub has **one** interface facing every spoke:

Default behaviour	Problem on a DMVPN hub	Fix	Needed in
Split horizon — don't advertise a route out the interface it was learned on	Spoke 1's LAN, learned on Tunnel0, is never sent to Spoke 2 — also on Tunnel0	`no ip split-horizon eigrp 100` on the hub tunnel	Phase 1/2/3 whenever spokes need specific routes from each other (unnecessary only if the hub sends just a summary/default)
Next-hop-self — EIGRP sets itself as next hop	Spoke 2 sees Spoke 1's LAN via 10.0.0.1, so traffic always hairpins	`no ip next-hop-self eigrp 100` on the hub tunnel	**Phase 2 only**

Worked example — split horizon on the hub. Spoke 1 advertises 10.11.0.0/24 to the hub; it arrives on the hub's Tunnel0. Spoke 2 is also reached through Tunnel0. Split horizon says "never advertise a route back out the interface you learned it on", so the hub never tells Spoke 2 about 10.11.0.0/24. On Spoke 2, `show ip route 10.11.0.0` returns nothing. After `no ip split-horizon eigrp 100` on the hub's Tunnel0, the hub advertises it, and Spoke 2 learns 10.11.0.0/24 — via 10.0.0.1 with next-hop-self on (Phases 1 and 3), or via 10.0.0.11 with next-hop-self off (Phase 2). Split horizon was protecting against loops on a link with one neighbour; on a multipoint tunnel the "same interface" leads to many different routers.

Phase 3 leaves next-hop-self **on** — the hub should be the next hop until NHRP redirects. On named-mode EIGRP the same knobs are `no split-horizon` and `no next-hop-self` under `af-interface Tunnel0`.

OSPF

OSPF cares about network type, and the default on a tunnel interface is **point-to-point** — which permits **one** neighbour. On an mGRE hub with several spokes, adjacencies flap as each new spoke replaces the last.

Network type	DR?	Next hop to other spokes	Fits
point-to-point (default)	No	—	**Never on the mGRE hub** (one neighbour max)
broadcast	Yes — **hub must be DR**; spokes `ip ospf priority 0`	Preserved (the other spoke)	**Phase 2**
point-to-multipoint	No	Rewritten to the hub	Phase 1 and **Phase 3**
non-broadcast	Yes	Preserved	Phase 2, but needs `neighbor` statements on the hub

Hub and spokes must use the same network type. The types don't just change DR behaviour; they change the default timers — broadcast and point-to-point use hello 10 s / dead 40 s, while point-to-multipoint and non-broadcast use hello 30 s / dead 120 s. A hub on point-to-multipoint and a spoke on broadcast therefore have mismatched hello/dead intervals and **never form an adjacency**. Even where the timers happen to match (point-to-point vs broadcast), mixing a DR type with a non-DR type gives inconsistent LSAs and missing routes. Pick one type for the whole DMVPN cloud and set it on every tunnel interface.

Why must the hub be DR in broadcast mode? Spokes only have adjacencies with the hub (they can't exchange hellos until a spoke-to-spoke tunnel exists, and multicast only flows hub↔spoke). If a spoke became DR, other spokes couldn't reach it to synchronise — the LSDB would fragment. Priority 0 on every spoke guarantees the hub wins.

One OSPF limitation to remember: all DMVPN routers share one area for the tunnel, and OSPF can only summarize at an ABR/ASBR — so OSPF over DMVPN can't give spokes "just a summary" as easily as EIGRP or BGP. That's why large Phase 3 designs favour EIGRP or BGP.

BGP and dynamic neighbors

A hub configured with one `neighbor` statement per spoke would need editing for every new branch — defeating the point of DMVPN. **BGP dynamic neighbors** fix that: the hub listens for BGP sessions from a whole subnet.

```
router bgp 65000
 neighbor SPOKES peer-group                          ! create the peer group FIRST
 neighbor SPOKES remote-as 65000                     ! iBGP spokes
 bgp listen range 10.0.0.0/24 peer-group SPOKES   ! accept any spoke from the tunnel subnet
 bgp listen limit 200                                ! cap on dynamic peers
 neighbor SPOKES timers 10 30
 !
 address-family ipv4
  neighbor SPOKES activate
  neighbor SPOKES route-reflector-client             ! iBGP: reflect spoke routes to spokes
  network 10.10.0.0 mask 255.255.255.0
 exit-address-family
```

Type it in that order: `bgp listen range … peer-group SPOKES` is rejected if the peer group doesn't exist yet. (In `show running-config`, IOS may display the `bgp listen` lines first — that's display order, not the order you can enter them.)

Spokes configure a normal static `neighbor 10.0.0.1 remote-as 65000`; the **spoke** initiates and the hub accepts because the source falls inside the listen range. Dynamic peers show up in `show ip bgp summary` with an asterisk:

```
Neighbor        V    AS MsgRcvd MsgSent   TblVer  InQ OutQ Up/Down  State/PfxRcd
*10.0.0.11      4 65000      22      25        9    0    0 00:09:41        1
*10.0.0.12      4 65000      21      25        9    0    0 00:09:30        1
* Dynamically created based on a listen range command
```

Why route-reflector-client? iBGP's split-horizon rule says a route learned from one iBGP peer isn't passed to another (Chapter 28). The hub must be a route reflector for spokes to learn each other's LANs. A route reflector **doesn't change the next hop**, which suits Phase 2; for Phase 3 you can instead send spokes only a default or summary (`neighbor SPOKES default-originate` plus outbound filtering). With **eBGP** spokes in different ASes, a dynamic peer-group still needs to know which ASes to accept: give the first with `neighbor SPOKES remote-as 65001` and list the rest with `neighbor SPOKES alternate-as 65002 65003`.

30.9 Verification

Why these commands? DMVPN is four layers stacked on each other — underlay, IPsec, NHRP, routing — and each has its own `show` command. Verification means checking each layer is up, bottom first.

```
Hub# show dmvpn
Legend: Attrb --> S - Static, D - Dynamic, I - Incomplete
        N - NATed, L - Local, X - No Socket
        T1 - Route Installed, T2 - Nexthop-override
        # Ent --> Number of NHRP entries with same NBMA peer

Interface: Tunnel0, IPv4 NHRP Details
Type:Hub, NHRP Peers:2,

 # Ent  Peer NBMA Addr Peer Tunnel Add State  UpDn Tm Attrb
 ----- --------------- --------------- ----- -------- -----
     1 203.0.113.11          10.0.0.11    UP 00:14:03     D
     1 203.0.113.12          10.0.0.12    UP 00:13:50     D
```

```
Spoke1# show dmvpn
Interface: Tunnel0, IPv4 NHRP Details
Type:Spoke, NHRP Peers:2,

 # Ent  Peer NBMA Addr Peer Tunnel Add State  UpDn Tm Attrb
 ----- --------------- --------------- ----- -------- -----
     1 198.51.100.1           10.0.0.1    UP 00:14:05     S
     2 203.0.113.12          10.0.0.12    UP 00:00:41   DT1
```

How to read them: on the **hub**, every spoke is **D** (dynamic — learned by registration). On the **spoke**, the hub is **S** (static — configured with `nhs`), and Spoke 2 appeared **D** only once traffic flowed: that's a live spoke-to-spoke tunnel, and **T1** says an NHRP shortcut route was installed. The **State** column tells you how far each peer has got, and the states run in the order the layers come up:

State	Meaning	Look at
`IKE`	IKE (phase 1) negotiation not yet complete	Underlay reachability, ISAKMP policy, PSK/certificates
`IPsec`	IKE done; IPsec (phase 2) SA not yet established	Transform set / IPsec profile mismatch
`NHRP`	Crypto is up; NHRP registration or resolution not complete	NHRP authentication string, NHS/NBMA mapping, `ip nhrp network-id` present
`UP`	Crypto and NHRP both up	Routing, if traffic still fails

If a peer is stuck, the state names the layer that's broken. (Without `tunnel protection`, the crypto states are skipped.)

Command	Use it to see
`show dmvpn detail`	Per-peer detail including crypto session state
`show ip nhrp`	The NHRP cache: tunnel IP → NBMA mappings, static/dynamic, expiry
`show ip nhrp nhs detail`	Spoke's view of its hub: registration requests sent/replies received
`show crypto isakmp sa`	IKE phase 1 — `QM_IDLE` is healthy; `MM_NO_STATE` means phase 1 failing
`show crypto ipsec sa`	IPsec SAs per peer — `#pkts encaps/decaps` should both rise
`show ip route nhrp`	Phase 3 shortcut routes (code `H`)
`debug nhrp packet`	Registration, resolution and redirect messages as they happen

30.10 Troubleshooting DMVPN

Work **underlay → IPsec → NHRP → routing**, in that order: each layer depends on the one before. (With `tunnel protection`, IPsec must come up before NHRP messages can even pass — the same order `show dmvpn` reports: IKE → IPsec → NHRP → UP.)

Symptom	Likely cause	Command that proves it
Spoke never appears on hub	Underlay: no route to hub NBMA, or ACL/ISP blocking **UDP 500/4500 and ESP (IP 50)**	`ping 198.51.100.1 source Gi0/0`, `show crypto isakmp sa`
`show dmvpn` state stuck at `IKE`	IKE phase 1 failing: PSK mismatch, ISAKMP policy mismatch, UDP 500/4500 blocked	`show crypto isakmp sa` (`MM_NO_STATE`), `debug crypto isakmp`
`show dmvpn` state stuck at `IPsec`	Phase 2 mismatch: transform set or mode in the IPsec profile	`show crypto ipsec sa` (no SAs), `debug crypto ipsec`

Symptom	Likely cause	Command that proves it
`show dmvpn` state stuck at `NHRP`	Crypto is fine; NHRP isn't: authentication string mismatch, wrong `nhs`/NBMA mapping	`show ip nhrp nhs detail`, `debug nhrp packet`
Registration fails, IPsec up	**NHRP authentication** string mismatch, or wrong NHS/NBMA mapping on the spoke	`show ip nhrp nhs detail` (requests sent, no replies), `debug nhrp packet`
Nothing at all over the tunnel; GRE counters rise on one side only	**`tunnel key` mismatch** — GRE packets with the wrong key are silently discarded	`show interface Tunnel0` (key shown), compare configs
`show dmvpn` all UP, but **no routing neighbours**	Hub missing `ip nhrp map multicast dynamic`, or spoke missing `multicast` on its NHS/map	`show ip eigrp neighbors`, `show ip nhrp multicast`
OSPF adjacencies to spokes keep flapping	Hub tunnel left at OSPF network type **point-to-point**	`show ip ospf interface Tunnel0`
OSPF hub and spoke never become neighbours	**Network-type mismatch** (e.g. point-to-multipoint vs broadcast) → hello/dead 30/120 vs 10/40	`show ip ospf interface Tunnel0` on both — compare type and timers
Spokes learn each other's routes but traffic still hairpins (Phase 2)	`no ip next-hop-self eigrp` missing on the hub	`show ip route` on spoke: next hop is 10.0.0.1
Spokes don't learn each other's routes at all	Split horizon still on at the hub; or iBGP hub isn't a route reflector	`show ip eigrp topology`, `show ip bgp neighbors ... advertised-routes`
Phase 3: no spoke-to-spoke tunnel ever forms	`ip nhrp redirect` missing on hub or `ip nhrp shortcut` missing on spoke	`show ip nhrp traffic` (no redirects), `show dmvpn` (no D entries on spoke)
Pings work, web pages / file copies hang	**MTU**: GRE + IPsec overhead pushes full-size packets past 1500 with DF set	`ping 10.12.0.10 size 1400 df-bit`, check `ip mtu 1400` + `ip tcp adjust-mss 1360`
`%TUN-5-RECURDOWN: Tunnel0 temporarily disabled due to recursive routing`	The route to the **NBMA** address is being learned **through the tunnel**	`show ip route 198.51.100.1` — must point to the physical interface
Spoke behind NAT can't build spoke-to-spoke	NAT on underlay; needs NAT-T (UDP 4500) and NHRP sees the post-NAT address	`show dmvpn` attribute **N** (NATed)
NATed spoke drops off after idle periods	NAT translation for UDP 4500 timed out	Add `crypto isakmp nat keepalive 20`; `show crypto isakmp sa detail`

Worked example — ticket: "DMVPN is up but the branches have no routes."

- **Symptom:** a new hub was just deployed. Spokes can ping the hub's tunnel address 10.0.0.1, but `show ip route eigrp` on every spoke is empty.

- **Step 1 — underlay, IPsec and NHRP:** `show dmvpn` on the hub lists both spokes as `UP` with attribute `D`. So the bottom three layers are fine.
- **Step 2 — routing:** `show ip eigrp neighbors` on the hub has no stable neighbours: spokes appear briefly and drop with `retry limit exceeded`, because the hub's unicast updates go unacknowledged by spokes that have never heard a hub hello. On a spoke, `debug eigrp packets hello` shows hellos sent but none received from the hub.
- **Step 3 — where do the hub's hellos go?**

```
Hub# show ip nhrp multicast
Hub#
```

- **Reading it:** an empty list. The hub has nowhere to send multicast, so its EIGRP hellos (to 224.0.0.10) leave Tunnel0 to no one. (Spokes can send hellos, because their `nhs ... multicast` names the hub — but EIGRP needs hellos both ways.)
- **Cause:** `ip nhrp map multicast dynamic` is missing on the hub's Tunnel0.
- **Fix:** add it. Each spoke's next registration adds it to the list:

```
Hub# show ip nhrp multicast
  I/F     NBMA address
Tunnel0    203.0.113.11       Flags: dynamic          (Enabled)
Tunnel0    203.0.113.12       Flags: dynamic          (Enabled)
```

Seconds later, `%DUAL-5-NBRCHANGE: EIGRP-IPv4 100: Neighbor 10.0.0.11 (Tunnel0) is up: new adjacency` appears, and the spokes learn the hub's summary.

Worked example — ticket: "the tunnel keeps going down by itself."

- **Symptom:** every minute or so, `%TUN-5-RECURDOWN: Tunnel0 temporarily disabled due to recursive routing` on Spoke 1.
- **Check:** `show ip route 198.51.100.1` → `D EX 198.51.0.0/16 via 10.0.0.1, Tunnel0`. The route to the hub's internet address points into the tunnel.
- **Why it breaks:** to send a GRE packet to 198.51.100.1, the router must send it through Tunnel0 — which needs another GRE packet to 198.51.100.1, and so on. The router detects the loop and shuts the tunnel; EIGRP drops; the bad route disappears; the tunnel comes back; EIGRP relearns the route… and it repeats.
- **Cause:** someone redistributed a 198.51.0.0/16 route into EIGRP at the hub, so it's advertised over the tunnel. The spoke's only underlay route was its static default (0.0.0.0/0 to the ISP). The /16 is **more specific**, and longest match beats everything — administrative distance is only compared between routes to the same prefix — so the tunnel route wins.
- **Fix:** stop advertising underlay prefixes into the overlay routing protocol (filter them on the hub, or with a distribute-list inbound on the spoke). The rule: **never let the tunnel learn the route to its own tunnel destination.**

Why is `ip tcp adjust-mss` needed if `ip mtu` is already set? `ip mtu 1400` makes the router fragment (or drop, if DF is set) anything bigger than 1400 entering the tunnel. Many hosts set DF and rely on Path MTU Discovery — which breaks when an ICMP "fragmentation needed" message is filtered somewhere. `adjust-mss` rewrites the MSS in TCP SYNs passing through, so the two hosts agree on segments small enough to fit **before** the first big packet is ever sent. `ip mtu` handles the packet that's too big; `adjust-mss` stops TCP from ever making one. (The TCP mechanics are in

CCNA guide 2.7.)

Chapter 31: Infrastructure Security: AAA, ACLs, uRPF, CoPP & IPv6 First-Hop Security

> **Blueprint:** ENARSI 3.1 — Troubleshoot device security using IOS AAA (TACACS+, RADIUS, local database); ENARSI 3.2 — Troubleshoot router security features: 3.2.a IPv4 ACLs (standard, extended, time-based), 3.2.b IPv6 traffic filter, 3.2.c uRPF; ENARSI 3.3 — Troubleshoot CoPP (Telnet, SSH, HTTP(S), SNMP, EIGRP, OSPF, BGP); ENARSI 3.4 — Describe IPv6 first-hop security (RA guard, DHCP guard, binding table, ND inspection/snooping, source guard).

Chapter 17 taught you to **build** AAA, ACLs and CoPP for ENCOR. ENARSI's verbs are different: almost everything here says **troubleshoot**. That means you need to know how each feature fails, what the failure looks like from the outside, and which command proves the cause. Every section ends with that table.

There's a theme running through the whole chapter, and it's worth naming up front: **security features fail closed.** A broken routing protocol usually announces itself. A broken security feature quietly drops something — a login, a routing hello, a neighbour discovery packet — and the symptom shows up somewhere else entirely.

31.1 IOS AAA — How Method Lists Really Behave

The problem it solves: with local usernames on every router, a company with 300 routers has 300 copies of every admin's password — and when someone leaves, 300 places to remove them. **AAA** (Authentication, Authorization and Accounting) moves those decisions to a central server: **authentication** checks who you are, **authorization** decides what you may do, and **accounting** records what you did. The two server protocols are **TACACS+** (Terminal Access Controller Access-Control System Plus, Cisco's) and **RADIUS** (Remote Authentication Dial-In User Service, the open standard).

A **method list** is an ordered list of places to check credentials:

```
aaa authentication login VTY-AUTH group TACACS-GRP local
```

"For logins using the VTY-AUTH list, ask the TACACS-GRP servers first; then the local user database."

The single most important — and most misunderstood — rule:

Server's answer	What IOS does next
PASS (accept)	Logs you in

Server's answer	What IOS does next
FAIL (reject — wrong password, unknown user)	**Stops. Login denied.** The next method is **not** tried
ERROR (no response, timeout, unreachable, can't decrypt)	Tries the **next method** in the list

Why doesn't IOS fall back to `local` when the server says "wrong password"? Because then the fallback would be a security hole: anyone the central server rejects — a fired employee whose account was disabled — could try again against a stale local account. The fallback exists for **availability** (the server is down, so let admins in with the emergency local account), not as a second chance. Local is the spare key for when the lock is broken, not a second lock you can try when the first one says no.

That rule produces two of the most common exam scenarios:

1. **"The TACACS server is down and I can't log in."** The list ends with `group TACACS-GRP` and no `local` — no fallback exists. Or `local` is there but **no local username was ever created**, so the fallback has nothing to check.
2. **"My local password works on the console but not on SSH."** The VTY list's first method is answering FAIL (the server is up and rejecting) — so local is never consulted.

A typical working configuration

```
aaa new-model
!
username admin privilege 15 secret 9 $9$...      ! the emergency local account
!
tacacs server ISE-1
 address ipv4 10.10.10.5
 key 7 0822455D0A16                               ! must match the server exactly
!
aaa group server tacacs+ TACACS-GRP
 server name ISE-1
 ip tacacs source-interface Loopback0             ! the IP the server knows us by
!
aaa authentication login default local            ! console etc.: local only (safe)
aaa authentication login VTY-AUTH group TACACS-GRP local
aaa authorization exec VTY-AUTH group TACACS-GRP local if-authenticated
aaa accounting exec default start-stop group TACACS-GRP
!
line vty 0 15
 login authentication VTY-AUTH
 authorization exec VTY-AUTH
 transport input ssh
```

Authenticating the `enable` command

Login authentication only gets you to the EXEC prompt. With `aaa new-model`, the **`enable`** command has its own method list:

```
aaa authentication enable default group TACACS-GRP enable
!                                 ^ ask TACACS+ first   ^ fall back to the local enable secret
```

The same PASS/FAIL/ERROR rule applies: if TACACS+ is unreachable (ERROR), the router falls back to the locally configured `enable secret`; if TACACS+ rejects the password (FAIL), you're denied. Two things catch people. First, a list of just `group TACACS-GRP` with no `enable` fallback means a server outage leaves nobody able to reach privileged mode. Second, if the `enable` method is listed but no `enable secret` was ever configured, the fallback has nothing to check. (On the TACACS+ server, enable passwords are a separate setting from the login password, so a user who logs in fine can still be rejected at `enable`.) A user stuck at the `>` prompt usually has an enable-authentication problem, not a login problem — `debug aaa authentication` shows `ENABLE` as the action.

How a TACACS+ SSH login works, step by step (user `jsmith` SSHes to R1, which uses the configuration above):

1. The VTY line uses list **VTY-AUTH**, whose first method is the TACACS-GRP group.
2. R1 opens a **TCP 49** connection from Loopback0 to 10.10.10.5 and sends an authentication **START** with the username. Everything after the TACACS+ header is encrypted with the shared key.
3. The server replies **GETPASS**; R1 prompts the user for a password and sends it back in a **CONTINUE** message.
4. The server checks the password and replies **PASS** (or **FAIL**). On PASS, authentication is done.
5. Because of `aaa authorization exec VTY-AUTH`, R1 sends a separate **authorization** request: "may jsmith start an EXEC shell, and at what level?" The server returns attributes such as `priv-lvl=15`. The user lands at `R1#`.
6. Because of `aaa accounting exec`, R1 sends an accounting **START** record; when the user logs out, a **STOP** record.

That separation of steps 4, 5 and 6 is why TACACS+ can authorize **each command** separately — every command the user types can trigger its own authorization request.

The RADIUS version is shorter: R1 sends one **Access-Request** over UDP 1812 (username plus the password, hidden using the shared secret); the server replies **Access-Accept** — carrying the authorization attributes in the same packet, for example the Cisco AV-pair `shell:priv-lvl=15` — or **Access-Reject**. Authentication and authorization arrive together, which is why RADIUS can't authorize individual commands.

Story Time : TACACS+ is a security guard who checks your ID at the front door, then checks a list again every time you try to open an office door, writing everything in a logbook. RADIUS is a guard who checks your ID once and hands you a badge already printed with the rooms you may enter. The badge is quicker; the guard at every door is finer-grained. That's why TACACS+ is used for router admins (who type commands) and RADIUS for network access (who just need to be let in).

RADIUS looks the same with `radius server ISE-1` / `address ipv4 10.10.10.5 auth-port 1812 acct-port 1813` / `key ...`, `aaa group server radius RADIUS-GRP`, and `ip radius source-interface`.

Protocol	Transport	Encrypts	Separates authN / authZ / accounting	Typical use
TACACS+	**TCP 49**	The whole payload	**Yes** — per-command authorization possible	Device administration
RADIUS	**UDP 1812/1813** (legacy 1645/1646)	Only the password	No — authN and authZ combined	Network access (802.1X, VPN)

Why does `aaa new-model` lock people out? The moment you enter it, every line that doesn't name a list uses the **default** list — and if there is no default list, IOS uses **local** authentication on every such line, console and VTYs alike (see 17.3). The VTYs are where it bites. If you had `line vty` with `password cisco` and no local usernames, you've just made remote login impossible. **Always create a local privileged user** before `aaa new-model`, and keep a console session open while you test.

Authorization — logged in, but at the wrong level

Authentication says who you are; **exec authorization** gives you your privilege level. A classic symptom: TACACS+ login succeeds, but you land at `Router>` instead of `Router#`. The server authenticated you, but either `aaa authorization exec` isn't configured (so IOS never asks for your privilege level) or the server's profile doesn't return `priv-lvl=15`. Note that **console** lines aren't subject to exec authorization unless you also configure `aaa authorization console`.

> **Exam trap:** `aaa authorization commands 15 default group TACACS-GRP` with **no `local` or `if-authenticated` fallback** means that if the server goes down, every privileged command you type is refused — including the commands to fix it.

The troubleshooting toolkit

```
R1# test aaa group TACACS-GRP admin Str0ngPass legacy
Sending password
User successfully authenticated
```

`test aaa` exercises the server **without risking a login** — the first thing to run. (`legacy` is the older output format; RADIUS tests typically use `new-code`.)

```
R1# debug aaa authentication
R1# debug tacacs
AAA/AUTHEN/LOGIN (00000041): Pick method list 'VTY-AUTH'
TPLUS: Queuing AAA Authentication request 65 for processing
TPLUS(00000041)/0/NB_WAIT: socket event 2
TPLUS: Received authen response status GET_PASSWORD (8)
TPLUS: Received authen response status FAIL (2)          ! the server REJECTED -> stop
AAA/AUTHEN (00000041): status = FAIL
```

That `FAIL` is the server's verdict, so local isn't tried. Compare with an unreachable server:

```
TPLUS(00000042)/0/NB_WAIT: timed out                     ! no answer -> ERROR
AAA/AUTHEN/LOGIN (00000042): Method=LOCAL                ! fell back to local
```

Symptom	Likely cause	Command that proves it
Every login times out, then local works	Server unreachable: routing, ACL, or **TCP 49 / UDP 1812** blocked	`test aaa group ...`, `show tacacs` / `show aaa servers` (state DEAD, timeouts)
Server logs "unknown device" / no record of the request	Router sourcing from the wrong IP — the server only trusts registered device IPs	`debug tacacs` shows source; fix `ip tacacs source-interface`
TACACS+: "invalid packet" / decryption errors; RADIUS: server drops silently	**Shared key mismatch**	Server logs; `show aaa servers` failed counts; re-enter key on both
Correct password rejected, local password also rejected	Server is answering **FAIL** (account/policy), so local never tried	`debug aaa authentication` shows `status = FAIL`
Server down and nobody can log in	No `local` fallback, or no local user exists	`show running-config | include aaa authentication|username`
Logs in at `>` instead of `#`	No `aaa authorization exec`, or server not returning priv-lvl 15	`debug aaa authorization`, `show privilege`
Configuration commands refused	Command authorization with dead server and no fallback	`debug aaa authorization` (`FAIL`/`ERROR` per command)
Console locked out too	`default` list used on console points to server only	Console list should be local; `line con 0` → `login authentication`

Worked example — ticket: "Every SSH login takes 30 seconds, then only the emergency account works."

- **Symptom:** TACACS+ users can't log in to R1. After a long pause, the local `admin` account works. Other routers authenticate the same users fine.
- **Step 1 — test without logging in:** `test aaa group TACACS-GRP jsmith Pa55word legacy` hangs for several seconds and never returns a PASS or FAIL from the server — R1 isn't getting an answer at all.
- **Step 2 — server state:**

```
R1# show tacacs
Tacacs+ Server -  public  :
               Server name: ISE-1
            Server address: 10.10.10.5
               Server port: 49
             Socket opens:          12
            Socket closes:           0
            Socket aborts:           0
            Socket errors:           0
         Socket Timeouts:          12
  Failed Connect Attempts:          12
```

- **Reading it:** twelve connection attempts, twelve timeouts — R1 never gets a TCP answer. That's an **ERROR**, which is why R1 falls back to `local` after waiting.
- **Step 3 — why can't it connect?** `ping 10.10.10.5 source Loopback0` fails, but `ping 10.10.10.5` (sourced from the outgoing interface) works. The network has no route back to R1's loopback — and the TACACS+ traffic is sourced from Loopback0 because of `ip tacacs`

`source-interface Loopback0`.

- **Cause:** R1's Loopback0 wasn't advertised into the IGP when the router was installed.
- **Fix:** advertise the loopback. `test aaa` now returns `User successfully authenticated`, and logins are instant again. (Had the server been reachable but answering FAIL, local would never have been tried — the opposite symptom.)

31.2 IPv4 ACLs — Standard, Extended and Time-Based

You've built ACLs since CCNA (CCNA guide Ch 19) and Chapter 17 reviewed them. ENARSI asks you to troubleshoot them. **Why do ACLs cause so many faults?** Because an ACL does exactly what it says, not what you meant, and it fails silently — a dropped packet doesn't log anything unless you ask it to.

Story Time : An ACL is a bouncer with a printed list, reading it **from the top** for every person who arrives. The first line that describes you decides your fate, and the bouncer stops reading. If your name isn't on the list at all, you're turned away — that's the invisible last line, `deny any`. Most ACL faults are one of two things: a broad line near the top catches people a specific line further down was meant to treat differently, or someone you forgot to list (a routing hello, a DHCP reply) is quietly turned away.

How a packet is checked, step by step — a packet from 10.1.1.5 to 172.16.1.10, TCP destination port 443, against this ACL applied inbound:

```
ip access-list extended LAN-IN
 10 deny   ip 10.1.0.0 0.0.255.255 172.16.0.0 0.0.255.255
 20 permit tcp host 10.1.1.5 host 172.16.1.10 eq 443
 30 permit ip any any
```

1. Line 10: is the source inside 10.1.0.0/16 and the destination inside 172.16.0.0/16? Yes → **deny**. Checking stops.
2. Lines 20 and 30 are never consulted for this packet. Line 20's counter stays at 0 forever — the giveaway in `show access-lists`.
3. Fix: move the specific permit above the broad deny (`ip access-list extended LAN-IN` then `5 permit tcp host 10.1.1.5 host 172.16.1.10 eq 443`). Sequence numbers let you insert a line without retyping the ACL.

Standard versus extended — what changes when troubleshooting. A **standard** ACL (1–99, 1300–1999, or named standard) matches **only the source address**. It can't tell web traffic from routing hellos, so it is usually placed close to the destination, or used where only "who" matters — VTY `access-class`, SNMP community restrictions, NAT and route filtering. An **extended** ACL (100–199, 2000–2699, or named extended) matches protocol, source, destination and ports, so it can be precise — and precise ACLs are where the order and port mistakes below happen.

Almost every ACL fault is one of six things:

Fault	Example	Proof
Order — a broader line above a specific one	`deny ip 10.1.0.0 0.0.255.255 any` above `permit tcp host 10.1.1.5 ...`	`show access-lists`: the specific line's counter stays 0
Implicit deny catches traffic you forgot	Routing hellos, DHCP, ICMP unreachable	Add `deny ip any any log` temporarily and watch the log
Wrong direction	Filtering `in` on the wrong interface	`show ip interface Gi0/1 | include access list`
Wrong wildcard	`10.1.1.0 0.0.0.255` written as `10.1.1.0 255.255.255.0`	`show access-lists` — IOS shows what it stored
Wrong protocol / port side	`eq 80` on the **source** port of a client request	Think through one packet's actual header
Outbound ACLs don't see router-originated traffic	A ping from the router passes an outbound ACL that denies it	Test from a host behind the router instead

Why do inbound ACLs so often break routing protocols? Because the implicit `deny ip any any` doesn't distinguish user traffic from control traffic. An inbound extended ACL on a WAN link that permits only "the business traffic" also drops **OSPF (IP protocol 89)**, **EIGRP (88)**, **BGP (TCP 179)**, **BFD (UDP 3784/3785)**, and HSRP hellos. The adjacency drops after the dead timer — and it looks like a routing problem. Every inbound ACL on a routed link should explicitly permit the control protocols that link carries.

Worked example — ticket: "OSPF dropped 40 seconds after the security team's change."

- **Symptom:** the log shows `%OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/0 from FULL to DOWN, Neighbor Down: Dead timer expired`.
- **Step 1:** `show ip interface Gi0/0 | include access list` → `Inbound access list is WAN-IN`.
- **Step 2:**

```
R1# show access-lists WAN-IN
Extended IP access list WAN-IN
    10 permit tcp any 10.1.0.0 0.0.255.255 eq 443 (5212 matches)
    20 permit udp any 10.1.0.0 0.0.255.255 eq domain (310 matches)
```

- **Reading it:** nothing permits OSPF (IP protocol 89, sent to 224.0.0.5). The neighbour's hellos hit the implicit deny, and after the 40-second dead interval the adjacency drops. The timing — "about 40 seconds after the change" — is the clue.
- **Fix:** `ip access-list extended WAN-IN` then `5 permit ospf any any`. The next hello is accepted and the adjacency returns to FULL.

Time-based ACLs

The problem it solves: some rules should only apply at certain times — guests may browse during office hours, a contractor may connect for two months. Instead of an engineer editing the ACL twice a day, a **time range** makes ACL entries active only at certain times:

```
time-range BUSINESS-HOURS
 periodic weekdays 8:00 to 18:00            ! recurring
time-range CONTRACTOR-ACCESS
 absolute start 00:00 01 November 2026 end 23:59 31 December 2026   ! one-off window
!
ip access-list extended GUEST-IN
 10 permit tcp 10.50.0.0 0.0.0.255 any eq 443 time-range BUSINESS-HOURS
```

```
R1# show time-range
time-range entry: BUSINESS-HOURS (inactive)
   periodic weekdays 8:00 to 18:00
   used in: IP ACL entry
R1# show access-lists GUEST-IN
Extended IP access list GUEST-IN
    10 permit tcp 10.50.0.0 0.0.0.255 any eq 443 time-range BUSINESS-HOURS (inactive)
```

Why is a time-based ACL really an NTP problem? Because the router judges "is it 9 a.m. on a weekday?" against **its own clock**. A router that rebooted without NTP may believe it's 00:00 on 1 March 1993 — every `periodic weekdays` entry is wrong, and every `absolute` window is either not yet started or long over. When a time-based ACL "randomly" blocks things, the first commands are `show clock` and `show ntp status` (NTP in Chapter 8). Remember too that `periodic` times are in the router's **configured time zone**.

Worked example — ticket: "Guests can't browse, and it's 10 a.m. on a Tuesday."

- `show access-lists GUEST-IN` → line 10 shows `time-range BUSINESS-HOURS (inactive)`. The router thinks it's outside business hours.
- `show clock` → `*00:41:12.003 UTC Mon Mar 1 1993`. The **asterisk** means the clock is not authoritative — it was never set or synchronised. The router rebooted after a power cut and lost its time.
- `show ntp associations` → no peers.
- **Cause:** the NTP server line was removed during a clean-up, so the router never set its clock after the reboot.
- **Fix:** restore `ntp server 10.1.1.1`. Once synchronised, `show time-range` reports BUSINESS-HOURS `(active)` and line 10 starts counting matches.

31.3 IPv6 Traffic Filters

The problem it solves: IPv6 traffic needs filtering just like IPv4 — an IPv4 ACL never even looks at an IPv6 packet, so a router with only IPv4 ACLs is wide open over IPv6. IPv6 ACLs are always **named**, use **prefix lengths** instead of wildcards, and are applied with **`ipv6 traffic-filter`** rather than `ip access-group`:

```
ipv6 access-list V6-EDGE-IN
 permit tcp any host 2001:DB8:10::80 eq 443
 permit icmp any any echo-request
 deny ipv6 any any log
!
interface GigabitEthernet0/0
 ipv6 traffic-filter V6-EDGE-IN in
```

The trap is at the bottom. Every IPv6 ACL ends with **three** implicit lines, not one:

```
 permit icmp any any nd-na         ! implicit
 permit icmp any any nd-ns         ! implicit
 deny ipv6 any any                 ! implicit
```

Why does IPv6 implicitly permit Neighbor Discovery? Because ND is IPv6's ARP (CCNA guide 13.8). In IPv4, ARP isn't IP, so an IP ACL can't block it. In IPv6, neighbour solicitations and advertisements **are** ICMPv6 packets, so a plain implicit deny would block address resolution and

every neighbour on the link would become unreachable. The implicit permits keep ND alive.

And why does the ACL above break the link anyway? Because it ends with an **explicit** `deny ipv6 any any log`. The explicit deny matches first, so the implicit ND permits underneath it are never reached. Neighbour entries go stale, then fail — and the interface "mysteriously" stops forwarding a few minutes after the ACL is applied.

How the delayed failure happens, step by step — why it isn't instant:

1. When the ACL goes on, R1 already has a **REACHABLE** neighbour cache entry for its next hop (learned before the ACL existed). Traffic keeps flowing.
2. After the reachable time (about 30 s by default) the entry becomes **STALE**. A stale entry is still used — traffic still flows.
3. The next time R1 sends to that neighbour, the entry moves to **DELAY**, and after about 5 s to **PROBE**: R1 sends unicast Neighbor Solicitations to check the neighbour is still there.
4. The neighbour answers with a Neighbor Advertisement — which arrives **inbound** and hits the explicit `deny ipv6 any any`. After the probes go unanswered, R1 deletes the entry.
5. Now R1 must resolve the neighbour from scratch, and the replies are blocked again. `show ipv6 neighbors` shows **INCMP** (incomplete), and traffic stops.

So the outage begins somewhere between seconds and minutes after the change, depending on when each neighbour entry happens to be re-checked — which is exactly why it looks unrelated to the ACL.

The fix: whenever you write your own final deny, add the ND permits **above** it yourself:

```
ipv6 access-list V6-EDGE-IN
 permit icmp any any nd-na
 permit icmp any any nd-ns
 permit icmp any any router-advertisement   ! if this side must learn a router via RA
 permit tcp any host 2001:DB8:10::80 eq 443
 deny ipv6 any any log
```

Note what the implicit lines **don't** include: router solicitations/advertisements. An inbound filter on a host-facing or ISP-facing interface that relies on SLAAC or RA-learned default routes must permit them explicitly.

Symptom	Likely cause	Command that proves it
Link works for minutes, then IPv6 neighbours fail	Explicit `deny ipv6 any any` blocking NS/NA	`show ipv6 access-list` (deny counter rising), `show ipv6 neighbors` (INCMP/STALE)
Router loses its IPv6 default route	RAs denied inbound	`show ipv6 route ::/0`, `show ipv6 routers`
OSPFv3 / EIGRP for IPv6 neighbour down	Link-local multicast routing traffic denied (OSPFv3 = IP proto 89 to FF02::5/6)	`show ipv6 access-list` counters
ACL applied but nothing filtered	Applied with `ip access-group` (wrong family) or wrong direction	`show ipv6 interface Gi0/0 | include access list`

31.4 Unicast Reverse Path Forwarding (uRPF)

The problem it solves: attackers **spoof** (fake) source addresses so their traffic can't be traced, and so replies (in reflection attacks) hit a victim. Writing ACLs that list every legitimate source on every interface doesn't scale. **uRPF** (Unicast Reverse Path Forwarding) uses the routing table you already have to drop packets whose **source** address doesn't make sense, by checking the source against the FIB. It needs **CEF** (on by default).

Mode	Command	A packet passes if…	Use where
Strict	`ip verify unicast source reachable-via rx`	The best route back to the source points **out the interface the packet arrived on**	Single-homed edges — customer-facing links, access layer
Loose	`ip verify unicast source reachable-via any`	**Any** route to the source exists (not via Null0)	Multihomed / asymmetric edges — internet links
VRF-aware	uRPF on an interface in a VRF checks **that VRF's** FIB; `ip verify unicast vrf NAME permit` (or `deny`) checks the source against another VRF	Provider edges, remote-triggered blackholing per VRF	

Story Time : Imagine a post office that receives mail from several regional sorting centres. A letter arrives on the truck from the **north** centre, but the return address says it's from a town in the **south**. Strict checking says: "if this letter were genuine, it would have come on the south truck — reject it." Loose checking only asks: "does that town exist at all?" — so it rejects letters from made-up towns but accepts a southern letter arriving on any truck. Where it breaks: in a real postal system, letters often take unusual routes for good reasons — and on the internet they do too (asymmetric routing), which is exactly when strict mode drops good traffic.

How the check works, step by step — R1 has strict uRPF on Gi0/1 (facing a branch whose LAN is 10.20.0.0/24):

1. A packet arrives on **Gi0/1** with source **10.20.0.5**.
2. R1 looks up the **source** (not the destination) in its FIB: best route to 10.20.0.5 is 10.20.0.0/24 via **Gi0/1**.
3. Strict mode: is that the interface the packet arrived on? **Yes → forward normally.**
4. Now a packet arrives on Gi0/1 claiming source **8.8.8.8**. The FIB's best route to 8.8.8.8 is the default route via **Gi0/0** (the internet link). Strict: Gi0/0 ≠ Gi0/1 → **drop**, and `verification drops` increases by one. A branch user can't pretend to be an internet host.
5. The same packet under **loose** mode: is there any route to 8.8.8.8? Only the default route — and without `allow-default`, a default route doesn't count → **drop**. A packet sourced from 10.99.99.99 (an address the router has no route for at all) would be dropped by loose mode too.

Options worth knowing:

- `allow-default` — let the default route satisfy the check. Without it, a source matched only by `0.0.0.0/0` **fails** (otherwise loose mode would pass everything on a router with a default route, making it useless).
- `allow-self-ping` — let the router ping itself (off by default).
- **An ACL number** at the end — `... reachable-via rx 120` — is consulted for packets that **fail** the check: `permit` forwards them anyway, `deny` drops them; with `log`, you see who's failing.

Why does strict mode break on asymmetric routing? Because it assumes the path to a source is the path from it. Picture a site dual-homed to two ISPs: traffic from 198.51.100.7 arrives on ISP-B's link, but your best route to 198.51.100.7 is via ISP-A. Strict uRPF on the ISP-B interface says "if this were genuine, it would have come in on ISP-A" — and drops perfectly legitimate traffic. That's why strict mode belongs on single-homed links and **loose** mode on multihomed ones.

```
R1# show ip interface Gi0/1 | include verify|drops
  IP verify source reachable-via RX, allow default
    173 verification drops
    0 suppressed verification drops
R1# show ip traffic | include RPF
        0 no route, 173 unicast RPF, 0 forced drop
```

Symptom	Likely cause	Command that proves it
Legitimate traffic from some sources dropped on a multihomed edge	**Strict** mode with asymmetric routing	`show ip interface ... verification drops`, `show ip route <source>` (best path is a different interface)
Loose mode drops nothing — even spoofed bogons	`allow-default` configured with a default route present	`show run interface`
Everything from a branch dropped after a routing change	Route to the branch's subnet moved to a different interface (strict)	`show ip cef <source>`
uRPF command accepted but no drops ever	CEF disabled	`show ip cef summary`

31.5 Control Plane Policing (CoPP) — Troubleshooting

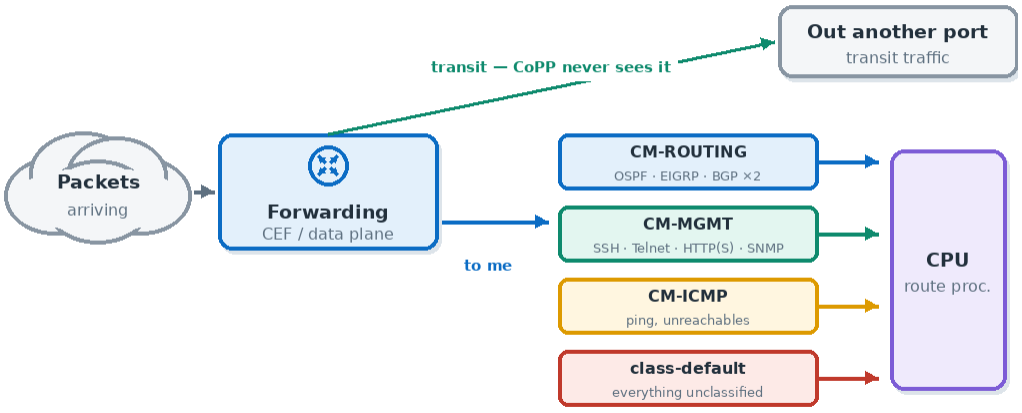
The problem it solves: a router's CPU (its **control plane**) is small compared with its forwarding hardware. Every routing hello, SSH keystroke, SNMP poll and ping to the router must be handled by that CPU. Flood it — by attack or by accident — and the CPU can't keep up: routing adjacencies time out and the network falls over. **CoPP** (Control Plane Policing) rations how much of each kind of traffic may reach the CPU.

Story Time : CoPP is the receptionist in front of a busy CEO. Calls from the board (routing protocols) are put straight through, generously. Calls from staff (SSH, SNMP from the management subnet) get a fair share. Cold callers (anything unrecognised) share one small line — if too many ring at once, the rest get a busy tone. The CEO stays able to work however many

people call. Note that the receptionist only handles calls **for the CEO**: traffic passing through the building (transit packets) never meets her.

Chapter 17 built CoPP: classify traffic **destined to the router itself** (management, routing, ICMP), police each class, attach the policy to `control-plane`. ENARSI lists the protocols explicitly — Telnet, SSH, HTTP(S), SNMP, EIGRP, OSPF, BGP — so you must be able to match each one correctly.

CoPP — only traffic addressed to the router meets the policers



first match wins, each class policed — unclassified traffic shares class-default

Protocol	Match	Easy mistake
Telnet	`tcp ... eq telnet` (23)	—
SSH	`tcp ... eq 22`	—
HTTP / HTTPS	`tcp ... eq www` / `eq 443`	Forgetting 443 when RESTCONF/GUI uses HTTPS
SNMP	`udp ... eq snmp` (161)	Using `tcp`, or matching 162 (traps go from the router)
EIGRP	`permit eigrp any any` (IP protocol **88**)	Matching a port — EIGRP has none
OSPF	`permit ospf any any` (IP protocol **89**)	Same
BGP	`tcp any any eq bgp` **and** `tcp any eq bgp any` (179)	Matching only one direction — see below

Why does BGP need two lines? A BGP session is one TCP connection; one side connects to port 179 from a random port, and the other side's packets come back from port 179 to that random port. If the neighbour opened the session, its packets arrive with **destination** 179. If you opened it, they arrive with **source** 179. Match only `eq bgp` as the destination and your router polices or drops half of all sessions — which half depends on who happened to connect first, so BGP fails "randomly" after a reset.

```
ip access-list extended COPP-ROUTING
 permit ospf any any
 permit eigrp any any
 permit tcp any any eq bgp
 permit tcp any eq bgp any
ip access-list extended COPP-MGMT
 permit tcp 10.99.0.0 0.0.0.255 any eq 22
 permit tcp 10.99.0.0 0.0.0.255 any eq telnet
 permit tcp 10.99.0.0 0.0.0.255 any eq www
 permit tcp 10.99.0.0 0.0.0.255 any eq 443
 permit udp 10.99.0.0 0.0.0.255 any eq snmp
```

Two rules for reading a CoPP ACL:

1. `deny` **in a classification ACL doesn't drop anything.** It means "this packet doesn't belong in this class" — the packet moves on to the next class. Only the **policer's actions** drop.
2. **Classes are checked top to bottom, first match wins**, and anything unmatched lands in `class-default`. That's where surprises live: NTP, BFD, HSRP/VRRP, DHCP relay replies, LDP, IP SLA responses — anything you didn't classify — share class-default's (usually small) rate.

```
R1# show policy-map control-plane
 Control Plane
  Service-policy input: PM-COPP
    Class-map: CM-MGMT (match-all)
      2380 packets, 301922 bytes
      5 minute offered rate 3000 bps, drop rate 0000 bps
      Match: access-group name COPP-MGMT
      police:
          cir 256000 bps, bc 8000 bytes
        conformed 2380 packets, 301922 bytes; actions: transmit
        exceeded 0 packets, 0 bytes; actions: drop
    Class-map: class-default (match-any)
      9113 packets, 812540 bytes
      5 minute offered rate 9000 bps, drop rate 2000 bps
        exceeded 1408 packets, 129536 bytes; actions: drop     ! something is being dropped
```

Symptom	Likely cause	Command that proves it
SSH to router slow or refused; works from some hosts	Source subnet not in the MGMT ACL → class-default policer drops	`show policy-map control-plane` (class-default exceeded rising)
BGP sessions fail after a reset, randomly	Only one BGP direction classified	`show access-lists COPP-ROUTING` (one line counting, other 0)
OSPF/EIGRP neighbours flap under load	Routing in a class with `exceed-action drop` and too low a rate	`exceeded` counter in that class
BFD / HSRP / NTP flapping	Not classified → starved in class-default	Add a class; watch class-default drops fall
SNMP polls time out, CPU is fine	SNMP matched as TCP, or policed hard	`show access-lists`, `show snmp` (packets input not rising)
Policy configured, nothing counted	`service-policy` not attached to `control-plane`, or attached `output`	`show run | section control-plane`

How one packet is handled, step by step — an SSH packet from 10.50.0.9 (the helpdesk subnet, not in COPP-MGMT) to R1:

1. The packet is addressed to R1 itself, so it's punted toward the CPU — and passes through PM-COPP on the way.
2. **CM-ROUTING:** does COPP-ROUTING match? It's TCP port 22, not OSPF, EIGRP or BGP → no.
3. **CM-MGMT:** COPP-MGMT only permits sources in 10.99.0.0/24 → no.
4. **CM-ICMP:** not ICMP → no.

5. **class-default:** everything left lands here. Its policer allows **64,000 bps**. A **policer** measures traffic against a rate (the **CIR**, committed information rate): traffic under the rate **conforms** and is transmitted; traffic over it **exceeds** and, here, is dropped.
6. One SSH session is a few kbps, so it conforms and works — until a busy minute when NTP, BFD, a backup script and this SSH session all share that 64 kbps together. Then the session freezes intermittently.

Worked example — ticket: "SSH from the helpdesk freezes; from the NOC it's fine." `show policy-map control-plane` shows class-default `exceeded` packets rising at the moment the helpdesk complains, while CM-MGMT shows none. The step-by-step above explains why: the helpdesk's 10.50.0.0/24 isn't in COPP-MGMT. **Fix:** add `permit tcp 10.50.0.0 0.0.0.255 any eq 22` to COPP-MGMT — if the helpdesk is meant to manage the router — and its SSH moves into the management class with its own 256 kbps.

> On **Catalyst 9000** switches CoPP isn't user-built in the same way: they ship with a fixed system policy (`system-cpp-policy`) whose class rates you tune. `show policy-map control-plane` still shows the counters.

31.6 IPv6 First-Hop Security

The problem it solves: IPv6 hosts trust what they hear on the local link: whoever sends a **Router Advertisement** (RA) becomes a default gateway; whoever answers a **Neighbor Solicitation** (NS) with a **Neighbor Advertisement** (NA) owns that address. Nothing in **NDP** (Neighbor Discovery Protocol) itself checks that the sender is telling the truth. **First-hop security** is a set of features on the access switch — the "first hop" a host's packets cross — that checks for it. ENARSI only asks you to **describe** the protections, so focus on matching each feature to the attack it stops.

Story Time : Think of an apartment building's front desk. **RA Guard** is the rule "only the building manager may post notices in the lobby" — a resident pinning up a fake "the exit is now through my flat" notice is torn down. **DHCPv6 Guard** is "only the manager hands out keys". The **binding table** is the desk's **resident register**: flat number, name, and which door they live behind. **ND inspection** checks every "I'm the person in flat 5" claim against the register. **Source Guard** checks every outgoing parcel: the sender's flat on the label must match the door it came out of.

Attack	How it works	Feature that stops it
Rogue RA	A host sends RAs: hosts adopt it as default gateway (man-in-the-middle), or a fake prefix, or router lifetime 0 (kills the real gateway)	**RA Guard**
Rogue DHCPv6 server	A host answers DHCPv6 with its own DNS server or addresses	**DHCPv6 Guard**
ND spoofing	A forged Neighbor Advertisement claims another host's IPv6 address — IPv6's ARP poisoning	**ND Inspection / snooping** (validated against the **binding table**)

Attack	How it works	Feature that stops it
Source address spoofing	A host sends traffic using an address it doesn't own	**IPv6 Source Guard** (uses the binding table)
DAD denial of service	A host answers every Duplicate Address Detection probe, so no one can claim an address	ND inspection with the binding table

How the pieces fit:

- **RA Guard** — on host-facing ports, **drop RAs and ICMPv6 redirects**. Only ports facing real routers (`device-role router`) may send them. The IPv6 version of "only trusted ports may speak for the gateway".
- **DHCPv6 Guard** — on host-facing ports (`device-role client`), drop DHCPv6 **server** messages (Advertise, Reply). The IPv6 equivalent of DHCP snooping's trusted/untrusted ports (CCNA guide 18.3).
- **Binding table** — the foundation. The switch **snoops** ND and DHCPv6 to build a table of **IPv6 address ↔ MAC ↔ port ↔ VLAN** — the IPv6 version of the DHCP snooping binding table. On IOS XE this is **SISF device-tracking** (`show device-tracking database`).
- **ND Inspection / ND snooping** — checks NS/NA messages against the binding table and drops ones that claim an address owned by someone else.
- **IPv6 Source Guard** — drops **data** traffic whose source address isn't in the binding table for that port — the IPv6 counterpart of IPv4 IP Source Guard.

Worked example — each feature on one access switch. VLAN 10, the real router on Gi1/0/48, hosts on Gi1/0/1–46. Host A (Gi1/0/5) has 2001:db8:10::5 with MAC 0050.5600.0005. The binding table, built by snooping Host A's own ND and DHCPv6 messages, holds:

IPv6 address	MAC	Port	VLAN	Learned from
2001:db8:10::5	0050.5600.0005	Gi1/0/5	10	NDP (Host A's DAD probe)

An attacker on **Gi1/0/9** (MAC 0050.5600.0009) tries four attacks:

Attack from Gi1/0/9	What the switch checks	Result
Sends an **RA** announcing itself as gateway	RA Guard: Gi1/0/9 has `device-role host` — hosts may not send RAs	**Dropped**; hosts keep the real router (Gi1/0/48) as gateway
Answers a **DHCPv6 Solicit** with its own DNS server	DHCPv6 Guard: Gi1/0/9 is `device-role client` — clients may not send Advertise/Reply	**Dropped**; only the real server's reply reaches the host
Sends an **NA**: "2001:db8:10::5 is at 0050.5600.0009"	ND inspection: the table says ::5 belongs to a different MAC on Gi1/0/5	**Dropped**; nobody's neighbour cache is poisoned
Sends **data** with source 2001:db8:10::5	Source Guard: ::5 is not bound to Gi1/0/9	**Dropped**; it can't impersonate Host A

The first two checks need only the port role; the last two need the binding table, which is why the device-tracking policy must be attached to the same ports.

What the policies look like (for recognition):

```
ipv6 nd raguard policy HOST-PORTS
 device-role host                      ! RAs from these ports are dropped
!
ipv6 dhcp guard policy DHCP-CLIENTS
 device-role client                    ! DHCPv6 server replies from these ports are dropped
!
device-tracking policy TRACK-HOSTS     ! builds the binding table (SISF)
 tracking enable
!
interface range GigabitEthernet1/0/1 - 46
 ipv6 nd raguard attach-policy HOST-PORTS
 ipv6 dhcp guard attach-policy DHCP-CLIENTS
 device-tracking attach-policy TRACK-HOSTS
```

ND inspection and Source Guard follow the same policy-and-attach pattern. On releases that still offer the classic commands:

```
ipv6 nd inspection policy ND-HOSTS
 device-role host                      ! validate NS/NA from these ports against the table
!
ipv6 source-guard policy SG-HOSTS     ! data packets must come from a bound address
!
interface range GigabitEthernet1/0/1 - 46
 ipv6 nd inspection attach-policy ND-HOSTS
 ipv6 source-guard attach-policy SG-HOSTS
```

Newer SISF-based IOS XE releases fold ND inspection into the device-tracking policy itself (for example `security-level guard`, which drops ND messages that conflict with the binding table) rather than a separate `ipv6 nd inspection` policy. Either way the dependency is the same: Source Guard and ND inspection are only as good as the binding table beneath them, so the device-tracking/snooping policy must be attached to the same ports.

Why does almost every feature depend on the binding table? Because "is this message lying?" can only be answered if the switch already knows the truth. RA Guard and DHCPv6 Guard are simple port-role rules; but deciding that an NA for 2001:db8::5 is forged, or that a data packet's source is spoofed, requires a record of which port actually owns 2001:db8::5. First build the truth (binding table), then police against it (ND inspection, source guard).

> **Exam trap:** RA Guard has a known weakness — an attacker can hide the RA behind **IPv6 extension headers** or fragments so a simple inspection never sees it. Modern implementations drop such packets; know that the evasion exists.

Chapter 32: Infrastructure Services: Management, SNMP, Logging, DHCP, IP SLA, NetFlow & Assurance

> **Blueprint:** ENARSI 4.1 — Troubleshoot device management: 4.1.a console and VTY, 4.1.b Telnet, HTTP, HTTPS, SSH, SCP, 4.1.c (T)FTP; ENARSI 4.2 — Troubleshoot SNMP (v2c, v3); ENARSI 4.3 — Troubleshoot network problems using logging (local, syslog, debugs, conditional debugs, timestamps); ENARSI 4.4 — Troubleshoot IPv4 and IPv6 DHCP (client, IOS DHCP server, relay, options); ENARSI 4.5 — Troubleshoot network performance issues using IP SLA (jitter, tracking objects, delay, connectivity); ENARSI 4.6 — Troubleshoot NetFlow (v5, v9, Flexible NetFlow); ENARSI 4.7 — Troubleshoot network problems using Cisco DNA Center (Catalyst Center) Assurance.

This is the **25%** domain of ENARSI — a quarter of the exam — and it's made of services you've met before: SSH (CCNA), SNMP, syslog, DHCP (CCNA Ch 17), NetFlow and IP SLA (Chapter 15), Catalyst Center (Chapter 16). What's new is the angle. ENARSI hands you a broken service and asks **why**. So each section here is short on "how to build it" (you have that) and long on "how it breaks and how you prove it".

32.1 Device Management Access

The problem it solves: you can only fix a router you can reach. Management access is the set of doors into the device — the console port, remote sessions, the web interface, file transfers — and each door has its own lock. When "I can't get in" is the ticket, you check which door and which lock.

Console and VTY lines

Every way into a router lands on a **line**: the console (`line con 0`) or a virtual terminal (`line vty 0 15` on most IOS XE platforms; `0 4` on older ones). Each line has its own login method, timeout, allowed protocols and access list.

```
aaa new-model
username admin privilege 15 secret 9 $9$...     ! local fallback account
aaa authentication login CONSOLE local          ! console: local only, works with servers down
aaa authentication login VTY-AUTH group TACACS-GRP local    ! VTYs: as in 31.1
!
line con 0
 exec-timeout 10 0
 logging synchronous                    ! log messages don't overwrite what you're typing
 login authentication CONSOLE           ! the local-only list defined above
line vty 0 15
 exec-timeout 15 0
 access-class MGMT-HOSTS in             ! who may connect TO the router
 transport input ssh                    ! which protocols may connect
 login authentication VTY-AUTH
```

Symptom	Likely cause	Command that proves it
`% Connection refused by remote host`	Protocol not in `transport input` (e.g. Telnet when only SSH allowed)	`show run | section line vty`
Connection refused only from some hosts	`access-class` doesn't permit that source	`show access-lists MGMT-HOSTS` (deny counter rises)
Connects, then immediately closed: "Password required, but none set"	`login` with no `password` on the line (no AAA)	`show run | section line vty`
Worked, now new sessions are refused	All VTY lines in use (stuck sessions)	`show users`, then `clear line vty N`
Sessions drop after idle minutes	`exec-timeout` (default 10 min)	`show line vty 0`

> **Exam trap:** `access-class` on a VTY line filters connections **to** the router (`in`) — or, with `out`, connections the router initiates from that line. It's not `access-group`, and applying a standard ACL as `ip access-group` on an interface to "protect SSH" also filters all transit traffic.

Telnet versus SSH

Telnet (TCP 23) sends everything — including passwords — in clear text. SSH (TCP 22) encrypts the session. SSH needs four things in place, and a missing one is the usual fault:

```
hostname R1                              ! 1. a hostname other than "Router"
ip domain name example.com               ! 2. a domain name (the RSA key is named host.domain)
crypto key generate rsa modulus 2048     ! 3. an RSA key pair
ip ssh version 2
username admin privilege 15 secret ... ! 4. something to authenticate against
line vty 0 15
 transport input ssh
 login local                             ! or an AAA list
```

```
R1# show ip ssh
SSH Enabled - version 2.0
Authentication methods:publickey,keyboard-interactive,password
Authentication timeout: 120 secs; Authentication retries: 3
```

Why does SSH need a domain name? Because IOS names the RSA key pair after the router's fully qualified name — `R1.example.com`. Without a hostname and domain, `crypto key generate rsa` refuses to run, and without a key SSH is simply disabled (`show ip ssh` → "SSH Disabled"). Key size matters too: SSH version 2 needs a modulus of at least **768** bits, and modern clients reject anything under **2048**.

Symptom	Likely cause	Proof
SSH refused; Telnet works	No RSA key (missing hostname/domain), or `transport input telnet` only	`show ip ssh`, `show crypto key mypubkey rsa`

Symptom	Likely cause	Proof
Client: "no matching key exchange / host key"	Key too small or old algorithms	`show ip ssh`; regenerate with `modulus 2048` or larger
Prompts for password, always rejected	Authentication method/list issue	`debug ip ssh`, `debug aaa authentication`

Worked example — ticket: "SSH to the new branch router is refused; console works."

- **Step 1:** from the console, `show ip ssh` → `SSH Disabled - version 1.99` and `%Please create RSA keys to enable SSH`.
- **Step 2:** `show run | include hostname|domain` → `hostname Router`, no `ip domain name`.
- **Cause:** the router was never given a hostname and domain, so no RSA key was ever generated, so SSH never started.
- **Fix:** `hostname BR12-RTR`, `ip domain name example.com`, `crypto key generate rsa modulus 2048`, `ip ssh version 2`. `show ip ssh` now says `SSH Enabled - version 2.0`, and the SSH login succeeds.

HTTP and HTTPS

The web server serves the device GUI (WebUI) and **RESTCONF** (Chapter 20).

```
no ip http server                       ! disable clear-text HTTP
ip http secure-server                   ! HTTPS (TCP 443) — needed by RESTCONF
ip http authentication local            ! or: aaa
ip http access-class ipv4 MGMT-HOSTS    ! restrict who may connect
```

Common faults: HTTPS server not enabled (RESTCONF fails to connect); `ip http authentication` left at the default **enable** method so users are prompted for the enable password; users below privilege 15 getting read-only or refused; the access-class excluding the client.

SCP, TFTP and FTP — moving files

Protocol	Transport	Authentication	Encrypted	Typical failure
TFTP	**UDP 69**	**None**	No	Wrong source IP; server directory/permissions; file size (older servers cap at **32 MB**)
FTP	**TCP 21** (+ data channel)	Username/password	No	Credentials; active/passive data channel blocked by a firewall
SCP	**TCP 22** (runs over SSH)	SSH user	**Yes**	SCP server not enabled; user lacks exec authorization at privilege 15

```
ip tftp source-interface Loopback0      ! the IP the TFTP server sees
ip ftp source-interface Loopback0
ip ftp username backup
ip ftp password 7 ...
!
R1# copy running-config tftp://10.1.1.50/R1-confg
R1# copy flash:c8000v-universalk9.17.12.04.SPA.bin ftp://10.1.1.50/images/
```

SCP server on the router (so a PC can push or pull files with `scp`):

```
aaa new-model
aaa authentication login default local
aaa authorization exec default local    ! SCP needs the user to land at privilege 15
ip scp server enable
```

Why does SCP fail with "Privilege denied" when SSH logins work? Because an interactive SSH user can type `enable`, but an SCP transfer can't — it needs the session to start at privilege 15. That only happens when **exec authorization** assigns the privilege level from the user's account. No `aaa authorization exec` → the transfer starts at level 1 → refused.

Why does `ip tftp source-interface` fix "timed out" so often? A router with several interfaces sources the transfer from whichever interface faces the server. If the server only allows (or has a firewall rule for) the management loopback, or there's no return route to the egress interface's subnet, the request leaves but the reply never arrives. Pinning the source makes the device's identity predictable — the same reason we pin TACACS+, SNMP and syslog sources.

32.2 SNMP v2c and v3

Chapter 15 covered SNMP's model (manager, agent, MIB, OIDs; polls on **UDP 161**, traps/informs to **UDP 162**). A quick reminder of the words: **SNMP** is the Simple Network Management Protocol; the **NMS** (network management system) is the manager software; the **agent** runs on the router; the **MIB** (Management Information Base) is the catalogue of everything the agent can report; and each item has an **OID** (object identifier), a dotted number like 1.3.6.1.2.1.2.2.1.10.

Story Time : Polling is the NMS **phoning** the router every five minutes: "how many bytes on interface 1?" A trap is the router **sending a text message** the moment something happens: "interface 1 just went down." Phone calls get an answer or you know they failed; a text message might never arrive and nobody notices — that's the trap-versus-inform difference later in this section.

How one SNMPv2c poll works, step by step:

1. The NMS (10.1.1.60) sends a **GetRequest** to R1 on **UDP 161**, carrying the community string `N3tM0n` and the OID **1.3.6.1.2.1.2.2.1.10.1** (ifInOctets — bytes received — for ifIndex 1).
2. R1 checks the community string. Unknown community → the request is **silently dropped** and the "Unknown community name" counter increases. No error is sent back, so the NMS just sees a timeout.
3. R1 checks ACL 20 attached to the community. 10.1.1.60 is permitted.
4. R1 checks that the community's view (all OIDs, by default) includes this OID, and that the operation is allowed (`RO` allows Get, not Set).

5. R1 sends a **GetResponse** back to the NMS: `ifInOctets.1 = 3482911204`. The NMS polls again five minutes later, subtracts, and draws the traffic graph.

Here's what breaks.

SNMPv2c

```
access-list 20 permit 10.1.1.60                  ! only the NMS
snmp-server community N3tM0n RO 20               ! community + permission + ACL
snmp-server location DC1-Row4
snmp-server host 10.1.1.60 version 2c N3tM0n     ! where traps go
snmp-server enable traps
snmp-server trap-source Loopback0
snmp-server ifindex persist                      ! keep ifIndex numbers stable across reloads
```

Why `ifindex persist`? The NMS stores interface statistics by **ifIndex**, an SNMP number for each interface. Without persistence, adding a module or reloading can renumber interfaces, and the NMS silently starts graphing the wrong interface under the old name. Graphs that "jumped" after a reload are an ifIndex problem, not a traffic problem.

The community is a **password sent in clear text**, which is why v2c should be read-only and ACL-restricted.

SNMPv3

The problem it solves: v2c's community string crosses the network in clear text, so anyone capturing one packet can read it — and with an RW community, change the router's configuration. v3 adds **authentication** (HMAC — SHA/MD5: proof the message came from someone who knows the password and wasn't altered) and **privacy** (encryption — AES/DES: nobody else can read it), through a hierarchy of **view → group → user**. Read the hierarchy as: a **view** says which OIDs, a **group** says what security level is required and which views it may read or write, and a **user** belongs to a group and holds the passwords.

```
snmp-server view FULL iso included                          ! which OIDs are visible
snmp-server group NMS-GRP v3 priv read FULL write FULL access 20
snmp-server user nmsuser NMS-GRP v3 auth sha AuthPass123 priv aes 128 PrivPass123
snmp-server host 10.1.1.60 version 3 priv nmsuser
```

Security level	Keyword	Authentication	Encryption
noAuthNoPriv	`noauth`	Username only	No
authNoPriv	`auth`	Yes	No
authPriv	`priv`	Yes	**Yes**

Why aren't SNMPv3 users shown in the running configuration? Because the user's passwords are converted into **localized keys** — hashed together with the router's **SNMP engine ID** — and only the keys are stored. That has two consequences you'll be tested on: `show running-config` won't show `snmp-server user` lines (use `show snmp user`), and **changing the engine ID invalidates every user** — they must be re-created. The engine ID defaults to one derived from a MAC address, so it can change if you replace hardware or set `snmp-server engineID local` later.

Symptom	Likely cause	Command that proves it
v2c polls time out	Wrong community (the agent **silently ignores** it) or ACL denies the NMS	`show snmp` → "Unknown community name" / "Illegal operation" counters rise
Reads work, writes fail	Community is `RO`, or the v3 group has no `write` view	`show snmp community`, `show snmp group`
Some OIDs return noSuchObject	The view excludes them	`show snmp view`
v3: "authorizationError" or unknown user	User's security level higher than the group's, or user created under a different group	`show snmp user`, `show snmp group`
v3: "wrong digests" / decryption errors	Auth or priv password/algorithm mismatch with the NMS	`show snmp` (wrong digests counter), `debug snmp packets`
v3 stopped after an engine ID change	Localized keys invalid	`show snmp engineID`; re-create users
Traps never arrive	No `snmp-server host`, traps not enabled, wrong trap source, UDP 162 blocked	`show snmp host`, `debug snmp packets`

Worked example — ticket: "The NMS shows R1 as unreachable, but I can ping it."

```
R1# show snmp
Chassis: FGL2231A0BC
412 SNMP packets input
    0 Bad SNMP version errors
    412 Unknown community name
    0 Illegal operation for community name supplied
    0 Encoding errors
    0 Number of requested variables
```

- **Reading it:** every one of the 412 packets that arrived used a community R1 doesn't know. The NMS is reaching R1 (so routing and UDP 161 are fine) — it's sending the wrong password, and R1 is silently ignoring it.
- **Cause:** the community was changed on the routers last week but not in the NMS's device profile.
- **Fix:** update the NMS. On the next poll, `Unknown community name` stops rising and `Number of requested variables` starts counting.
- **Contrast:** if `SNMP packets input` were **0**, the requests aren't arriving at all — look at routing, firewalls and UDP 161 instead. If the community were right but the NMS's address missing from ACL 20, the packets would also be dropped silently and you'd confirm it with `show access-lists 20`.

Traps or informs? A **trap** is fire-and-forget over UDP — if it's lost, nobody knows. An **inform** is acknowledged and retransmitted. When an NMS is missing events from a lossy site, informs are the fix.

SNMPv3 informs need two extra lines, and their absence is a classic "informs never arrive" fault. For an inform, the **receiver** (the NMS) is the authoritative SNMP engine, so the router must localize the user's keys with the **NMS's** engine ID, not its own:

```
snmp-server engineID remote 10.1.1.60 800000090300AABBCCDDEEFF   ! the NMS's engine ID
snmp-server user nmsuser NMS-GRP remote 10.1.1.60 v3 auth sha AuthPass1 priv aes 128 PrivPass1
snmp-server host 10.1.1.60 informs version 3 priv nmsuser
snmp-server enable traps
```

Configure the remote engine ID **before** the remote user (the keys are computed from it when the user is created). `show snmp engineID` lists both the local and the remote engine IDs; `show snmp user` shows the remote user against that engine. Traps don't need any of this — for a trap the router itself is authoritative, which is why v3 traps work with only the local user.

32.3 Logging

The problem it solves: a router notices hundreds of events — links flapping, neighbours dropping, logins failing — and you need a record of them to work out what happened, especially after the fact. Logging decides **which** events are kept (by severity) and **where** they go (screen, memory, a central server).

Reading a log message

```
000123: Oct 26 09:14:02.331 EST: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet0/1, changed state to down
```

Part	Example	Meaning
Sequence number	`000123`	Message number (from `service sequence-numbers`) — gaps on the server mean lost messages
Timestamp	`Oct 26 09:14:02.331 EST`	When it happened, to the millisecond, in local time with the zone shown (the `service timestamps log datetime msec localtime show-timezone` setting in this section)
Facility	`LINEPROTO`	Which part of IOS raised it
Severity	`5`	How serious (table below) — 5 = notification
Mnemonic	`UPDOWN`	A short code for the event type
Text	`Line protocol on ... down`	The human-readable detail

A leading **asterisk** on the timestamp (`*Oct 26 ...`) means the clock is **not authoritative** — not synchronised with NTP — so don't trust that time when correlating with other devices. A leading **dot** means the clock is considered authoritative (it was set or synchronised earlier) but is **not currently synchronised** with NTP — for example, it has lost contact with its NTP server — so treat it with some caution too.

Severity and destinations

Memory trick: "**E**very **A**wesome **C**isco **E**ngineer **W**ill **N**eed **I**ce cream **D**aily" — Emergencies, Alerts, Critical, Errors, Warnings, Notifications, Informational, Debugging, levels 0 to 7. Lower number = more serious.

Level	Keyword	Level	Keyword
0	emergencies	4	warnings
1	alerts	5	notifications
2	critical	6	informational
3	errors	7	debugging

Setting a level means "**this level and everything more severe**" — `logging trap warnings` sends 0–4.

Destination	Command	Default	Notes
Console	`logging console <level>`	debugging	Slow — a debug flood here can hang the router
VTY (monitor)	`logging monitor <level>`	debugging	Shown only after `terminal monitor` in **that** session
Local buffer	`logging buffered <size> <level>`	on, in RAM	`show logging`; lost at reload
Syslog server	`logging host 10.1.1.70` + `logging trap <level>`	trap level **informational**	UDP 514; set `logging source-interface`

Why is "I enabled debug but see nothing" so common? Three independent switches must line up: the debug must be on; the destination you're looking at must have a level of **debugging (7)**; and on a VTY, `terminal monitor` must be enabled in your session. Debug output is severity 7, so a buffer set to `informational` silently discards it.

Timestamps — making logs useful

```
service timestamps log datetime msec localtime show-timezone
service timestamps debug datetime msec localtime show-timezone
service sequence-numbers
clock timezone EST -5 0
ntp server 10.1.1.1
```

Why do timestamps decide whether logs are evidence or noise? Troubleshooting is correlation: "the BGP session dropped at 14:02:17.331 — what did the firewall log at the same instant?" That requires every device's clock to agree (**NTP**, Chapter 8) and to log with **datetime and milliseconds**. The default `uptime` format ("3w2d") can't be lined up against anything. Sequence numbers reveal whether messages were lost in transit to the syslog server.

Conditional debugging

A plain `debug ip packet` on a busy router is dangerous: it tries to print every packet. Narrow the output before you enable it:

```
R1# debug condition interface GigabitEthernet0/1     ! only events tied to this interface
R1# debug ip packet 120 detail                       ! only packets matching ACL 120
R1# show debug condition
R1# undebug all                                      ! ALWAYS have this ready
R1# no debug condition all                           ! conditions survive undebug all
```

Conditions filter debugs that support them (most interface- and protocol-related debugs do); an ACL attached to `debug ip packet` restricts which packets are shown. Note that `debug ip packet` only sees **process-switched** traffic — CEF-switched transit packets never appear. Conditions are **separate from the debugs**: `undebug all` turns off the debugs but leaves the conditions configured, so a later debug may show far less than you expect. Clear them with `no debug condition all` when you're done (or remove one with `no debug condition <id>`). Debug the smallest thing you can, to the buffer, with timestamps — and know your undo commands before you press Enter.

Worked example — debugging one user's traffic on a busy router. Users say pings from host 10.1.20.55 to the server 10.1.1.10 fail; everyone else is fine. The router forwards 2 Gbps, so an unfiltered packet debug is out of the question.

```
R1(config)# access-list 120 permit icmp host 10.1.20.55 host 10.1.1.10
R1(config)# access-list 120 permit icmp host 10.1.1.10 host 10.1.20.55
R1(config)# logging buffered 64000 debugging       ! buffer keeps level 7
R1(config)# no logging console                     ! don't print to the slow console
R1(config)# end
R1# debug ip packet 120 detail
R1# show logging | include 10.1.20.55
```

Only the two hosts' ICMP packets are reported, into the buffer, with millisecond timestamps. One caution: on a CEF-switched router, transit packets may not appear at all (only process-switched ones do) — if the debug stays empty while pings are running, that's why, not proof that the packets never arrived. Tools such as Embedded Packet Capture or an ACL with `log` on the interface see switched traffic. Finish with `undebug all` and `no debug condition all`.

Symptom	Likely cause	Command that proves it
Syslog server receives nothing	No route/ACL, wrong `logging host`, UDP 514 blocked, trap level too low	`show logging` (host line: "xx message lines logged"), `ping` from the source interface
Server receives logs from an "unknown" IP	Source not pinned	Set `logging source-interface Loopback0`
Timestamps don't match other devices	No NTP / wrong time zone / uptime format	`show clock detail`, `show ntp status`
Debug output missing	Buffer level < 7, or no `terminal monitor`	`show logging` (levels per destination)
Router sluggish after debugging	Console logging of a high-volume debug	`no logging console`; `undebug all`

32.4 DHCP for IPv4

The problem it solves: DHCP (Dynamic Host Configuration Protocol) gives every client its IP address, mask, gateway and DNS server automatically. Its weak point is that a new client has no address yet, so it can only **broadcast** — and routers don't forward broadcasts. A **relay agent** (configured with `ip helper-address`) carries those broadcasts to a central server on another subnet.

The IOS DHCP server and relay

```
ip dhcp excluded-address 10.1.10.1 10.1.10.20   ! never hand these out
ip dhcp pool VLAN10
 network 10.1.10.0 255.255.255.0
 default-router 10.1.10.1
 dns-server 10.1.1.53
 domain-name example.com
 option 150 ip 10.1.1.80                         ! Cisco IP phones: TFTP server(s)
 lease 2                                         ! days
!
interface Vlan20                                 ! on the L3 switch where clients live
 ip address 10.1.20.1 255.255.255.0
 ip helper-address 10.1.1.10                     ! relay DHCP to the central server
```

How does a central server know which pool to use for a relayed request? The relay agent writes the IP address of the interface the request **arrived on** into the **giaddr** field before unicasting it to the server. The server picks the pool whose `network` contains giaddr, and sends the reply back to the relay. That's why a missing pool for 10.1.20.0/24 on the server — or a relay interface with the wrong IP — means no address, even though the relay "works".

Story Time : A relay agent is the mailroom on your office floor. You (the client) shout "I need an address!" into the corridor. The floor mailroom hears you, writes your request on a form, adds "**reply to: Floor 20 mailroom**" (that's **giaddr**), and posts it to head office (the DHCP server). Head office reads "Floor 20", picks an address from Floor 20's range, and posts the answer back to the Floor 20 mailroom, which hands it to you. Head office never needed to hear you shout.

How a relayed DHCP exchange works, step by step (client in VLAN 20; relay SVI 10.1.20.1 with `ip helper-address 10.1.1.10`; server 10.1.1.10):

1. **Discover.** The client broadcasts from 0.0.0.0 to 255.255.255.255 (UDP 68 → 67).
2. **Relay.** The SVI receives the broadcast, sets **giaddr = 10.1.20.1** (its own address on the client's subnet), and **unicasts** the Discover to 10.1.1.10, sourced from its own address.
3. **Offer.** The server matches giaddr 10.1.20.1 to a pool whose `network` contains it (10.1.20.0/24), checks (by pinging) that the first free address — say 10.1.20.21 — isn't already in use, and unicasts an Offer **back to giaddr** 10.1.20.1.
4. **Relay back.** The SVI passes the Offer to the client on VLAN 20 (broadcast or unicast, depending on the client's broadcast flag).
5. **Request and Ack** follow the same path. The client now has 10.1.20.21/24, gateway 10.1.20.1, and the DNS server from the pool.

Every failure in the table below breaks one numbered step: no helper (step 2), no route between relay and server (steps 2–3), no matching pool (step 3).

`ip helper-address` relays more than DHCP: by default it also forwards TFTP (69), DNS (53), Time (37), TACACS (49), NetBIOS (137/138) and IEN-116 name service (42). Use `no ip forward-protocol udp <port>` to stop the ones you don't want.

Option	Carries	Typical use
3	Default router	`default-router`
6	DNS servers	`dns-server`

Option	Carries	Typical use
43	Vendor-specific information	APs finding their **WLC** (sub-option 0xF1 + controller IPs, in hex)
66	TFTP server **name**	Phones/devices that take a hostname
150	TFTP server **IP list** (Cisco)	Cisco IP phones fetching config
82	Relay Agent Information	Inserted by relays/snooping switches: which switch and port the client is on

How option 82 is added — and how it breaks a relay, step by step. An access switch SW1 runs DHCP snooping; the client is on Gi1/0/5, VLAN 20. The router above it is the relay.

1. The client broadcasts a Discover.
2. SW1's DHCP snooping inserts **option 82** by default: a **circuit ID** (VLAN 20, port Gi1/0/5) and a **remote ID** (SW1's MAC address). This tells the server exactly where the client is plugged in — useful for per-port address policy.
3. SW1 is a Layer 2 switch, not a relay, so it leaves **giaddr = 0.0.0.0** and floods the Discover up the VLAN.
4. The router's SVI receives a packet that **has option 82 but giaddr 0**. RFC-wise that's inconsistent (option 82 is meant to be added by a relay, which would also set giaddr), so on an untrusted interface IOS **drops it**. `debug ip dhcp server packet` shows:

```
DHCPD: inconsistent relay information.
DHCPD: relay information option exists, but giaddr is zero.
```

5. The client never gets an Offer and falls back to 169.254.x.x.

Fixes, pick one: on the router, trust that interface (`ip dhcp relay information trusted` on the SVI) or all interfaces (`ip dhcp relay information trust-all`); or on SW1, stop inserting the option (`no ip dhcp snooping information option`).

DHCP client

```
interface GigabitEthernet0/0
 ip address dhcp                        ! e.g. an internet-facing link
R1# show dhcp lease
```

A DHCP-learned default route is installed with **AD 254**, so any static or dynamic default wins over it.

What breaks

Symptom	Likely cause	Command that proves it
Clients get 169.254.x.x (APIPA)	No server reachable: missing `ip helper-address`, server not reachable from relay	`debug ip dhcp server packet` on server (nothing arrives)
Clients get APIPA, relay seems configured	`ip helper-address` on the **wrong interface** — it belongs on the **client-facing** interface/SVI (where the broadcast arrives), not the uplink toward the server	`show ip interface Vlan10 | include Helper`
Relay forwards, server never answers	**No pool matching giaddr**	`show ip dhcp pool`, `debug ip dhcp server events`
Server ignores relayed requests from a switch running DHCP snooping	Switch inserted **option 82 with giaddr 0**; IOS server drops it as untrusted	`debug ip dhcp server packet`; fix with `no ip dhcp snooping information option` on the switch or `ip dhcp relay information trust-all` on the relay/server
Pool exhausted / addresses skipped	Addresses found in use by **conflict detection** (server pings before offering)	`show ip dhcp conflict`, `show ip dhcp pool` (leased vs total)
Clients get an address that conflicts with a static device	Static range not in `excluded-address`	`show ip dhcp binding`
Phones/APs don't find TFTP/WLC	Option 150/43 missing or malformed	`show run | section dhcp pool`
Nothing works at all	`no service dhcp` — disables the server and relay	`show run | include service dhcp`

32.5 DHCP for IPv6

The problem it solves: IPv6 hosts can build their own addresses (**SLAAC**, Stateless Address Autoconfiguration), so DHCP is optional in IPv6 — but organisations often still want to control and record addresses, or hand out DNS servers. IPv6 splits address assignment between **Router Advertisements** and **DHCPv6**, controlled by flags in the RA: the **M** (Managed) flag and the **O** (Other configuration) flag.

RA flags	Model	Host gets address from	Host gets DNS etc. from
M=0, O=0	**SLAAC** only	SLAAC	RDNSS in the RA (if supported)
M=0, **O=1**	**Stateless DHCPv6**	SLAAC	DHCPv6
M=1	**Stateful DHCPv6**	DHCPv6	DHCPv6

The one thing DHCPv6 never provides is a default gateway. Hosts always learn their default router from RAs. A host with a perfect DHCPv6 address and no RA has no way off the link.

```
ipv6 dhcp pool V6-VLAN10
 address prefix 2001:DB8:10::/64 lifetime infinite infinite    ! stateful only
 dns-server 2001:DB8:1::53
 domain-name example.com
!
interface Vlan10
 ipv6 address 2001:DB8:10::1/64
 ipv6 dhcp server V6-VLAN10
 ipv6 nd managed-config-flag             ! M=1: use DHCPv6 for addresses
 ipv6 nd other-config-flag               ! O=1: and for other information
 ipv6 nd prefix 2001:DB8:10::/64 2592000 604800 no-autoconfig   ! stop SLAAC too
```

On a relay (the server is elsewhere):

```
interface Vlan20
 ipv6 dhcp relay destination 2001:DB8:1::10
```

DHCPv6 runs on **UDP 546 (client) / 547 (server/relay)**; clients solicit to the multicast address **FF02::1:2** (All DHCP Relay Agents and Servers) — there's no broadcast in IPv6. A router can be a client with `ipv6 address dhcp`.

How a relayed stateful DHCPv6 exchange works, step by step (client in VLAN 20; relay interface 2001:DB8:20::1; server 2001:DB8:1::10):

1. The client receives an **RA** with **M=1**: "get your address from DHCPv6." The RA also gives it its default gateway (the router's link-local address).
2. The client sends a **Solicit** from its link-local address to **FF02::1:2**.
3. The relay wraps the Solicit in a **Relay-Forward** message, writes its own address on the client's link (2001:DB8:20::1) into the **link-address** field — the IPv6 equivalent of giaddr — and unicasts it to 2001:DB8:1::10.
4. The server picks the pool matching that link address and returns an **Advertise** inside a **Relay-Reply**; the relay unwraps it and passes it to the client.
5. **Request** and **Reply** complete the exchange. The client now holds, for example, 2001:DB8:20::A1B2/128 plus the DNS server — and still uses the RA-learned router as its gateway.

(DHCPv4 has four messages, Discover–Offer–Request–Ack; DHCPv6's equivalents are **Solicit–Advertise–Request–Reply**.)

Symptom	Likely cause	Command that proves it
Hosts use SLAAC and ignore DHCPv6	M flag not set	`show ipv6 interface Vlan10` ("hosts use DHCP to obtain routable addresses")
Hosts get **two** addresses (SLAAC + DHCPv6)	M=1 but the prefix still has autonomous (A) flag set	Add `no-autoconfig` to `ipv6 nd prefix`
DHCPv6 address but no off-link connectivity	RAs suppressed (`ipv6 nd ra suppress`) or filtered — no default route	`show ipv6 routers` on a client router; check RA Guard (31.6)

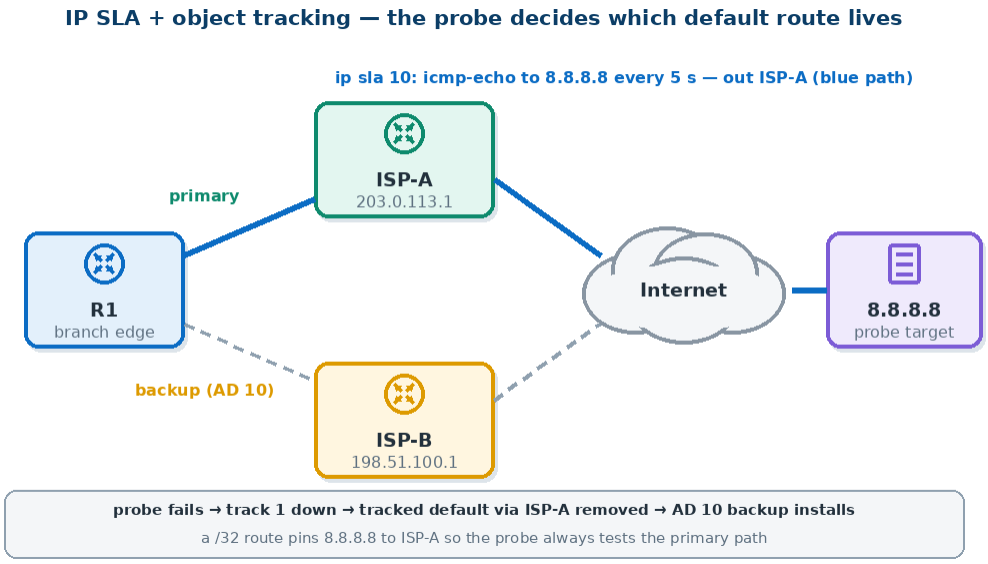
Symptom	Likely cause	Command that proves it
Relayed clients get nothing	Missing `ipv6 dhcp relay destination`, or server has no route back to the relay's link	`debug ipv6 dhcp`, `show ipv6 dhcp interface`
Stateless clients have no DNS	O flag not set or pool has no `dns-server`	`show ipv6 dhcp pool`

32.6 IP SLA and Object Tracking

The problem it solves: a router only knows its own links are up. It can't tell that the ISP's network behind the link has failed, or that the WAN has become too slow for voice. **IP SLA** (IP Service Level Agreement) makes the router **generate synthetic traffic** and measure the result (Chapter 15 introduced it). ENARSI uses it for two jobs: measuring performance (**delay, jitter, loss**) and testing **connectivity** to drive failover through **object tracking**.

Connectivity and failover

```
ip sla 10
 icmp-echo 8.8.8.8 source-interface GigabitEthernet0/0   ! probe *through* ISP-A
 threshold 300
 timeout 1000
 frequency 5
ip sla schedule 10 life forever start-time now
!
track 1 ip sla 10 reachability
 delay down 10 up 30                                     ! dampen flaps
!
ip route 0.0.0.0 0.0.0.0 203.0.113.1 track 1             ! primary: only while track 1 is up
ip route 0.0.0.0 0.0.0.0 198.51.100.1 10                 ! floating backup (AD 10)
ip route 8.8.8.8 255.255.255.255 203.0.113.1             ! pin the probe to ISP-A
```



Why the extra host route to 8.8.8.8? Without it, when ISP-A fails, the backup default takes over — and the probe itself now follows the backup route through ISP-B, succeeds, brings track 1 back up, which reinstalls the primary route, which fails the probe, which… The route flaps forever.

Pinning the probe target to ISP-A means the probe always tests the **primary** path, whatever the routing table currently prefers. A probe must test the path you're deciding about, not whichever path happens to be in use.

Hardening the pinned route. `ip route 8.8.8.8 255.255.255.255 203.0.113.1` has its own weakness: if ISP-A's interface goes **down**, the next hop becomes unreachable, the host route is removed, and the probe falls back to the default — through ISP-B. Two common fixes keep the probe on ISP-A no matter what:

```
ip route 8.8.8.8 255.255.255.255 GigabitEthernet0/0 203.0.113.1   ! interface-qualified
ip route 8.8.8.8 255.255.255.255 Null0 250                        ! floating discard
```

The interface-qualified route ties the probe to Gi0/0. The floating Null0 route (AD 250) takes over when the primary host route disappears, so the probe is **dropped** instead of leaking out through ISP-B. A dropped probe correctly reports ISP-A as down. The probe must fail when the path fails; it must never succeed by taking a different path.

How a failover plays out, second by second (the configuration above: probe every 5 s, 1000 ms timeout, `delay down 10 up 30`):

1. **10:00:00** — ISP-A's upstream fails. Gi0/0 stays up (the local link is fine), so nothing else in the router notices.
2. **≈10:00:05** — the next probe to 8.8.8.8 gets no reply within 1000 ms. Return code: **Timeout**. The track would now go down — but `delay down 10` makes it wait in case this is a blip.
3. **≈10:00:15** — probes are still failing after 10 s, so **track 1 goes Down**. The static route tied to track 1 is removed from the routing table.
4. Immediately, the floating default (`198.51.100.1`, AD 10) is the best default and is installed. Traffic now leaves through ISP-B. The probe, pinned to ISP-A, keeps failing — correctly.
5. **Later** — ISP-A recovers; probes succeed. `delay up 30` waits 30 s of success before declaring the track Up, so a link that's flapping doesn't drag traffic back and forth. Then the primary route returns and traffic moves back to ISP-A.

What you'd see during the outage:

```
R1# show track 1
Track 1
  IP SLA 10 reachability
  Reachability is Down
    3 changes, last change 00:00:12
  Delay up 30 secs, down 10 secs
  Latest operation return code: Timeout
  Tracked by:
    Static IP Routing 0
```

Total failover time is roughly one probe interval plus the down delay — about 15 s here. Shorten `frequency` and `delay down` for faster failover, at the cost of reacting to brief blips.

Why probe 8.8.8.8 instead of the ISP's next hop? The next hop (203.0.113.1) answers as long as the local link is up — even when the ISP's own upstream is down. Probing something **beyond** the ISP tests the path you actually care about.

Track type	Up when…
`track 1 ip sla 10 reachability`	Return code is **OK** or **Over threshold**
`track 1 ip sla 10 state`	Return code is **OK** only — slow counts as down
`track 2 interface Gi0/0 line-protocol`	Interface line protocol up

Jitter and delay

Voice and video care about **one-way delay**, **jitter** (variation in delay) and **loss** — none of which a ping measures properly.

Story Time : A bus that is always exactly 10 minutes late is annoying but predictable — you just leave 10 minutes later. A bus that is sometimes 2 minutes late and sometimes 18 minutes late is far worse, even with the same average. That variation is **jitter**. A voice call can live with steady delay, but jittery packets arrive bunched up and spaced out, so the phone must hold a buffer to smooth them — and packets arriving after the buffer has played out are as good as lost.

The **udp-jitter** operation sends a stream of timestamped UDP packets:

```
! On the far-end router — REQUIRED for udp-jitter:
ip sla responder
!
! On the measuring router:
ip sla 20
 udp-jitter 10.2.2.2 16384 codec g711ulaw     ! simulate a G.711 voice call
 tos 184                                       ! mark like real voice (DSCP EF)
 frequency 60
ip sla schedule 20 life forever start-time now
```

Why does udp-jitter need a responder? Because the far end must **timestamp** each packet when it arrives and when it replies, so the sender can separate the outbound and return legs and subtract the responder's own processing time. An ordinary host would just drop the packets. One-way delay values additionally need both routers' clocks synchronised with **NTP**; round-trip and jitter don't.

```
R1# show ip sla statistics 20
IPSLAs Latest Operation Statistics
IPSLA operation id: 20
Type of operation: udp-jitter
        Latest RTT: 41 milliseconds
Latest operation return code: OK
RTT Values:
        Number Of RTT: 1000       RTT Min/Avg/Max: 38/41/77 milliseconds
Packet Loss Values:
        Loss Source to Destination: 0   Loss Destination to Source: 0
Jitter Time:
        Source to Destination Jitter Min/Avg/Max: 0/2/18 milliseconds
        Destination to Source Jitter Min/Avg/Max: 0/1/9 milliseconds
Voice Score Values:
        Calculated Planning Impairment Factor (ICPIF): 1
        MOS score: 4.34
```

Worked example — reading the udp-jitter statistics above. Average round-trip 41 ms is fine for voice (the usual guideline is under 150 ms one-way). Average jitter of 2 ms source-to-destination and 1 ms on the way back, peaking at 18 ms, is well within what a phone's jitter buffer absorbs (a common target is under 30 ms). Zero loss. **MOS** (Mean Opinion Score — a

1 to 5 estimate of call quality) of 4.34 is close to the G.711 maximum of about 4.4. Verdict: this path is healthy for voice. If users still complain, the problem is elsewhere — the LAN, the phones, or a different time of day (the probe already runs continuously with `life forever`, so look at its history for the hours users complain about).

Symptom	Likely cause	Command that proves it
Statistics show nothing / "Number of successes: 0", never ran	Operation **not scheduled** — the most common fault	`show ip sla configuration 10` (look for schedule), `show ip sla summary`
udp-jitter return code "No connection" / timeout	`ip sla responder` **missing** on the far end, or UDP blocked	`show ip sla responder` on the target
Probe fails though path is fine	Wrong source interface, or ACL/CoPP drops probes	`show ip sla statistics`, `debug ip sla trace`
Track never goes down when ISP fails	Probing the ISP next hop instead of beyond it	`show track`
Default route flaps every few seconds	Probe follows the backup path after failover	Missing host route pinning the probe
Failover too slow	`frequency`/`timeout` too long, or track `delay down` too long	`show ip sla configuration`, `show track`
One-way delay values nonsensical	Clocks not NTP-synced	`show ntp status` both ends

32.7 NetFlow — v5, v9 and Flexible NetFlow

NetFlow answers "**who** is talking to **whom**, using **what**, and how much?" — the question SNMP interface counters can't. A **flow** is a set of packets sharing the same **key fields**; the router keeps a counter per flow in its **cache** and exports expired flows to a **collector**. Chapter 15 configured Flexible NetFlow; here we compare versions and troubleshoot.

Story Time : SNMP interface counters are like your phone bill's **total**: "you used 4,000 minutes this month." NetFlow is the **itemised bill**: every call, who you called, when, and for how long. When the bill is suddenly huge, only the itemised version tells you who to blame.

How NetFlow builds and exports a flow, step by step:

1. A packet arrives on an interface with a monitor applied: 10.1.20.55 → 172.16.5.10, TCP, source port 51514, destination port 443.
2. The router reads the **key fields** (here: source/destination address, protocol, source/destination port, input interface) and looks for a matching entry in the **flow cache**.
3. No match → it creates a new flow entry. Match → it adds this packet's bytes and a packet count to the existing entry. So a 10,000-packet download is **one** flow with one set of counters, not 10,000 records.
4. A flow is **expired** (removed from the cache and sent) when: the TCP session ends (FIN or RST seen), it has been idle for the **inactive timeout** (default 15 s), it has been running for the **active timeout** (FNF default 1800 s — long flows are reported in slices), or the cache fills up.

5. Expired flows are bundled into export packets and sent by UDP to the **collector** (for example 10.1.1.70 port 2055), which stores them and draws "top talkers" reports.

Worked example — what counts as a separate flow? Host 10.1.20.55 opens two browser tabs to the same web server, 172.16.5.10:443, and pings it once:

Traffic	Key fields that differ	Flows created
Tab 1, source port 51514	—	Flow 1
Tab 2, source port 51515	Source port	Flow 2 (different key)
Ping	Protocol ICMP, no ports	Flow 3
Server's replies (arriving on another interface)	Source/destination swapped	Counted only if a monitor is applied on **that** interface in the right direction

That last row is the reason "traffic totals are half of reality" in the table below.

Version	Format	Flexible?	IPv6 / MPLS	Key fields
v5	**Fixed** record	No	**No** — IPv4 only	Fixed 7: src/dst IP, src/dst port, protocol, ToS, input interface
v9	**Template-based** — the exporter first sends a template describing its records	Record content set by the feature	Yes	Defined by the template
Flexible NetFlow	Cisco's configuration model, exporting as **v5, v9 or IPFIX**	**Yes** — you choose `match` (key) and `collect` (non-key) fields	Yes	Whatever you `match`

Why did NetFlow move to templates? Because a fixed v5 record can't grow: there's no room for IPv6 addresses, MPLS labels, application IDs or anything invented later. A v9 exporter sends a **template** first ("my records are: field 8 for 4 bytes, field 12 for 4 bytes..."), then data records that follow it. New fields need a new template, not a new protocol version. The catch: **the collector can't decode any data until it has received the template** — the source of a whole class of v9 problems.

Traditional (v5/v9) configuration, for recognition:

```
interface GigabitEthernet0/1
 ip flow ingress                              ! collect flows entering this interface
!
ip flow-export version 9                     ! or: ip flow-export version 5
ip flow-export destination 10.1.1.70 2055
ip flow-export source Loopback0
R1# show ip cache flow
R1# show ip flow export
```

The only difference between a v5 and a v9 export in traditional NetFlow is the `ip flow-export version` line. **Set it explicitly and match the collector**: a collector expecting v9 templates can't parse v5 records and vice versa, and only v9 can carry IPv6 flows. `show ip flow export` shows the version in use, the destination, and the count of export failures.

Flexible NetFlow (record → exporter → monitor → interface):

```
flow record FR-BASIC
 match ipv4 source address
 match ipv4 destination address
 match ipv4 protocol
 match transport source-port
 match transport destination-port
 match interface input
 collect counter bytes long
 collect counter packets long
!
flow exporter FE-COLLECTOR
 destination 10.1.1.70
 source Loopback0
 transport udp 2055
 export-protocol netflow-v9
 template data timeout 60                ! resend templates every 60 s
!
flow monitor FM-BASIC
 record FR-BASIC
 exporter FE-COLLECTOR
 cache timeout active 60
!
interface GigabitEthernet0/1
 ip flow monitor FM-BASIC input
```

Symptom	Likely cause	Command that proves it
Collector shows nothing; cache full of flows	Exporter destination, **UDP port** mismatch (2055 / 9995 / 9996 / IPFIX 4739 — must match the collector), route, or ACL	`show flow exporter statistics` (packets sent), `show flow exporter FE-COLLECTOR`
Collector receives packets but can't decode them (v9)	**Template not received** — collector started after the template was sent	Lower `template data timeout`; check collector logs
Collector lists the router under an unexpected IP	No `source` on the exporter	`show flow exporter`
Cache empty	Monitor not applied to the interface, or applied in wrong direction	`show flow interface Gi0/1`, `show flow monitor FM-BASIC cache`
Traffic totals half of reality	Only `input` monitored on some interfaces — traffic in the other direction never seen	`show flow interface` on every relevant interface
Long transfers appear late or as one giant spike	Active timeout too long (FNF default **1800 s**)	`show flow monitor FM-BASIC` — lower `cache timeout active`
Collector shows no IPv6	Exporting **v5** (IPv4-only) or record only matches IPv4	`show flow exporter`, record definition

32.8 Catalyst Center (formerly DNA Center) Assurance

Chapter 16 introduced Catalyst Center. **Assurance** is its troubleshooting half: it ingests telemetry from every device and client — streaming telemetry, SNMP, syslog, NetFlow, ISE and DHCP/DNS data, wireless data — and turns it into **health scores** and **issues**. ENARSI asks you to use it for four things named in the blueprint: **network health, device health, connectivity, and monitoring**.

View	Tells you	Typical use
Network Health	The **percentage of healthy devices**, broken down by site and device role (core, distribution, access, router, WLC, AP)	"Is anything wrong right now, and where?"
Device Health / Device 360	A **1–10 score** per device: **Poor 1–3 (red), Fair 4–7 (orange), Good 8–10 (green)** — built from KPIs such as CPU, memory, reachability, interface errors, link status	Drilling into one switch: which KPI dragged its score down
Client Health / Client 360	Per-client onboarding and connectivity: association, authentication, DHCP, DNS, RF quality for wireless	"Why can't this user connect?" — showing which onboarding step failed
Issues	Detected problems with **severity**, affected devices/clients, **root cause** and **suggested actions** (guided remediation)	Working the queue: what's broken, why, and the CLI checks Catalyst Center recommends
Path Trace	The hop-by-hop **L2/L3 path** between two endpoints, with interfaces, and whether **ACLs** along the path permit the flow	"The app server is unreachable from this VLAN" — find the hop or ACL in the way
Time travel	The network's health at a past moment	"It was broken at 9:40 and fine now" — see what it looked like then

Why is Assurance more than a fancy SNMP poller? Because it **correlates**. A classic NMS tells you "interface Gi1/0/48 errors increased" and "client 10.1.20.55 failed DHCP" as two unrelated alarms. Assurance knows the client is attached to that switch, that DHCP is relayed through that uplink, and that the errors started at the same minute — so it raises **one issue** with a probable root cause. Its AI-driven analytics (Chapter 16) go further, learning each site's **baseline** so it can flag "abnormal for this network" rather than relying on fixed thresholds.

A troubleshooting workflow that matches how the exam frames it:

1. **Network Health** — find the site/role with a falling score.

2. **Device 360** — open the worst device; see which KPI (CPU, memory, link errors, reachability) is red and when it changed.
3. **Issues** — read the root cause and **suggested actions**; run the recommended commands (Catalyst Center can run them for you and show the output).
4. **Client 360** — if users report it, confirm which onboarding step fails (association → authentication → DHCP → DNS).
5. **Path Trace** — prove whether the forwarding path and ACLs allow the flow end to end.
6. **Time travel** — for intermittent problems, compare "then" with "now".

Story Time : A traditional NMS is a room full of separate alarms — smoke detector, door sensor, thermostat — each beeping on its own. Assurance is the doctor reading all your test results together: "your temperature, your blood test and your cough started on the same day — it's one infection, and here's the treatment." Same raw data; the difference is that it's connected.

Worked example — ticket: "Users on the third floor can't get on the network since 8:45."

1. **Network Health** shows Site HQ / Floor 3 with the access role dropping to 60% healthy.
2. **Client Health** shows dozens of Floor 3 clients failing onboarding. **Client 360** for one of them shows association and authentication **succeeded**, but the **DHCP** step failed — the client never got an address.
3. **Issues** lists "DHCP server not responding for clients on VLAN 30" with the affected switch and a suggested action: check the relay configuration on the Floor 3 distribution SVI.
4. **Device 360** for the distribution switch shows a configuration change at 08:43. Running the suggested command from Catalyst Center shows `ip helper-address` missing from interface Vlan30 — removed by mistake during that change.
5. Restore the helper. Client Health recovers within minutes; **time travel** to 08:40 confirms the floor was healthy before the change.

The value: without Assurance you'd have had 40 separate "can't connect" calls and a switch config change, and connected them yourself. Assurance tied them together in one issue.

> **Exam trap:** health scores come from data Catalyst Center **receives**. A device that's unmanaged, unreachable, or not sending telemetry/syslog/SNMP to Catalyst Center can look "unknown" or stay stale — that's a monitoring gap, not a healthy device. When one device's data looks wrong, check its reachability and telemetry configuration from Catalyst Center (inventory: **Managed** vs **Unreachable** / sync errors) before trusting its score.

Part III — Passing Both Exams

Chapter 33: Exam Strategy & Study Plans

33.1 How the Certification Fits Together

CCNP Enterprise is **two exams**, and each one earns something on its own:

You pass	You earn
ENCOR 350-401 alone	Cisco Certified Specialist – Enterprise Core, and eligibility to book the **CCIE Enterprise Infrastructure** lab (CCIE Enterprise Wireless now qualifies through WLCOR 350-101 instead)
ENARSI 300-410 alone	Cisco Certified Specialist – Enterprise Advanced Infrastructure Implementation
Both (in either order)	**CCNP Enterprise**

The certification is valid for **three years**. You can recertify by passing exams again or by earning Continuing Education credits, and passing ENCOR again on its own also recertifies it.

Why does ENCOR come first in this guide, if the order is up to you? Because ENARSI is built on top of it. ENCOR asks you to configure simple OSPF and eBGP; ENARSI asks you to troubleshoot multi-area OSPF, iBGP route reflection and redistribution between them. Studying ENARSI first means learning the foundation and the advanced layer at the same time, while ENCOR first means Part II feels like the next floor of a building you already know. The rare exception: if your day job is heavy routing work and you want a quick early win, ENARSI first is defensible. For everyone else, take ENCOR first, then ENARSI while the core is still fresh.

33.2 The Exams Themselves

Exam	Duration	Blueprint version	Domains
ENCOR 350-401	120 minutes	**v1.2** (since 19 March 2026)	6 — wide coverage
ENARSI 300-410	90 minutes	**v1.1**	4 — deep, troubleshooting-heavy

These are the versions **at the time of writing (October 2026)**. Cisco refreshes blueprints periodically, and ENARSI v1.1 is the older of the two, so confirm the current version on the Cisco Learning Network before you book. If a version changes, compare its topic list against Appendix A or B line by line.

Things that hold for both:

- **You cannot go back.** Once you submit an answer, it is final. Answer each question as if it were the last one.
- **Question types** include multiple choice (single and multiple answer), drag-and-drop, and hands-on simulation or lab-style items. Cisco does not publish question counts and adjusts formats over time, so check the current exam page on the Cisco Learning Network the week you book.
- **Scores are scaled** and the passing mark is not published. Don't try to calculate it during the exam; just make every answer count.

Why do the two exams feel so different, even though they share topics? Look at the verbs in each blueprint. ENCOR mostly says describe, explain, interpret and configure: can you recognise it, and can you set it up? ENARSI says **troubleshoot** almost every time: here's a broken network, find the fault. ENCOR rewards breadth: a little of everything from QoS to Python. ENARSI rewards depth: given a stuck BGP session, can you name the four likely causes and the command that proves each one? Study for ENCOR with understanding plus recall; study for ENARSI by breaking labs on purpose.

What each verb looks like as a question. These are examples written for this guide in the style of each verb, not real exam items:

Verb	What it asks of you	Example of the style
Describe / Explain	Recognise a feature and say what it does or why	"Which SD-Access component holds the host tracking database?" (the control plane node, 14.7)
Interpret	Read a config or output and say what happens	An exhibit of a `policy-map` with `priority percent 10` on class VOICE: "What happens to voice traffic above 10% during congestion?" (it's policed and dropped, 11.7)
Configure / Construct	Produce the right commands or file	"Drag the lines into the right order to build an EEM applet that saves the config when an interface goes down" (21.1)
Troubleshoot	Find the fault from symptoms and output	`show ip ospf neighbor` stuck in EXSTART plus two `show interface` outputs with MTU 1500 and 1400: "What is the cause?" (MTU mismatch, 27.2)

Notice how the troubleshoot item gives you evidence, not a description of the fault. Practise reading output until the fault jumps out.

33.3 Your Lab

You already run **EVE-NG**, and that covers almost everything. What to run where:

Topic area	Best platform	Notes
Switching, STP/MST, EtherChannel (Ch 3–4)	**IOL L2** images in EVE-NG	Lightweight; lab with 4–6 switches at once
Routing, redistribution, BGP, PBR, VRF-Lite, BFD (Ch 5–7, 22–28)	**IOL L3** or **vIOS**	Run many routers on modest hardware
HSRP/VRRP, NAT, NTP, multicast, QoS policies (Ch 8–11)	IOL L3 / vIOS	Multicast needs a source and a receiver — use VPCS or Linux nodes
GRE, IPsec, DMVPN, MPLS L3VPN (Ch 12, 29, 30)	IOL L3 or **Catalyst 8000v**	All work on IOL; the 8000v gives you IKEv2 defaults identical to production
NETCONF, RESTCONF, EEM, Flexible NetFlow, IP SLA (Ch 15, 20, 21, 32)	**Catalyst 8000v** (IOS XE)	Model-driven APIs need IOS XE
Python, Ansible, JSON (Ch 19–21)	A Linux node in EVE-NG, or your laptop	Point scripts at the 8000v
Catalyst Center, SD-WAN, SD-Access (Ch 14, 16, 20)	**Cisco DevNet sandboxes**	Real controllers without the hardware; several are always-on

Why bother with a full lab when the blueprint says "describe" for so much of it? Because the configure and troubleshoot items are where people lose marks, and you can't fake those. A lab is also the only reliable way to find out you've misunderstood something: reading lets you believe a wrong idea indefinitely, but a router tells you within seconds. Every chapter with config in it should end with you typing that config and breaking it once.

33.4 The Study Loop

The same loop that got you through CCNA, extended slightly for CCNP:

1. **Read the chapter.** Note every **Why?** you couldn't have answered before reading.
2. **Lab it.** Type the configs yourself rather than pasting them. Then **break one thing on purpose** and fix it using only `show` and `debug` output. ENARSI is a troubleshooting exam, so practise troubleshooting.
3. **Explain it out loud (the Feynman test).** Explain the chapter's core mechanism as if teaching someone, without notes. Wherever you hand-wave, you've found a gap. Go back to that section. Having Claude play the skeptical student who keeps asking "but why?" works well here.
4. **Check against the blueprint.** Find the chapter's lines in Appendix A or B. For each one, can you do what the verb says (describe, configure, troubleshoot)?
5. **Move on.** Don't wait for perfection. Review comes from the schedule below, not from re-reading the same chapter five times.

Why the Feynman step, when reading again feels more productive? Because re-reading creates familiarity, and familiarity feels like understanding. Explaining from memory forces retrieval, and retrieval is what both the exam and a 3 a.m. outage demand. If you can't explain it,

you don't know it yet. You only recognise it.

33.5 ENCOR Study Plans

The 10-Week Plan (steady pace, ~1–1.5 hrs/day)

Week	Chapters	Focus	Lab
1	1–2	Design models, high availability	Build the base campus topology you'll reuse
2	3–4	Trunking/EtherChannel faults, RSTP, MST, STP guards	Break DTP and LACP; migrate PVST+ to MST
3	5–6	EIGRP vs OSPF, multi-area OSPF, OSPFv3	Three areas, summarization, filtering
4	7	eBGP and best-path selection	Two ASes; steer traffic with weight and local-pref
5	8–9	NTP, PTP, NAT/PAT, HSRP, VRRP	HSRP with tracking; PAT with a pool
6	10–11	Multicast, QoS	PIM-SM with a static RP; read your own `show policy-map`
7	12–14	VRF, GRE, IPsec, LISP, VXLAN, SD-WAN, SD-Access	GRE over IPsec with a VTI; explore the DevNet SD-WAN sandbox
8	15–16	Debugs, SNMP, syslog, Flexible NetFlow, SPAN, IP SLA, Catalyst Center	Flexible NetFlow + ERSPAN on an 8000v
9	17–18	AAA, ACLs, CoPP, security design, REST API security	Local fallback AAA; a CoPP policy
10	19–21 + review	Python, JSON, YANG, NETCONF/RESTCONF, EEM, Ansible, AI	RESTCONF from Python; three EEM applets. Then Appendix A as a checklist

The 5-Week Sprint (~2.5–3 hrs/day)

Week	Chapters	The one thing you must leave the week able to do
1	1–4	Read `show etherchannel summary` and `show spanning-tree mst` cold, and say what's wrong
2	5–9	Configure multi-area OSPF and eBGP from a blank router, and recite best-path order
3	10–14	Explain an SD-Access packet flow: LISP lookup, VXLAN encapsulation, SGT enforcement
4	15–18	Configure AAA with fallback, CoPP and Flexible NetFlow without notes
5	19–21 + review	Predict a Python script's output, and write valid JSON and an EEM applet from scratch

If you fall behind, cut in this order: depth sections first, then the second lab in a week (keep the first), then shorten the review week. **Never cut Infrastructure (Ch 3–10).** It's 30% of the exam.

33.6 ENARSI Study Plans

The 6-Week Plan (steady pace, ~1–1.5 hrs/day)

Week	Chapters	Lab — always finish by breaking it
1	22–23	AD puzzles; prefix-list ge/le drills; route maps with tags
2	24–25	Two-point mutual redistribution, cause the loop, fix it with tags; PBR; VRF-Lite; BFD
3	26–27	EIGRP named mode with stubs and variance; OSPF NSSA plus a virtual link
4	28	iBGP with a route reflector, communities, AS-path filters
5	29–30	A small MPLS L3VPN; DMVPN Phase 3 with IPsec
6	31–32 + review	AAA lockout recovery, uRPF, CoPP, SNMPv3, DHCP relay, IP SLA tracking. Then Appendix B as a checklist

The 3-Week Sprint (~3 hrs/day)

Week	Chapters
1	22–25: the tools (AD, route maps, redistribution, PBR, VRF, BFD)
2	26–28: the three protocols, in depth
3	29–32 + review: VPNs, security, services

ENARSI's honest truth: reading is the smaller half of the work. If you only have so many hours, put more of them into **breaking labs** than into reading. A good ENARSI week is one where you've seen every `show` command in the chapter return output that tells you something is broken.

33.7 Exam-Day Tactics

- **Read the last sentence first.** CCNP stems are long, and the question is often in the last line. Knowing what's being asked tells you which details in the scenario matter. Example (written for this guide, not a real exam item): a six-line stem describes two sites, three routing protocols and a recent change, and the last line asks "Which command shows the route's administrative distance?" Most of the scenario turns out to be background; you only needed to know the answer is `show ip route <prefix>`.

- **"Choose two" means two.** Most people who miss a multiple-answer question found one right answer and stopped thinking. Example (guide-written): "Which two fields must match for OSPF neighbours to form? (Choose two.)" with options area ID, router ID, hello interval, process ID, interface cost. Area ID jumps out first; keep going and hello interval is the second. Router ID must be different, and process ID and cost are local.
- **Budget the clock.** You can't go back, so time can't be reclaimed later. Glance at the clock every ten questions and keep a steady pace instead of saving time for review that won't exist.
- **Sims:** read every task before you type anything, verify each task with a `show` command, and remember that what's graded is usually the running config's effect, not how elegant it is.
- **Unsure? Eliminate.** CCNP distractors are usually real commands or real features used in the wrong place. Ask "would this even apply here?" before "is this right?" Example (guide-written): "Which protocol does the SD-WAN Controller use to distribute routes to WAN Edges?" with options LISP, OMP, BGP EVPN, IS-IS. All four are real overlay or fabric protocols. LISP belongs to SD-Access, BGP EVPN to data-center VXLAN, and IS-IS to the SD-Access underlay. What's left is OMP.

33.8 Must-Memorize Cheat Sheet

Administrative distance

Source	AD	Source	AD
Connected	0	OSPF	110
Static	1	IS-IS	115
EIGRP summary	5	RIP	120
eBGP	**20**	EIGRP external	170
EIGRP internal	90	**iBGP**	**200**
Unknown / unusable	255		

Protocols, ports and addresses

What	Value
OSPF	IP protocol 89 · 224.0.0.5 (all OSPF) · 224.0.0.6 (DR/BDR) · FF02::5 / FF02::6
EIGRP	IP protocol 88 · 224.0.0.10 · FF02::A
PIM / IGMP	224.0.0.13 (PIM) · IGMPv3 reports to 224.0.0.22 · SSM range 232.0.0.0/8
HSRP	UDP 1985 · v1 224.0.0.2 · v2 224.0.0.102 · vMAC v1 0000.0c07.acXX, v2 0000.0c9f.fXXX
VRRP	IP protocol 112 · 224.0.0.18 · vMAC 0000.5e00.01XX
BGP	TCP 179
LDP	UDP 646 (hellos) · TCP 646 (session)
GRE / ESP / AH	IP protocols 47 / 50 / 51
IKE	UDP 500, and UDP 4500 with NAT traversal
VXLAN · LISP	UDP 4789 · UDP 4342 (control) / 4341 (data)
NTP · PTP	UDP 123 · UDP 319 (event) / 320 (general)
SNMP · syslog	UDP 161 (polls), 162 (traps/informs) · UDP 514
TACACS+ · RADIUS	TCP 49 · UDP 1812 auth / 1813 accounting
NETCONF · RESTCONF	SSH, TCP 830 · HTTPS, TCP 443

BGP best path, in order

N WLLA OMNI (the same mnemonic Chapter 7 teaches): **N**ext hop reachable → highest **W**eight → highest **L**ocal preference → **L**ocally originated → shortest **A**S path → lowest **O**rigin (IGP < EGP < incomplete) → lowest **M**ED → **N**eighbour type (eBGP over iBGP) → lowest **I**GP metric to the next hop. Then the tie-breakers: oldest eBGP path → lowest router ID → shortest cluster list → lowest neighbour address. Chapter 7.7 walks through every step and Chapter 28.6 shows how to manipulate each one.

OSPF LSAs

Type	Name	Generated by	Blocked by
1	Router	Every router	—
2	Network	DR	—
3	Summary	ABR	Totally stubby, totally NSSA
4	ASBR summary	ABR	Stub, totally stubby, NSSA, totally NSSA
5	External	ASBR	Stub, totally stubby, NSSA, totally NSSA
7	NSSA external	ASBR in an NSSA	(converted to Type 5 by the ABR)

QoS markings

DSCP for AFxy = **8x + 2y**: AF11 = 10, AF21 = 18, AF31 = 26, AF41 = 34. **EF = 46** (voice). CSn = 8n: CS3 = 24 (signalling), CS6 = 48 (routing).

HTTP status codes

200 OK · **201** Created · **202** Accepted (async task started — Catalyst Center returns a task ID) · **204** No Content · **400** Bad Request · **401** Unauthorized (not authenticated) · **403** Forbidden (authenticated but not allowed) · **404** Not Found · **405** Method Not Allowed · **409** Conflict · **415** Unsupported Media Type · **429** Too Many Requests (rate-limited) · **500** Internal Server Error · **503** Service Unavailable.

Appendix A: ENCOR 350-401 v1.2 Blueprint Coverage Map

Cisco publishes the official topic list for **ENCOR 350-401 v1.2**, the version in effect since 19 March 2026. This table maps **every** item on it to the sections that teach it. Use it two ways: as the final checklist before you book the exam, and as the index to go back to when a practice question catches you out. A **bold** section is the main treatment. Sections in (italics) are the ENARSI-depth follow-ups in Part II: not needed for ENCOR, but the next place to go if you want more.

The verb matters. **Describe/Explain** means recognise and explain it. **Interpret** means read a config or output and say what it does. **Configure/Construct** means produce it from scratch. **Troubleshoot** means find what's broken.

1.0 Architecture (15%)

#	Blueprint topic	Where
1.1	Explain the different design principles used in an enterprise network	**1.2**, 1.9
1.1.a	High-level enterprise network design such as 2-tier, 3-tier, fabric, and cloud	**1.3–1.6**, 1.7, **1.8**
1.1.b	High availability techniques such as redundancy, FHRP, and SSO	**2.1–2.8**, 9.1, 9.4
1.2	Explain the working principles of the Cisco Catalyst SD-WAN solution	**14.1–14.5**, 14.9
1.2.a	SD-WAN control and data planes elements	**14.2–14.4**
1.2.b	Benefits and limitations of Catalyst SD-WAN solution	**14.1**, **14.5**
1.3	Explain the working principles of the Cisco SD-Access solution	**14.6–14.8**, 14.9
1.3.a	SD-Access control and data planes elements	**14.6**, **14.7** (LISP and VXLAN themselves: 13)
1.3.b	Traditional campus interoperating with SD-Access	**14.8**
1.4	Interpret QoS configurations	11.1–11.6, **11.7** (line-by-line reading)

2.0 Virtualization (10%)

#	Blueprint topic	Where
2.1	Describe device virtualization technologies	**12.1–12.4**
2.1.a	Hypervisor type 1 and 2	**12.2**
2.1.b	Virtual machine	**12.1**, **12.3**
2.1.c	Virtual switching	**12.4**
2.2	Configure and verify data path virtualization technologies	**12.5–12.11**
2.2.a	VRF (configure and verify)	**12.5**, **12.6**, 12.11 (25.2 VRF-Lite with routing protocols)
2.2.b	GRE and IPsec tunneling (configure and verify)	**12.7–12.11** (30 DMVPN)
2.3	Describe network virtualization concepts	**13.1–13.5**
2.3.a	LISP	**13.1–13.3**, 13.5
2.3.b	VXLAN	13.2, **13.4**, 13.5

3.0 Infrastructure (30%)

#	Blueprint topic	Where
3.1	Layer 2	**3**, **4**
3.1.a	Troubleshoot static and dynamic 802.1q trunking protocols	**3.1–3.4**, 3.9
3.1.b	Troubleshoot static and dynamic EtherChannels	**3.5–3.9**
3.1.c	Configure and verify RSTP, MST, and STP enhancements (root guard, BPDU guard)	**4.1–4.2** RSTP, **4.3** MST, **4.4–4.6** enhancements
3.2	Layer 3	**5**, **6**, **7**
3.2.a	Compare EIGRP and OSPF (DV vs link state, load balancing, path selection, path operations, metrics, area types)	**5.1–5.8** (26 EIGRP, 27 OSPF in depth)
3.2.b	Configure simple OSPFv2/v3 — multiple normal areas, summarization, filtering, adjacency, p2p/broadcast, passive-interface	**6.1–6.9** (27)
3.2.c	Configure and verify eBGP between directly connected neighbors (best path, neighbor relationships)	**7.1–7.9** (28)
3.2.d	Describe policy-based routing	**5.9** (25.1 configuration)
3.3	IP Services	**8**, **9**, **10**
3.3.a	Interpret NTP and PTP configurations	**8.1–8.4**
3.3.b	Configure NAT/PAT	**8.5**
3.3.c	Configure FHRPs such as HSRP and VRRP	**9.1–9.5**, 9.7, 9.8 (GLBP comparison 9.6)
3.3.d	Describe multicast — RPF check, PIM SM, IGMP v2/v3, SSM, bidir, MSDP	**10.2–10.6** (IGMP 10.3, PIM-SM 10.4–10.5, RPF 10.5, SSM/bidir/MSDP 10.6)

4.0 Network Assurance (10%)

#	Blueprint topic	Where
4.1	Diagnose problems using debugs, conditional debugs, traceroute, ping, SNMP, and syslog	**15.1–15.4**, 15.8 (32 troubleshooting depth)
4.2	Configure and verify Flexible NetFlow	**15.5** (32)
4.3	Configure SPAN/RSPAN/ERSPAN	**15.6**
4.4	Configure and verify IP SLA	**15.7**, tracking in 9 (32)
4.5	Describe how Catalyst Center applies configuration, monitoring and management using traditional and AI-powered workflows	**16.1–16.5**
4.6	Configure and verify NETCONF and RESTCONF	**20.2–20.4**

5.0 Security (20%)

#	Blueprint topic	Where
5.1	Configure and verify device access control	**17.1–17.4**
5.1.a	Lines and local user authentication	**17.1**, **17.2**
5.1.b	Authentication and authorization using AAA	**17.3**, **17.4** (31 troubleshooting)
5.2	Configure and verify infrastructure security features	**17.5–17.7**
5.2.a	ACLs	**17.5** (31)
5.2.b	CoPP	**17.6**, 17.7 (31)
5.3	Describe REST API security	**18.7**
5.4	Describe the components of network security design	**18.1–18.6**
5.4.a	Threat defense	**18.1**, **18.2**
5.4.b	Endpoint security	**18.3**
5.4.c	Next-generation firewall	**18.4**
5.4.d	TrustSec and MACsec	**18.5**, **18.6**

6.0 Automation and Artificial Intelligence (15%)

#	Blueprint topic	Where
6.1	Interpret basic Python components and scripts	**19.1–19.7**
6.2	Construct valid JSON-encoded files	**19.8–19.10**
6.3	Describe data modeling language principles and benefits, such as YANG	**20.1**
6.4	Describe APIs for Catalyst Center and SD-WAN Manager	**20.5**, **20.6**
6.5	Interpret REST API response codes and payload results (Catalyst Center, RESTCONF)	**20.7**, 20.5
6.6	Construct an EEM applet to automate configuration, troubleshooting, or data collection	**21.1**, **21.2**
6.7	Compare agent vs. agentless orchestration tools	**21.3**, **21.4**
(note)	The "Artificial Intelligence" half of the domain title — not a numbered item	**21.5**, and 16.3–16.4 (AI-powered workflows)

Appendix B: ENARSI 300-410 v1.1 Blueprint Coverage Map

Every item on Cisco's official **ENARSI 300-410 v1.1** topic list, mapped to the sections that teach it. A **bold** section is the main treatment. Nearly every line says troubleshoot, so for each one ask yourself: given a broken lab, could I find this fault with `show` and `debug` alone?

1.0 Layer 3 Technologies (35%)

#	Blueprint topic	Where
1.1	Troubleshoot administrative distance (all routing protocols)	**22.3**, **22.5–22.7** (method 22.1–22.3, RIB/FIB 22.4)
1.2	Troubleshoot route map for any routing protocol (attributes, tagging, filtering)	**23.2–23.5**, 23.7
1.3	Troubleshoot loop prevention mechanisms (filtering, tagging, split horizon, route poisoning)	**23.6**, 23.5, 24.4
1.4	Troubleshoot redistribution between any routing protocols or sources	**24.1–24.4**, **24.8** (default-route origination)
1.5	Troubleshoot manual and auto-summarization with any routing protocol	**24.5–24.7**
1.6	Configure and verify policy-based routing	**25.1** (concept: 5.9)
1.7	Configure and verify VRF-Lite	**25.2** (concept: 12.5–12.6)
1.8	Describe Bidirectional Forwarding Detection	**25.3**
1.9	Troubleshoot EIGRP (classic and named mode; VRF and global)	**26.1**, **26.3**, 26.11 (concepts: 5.2–5.6)
1.9.a	Address families (IPv4, IPv6)	**26.1**, **26.2**
1.9.b	Neighbor relationship and authentication	**26.4**, **26.5**
1.9.c	Loop-free path selection (RD, FD, FC, successor, feasible successor, stuck in active)	**26.6**, **26.7**
1.9.d	Stubs	**26.8**
1.9.e	Load balancing (equal and unequal cost)	**26.9**
1.9.f	Metrics	**26.10**
1.10	Troubleshoot OSPF (v2/v3)	**27.1–27.10**, 24.8 (default origination) (foundation: 6)
1.10.a	Address families (IPv4, IPv6)	**27.1**
1.10.b	Neighbor relationship and authentication	**27.2**, **27.3**
1.10.c	Network types, area types, and router types	**27.4**, **27.5**

#	Blueprint topic	Where
1.10.c.i	Network types: point-to-point, multipoint, broadcast, nonbroadcast	**27.4**
1.10.c.ii	Area types: backbone, normal, transit, stub, NSSA, totally stub	**27.5**
1.10.c.iii	Router types: internal, backbone, ABR, ASBR	**27.5**
1.10.c.iv	Virtual link	**27.6**
1.10.d	Path preference	**27.7**
1.11	Troubleshoot BGP (internal and external, unicast, and VRF-Lite)	**28.1**, **28.9**, 28.10 (foundation: 7)
1.11.a	Address families (IPv4, IPv6)	**28.4**
1.11.b	Neighbor relationship and authentication (next-hop, multihop, 4-byte AS, private AS, route refresh, synchronization, operation, peer group, states and timers)	**28.1–28.5**, 28.11 (maximum-prefix, local-as, default-originate): multihop 28.1, next-hop 28.2, auth/states/timers 28.3, 4-byte and private AS 28.4, route refresh/synchronization/peer groups 28.5
1.11.c	Path preference (attributes and best-path)	**28.6** (algorithm first met in 7.7)
1.11.d	Route reflector (excluding multiple route reflectors, confederations, dynamic peer)	**28.7**
1.11.e	Policies (inbound/outbound filtering, path manipulation)	**28.8**, **28.11** (allowas-in, as-override, multipath, as-path ignore)

2.0 VPN Technologies (20%)

#	Blueprint topic	Where
2.1	Describe MPLS operations (LSR, LDP, label switching, LSP)	**29.1–29.5**
2.2	Describe MPLS Layer 3 VPN	**29.6–29.13**
2.3	Configure and verify DMVPN (single hub)	**30.1–30.10**
2.3.a	GRE/mGRE	**30.2**, 30.7 (GRE itself: 12.7)
2.3.b	NHRP	**30.3–30.5**
2.3.c	IPsec	**30.6** (IPsec itself: 12.8–12.10)
2.3.d	Dynamic neighbor	**30.8**
2.3.e	Spoke-to-spoke	**30.4**, **30.5**

3.0 Infrastructure Security (20%)

#	Blueprint topic	Where
3.1	Troubleshoot device security using IOS AAA (TACACS+, RADIUS, local database)	**31.1** (configuration: 17.3–17.4)
3.2	Troubleshoot router security features	**31.2–31.4**
3.2.a	IPv4 access control lists (standard, extended, time-based)	**31.2** (17.5)
3.2.b	IPv6 traffic filter	**31.3**
3.2.c	Unicast reverse path forwarding (uRPF)	**31.4**
3.3	Troubleshoot control plane policing (Telnet, SSH, HTTP(S), SNMP, EIGRP, OSPF, BGP)	**31.5** (configuration: 17.6)
3.4	Describe IPv6 First Hop security (RA guard, DHCP guard, binding table, ND inspection/snooping, source guard)	**31.6**

4.0 Infrastructure Services (25%)

#	Blueprint topic	Where
4.1	Troubleshoot device management	**32.1** (lines: 17.1)
4.1.a	Console and VTY	**32.1** (17.1)
4.1.b	Telnet, HTTP, HTTPS, SSH, SCP	**32.1**
4.1.c	(T)FTP	**32.1**
4.2	Troubleshoot SNMP (v2c, v3)	**32.2** (15.4)
4.3	Troubleshoot using logging (local, syslog, debugs, conditional debugs, timestamps)	**32.3** (15.2, 15.3)
4.4	Troubleshoot IPv4 and IPv6 DHCP (client, IOS server, relay, options)	**32.4** IPv4, **32.5** IPv6
4.5	Troubleshoot performance using IP SLA (jitter, tracking objects, delay, connectivity)	**32.6** (15.7)
4.6	Troubleshoot NetFlow (v5, v9, flexible NetFlow)	**32.7** (15.5)
4.7	Troubleshoot using Catalyst Center (formerly DNA Center) assurance — connectivity, monitoring, device health, network health	**32.8** (16)

Appendix C: The CCNA Foundations to Re-Read First

CCNP doesn't re-teach CCNA. It assumes it. These are the sections of the **CCNA Study Guide** that each part of this book builds on directly. Many of them are the depth sections, which were written to be the runway for exactly this. If one feels shaky, spend twenty minutes on it before starting the CCNP chapter. It's cheaper than getting lost halfway through.

Before CCNP chapter	Re-read in the CCNA guide	Why it matters here
1 — Design	1.7 Architectures, 1.8 SOHO/on-prem/cloud	Chapter 1 starts where those stop
2, 9 — HA and FHRP	14.7 FHRP & HSRP	Chapter 9 assumes the active/standby/virtual-IP idea

Before CCNP chapter	Re-read in the CCNA guide	Why it matters here
3 — Trunking & EtherChannel	8.1–8.2 802.1Q and DTP, 10.2–10.4 EtherChannel	The mode matrices extend these tables
4 — STP	9.8–9.11 PVST+, guards, BPDU mechanics, RSTP sync	MST and the guards assume the decision order and port roles cold
5–6, 27 — OSPF	16.9–16.11 neighbour requirements, DR/BDR, LSAs, area types	Multi-area starts from LSA types 1–7
5, 22 — Routing logic	14.3–14.5 longest match, AD, metric	The three-step path selection is the spine of Part II
8 — NTP & NAT	17.3 NAT, 17.4 NTP	Inside/outside local/global terms are used without definition
10 — Multicast	5.4 unicast/broadcast/multicast	Multicast MAC mapping builds on Ethernet addressing
11 — QoS	17.7 QoS	The toolkit vocabulary (classify, mark, queue, police) carries over
12 — VRF & tunnels	1.9 VRFs, 2.7 TCP (MSS and MTU)	GRE/IPsec MTU problems are 2.7's MSS story in practice
15, 32 — Assurance	17.5 Syslog, 17.6 SNMP	Severity levels and SNMP versions are assumed
17, 31 — Security	18.4 AAA, 18.6 802.1X, 19 ACLs	AAA method lists and ACL logic are the starting point
19–21 — Automation	22.3–22.6 APIs, REST, JSON/YAML/XML, tools	Part I's automation chapters go from concept to working code
13 & 31 — IPv6 pieces	13.8 NDP & SLAAC	RA guard and ND inspection protect the messages taught there
Everything	14.8 A Day in the Life of a Packet	Still the best troubleshooting method you own

If you can explain every row's CCNA section out loud, you're ready for Part I.

Glossary (Quick Definitions)

Terms the CCNA glossary already covers (VLAN, ARP, OSPF and so on) aren't repeated here. The chapter number shows where each term is taught.

Term	Meaning	Ch
802.1X	Port-based access control: supplicant ↔ authenticator (switch) ↔ authentication server (RADIUS/ISE), using EAP	18
AAA	Authentication, Authorization and Accounting — the framework for who can log in, what they can do, and what they did	17, 31
ABR	Area Border Router — OSPF router with interfaces in Area 0 and another area; generates Type 3 LSAs	6, 27
Address family	A protocol's per-address-type configuration context (IPv4, IPv6, VPNv4, a VRF)	7, 26–28
Agentless	Orchestration that needs no software on the managed device (e.g. Ansible over SSH/NETCONF)	21
AIOps	Using machine learning and AI to detect, correlate and explain operational problems	16, 21
Ansible / Puppet / Chef / Salt	Orchestration tools: Ansible is agentless push over SSH/NETCONF; Puppet and Chef are agent-based pull; Salt is agent-based (minions) and can also run agentless over SSH	21
Anycast RP	Several multicast RPs sharing one address, kept in sync (often with MSDP)	10
AS_PATH	BGP attribute listing every AS a route has crossed; used for loop prevention and path length	7, 28
ASBR	Autonomous System Boundary Router — injects external routes into OSPF	6, 27
BFD	Bidirectional Forwarding Detection — lightweight, sub-second link-failure detection for routing protocols	25
Bidir PIM	PIM variant that uses only shared trees, suited to many-to-many groups	10

Term	Meaning	Ch
Border node	SD-Access node connecting the fabric to external networks	14
Catalyst Center	Cisco's campus controller and assurance platform (formerly DNA Center)	16, 20
CEF	Cisco Express Forwarding — the FIB and adjacency table built from the RIB for fast forwarding	5, 22
CIST / IST	MST's Common and Internal Spanning Tree — instance 0, which carries BPDUs and interoperates with other regions	4
Cluster list	BGP attribute route reflectors add to prevent reflection loops	28
Community	BGP tag attached to routes to drive policy (no-export, no-advertise, custom values)	28
Conditional debug	A debug limited to one interface, address or peer, so it's safe on a busy box	15, 32
CoPP	Control Plane Policing — MQC policy that rate-limits traffic destined to the router's CPU	17, 31
Data model	A structured, machine-readable schema for configuration and state (see YANG)	20
Datastore	A NETCONF configuration store: running, candidate or startup	20
DMVPN	Dynamic Multipoint VPN — mGRE + NHRP (+ IPsec) giving hub-and-spoke with on-demand spoke-to-spoke tunnels	30
DSCP	6-bit DiffServ field in the IP header used for QoS marking	11
DUAL	EIGRP's Diffusing Update Algorithm — computes loop-free successors and feasible successors	5, 26
EAP	Extensible Authentication Protocol — the 802.1X method family (EAP-TLS, PEAP, EAP-FAST)	18

Term	Meaning	Ch
EAPOL	EAP over LAN — how 802.1X carries EAP between the supplicant and the switch	18
Edge node	SD-Access access-layer switch where endpoints attach; acts as the VXLAN tunnel endpoint	14
EEM	Embedded Event Manager — on-box automation triggered by syslog, CLI, timers, tracking and other events	21
EID / RLOC	LISP's Endpoint Identifier (who) and Routing Locator (where)	13
ERSPAN	Encapsulated Remote SPAN — mirrored traffic carried in GRE across a routed network	15
Feasible distance (FD)	EIGRP's best total metric to a destination	5, 26
Feasible successor	EIGRP backup route that passes the feasibility condition (RD < FD) and is guaranteed loop-free	5, 26
FIB	Forwarding Information Base — the data-plane copy of the best routes	22
Flexible NetFlow	NetFlow with user-defined records (key and non-key fields), exporters and monitors	15, 32
GRE	Generic Routing Encapsulation — a simple tunnel that can carry any protocol, including multicast	12
HSRP	Cisco's Hot Standby Router Protocol — active/standby virtual gateway	9
iACL	Infrastructure ACL — an edge ACL protecting the network's own addresses	17
iBGP / eBGP	BGP between routers in the same AS / in different ASes	7, 28
Idempotent	An operation that gives the same result however many times it runs (PUT, DELETE; Ansible tasks)	20, 21
IKE	Internet Key Exchange — negotiates IPsec security associations (IKEv1, IKEv2)	12, 30

Term	Meaning	Ch
Intent-based networking	Declaring the desired outcome and letting a controller translate and verify it	16
IP SLA	Synthetic probes that measure reachability, delay and jitter, often driving tracked failover	15, 32
ISE	Identity Services Engine — Cisco's RADIUS/TACACS+ policy server for 802.1X, posture and TrustSec	18
JSON	JavaScript Object Notation — the text data format most network APIs use	19, 20
LDP	Label Distribution Protocol — distributes MPLS label bindings between neighbors	29
LFIB	Label Forwarding Information Base — the MPLS data-plane table	29
LISP	Locator/ID Separation Protocol — the SD-Access control plane	13, 14
LLQ	Low Latency Queuing — CBWFQ plus a policed strict-priority queue for voice	11
Local preference	BGP attribute (higher wins) that steers outbound traffic across an AS	7, 28
MAB	MAC Authentication Bypass — authenticating a non-802.1X device by its MAC address	18
MACsec	IEEE 802.1AE hop-by-hop Layer 2 encryption, keyed by MKA	18
MED	Multi-Exit Discriminator — BGP hint (lower wins) to a neighbor AS about which entry point to use	7, 28
Method list	Named or default ordered list of AAA sources (group, local, enable, none) applied to lines or services	17, 31
mGRE	Multipoint GRE — one tunnel interface reaching many peers	30

Term	Meaning	Ch
MKA	MACsec Key Agreement — the protocol that establishes MACsec keys	18
MP-BGP	Multiprotocol BGP — address families beyond IPv4 unicast (IPv6, VPNv4)	28, 29
MPLS	Multiprotocol Label Switching — forwarding on short labels instead of IP lookups	29
MQC	Modular QoS CLI — class-map, policy-map, service-policy	11, 17
MSDP	Multicast Source Discovery Protocol — shares active sources between RPs	10
MST	Multiple Spanning Tree (802.1s) — maps many VLANs onto a few spanning-tree instances	4
NETCONF	XML-over-SSH (TCP 830) protocol for model-driven configuration	20
Next-hop-self	Makes an iBGP speaker advertise itself as next hop, fixing unreachable eBGP next hops	28
NGFW	Next-generation firewall — stateful firewall plus application awareness, IPS, URL and malware filtering	18
NHRP	Next Hop Resolution Protocol — DMVPN's tunnel-to-public-address resolution	30
NSF / SSO	Non-Stop Forwarding / Stateful Switchover — keep forwarding while a supervisor or control plane fails over	2
NSSA	Not-So-Stubby Area — stub area that may still import externals as Type 7 LSAs	6, 27
OAuth 2.0	Token-based authorization framework for APIs; the client presents a short-lived bearer token	18

Term	Meaning	Ch
OMP	Overlay Management Protocol — SD-WAN's BGP-like control protocol between edges and controllers	14
Orchestration	Automating multi-step change across many devices (Ansible, Puppet, Chef, Salt)	21
PBR	Policy-Based Routing — forwarding decided by a route map instead of only the destination	5, 25
PHP	Penultimate Hop Popping — the second-to-last LSR removes the transport label	29
PIM-SM	Protocol Independent Multicast – Sparse Mode — explicit-join multicast routing via an RP	10
Prefix list	Ordered list matching prefixes and lengths (ge/le) for route filtering	23
Privilege level	IOS access level 0–15; 1 is user EXEC, 15 is full control	17
PTP	Precision Time Protocol (IEEE 1588) — sub-microsecond clock synchronisation	8
RADIUS	AAA protocol over UDP 1812/1813; combines authentication and authorization; encrypts only the password	17, 31
RD / RT	Route Distinguisher (makes VPN prefixes unique) / Route Target (controls VRF import and export)	29
Redistribution	Injecting routes from one routing source into another protocol	24
Reported distance (RD)	EIGRP: a neighbor's own metric to the destination (also called advertised distance)	26
RESTCONF	HTTPS-based, REST-style access to YANG-modelled data	20
RIB	Routing Information Base — the routing table	22

Term	Meaning	Ch
Route map	Ordered if/then policy (match/set) used by redistribution, BGP, PBR and more	23
Route reflector	iBGP speaker allowed to re-advertise iBGP routes (client routes to clients and non-clients, non-client routes to clients), removing the full mesh	28
RP	Rendezvous Point — PIM-SM's meeting point for sources and receivers	10
RPF check	Multicast loop check: accept a packet only on the interface towards its source (or RP)	10
RSPAN	Remote SPAN — mirrored traffic carried in a dedicated VLAN	15
SD-Access	Cisco's campus fabric: LISP control plane, VXLAN data plane, TrustSec policy plane	14
SD-WAN Manager / Controller / Validator	Catalyst SD-WAN's management, control and orchestration components (formerly vManage, vSmart, vBond)	14, 20
SGACL	Security Group ACL — a TrustSec policy written between source and destination SGTs	18
SGT	Security Group Tag — TrustSec's role-based label carried with traffic	14, 18
SIA	Stuck in Active — an EIGRP query that never got all its replies	26
SSM	Source-Specific Multicast — receivers join (S,G) directly; range 232/8; needs IGMPv3	10
StackWise Virtual	Two Catalyst chassis acting as one logical switch for multichassis EtherChannel	2
Stub router (EIGRP)	Router that tells neighbors not to query it, bounding the query domain	26
Successor	EIGRP's best route to a destination	5, 26

Term	Meaning	Ch
SXP	SGT Exchange Protocol — carries IP-to-SGT mappings over TCP 64999 to devices that can't tag inline	18
TACACS+	Cisco AAA protocol over TCP 49; separates authentication, authorization and accounting; encrypts the whole payload	17, 31
TLOC	Transport Locator — an SD-WAN edge's attachment point to a transport (system IP, colour, encapsulation)	14
TrustSec	Cisco role-based segmentation using SGTs and SGACLs	18
uRPF	Unicast Reverse Path Forwarding — drops packets whose source isn't reachable back through the arrival interface (strict) or at all (loose)	31
Variance	EIGRP multiplier enabling unequal-cost load balancing across feasible successors	5, 26
Virtual link	OSPF logical link through a transit area to connect a stranded area to Area 0	27
VNI	VXLAN Network Identifier — 24-bit segment ID (~16 million segments)	13
VRF / VRF-Lite	Virtual routing table on one router; VRF-Lite is VRFs without MPLS	12, 25
VRRP	Standards-based FHRP (master/backup)	9
VTEP	VXLAN Tunnel Endpoint — encapsulates and decapsulates VXLAN	13
VTI	Virtual Tunnel Interface — routable IPsec tunnel interface	12
VXLAN	L2-over-UDP (port 4789) overlay with 24-bit segment IDs	13
Weight	Cisco-only, router-local BGP attribute; highest wins, checked first	7, 28

Term	Meaning	Ch
XSRF token	Anti-cross-site-request-forgery token SD-WAN Manager requires on POST/PUT/DELETE calls	20
YANG	Data modelling language describing device configuration and state	20

That's Both Exams

ENCOR proves you understand the whole enterprise network. ENARSI proves you can find the fault in it. The habit that carries you through both is the one from the CCNA guide: refuse to accept that something works until you know why. Book ENCOR, lab everything, explain it out loud, and come back to Chapter 33 when you're ready for the next one.